

Layer Explanation Plan

20 ABFYCTA 2026 Г.

you 20/08/2026, 11:03:48

tum ye file completely read kro
iss file ko ache se depth tk understand kro
phir mujhe ek file/ project ko layer by layer explain kro complete depth me
tum ek example lo jo pdf me nhi liya ho pr wo example sbhi layer me use hona chahiye
uss example se iss project ki saari ondition full fill hona chahiye
isliye ek bahut bada example lo taki meri iss project ki understanding bad ske
hrr layer ka input kya hoga output kya hoga layer kaam kese kr rhi hai
hrr layer me error, mistakes , missing part ho toh wo bhi btao
exception handling kese kr rhe hai wo bhi btao
pr ek time pr ek hi

DEEP RESEARCH 20/08/2026, 11:03:54

Make sure to include `filecite` in your response to cite this file, or to surface it as a link. Replace the placeholder line range with the actual line range from the result that best fits the snippet you want to surface with the citation.

Use `ref_id "turn0file0"` to reference this file. For broad questions or unknown locations, retrieve relevant chunks with `files.search` scoped to this file. Use `files.find` only for exact text, and use `files.read` for a known range or to expand relevant results. Full file size: 2067 file lines.

[L1] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L3] > **Project Type:** Minor Project (M.Tech)

[L4] > **Date:** August 2026

[L5] > **Author:** [Your Name]

[L9] ## Abstract

[L11] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L13] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS_A, PART_OF) and semantic (USES, CAUSES) relationships.

[L15] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L17] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L19] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L23] ## Table of Contents

[L25] 1. Introduction

[L26] 2. Literature Review

[L27] 3. System Architecture

[L28] 4. Data Model — Layer 0

[L29] 5. Offline Pipeline — Layers 1-5

[L30] 6. Online Pipeline — Layers 6-7

[L31] 7. Operations Layer — Layer 8

[L32] 8. Novel Contributions

[L33] 9. Mathematical Framework

[L34] 10. Implementation

[L35] 11. Evaluation Framework

[L36] 12. Risk Analysis

[L37] 13. Conclusion & Future Work

[L41] ## 1. Introduction

[L43] ### 1.1 Problem Statement

[L45] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L47] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L48] 2. Embed each chunk into a vector space

[L49] 3. At query time, retrieve the top-k most similar chunks

[L50] 4. Feed these chunks as context to an LLM for answer generation

[L52] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L54] | Problem | Description | Impact |

[L55] |-----|-----|-----|

[L56] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L57] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L58] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L59] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |

[L60] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L62] ### 1.2 Proposed Solution: SHIA-RAG

[L64] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L67] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L68] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

- [L71] The system:
- [L72] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents
 - [L73] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees
 - [L74] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth
 - [L75] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints
 - [L76] 5. **Generates verified answers** — with claim-level evidence verification and citation binding
 - [L77] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure
- [L79] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L81] ### 1.3 Objectives

- [L83] 1. Design a complete 8-layer architecture for hierarchical RAG
- [L84] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)
- [L85] 3. Implement a self-evolving feedback loop that improves retrieval quality over time
- [L86] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)
- [L87] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality
- [L88] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L90] ### 1.4 Contributions

- [L92] This work makes five novel contributions to the RAG literature:
- | | |
|--------|--|
| [L94] | # Contribution Novelty Level |
| [L95] | -- ----- ----- |
| [L96] | 1 Precedence-Constrained Tree Knapsack for context selection Incremental (Novel Application) |
| [L97] | 2 Self-Evolving Knowledge Forest via retrieval feedback Moderately Novel |
| [L98] | 3 Multi-Objective Energy Minimization for hierarchy induction Moderately Novel |
| [L99] | 4 Adaptive Depth Traversal via query complexity classification Incremental |
| [L100] | 5 SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG Incremental (Novel Combination) |

[L104] ## 2. Literature Review

[L106] ### 2.1 Related Work

[L108] #### 2.1.1 Flat-Chunk RAG Systems

[L110] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-reflection, but all operate on flat text chunks with no structural awareness.

[L112] ##### 2.1.2 Structured RAG Systems

[L114] Recent work has begun exploring structured retrieval:

[L116] | System | Approach | Key Limitation |

[L117] |-----|-----|-----|

[L118] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L119] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L120] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L121] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L123] ##### 2.1.3 Knowledge Graph Construction

[L125] Ontology learning and taxonomy induction have a rich history in NLP:

[L126] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L127] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L128] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L130] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L132] ### 2.2 Research Gap

[L135] [L136] System	Structured Retrieval?	Concept-Level Extraction?	Self-Evolving?	Hierarchy-Aware Context Selection?
[L137]				
[L138] Traditional RAG	x	x	x	x
[L139] RAPTOR	✓ (summaries)	x	x	x
[L140] GraphRAG	✓ (subgraph)	✓ (partial)	x	x
[L141] TreeRAG	✓ (tree)	x	x	x
[L142] HiChunk	✓ (chunks)	x	x	x
[L143]				
[L144] SHIA-RAG (Ours)	✓ (forest)	✓ (full)	✓	✓

[L147] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L149] ### 2.3 Literature Search Methodology

[L151] | Parameter | Details |

[L152] |-----|-----|

[L153] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L154] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L155] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L156] | **Time Range** | 2020–2026 |

[L157] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L161] ## 3. System Architecture

[L163] ### 3.1 Overview

[L165] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L167] | Phase | Layers | Mode | Purpose |

[L168] |-----|-----|-----|-----|

[L169] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L170] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L171] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L173] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L175]

```
[L176] graph TD
[L177]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L178]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L179]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L180]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L181]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L182]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L182]     end

[L185]     subgraph "Phase 2: Online – Retrieval & Generation"
[L186]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L187]         KF --> L6
[L188]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L189]         L7 --> A["Answer + Citations"]
[L189]     end

[L192]     subgraph "Phase 3: Continuous – Operations"
[L193]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L194]         A --> L8
[L195]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L195]     end
```

[L199] ### 3.2 Data Flow

```
[L202] Raw Document (PDF/DOCX/HTML)
[L203]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L204] Normalized Text + Metadata
[L205]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L206] [Page → Region → Block] hierarchy
[L207]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L208] KnowledgeUnit objects (candidate concepts)
[L209]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L210] KnowledgeNode objects (canonical, deduplicated)
[L211]     ↓ Layer 4: Validate relations against ontology, score confidence
[L212] KnowledgeEdge objects (typed, scored)
[L213]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L214] Knowledge Forest (rooted trees + cross-links)
[L215]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L216] Selected context nodes (within token budget)
[L217]     ↓ Layer 7: Build prompt, invoke LLM
[L218]     ↓ Layer 7: Verify claims against evidence, bind citations
[L219] Answer with citations + confidence score
[L220]     ↓ Layer 8: Log, collect feedback, update forest weights
```

[L223] ### 3.3 Layer Summary

[L225] | Layer | Name | Input | Output | Key Algorithms |
[L226] |-----|-----|-----|-----|-----|
[L227] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L228] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L229] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L230] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L231] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L232] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L233] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L234] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L235] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L239] ## 4. Data Model — Layer 0

[L241] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` (concepts) and `KnowledgeEdge` (relationships).

[L243] ### 4.1 KnowledgeNode Schema

[L245] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :

[L247] ```json
{
[L249] "node_id": "NODE_000789",
[L250] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L251] "canonical_name": "Round Robin",
[L252] "aliases": ["RR", "Round-Robin Scheduling"],
[L253] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L254] "summaries": ["CPU scheduling method using time slices"],
[L255] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L256] "domain": "Operating Systems",
[L257] "properties": {
[L258] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L259] "preemptive": {"type": "boolean", "value": true}
},
[L261] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L262] "parent": "NODE_000456",
[L263] "children": ["NODE_001001", "NODE_001002"],
[L264] "related": ["NODE_000890"],
[L265] "incoming_edges": ["EDGE_001"],
[L266] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L

CHATGPT 20/08/2026, 11:03:54

{"paths":["files"]}

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `filecite[turn1file0]`

[L1] metadata:

[L2] query_result_index: 0

[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4).md

[L6] mime_type: text/markdown

[L7] surface: conversation

[L8] score: 0.03252247488101534

[L9] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--51

[L10] snippet:

[L11] ### 11.4 Statistical Significance Testing

[L12]

[L13] | Test | Application | Details |

[L14] |-----|-----|-----|

[L15] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |

[L16] | Significance level | All tests | $\alpha = 0.05$ |

[L17] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |

[L18] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |

[L19] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |

[L20]

[L21] ### 11.5 Error Analysis

[L22]

[L23] | Analysis Type | Method | Details |

[L24] |-----|-----|-----|

[L25] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |

[L26] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |

[L27] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |

[L28] | Per-category breakdown | Stratified metrics | Report accuracy per query type |

[L29]

[L30] ### 11.6 Computational Cost Analysis

[L31]

[L32] | Metric | How Measured | Report Format |

[L33] |-----|-----|-----|

[L34] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L35] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L36] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L37] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L38] | Memory usage | Peak RAM + VRAM | GB |

[L39] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L40]

[L41] ---

[L42]

[L43] ## 12. Risk Analysis

[L44]

[L45] | Risk | Impact | Probability | Mitigation |

[L46] |-----|-----|-----|-----|

[L47] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L48] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L49] | $O(N^2)$ operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L50] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning, conflict logging |

[L51] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L52] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L53] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L54] | Self-evolution reinforces wrong paths | Medium | Medium | ϵ -greedy exploration ($\epsilon=0.1$), periodic weight normalization, anomaly detection |

[L55]

[L56] ---

[L57]

[L58] ## 12.5 Limitations & Current Status

[L59]

[L60] This section explicitly documents known limitations and the current project status.

[L61]

[L62] ### Current Status

[L63]

[L64] SHIA-RAG is currently at the **specification stage**. This report provides a complete architectural specification with schemas, algorithms, pseudocode, and implementation contracts. However:

[L65]

[L66] - **No implementation exists yet** — the system has not been built or deployed

[L67] - **No experimental results exist** — all evaluation metrics are planned, not measured

[L68] - **Cost/latency numbers are projections** — actual measurements require a running system

[L69]

[L70] The implementation roadmap (Section 10.3) details the path from specification to working system.

[L71]

[L72] ### Known Limitations

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `filecite[turn1file1]`

[L73] metadata:

[L74] query_result_index: 1

[L75] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L76] version_id: 1

[L77] name: SHIA_RAG_Final_Report(4).md

[L78] mime_type: text/markdown

[L79] surface: conversation

[L80] score: 0.032018442622950824

[L81] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--6

[L82] snippet:

[L83]

```
[L84] graph TD
[L85]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L86]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Par
[L87]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L88]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L89]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L90]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L91]     end
[L92]
[L93]     subgraph "Phase 2: Online – Retrieval & Generation"
[L94]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RBO"]
[L95]         KF --> L6
[L96]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L97]         L7 --> A["Answer + Citations"]
[L98]     end
[L99]
[L100]    subgraph "Phase 3: Continuous – Operations"
[L101]        L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L102]        A --> L8
[L103]        L8 -->| "Self-Evolution<br/>Feedback" | KF
[L104]    end
[L105]
```

[L106]

[L107] ### 3.2 Data Flow

[L108]

[L109]

```
[L110] Raw Document (PDF/DOCX/HTML)
[L111]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L112] Normalized Text + Metadata
[L113]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L114] [Page → Region → Block] hierarchy
[L115]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L116] KnowledgeUnit objects (candidate concepts)
[L117]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L118] KnowledgeNode objects (canonical, deduplicated)
[L119]     ↓ Layer 4: Validate relations against ontology, score confidence
[L120] KnowledgeEdge objects (typed, scored)
[L121]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L122] Knowledge Forest (rooted trees + cross-links)
[L123]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L124] Selected context nodes (within token budget)
[L125]     ↓ Layer 7: Build prompt, invoke LLM
[L126]     ↓ Layer 7: Verify claims against evidence, bind citations
[L127] Answer with citations + confidence score
[L128]     ↓ Layer 8: Log, collect feedback, update forest weights
[L129]
```

[L130]

[L131] ### 3.3 Layer Summary

[L132]

[L133] | Layer | Name | Input | Output | Key Algorithms |

[L134] |-----|-----|-----|-----|-----|

[L135] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L136] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L137] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L138] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L139] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L140] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L141] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L142] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L143] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L144]
[L145] ---
[L146]
[L147] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `[[filecite]]turn1file2[[`

[L148] metadata:
[L149] query_result_index: 2
[L150] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L151] version_id: 1
[L152] name: SHIA_RAG_Final_Report(4).md
[L153] mime_type: text/markdown
[L154] surface: conversation
[L155] score: 0.03200204813108039
[L156] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--5
[L157] snippet:
[L158] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.
[L159]
[L160] ### 2.3 Literature Search Methodology
[L161]
[L162] | Parameter | Details |
[L163] |-----|-----|
[L164] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |
[L165] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |
[L166] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |
[L167] | **Time Range** | 2020–2026 |
[L168] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |
[L169]
[L170] ---
[L171]
[L172] ## 3. System Architecture

[L173]

[L174] ### 3.1 Overview

[L175]

[L176] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L177]

[L178] | Phase | Layers | Mode | Purpose |

[L179] |-----|-----|-----|-----|

[L180] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L181] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L182] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L183]

[L184] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L185]

[L186]

```
[L187] graph TD
[L188]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L189]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L190]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L191]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L192]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L193]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L194]     end
[L195]
[L196]     subgraph "Phase 2: Online – Retrieval & Generation"
[L197]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L198]         KF --> L6
[L199]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L200]         L7 --> A["Answer + Citations"]
[L201]     end
[L202]
[L203]     subgraph "Phase 3: Continuous – Operations"
[L204]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L205]         A --> L8
[L206]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L207]     end
[L208]
```

[L209]

[L210] ### 3.2 Data Flow

[L211]

[L212] ```

[L213] Raw Document (PDF/DOCX/HTML)

[L214] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L215] Normalized Text + Metadata

[L216] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L217] [Page → Region → Block] hierarchy

[L218] ↓ Layer 3: Split sentences,

[L219] metadata:

[L220] query_result_index: 3

[L221] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L222] version_id: 1

[L223] name: SHIA_RAG_Final_Report(4).md

[L224] mime_type: text/markdown

[L225] surface: conversation

[L226] score: 0.03149801587301587

[L227] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--54

[L228] snippet:

[L229] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L230] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L231] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L232] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L233] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L234]

[L235] The complete system specification covers:

[L236] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L237] - **12 defined functions** with complete implementations

[L238] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L239] - **8 schemas** covering all data objects from document blocks to query logs

[L240] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L241] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L242]

[L243] ### 13.2 Key Novelties

[L244]

[L245] | # | Novelty | Level | Why It's New |

[L246] |--|-----|-----|-----|

[L247] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L248] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L249] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L250] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L251] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L252]

[L253] ### 13.3 Next Steps

[L254]

- [L255] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)
- [L256] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)
- [L257] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)
- [L258] 4. **Build Layer 3** (Knowledge extraction with LLM)
- [L259] 5. **Build Layer 4** (Relation management with ontology validation)
- [L260] 6. **Build Layer 5** (SHIA Core with algorithms)
- [L261] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
- [L262] 8. **Build Layer 7** (Generation with verification)
- [L263] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
- [L264] 10. **Run first evaluation** on OS textbook dataset using SHEF
- [L265] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
- [L266] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
- [L267]
- [L268] ### 13.4 Future Work

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `[[filecite]]turn1file4[[`

- [L269] metadata:
- [L270] query_result_index: 4
- [L271] file_id: file_0000000012688211b4f57f5b8bcb8b42
- [L272] version_id: 1
- [L273] name: SHIA_RAG_Final_Report(4).md
- [L274] mime_type: text/markdown
- [L275] surface: conversation
- [L276] score: 0.030776515151515152
- [L277] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--2
- [L278] snippet:
- [L279] | Problem | Description | Impact |
- [L280] |-----|-----|-----|
- [L281] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |
- [L282] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |
- [L283] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |
- [L284] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |
- [L285] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |
- [L286]
- [L287] ### 1.2 Proposed Solution: SHIA-RAG
- [L288]
- [L289] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five

limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L290]

[L291]

[L292] Traditional RAG: Document → [Chunk₁, Chunk₂, ..., Chunk_n] → Top-k → LLM

[L293] SHIA-RAG: Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM

[L294]

[L295]

[L296] The system:

[L297] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L298] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L299] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L300] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L301] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L302] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L303]

[L304] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L305]

[L306] ### 1.3 Objectives

[L307]

[L308] 1. Design a complete 8-layer architecture for hierarchical RAG

[L309] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L310] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L311] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L312] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L313] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L314]

[L315] ### 1.4 Contributions

[L316]

[L317] This work makes five novel contributions to the RAG literature:

[L318]

[L319] | # | Contribution | Novelty Level |

[L320] |---|-----|-----|

[L321] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L322] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L323] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L324] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L325] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L326]

[L327] ---

[L328]

[L329] ## 2. Literature Review

[L330]

[L331] ### 2.1 Related Work

[L332]

[L333] ##### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `filecite{turn1file5}`

[L334] metadata:

[L335] query_result_index: 5

[L336] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L337] version_id: 1

[L338] name: SHIA_RAG_Final_Report(4).md

[L339] mime_type: text/markdown

[L340] surface: conversation

[L341] score: 0.0304147465437788

[L342] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--37

[L343] snippet:

[L344]

```
[L345] @dataclass
[L346] class QueryLog:
[L347]     event_id: str
[L348]     timestamp: str
[L349]     query: str
[L350]     intent: str          # DEFINITION, COMPARISON, etc.
[L351]     complexity: str      # SIMPLE, MODERATE, COMPLEX
[L352]     seed_nodes: list
[L353]     candidate_count: int
[L354]     selected_count: int
[L355]     traversal_edges: list # For feedback attribution
[L356]     answer_length: int
[L357]     claim_count: int
[L358]     supported_claims: int
[L359]     unsupported_claims: int
[L360]     overall_confidence: float
[L361]     model_used: str
[L362]     latency_ms: float
[L363]     user_feedback: str = None # HELPFUL / NOT_HELPFUL (filled later)
[L364]
```

[L365]

[L366] ### 7.2 Feedback Acquisition & Attribution

[L367]

[L368] User feedback is collected via API:

[L369]

[L370]

```
[L371] POST /api/feedback { "event_id": "...", "feedback": "HELPFUL" | "NOT_HELPFUL" }
[L372]
```

[L373]

[L374] **Attribution strategy:** Feedback applies ONLY to edges in the recorded `traversal_path`. This isolates retrieval quality from generation quality.

[L375]

[L376] **Safeguards:**

[L377] - Edge weight updates use a dampened learning rate ($\eta=0.02$, small increments)

[L378] - Layer 7 verification metrics (supported/unsupported claims) are tracked separately

[L379] - Anomaly detection: if `confidence > 0.8` AND `feedback == "NOT_HELPFUL"`, log a warning (possible answer format issue, not retrieval issue)

[L380]

[L381] **7.3 Self-Evolution Update Rule**

[L382]

[L383] After each query with feedback signal $f \in \{+1, -1\}$:

[L384]

[L385]
$$w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \forall e \in \text{traversal_path}$$

[L386]

[L387] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.

[L388]

[L389]

```
[L390] import math
[L391] import random
[L392]
[L393] ETA = 0.02
[L394] WEIGHT_MIN, WEIGHT_MAX = 0.1, 10.0
[L395] RESTRUCTURE_THRESHOLD = 0.3
[L396] EPSILON = 0.1 # ε-greedy exploration rate
[L397]
[L398] def update_forest_from_feedback(selected_nodes, user_feedback, forest):
[L399]     f = +1 if user_feedback == "HELPFUL" else -1
[L400]     path_edges = get_traversal_path_edges(selected_nodes, forest)
[L401]
[L402]     for edge in path_edges:
[L403]         edge.weight = edge.weight * math.exp(ETA * f)
[L404]         edge.weight = max(WEIGHT_MIN, min(edge.weight, WEIGHT_MAX))
[L405]         edge.access_count += 1
[L406]
[L407]         if f == -1 and edge.weight < RESTRUCTURE_THRESHOLD:
[L408]             flag_for_restructure(edge, forest)
[L409]
[L410]     if forest.total_queries % 100 == 0:
[L411]         rebalance_forest(forest)
[L412]     forest.total_queries += 1
[L413]
```

[L414]

[L415] **ε-Greedy Exploration:** To prevent the self-evolution loop from reinforcing wrong paths (e.g., an incorrect path gets positive feedback because the LLM produces a convincing-looking answer), retrieval includes ε-greedy exploration:

[L416] metadata:
[L417] query_result_index: 6
[L418] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L419] version_id: 1
[L420] name: SHIA_RAG_Final_Report(4).md
[L421] mime_type: text/markdown
[L422] surface: conversation
[L423] score: 0.030309988518943745
[L424] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--7
[L425] snippet:
[L426] deduplicated)
[L427] ↓ Layer 4: Validate relations against ontology, score confidence
[L428] KnowledgeEdge objects (typed, scored)
[L429] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L430] Knowledge Forest (rooted trees + cross-links)
[L431] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L432] Selected context nodes (within token budget)
[L433] ↓ Layer 7: Build prompt, invoke LLM
[L434] ↓ Layer 7: Verify claims against evidence, bind citations
[L435] Answer with citations + confidence score
[L436] ↓ Layer 8: Log, collect feedback, update forest weights
[L437]

```
[L438]
[L439] ### 3.3 Layer Summary
[L440]
[L441] | Layer | Name | Input | Output | Key Algorithms |
[L442] |-----|-----|-----|-----|-----|
[L443] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L444] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L445] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L446] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L447] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L448] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L449] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L450] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L451] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L452]
[L453] ---
[L454]
[L455] ## 4. Data Model – Layer 0
[L456]
[L457] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L458]
[L459] ### 4.1 KnowledgeNode Schema
[L460]
[L461] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L462]
[L463]
```

json

```
[L464] {
[L465] "node_id": "NODE_000789",
[L466] "uuid": "550e8400-e29b-41d4-a716-446655440000",
```

[L467] "canonical_name": "Round Robin",
 [L468] "aliases": ["RR", "Round-Robin Scheduling"],
 [L469] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
 [L470] "summaries": ["CPU scheduling method using time slices"],
 [L471] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
 [L472] "domain": "Operating Systems",
 [L473] "properties": {
 [L474] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
 [L475] "preemptive": {"type": "boolean", "value": true}
 [L476] },
 [L477] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
 [L478] "parent": "NODE_000456",
 [L479] "children": ["NODE_001001", "NODE_001002"],
 [L480] "related": ["NODE_000890"],
 [L481] "incoming_edges": ["EDGE_001"],
 [L482] "outgoing_edges": ["EDGE_002", "EDGE_003"],
 [L483] "evidence": [
 [L484] {
 [L485] "evidence_type": "DEFINITION",
 [L486] "text": "Round Robin is a preemptive scheduling algorithm...",
 [L487] "source_doc": "DOC_000001",
 [L488] "source_block": "BLK_000234",
 [L489] "page": 42
 [L490] }
 [L491]],

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `filecite`turn1file7

[L492] metadata:
 [L493] query_result_index: 7
 [L494] file_id: file_0000000012688211b4f57f5b8bcb8b42
 [L495] version_id: 1
 [L496] name: SHIA_RAG_Final_Report(4).md
 [L497] mime_type: text/markdown
 [L498] surface: conversation
 [L499] score: 0.03009207275993712
 [L500] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--1
 [L501] snippet:
 [L502] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing
 [L503]
 [L504] ---
 [L505]
 [L506] ## Table of Contents

[L507]

[L508] 1. Introduction

[L509] 2. Literature Review

[L510] 3. System Architecture

[L511] 4. Data Model — Layer 0

[L512] 5. Offline Pipeline — Layers 1-5

[L513] 6. Online Pipeline — Layers 6-7

[L514] 7. Operations Layer — Layer 8

[L515] 8. Novel Contributions

[L516] 9. Mathematical Framework

[L517] 10. Implementation

[L518] 11. Evaluation Framework

[L519] 12. Risk Analysis

[L520] 13. Conclusion & Future Work

[L521]

[L522] ---

[L523]

[L524] ## 1. Introduction

[L525]

[L526] ### 1.1 Problem Statement

[L527]

[L528] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L529]

[L530] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L531] 2. Embed each chunk into a vector space

[L532] 3. At query time, retrieve the top-k most similar chunks

[L533] 4. Feed these chunks as context to an LLM for answer generation

[L534]

[L535] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L536]

[L537] | Problem | Description | Impact |

[L538] |-----|-----|-----|

[L539] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L540] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L541] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L542] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., $A \rightarrow B \rightarrow C$) |

[L543] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L544]

[L545] ### 1.2 Proposed Solution: SHIA-RAG

[L546]

[L547] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L548]

[L549]

[L550]	Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L551]	SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L552]		

[L553]

[L554] The system:

[L555] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L556] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L557] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L558] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L559] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L560] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

DEEP RESEARCH 20/08/2026, 11:03:59

Citation Marker: `[[filecite]]turn1file8`

[L561] metadata:

[L562] query_result_index: 8

[L563] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L564] version_id: 1

[L565] name: SHIA_RAG_Final_Report(4).md

[L566] mime_type: text/markdown

[L567] surface: conversation

[L568] score: 0.02967032967032967

[L569] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--53

[L570] snippet:

[L571] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") | Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L572] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is O(nB) where B = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L573] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L574] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L575] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds and human review |

[L576] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors. ϵ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by ϵ -greedy; future: Thompson sampling |

[L577] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L578]

[L579] ---

[L580]

[L581] ## 13. Conclusion & Future Work

[L582]

[L583] ### 13.1 Summary

[L584]

[L585] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L586]

[L587] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L588] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L589] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L590] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L591] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L592]

[L593] The complete system specification covers:

[L594] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L595] - **12 defined functions** with complete implementations

[L596] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L597] - **8 schemas** covering all data objects from document blocks to query logs

[L598] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L599] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L600]

[L601] ### 13.2 Key Novelties

[L602]

[L603] | # | Novelty | Level | Why It's New |

[L604] |---|-----|-----|-----|

[L605] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L606] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

[L607] metadata:
 [L608] query_result_index: 9
 [L609] file_id: file_0000000012688211b4f57f5b8bcb8b42
 [L610] version_id: 1
 [L611] name: SHIA_RAG_Final_Report(4).md
 [L612] mime_type: text/markdown
 [L613] surface: conversation
 [L614] score: 0.02964426877470356
 [L615] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--31
 [L616] snippet:
 [L617] ##### 5.5.5 CLD — Cross-Link Discoverer
 [L618]
 [L619] After hierarchical placement, discover semantic (non-hierarchical) links:
 [L620]
 [L621]

```
[L622] def cld_discover(node, forest, config):
[L623]     """Find semantic cross-links for a newly placed node."""
[L624]     cross_links = []
[L625]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L626]
[L627]     for neighbor in neighbors:
[L628]         if neighbor.id == node.id or neighbor.id == node.parent:
[L629]             continue
[L630]
[L631]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L632]         if sim > config.cross_link_threshold:
[L633]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L634]             if relation_type:
[L635]                 forest.add_edge(node.id, neighbor.id,
[L636]                                 edge_class='SEMANTIC', relation_type=relation_type,
[L637]                                 confidence=sim, weight=1.0)
[L638]                 cross_links.append((neighbor.id, relation_type, sim))
[L639]
[L640]     return cross_links
[L641]
```

[L642]
 [L643] ---
 [L644]
 [L645] ## 6. Online Pipeline — Layers 6-7
 [L646]
 [L647] ### 6.1 Layer 6: Retrieval Engine
 [L648]
 [L649] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.
 [L650]
 [L651] **Input:** User query string → **Output:** RetrievalResult
 [L652]
 [L653] ##### Query Complexity Classification
 [L654]
 [L655] | Intent | Example | Complexity | Traversal Depth |

```

[L656] |-----|-----|-----|-----|
[L657] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |
[L658] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |
[L659] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |
[L660] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |
[L661] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |
[L662]
[L663] ##### Retrieval Pipeline
[L664]
[L665] ```python
[L666] @dataclass
[L667] class RetrievalResult:
[L668] """Output of Layer 6 — input to Layer 7."""
[L669] selected_nodes: list # Nodes selected by Tree Knapsack
[L670] context_text: str # Assembled context string
[L671] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L672] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L673] query_intent: str # DEFINITION, COMPARISON, etc.
[L674] seed_node_ids: list # Initial seed nodes from vector search
[L675] token_count: int # Estimated tokens in context
[L676]
[L677] class RetrievalEngine:
[L678] def __init__(self, forest, vector_index, cache_manager, config):
[L679] self.forest = forest
[L680] self.vector_index = vector_index
[L681] self.cache = cache_manager
[L682] self.config = config
[L683]
[L684] def retrieve(self, query) -> RetrievalResult:
[L685] # Step 0: Cache check
[L686] cached = self.cache.get_query_result(hash_query(query))
[L687] if cached:
[L688] return cached
[L689]
[L690] # Step 1: Classify query intent and complexity
[L691] intent = detect_intent(query)
[L692] complexity = classify_complexity(intent)
[L693]
[L694] # Step 2: Embed query and find seed nodes
[L695] query_emb = embed_query(query) # Asymmetric embedding model
[L696] seed_nodes = self.vector_index.search(query_emb, k=5)
[L697]
[L698] # Step 3: Adaptive depth traversal
[L699] depth_config = {
[L700] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},

```

[L701] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L702] 'COMPLEX': {'max_depth': 3, 'parents': True,

CHATGPT 20/08/2026, 11:03:59

Bilkul. Maine file ko **sirf upar-upar se nahi**, balki iski complete architecture, data flow, algorithms, retrieval, generation, self-evolution, risk analysis aur known limitations ke perspective se study karna start kar diya hai.

Is file ka core idea ye hai ki normal RAG ke:

Document → Chunks → Top-k chunks → LLM

approach ko replace karke:

Document → Concepts + Relations → Knowledge Forest → Adaptive Traversal → Tree Knapsack → Verified Answer → Feedback-based Evolution

banaya gaya hai. `filecite\turn1file4\L279-L302`

Aur complete system **Layer 0 se Layer 8** tak divided hai: offline knowledge construction, online retrieval/generation, aur continuous self-evolution. `filecite\turn1file1\L83-L143`

Hum isko kaise padhenge

Tumne jo requirement di hai, uske according **ek time par sirf ek layer** karenge.

Har layer ke liye main same structure follow karunga:

1. Layer ka exact purpose
2. Is layer ko exact input kya milta hai
3. Input ka actual format/data structure
4. Hamare bade common example me iss layer ko kya input milega
5. Layer internally step-by-step kya karti hai
6. Har algorithm exactly kahan aur kyun use hota hai
7. Har step ke baad intermediate output kya banta hai
8. Final output ka exact format
9. Output next layer ko kaise diya jata hai
10. Why this layer is necessary
11. Possible errors
12. Possible mistakes / failure cases
13. File me jo safeguards defined hain
14. Exception handling
15. File me missing/weak/unclear part kya hai
16. Hamare common example ka layer-wise state

Ek hi bada common example use karenge

Main har layer me **same example** use karunga, taaki tum dekh sako ki ek hi information:

Raw Document → Text → Blocks → Concepts → Relations → Forest → Retrieved Context → Verified Answer → Feedback

kaise transform hoti hai.

Hamara common example: Smart Hospital Knowledge Base

Maan lo hum SHIA-RAG me ek bahut bada document set upload karte hain:

Document A — Hospital System Architecture

Topics:

- Smart Hospital
- Hospital Information System (HIS)
- Electronic Health Record (EHR)
- Patient
- Doctor
- Appointment
- Medical Record
- Database
- Authentication
- Role-Based Access Control

Document B — Emergency Workflow

Topics:

- Emergency Department
- Patient Triage
- Doctor Assignment
- Critical Patient
- ICU
- Bed Allocation
- Ambulance Arrival

Document C — Data Security

Topics:

- Patient Data
- Encryption
- Authentication
- RBAC
- Audit Log
- Data Breach
- Access Control

 Document D — AI Diagnostic Module

Topics:

- AI Diagnosis
- Medical Image
- X-ray
- CNN
- Disease Prediction
- Doctor Verification
- Diagnostic Confidence

 Document E — Conflicting / Cross-document Information

Isme same concepts different names se honge:

- EHR
- Electronic Health Record
- Digital Patient Record

Aur relations bhi multiple honge:

- AI Diagnosis **USES** Medical Image
- CNN **IS_A** Deep Learning Model
- Deep Learning **IS_A** Machine Learning
- Medical Image **PART_OF** Patient Diagnostic Data
- Doctor Verification **VALIDATES** / supported ontology ke according appropriate relation
- Weak evidence ya conflicting relations bhi honge

Final user query bhi isi example par hogi:

"Explain how an AI-assisted emergency diagnosis system processes a critical patient from arrival to final doctor-verified diagnosis, and how security controls protect the patient's medical data."

Ye intentionally **complex multi-hop query** hai.

Is query ko answer karne ke liye system ko multiple trees aur multiple relationships traverse karne padenge:

```

Patient Arrival
  ↓
Emergency Triage
  ↓
Critical Patient
  ↓
Medical Image
  ↓
AI Diagnosis
  ↓
Disease Prediction
  ↓
Doctor Verification
  ↓
Final Diagnosis

Security branch:
Patient Data
  ↓
Authentication
  ↓
RBAC
  ↓
Encryption
  ↓
Audit Log

```

Aur in dono branches ko combine karke final answer dena hoga.

Yahi same example Layer 0 se Layer 8 tak chalega.

Important: File ko blindly correct nahi maanenge

Main explanation me do cheezein clearly separate karunga:

A. Report me exactly kya defined hai

Ye file ke according hoga.

B. Meri technical analysis

Yahan main bataunga:

- architecture me gap kya hai
- algorithm me potential problem kya hai
- pseudocode aur production system me difference kya hai
- koi field missing hai kya
- exception handling incomplete hai kya
- future implementation me kya add karna chahiye

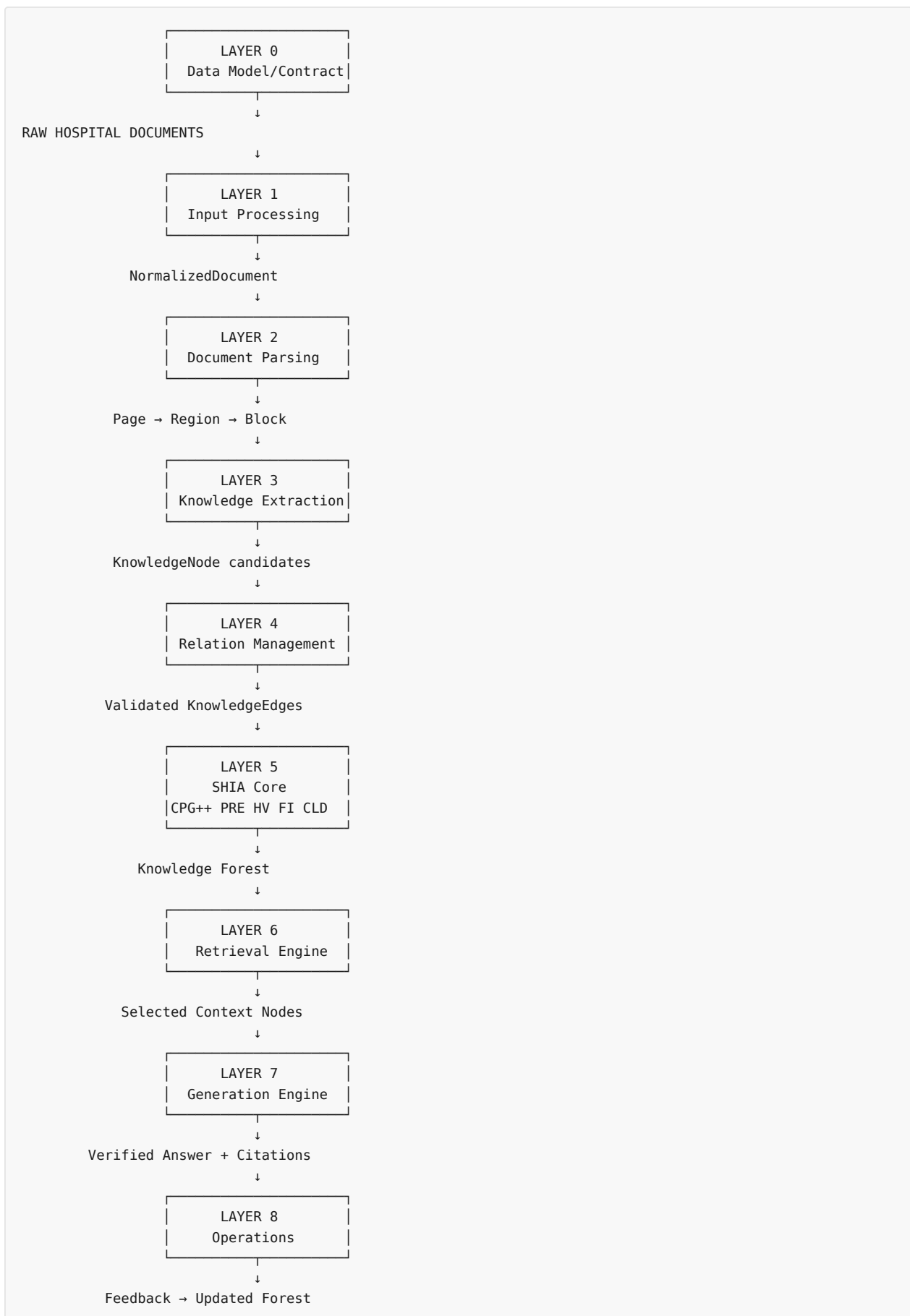
Ye important hai kyunki report khud clearly kehti hai ki **SHIA-RAG abhi specification stage par hai; implementation aur experimental results abhi exist nahi karte.** `filecite{turn1file0L58-L70}`

Aur report ne kuch known limitations bhi explicitly accept ki hain, jaise:

- greedy Tree Knapsack exact optimal nahi hai,
- incremental updates missing hain,

- abstraction-level heuristic empirically validated nahi hai,
- LLM extraction errors propagate ho sakte hain,
- self-evolution wrong paths ko reinforce kar sakta hai. [filecite\turn1file8\L571-L577](#)

Overall journey jo hum follow karenge



Ye architecture exactly report ke offline, online aur continuous phases ke structure ko follow karta hai.

filecite\turn1file2\L172-L218

Next: Layer 0 — Data Model

Ab hum **Layer 0 se start karenge**, kyunki bina Layer 0 samjhe baaki kisi layer ka input/output properly samajhna possible nahi hai.

Layer 0 me hum depth me dekhenge:

- `KnowledgeNode` exactly kya hai
- `KnowledgeEdge` exactly kya hai
- hierarchical edge vs semantic edge
- source of truth kya hai
- `parent/children` field ka update anomaly problem
- single-parent constraint
- cycle detection
- node confidence propagation
- Kahn's topological sort kyun
- evidence ka format
- embedding ka format
- common hospital example ke actual `KnowledgeNode` aur `KnowledgeEdge` objects
- possible schema errors
- missing validation
- exception handling
- aur **Layer 0 ka output Layer 1–8 ko exactly kaise support karta hai**

Next message me sirf Layer 0 ko complete depth me samjhayenge.

YOU 20/08/2026, 11:21:46

thik hai mujhe layer 0 smjhao

CHATGPT 20/08/2026, 11:21:52

```
{"uri":"/response/turn1","start_line":455,"num_lines":10}
```

DEEP RESEARCH 20/08/2026, 11:21:54

Citation Marker: `filecite`turn1file0

[L455] ## 4. Data Model — Layer 0

[L456]

[L457] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` (concepts) and `KnowledgeEdge` (relationships).

[L458]

[L459] ### 4.1 KnowledgeNode Schema

[L460]

[L461] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :

[L462]

[L463] ```json

[L464] {

CHATGPT 20/08/2026, 11:21:54

{"uri":"/response/turn1","start_line":465,"num_lines":180}

DEEP RESEARCH 20/08/2026, 11:21:55

Citation Marker: [filecite]turn1file0[

[L465] "node_id": "NODE_000789",
[L466] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L467] "canonical_name": "Round Robin",
[L468] "aliases": ["RR", "Round-Robin Scheduling"],
[L469] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L470] "summaries": ["CPU scheduling method using time slices"],
[L471] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L472] "domain": "Operating Systems",
[L473] "properties": {
[L474] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L475] "preemptive": {"type": "boolean", "value": true}
[L476] },
[L477] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L478] "parent": "NODE_000456",
[L479] "children": ["NODE_001001", "NODE_001002"],
[L480] "related": ["NODE_000890"],
[L481] "incoming_edges": ["EDGE_001"],
[L482] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L483] "evidence": [
[L484] {
[L485] "evidence_type": "DEFINITION",
[L486] "text": "Round Robin is a preemptive scheduling algorithm...",
[L487] "source_doc": "DOC_000001",
[L488] "source_block": "BLK_000234",
[L489] "page": 42
[L490] }
[L491]],
[L492] metadata:
[L493] query_result_index: 7
[L494] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L495] version_id: 1
[L496] name: SHIA_RAG_Final_Report(4).md
[L497] mime_type: text/markdown
[L498] surface: conversation
[L499] score: 0.03009207275993712
[L500] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--1
[L501] snippet:
[L502] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-

Evolving Knowledge Systems, Natural Language Processing

[L503]

[L504] ---

[L505]

[L506] ## Table of Contents

[L507]

[L508] 1. Introduction

[L509] 2. Literature Review

[L510] 3. System Architecture

[L511] 4. Data Model — Layer 0

[L512] 5. Offline Pipeline — Layers 1-5

[L513] 6. Online Pipeline — Layers 6-7

[L514] 7. Operations Layer — Layer 8

[L515] 8. Novel Contributions

[L516] 9. Mathematical Framework

[L517] 10. Implementation

[L518] 11. Evaluation Framework

[L519] 12. Risk Analysis

[L520] 13. Conclusion & Future Work

[L521]

[L522] ---

[L523]

[L524] ## 1. Introduction

[L525]

[L526] ### 1.1 Problem Statement

[L527]

[L528] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L529]

[L530] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L531] 2. Embed each chunk into a vector space

[L532] 3. At query time, retrieve the top-k most similar chunks

[L533] 4. Feed these chunks as context to an LLM for answer generation

[L534]

[L535] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L536]

[L537] | Problem | Description | Impact |

[L538] |-----|-----|-----|

[L539] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L540] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L541] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L542] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,

A → B → C) |

[L543] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L544]

[L545] ### 1.2 Proposed Solution: SHIA-RAG

[L546]

[L547] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L548]

[L549]

[L550] Traditional RAG:	Document → [Chunk ₁ , Chunk ₂ , ..., Chunk _n] → Top-k → LLM
[L551] SHIA-RAG:	Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM
[L552]	

[L553]

[L554] The system:

[L555] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L556] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L557] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L558] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L559] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L560] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L561] metadata:

[L562] query_result_index: 8

[L563] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L564] version_id: 1

[L565] name: SHIA_RAG_Final_Report(4).md

[L566] mime_type: text/markdown

[L567] surface: conversation

[L568] score: 0.02967032967032967

[L569] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--53

[L570] snippet:

[L571] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") |

Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L572] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is O(nB) where B = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L573] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L574] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L575] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds

and human review |

[L576] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors. ϵ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by ϵ -greedy; future: Thompson sampling |

[L577] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L578]

[L579] ---

[L580]

[L581] ## 13. Conclusion & Future Work

[L582]

[L583] ### 13.1 Summary

[L584]

[L585] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L586]

[L587] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L588] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L589] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L590] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L591] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L592]

[L593] The complete system specification covers:

[L594] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L595] - **12 defined functions** with complete implementations

[L596] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L597] - **8 schemas** covering all data objects from document blocks to query logs

[L598] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L599] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L600]

[L601] ### 13.2 Key Novelties

[L602]

[L603] | # | Novelty | Level | Why It's New |

[L604] |---|-----|-----|-----|

[L605] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L606] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

[L607] metadata:

[L608] query_result_index: 9

[L609] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L610] version_id: 1
 [L611] name: SHIA_RAG_Final_Report(4).md
 [L612] mime_type: text/markdown
 [L613] surface: conversation
 [L614] score: 0.02964426877470356
 [L615] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--31
 [L616] snippet:
 [L617] ##### 5.5.5 CLD — Cross-Link Discoverer
 [L618]
 [L619] After hierarchical placement, discover semantic (non-hierarchical) links:
 [L620]
 [L621]

```
[L622] def cld_discover(node, forest, config):
[L623]     """Find semantic cross-links for a newly placed node."""
[L624]     cross_links = []
[L625]     neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L626]
[L627]     for neighbor in neighbors:
[L628]         if neighbor.id == node.id or neighbor.id == node.parent:
[L629]             continue
[L630]
[L631]         sim = cosine_similarity(node.embedding, neighbor.embedding)
[L632]         if sim > config.cross_link_threshold:
[L633]             relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L634]             if relation_type:
[L635]                 forest.add_edge(node.id, neighbor.id,
[L636]                     edge_class='SEMANTIC', relation_type=relation_type,
[L637]                     confidence=sim, weight=1.0)
[L638]                 cross_links.append((neighbor.id, relation_type, sim))
[L639]
[L640]     return cross_links
[L641]
```

[L642]
 [L643] ---
 [L644]

CHATGPT 20/08/2026, 11:21:55

{"uri":"/response/turn1","query":"### 4.2 KnowledgeEdge Schema","start_line":1}

CHATGPT 20/08/2026, 11:21:57

{"uri":"/response/turn1","start_line":1,"num_lines":5}

DEEP RESEARCH 20/08/2026, 11:21:58

Citation Marker: □filecite□turn1file0□

[L1] metadata:
 [L2] query_result_index: 0
 [L3] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4).md

CHATGPT 20/08/2026, 11:21:58

{"uri":"/response/turn1","start_line":6,"num_lines":3}

DEEP RESEARCH 20/08/2026, 11:21:59

Citation Marker: `filecite`turn1file0

[L6] mime_type: text/markdown

[L7] surface: conversation

[L8] score: 0.03252247488101534

CHATGPT 20/08/2026, 11:21:59

{"paths":["files"],"query":"read"}

DEEP RESEARCH 20/08/2026, 11:22:02

Citation Marker: `filecite`turn2file0

[L1] metadata:

[L2] ref_id: turn1file0

[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4).md

[L6] lines_shown: 455-714

[L7] total_file_lines: 2066

[L8] has_more: True

[L9] next_start_line: 715

[L10] next_read: {"ref_id":"turn1file0","mode":"full","start_line":715,"max_lines":260,"include_images":false}

[L11] content:

[L12] | HTML | BeautifulSoup4 | Web pages |

[L13] | Language detection | langdetect | Auto-detect document language |

[L14] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |

[L15]

[L16] ##### Document Deduplication

[L17]

[L18] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document registry, reject the upload:

[L19]

[L20]


```

[L21] import hashlib
[L22]
[L23] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L24]     """Return True if document is a duplicate."""
[L25]     with open(file_path, 'rb') as f:
[L26]         content_hash = hashlib.sha256(f.read()).hexdigest()
[L27]     if content_hash in doc_registry:
[L28]         raise DuplicateDocumentError(
[L29]             f"Document already ingested as {doc_registry[content_hash]}")
[L30]     return content_hash
[L31]

```

[L32]

[L33] ##### OCR Detection (Multi-Signal)

[L34]

[L35] A single heuristic (`text_chars / page_area < 0.1`) is unreliable. For example, a page with only a large heading ("RETURN POLICY") has little text but is NOT scanned. We use a **multi-signal approach**:

[L36]

[L37]

```

[L38] def needs_ocr(page) -> bool:
[L39]     """Multi-signal OCR detection – more robust than text/area ratio alone.
[L40]
[L41]     Signal 1: Text density (text chars / page area)
[L42]     Signal 2: Font metadata availability (scanned pages lack font info)
[L43]     Signal 3: Image coverage ratio (scanned pages are mostly images)
[L44]     """
[L45]     text = page.get_text()
[L46]     area = page.rect.width * page.rect.height
[L47]
[L48]     # Signal 1: Text density
[L49]     text_density = len(text.strip()) / max(area, 1)
[L50]
[L51]     # Signal 2: Font metadata (scanned PDFs have no embedded fonts)
[L52]     has_fonts = len(page.get_fonts()) > 0
[L53]
[L54]     # Signal 3: Image coverage (scanned pages are large images)
[L55]     images = page.get_images()
[L56]     image_area = sum(img[2] * img[3] for img in images) if images else 0
[L57]     image_coverage = image_area / max(area, 1)
[L58]
[L59]     # Decision: OCR needed if text is sparse AND (no fonts OR high image coverage)
[L60]     if text_density < 0.1 and (not has_fonts or image_coverage > 0.5):
[L61]         return True
[L62]     return False
[L63]

```

[L64]

[L65] ##### OCR Confidence Calculation

[L66]

[L67]

```

[L68] def calculate_ocr_confidence(ocr_result) -> float:
[L69]     """Calculate OCR confidence from Tesseract per-word confidence scores.
[L70]
[L71]     Tesseract returns per-word confidence (0-100). We:
[L72]     1. Get per-word confidences
[L73]     2. Filter out words with confidence < 10 (noise)
[L74]     3. Compute weighted average by word length (longer words = more reliable signal)
[L75]     4. Normalize to [0.0, 1.0]
[L76]     """
[L77]     word_data = ocr_result.get('words', [])
[L78]     if not word_data:
[L79]         return 0.0
[L80]
[L81]     total_weight, weighted_sum = 0, 0
[L82]     for word in word_data:
[L83]         if word['confidence'] >= 10: # Filter noise
[L84]             weight = len(word['text']) # Weight by word length
[L85]             weighted_sum += word['confidence'] * weight
[L86]             total_weight += weight
[L87]
[L88]     return (weighted_sum / max(total_weight, 1)) / 100.0
[L89]

```

[L90]

[L91] ##### Text Cleaning (Safe)

[L92]

[L93]

```

[L94] def clean_text(text: str, pages: list) -> str:
[L95]     """Clean extracted text while preserving legitimate content.
[L96]
[L97]     Safeguards against false header/footer removal:
[L98]     1. Identify candidate headers/footers by frequency across pages
[L99]     2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L100]     3. Never remove lines that appear on fewer than 3 pages
[L101]     4. Normalize whitespace but preserve paragraph boundaries
[L102]     """
[L103]     if len(pages) >= 3:
[L104]         # Frequency-based header/footer detection
[L105]         first_lines = [p.split('\n')[0].strip() for p in pages if p.strip()]
[L106]         last_lines = [p.split('\n')[-1].strip() for p in pages if p.strip()]
[L107]
[L108]         from collections import Counter
[L109]         header_counts = Counter(first_lines)
[L110]         footer_counts = Counter(last_lines)
[L111]
[L112]         threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L113]         headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L114]         footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L115]
[L116]         lines = text.split('\n')
[L117]         lines = [l for l in lines
[L118]                 if l.strip() not in headers_to_remove
[L119]                 and l.strip() not in footers_to_remove]
[L120]         text = '\n'.join(lines)
[L121]
[L122]     # Normalize whitespace (preserve paragraph breaks)
[L123]     import re
[L124]     text = re.sub(r'[ \t]+', ' ', text) # Collapse horizontal space
[L125]     text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L126]     return text.strip()
[L127]

```

[L128]

[L129] ##### Language Detection

[L130]

[L131]

```
[L132] from langdetect import detect
[L133]
[L134] def detect_language(text: str) -> str:
[L135]     """Detect document language for OCR configuration.
[L136]
[L137]     Used to configure Tesseract's language parameter.
[L138]     Falls back to 'eng' if detection fails."""
[L139]     try:
[L140]         lang = detect(text[:5000]) # Sample first 5000 chars
[L141]         # Map langdetect codes to Tesseract codes
[L142]         lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L143]         return lang_map.get(lang, 'eng')
[L144]     except Exception:
[L145]         return 'eng'
[L146]
```

[L147]

[L148] ##### Partial Extraction Policy

[L149]

[L150] If some pages fail during extraction, the system uses a **best-effort policy**:

[L151]

[L152] | Condition | Action |

[L153] |-----|-----|

[L154] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |

[L155] | <80% pages succeed | Reject entire document, queue for manual review |

[L156] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |

[L157]

[L158]

```

[L159] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L160]     """Layer 1: Input Processing pipeline with all safeguards."""
[L161]     # Step 0: Duplicate check
[L162]     content_hash = check_duplicate(file_path, doc_registry)
[L163]
[L164]     fmt = detect_format(file_path)
[L165]     validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L166]
[L167]     failed_pages = []
[L168]     if fmt == 'pdf':
[L169]         text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L170]         # Partial extraction policy
[L171]         success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L172]         if success_rate < 0.8:
[L173]             raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L174]     elif fmt == 'docx':
[L175]         text, pages = extract_docx(file_path), []
[L176]         is_ocr, ocr_conf = False, 1.0
[L177]     elif fmt == 'html':
[L178]         text, pages = extract_html(file_path), []
[L179]         is_ocr, ocr_conf = False, 1.0
[L180]     else: # txt
[L181]         text = open(file_path, 'r', encoding='utf-8').read()
[L182]         pages, is_ocr, ocr_conf = [], False, 1.0
[L183]
[L184]     language = detect_language(text)
[L185]     text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L186]     metadata = extract_metadata(file_path, fmt)
[L187]
[L188]     doc = NormalizedDocument(
[L189]         doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L190]         format=DocumentFormat(fmt), text=text, pages=pages,
[L191]         metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L192]         char_count=len(text), page_count=len(pages) or 1,
[L193]         content_hash=content_hash, language=language,
[L194]         partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L195]     )
[L196]     doc_registry[content_hash] = doc.doc_id # Register for dedup
[L197]     return doc
[L198]

```

[L199]

[L200] ---

[L201]

[L202] ### 5.2 Layer 2: Document Parsing

[L203]

[L204] **Responsibility:** Parse normalized text into a structured [Page → Region → Block] hierarchy with block classification and reading order.

[L205]

[L206] **Input:** NormalizedDocument → **Output:** List of DocumentBlock

[L207]

[L208]

```

[L209] class BlockType(Enum):
[L210]     HEADING = "heading"
[L211]     PARAGRAPH = "paragraph"
[L212]     LIST = "list"
[L213]     TABLE = "table"
[L214]     CODE = "code"
[L215]     ALGORITHM = "algorithm"
[L216]     FIGURE_CAPTION = "figure_caption"
[L217]     EQUATION = "equation"
[L218]     DEFINITION = "definition"
[L219]
[L220] @dataclass
[L221] class DocumentBlock:
[L222]     """Atomic unit of document structure."""
[L223]     block_id: str
[L224]     doc_id: str
[L225]     page_number: int
[L226]     region_index: int          # Which region on the page
[L227]     block_type: BlockType
[L228]     text: str
[L229]     reading_order: int         # Global ordering across the document
[L230]     confidence: float = 1.0    # Classification confidence
[L231]     table_cells: list = None   # Parsed table cells (for TABLE blocks only)
[L232]

```

[L233]

[L234] The parser uses rule-based heuristics for block classification:

[L235] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase

[L236] - **List:** Starts with -, *, •, or 1.

[L237] - **Table:** Contains 3+ pipe characters (|) — uses regex `re.match(r'^(def |class |import |from )', line)` for code

[L238] - **Code:** Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")

[L239] - **Definition:** Contains phrases like "is defined as", "refers to"

[L240] - **Default:** Paragraph

[L241]

[L242] ##### Table Cell-Level Parsing

[L243]

[L244] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-level structure:

[L245]

[L246]

```
[L247] def parse_table_cells(page, table_region) -> list:
[L248]     """Extract table structure with cell-level detail.
[L249]
[L250]     Uses pdfplumber for text-based tables, Camelot for image-based.
[L251]     Returns list of rows, each row is a list of cell strings."""
[L252]     try:
[L253]         # Primary: pdfplumber table extraction
[L254]         table = page.extract_table(table_region)
[L255]         if table and len(table) > 1: # At least header + 1 data row
[L256]             return table
[L257]     except Exception:
[L258]         pass
[L259]
[L260]     # Fallback: Camelot for image-based tables
[L261]     try:
[L262]         import camelot
[L263]         tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L264]         if tables:
[L265]             return tables[0].df.values.tolist()
[L266]     except Exception:
[L267]         pass
[L268]
[L269]     return None # Table structure could not be parsed
[L270]
```

DEEP RESEARCH 20/08/2026, 11:22:05

Citation Marker: `filecite`turn3file0

[L1] Find: find=0 ref=turn1file0 name=SHIA_RAG_Final_Report(4).md pattern=### 4.2 KnowledgeEdge Schema

[L2] Scope: full file surface=conversation

[L3] Matches: shown=1 total=1

[L4] [match 0 | match_lines 296 | shown_lines 291-301]

[L5] - **parent / children** are **convenience-only** fields derived from hierarchical edges. In production, always derive these from edge queries to avoid update anomalies (e.g., `parent` field says `NODE_100` but the actual edge points to `NODE_200`). These fields MUST NOT be treated as the source of truth.

[L6] - **confidence** is propagated from extraction confidence, parent confidence, and evidence confidence using topological sort (see Section 4.4).

[L7] - **abstraction_level** (0.0 = very specific, 1.0 = very abstract) is used by CPG++ for parent filtering.

[L8] - **extraction_confidence** (not shown above) stores the original LLM extraction confidence before propagation. This is the immutable baseline.

[L9]

[L10] ### 4.2 KnowledgeEdge Schema

[L11]

[L12] Relationships between nodes are represented as `KnowledgeEdge` objects. There are **two edge classes**:

[L13]

[L14] | Edge Class | Structural Rule | Example |

[L15] |-----|-----|-----|

DEEP RESEARCH 20/08/2026, 11:22:05

Citation Marker: `filecite`turn3file1

[L16] Find: find=1 ref=turn1file0 name=SHIA_RAG_Final_Report(4).md pattern=### 4.3 Edge Type Ontology

[L17] Scope: full file surface=conversation

[L18] Matches: shown=0 total=0

[L19] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 20/08/2026, 11:22:05

Citation Marker: `filecite`turn3file2

[L20] Find: find=2 ref=turn1file0 name=SHIA_RAG_Final_Report(4).md pattern=### 4.4 Schema Validation

[L21] Scope: full file surface=conversation

[L22] Matches: shown=0 total=0

[L23] No matches were found in the requested range. A zero-match find does not establish that the answer is absent. If the wording or location is uncertain, call `files.search` for ranked semantic retrieval; otherwise retry `files.find` with a broader or alternate exact pattern.

DEEP RESEARCH 20/08/2026, 11:22:05

Citation Marker: `filecite`turn3file3

[L24] Find: find=3 ref=turn1file0 name=SHIA_RAG_Final_Report(4).md pattern=KnowledgeEdge

[L25] Scope: full file surface=conversation

[L26] Matches: shown=10 total=10

[L27] [match 0 | match_lines 212 | shown_lines 207-217]

[L28] ↓ Layer 3: Split sentences, extract concepts/definitions/relations

[L29] KnowledgeUnit objects (candidate concepts)

[L30] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)

[L31] KnowledgeNode objects (canonical, deduplicated)

[L32] ↓ Layer 4: Validate relations against ontology, score confidence

[L33] KnowledgeEdge objects (typed, scored)

[L34] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)

[L35] Knowledge Forest (rooted trees + cross-links)

[L36] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection

[L37] Selected context nodes (within token budget)

[L38] ↓ Layer 7: Build prompt, invoke LLM

[L39] [matches 1-3 | match_lines 227,231,241 | shown_lines 222-246]

[L40]

[L41] ### 3.3 Layer Summary

[L42]

[L43] | Layer | Name | Input | Output | Key Algorithms |

[L44] |-----|-----|-----|-----|-----|

[L45] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L46] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L47] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L48] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |

[L49] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |

[L50] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |

[L51] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |

[L52] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |

[L53] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |

[L54]

[L55] ---

[L56]

[L57] ## 4. Data Model — Layer 0

[L58]

[L59] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` (concepts) and `KnowledgeEdge` (relationships).

[L60]

[L61] ### 4.1 KnowledgeNode Schema

[L62]

[L63] Each concept in the Knowledge Forest is represented as a `KnowledgeNode` :

[L64] [matches 4-5 | match_lines 296,298 | shown_lines 291-303]

[L65] - **parent** / **children** are **convenience-only** fields derived from hierarchical edges. In production, always derive these from edge queries to avoid update anomalies (e.g., `parent` field says `NODE_100` but the actual edge points to `NODE_200`). These fields MUST NOT be treated as the source of truth.

[L66] - **confidence** is propagated from extraction confidence, parent confidence, and evidence confidence using topological sort (see Section 4.4).

[L67] - **abstraction_level** (0.0 = very specific, 1.0 = very abstract) is used by CPG++ for parent filtering.

[L68] - **extraction_confidence** (not shown above) stores the original LLM extraction confidence before propagation. This is the immutable baseline.

[L69]

[L70] ### 4.2 KnowledgeEdge Schema

[L71]

[L72] Relationships between nodes are represented as `KnowledgeEdge` objects. There are **two edge classes**:

[L73]

[L74] | Edge Class | Structural Rule | Example |

[L75] |-----|-----|-----|

[L76] | **HIERARCHICAL** | Must form an **acyclic forest** (single parent per node) | IS_A, PART_OF, SUBCLASS_OF, INSTANCE_OF |

[L77] | **SEMANTIC** | Unrestricted **graph** (cycles allowed) | USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIMILAR_TO |

[L78] [matches 6-7 | match_lines 906,908 | shown_lines 901-913]

[L79]

[L80] ---

[L81]

[L82] ### 5.4 Layer 4: Relation Management

[L83]

[L84] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve cross-document conflicts, and produce final `KnowledgeEdge` objects.

[L85]

[L86] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L87]

[L88] **Pipeline:**

[L89] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L90] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L91] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L92] [match 8 | match_lines 933 | shown_lines 928-938]

[L93] ##### Cross-Document Conflict Resolution

[L94]

[L95] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Round Robin IS_A Scheduling" but Document B says "Round Robin PART_OF OS"), the system resolves conflicts using evidence-weighted voting:

[L96]

[L97]

```
[L98] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:
[L99]     """Resolve conflicting relations from different documents.
[L100]
[L101]     Strategy: Evidence-weighted voting
[L102]     1. Group edges by (source_node, target_node) pair
[L103]     2. For each group, tally votes per relation_type weighted by evidence confidence
[L104] [match 9 | match_lines 993 | shown_lines 988-998]
[L105]
[L106] ### 5.5 Layer 5: SHIA Core – Hierarchy Construction
[L107]
[L108] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage
[L109]
[L110] **Input:** `KnowledgeNode[]` + `KnowledgeEdge[]` → **Output:** Knowledge Forest
[L111]
[L112] ##### Pipeline Overview
[L113]
[L114]
```

mermaid

[L115] graph LR

DEEP RESEARCH 20/08/2026, 11:22:08

Citation Marker: `filecite[turn4file0]`

[L1] metadata:

[L2] ref_id: turn1file0

[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L4] version_id: 1

[L5] name: SHIA_RAG_Final_Report(4).md

[L6] lines_shown: 270-449

[L7] total_file_lines: 2066

[L8] has_more: True

[L9] next_start_line: 450

[L10] next_read: {"ref_id":"turn1file0","mode":"full","start_line":450,"max_lines":180,"include_images":false}

[L11] content:

[L12] "text": "Round Robin is a preemptive scheduling algorithm...",

[L13] "source_doc": "DOC_000001",

[L14] "source_block": "BLK_000234",

[L15] "page": 42

[L16] }

```

[L17] ],
[L18] "embeddings": [
[L19] {
[L20] "model": "bge-base-en-v1.5",
[L21] "dimension": 768,
[L22] "vector": [0.123, -0.456, "..."],
[L23] "created_at": "2026-08-17T10:00:00Z"
[L24] }
[L25] ],
[L26] "confidence": 0.92,
[L27] "abstraction_level": 0.35,
[L28] "statistics": {"access_count": 0, "update_count": 1, "merge_count": 0}
[L29] }
[L30]

```

```

[L31]
[L32] **Key design decisions:**
[L33] - **`parent` / `children`** are **convenience-only** fields derived from hierarchical edges. In produc
[L34] - **`confidence`** is propagated from extraction confidence, parent confidence, and evidence confidenc
[L35] - **`abstraction_level`** (0.0 = very specific, 1.0 = very abstract) is used by CPG++ for parent filter
[L36] - **`extraction_confidence`** (not shown above) stores the original LLM extraction confidence before p
[L37]
[L38] ### 4.2 KnowledgeEdge Schema
[L39]
[L40] Relationships between nodes are represented as `KnowledgeEdge` objects. There are **two edge classes**
[L41]
[L42] | Edge Class | Structural Rule | Example |
[L43] |-----|-----|-----|
[L44] | `HIERARCHICAL` | Must form an **acyclic forest** (single parent per node) | IS_A, PART_OF, SUBCLASS_
[L45] | `SEMANTIC` | Unrestricted **graph** (cycles allowed) | USES, CAUSES, DEPENDS_ON, CONTRASTS_WITH, SIM
[L46]
[L47]

```

json

```

[L48] {
[L49] "edge_id": "EDGE_001",
[L50] "source_node": "NODE_000789",
[L51] "target_node": "NODE_000456",
[L52] "edge_class": "HIERARCHICAL",
[L53] "relation_type": "IS_A",
[L54] "confidence": 0.95,
[L55] "weight": 1.0,
[L56] "evidence": [{"text": "Round Robin is a type of CPU scheduling algorithm.", "source_doc": "DOC_001", "page":
42}],
[L57] "metadata": {"source_docs": ["DOC_001"]}
[L58] }
[L59]

```

```

[L60]
[L61] **Edge class enforcement:**
[L62]
[L63]

```

python

```
[L64] class NodeNotFoundError(Exception):
[L65] """Raised when a referenced node does not exist in the forest."""
[L66] pass
[L67]
[L68] def validate_edge(edge, forest):
[L69] """Validate edge based on its class.
[L70]
[L71] NOTE: We use explicit exceptions instead of assert statements because
[L72] Python's -O flag disables assertions in production."""
[L73] # Both edge classes require valid nodes
[L74] if not forest.has_node(edge.source_node):
[L75] raise NodeNotFoundError(f"Source node {edge.source_node} not found")
[L76] if not forest.has_node(edge.target_node):
[L77] raise NodeNotFoundError(f"Target node {edge.target_node} not found")
[L78]
[L79] if edge.edge_class == "HIERARCHICAL":
[L80] # MUST be acyclic — check for cycles
[L81] if creates_cycle(edge.source_node, edge.target_node, forest):
[L82] raise CycleError(f"HIERARCHICAL edge would create a cycle")
[L83] # MUST respect single-parent constraint
[L84] validate_single_parent(forest.get_node(edge.target_node), edge.source_node, forest)
[L85] elif edge.edge_class == "SEMANTIC":
[L86] # Cycles are ALLOWED — node existence already validated above
[L87] pass
[L88]
```

```
[L89]
[L90] ### 4.3 Relation Ontology
[L91]
[L92] The complete set of supported relation types:
[L93]
[L94]
```

yaml

```
[L95] hierarchical_relations:
[L96] - IS_A # "Round Robin IS_A CPU Scheduling Algorithm"
[L97] - PART_OF # "ALU PART_OF CPU"
[L98] - SUBCLASS_OF # "OSPF SUBCLASS_OF Routing Protocol"
[L99] - INSTANCE_OF # "Linux INSTANCE_OF Operating System"
[L100]
[L101] semantic_relations:
[L102] - USES # "TCP USES Congestion Control"
[L103] - CAUSES # "Deadlock CAUSES System Hang"
[L104] - DEPENDS_ON # "Virtual Memory DEPENDS_ON Paging"
[L105] - CONTRASTS_WITH # "Preemptive CONTRASTS_WITH Non-Preemptive"
[L106] - SIMILAR_TO # "Round Robin SIMILAR_TO Fair-Share"
[L107] - HAS_PROPERTY # "Round Robin HAS_PROPERTY Time Quantum"
[L108] - PRODUCES # "Compiler PRODUCES Machine Code"
```

[L109] - PREREQUISITE # "Data Structures PREREQUISITE Algorithms"

[L110]

```
[L111]
[L112] ### 4.4 Confidence Propagation
[L113]
[L114] Node confidence is propagated from roots to leaves using topological sort (Kahn's algorithm) to c
[L115]
[L116]
```

python

[L117] from collections import deque

[L118]

[L119] # Confidence weighting factors (must sum to 1.0)

[L120] ALPHA = 0.4 # Node's own extraction confidence weight

[L121] BETA = 0.4 # Parent confidence weight

[L122] GAMMA = 0.2 # Evidence confidence weight

[L123] DECAY = 0.95 # Per-level decay factor

[L124] BASE_CONFIDENCE = 0.5 # Default for root nodes (configurable)

[L125]

[L126] def propagate_confidence(forest):

[L127] """Propagate confidence scores from roots to leaves.

[L128]

[L129] Uses Kahn's algorithm (topological ordering) to ensure

[L130] parents are always processed before their children.

[L131] Processing in arbitrary order would give children stale

[L132] parent confidence values."""

[L133] in_degree = {n.id: len(forest.parents(n)) for n in forest.nodes() }

[L134] queue = deque([n for n in forest.nodes() if in_degree[n.id] == 0]) # Roots first

[L135]

[L136] while queue:

[L137] node = queue.popleft()

[L138] if forest.parents(node):

[L139] parent_confs = [p.confidence for p in forest.parents(node)]

[L140] avg_parent_conf = sum(parent_confs) / len(parent_confs)

[L141] evidence_conf = (

[L142] sum(e.confidence for e in node.evidence) / len(node.evidence)

[L143] if node.evidence else 0.5

[L144])

[L145] node.confidence = (

[L146] ALPHA *node.extraction_confidence* +

[L147] BETA avg_parent_conf +

[L148] GAMMA *evidence_conf*

[L149]) DECAY

[L150] else:

[L151] node.confidence = node.extraction_confidence or BASE_CONFIDENCE

[L152]

[L153] for child in forest.children(node):

```
[L154] in_degree[child.id] -= 1
[L155] if in_degree[child.id] == 0:
[L156] queue.append(child)
[L157]
```

```
[L158]
[L159] ---
[L160]
[L161] ## 5. Offline Pipeline – Layers 1-5
[L162]
[L163] ### 5.1 Layer 1: Input Processing
[L164]
[L165] **Responsibility:** Accept raw documents (PDF, DOCX, HTML, TXT), validate format, deduplicate, extract
[L166]
[L167] **Input:** Raw document file → **Output:** `NormalizedDocument`
[L168]
[L169]
```

python

```
[L170] @dataclass
[L171] class NormalizedDocument:
[L172] """Output of Layer 1 — ready for Layer 2 parsing."""
[L173] doc_id: str
[L174] original_filename: str
[L175] format: DocumentFormat # PDF, DOCX, HTML, TXT
[L176] text: str # Full extracted text
[L177] pages: list # List of page texts (for PDFs)
[L178] metadata: dict # Author, title, date, etc.
[L179] is_ocr: bool = False # True if OCR was used
[L180] ocr_confidence: float = 1.0 # Average OCR confidence score
[L181] char_count: int = 0
[L182] page_count: int = 0
[L183] content_hash: str = "" # SHA-256 hash for deduplication
[L184] language: str = "en" # Detected language code
[L185] partial_extraction: bool = False # True if some pages failed
[L186] failed_pages: list = None # List of page numbers that failed
[L187] ```
[L188]
[L189] Technology choices:
[L190]
[L191] | Component | Technology | Purpose |
```

CHATGPT 20/08/2026, 11:22:08

Layer 0 — Data Model / Foundation

Ab hum sirf **Layer 0** samjhenge.

Sabse pehle ek important baat:

Layer 0 koi document ko process nahi karti.

Iska main kaam hai poore SHIA-RAG system ke liye **common data structures, schemas aur rules define karna**.

Report ke according Layer 0 ke do core objects hain:

- **KnowledgeNode** → knowledge/concept represent karta hai
- **KnowledgeEdge** → do concepts ke beech relationship represent karta hai

Aur poori pipeline ka basic contract ye hai ki Layer 3 Nodes banayegi, Layer 4 Edges banayegi, aur Layer 5 in Nodes + Edges se Knowledge Forest banayegi.

1. Layer 0 ko simple language me samjho

Maan lo hum ek **Smart Hospital** ka knowledge system bana rahe hain.

Humein concepts milte hain:

```
Patient
Emergency Triage
Critical Patient
Medical Image
AI Diagnosis
CNN
Disease Prediction
Doctor Verification
Final Diagnosis
Authentication
RBAC
Encryption
Audit Log
```

Ab computer ko ye kaise pata chalega ki:

- **AI Diagnosis** kya hai?
- iska evidence kahan se aaya?
- iska alias kya hai?
- iska parent kaun hai?
- ye kis concept se related hai?
- relationship **IS_A** hai ya **USES** ?
- relation trustworthy hai ya nahi?
- retrieval ke time iska vector kaunsa use karna hai?

Iske liye **Layer 0** pehle se structure define karti hai.

Simple form me:

```
Real-world knowledge
↓
KnowledgeNode
+
KnowledgeEdge
↓
Knowledge Forest
```

2. Layer 0 ka exact input kya hai?

Strictly speaking: Layer 0 ka runtime input nahi hai

Architecture summary me Layer 0 ka input essentially **schemas/data contracts** hain aur output bhi schemas/types hi hain. `filecite\turn1file1\L131-L143`

Isliye:

```
Layer 0

Input:
    No runtime document/query input

Processing:
    Define data contracts + validation rules

Output:
    KnowledgeNode schema
    KnowledgeEdge schema
    Relation ontology
    Confidence propagation rules
```

Important difference

Layer 1 ko actual PDF milti hai.

```
PDF → Layer 1
```

Lekin Layer 0 ko aisa kuch directly nahi milta.

Layer 0 pehle se system ko bolti hai:

"Future me agar tum concept store karoge, to iss format me karna."

Aur:

"Future me agar do concepts connect karoge, to iss format aur in rules ke according karna."

3. Layer 0 ko foundation kyun bolte hain?

Socho Layer 0 nahi hai.

Layer 3 bol sakti hai:

```
{
  "name": "AI Diagnosis",
  "confidence": "high"
}
```

Layer 4 bol sakti hai:

```
{
  "from": "AI Diagnosis",
  "to": "Medical Image",
  "relation": "uses"
}
```

Layer 5 shayad expect kare:

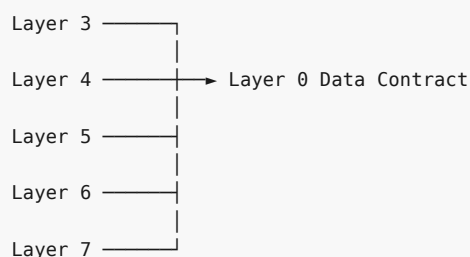
```
{
  "source_node": "...",
  "target_node": "...",
  "relation_type": "USES"
}
```

Yahan problem aa gayi:

name	≠ canonical_name
from	≠ source_node
to	≠ target_node
uses	≠ USES
"high"	≠ 0.92

Isliye har layer apna random format use nahi kar sakti.

Layer 0 ek contract banati hai.



4. Layer 0 ke do main objects

A. KnowledgeNode

Ek KnowledgeNode = ek canonical knowledge concept.

Example:

```
AI Diagnosis
```

Ye sirf ek string nahi hai.

SHIA-RAG me iske saath associated ho sakta hai:

- unique ID
- UUID
- canonical name
- aliases
- definitions
- summaries
- keywords
- domain
- properties
- examples
- parent
- children
- related nodes

- incoming/outgoing edges
- evidence
- embeddings
- confidence
- abstraction level
- statistics

Report ke sample `KnowledgeNode` schema me exactly isi type ki information define ki gayi hai.

filecite\turn1file6\L455-L490\ filecite\turn4file0\L18-L29

5. Hamare Smart Hospital example me KnowledgeNode

Maan lo Layer 3 ne concept extract kiya:

AI-assisted emergency diagnosis

Ab Layer 0 ke schema ke according ise approximately is tarah represent kiya jayega:

```
KnowledgeNode
├── Identity
│   ├── node_id
│   └── uuid
├── Meaning
│   ├── canonical_name
│   ├── aliases
│   ├── definitions
│   └── summaries
├── Searchability
│   ├── keywords
│   └── embeddings
├── Knowledge attributes
│   ├── domain
│   ├── properties
│   └── examples
├── Relationships
│   ├── parent
│   ├── children
│   ├── related
│   ├── incoming_edges
│   └── outgoing_edges
├── Trust
│   ├── evidence
│   └── confidence
└── Hierarchy
    └── abstraction_level
```

Ab isko ek actual example se samjho.

6. node_id aur uuid

Example:

```
{
  "node_id": "NODE_000245",
  "uuid": "...
}
```

node_id

System ke andar readable identifier:

```
NODE_000245
```

Iska use:

- Graph database references
- Debugging
- Logging
- Edge connections

Example:

```
NODE_000245 = AI Diagnosis
NODE_000246 = Medical Image
```

Relation:

```
EDGE_000501

AI Diagnosis —USES→ Medical Image
```

Internally:

```
NODE_000245 —USES→ NODE_000246
```

uuid

Ye globally unique identifier hai.

Simple ID kabhi migration ya distributed systems me conflict create kar sakti hai. UUID ka purpose globally unique identity maintain karna hai.

7. canonical_name

Ye concept ka **main standardized naam** hai.

Hamare documents me same cheez multiple names se aa sakti hai:

```
AI-assisted diagnosis
AI diagnosis
Automated medical diagnosis
```

Layer 3 alias resolution ke baad system ek canonical name choose kar sakta hai:

```
canonical_name = "AI Diagnosis"
```

Baaki:

```
aliases = [  
    "AI-assisted diagnosis",  
    "Automated medical diagnosis"  
]
```

Why important?

Agar canonicalization nahi hogi:

```
AI Diagnosis  
AI-assisted Diagnosis  
Automated Diagnosis
```

teen alag Nodes ban sakte hain.

Phir retrieval fragmented ho jayega.

8. definitions

Ye concept ki detailed meaning store karta hai.

Example:

```
AI Diagnosis:  
  
"An automated diagnostic process in which an AI model analyzes  
medical data or medical images to assist in disease prediction."
```

Important point:

Ek concept ke **multiple definitions** ho sakte hain.

```
definitions = [  
    Definition from Document A,  
    Definition from Document D  
]
```

Isse cross-document knowledge preserve hoti hai.

9. summaries

Definition detailed hoti hai.

Summary compact hota hai.

Example:

```
Definition:  
AI diagnosis uses computational models to analyze medical data.  
  
Summary:  
AI-assisted disease prediction.
```

Difference

```
Definition → Accurate explanation  
Summary → Fast retrieval / compact context
```

Later Layer 6 ke token budget me ye useful ho sakta hai.

10. keywords

Example:

```
[  
  "AI",  
  "medical diagnosis",  
  "disease prediction",  
  "medical image",  
  "emergency"  
]
```

Ye retrieval aur semantic understanding me supporting metadata provide karte hain.

11. domain

Concept kis domain ka hai?

Example:

```
domain = "Healthcare AI"
```

Ya:

```
domain = "Hospital Information Systems"
```

Why useful?

Same word different domains me different meaning de sakta hai.

Example:

```
"Admission"  
  
Hospital → patient admission  
University → student admission
```

Domain disambiguation me help karta hai.

12. properties

Properties structured facts ko represent karti hain.

Example:

```
AI Diagnosis
```

ki properties:

```
{
  "requires_doctor_verification": {
    "type": "boolean",
    "value": true
  },
  "input_type": {
    "type": "string",
    "value": "Medical Image"
  }
}
```

Report ke example me `time_quantum` aur `preemptive` jaise structured properties use kiye gaye hain.

📄filecite🔄turn1file6📄L467-L477📄

Why normal text enough nahi?

Agar user pooche:

"Does AI Diagnosis require doctor verification?"

Structured property se answer directly support ho sakta hai.

13. examples

Example:

```
"A critical patient arrives at the emergency department.
An X-ray is captured. The AI model analyzes it and predicts a
possible lung abnormality. A doctor then verifies the prediction."
```

Examples ka purpose concept ko sirf abstract definition tak limited nahi rakhna.

14. Evidence — Layer 0 ka bahut important part

Ye SHIA-RAG ke important parts me se ek hai.

Concept sirf:

AI Diagnosis exists

nahi.

System ko pata hona chahiye:

Ye knowledge aayi kahan se?

Example:

```
{
  "evidence_type": "DEFINITION",
  "text": "The AI diagnostic module analyzes medical images...",
  "source_doc": "DOC_HOSPITAL_AI",
  "source_block": "BLK_00452",
  "page": 27
}
```

Report ke schema me evidence ke saath `evidence_type`, original text, source document, source block aur page information rakhi gayi hai. 📄filecite🔄turn1file6📄L483-L490📄

Hamare example me

KnowledgeNode: AI Diagnosis

Evidence 1:
Document D
Page 12
Block BLK_044

Evidence 2:
Document A
Page 37
Block BLK_190

Isse later Layer 7 bol sakti hai:

"Maine jo claim generate kiya hai, iska actual evidence ye source hai."

15. Embeddings

Report ke Node schema me embeddings ka structure bhi defined hai:

- model name
- vector dimension
- vector
- creation time `filecite\turn4file0\18-L24`

Example:

```
AI Diagnosis
  ↓
Embedding Model
  ↓
[0.123, -0.456, 0.891, ...]
```

Saath me:

```
model = bge-base-en-v1.5
dimension = 768
```

Why model name store karna important hai?

Agar future me embedding model change kar diya:

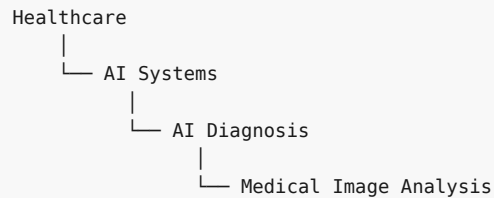
```
Old model → 768 dimensions
New model → 1024 dimensions
```

Agar metadata nahi hoga to vectors incompatible ho sakte hain.

16. parent, children, related

Yahan ek **bahut important design decision** hai.

Hamare example me:



To:

```
parent(AI Diagnosis) = AI Systems
```

Aur:

```
children(AI Diagnosis) =
[
  Medical Image Analysis,
  Disease Prediction
]
```

`related` me non-hierarchical concepts aa sakte hain:

```
related(AI Diagnosis) =
[
  Doctor Verification,
  Medical Image
]
```

17. Lekin parent/children source of truth nahi hain ⚠

Report explicitly kehti hai:

parent aur children convenience-only fields hain.

Production me inhe hierarchical edges se derive karna chahiye, kyunki otherwise update anomaly ho sakti hai.

📄filecite🔄turn4file0📄L32-L36📄

Example of dangerous inconsistency

Node me:

```
AI Diagnosis.parent = AI Systems
```

Lekin actual graph edge:

```
AI Diagnosis —IS_A→ Medical AI
```

Ab system ke paas do different truths hain.

```
Node field says:    AI Systems
Edge database says: Medical AI
```

Which one is correct?

Isliye report ka rule:

```
SOURCE OF TRUTH = EDGE
parent/children = derived convenience fields
```

Ye bahut important design principle hai.

18. confidence

Har Node equally trustworthy nahi hoga.

Example:

```
AI Diagnosis → 0.94
CNN → 0.98
Experimental Diagnosis Method → 0.51
```

Report me confidence propagation ka mechanism defined hai.

Node ka final confidence combine karta hai:

1. Node ki own extraction confidence
2. Parent confidence
3. Evidence confidence

Weights:

```
ALPHA = 0.4 → extraction confidence
BETA = 0.4 → parent confidence
GAMMA = 0.2 → evidence confidence

DECAY = 0.95
```

Report ka formula:

```
final node confidence
=
(
  0.4 × extraction confidence
+ 0.4 × parent confidence
+ 0.2 × evidence confidence
)
× 0.95
```

filecite\turn4file0\L112-L157

19. Hamare example me confidence calculation

Suppose:

```
AI Diagnosis

Extraction confidence = 0.90
Parent confidence     = 0.95
Evidence confidence   = 0.85
```

Then:

$$\begin{aligned}
 &= (0.4 \times 0.90 + 0.4 \times 0.95 + 0.2 \times 0.85) \times 0.95 \\
 &= (0.36 + 0.38 + 0.17) \times 0.95 \\
 &= 0.91 \times 0.95 \\
 &= 0.8645
 \end{aligned}$$

So approximately:

AI Diagnosis confidence = 0.8645

20. Topological Sort kyun use hota hai?

Report **Kahn's algorithm** use karti hai.

Reason:

Parent ki confidence pehle calculate honi chahiye, phir child ki.

Report specifically arbitrary processing order se stale parent confidence ka problem batati hai.

□filecite□turn4file0□L126-L156□

Example:

```

Healthcare
  ↓
AI Systems
  ↓
AI Diagnosis
  ↓
Disease Prediction
  
```

Correct processing:

1. Healthcare
2. AI Systems
3. AI Diagnosis
4. Disease Prediction

Wrong processing:

1. AI Diagnosis ✖
2. Healthcare
3. AI Systems

First case me AI Diagnosis ko final parent confidence available hai.

Second case me stale/default confidence use ho sakti hai.

21. extraction_confidence VS confidence

Report me ek subtle but important distinction hai.

Extraction confidence

Original LLM confidence

Example:

```
extraction_confidence = 0.90
```

Ye original baseline hai.

Final confidence

Propagation ke baad:

```
confidence = 0.8645
```

Report explicitly kehti hai ki `extraction_confidence` immutable baseline hai aur final `confidence` propagation se update hoti hai. `filecite\turn4file0\L32-L36`

22. abstraction_level

Ye Layer 0 ka bahut important field hai.

Range:

```
0.0 → very specific  
1.0 → very abstract
```

Report ke according CPG++ hierarchy construction me parent filtering ke liye use hota hai. `filecite\turn4file0\L32-L36`

Hamare example me:

Healthcare	abstraction = 0.95
Emergency Diagnosis	abstraction = 0.70
AI Diagnosis	abstraction = 0.50
Chest X-ray Classification	abstraction = 0.20
Patient X-ray #123	abstraction = 0.02

Hierarchy ideally:

```
Abstract  
↓  
Healthcare  
↓  
Emergency Diagnosis  
↓  
AI Diagnosis  
↓  
Specific  
Chest X-ray Classification
```

Ye Layer 5 me bahut important hoga.

23. Ab KnowledgeEdge samjho

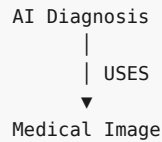
Node bolta hai:

"Main kya hoon?"

Edge bolta hai:

"Mera dusre knowledge se kya relationship hai?"

Example:



Yahan:

```

AI Diagnosis = KnowledgeNode
Medical Image = KnowledgeNode
USES = KnowledgeEdge relation
  
```

24. KnowledgeEdge ka exact structure

Report ke sample me edge ke paas hota hai:

```

edge_id
source_node
target_node
edge_class
relation_type
confidence
weight
evidence
metadata
  
```

filecite\turn4file0\L38-L59

Hamare example:

```

{
  "edge_id": "EDGE_000501",
  "source_node": "NODE_AI_DIAGNOSIS",
  "target_node": "NODE_MEDICAL_IMAGE",
  "edge_class": "SEMANTIC",
  "relation_type": "USES",
  "confidence": 0.93,
  "weight": 1.0
}
  
```

Meaning:

AI Diagnosis — USES → Medical Image

25. Do types ke Edges

Type 1: HIERARCHICAL

Rule:

Ye *acyclic forest* banana chahiye aur har node ka single parent hona chahiye.

Supported examples:

```
IS_A
PART_OF
SUBCLASS_OF
INSTANCE_OF
```

Type 2: SEMANTIC

Rule:

Ye unrestricted graph hai aur cycles allowed hain.

Examples:

```
USES
CAUSES
DEPENDS_ON
CONTRASTS_WITH
SIMILAR_TO
```

Ye distinction report me explicitly defined hai. `filecite\turn4file0\L38-L45`

26. Hamare example me HIERARCHICAL edges

```
AI Diagnosis
  IS_A
  ↓
Medical AI
```

```
CNN
  IS_A
  ↓
Deep Learning Model
```

```
Medical Image
  PART_OF
  ↓
Patient Diagnostic Data
```

Ye hierarchy banane me use honge.

27. Hamare example me SEMANTIC edges

```
AI Diagnosis —USES→ Medical Image
AI Diagnosis —PRODUCES→ Disease Prediction
Disease Prediction —DEPENDS_ON→ Medical Image
Authentication —DEPENDS_ON→ User Identity
RBAC —HAS_PROPERTY→ Role Permissions
```

Ye hierarchy nahi hain.

Inme complex graph connections allowed hain.

28. HIERARCHICAL edge me cycle allowed kyun nahi?

Suppose:

```
AI Diagnosis
  ↓ IS_A
Medical AI
  ↓ IS_A
Healthcare AI
  ↓ IS_A
AI Diagnosis ❌
```

Ab cycle ban gayi.

System endlessly traverse kar sakta hai.

Isliye validation:

Before adding hierarchical edge:

1. Check source exists
2. Check target exists
3. Check cycle banega?
4. Check single-parent rule break hoga?
5. Then add edge

Report me `NodeNotFoundError`, cycle check aur single-parent validation explicitly defined hain.

📄filecite🔄turn4file0📄L61-L88📄

29. Single-parent constraint samjho

Suppose:

```
Healthcare AI
  |
  |
AI Diagnosis
  |
Disease Prediction
```

Forest model me ek hierarchical node ka **single parent** expected hai.

Agar:

```
AI Diagnosis
  |
  ├── parent = Healthcare AI
  └── parent = Machine Learning ❌
```

to report ke current design me ye problem create karega.

Isko later report ki known limitation bhi mana gaya hai: real-world knowledge me multiple valid parents ho sakte hain, lekin current design single-parent forest constraint use karta hai; DAG support future work hai.

📄filecite🔄turn1file8📄L571-L577📄

30. Complete Relation Ontology

Report ke according supported relations hain:

Hierarchical

```
IS_A
PART_OF
SUBCLASS_OF
INSTANCE_OF
```

Semantic

```
USES
CAUSES
DEPENDS_ON
CONTRASTS_WITH
SIMILAR_TO
HAS_PROPERTY
PRODUCES
PREREQUISITE
```

filecite turn4file0 L90-L110

Hamare project ke liye iska meaning:

```
CNN IS_A Deep Learning Model

Medical Image PART_OF Patient Diagnostic Data

AI Diagnosis USES Medical Image

AI Diagnosis PRODUCES Disease Prediction

Disease Prediction DEPENDS_ON Medical Image

AI Diagnosis HAS_PROPERTY Diagnostic Confidence
```

31. Layer 0 me actual data flow

Hamare example ke through:

Step 1 — Initially concepts raw form me milenge

Layer 3 future me extract karegi:

```
AI diagnosis
Medical image
Disease prediction
Doctor verification
```

Layer 0 already define kar chuki hai:

```
"Concept ko KnowledgeNode banana hai."
```

So:

```
Raw Concept
↓
KnowledgeNode schema
↓
Validated Node Object
```

Step 2 — Relations milengi

Example:

```
AI diagnosis uses medical images.
```

Layer 0 bolti hai:

```
Relationship ko KnowledgeEdge schema me convert karo.
```

So:

```
Raw relation
↓
KnowledgeEdge
↓
Source + Target + Type + Class + Confidence + Evidence
```

Step 3 — Hierarchy validate hogi

```
AI Diagnosis
  IS_A
Medical AI
```

Check:

```
✓ Nodes exist?
✓ Relation valid?
✓ Edge class correct?
✓ Cycle create nahi?
✓ Single-parent rule follow?
```

Step 4 — Confidence propagate hogi

```
Root
↓
Parent confidence
↓
Child confidence
↓
Leaf
```

Kahn's topological order ensure karta hai ki parent pehle process ho. [filecite turn4file0 L112-L157](#)

32. Layer 0 ka final output exactly kya hai?

Layer 0 ka output **document nahi** hai.

Na hi answer.

Na hi Knowledge Forest directly.

Layer 0 ka output hai:

```
DATA CONTRACT
```

More specifically:

KnowledgeNode Schema

- identity
- semantic information
- evidence
- embeddings
- confidence
- abstraction level

KnowledgeEdge Schema

- source
- target
- relation type
- edge class
- confidence
- weight
- evidence

Validation Rules

- node existence
- no hierarchical cycles
- single-parent constraint
- ontology validation

Confidence Rules

- extraction confidence
- parent confidence
- evidence confidence
- topological propagation

33. Ye next layers ko kaise jata hai?

Layer 1

Layer 1 direct `KnowledgeNode` nahi banati.

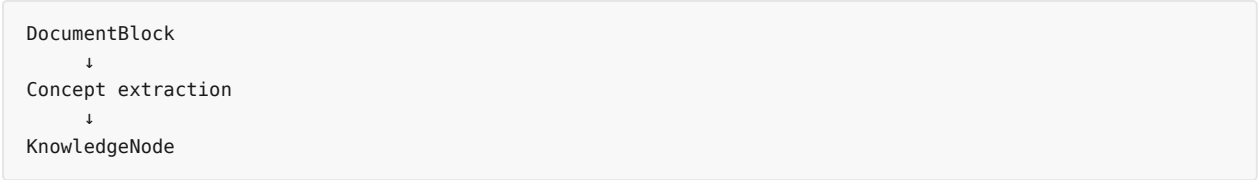
But Layer 1 ka `NormalizedDocument` later knowledge extraction ke input chain ka part hai.

Layer 2

`DocumentBlock` banata hai.

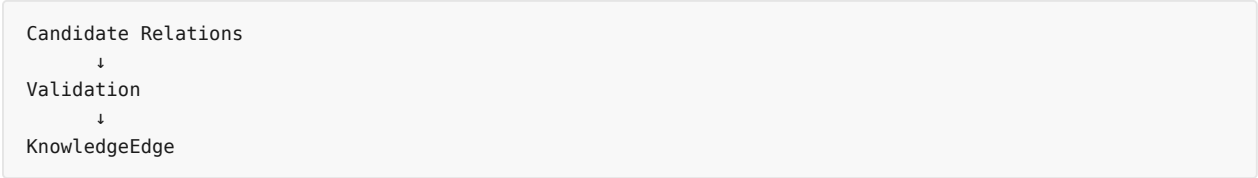
Layer 3

Layer 0 ke `KnowledgeNode` contract ko use karegi.

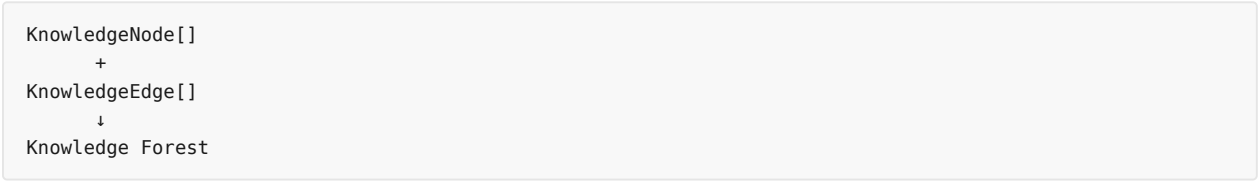


Layer 4

Layer 0 ke KnowledgeEdge contract ko use karegi.



Layer 5



Layer 6

Node embeddings, confidence aur graph relationships retrieval me use honge.

Layer 7

Evidence fields verification aur citation binding me useful honge.

Layer 8

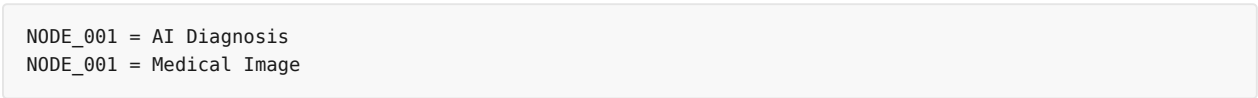
Edges ke weights aur access statistics feedback loop me update honge.

Isliye Layer 0 indirectly **har layer ko support karti hai**.

34. Layer 0 me possible errors

Ab important part.

Error 1: Duplicate Node IDs



Problem

Identity collision.

Safeguard

Unique ID generation + database uniqueness constraint.

Report ke shown Layer 0 section me exact database-level uniqueness mechanism specify nahi kiya gaya, so implementation phase me ye explicitly add karna padega.

Error 2: Node references nonexistent node

Edge:

```
AI Diagnosis —USES→ NODE_999999
```

But target node exists hi nahi.

Report ka handling

`validate_edge()` pehle source aur target dono check karta hai.

Missing hone par:

```
NodeNotFoundError
```

raise hota hai. `filecite[turn4file0][L61-L88]`

Error 3: Hierarchical cycle

```
A → B → C → A
```

Handling

`creates_cycle()` check.

Cycle banne par:

```
CycleError
```

Error 4: Two parents

```
AI Diagnosis
  ↑      ↑
Parent A  Parent B
```

Current forest model me invalid.

Handling

```
validate_single_parent(...)
```

Report ke validation logic me defined hai. `filecite[turn4file0][L79-L87]`

Error 5: Wrong relation type

Example:

```
AI Diagnosis —FLIES_T0→ Medical Image
```

Ye ontology me nahi hai.

Layer 0 ontology sirf supported relation types define karti hai. [filecite]turn4file0[L90-L110]

Unknown relation ko later Layer 4 validate/infer karegi.

35. Possible mistakes / missing parts in Layer 0

Ye meri technical analysis hai; main ise report ka explicitly stated content nahi keh raha.

Missing Part 1: Schema versioning

Suppose:

```
KnowledgeNode Schema v1
```

Baad me new field add:

```
temporal_validity
```

Old nodes ka kya hoga?

Report ke Layer 0 section me explicit:

```
schema_version  
migration strategy  
backward compatibility
```

detail nahi dikh rahi.

Production implementation me add karna useful hoga.

Missing Part 2: Temporal knowledge

Medical knowledge change ho sakta hai.

Example:

```
Old clinical guideline  
New clinical guideline
```

Current basic Node schema me clearly:

```
valid_from  
valid_to  
superseded_by
```

jaisi temporal semantics Layer 0 ke shown schema me specified nahi hain.

Ye future gap ho sakta hai.

Missing Part 3: Evidence confidence schema inconsistency risk

Confidence propagation pseudocode:

```
e.confidence
```

use karta hai. `filecite\turn4file0\L141-L149`

Lekin displayed sample evidence object me:

```
evidence_type
text
source_doc
source_block
page
```

dikhte hain. `filecite\turn1file6\L483-L490`

Isliye implementation se pehle **Evidence schema me explicit `confidence` field mandatory/optional ka rule clearly define karna chahiye.**

Warna runtime issue ho sakta hai:

```
AttributeError:
Evidence object has no confidence
```

Missing Part 4: Multiple parents real-world problem

Report khud accept karti hai ki real knowledge me multiple valid parents ho sakte hain, lekin current forest model single-parent constraint lagata hai; DAG support future work hai. `filecite\turn1file8\L571-L577`

Hamare example me:

```
AI Diagnosis
├─ Healthcare AI
└─ Machine Learning Application
```

Dono valid parents ho sakte hain.

Current model ko ek choose karna padega.

Dusra connection semantic cross-link ban sakta hai, lekin true hierarchical multiple-parent representation abhi limitation hai.

36. Exception handling summary

Problem	System action
Source node missing	`NodeNotFoundError`
Target node missing	`NodeNotFoundError`
Hierarchical cycle	`CycleError`
Multiple parent violation	`validate_single_parent()`
Semantic cycle	Allowed
Root node	Default/base confidence
No evidence	Default evidence confidence `0.5` in propagation
Arbitrary confidence processing	Kahn's topological sort
Parent/children inconsistency	Edge is source of truth

Confidence propagation aur root/default handling report ke pseudocode me defined hai. `filecite\turn4file0\L112-L157`

37. Hamare common example ka Layer 0 final state

Layer 0 ke baad actual hospital documents abhi process nahi hue hain.

But system ke paas contract ready hai.

```
KnowledgeNode Schema Ready
  ↓
"AI Diagnosis" future me kaise store hoga? ✓

KnowledgeEdge Schema Ready
  ↓
"AI Diagnosis USES Medical Image"
future me kaise store hoga? ✓

Relation Ontology Ready
  ↓
IS_A / PART_OF / USES / DEPENDS_ON etc. ✓

Validation Rules Ready
  ↓
Cycle? Missing node? Multiple parent? ✓

Confidence Rules Ready
  ↓
Extraction + Parent + Evidence ✓
```

Sabse important conclusion

Layer 0 knowledge create nahi karti. Layer 0 decide karti hai ki future me create hone wali knowledge ka structure, identity, relationship, validation aur trust representation kaisa hoga.

One-line memory trick

```
Layer 0 = "Knowledge ka database contract"
Layer 3 = "Knowledge kya hai?"
Layer 4 = "Knowledge ka relationship kya hai?"
Layer 5 = "Knowledge hierarchy me kahan jayegi?"
```

Abhi Layer 0 complete.

YOU 20/08/2026, 11:42:21

ab mujhe layer 1 smjhao same jese layer 0 smjhaya hai
kuch bhi miss nhi krna

CHATGPT 20/08/2026, 11:42:27

```
{"paths":["files"],"query":"read"}
```

DEEP RESEARCH 20/08/2026, 11:42:30

Citation Marker: fileciteturn5file0

[L1] metadata:

[L2] ref_id: turn1file0

[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42

[L4] version_id: 1

```

[L5] name: SHIA_RAG_Final_Report(4).md
[L6] lines_shown: 430-899
[L7] total_file_lines: 2066
[L8] has_more: True
[L9] next_start_line: 900
[L10] next_read: {"ref_id":"turn1file0","mode":"full","start_line":900,"max_lines":470,"include_images":false}
[L11] content:
[L12] """Output of Layer 1 — ready for Layer 2 parsing."""
[L13] doc_id: str
[L14] original_filename: str
[L15] format: DocumentFormat # PDF, DOCX, HTML, TXT
[L16] text: str # Full extracted text
[L17] pages: list # List of page texts (for PDFs)
[L18] metadata: dict # Author, title, date, etc.
[L19] is_ocr: bool = False # True if OCR was used
[L20] ocr_confidence: float = 1.0 # Average OCR confidence score
[L21] char_count: int = 0
[L22] page_count: int = 0
[L23] content_hash: str = "" # SHA-256 hash for deduplication
[L24] language: str = "en" # Detected language code
[L25] partial_extraction: bool = False # True if some pages failed
[L26] failed_pages: list = None # List of page numbers that failed
[L27]

```

```

[L28]
[L29] **Technology choices:**
[L30]
[L31] | Component | Technology | Purpose |
[L32] |-----|-----|-----|
[L33] | PDF extraction | PyMuPDF (fitz) | Primary text extraction |
[L34] | PDF fallback | pdfplumber | Complex layouts |
[L35] | OCR | Tesseract OCR | Scanned document pages |
[L36] | DOCX | python-docx | Word documents |
[L37] | HTML | BeautifulSoup4 | Web pages |
[L38] | Language detection | langdetect | Auto-detect document language |
[L39] | Deduplication | hashlib (SHA-256) | Prevent duplicate ingestion |
[L40]
[L41] #### Document Deduplication
[L42]
[L43] Before processing, compute a SHA-256 hash of the file content. If the hash already exists in the document registry, skip processing.
[L44]
[L45]

```

python

```

[L46] import hashlib
[L47]
[L48] def check_duplicate(file_path: str, doc_registry: dict) -> bool:
[L49] """Return True if document is a duplicate."""
[L50] with open(file_path, 'rb') as f:
[L51] content_hash = hashlib.sha256(f.read()).hexdigest()
[L52] if content_hash in doc_registry:
[L53] raise DuplicateDocumentError(

```

[L54] f"Document already ingested as {doc_registry[content_hash]}")

[L55] return content_hash

[L56]

```
[L57]
[L58] ##### OCR Detection (Multi-Signal)
[L59]
[L60] A single heuristic ('text_chars / page_area < 0.1') is unreliable. For example, a page with only a lar
[L61]
[L62]
```

python

[L63] def needs_ocr(page) -> bool:

[L64] """Multi-signal OCR detection — more robust than text/area ratio alone.

[L65]

[L66] Signal 1: Text density (text chars / page area)

[L67] Signal 2: Font metadata availability (scanned pages lack font info)

[L68] Signal 3: Image coverage ratio (scanned pages are mostly images)

[L69] """

[L70] text = page.get_text()

[L71] area = page.rect.width * page.rect.height

[L72]

[L73] # Signal 1: Text density

[L74] text_density = len(text.strip()) / max(area, 1)

[L75]

[L76] # Signal 2: Font metadata (scanned PDFs have no embedded fonts)

[L77] has_fonts = len(page.get_fonts()) > 0

[L78]

[L79] # Signal 3: Image coverage (scanned pages are large images)

[L80] images = page.get_images()

[L81] image_area = sum(img[2] * img[3] for img in images) if images else 0

[L82] image_coverage = image_area / max(area, 1)

[L83]

[L84] # Decision: OCR needed if text is sparse AND (no fonts OR high image coverage)

[L85] if text_density < 0.1 and (not has_fonts or image_coverage > 0.5):

[L86] return True

[L87] return False

[L88]

```
[L89]
[L90] ##### OCR Confidence Calculation
[L91]
[L92]
```

python

[L93] def calculate_ocr_confidence(ocr_result) -> float:

[L94] """Calculate OCR confidence from Tesseract per-word confidence scores.

[L95]

[L96] Tesseract returns per-word confidence (0-100). We:

[L97] 1. Get per-word confidences

```

[L98] 2. Filter out words with confidence < 10 (noise)
[L99] 3. Compute weighted average by word length (longer words = more reliable signal)
[L100] 4. Normalize to [0.0, 1.0]
[L101] """
[L102] word_data = ocr_result.get('words', [])
[L103] if not word_data:
[L104]     return 0.0
[L105]
[L106] total_weight, weighted_sum = 0, 0
[L107] for word in word_data:
[L108]     if word['confidence'] >= 10: # Filter noise
[L109]         weight = len(word['text']) # Weight by word length
[L110]         weighted_sum += word['confidence'] * weight
[L111]     total_weight += weight
[L112]
[L113] return (weighted_sum / max(total_weight, 1)) / 100.0
[L114]

```

```

[L115]
[L116] ##### Text Cleaning (Safe)
[L117]
[L118]

```

python

```

[L119] def clean_text(text: str, pages: list) -> str:
[L120]     """Clean extracted text while preserving legitimate content.
[L121]
[L122]     Safeguards against false header/footer removal:
[L123]     1. Identify candidate headers/footers by frequency across pages
[L124]     2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L125]     3. Never remove lines that appear on fewer than 3 pages
[L126]     4. Normalize whitespace but preserve paragraph boundaries
[L127]     """
[L128]     if len(pages) >= 3:
[L129]         # Frequency-based header/footer detection
[L130]         first_lines = [p.split('\n')[0].strip() for p in pages if p.strip()]
[L131]         last_lines = [p.split('\n')[-1].strip() for p in pages if p.strip()]
[L132]
[L133]         from collections import Counter
[L134]         header_counts = Counter(first_lines)
[L135]         footer_counts = Counter(last_lines)
[L136]
[L137]         threshold = max(3, int((len(pages) * 0.6))) # 60% of pages OR at least 3
[L138]         headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L139]         footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L140]
[L141]         lines = text.split('\n')
[L142]         lines = [l for l in lines

```



```

[L143] if l.strip() not in headers_to_remove
[L144] and l.strip() not in footers_to_remove]
[L145] text = '\n'.join(lines)
[L146]
[L147] # Normalize whitespace (preserve paragraph breaks)
[L148] import re
[L149] text = re.sub(r'[\t]+' , ' ', text) # Collapse horizontal space
[L150] text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L151] return text.strip()
[L152]

```

```

[L153]
[L154] ##### Language Detection
[L155]
[L156]

```

```

python
[L157] from langdetect import detect
[L158]
[L159] def detect_language(text: str) -> str:
[L160] """Detect document language for OCR configuration.
[L161]
[L162] Used to configure Tesseract's language parameter.
[L163] Falls back to 'eng' if detection fails."""
[L164] try:
[L165] lang = detect(text[:5000]) # Sample first 5000 chars
[L166] # Map langdetect codes to Tesseract codes
[L167] lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L168] return lang_map.get(lang, 'eng')
[L169] except Exception:
[L170] return 'eng'
[L171]

```

```

[L172]
[L173] ##### Partial Extraction Policy
[L174]
[L175] If some pages fail during extraction, the system uses a best-effort policy:
[L176]
[L177] | Condition | Action |
[L178] |-----|-----|
[L179] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |
[L180] | <80% pages succeed | Reject entire document, queue for manual review |
[L181] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L182]
[L183]

```

```

python
[L184] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L185] """Layer 1: Input Processing pipeline with all safeguards."""
[L186] # Step 0: Duplicate check
[L187] content_hash = check_duplicate(file_path, doc_registry)
[L188]

```

```

[L189] fmt = detect_format(file_path)
[L190] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L191]
[L192] failed_pages = []
[L193] if fmt == 'pdf':
[L194] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L195] # Partial extraction policy
[L196] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L197] if success_rate < 0.8:
[L198] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L199] elif fmt == 'docx':
[L200] text, pages = extract_docx(file_path), []
[L201] is_ocr, ocr_conf = False, 1.0
[L202] elif fmt == 'html':
[L203] text, pages = extract_html(file_path), []
[L204] is_ocr, ocr_conf = False, 1.0
[L205] else: # txt
[L206] text = open(file_path, 'r', encoding='utf-8').read()
[L207] pages, is_ocr, ocr_conf = [], False, 1.0
[L208]
[L209] language = detect_language(text)
[L210] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L211] metadata = extract_metadata(file_path, fmt)
[L212]
[L213] doc = NormalizedDocument(
[L214] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L215] format=DocumentFormat(fmt), text=text, pages=pages,
[L216] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L217] char_count=len(text), page_count=len(pages) or 1,
[L218] content_hash=content_hash, language=language,
[L219] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L220] )
[L221] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L222] return doc
[L223]

```

```

[L224]
[L225] ---
[L226]
[L227] ### 5.2 Layer 2: Document Parsing
[L228]
[L229] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L230]
[L231] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L232]
[L233]

```

python

```
[L234] class BlockType(Enum):
```

```

[L235] HEADING = "heading"
[L236] PARAGRAPH = "paragraph"
[L237] LIST = "list"
[L238] TABLE = "table"
[L239] CODE = "code"
[L240] ALGORITHM = "algorithm"
[L241] FIGURE_CAPTION = "figure_caption"
[L242] EQUATION = "equation"
[L243] DEFINITION = "definition"
[L244]
[L245] @dataclass
[L246] class DocumentBlock:
[L247] """Atomic unit of document structure."""
[L248] block_id: str
[L249] doc_id: str
[L250] page_number: int
[L251] region_index: int # Which region on the page
[L252] block_type: BlockType
[L253] text: str
[L254] reading_order: int # Global ordering across the document
[L255] confidence: float = 1.0 # Classification confidence
[L256] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L257]

```

```

[L258]
[L259] The parser uses rule-based heuristics for block classification:
[L260] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L261] - List: Starts with ` - `, `* `, `• `, or `1.`
[L262] - Table: Contains 3+ pipe characters (`|`) - uses regex `re.match(r'^(def |class |import |from )'
[L263] - Code: Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")
[L264] - Definition: Contains phrases like "is defined as", "refers to"
[L265] - Default: Paragraph
[L266]
[L267] ##### Table Cell-Level Parsing
[L268]
[L269] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-
[L270]
[L271]

```

python

```

[L272] def parse_table_cells(page, table_region) -> list:
[L273] """Extract table structure with cell-level detail.
[L274]
[L275] Uses pdfplumber for text-based tables, Camelot for image-based.
[L276] Returns list of rows, each row is a list of cell strings."""
[L277] try:
[L278] # Primary: pdfplumber table extraction
[L279] table = page.extract_table(table_region)
[L280] if table and len(table) > 1: # At least header + 1 data row
[L281] return table
[L282] except Exception:

```

```

[L283] pass
[L284]
[L285] # Fallback: Camelot for image-based tables
[L286] try:
[L287] import camelot
[L288] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L289] if tables:
[L290] return tables[0].df.values.tolist()
[L291] except Exception:
[L292] pass
[L293]
[L294] return None # Table structure could not be parsed
[L295]

```

```

[L296]
[L297] ##### Table Detection Fallback
[L298]
[L299] If table detection confidence is low, the block is flagged:
[L300]
[L301]

```

```

python
[L302] def classify_block_with_fallback(text, page) -> tuple:
[L303] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L304] block_type, confidence = classify_block(text)
[L305]
[L306] if block_type == BlockType.TABLE and confidence < 0.6:
[L307] # Attempt cell-level parsing to confirm
[L308] cells = parse_table_cells(page, text)
[L309] if cells is None:
[L310] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L311] else:
[L312] return BlockType.TABLE, 0.8 # Confirmed by parser
[L313]
[L314] return block_type, confidence
[L315]

```

```

[L316]
[L317] ##### Reading Order Validation
[L318]
[L319] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks re
[L320]
[L321]

```

```

python
[L322] def assign_reading_order(blocks: list) -> list:
[L323] """Assign global reading order with confidence.
[L324]
[L325] Multi-column layouts can produce incorrect reading order.
[L326] We detect potential issues and flag low confidence."""

```

```

[L327] # Sort by page, then y-coordinate, then x-coordinate
[L328] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L329]
[L330] for i, block in enumerate(sorted_blocks):
[L331]     block.reading_order = i
[L332]
[L333] # Detect potential multi-column issues
[L334] if i > 0:
[L335]     prev = sorted_blocks[i - 1]
[L336]     if (prev.page_number == block.page_number and
[L337]         abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L338]         prev.x_pos > block.x_pos): # But previous is to the right
[L339]         block.confidence = 0.7 # Reduce confidence — possible column issue
[L340]
[L341] return sorted_blocks
[L342]

```

```

[L343]
[L344] ---
[L345]
[L346] ### 5.3 Layer 3: Knowledge Extraction
[L347]
[L348] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then
[L349]
[L350] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L351]
[L352] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, c
[L353]
[L354] ##### Extraction Prompt
[L355]
[L356]

```

```

[L357] You are a knowledge extraction system. Given a text block, extract:
[L358] 1. Concepts: Named entities, technical terms
[L359] 2. Definitions: If the text defines any concept, extract the definition
[L360] 3. Properties: Key attributes with types and values
[L361] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L362] 5. Domain: Subject area (e.g., "Operating Systems")
[L363]
[L364] Respond in JSON format:
[L365] {
[L366]   "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."]},
[L367]   "properties": [{"key": {"type": "...", "value": "..."}},
[L368]   "relations": [{"source": "...", "type": "IS_A/PART_OF/...", "target": "...",
[L369]   "evidence": "..."}],
[L370]   "domain": "..."
[L371] }
[L372] ``
[L373]
[L374] ##### Definition Extraction (Dependency-Parse Based)

```

[L375]

[L376] Simple regex patterns like `^(.) is (a|an|the)? (.*)`` are too broad — they match non-definitions like "The sky is blue today." Instead, we use **dependency parsing**

CHATGPT 20/08/2026, 11:42:30

Layer 1 — Input Processing

Ab hum **sirf Layer 1** ko complete depth me samjhenge — same approach se jo Layer 0 me kiya tha.

Ham **wahi same Smart Hospital example** use karenge:

AI-assisted emergency diagnosis system + hospital data security

Aur yaad rakho: Layer 0 ne sirf **data contracts/schemas** define kiye the. Ab **Layer 1 me actual raw documents system me enter hote hain**.

Report ke according Layer 1 ki responsibility hai:

Raw documents ko accept karna, format validate karna, duplicates rokna, text extract karna, OCR fallback use karna, metadata preserve karna, aur ek `NormalizedDocument` banana. `filecite` turn5file0 L29-L39

1. Layer 1 ko simple language me samjho

Agar poora SHIA-RAG ek factory hai:

Layer 0 = Factory ke rules aur container design
Layer 1 = Raw material receiving + cleaning section

Different documents aa sakte hain:

PDF
DOCX
HTML
TXT
Scanned PDF
Complex-layout PDF
Partial/corrupted PDF
Duplicate file

Layer 1 ka kaam **knowledge extract karna nahi** hai.

Ye ye decide nahi karti:

"AI Diagnosis ek important concept hai."

Wo Layer 3 karegi.

Layer 1 sirf ensure karti hai:

"Jo raw document system ke andar aaya hai, use safely aur consistently extract karke next layer ke usable format me convert karo."

2. Layer 1 ka exact Input kya hai?

Input

Raw Document File

Supported formats:

PDF
DOCX
HTML
TXT

Report ki Layer 1 technology table me in formats ke extraction tools define kiye gaye hain. `filecite\turn5file0\L29-L39`

3. Hamare common example me Layer 1 ko kya milega?

Maan lo user ye 5 documents upload karta hai:

DOC A:
Smart_Hospital_Architecture.pdf

DOC B:
Emergency_Workflow.docx

DOC C:
Hospital_Security.html

DOC D:
AI_Diagnostic_Module.pdf

DOC E:
Hospital_Knowledge_Notes.txt

Aur maan lo AI_Diagnostic_Module.pdf scanned document hai.

To Layer 1 ka initial input hoga:

Raw Files

- |
- ├ Smart_Hospital_Architecture.pdf
- ├ Emergency_Workflow.docx
- ├ Hospital_Security.html
- ├ AI_Diagnostic_Module.pdf
- └ Hospital_Knowledge_Notes.txt

Har file **independently process** hogi.

4. Layer 1 ka complete pipeline

Report ke `layer1_process()` pseudocode ko simple flow me dekho:

```
RAW FILE
↓
Step 0: Duplicate Check
↓
Format Detection
↓
File Validation
↓
Format-specific Extraction
↓
PDF ho to OCR Detection
↓
OCR if needed
↓
Partial Extraction Check
↓
Language Detection
↓
Safe Text Cleaning
↓
Metadata Extraction
↓
NormalizedDocument
↓
Layer 2
```

Report ka complete orchestration pseudocode duplicate check se lekar `NormalizedDocument` return karne tak isi order ko define karta hai. `filecite\turn5file0\L173-L223`

Ab hum **har step ko depth me** samjhenge.

5. Step 0 — Duplicate Document Check

Sabse pehle system check karta hai:

Kya ye exact same file pehle ingest ho chuki hai?

Iske liye report **SHA-256 hash** use karti hai. `filecite\turn5file0\L41-L56`

SHA-256 hash kya karta hai?

Suppose file:

```
AI_Diagnostic_Module.pdf
```

Uski complete binary content:

```
101011010101010...
```

SHA-256 se process hone ke baad:

```
content_hash =
A8F34C...XYZ
```

System registry me:


```
Document Registry
```

```
A8F34C...XYZ
```

```
↓
```

```
DOC_000004
```

Ab user same file dobara upload karta hai.

System:

```
New File
```

```
↓
```

```
SHA-256
```

```
↓
```

```
A8F34C...XYZ
```

```
↓
```

```
Registry me already present? YES
```

```
↓
```

```
DuplicateDocumentError
```

Report me duplicate document milne par upload reject karne ka logic diya gaya hai. `filecite\turn5file0\L41-L56`

Hamare example me

Pehli baar:

```
AI_Diagnostic_Module.pdf
```

```
Hash → H1
```

```
Registry → empty
```

```
Result:
```

```
Accept
```

Doosri baar same exact file:

```
AI_Diagnostic_Module.pdf
```

```
Hash → H1
```

```
Registry → H1 already exists
```

```
Result:
```

```
REJECT
```

Why ye step necessary hai?

Agar duplicate check nahi hoga:

```
Same document
```

```
↓
```

```
Knowledge extraction twice
```

```
↓
```

```
Same concepts twice
```

```
↓
```

```
Same evidence twice
```

```
↓
```

```
Artificial confidence increase
```

```
↓
```

```
Duplicate graph structure
```

Example:

```
AI Diagnosis
AI Diagnosis
AI Diagnosis
```

System galat soch sakta hai ki 3 independent sources ne concept confirm kiya hai.

Actually:

```
1 document
uploaded 3 times
```

Possible Error: Same content, different file format

Example:

```
AI_Diagnostic_Module.pdf
AI_Diagnostic_Module_Copy.pdf
```

Agar binary content exactly same hai:

```
Hash same
→ duplicate detect
```

Lekin agar same text ko re-save karke PDF internally change kar diya gaya:

```
Content semantically same
Binary hash different
```

To SHA-256 exact duplicate detection miss kar sakta hai.

Important

Report yahan **SHA-256 based exact file deduplication** define karti hai. Semantic duplicate detection Layer 1 specification me described nahi hai. `filecite\turn5file0\L41-L56`

6. Step 1 — Format Detection

Duplicate check pass hone ke baad:

```
detect_format(file_path)
```

System identify karega:

```
PDF?
DOCX?
HTML?
TXT?
```

Hamare example:

```
Smart_Hospital_Architecture.pdf
→ PDF

Emergency_Workflow.docx
→ DOCX

Hospital_Security.html
→ HTML

Hospital_Knowledge_Notes.txt
→ TXT
```

Phir har format ka alag extraction path hoga.

7. Step 2 — File Validation

Report ke orchestration pseudocode me:

```
validate_file(file_path, fmt)
```

call hota hai.

Comment me constraints diye gaye hain:

```
Format check
Size check: maximum 100 MB
Page limit: maximum 5000 pages
```

filecite\turn5file0\L183-L191

Why?

Suppose user upload kare:

```
20 GB file ✗
100,000 page PDF ✗
Unsupported executable ✗
```

Agar validation nahi hogi:

```
Resource exhaustion
Memory issue
Very long processing
OCR overload
```

8. Step 3 — Format-specific Extraction

Ab actual text extraction start hota hai.

PDF

Primary:

```
PyMuPDF (fitz)
```

Fallback:

```
pdfplumber
```

Scanned PDF

```
Tesseract OCR
```

DOCX

```
python-docx
```

HTML

```
BeautifulSoup4
```

TXT

Direct UTF-8 reading.

Ye exact technology choices report me specified hain. `filecite\turn5file0\L29-L39`

9. PDF Extraction ko detail me samjho

Hamari file:

```
Smart_Hospital_Architecture.pdf
```

Suppose PDF digitally generated hai:

```
Page 1 → text layer available  
Page 2 → text layer available  
Page 3 → text layer available
```

Then:

```
PyMuPDF  
↓  
Direct text extraction
```

Output:

```
Full text  
+  
Page-wise text
```

PDF fallback kyun?

Kuch PDFs ka layout complex ho sakta hai:

```
Two columns  
Tables  
Text boxes  
Mixed layout
```

Isliye report:

```
Primary → PyMuPDF
Fallback → pdfplumber
```

define karti hai. `filecite\turn5file0\L31-L35`

10. Scanned PDF ka problem

Hamari file:

```
AI_Diagnostic_Module.pdf
```

maan lo scanner se scan hui hai.

Internally:

```
Page = Image
```

Normal text extraction try karo:

```
page.get_text()
↓
Almost empty text
```

Lekin iska matlab automatically OCR nahi.

Kyun?

Ek normal PDF page me bhi sirf:

```
CHAPTER 5
```

likha ho sakta hai.

Text kam hai, lekin page scanned nahi hai.

Report specifically warn karti hai ki single heuristic unreliable hai. `filecite\turn5file0\L58-L69`

11. OCR Detection — Multi-Signal Approach

Report 3 signals use karti hai:

Signal 1: Text Density

```
text density
=
number of extracted characters / page area
```

Low text density suspicious ho sakti hai.

Signal 2: Font Metadata

Digital PDF:

```
Embedded fonts usually exist
```

Scanned PDF:

```
Often no embedded text fonts
```

System:

```
has_fonts = len(page.get_fonts()) > 0
```

Signal 3: Image Coverage

Scanned page mostly:

```
Large image
```

System calculate karta hai:

```
image coverage
=
image area / page area
```

12. Final OCR Decision

Report ka rule:

```
OCR needed IF:

text is sparse

AND

(
  fonts absent
  OR
  image coverage > 50%
)
```

filecite\turn5file0\L58-L88

Hamare scanned PDF example me

```
AI_Diagnostic_Module.pdf

Text density = 0.03      → sparse
Has fonts = False       → no fonts
Image coverage = 0.95    → mostly image
```

Result:

```
needs_ocr = True
```

Then:

```
Page Image
↓
Tesseract OCR
↓
Extracted Text
```

13. Why Multi-Signal approach important hai?

Suppose page:

RETURN POLICY

Bas ek heading hai.

Text density = low

Agar system sirf text density dekhe:

Low density
→ OCR needed ❌

But page scanned nahi hai.

Multi-signal:

Low density
BUT fonts exist
AND image coverage low

→ OCR not needed ✓

Yahi report ka major safeguard hai. `filecite\turn5file0\L58-L88`

14. OCR ke baad confidence calculation

OCR perfect nahi hota.

Example original text:

AI Diagnosis

OCR output:

A1 Diagn0sis

Isliye report har OCR output ka confidence calculate karti hai.

Tesseract per-word confidence

Maan lo:

AI → confidence 98
diagnosis → confidence 95
system → confidence 92

Noise:

```
??? → confidence 5
```

Report ka process:

1. Har word ki confidence lo
2. Confidence < 10 remove karo
3. Remaining words ko word length se weight karo
4. Weighted average lo
5. 0-100 se 0-1 scale par normalize karo

filecite\turn5file0\L90-L114

15. Word length weighting kyun?

Example:

```
AI          confidence = 90, length = 2
Diagnosis confidence = 90, length = 9
```

Longer word ko zyada weight dena extraction quality ka relatively stronger signal deta hai.

Final:

```
ocr_confidence ∈ [0.0, 1.0]
```

16. OCR confidence low ho to kya hota hai?

Report ke partial extraction policy ke according:

```
OCR confidence < 0.6
↓
Page flagged for manual review
↓
But document me still included
```

filecite\turn5file0\L173-L181

Example

Page 27:

```
ocr_confidence = 0.48
```

System:

```
Document reject? → No, automatically nahi
Page include? → Yes
Manual review flag? → Yes
```

17. Step 4 — Partial Extraction Handling

Ye Layer 1 ka important exception-handling mechanism hai.

Maan lo:

```
100-page PDF
```

Extraction:

```
90 pages successful
10 pages failed
```

Success rate:

```
90 / 100 = 90%
```

Report policy:

```
≥ 80% pages successful
→ Accept document
→ partial_extraction = True
→ failed_pages record karo
```

But:

```
70 / 100 pages successful
= 70%
```

Then:

```
< 80%
→ Reject entire document
→ Manual review queue
```

filecite\turn5file0\L173-L181

18. Hamare example me partial extraction

```
Smart_Hospital_Architecture.pdf
```

```
Total pages = 120
```

```
Successful = 110
```

```
Failed = 10
```

```
Success rate = 91.67%
```

Output:

```
partial_extraction = True

failed_pages = [
    18, 19, 45, ...
]
```

Document continue kar sakta hai.

Lekin:

```

Successful = 70
Failed = 50

Success rate = 58.33%

```

Then:

```
ExtractionError
```

Report ke pseudocode me below-80% condition explicitly `ExtractionError` raise karti hai. `filecite\turn5file0\L183-L199`

19. Step 5 — Language Detection

Text extract hone ke baad:

```
detect_language(text)
```

Report:

```

First 5000 characters sample
↓
langdetect
↓
Language code
↓
Tesseract language mapping

```

Example mapping:

```

en → eng
hi → hin
de → deu
fr → fra
es → spa

```

Agar detection fail:

```
fallback → eng
```

`filecite\turn5file0\L154-L171`

Hamare example me

```
Emergency_Workflow.docx
```

First 5000 chars:

```
"The patient arrives at the emergency department..."
```

Detection:

```
en
```

OCR configuration code:

eng

20. Language detection OCR se pehle ya baad?

Report ke shown orchestration pseudocode me language detection **format-specific extraction ke baad** call hoti hai:

```

text_extract
  ↓
detect_language(text)
  ↓
clean_text

```

filecite\turn5file0\L193-L211

Isliye report ke exact implementation flow ke according hum isi order ko follow karenge.

21. Step 6 — Safe Text Cleaning

Raw extraction me unwanted cheezein aa sakti hain:

```

Page headers
Page footers
Extra spaces
Tabs
Many blank lines

```

Example:

```

NIT Hospital – Confidential
-----

AI Diagnosis ...

...
Page 12

```

Agar system blindly repeated text remove kare, useful content delete ho sakta hai.

Isliye report **safe frequency-based removal** use karti hai. filecite\turn5file0\L116-L151

22. Header/Footer removal ka algorithm

System pages ke:

```

First line
Last line

```

collect karta hai.

Example:

Page 1 first line:
Smart Hospital Architecture

Page 2 first line:
Smart Hospital Architecture

Page 3 first line:
Smart Hospital Architecture

Frequency count:

Smart Hospital Architecture = 80% pages

Candidate header.

Removal threshold

Report:

Remove only if line appears on:

60%+ pages

AND

Never remove if it appears on fewer than 3 pages

Threshold:

max(3, 60% of pages)

filecite turn5file0 L116-L151

23. Why 60% threshold?

Suppose medical document me ek important sentence sirf 2 pages me repeat hua:

Critical patient requires immediate evaluation.

Agar repeated line blindly remove hoti:

Important knowledge lost ✖

But report ka safeguard:

< 3 pages

→ Never treat as removable repeated header/footer

24. Whitespace normalization

Cleaning ke baad:

Multiple spaces
↓
Single space

5 consecutive newlines
↓
Maximum 2 consecutive newlines

But:

Paragraph boundaries preserve ki jati hain.

Report explicitly horizontal whitespace collapse karti hai, lekin paragraph separation preserve karne ke liye newline normalization limit karti hai. [filecite\turn5file0\L147-L151](#)

25. Important: Layer 1 text ka meaning change nahi karti

Ye bahut important hai.

Suppose original:

AI diagnosis does not replace doctor verification.

Layer 1 ka kaam nahi:

AI diagnosis replaces doctor verification ❌

Layer 1 sirf:

Extraction
Cleaning
Normalization

karti hai.

Semantic interpretation Layer 3 me aayegi.

26. Step 7 — Metadata Extraction

Report orchestration me:

```
metadata = extract_metadata(file_path, fmt)
```

call hota hai. [filecite\turn5file0\L209-L221](#)

Metadata examples:

Author
Title
Date
etc.

`NormalizedDocument` me metadata dictionary store hoti hai. [filecite\turn5file0\L12-L26](#)

Hamare example:

```
metadata = {  
    title: "AI Diagnostic Module",  
    author: "Hospital Research Team",  
    creation_date: "...",  
    ...  
}
```

27. Final Output — NormalizedDocument

Ab sabse important:

Layer 1 ka final output exactly kya hai?

NormalizedDocument

Report ke schema me fields hain:

```
doc_id  
original_filename  
format  
text  
pages  
metadata  
is_ocr  
ocr_confidence  
char_count  
page_count  
content_hash  
language  
partial_extraction  
failed_pages
```

filecite\turn5file0\L12-L26

28. Har field ko hamare example se samjho

Suppose:

AI_Diagnostic_Module.pdf

process ho chuka hai.

Output approximately:

```
NormalizedDocument
├── doc_id
│   └── DOC_000004
├── original_filename
│   └── AI_Diagnostic_Module.pdf
├── format
│   └── PDF
├── text
│   └── Complete cleaned extracted text
├── pages
│   └── [
│       ├── page 1 text,
│       ├── page 2 text,
│       └── ...
│   ]
├── metadata
│   └── {
│       ├── title,
│       ├── author,
│       └── date
│   }
├── is_ocr
│   └── True
├── ocr_confidence
│   └── 0.87
├── char_count
│   └── 245,381
├── page_count
│   └── 120
├── content_hash
│   └── SHA-256 hash
├── language
│   └── eng
├── partial_extraction
│   └── True/False
└── failed_pages
    └── [...]
```

29. Hamare 5 documents ka Layer 1 output

Initially:

5 Raw Documents

Layer 1 ke baad:

```
NormalizedDocument[]
|
|--- DOC_000001
|      Smart Hospital Architecture
|
|--- DOC_000002
|      Emergency Workflow
|
|--- DOC_000003
|      Hospital Security
|
|--- DOC_000004
|      AI Diagnostic Module
|
|--- DOC_000005
|      Hospital Knowledge Notes
```

Concept extraction **abhi nahi hui**.

Ye yaad rakhna bahut important hai.

At this point system ko abhi ye formally nahi pata:

```
AI Diagnosis = Concept?
CNN = Concept?
RBAC = Concept?
```

Abhi sirf clean, normalized document representation hai.

30. Layer 1 internally kya-kya algorithms/techniques use karti hai?

Summary

```
| Step | Algorithm / Technique |
|---|---|
| Duplicate check | SHA-256 |
| Format handling | Format detection |
| PDF extraction | PyMuPDF |
| Complex PDF fallback | pdfplumber |
| OCR | Tesseract |
| OCR decision | Multi-signal heuristic |
| OCR confidence | Weighted per-word confidence |
| Language detection | langdetect |
| Header/footer removal | Frequency-based detection |
| Whitespace cleanup | Regex normalization |
| Partial extraction | 80% threshold policy |
```

Ye techniques Layer 1 ke report-defined pipeline me described hain. `filecite\turn5file0\L29-L223`

31. Layer 1 me possible errors aur handling

Ab hum exact error cases dekhenge.

Error 1 — Duplicate document

Input

Same exact file again

Detection

SHA-256 hash already exists

Handling

DuplicateDocumentError

Upload reject.

filecite\turn5file0\L41-L56

Error 2 — Invalid format

Example:

malware.exe

Expected:

Format validation failure

Layer 1 orchestration `validate_file()` call karti hai. Report ke shown comment me format validation included hai.

filecite\turn5file0\L183-L191

Lekin exact exception class aur unsupported-format response shown section me detail se define nahi hai.

Error 3 — Oversized document

150 MB

Maximum shown limit:

100 MB

Result:

Validation failure

filecite\turn5file0\L189-L191

Error 4 — Too many pages

7000 pages

Shown limit:

5000 pages

Result:

Validation failure

Error 5 — OCR wrongly triggered

Problem

Page has little text but is not scanned.

Safeguard

Single text-density heuristic use nahi hoti.

Use:

Text density
+
Font metadata
+
Image coverage

filecite\turn5file0\L58-L88

Error 6 — OCR poor quality

OCR confidence < 0.6

Handling

Page flag for manual review
+
Still include page

filecite\turn5file0\L173-L181

Error 7 — Some pages fail

≥80% successful

Handling

Accept
partial_extraction = True
failed_pages = [...]

Error 8 — Too many pages fail

<80% successful

Handling

Reject entire document
ExtractionError
Manual review queue

filecite\turn5file0\L173-L181\ filecite\turn5file0\L193-L199\

Error 9 — Language detection fails

Handling

Fallback → eng

filecite\turn5file0\L154-L171\

Error 10 — Header mistakenly removed

Safeguard

Remove only when:

Appears $\geq 60\%$ of pages
AND
at least 3 occurrences

filecite\turn5file0\L116-L151\

32. Layer 1 me mistakes kaise ho sakti hain?

Ab ye **technical analysis** hai. Main report ke content se alag clearly bata raha hoon.

Mistake A — Hash exact duplicate hi detect karega

Same knowledge
Different PDF binary

SHA-256 different hoga.

So:

Semantic duplicate may enter

Report ki Layer 1 specification exact-content hash deduplication define karti hai; semantic deduplication yahan specified nahi hai. filecite\turn5file0\L41-L56\

Mistake B — OCR detection heuristic perfect nahi

Even 3 signals ke baad unusual documents me:

False positive
False negative

possible hain.

Example:

```
Digital PDF
+
large full-page diagram
+
very little text
```

OCR incorrectly trigger ho sakta hai.

Mistake C — First 5000 characters language sample representative na ho

Suppose document:

```
First 5000 chars → English
Rest → Hindi
```

Current shown logic:

```
First 5000 chars detect
```

Language result entire document ke liye representative nahi bhi ho sakta. Report ka code first 5000 characters sample karta hai. `filecite\turn5file0\L154-L170`

Mistake D — Partial extraction knowledge loss

Suppose:

```
120-page document
119 pages extract
Page 120 fails
```

Success rate:

```
99.1%
→ accept
```

But Page 120 me:

```
Most important security policy
```

ho sakti hai.

System technical success rate dekhta hai, **semantic importance of failed page** shown policy me evaluate nahi hoti.

33. Layer 1 ke missing/unclear parts

Ye bhi meri technical review hai.

Missing/unclear Part 1 — Exact `detect_format()` logic

Report orchestration me function call hai:

```
detect_format(file_path)
```

Lekin shown Layer 1 section me exact implementation detail—extension-based, MIME-based, ya magic-byte-based—specified nahi hai. [filecite\turn5file0\L183-L190](#)

Missing/unclear Part 2 — `validate_file()` detailed rules

Report comment me:

```
format check
100 MB max
5000 pages max
```

given hai, but detailed validation logic shown nahi hai. [filecite\turn5file0\L189-L191](#)

For production, explicit rules useful honge:

```
MIME verification
Corrupt-file handling
Password-protected PDF
Encrypted PDF
ZIP bomb / decompression concerns
```

Ye shown Layer 1 specification me detail se defined nahi hain.

Missing/unclear Part 3 — OCR engine failure recovery

OCR confidence handling defined hai.

Lekin agar:

```
Tesseract process itself crashes
```

to retry policy / alternate OCR engine / failure queue ka detailed behavior shown section me specified nahi hai.

Missing/unclear Part 4 — Mixed-language documents

Fallback:

```
eng
```

defined hai if detection fails. [filecite\turn5file0\L154-L170](#)

Lekin Hindi + English + code mixed document ko per-page/per-block language detection se kaise handle karna hai, ye shown Layer 1 specification me explicit nahi hai.

34. Layer 1 ka output Layer 2 ko exactly kaise jata hai?

Layer 1:

```
Raw File
↓
NormalizedDocument
```

Layer 2:

```
NormalizedDocument
↓
Page → Region → Block
```

Report Layer 2 ka input explicitly `NormalizedDocument` aur output `DocumentBlock[]` define karti hai.

filecite\turn5file0\L227-L257

Hamare example me:

```
NormalizedDocument
|
├─ Full cleaned text
├─ Page-wise text
├─ OCR information
├─ Language
├─ Metadata
├─ Failed page information
└─ Document identity
    ↓
    Layer 2
```

35. Complete Layer 1 — Hamare common example ka end-to-end walkthrough

Chalo ek file ko complete trace karte hain.

Input

```
AI_Diagnostic_Module.pdf
120 pages
Scanned PDF
```

Step 1: SHA-256

```
Hash = H_AI_001
Registry me nahi
→ Accept
```

Step 2: Format

```
.pdf
→ PDF pipeline
```

Step 3: Validation

```
Size = 25 MB ✓
Pages = 120 ✓
```

Step 4: Primary extraction

```
PyMuPDF
→ Very sparse text
```

Step 5: OCR Detection

Text density = low
Fonts = absent
Image coverage = 93%

→ OCR required

Step 6: Tesseract OCR

Page-wise text extracted

Step 7: OCR confidence

Page 1 = 0.94
Page 2 = 0.91
Page 3 = 0.52 ▲
...

Page 3:

Manual review flag
But included

Step 8: Extraction failures

Failed pages = [87, 88]

Success rate = 118 / 120
 = 98.3%

→ Accept
→ partial_extraction = True

Step 9: Language

English
→ eng

Step 10: Safe cleaning

Repeated header removed
Repeated footer removed
Extra spaces normalized
Paragraph boundaries preserved

Step 11: Metadata

Title extracted
Author extracted
Date extracted

Final Output

```
NormalizedDocument(
  doc_id="DOC_000004",
  format="PDF",
  is_ocr=True,
  ocr_confidence=0.89,
  page_count=120,
  partial_extraction=True,
  failed_pages=[87, 88]
)
```

36. Layer 1 ka sabse important difference Layer 0 se

```
Layer 0:
"Knowledge ko store kaise karna hai?"
  ↓
Schema / Contract

Layer 1:
"Raw document ko usable standard form me kaise lana hai?"
  ↓
NormalizedDocument
```

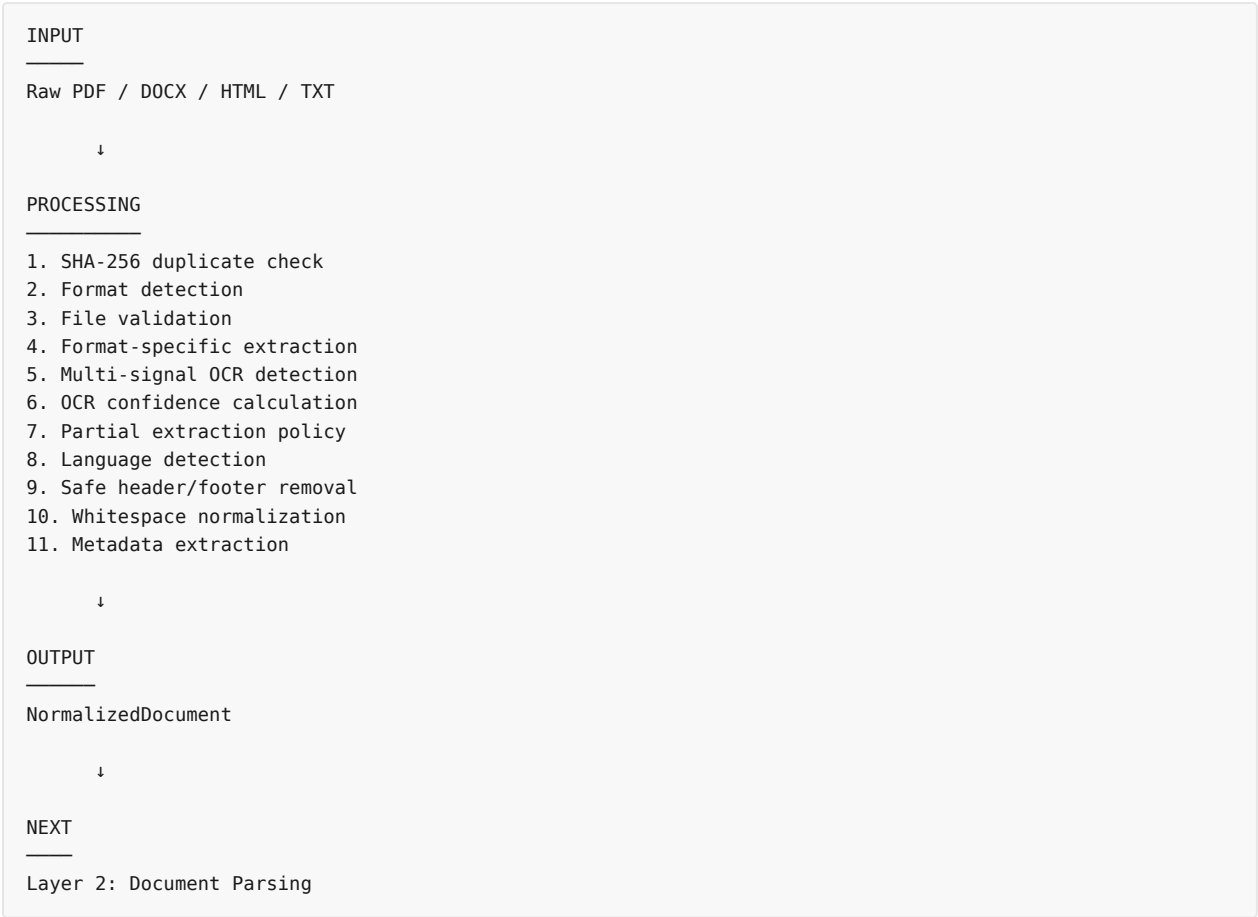
37. One-line memory trick

Layer 1 = Ingestion + Validation + Extraction + OCR + Cleaning + Normalization

Ya visual form me:

```
MESSY DOCUMENT
  ↓
Check
  ↓
Extract
  ↓
OCR if needed
  ↓
Verify extraction
  ↓
Detect language
  ↓
Clean safely
  ↓
NORMALIZED DOCUMENT
```

38. Layer 1 ka final Input → Output summary



Layer 1 complete.

YOU 20/08/2026, 14:17:11

ab mujhe layer 2 smjhao
same layer 0 aur layer 1 ke jese

CHATGPT 20/08/2026, 14:17:17

{ "paths": ["files"], "query": "search" }

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: fileciteturn6file0

- [L1] metadata:
- [L2] query_result_index: 0
- [L3] file_id: file_000000003218820b918ee3d86ec345ae
- [L4] version_id: 1
- [L5] name: SHIA_RAG_Final_Report(1).md
- [L6] mime_type: text/markdown
- [L7] surface: library
- [L8] score: 0.01821162444113264
- [L9] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--6
- [L10] snippet:
- [L11]

```
[L12] graph TD
[L13]   subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14]     L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Par
[L15]     L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L16]     L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L17]     L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L18]     L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L19]   end
[L20]
[L21]   subgraph "Phase 2: Online – Retrieval & Generation"
[L22]     Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L23]     KF --> L6
[L24]     L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L25]     L7 --> A["Answer + Citations"]
[L26]   end
[L27]
[L28]   subgraph "Phase 3: Continuous – Operations"
[L29]     L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L30]     A --> L8
[L31]     L8 -->|"Self-Evolution<br/>Feedback"| KF
[L32]   end
[L33]
```

[L34]

[L35] ### 3.2 Data Flow

[L36]

[L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47]   ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53]   ↓ Layer 7: Build prompt, invoke LLM
[L54]   ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56]   ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

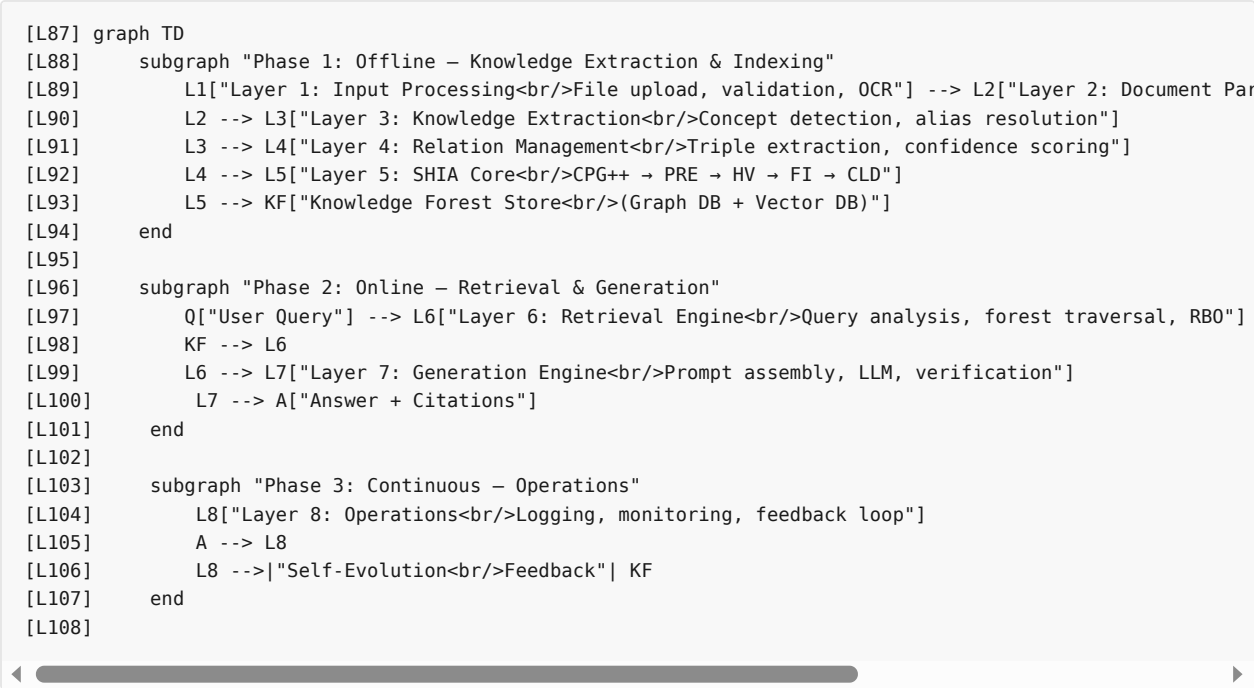
[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `filecite`turn6file1

[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(2).md
[L81] mime_type: text/markdown
[L82] surface: library
[L83] score: 0.017947214076246335
[L84] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--6
[L85] snippet:
[L86]



[L109]
[L110] ### 3.2 Data Flow

[L111]
[L112]

```
[L113] Raw Document (PDF/DOCX/HTML)
[L114]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L115] Normalized Text + Metadata
[L116]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L117] [Page → Region → Block] hierarchy
[L118]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L119] KnowledgeUnit objects (candidate concepts)
[L120]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L121] KnowledgeNode objects (canonical, deduplicated)
[L122]   ↓ Layer 4: Validate relations against ontology, score confidence
[L123] KnowledgeEdge objects (typed, scored)
[L124]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L125] Knowledge Forest (rooted trees + cross-links)
[L126]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L127] Selected context nodes (within token budget)
[L128]   ↓ Layer 7: Build prompt, invoke LLM
[L129]   ↓ Layer 7: Verify claims against evidence, bind citations
[L130] Answer with citations + confidence score
[L131]   ↓ Layer 8: Log, collect feedback, update forest weights
[L132]
```

[L133]
[L134] ### 3.3 Layer Summary
[L135]
[L136] | Layer | Name | Input | Output | Key Algorithms |
[L137] |-----|-----|-----|-----|-----|
[L138] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L139] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L140] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L141] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L142] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L143] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L144] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L145] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L146] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L147]
[L148] ---
[L149]
[L150] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:17:23
Citation Marker: `[[filecite]]turn6file2[[`

[L151] metadata:
[L152] query_result_index: 2
[L153] file_id: file_000000003c8081fab658b1752549d64
[L154] version_id: 1
[L155] name: SHIA_RAG_Complete_Report(5).md
[L156] mime_type: text/markdown

[L157] surface: library
 [L158] score: 0.01769119769119769
 [L159] document_chunk_id: file_000000003c8081fab658b1752549d64--1--31
 [L160] snippet:
 [L161] 2f} below threshold")
 [L162]
 [L163] return full_text, pages, is_ocr, ocr_conf
 [L164]

```
[L165]
[L166] **Error handling:** See Section 7.7 for `OCRFailure`, `UnsupportedFormatError`, and retry logic.
[L167]
[L168] ---
[L169]
[L170] ### 7.3 Layer 2: Document Parsing
[L171]
[L172] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L173]
[L174] **Input:** `NormalizedDocument` from Layer 1
[L175] **Output:** List of `DocumentBlock` objects
[L176]
[L177]
```

python

```
[L178] from enum import Enum
[L179] from dataclasses import dataclass
[L180]
[L181] class BlockType(Enum):
[L182]     HEADING = "heading"
[L183]     PARAGRAPH = "paragraph"
[L184]     LIST = "list"
[L185]     TABLE = "table"
[L186]     CODE = "code"
[L187]     ALGORITHM = "algorithm"
[L188]     FIGURE_CAPTION = "figure_caption"
[L189]     EQUATION = "equation"
[L190]     DEFINITION = "definition"
[L191]
[L192] @dataclass
[L193] class DocumentBlock:
[L194]     """Atomic unit of document structure."""
[L195]     block_id: str
[L196]     doc_id: str
[L197]     page_number: int
[L198]     region_index: int # Which region on the page
[L199]     block_type: BlockType
[L200]     text: str
[L201]     reading_order: int # Global ordering across the document
[L202]     bounding_box: Optional[dict] = None # For PDF: {x0, y0, x1, y1}
[L203]     confidence: float = 1.0 # Classification confidence
```

```
[L204]
[L205] def layer2_parse(doc: NormalizedDocument) -> list:
[L206] """Layer 2: Document Parsing pipeline."""
[L207] blocks = []
[L208] global_order = 0
[L209]
[L210] for page_num, page_text in enumerate(doc.pages or [doc.text]):
[L211] # Step 1: Segment page into regions (contiguous content areas)
[L212] regions = segment_into_regions(page_text)
[L213]
[L214] for region_idx, region in enumerate(regions):
[L215] # Step 2: Split region into blocks
[L216] raw_blocks = split_into_blocks(region)
[L217]
[L218] for raw_block in raw_blocks:
[L219] # Step 3: Classify block type
[L220] block_type, conf = classify_block(raw_block)
[L221]
[L222] blocks.append(DocumentBlock(
[L223] block_id=f"BLK_{len(blocks):06d}",
[L224] doc_id=doc.doc_id,
[L225] page_number=page_num + 1,
[L226] region_index=region_idx,
[L227] block_type=block_type,
[L228] text=raw_block.strip(),
[L229] reading_order=global_order,
[L230] confidence=conf
[L231] ))
[L232] global_order += 1
[L233]
[L234] return blocks
[L235]
[L236] def classify_block(text: str) -> tuple:
[L237] """Classify a text block into a BlockType using rule-based heuristics."""
[L238] text_stripped = text.strip()
[L239]
[L240] # Heading detection: short line, no ending punctuation, often capitalized
[L241] if len(text_stripped) < 100 and not text_stripped.endswith('.') and text_stripped[0].isupper():
[L242] if text_stripped.startswith('#') or text_stripped.isupper():
[L243] return BlockType.HEADING, 0.9
[L244]
[L245] # List detection
[L246] if any(text_stripped.startswith(p) for p in ['- ', '* ', '• ', '1.', '2.
```

[L247] metadata:

[L248] query_result_index: 3

[L249] file_id: file_00000000fb2c81fd82f14a7a48993554

[L250] version_id: 1

[L251] name: SHIA_RAG_Final_Report.md

[L252] mime_type: text/markdown

[L253] surface: library

[L254] score: 0.01744318181818182

[L255] document_chunk_id: file_00000000fb2c81fd82f14a7a48993554--1--6

[L256] snippet:

[L257]

```
[L258] graph TD
[L259]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L260]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L261]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L262]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L263]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L264]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L265]     end
[L266]
[L267]     subgraph "Phase 2: Online – Retrieval & Generation"
[L268]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L269]         KF --> L6
[L270]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L271]         L7 --> A["Answer + Citations"]
[L272]     end
[L273]
[L274]     subgraph "Phase 3: Continuous – Operations"
[L275]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L276]         A --> L8
[L277]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L278]     end
[L279]
```

[L280]

[L281] ### 3.2 Data Flow

[L282]

[L283]

```
[L284] Raw Document (PDF/DOCX/HTML)
[L285]   ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L286] Normalized Text + Metadata
[L287]   ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L288] [Page → Region → Block] hierarchy
[L289]   ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L290] KnowledgeUnit objects (candidate concepts)
[L291]   ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L292] KnowledgeNode objects (canonical, deduplicated)
[L293]   ↓ Layer 4: Validate relations against ontology, score confidence
[L294] KnowledgeEdge objects (typed, scored)
[L295]   ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L296] Knowledge Forest (rooted trees + cross-links)
[L297]   ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L298] Selected context nodes (within token budget)
[L299]   ↓ Layer 7: Build prompt, invoke LLM
[L300]   ↓ Layer 7: Verify claims against evidence, bind citations
[L301] Answer with citations + confidence score
[L302]   ↓ Layer 8: Log, collect feedback, update forest weights
[L303]
```

```
[L304]
[L305] ### 3.3 Layer Summary
[L306]
[L307] | Layer | Name | Input | Output | Key Algorithms |
[L308] |-----|-----|-----|-----|-----|
[L309] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L310] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L311] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L312] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L313] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L314] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L315] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L316] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L317] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L318]
[L319] ---
[L320]
[L321] ## 4. Data Model — Layer 0
```

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `[[filecite]]turn6file4`

```
[L322] metadata:
[L323] query_result_index: 4
[L324] file_id: file_00000000f620821186e54e582d59731e
[L325] version_id: 1
[L326] name: SHIA_RAG_Final_Report(3).md
[L327] mime_type: text/markdown
[L328] surface: library
[L329] score: 0.01744318181818182
```


[L330] document_chunk_id: file_00000000f620821186e54e582d59731e--1--6

[L331] snippet:

[L332]

```
[L333] graph TD
[L334]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L335]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L336]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L337]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L338]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L339]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L340]     end
[L341]
[L342]     subgraph "Phase 2: Online – Retrieval & Generation"
[L343]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L344]         KF --> L6
[L345]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L346]         L7 --> A["Answer + Citations"]
[L347]     end
[L348]
[L349]     subgraph "Phase 3: Continuous – Operations"
[L350]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L351]         A --> L8
[L352]         L8 -->|"Self-Evolution<br/>Feedback"| KF
[L353]     end
[L354]
```

[L355]

[L356] ### 3.2 Data Flow

[L357]

[L358]

```
[L359] Raw Document (PDF/DOCX/HTML)
[L360]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L361] Normalized Text + Metadata
[L362]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L363] [Page → Region → Block] hierarchy
[L364]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L365] KnowledgeUnit objects (candidate concepts)
[L366]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L367] KnowledgeNode objects (canonical, deduplicated)
[L368]     ↓ Layer 4: Validate relations against ontology, score confidence
[L369] KnowledgeEdge objects (typed, scored)
[L370]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L371] Knowledge Forest (rooted trees + cross-links)
[L372]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L373] Selected context nodes (within token budget)
[L374]     ↓ Layer 7: Build prompt, invoke LLM
[L375]     ↓ Layer 7: Verify claims against evidence, bind citations
[L376] Answer with citations + confidence score
[L377]     ↓ Layer 8: Log, collect feedback, update forest weights
[L378]
```

[L379]

[L380] ### 3.3 Layer Summary

[L381]

[L382] | Layer | Name | Input | Output | Key Algorithms |

[L383] |-----|-----|-----|-----|-----|

[L384] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
 [L385] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
 [L386] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
 [L387] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
 [L388] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
 [L389] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
 [L390] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
 [L391] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
 [L392] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
 [L393]
 [L394] ---
 [L395]
 [L396] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: □filecite□turn6file5□

[L397] metadata:
 [L398] query_result_index: 5
 [L399] file_id: file_0000000012688211b4f57f5b8bcb8b42
 [L400] version_id: 1
 [L401] name: SHIA_RAG_Final_Report(4).md
 [L402] mime_type: text/markdown
 [L403] surface: conversation
 [L404] score: 0.017198679141441936
 [L405] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--17
 [L406] snippet:
 [L407] 1)
 [L408] if success_rate < 0.8:
 [L409] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
 [L410] elif fmt == 'docx':
 [L411] text, pages = extract_docx(file_path), []
 [L412] is_ocr, ocr_conf = False, 1.0
 [L413] elif fmt == 'html':
 [L414] text, pages = extract_html(file_path), []
 [L415] is_ocr, ocr_conf = False, 1.0
 [L416] else: # txt
 [L417] text = open(file_path, 'r', encoding='utf-8').read()
 [L418] pages, is_ocr, ocr_conf = [], False, 1.0
 [L419]
 [L420] language = detect_language(text)
 [L421] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
 [L422] metadata = extract_metadata(file_path, fmt)
 [L423]
 [L424] doc = NormalizedDocument(
 [L425] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),

```

[L426] format=DocumentFormat(fmt), text=text, pages=pages,
[L427] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L428] char_count=len(text), page_count=len(pages) or 1,
[L429] content_hash=content_hash, language=language,
[L430] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L431] )
[L432] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L433] return doc
[L434]

```

```

[L435]
[L436] ---
[L437]
[L438] ### 5.2 Layer 2: Document Parsing
[L439]
[L440] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L441]
[L442] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L443]
[L444]

```

python

```

[L445] class BlockType(Enum):
[L446]     HEADING = "heading"
[L447]     PARAGRAPH = "paragraph"
[L448]     LIST = "list"
[L449]     TABLE = "table"
[L450]     CODE = "code"
[L451]     ALGORITHM = "algorithm"
[L452]     FIGURE_CAPTION = "figure_caption"
[L453]     EQUATION = "equation"
[L454]     DEFINITION = "definition"
[L455]
[L456] @dataclass
[L457] class DocumentBlock:
[L458]     """Atomic unit of document structure."""
[L459]     block_id: str
[L460]     doc_id: str
[L461]     page_number: int
[L462]     region_index: int # Which region on the page
[L463]     block_type: BlockType
[L464]     text: str
[L465]     reading_order: int # Global ordering across the document
[L466]     confidence: float = 1.0 # Classification confidence
[L467]     table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L468]

```

```

[L469]
[L470] The parser uses rule-based heuristics for block classification:
[L471] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L472] - List: Starts with ` - `, `* `, `• `, or `1.`
[L473] - Table: Contains 3+ pipe characters (`|`) – uses regex `re.match(r'^(def |class |import |from )'
[L474] - Code: Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")
[L475] - Definition: Contains phrases like "is defined as", "refers to"
[L476] - Default: Paragraph
[L477]
[L478] ##### Table Cell-Level Parsing
[L479]
[L480] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-
[L481]
[L482]

```

python

[L483] def parse_table_cells(page, table_region) -> list:

[L484] """Extract table structure with cell-level detail.

[L485]

[L486] Uses pdfplumber for text-based tables, Camelot for image-based.

[L487] Returns list of rows, each row is a list of cell strings."""

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `filecite[turn6file6]`

[L488] metadata:

[L489] query_result_index: 6

[L490] file_id: file_000000001dd481fa9855f2731dbc53d5

[L491] version_id: 1

[L492] name: SHIA_RAG_Complete_Report(6).md

[L493] mime_type: text/markdown

[L494] surface: library

[L495] score: 0.01696969696969697

[L496] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--31

[L497] snippet:

[L498] 2f} below threshold")

[L499]

[L500] return full_text, pages, is_ocr, ocr_conf

[L501]

```

[L502]
[L503] Error handling: See Section 7.7 for `OCRFailure`, `UnsupportedFormatError`, and retry logic.
[L504]
[L505] ---
[L506]
[L507] ### 7.3 Layer 2: Document Parsing
[L508]
[L509] Responsibility: Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L510]
[L511] Input: `NormalizedDocument` from Layer 1
[L512] Output: List of `DocumentBlock` objects
[L513]
[L514]

```

```

python
[L515] from enum import Enum
[L516] from dataclasses import dataclass
[L517]
[L518] class BlockType(Enum):
[L519]     HEADING = "heading"
[L520]     PARAGRAPH = "paragraph"
[L521]     LIST = "list"
[L522]     TABLE = "table"
[L523]     CODE = "code"
[L524]     ALGORITHM = "algorithm"
[L525]     FIGURE_CAPTION = "figure_caption"
[L526]     EQUATION = "equation"
[L527]     DEFINITION = "definition"
[L528]
[L529] @dataclass
[L530] class DocumentBlock:
[L531]     """Atomic unit of document structure."""
[L532]     block_id: str
[L533]     doc_id: str
[L534]     page_number: int
[L535]     region_index: int # Which region on the page
[L536]     block_type: BlockType
[L537]     text: str
[L538]     reading_order: int # Global ordering across the document
[L539]     bounding_box: Optional[dict] = None # For PDF: {x0, y0, x1, y1}
[L540]     confidence: float = 1.0 # Classification confidence
[L541]
[L542] def layer2_parse(doc: NormalizedDocument) -> list:
[L543]     """Layer 2: Document Parsing pipeline."""
[L544]     blocks = []
[L545]     global_order = 0
[L546]
[L547]     for page_num, page_text in enumerate(doc.pages or [doc.text]):
[L548]         # Step 1: Segment page into regions (contiguous content areas)
[L549]         regions = segment_into_regions(page_text)
[L550]
[L551]         for region_idx, region in enumerate(regions):
[L552]             # Step 2: Split region into blocks
[L553]             raw_blocks = split_into_blocks(region)
[L554]
[L555]             for raw_block in raw_blocks:
[L556]                 # Step 3: Classify block type
[L557]                 block_type, conf = classify_block(raw_block)
[L558]
[L559]                 blocks.append(DocumentBlock(

```

```

[L560] block_id=f"BLK_{len(blocks):06d}",
[L561] doc_id=doc.doc_id,
[L562] page_number=page_num + 1,
[L563] region_index=region_idx,
[L564] block_type=block_type,
[L565] text=raw_block.strip(),
[L566] reading_order=global_order,
[L567] confidence=conf
[L568] ))
[L569] global_order += 1
[L570]
[L571] return blocks
[L572]
[L573] def classify_block(text: str) -> tuple:
[L574] """Classify a text block into a BlockType using rule-based heuristics."""
[L575] text_stripped = text.strip()
[L576]
[L577] # Heading detection: short line, no ending punctuation, often capitalized
[L578] if len(text_stripped) < 100 and not text_stripped.endswith('.') and text_stripped[0].isupper():
[L579] if text_stripped.startswith('#') or text_stripped.isupper():
[L580] return BlockType.HEADING, 0.9
[L581]
[L582] # List detection
[L583] if any(text_stripped.startswith(p) for p in ['- ', '* ', '• ', '1.', '2.

```

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `filecite[turn6file7]`

```

[L584] metadata:
[L585] query_result_index: 7
[L586] file_id: file_0000000033088207b45d2ae58cbd406c
[L587] version_id: 1
[L588] name: SHIA_RAG_Complete_Report(8).md
[L589] mime_type: text/markdown
[L590] surface: library
[L591] score: 0.016950228542407315
[L592] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--34
[L593] snippet:
[L594] > [!IMPORTANT]
[L595] > Layer 3 / Layer 4 boundary: Layer 3 performs candidate extraction — it identifies entities, definitions, and
raw relation triples from text. Layer 4 performs validation and scoring — it validates extracted relations against the
ontology, resolves conflicts, and assigns confidence scores. Layer 3 outputs raw KnowledgeUnit objects; Layer 4
produces scored KnowledgeEdge objects.
[L596]
[L597] Input: List of DocumentBlock from Layer 2
[L598] Output: List of KnowledgeNode objects (canonical, deduplicated)

```

```

[L599]
[L600] ```python
[L601] from dataclasses import dataclass
[L602]
[L603] @dataclass
[L604] class KnowledgeUnit:
[L605] """Atomic fact extracted from a single document block.
[L606] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L607] unit_id: str
[L608] concept_name: str # Raw extracted concept name
[L609] aliases: list # Alternative names found in text
[L610] definitions: list # Definition sentences
[L611] properties: dict # Key-value properties (typed)
[L612] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]
[L613] source_doc: str
[L614] source_block: str
[L615] page: int
[L616] extraction_confidence: float # LLM extraction confidence
[L617] domain_hint: str # LLM-suggested domain
[L618]
[L619] # --- Extraction Pipeline ---
[L620]
[L621] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L622]
[L623] 1. Concepts: Named entities, technical terms, or ideas discussed
[L624] 2. Definitions: If the text defines any concept, extract the definition
[L625] 3. Properties: Key attributes with types and values
[L626] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L627] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")
[L628]
[L629] Respond in JSON format:
[L630] {
[L631] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."], "properties": {"key": {"type": "...", "value": "..."}}}],
[L632] "relations": [{"source": "...", "type": "IS_A|PART_OF|USES|CAUSES|...", "target": "...", "evidence": "..."}],
[L633] "domain": "...
[L634] }
[L635]
[L636] TEXT BLOCK:
[L637] {text}
[L638] """
[L639]
[L640] def layer3_extract(blocks: list, llm_client) -> list:
[L641] """Layer 3: Knowledge Extraction pipeline."""
[L642] knowledge_units = []
[L643]

```

```

[L644] for block in blocks:
[L645] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L646] continue # Skip non-content blocks
[L647]
[L648] # Step 1: LLM-based extraction
[L649] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L650] try:
[L651] response = llm_client.query(prompt, response_format="json")
[L652] extracted = json.loads(response)
[L653] except (json.JSONDecodeError, LLMLTimeout) as e:
[L654] # Fallback: rule-based extraction
[L655] extracted = rule_based_extract(block.text)
[L656]
[L657] # Step 2: Create KnowledgeUnit per concept
[L658] for concept in extracted.get('concepts', []):
[L659] unit = KnowledgeUnit(
[L660] unit_id=f"KU_{len(knowledge_units):06d}",
[L661] concept_name=concept['name'],
[L662] aliases=concept.get('aliases', []),
[L663] definitions=concept.get('definitions', []),
[L664] properties=concept.get('properties', {}),
[L665] raw_relations=[r for r in extracted.get('relations', [])
[L666] if r['source'] == concept['name']],
[L667] source_doc=block.doc_id,
[L668] source_block=block.block_id,
[L669] page=block.page_number,

```

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `filecite[turn6file8]`

```

[L670] metadata:
[L671] query_result_index: 8
[L672] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L673] version_id: 1
[L674] name: SHIA_RAG_Final_Report(4).md
[L675] mime_type: text/markdown
[L676] surface: conversation
[L677] score: 0.016743554952510176
[L678] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--6
[L679] snippet:
[L680]

```



```
[L681] graph TD
[L682]     subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L683]         L1["Layer 1: Input Processing<br/>File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L684]         L2 --> L3["Layer 3: Knowledge Extraction<br/>Concept detection, alias resolution"]
[L685]         L3 --> L4["Layer 4: Relation Management<br/>Triple extraction, confidence scoring"]
[L686]         L4 --> L5["Layer 5: SHIA Core<br/>CPG++ → PRE → HV → FI → CLD"]
[L687]         L5 --> KF["Knowledge Forest Store<br/>(Graph DB + Vector DB)"]
[L688]     end
[L689]
[L690]     subgraph "Phase 2: Online – Retrieval & Generation"
[L691]         Q["User Query"] --> L6["Layer 6: Retrieval Engine<br/>Query analysis, forest traversal, RB0"]
[L692]         KF --> L6
[L693]         L6 --> L7["Layer 7: Generation Engine<br/>Prompt assembly, LLM, verification"]
[L694]         L7 --> A["Answer + Citations"]
[L695]     end
[L696]
[L697]     subgraph "Phase 3: Continuous – Operations"
[L698]         L8["Layer 8: Operations<br/>Logging, monitoring, feedback loop"]
[L699]         A --> L8
[L700]         L8 -->| "Self-Evolution<br/>Feedback" | KF
[L701]     end
[L702]
```

[L703]

[L704] ### 3.2 Data Flow

[L705]

[L706]

```
[L707] Raw Document (PDF/DOCX/HTML)
[L708]     ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L709] Normalized Text + Metadata
[L710]     ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L711] [Page → Region → Block] hierarchy
[L712]     ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L713] KnowledgeUnit objects (candidate concepts)
[L714]     ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L715] KnowledgeNode objects (canonical, deduplicated)
[L716]     ↓ Layer 4: Validate relations against ontology, score confidence
[L717] KnowledgeEdge objects (typed, scored)
[L718]     ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L719] Knowledge Forest (rooted trees + cross-links)
[L720]     ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L721] Selected context nodes (within token budget)
[L722]     ↓ Layer 7: Build prompt, invoke LLM
[L723]     ↓ Layer 7: Verify claims against evidence, bind citations
[L724] Answer with citations + confidence score
[L725]     ↓ Layer 8: Log, collect feedback, update forest weights
[L726]
```

[L727]

[L728] ### 3.3 Layer Summary

[L729]

[L730] | Layer | Name | Input | Output | Key Algorithms |

[L731] |-----|-----|-----|-----|-----|

[L732] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L733] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L734] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L735] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L736] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L737] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L738] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L739] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L740] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L741]
[L742] ---
[L743]
[L744] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:17:23

Citation Marker: `[[filecite]]turn6file9`

[L745] metadata:
[L746] query_result_index: 9
[L747] file_id: file_00000000f620821186e54e582d59731e
[L748] version_id: 1
[L749] name: SHIA_RAG_Final_Report(3).md
[L750] mime_type: text/markdown
[L751] surface: library
[L752] score: 0.016524064171122996
[L753] document_chunk_id: file_00000000f620821186e54e582d59731e--1--7
[L754] snippet:
[L755] deduplicated)
[L756] ↓ Layer 4: Validate relations against ontology, score confidence
[L757] KnowledgeEdge objects (typed, scored)
[L758] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L759] Knowledge Forest (rooted trees + cross-links)
[L760] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L761] Selected context nodes (within token budget)
[L762] ↓ Layer 7: Build prompt, invoke LLM
[L763] ↓ Layer 7: Verify claims against evidence, bind citations
[L764] Answer with citations + confidence score
[L765] ↓ Layer 8: Log, collect feedback, update forest weights
[L766]

```
[L767]
[L768] ### 3.3 Layer Summary
[L769]
[L770] | Layer | Name | Input | Output | Key Algorithms |
[L771] |-----|-----|-----|-----|-----|
[L772] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L773] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L774] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L775] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L776] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L777] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L778] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L779] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L780] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L781]
[L782] ---
[L783]
[L784] ## 4. Data Model – Layer 0
[L785]
[L786] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L787]
[L788] ### 4.1 KnowledgeNode Schema
[L789]
[L790] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L791]
[L792]
```

json

```
[L793] {
[L794]   "node_id": "NODE_000789",
[L795]   "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L796]   "canonical_name": "Round Robin",
[L797]   "aliases": ["RR", "Round-Robin Scheduling"],
[L798]   "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L799]   "summaries": ["CPU scheduling method using time slices"],
[L800]   "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L801]   "domain": "Operating Systems",
[L802]   "properties": {
[L803]     "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L804]     "preemptive": {"type": "boolean", "value": true}
[L805]   },
[L806]   "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L807]   "parent": "NODE_000456",
[L808]   "children": ["NODE_001001", "NODE_001002"],
[L809]   "related": ["NODE_000890"],
[L810]   "incoming_edges": ["EDGE_001"],
[L811]   "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L812]   "evidence": [
[L813]     {
[L814]       "evidence_type": "DEFINITION",
[L815]       "text": "Round Robin is a preemptive scheduling algorithm...",
[L816]       "source_doc": "DOC_000001",
[L817]       "source_block": "BLK_000234",
```

[L818] "page": 42

[L819] }

[L820]],

DEEP RESEARCH 20/08/2026, 14:17:26

Citation Marker: □filecite□turn7file0□

[L1] metadata:

[L2] ref_id: turn6file2

[L3] file_id: file_000000003c8081fab658b1752549d64

[L4] version_id: 1

[L5] name: SHIA_RAG_Complete_Report(5).md

[L6] lines_shown: 170-349

[L7] total_file_lines: 3316

[L8] has_more: True

[L9] next_start_line: 350

[L10] next_read: {"ref_id":"turn6file2","mode":"full","start_line":350,"max_lines":180}

[L11] content:

[L12] + inserted = False

[L13] + for parent in ranked_parents:

[L14] + if HV.validate(parent, node):

[L15] + FI.insert(node, parent)

[L16] + inserted = True

[L17] + break

[L18] + if not inserted:

[L19] + FI.insert_as_root(node) # Create new tree

[L20]

[L21]

[L22] ---

[L23]

[L24] ### 3.3 FATAL: Confidence Propagation Without Topological Sort (PDF 2)

[L25]

[L26] **The Bug:** `propagate_confidence()` iterates over nodes in arbitrary order. A child's confidence dep

[L27]

[L28]

diff

[L29] # BROKEN

[L30] - for node in forest.nodes(): # Arbitrary order

[L31] - if node.parents:

[L32] - node.confidence = avg(p.confidence for p in node.parents) *decay*

[L33]

[L34] # FIXED: Use topological sort (parents always before children)

[L35] # FIXED: Include node's extraction confidence and evidence confidence

[L36] + from collections import deque

[L37] +

[L38] + # Confidence weighting factors (must sum to 1.0)

[L39] + ALPHA = 0.4 # Node's own extraction confidence weight

```

[L40] + BETA = 0.4 # Parent confidence weight
[L41] + GAMMA = 0.2 # Evidence confidence weight
[L42] + DECAY = 0.95 # Per-level decay factor
[L43] +
[L44] + def propagate_confidence(forest):
[L45] + # Kahn's algorithm for topological ordering
[L46] + in_degree = {n: len(forest.parents(n)) for n in forest.nodes()}
[L47] + queue = deque([n for n, d in in_degree.items() if d == 0]) # Roots first
[L48] + while queue:
[L49] + node = queue.popleft()
[L50] + if forest.parents(node):
[L51] + parent_confs = [p.confidence for p in forest.parents(node)]
[L52] + avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L53] + evidence_conf = avg([e.confidence for e in node.evidence]) if node.evidence else 0.5
[L54] + node.confidence = (
[L55] + ALPHA node.extraction_confidence +
[L56] + BETA avg_parent_conf +
[L57] + GAMMA evidence_conf
[L58] + ) DECAY
[L59] + else:
[L60] + node.confidence = node.base_confidence # Root nodes (default: 0.5)
[L61] + for child in forest.children(node):
[L62] + in_degree[child] -= 1
[L63] + if in_degree[child] == 0:
[L64] + queue.append(child)
[L65]

```

```

[L66]
[L67] > [!NOTE]
[L68] > base_confidence is the standardized term used throughout (default: `0.5`, configurable). Previous
[L69]
[L70] ---
[L71]
[L72] ### 3.4 CRITICAL: RBO Knapsack Ignores Node Dependencies (PDF 6)
[L73]
[L74] The Bug: The Retrieval Budget Optimizer treats nodes as independent items in a 0-1 Knapsack. But i
[L75]
[L76] > [!WARNING]
[L77] > Implementation Note: The algorithm below is a greedy approximation for the Precedence-Constr
[L78]
[L79]

```

diff

```

[L80] # BROKEN: Standard Knapsack (independent items)
[L81] - nodes.sort(key=lambda n: n.utility / n.token_cost, reverse=True)
[L82] - for n in nodes:
[L83] - if budget >= n.token_cost:
[L84] - selected.append(n)
[L85] - budget -= n.token_cost
[L86]
[L87] # FIXED: Greedy Approximation for Tree Knapsack (precedence-constrained)

```

```

[L88] + def tree_knapsack(root, budget, forest):
[L89] + """Select nodes maximizing utility, but a child can only be
[L90] + selected if its parent is also selected.
[L91] +
[L92] + NOTE: This is a greedy approximation (not exact DP).
[L93] + Children are sorted by utility-density and greedily allocated.
[L94] + Exact DP is computationally prohibitive for large token budgets."""
[L95] + dp = {} # Memoization cache
[L96] +
[L97] + def solve(node, remaining_budget):
[L98] + if (node.id, remaining_budget) in dp:
[L99] + return dp[(node.id, remaining_budget)]
[L100] +
[L101] + # Option 1: Don't include this node (or any descendants)
[L102] + best = (0, set())
[L103] +
[L104] + # Option 2: Include this node, then greedily pick children
[L105] + if remaining_budget >= node.token_cost:
[L106] + child_budget = remaining_budget - node.token_cost
[L107] + total_util = node.utility
[L108] + selected = {node.id}
[L109] +
[L110] + # Greedily allocate remaining budget to children by utility-density
[L111] + children = sorted(forest.children(node),
[L112] + key=lambda c: c.utility / max(c.token_cost, 1),
[L113] + reverse=True)
[L114] + for child in children:
[L115] + if child_budget >= child.token_cost:
[L116] + cu, cs = solve(child, child_budget)
[L117] + total_util += cu
[L118] + selected |= cs
[L119] + child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L120] +
[L121] + if total_util > best[0]:
[L122] + best = (total_util, selected)
[L123] +
[L124] + dp[(node.id, remaining_budget)] = best
[L125] + return best
[L126] +
[L127] + return solve(root, budget)

```

[L128]

```

[L129]
[L130] ---
[L131]
[L132] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)
[L133]
[L134] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q`
[L135]
[L136] **Fix:** Estimate abstraction level from textual features before placement:
[L137]
[L138]

```

python

```

[L139] def estimate_abstraction_level(node):
[L140] """Multi-factor heuristic for estimating abstraction level of unplaced nodes.
[L141] Returns a score in [0.0, 1.0] where higher = more abstract.
[L142]
[L143] NOTE: Simple name-length alone is insufficient (e.g., 'A' → abstract? No).
[L144] We combine multiple signals for robustness."""
[L145] score = 0.0
[L146]
[L147] # Factor 1: Name length (shorter multi-word names tend to be more abstract)
[L148] word_count = len(node.name.split())
[L149] if word_count <= 2:
[L150] score += 0.15 # Short names like "Scheduling", "CPU" are abstract
[L151] elif word_count >= 4:
[L152] score -= 0.05 # Long names like "Advanced Encryption Standard" are specific
[L153]
[L154] # Factor 2: Number of source documents (more sources = more general)
[L155] score += min(len(node.sources) * 0.05, 0.20)
[L156]
[L157] # Factor 3: Number of sub-concept hints (more sub-concepts = more abstract)
[L158] score += min(len(node.children_hints) * 0.05, 0.25)
[L159]
[L160] # Factor 4: Definition keywords indicating abstraction
[L161] abstract_keywords = ['general', 'category', 'type', 'class', 'family', 'group',
[L162] 'framework', 'paradigm', 'approach', 'method', 'technique']
[L163] specific_keywords = ['specific', 'particular', 'exact', 'instance', 'example',
[L164] 'implementation', 'variant', 'version']
[L165] defn_text = ' '.join(node.definitions).lower() if node.definitions else ""
[L166] score += sum(0.05 for kw in abstract_keywords if kw in defn_text)
[L167] score -= sum(0.05 for kw in specific_keywords if kw in defn_text)
[L168]
[L169] # Factor 5: Hypernym detection (LLM-based, optional)
[L170] # If available, use LLM to check if node name is a hypernym
[L171] # score += hypernym_llm_score(node) # Expensive, use sparingly
[L172]

```

[L173] return max(0.0, min(score, 1.0)) # Clamp to [0, 1]

[L174]

```
[L175]
[L176] ---
[L177]
[L178] ### 3.6 CRITICAL:  $O(N^2)$  Canonicalization (PDF 2)
[L179]
[L180] **The Bug:** Alias merging compares every node pair: `for A in nodes: for B in nodes:`  $\rightarrow O(N^2)$ . For n
[L181]
[L182] **Fix:** Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L183]
[L184]
```

python

[L185] from datasketch import MinHashLSH

[L186]

[L187] def efficient_canonicalization(nodes, threshold=0.85):

[L188] lsh = MinHashLSH(threshold=threshold, num_perm=128)

[L189]

[L190] # Index all nodes by their text signature

[L191] for node in nodes:

DEEP RESEARCH 20/08/2026, 14:17:30

Citation Marker: `filecite`turn8file0

[L1] metadata:

[L2] ref_id: turn6file2

[L3] file_id: file_000000003c8081fab658b1752549d64

[L4] version_id: 1

[L5] name: SHIA_RAG_Complete_Report(5).md

[L6] lines_shown: 236-395

[L7] total_file_lines: 3316

[L8] has_more: True

[L9] next_start_line: 396

[L10] next_read: {"ref_id":"turn6file2","mode":"full","start_line":396,"max_lines":160}

[L11] content:

[L12]

[L13]


```

[L14] # BROKEN: Standard Knapsack (independent items)
[L15] - nodes.sort(key=lambda n: n.utility / n.token_cost, reverse=True)
[L16] - for n in nodes:
[L17] -     if budget >= n.token_cost:
[L18] -         selected.append(n)
[L19] -         budget -= n.token_cost
[L20]
[L21] # FIXED: Greedy Approximation for Tree Knapsack (precedence-constrained)
[L22] + def tree_knapsack(root, budget, forest):
[L23] +     """Select nodes maximizing utility, but a child can only be
[L24] +         selected if its parent is also selected.
[L25] +
[L26] +         NOTE: This is a greedy approximation (not exact DP).
[L27] +         Children are sorted by utility-density and greedily allocated.
[L28] +         Exact DP is computationally prohibitive for large token budgets."""
[L29] +     dp = {} # Memoization cache
[L30] +
[L31] +     def solve(node, remaining_budget):
[L32] +         if (node.id, remaining_budget) in dp:
[L33] +             return dp[(node.id, remaining_budget)]
[L34] +
[L35] +         # Option 1: Don't include this node (or any descendants)
[L36] +         best = (0, set())
[L37] +
[L38] +         # Option 2: Include this node, then greedily pick children
[L39] +         if remaining_budget >= node.token_cost:
[L40] +             child_budget = remaining_budget - node.token_cost
[L41] +             total_util = node.utility
[L42] +             selected = {node.id}
[L43] +
[L44] +             # Greedily allocate remaining budget to children by utility-density
[L45] +             children = sorted(forest.children(node),
[L46] +                             key=lambda c: c.utility / max(c.token_cost, 1),
[L47] +                             reverse=True)
[L48] +             for child in children:
[L49] +                 if child_budget >= child.token_cost:
[L50] +                     cu, cs = solve(child, child_budget)
[L51] +                     total_util += cu
[L52] +                     selected |= cs
[L53] +                     child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L54] +
[L55] +             if total_util > best[0]:
[L56] +                 best = (total_util, selected)
[L57] +
[L58] +             dp[(node.id, remaining_budget)] = best
[L59] +             return best
[L60] +
[L61] +     return solve(root, budget)
[L62]

```

[L63]

[L64] ---

[L65]

[L66] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)

[L67]

[L68] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q` is a new, unplaced node — it has no level/depth yet.

[L69]

[L70] **Fix:** Estimate abstraction level from textual features before placement:

[L71]

[L72]

```

[L73] def estimate_abstraction_level(node):
[L74]     """Multi-factor heuristic for estimating abstraction level of unplaced nodes.
[L75]     Returns a score in [0.0, 1.0] where higher = more abstract.
[L76]
[L77]     NOTE: Simple name-length alone is insufficient (e.g., 'A' → abstract? No).
[L78]     We combine multiple signals for robustness."""
[L79]     score = 0.0
[L80]
[L81]     # Factor 1: Name length (shorter multi-word names tend to be more abstract)
[L82]     word_count = len(node.name.split())
[L83]     if word_count <= 2:
[L84]         score += 0.15 # Short names like "Scheduling", "CPU" are abstract
[L85]     elif word_count >= 4:
[L86]         score -= 0.05 # Long names like "Advanced Encryption Standard" are specific
[L87]
[L88]     # Factor 2: Number of source documents (more sources = more general)
[L89]     score += min(len(node.sources) * 0.05, 0.20)
[L90]
[L91]     # Factor 3: Number of sub-concept hints (more sub-concepts = more abstract)
[L92]     score += min(len(node.children_hints) * 0.05, 0.25)
[L93]
[L94]     # Factor 4: Definition keywords indicating abstraction
[L95]     abstract_keywords = ['general', 'category', 'type', 'class', 'family', 'group',
[L96]                          'framework', 'paradigm', 'approach', 'method', 'technique']
[L97]     specific_keywords = ['specific', 'particular', 'exact', 'instance', 'example',
[L98]                          'implementation', 'variant', 'version']
[L99]     defn_text = ' '.join(node.definitions).lower() if node.definitions else ''
[L100]    score += sum(0.05 for kw in abstract_keywords if kw in defn_text)
[L101]    score -= sum(0.05 for kw in specific_keywords if kw in defn_text)
[L102]
[L103]    # Factor 5: Hypernym detection (LLM-based, optional)
[L104]    # If available, use LLM to check if node name is a hypernym
[L105]    # score += hypernym_llm_score(node) # Expensive, use sparingly
[L106]
[L107]    return max(0.0, min(score, 1.0)) # Clamp to [0, 1]
[L108]

```

[L109]

[L110] ---

[L111]

[L112] ### 3.6 CRITICAL: $O(N^2)$ Canonicalization (PDF 2)

[L113]

[L114] **The Bug:** Alias merging compares every node pair: `for A in nodes: for B in nodes:` → $O(N^2)$. For millions of nodes, this is impossible.

[L115]

[L116] **Fix:** Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:

[L117]

[L118]

```

[L119] from datasketch import MinHashLSH
[L120]
[L121] def efficient_canonicalization(nodes, threshold=0.85):
[L122]     lsh = MinHashLSH(threshold=threshold, num_perm=128)
[L123]
[L124]     # Index all nodes by their text signature
[L125]     for node in nodes:
[L126]         mh = create_minhash(node.name + " " + node.definition)
[L127]         lsh.insert(node.id, mh)
[L128]
[L129]     # Query each node for near-duplicates (sub-linear time)
[L130]     merge_groups = UnionFind()
[L131]     for node in nodes:
[L132]         mh = create_minhash(node.name + " " + node.definition)
[L133]         candidates = lsh.query(mh) # O(1) average
[L134]         for candidate_id in candidates:
[L135]             if candidate_id != node.id:
[L136]                 merge_groups.union(node.id, candidate_id)
[L137]
[L138]     return merge_groups.groups()
[L139]

```

[L140]

[L141] ---

[L142]

[L143] ### 3.7 Other Errors Found & Fixed

[L144]

[L145] | # | Error | Source PDF | Severity | Fix |

[L146] |---|-----|-----|-----|-----|

[L147] | 7 | PRE formula inconsistency (3 different versions across PDFs) | PDFs 1,2,3 | High | Standardize to: $\text{Score} = w_1 \cdot \text{CosSim} + w_2 \cdot \text{HypernymScore} + w_3 \cdot (1/(\text{depth}+1)) + w_4 \cdot \text{EvidenceSupport} + w_5 \cdot \text{ParentConfidence}, \sum w_i = 1$ |

[L148] | 8 | HV depth check uses `depth(N)` instead of `depth(P)+1` | PDF 3 | High | Change to `if depth(P) + 1 >= maxDepth: Reject` |

[L149] | 9 | Cross-link cycle check uses hierarchy validator | PDF 2 | High | Use separate `is_valid_crosslink()` that allows cycles in semantic graph |

[L150] | 10 | Forest Density metric inverted (`nodes/edges` instead of `edges/nodes`) | PDF 3 | Medium | Fix to `FD = |edges| / |nodes|` |

[L151] | 11 | Forest Connectivity undefined without start vertex | PDF 3 | Medium | Define as `FC = |largest_connected_component| / |total_nodes|` |

[L152] | 12 | `base_confidence` undefined in confidence propagation | PDF 2 | Medium | Define as `base_confidence = 0.5` (configurable default). Use `base_confidence` consistently across all modules |

[L153] | 13 | Token budget uses `len(text)` instead of tokenizer | PDF 4 | Medium | Use `tiktoken.encoding_for_model(model).encode(text)` |

[L154] | 14 | `block.text.contains("def ")` falsely matches "definitely" | PDF 4 | Low | Use regex: `re.match(r'^(\def |class |import |from )', block.text)` |

[L155] | 15 | Definition regex `"^(.) is (a|an|the)?(.)"` too broad | PDF 4 | Medium | Add NLP dependency parsing: subject must be a noun phrase, "is" must be copular |

[L156] | 16 | Embedding averaging collapses to mean | PDF 4 | High | Use max-pooling or weighted attention over unit embeddings |

[L157] | 17 | Symmetric embedding model used for asymmetric QA retrieval | PDF 4 | High | Use asymmetric model like `multi-qa-MiniLM-L6-cos-v1` or `bge-base-en-v1.5` |

[L158] | 18 | Alias resolution at 98% embedding similarity merges antonyms | PDF 5 | High | Add semantic role check:

require same hypernym AND high similarity |

[L159] | 19 | Timeline: Gantt previously showed 11 months, text said 8 months (now corrected) | PDF 1 | Low | Standardized to 6 months with clear milestones (consistent with Section 9.2) |

[L160] | 20 | Schema mismatch: `Page.regions` type differs between Layer 0 and Layer 2 | PDF 4 | Medium | Standardize to `regions: [LayoutRegion]` everywhere |

[L161] | 21 | `ExecutionPrompt` object used but never defined | PDF 4 | Medium | Add schema definition in Layer 0 |

[L162] | 22 | No database synchronization protocol for hybrid storage | PDF 5 | High | Add saga pattern or 2-phase commit for cross-DB transactions |

[L163] | 23 | Retrieval sort by utility ignores token cost | PDF 3 | High | Sort by `utility / token_cost` as stated in theory section |

[L164] | 24 | `tieBreak(P, bestParent)` crashes when `bestParent` is null | PDF 3 | Medium | Guard: `if bestParent is None or tieBreak(P, bestParent)` |

[L165]

[L166] ---

[L167]

[L168] ## 4. Cross-Document Inconsistencies

[L169]

[L170] > [!WARNING]

[L171] > The 6 PDFs evolved iteratively but were never reconciled. Below are the major contradictions.

DEEP RESEARCH 20/08/2026, 14:17:33

Citation Marker: `filecite[turn9file0]`

[L1] metadata:

[L2] `ref_id: turn6file2`

[L3] `file_id: file_000000003c8081fab658b1752549d64`

[L4] `version_id: 1`

[L5] `name: SHIA_RAG_Complete_Report(5).md`

[L6] `lines_shown: 300-519`

[L7] `total_file_lines: 3316`

[L8] `has_more: True`

[L9] `next_start_line: 520`

[L10] `next_read: {"ref_id": "turn6file2", "mode": "full", "start_line": 520, "max_lines": 220}`

[L11] content:

[L12]

[L13] NOTE: Simple name-length alone is insufficient (e.g., 'A' → abstract? No).

[L14] We combine multiple signals for robustness. ""

[L15] `score = 0.0`

[L16]

[L17] # Factor 1: Name length (shorter multi-word names tend to be more abstract)

[L18] `word_count = len(node.name.split())`

[L19] `if word_count <= 2:`

[L20] `score += 0.15 # Short names like "Scheduling", "CPU" are abstract`

[L21] `elif word_count >= 4:`

[L22] `score -= 0.05 # Long names like "Advanced Encryption Standard" are specific`

[L23]

```

[L24] # Factor 2: Number of source documents (more sources = more general)
[L25] score += min(len(node.sources) 0.05, 0.20)
[L26]
[L27] # Factor 3: Number of sub-concept hints (more sub-concepts = more abstract)
[L28] score += min(len(node.children_hints) 0.05, 0.25)
[L29]
[L30] # Factor 4: Definition keywords indicating abstraction
[L31] abstract_keywords = ['general', 'category', 'type', 'class', 'family', 'group',
[L32] 'framework', 'paradigm', 'approach', 'method', 'technique']
[L33] specific_keywords = ['specific', 'particular', 'exact', 'instance', 'example',
[L34] 'implementation', 'variant', 'version']
[L35] defn_text = ''.join(node.definitions).lower() if node.definitions else ""
[L36] score += sum(0.05 for kw in abstract_keywords if kw in defn_text)
[L37] score -= sum(0.05 for kw in specific_keywords if kw in defn_text)
[L38]
[L39] # Factor 5: Hypernym detection (LLM-based, optional)
[L40] # If available, use LLM to check if node name is a hypernym
[L41] # score += hypernym_llm_score(node) # Expensive, use sparingly
[L42]
[L43] return max(0.0, min(score, 1.0)) # Clamp to [0, 1]
[L44]

```

```

[L45]
[L46] ---
[L47]
[L48] ### 3.6 CRITICAL:  $O(N^2)$  Canonicalization (PDF 2)
[L49]
[L50] The Bug: Alias merging compares every node pair: `for A in nodes: for B in nodes: →  $O(N^2)$ . For mi
[L51]
[L52] Fix: Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L53]
[L54]

```

python

```

[L55] from datasketch import MinHashLSH
[L56]
[L57] def efficient_canonicalization(nodes, threshold=0.85):
[L58]     lsh = MinHashLSH(threshold=threshold, num_perm=128)
[L59]
[L60]     # Index all nodes by their text signature
[L61]     for node in nodes:
[L62]         mh = create_minhash(node.name + " " + node.definition)
[L63]         lsh.insert(node.id, mh)
[L64]
[L65]     # Query each node for near-duplicates (sub-linear time)
[L66]     merge_groups = UnionFind()
[L67]     for node in nodes:
[L68]         mh = create_minhash(node.name + " " + node.definition)
[L69]         candidates = lsh.query(mh) #  $O(1)$  average

```

```
[L70] for candidate_id in candidates:  
[L71] if candidate_id != node.id:  
[L72] merge_groups.union(node.id, candidate_id)  
[L73]  
[L74] return merge_groups.groups()  
[L75]
```

```

[L76]
[L77] ---
[L78]
[L79] ### 3.7 Other Errors Found & Fixed
[L80]
[L81] | # | Error | Source PDF | Severity | Fix |
[L82] |---|-----|-----|-----|-----|
[L83] | 7 | PRE formula inconsistency (3 different versions across PDFs) | PDFs 1,2,3 | High | Standardize t
[L84] | 8 | HV depth check uses `depth(N)` instead of `depth(P)+1` | PDF 3 | High | Change to `if depth(P) +
[L85] | 9 | Cross-link cycle check uses hierarchy validator | PDF 2 | High | Use separate `is_valid_crosslin
[L86] | 10 | Forest Density metric inverted (`nodes/edges` instead of `edges/nodes`) | PDF 3 | Medium | Fix
[L87] | 11 | Forest Connectivity undefined without start vertex | PDF 3 | Medium | Define as `FC = |largest_
[L88] | 12 | `base_confidence` undefined in confidence propagation | PDF 2 | Medium | Define as `base_confic
[L89] | 13 | Token budget uses `len(text)` instead of tokenizer | PDF 4 | Medium | Use `tiktoken.encoding_fc
[L90] | 14 | `block.text.contains("def ")` falsely matches "definitely" | PDF 4 | Low | Use regex: `re.match
[L91] | 15 | Definition regex `"(^|(?!(a|an|the))?.*)"` too broad | PDF 4 | Medium | Add NLP dependency p
[L92] | 16 | Embedding averaging collapses to mean | PDF 4 | High | Use max-pooling or weighted attention ov
[L93] | 17 | Symmetric embedding model used for asymmetric QA retrieval | PDF 4 | High | Use asymmetric mode
[L94] | 18 | Alias resolution at 98% embedding similarity merges antonyms | PDF 5 | High | Add semantic role
[L95] | 19 | Timeline: Gantt previously showed 11 months, text said 8 months (now corrected) | PDF 1 | Low |
[L96] | 20 | Schema mismatch: `Page.regions` type differs between Layer 0 and Layer 2 | PDF 4 | Medium | Sta
[L97] | 21 | `ExecutionPrompt` object used but never defined | PDF 4 | Medium | Add schema definition in Lay
[L98] | 22 | No database synchronization protocol for hybrid storage | PDF 5 | High | Add saga pattern or 2-
[L99] | 23 | Retrieval sort by utility ignores token cost | PDF 3 | High | Sort by `utility / token_cost` as
[L100] | 24 | `tieBreak(P, bestParent)` crashes when `bestParent` is null | PDF 3 | Medium | Guard: `if best
[L101]
[L102] ---
[L103]
[L104] ## 4. Cross-Document Inconsistencies
[L105]
[L106] > [!WARNING]
[L107] > The 6 PDFs evolved iteratively but were never reconciled. Below are the major contradictions.
[L108]
[L109] ### 4.1 PRE Scoring Formula (3 Versions!)
[L110]
[L111] | PDF | Formula |
[L112] |-----|-----|
[L113] | PDF 1 (first Pdf) | `Score = \alpha \cdot Sim + \beta \cdot Isa + \gamma \cdot Evidence + \delta \cdot General - \zeta \cdot Conflict` |
[L114] | PDF 2 (third Pdf) | `Score = \alpha \cdot sim + \beta \cdot depthPenalty + \gamma \cdot C_p` (3 terms) then later `w_1 \cdot cos + w_2 \cdot worc
[L115] | PDF 3 (4h pdf) | Energy minimization: `E(P,N) = \lambda_1 \cdot L_{sem} + \lambda_2 \cdot L_{depth} + \lambda_3 \cdot L_{conf} + \lambda_4 \cdot L_{onto}` |
[L116]
[L117] **Resolution:** Unify into a single canonical formula:
[L118]
[L119] 
$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max}}{D_{\max} + 1} - \text{tex}$$

[L120]
[L121] Where  $\sum_{i=1}^5 w_i = 1$ , defaults:  $w_1=0.30, w_2=0.25, w_3=0.15, w_4=0.20, w_5=0.10$ ,  $D_{\max}$ 
[L122]
[L123] ### 4.2 Architecture: 13 Modules vs 8 Layers
[L124]
[L125] | PDF | Architecture Style |
[L126] |-----|-----|
[L127] | PDFs 1, 5 | 13 sequential modules (Modules 1-13) |
[L128] | PDFs 4, 6 | 8 architectural layers (Layers 0-8) |
[L129]
[L130] **Resolution:** The 8-layer model is more mature. Map modules into layers:
[L131]
[L132] | Layer | Modules Contained |
[L133] |-----|-----|
[L134] | Layer 0: Data Model | Core schemas |
[L135] | Layer 1: Input Processing | Module 1 (Document Parser) |
[L136] | Layer 2: Document Parsing | Module 2 (Structure Analyzer) |
[L137] | Layer 3: Knowledge Extraction | Modules 3-4 (Extractor + Normalizer) |
[L138] | Layer 4: Relation Management | Module 5 (Relation Discovery) + Module 6 (KCE) |
[L139] | Layer 5: SHIA Core | Module 7 (Hierarchy Builder: CPG++, PRE, HV, FI, CLD) |
[L140] | Layer 6: Retrieval Engine | Modules 8-10 (Query Analyzer + Retrieval + Context Assembly) |
[L141] | Layer 7: Generation Engine | Module 11 (LLM Generator) + Module 12 (Citation Verifier) |
[L142] | Layer 8: Operations | Logging, monitoring, eval + Module 13 (Evaluation/SHEF) |
[L143]

```

```

[L144] ### 4.3 Tree vs DAG vs Graph
[L145]
[L146] | PDF | Claim |
[L147] |-----|-----|
[L148] | All PDFs | "Knowledge Forest is acyclic (no loops)" |
[L149] | All PDFs | "Cross-Link Discovery (CLD) adds semantic associations" |
[L150]
[L151] **Contradiction:** Cross-links create graph cycles. The forest is only acyclic in its **hierarchical
[L152]
[L153] **Resolution:** Define two distinct edge types:
[L154] 1. **Hierarchical edges** (IS_A, PART_OF) – must be acyclic, form a forest
[L155] 2. **Semantic edges** (USES, RELATED_TO, CAUSES) – may form cycles, separate graph layer
[L156]
[L157] > [!IMPORTANT]
[L158] > **Single-Parent Constraint:** SHIA-RAG enforces **strict single-parent trees** for hierarchical edges
[L159] > - Precedence constraints in Tree Knapsack:  $x_i \leq x_{\text{parent}(i)}$  is unambiguous
[L160] > - Confidence propagation: single parent path, no need for multi-parent averaging
[L161] > - Cycle detection: simple ancestor traversal
[L162] >
[L163] > Multiple semantic associations are handled via `SEMANTIC` edges (cross-links), which are unrestricted
[L164]
[L165]

```

python

```

[L166] # Explicit enforcement of single-parent constraint
[L167] def validate_single_parent(node, new_parent, forest):
[L168] """Ensure node has at most one hierarchical parent."""
[L169] existing_parents = [
[L170] e for e in forest.get_incoming_edges(node)
[L171] if e.edge_class == "HIERARCHICAL"
[L172] ]
[L173] if existing_parents:
[L174] raise ValueError(
[L175] f"Node {node.id} already has parent via edge {existing_parents[0].edge_id}. "
[L176] f"SHIA-RAG enforces single-parent trees. Use SEMANTIC edges for additional associations."
[L177] )
[L178]

```



```

[L179]
[L180] ---
[L181]
[L182] ## 5. Novelty Contributions
[L183]
[L184] > [!IMPORTANT]
[L185] > These are 5 novel contributions that make SHIA-RAG publishable and distinguish it from all existing
[L186]
[L187] ### Novelty 1: Precedence-Constrained Tree Knapsack for Context Selection
[L188]
[L189] What exists: All current RAG systems use flat top-k selection or simple greedy knapsack for context
[L190]
[L191] What SHIA-RAG proposes: A greedy approximation for the Tree Knapsack formulation where selecting
[L192]
[L193] 
$$\max \sum_{i=1}^n U_i \cdot x_i \quad \text{s.t.} \quad \sum_{i=1}^n C_i \cdot x_i \leq B, \quad x_i \in \{0, 1\}$$

[L194]
[L195] > [!NOTE]
[L196] > The implementation uses a greedy utility-density heuristic rather than exact DP, as exact solution is
[L197]
[L198] Why novel: No RAG paper has formulated context selection as a precedence-constrained optimization problem
[L199]
[L200] ---
[L201]
[L202] ### Novelty 2: Self-Evolving Knowledge Forest via Retrieval Feedback
[L203]
[L204] What exists: All current Knowledge Graphs and hierarchies are static after construction.
[L205]
[L206] What SHIA-RAG proposes: The forest learns from user queries. When a retrieval succeeds (user provides
[L207]
[L208] > [!NOTE]
[L209] > Unified formula: The update rule uses the exponential multiplicative update  $w \cdot \exp(\eta \cdot \text{feedback})$ 
[L210]
[L211] > [!IMPORTANT]
[L212] > Traversal path definition: traversal_path(retrieved_nodes) returns only edges along paths from root to
[L213]
[L214]

```

python

```

[L215] import math
[L216]
[L217] ETA = 0.02 # Learning rate
[L218] WEIGHT_MIN = 0.1 # Minimum edge weight (prevents zero-out)
[L219] WEIGHT_MAX = 10.0 # Maximum edge weight (prevents runaway)
[L220] REBALANCE_INTERVAL = 100 # Queries between rebalance cycles
[L221] RESTRUCTURE_THRESHOLD = 0.3 # Weight below which edge is flagged
[L222]
[L223] def update_forest_from_feedback(query, retrieved_nodes, selected_nodes, user_feedback, forest):
[L224]     """Update edge weights along the retrieval path based on user feedback.
[L225]
[L226]     Args:
[L227]     query: The user query that triggered retrieval
[L228]     retrieved_nodes: All nodes visited during traversal
[L229]     selected_nodes: Nodes included in final context (subset of retrieved)
[L230]     user_feedback: +1 (HELPFUL) or -1 (NOT_HELPFUL)
[L231]     forest: The Knowledge Forest instance

```

Layer 2 — Document Parsing

Bilkul. Ab hum **Layer 2** ko bhi **Layer 0** aur **Layer 1** ki tarah **complete depth** me samjhenge.

Aur hum **wahi same Smart Hospital example** use karenge, taaki Layer 0 → Layer 1 → Layer 2 ka connection continuously clear rahe.

***Important:** Main is explanation ko report ke Layer 2 design ke basis par rakh raha hoon. Jahan report me koi detail explicitly defined nahi hai aur main technical analysis/additional observation de raha hoon, wahan main clearly mention karunga.*

1. Sabse pehle: Layer 2 actually karta kya hai?

Layer 1 ke end me hume kya mila tha?

```
Raw Document
↓
Layer 1
↓
NormalizedDocument
```

Ab problem ye hai ki `NormalizedDocument` me text normalized hai, **lekin document ka structure abhi knowledge system ke liye sufficiently explicit nahi hai.**

Example:

```
1. AI-Assisted Emergency Diagnosis

AI-assisted diagnosis uses machine learning models
to analyze patient symptoms and medical images.

The diagnostic pipeline consists of:

1. Data collection
2. Feature extraction
3. Risk prediction

Table 2: Emergency Risk Levels

| Level | Condition |
|-----|-----|
| High  | Immediate treatment |
| Low   | Routine monitoring |

Algorithm 1: Patient Risk Scoring
```

Layer 1 ke baad system ke paas largely normalized text/page content hai.

Lekin Layer 3 ko directly ye samajhne ke liye diya jaye:

Ye heading hai ya paragraph?

Ye list hai ya normal text?

Ye table hai?

`Algorithm 1` algorithm block hai ya simple sentence?

Ye kaam **Layer 2** karta hai.

Report ke according Layer 2 ki responsibility hai normalized text ko:

```
Page
↓
Region
↓
Block
```

hierarchy me parse karna, block types classify karna, aur reading order preserve karna. `filecite turn6file2 L170-L175`

2. Factory analogy

Agar poora SHIA-RAG ek factory hai:

```
Layer 0
= Rules aur data containers design karta hai

Layer 1
= Raw documents receive, validate aur normalize karta hai

Layer 2
= Document ko meaningful structural pieces me todta hai
```

Simple language me:

Layer 1 document ko readable banata hai. Layer 2 document ki body ko organize karta hai.

3. Layer 2 ka exact Input kya hai?

Input

```
NormalizedDocument
```

Ye directly Layer 1 se aata hai. Layer 2 ka report-defined input `NormalizedDocument` aur output `DocumentBlock` objects ki list hai. `filecite turn6file2 L170-L175`

Hamare example me Layer 1 se:

```
NormalizedDocument(  
  doc_id = "DOC_000004",  
  original_filename = "AI_Diagnostic_Module.pdf",  
  
  text = "...complete cleaned text...",  
  
  pages = [  
    "Page 1 text...",  
    "Page 2 text...",  
    "Page 3 text..."  
  ],  
  
  metadata = {...},  
  
  is_ocr = True,  
  ocr_confidence = 0.89,  
  
  language = "eng",  
  
  partial_extraction = True,  
  failed_pages = [87, 88]  
)
```

Ye poora object Layer 2 ko milega.

4. Layer 2 ka complete Input → Output flow

```
NormalizedDocument  
  |  
  ▼  
Page-wise processing  
  |  
  ▼  
Region Segmentation  
  |  
  ▼  
Block Segmentation  
  |  
  ▼  
Block Classification  
  |  
  ▼  
Reading Order Assignment  
  |  
  ▼  
DocumentBlock[]  
  |  
  ▼  
Layer 3
```

Report ke data flow me Layer 2 ka transformation explicitly:

```
Normalized Text + Metadata  
  ↓  
Detect layout  
  ↓  
Segment into blocks  
  ↓  
Classify types  
  ↓  
Reading order  
  ↓  
[Page → Region → Block] hierarchy
```

ke form me diya gaya hai. `filecite\turn6file0\L35-L44`

5. Hamara same common example

Hum ek **bada Smart Hospital document** lete hain.

Maan lo:

```
AI_Diagnostic_Module.pdf
```

me 120 pages hain.

Kuch sections:

```
Page 1
  Smart Hospital AI System

Page 2–10
  Introduction and Architecture

Page 11–30
  Emergency Diagnosis

Page 31–45
  Risk Prediction Models

Page 46–60
  Security and Access Control

Page 61–80
  Algorithms and System Workflow

Page 81–100
  Tables and Performance Results

Page 101–120
  Definitions, Limitations and References
```

Layer 2 ka goal:

```
120 pages ka document
      ↓
Thousands of structured blocks
```

Example:

```
BLK_000000 → Heading
BLK_000001 → Paragraph
BLK_000002 → Paragraph
BLK_000003 → List
BLK_000004 → Table
BLK_000005 → Algorithm
...
```

6. Layer 2 ka sabse important concept — Page → Region → Block

Isko bahut ache se samajhna zaroori hai.

Page

Document ka physical/logical page.

Example:

Page 15

Region

Ek page ke andar **contiguous content area**.

Maan lo page aisa hai:

3. Emergency Diagnosis

Patient symptoms are first...

Diagnostic model analyzes...

1. Collect symptoms

2. Run diagnostic model

3. Generate risk score

Table 4: Risk Classification

High | Immediate action

Low | Routine monitoring

← Region 0

← Region 1

← Region 2

← Region 3

Block

Region ke andar **atomic structural unit**.

Example:

Region 0

- Block: HEADING

Region 1

- Block: PARAGRAPH
- Block: PARAGRAPH

Region 2

- Block: LIST

Region 3

- Block: FIGURE_CAPTION / table caption
- Block: TABLE

Layer 2 ka final atomic output **DocumentBlock** hai. [filecite]turn6file2[L192-L203]

7. Layer 2 ka main data structure — DocumentBlock

Report ke according:

DocumentBlock

ke fields hain:

```
block_id
doc_id
page_number
region_index
block_type
text
reading_order
bounding_box
confidence
```

`bounding_box` PDF layout ke liye optional hai. `filecite` turn6file2 L192-L203

Chalo har field ko hamare example se samjhte hain.

7.1 `block_id`

```
BLK_000127
```

Har block ka unique identifier.

Example:

```
BLK_000000
BLK_000001
BLK_000002
```

Ye later evidence tracking ke liye important hai.

Suppose Layer 3 ne:

```
"Risk Prediction"
```

concept extract kiya.

To baad me hum track kar sakte hain:

```
Concept
  ↓
Source Block = BLK_000127
  ↓
Document = DOC_000004
  ↓
Page = 27
```

7.2 `doc_id`

```
DOC_000004
```

Ye Layer 1 se aata hai.

Isse pata chalta hai:

Ye block kis original document se belong karta hai?

7.3 `page_number`

Example:

```
page_number = 15
```

Ye provenance ke liye important hai.

Later answer:

"AI-assisted diagnosis requires doctor verification."

System trace kar sakta hai:

```
DOC_000004  
Page 15  
Block BLK_000127
```

7.4 region_index

Example:

```
region_index = 2
```

Meaning:

```
Page 15  
↓  
Region 2  
↓  
This block
```

Report me region ko page ke contiguous content area ke roop me process kiya gaya hai. [filecite]turn6file2[L205-L216]

7.5 block_type

Possible types:

```
HEADING  
PARAGRAPH  
LIST  
TABLE  
CODE  
ALGORITHM  
FIGURE_CAPTION  
EQUATION  
DEFINITION
```

Report me ye exact block taxonomy defined hai. [filecite]turn6file2[L181-L190]

7.6 text

Actual block content.

Example:

```
"AI-assisted diagnosis uses machine learning  
models to analyze patient symptoms and medical images."
```


7.7 reading_order

Example:

```
reading_order = 127
```

Ye **document ke andar global order** hai.

Important point:

```
Page 1, Block 0 → order 0
Page 1, Block 1 → order 1
Page 2, Block 0 → order 2
...
```

Report ke pseudocode me **global_order** initialize hota hai aur har new block ke baad increment hota hai.

filecite\turn6file2\L205-L234

7.8 bounding_box

PDF ke visual location ke liye:

```
{
  x0,
  y0,
  x1,
  y1
}
```

Example:

```
{
  x0: 72,
  y0: 120,
  x1: 530,
  y1: 250
}
```

Meaning:

Page par ye block exactly kahan tha.

Report me bounding box optional PDF field ke roop me defined hai. filecite\turn6file2\L192-L203

7.9 confidence

Block classification confidence.

Example:

```
Block:
"3. Emergency Diagnosis"

Detected:
HEADING

Confidence:
0.90
```

8. Layer 2 ka actual internal algorithm

Report ka main pseudocode conceptually ye hai:

```
for each page:
    segment page into regions

    for each region:
        split region into blocks

        for each block:
            classify block

            assign ID
            assign page number
            assign region index
            assign reading order
            assign confidence

return all blocks
```

Report ke actual Layer 2 pipeline me bhi isi sequence ko use kiya gaya hai. [filecite\turn6file2\L205-L234](#)

Ab hum ek-ek step deep me samjhenge.

9. Step 1 — Page Selection / Iteration

Code ka basic logic:

```
doc.pages
```

Agar pages available hain:

```
Page 1
Page 2
Page 3
...
```

Otherwise fallback:

```
[doc.text]
```

Report ka pseudocode:

```
for page_num, page_text in enumerate(doc.pages or [doc.text])
```

use karta hai. [filecite\turn6file2\L205-L213](#)

Hamare example me

```
DOC_000004

Page 1
Page 2
...
Page 120
```

Layer 2:

```
Start with Page 1
      ↓
Finish Page 1
      ↓
Page 2
      ↓
...
```

10. Step 2 — Region Segmentation

Code:

```
regions = segment_into_regions(page_text)
```

Report ke comment ke according:

Region = contiguous content area. filecite turn6file2 L210-L216

Simple example:

PAGE 15

3. EMERGENCY DIAGNOSIS

Patient data is collected from multiple hospital systems.

The diagnostic engine processes symptoms and medical images.

1. Collect patient data
2. Validate input
3. Generate risk score

TABLE 4: RISK LEVELS

High	Immediate action
Medium	Doctor review
Low	Routine monitoring

Potential regions:

```
Region 0
3. EMERGENCY DIAGNOSIS

Region 1
Patient data is collected...
The diagnostic engine processes...

Region 2
1. Collect patient data
2. Validate input
3. Generate risk score

Region 3
TABLE 4: RISK LEVELS
High ...
Medium ...
Low ...
```

11. Region aur Block me exact difference

Ye point bahut important hai.

Region = container area

```
Region
```

ke andar multiple content units ho sakte hain.

Block = actual atomic unit

Example:

```
Region 1
|
├─ Block A: Paragraph
|
└─ Block B: Paragraph
```

Visual:

```
Page
├─ Region 0
│   └─ Block 0
├─ Region 1
│   ├── Block 1
│   └─ Block 2
└─ Region 2
    ├── Block 3
    └─ Block 4
```

12. Step 3 — Region ko Blocks me split karna

Code:

```
raw_blocks = split_into_blocks(region)
```

Example Region:

AI diagnosis uses historical patient data to estimate disease probability.

The system does not replace doctors.
It provides decision support.

Ye 2 paragraphs ho sakte hain:

Block 1:
AI diagnosis uses historical patient data...

Block 2:
The system does not replace doctors...

13. Block segmentation kyun needed hai?

Agar poora page ek hi block bana diya:

Heading
+
Paragraph
+
List
+
Table
+
Algorithm

sab mix ho jayega.

Then Layer 3 ko ek massive mixed input milega:

"3. Emergency Diagnosis Patient data...
1. Collect...
Table 4..."

Result:

Context loss
Structural ambiguity
Wrong knowledge extraction

Isliye:

Mixed Page
↓
Atomic Blocks

14. Step 4 — Block Classification

Ab har raw block ke liye:

```
block_type, confidence = classify_block(raw_block)
```

Report ke according classification **rule-based heuristics** se hoti hai. `filecite\turn6file2\L236-L246`

Ab har type ko deep me samjhte hain.

15. Block Type 1 — HEADING

Example:

```
3. EMERGENCY DIAGNOSIS
```

Ya:

```
AI-ASSISTED RISK PREDICTION
```

Report ki heading heuristic:

```
Short line  
< 100 characters  
  
No ending period  
  
First character uppercase  
  
And:  
  
starts with '#'  
OR  
entire text uppercase
```

To classification:

```
HEADING  
confidence = 0.9
```

filecite\turn6file2\L236-L244

Hamare example me

```
EMERGENCY DIAGNOSIS
```

Check:

```
Length < 100      ✓  
Ends with "."? No ✓  
Starts uppercase  ✓  
All uppercase     ✓
```

Result:

```
BlockType.HEADING  
Confidence = 0.9
```

16. HEADING classification me possible mistake

Example:

```
CRITICAL PATIENT
```

Ye heading bhi ho sakta hai.

But:

```
CRITICAL PATIENT  
REQUIRES IMMEDIATE TREATMENT
```

document ke paragraph ka emphasized sentence bhi ho sakta hai.

Rule-based heuristic context nahi samajhta.

So possible:

```
False Positive
```

Report confidence store karti hai, jo downstream handling ke liye useful signal deta hai. `filecite\turn6file2\L192-L203`

17. Block Type 2 — LIST

Example:

```
1. Collect patient symptoms  
2. Validate patient history  
3. Run AI model
```

Ya:

```
- Fever  
- Blood pressure  
- Oxygen level
```

Ya:

```
• Emergency symptoms  
• Risk factors
```

Report ke rules me prefixes include hain:

```
-  
*  
•  
1.  
2.
```

`filecite\turn6file2\L245-L246`

Result:

```
BlockType.LIST
```

18. LIST ko paragraph kyu nahi banana?

Example:

```
1. Collect symptoms
2. Run diagnostic model
3. Verify with doctor
```

Isme **sequence** hai.

Agar paragraph bana diya:

```
Collect symptoms run diagnostic model verify...
```

To structural information lose ho sakti hai:

```
Step 1
Step 2
Step 3
```

Later Layer 3 ke liye list/sequence knowledge useful ho sakta hai.

19. Block Type 3 — TABLE

Example:

```
| Risk Level | Action |
|-----|-----|
| High      | Immediate treatment |
| Medium    | Doctor review |
| Low       | Routine monitoring |
```

Report ki Layer 2 classification notes me table ke liye pipe-character heuristic describe ki gayi hai.

filecite turn6file5 L470-L476

Agar:

```
3+ pipe characters
```

to table detect kiya ja sakta hai.

20. Table special kyun hai?

Table:

```
High → Immediate treatment
Low  → Routine monitoring
```

normal paragraph nahi hai.

Iska structure:

```
Rows
+
Columns
+
Cell relationships
```


Report ke newer Layer 2 description me table blocks ke liye cell-level parsing bhi described hai. `TABLE` block ke case me `table_cells` preserve kiye ja sakte hain, aur PDF tables ke liye `pdfplumber` / `Camelot` reconstruction use karne ki baat hai. `filecite` turn6file5L456-L480

21. Table ka detailed example

Input:

```
| Risk Score | Category | Action |
|-----|-----|-----|
| 0-30      | Low      | Monitor |
| 31-70     | Medium   | Review  |
| 71-100    | High     | Immediate treatment |
```

Cell-level representation:

```
table_cells = [
    ["Risk Score", "Category", "Action"],
    ["0-30", "Low", "Monitor"],
    ["31-70", "Medium", "Review"],
    ["71-100", "High", "Immediate treatment"]
]
```

Why important?

Agar table flatten ho jaye:

```
Risk Score Category Action 0-30 Low Monitor...
```

to relationships confuse ho sakte hain.

Cell structure preserve karne se:

```
71-100
  ↓
High
  ↓
Immediate treatment
```

relationship clear rehta hai.

22. Table detection me possible error

Har `|` wala text table nahi hota.

Example:

```
A | B | C represents three variables.
```

Heuristic:

```
Many pipes
→ TABLE
```

False positive possible.

Isliye report ka rule-based approach fast hai, lekin perfect semantic understanding nahi deta. Ye meri technical observation hai; report classification ko rule-based heuristics ke roop me define karti hai. [filecite\turn6file5\L470-L476](#)

23. Block Type 4 — CODE

Hamare Smart Hospital project me code example:

```
def calculate_risk(symptoms):
    score = model.predict(symptoms)
    return score
```

Report code detection ke liye regex-based matching use karne ki correction mention karti hai:

```
^(def |class |import |from )
```

instead of simply checking whether `"def "` occurs anywhere. [filecite\turn9file0\L79-L100](#)

24. Ye correction kyu necessary thi?

Broken logic:

```
if "def " in text:
    CODE
```

Problem:

```
"This definitely improves diagnosis."
```

`definitely` me `"def"` pattern aa sakta hai.

False code classification.

Correct approach:

Line beginning check:

```
def
class
import
from
```

So:

```
def calculate_risk():
```

→ CODE ✓

```
This definitely improves...
```

→ CODE ✗

Report ke error-fix section me exactly isi false-match problem aur regex fix ko identify kiya gaya hai.

[filecite\turn9file0\L89-L91](#)

25. Block Type 5 — ALGORITHM

Example:

```
Algorithm 1: Emergency Risk Scoring

Input:
Patient symptoms S

1. Validate S
2. Extract features F
3. Compute risk score R
4. Return R
```

Layer 2 ka kaam yahan:

| *Is structure ko algorithmic content ke roop me identify karna.*

Ye important hai because:

```
Algorithm
```

sirf normal paragraph nahi hota.

Isme:

```
Input
Steps
Sequence
Output
```

ho sakte hain.

Report ke BlockType taxonomy me ALGORITHM explicitly included hai. `filecite{turn6file2}{L181-L190}`

However: report ke shown classification snippet me algorithm detection ka detailed rule explicitly visible nahi hai. Isliye exact detection heuristic ko invent nahi karenge.

26. Block Type 6 — FIGURE_CAPTION

Example:

```
Figure 8: Architecture of the AI-assisted emergency diagnosis pipeline.
```

Ye paragraph nahi hai.

Ye ek figure ko describe karta hai.

Block type:

```
FIGURE_CAPTION
```

Report ke taxonomy me explicitly defined hai. `filecite{turn6file2}{L181-L190}`

27. Figure caption ko separate block kyun?

Later Layer 3 me report ke shown pipeline me:

HEADING
FIGURE_CAPTION

jaise non-content blocks skip kiye ja sakte hain during primary knowledge extraction. [filecite\turn6file7\L640-L647](#)

Important distinction:

Caption preserve karna
≠
Usse primary concept extraction karna

28. Block Type 7 — EQUATION

Example:

Risk Score = 0.4 × SymptomScore
 + 0.3 × MedicalHistoryScore
 + 0.3 × ImageScore

Block:

EQUATION

Report taxonomy me equation explicitly included hai. [filecite\turn6file2\L181-L190](#)

Why separate?

Because equation ka structure:

variables
operators
mathematical relation

hota hai.

Normal paragraph me convert karne se mathematical semantics damage ho sakti hai.

29. Block Type 8 — DEFINITION

Example:

AI-assisted diagnosis is defined as the use of
artificial intelligence techniques to support clinical
decision-making.

Report definition detection ke liye phrases jaise:

"is defined as"
"refers to"

use karne ka mention karti hai. [filecite\turn6file5\L470-L476](#)

Result:

BlockType.DEFINITION

30. Definition detection me major possible mistake

Broken rule kuch aisa ho sakta tha:

```
X is Y  
→ DEFINITION
```

Problem:

```
"The patient is stable."
```

Ye definition nahi hai.

Report ke error analysis me definition regex ko **too broad** bataya gaya hai aur stronger NLP dependency parsing ki recommendation di gayi hai—subject noun phrase hona chahiye aur `"is"` copular relation ke role me validate hona chahiye. `filecite[turn9file0][L89-L92]`

Ye Layer 2/3 boundary par important correction hai.

31. Block Type 9 — PARAGRAPH

Agar koi specific rule match nahi karta:

```
Default → PARAGRAPH
```

Report classification summary me paragraph default type hai. `filecite[turn6file5][L470-L476]`

Example:

```
The emergency diagnosis engine receives patient data  
from multiple sources and produces a preliminary risk  
assessment for clinical review.
```

Result:

```
BlockType.PARAGRAPH
```

32. Complete classification example

Page 15:

3. EMERGENCY DIAGNOSIS

The system receives patient symptoms from the hospital information system.

The diagnostic workflow consists of:

1. Data collection
2. Feature extraction
3. Risk prediction

Score	Action
High	Immediate treatment

```
def calculate_risk(data):
    return model.predict(data)
```

AI-assisted diagnosis is defined as the use of AI to support clinical decision-making.

Layer 2 output:

```
BLK_000120
→ HEADING

BLK_000121
→ PARAGRAPH

BLK_000122
→ LIST

BLK_000123
→ TABLE

BLK_000124
→ CODE

BLK_000125
→ DEFINITION
```

33. Step 5 — Reading Order assign karna

Ye Layer 2 ka **bahut important part** hai.

Suppose document me blocks hain:

```
BLK_000120 → Heading
BLK_000121 → Paragraph
BLK_000122 → List
BLK_000123 → Table
```

Reading order:

```
120 → 121 → 122 → 123
```

Report ka pseudocode:

```
global_order = 0
```

se start karta hai aur har block banne ke baad:

```
global_order += 1
```

karta hai. `filecite`turn6file2L205-L234

34. Reading order kyun important hai?

Suppose:

```
Heading:
Emergency Diagnosis

Paragraph:
The model calculates patient risk.

List:
1. Collect symptoms
2. Extract features
3. Predict risk
```

Agar order lose ho gaya:

```
List
Heading
Paragraph
```

to document ka flow broken.

Layer 3 ko context milega:

```
1. Collect symptoms
2. Extract features

...some unrelated content...

Emergency Diagnosis
```

Semantic interpretation affect ho sakti hai.

35. Multi-column document problem

Yahan ek important practical case hai.

Suppose PDF:

LEFT COLUMN	RIGHT COLUMN
Paragraph A	Paragraph C
Paragraph B	Paragraph D

Correct reading order:

```
A
B
C
D
```

Wrong geometric extraction ho sakta hai:

A
C
B
D

Isliye report Layer 2 me:

```
Layout analysis
+
Region segmentation
+
Reading order
```

ko explicitly responsibility banati hai. [filecite\turn6file0\L38-L44](#)

But exact multi-column ordering algorithm shown Layer 2 pseudocode me detailed nahi hai. Ye report ka current implementation gap/unclear area hai.

36. Layer 2 ka complete pseudocode ko simple language me

Report ka conceptual logic:

```
blocks = []
global_order = 0

FOR every page:

    regions = segment page

    FOR every region:

        raw_blocks = split region

        FOR every raw block:

            type = classify block

            create DocumentBlock

            reading_order = global_order

            global_order += 1

RETURN blocks
```

Ye report ke Layer 2 pseudocode ka direct structural flow hai. [filecite\turn6file2\L205-L234](#)

37. Hamare Smart Hospital example ka full Layer 2 walkthrough

Chalo ek actual page process karte hain.

Layer 1 se input

```
NormalizedDocument

doc_id = DOC_000004
page_number = 15
```

Page 15:

3. EMERGENCY DIAGNOSIS

The emergency system collects symptoms, vital signs, medical history and diagnostic images from multiple hospital services.

The diagnostic workflow consists of:

1. Data Collection

2. Feature Extraction

3. Risk Prediction

4. Doctor Verification

Table 4: Risk Levels

Score	Category	Action
0-30	Low	Monitor
31-70	Medium	Review
71-100	High	Immediate Treatment

Algorithm 1: Risk Prediction

Step A — Page ko regions me divide karo

Potential output:

Region 0
Heading area

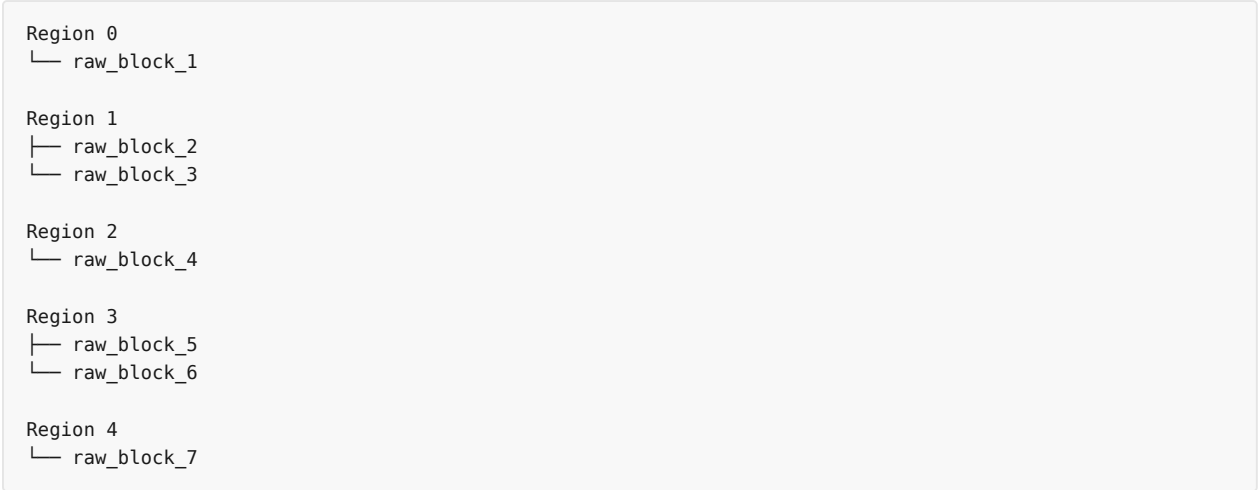
Region 1
Paragraph area

Region 2
List area

Region 3
Table area

Region 4
Algorithm area

Step B — Har region ko blocks me split karo



Step C — Classification

raw_block_1

"3. EMERGENCY DIAGNOSIS"

→ HEADING

confidence = 0.90

raw_block_2

"The emergency system collects symptoms..."

→ PARAGRAPH

raw_block_4

1. Data Collection

2. Feature Extraction

3. Risk Prediction

→ LIST

raw_block_6

Score	Category	Action
-------	----------	--------

→ TABLE

raw_block_7

Algorithm 1: Risk Prediction

→ ALGORITHM

Step D — IDs assign karo

```

BLK_000120
BLK_000121
BLK_000122
BLK_000123
BLK_000124
...

```

Step E — Reading order

```

BLK_000120 → 120
BLK_000121 → 121
BLK_000122 → 122
BLK_000123 → 123
BLK_000124 → 124

```

38. Final `DocumentBlock[]` output

Layer 2 output:

```

[
  DocumentBlock(
    block_id = "BLK_000120",
    doc_id = "DOC_000004",
    page_number = 15,
    region_index = 0,
    block_type = HEADING,
    text = "3. EMERGENCY DIAGNOSIS",
    reading_order = 120,
    confidence = 0.90
  ),

  DocumentBlock(
    block_id = "BLK_000121",
    doc_id = "DOC_000004",
    page_number = 15,
    region_index = 1,
    block_type = PARAGRAPH,
    text = "The emergency system collects...",
    reading_order = 121
  ),

  DocumentBlock(
    block_id = "BLK_000122",
    doc_id = "DOC_000004",
    page_number = 15,
    region_index = 2,
    block_type = LIST,
    text = "1. Data Collection...",
    reading_order = 122
  ),

  ...
]

```

39. Layer 2 me algorithms kya use ho rahe hain?

Report-defined techniques

- | Step | Technique |
- |---|---|
- | Page processing | Sequential iteration |
- | Region creation | Contiguous content area segmentation |
- | Block splitting | Structural segmentation |
- | Heading detection | Rule-based heuristic |
- | List detection | Prefix pattern matching |
- | Table detection | Pipe/structure heuristic |
- | Code detection | Regex pattern |
- | Definition detection | Phrase/rule heuristic |
- | Fallback | Paragraph |
- | Reading order | Global sequential counter |
- | Table structure | Cell-level reconstruction where supported |

Ye Layer 2 ke report-described structure aur rules se derived summary hai. [filecite\turn6file2\L181-L246](#)
[filecite\turn6file5\L470-L480](#)

40. Layer 2 me possible errors

Ab hum Layer 1 ki tarah complete error analysis karte hain.

Error 1 — Heading ko paragraph samajhna

Input:

Emergency Diagnosis

Agar heading rule fail:

PARAGRAPH ❌

Impact:

Later:

Section context lost

Error 2 — Paragraph ko heading samajhna

Input:

PATIENT SAFETY REQUIRES CONTINUOUS MONITORING

Ye emphasized sentence ho sakta hai.

But:

All uppercase
+
short

→ HEADING false positive.

Error 3 — List split ho jana

Original:

- ```
1. Collect symptoms
2. Validate history
3. Run prediction
```

Bad segmentation:

```
Block A:
1. Collect symptoms

Block B:
2. Validate history

Block C:
3. Run prediction
```

Kabhi-kabhi ye acceptable ho sakta hai, but agar list ek logical procedure hai to uski group structure break ho sakti hai.

---

### Error 4 — Table ko paragraph banana

Input:

```
Score Category Action
0-30 Low Monitor
```

PDF extraction me column boundaries lost ho jaye.

Result:

PARAGRAPH ❌

Then:

```
0-30
Low
Monitor
```

ke exact row relationship lose ho sakte hain.

---

### Error 5 — Non-table ko table samajhna

Example:

```
Risk | Severity | Action
```

Pipe heuristic se table detect ho sakta hai, even if it is mathematical/text notation.

False positive possible.

---

### Error 6 — Code misclassification

Report ne specifically ek bug identify kiya:

```
"definitely"
```

ko naive "def " detection se code samajhne ka risk.

Fix:

```
Regex anchored at line beginning
```

filecite\turn9file0\L89-L91

## Error 7 — Definition misclassification

Example:

```
"The patient is stable."
```

Naive:

```
X is Y
→ DEFINITION ❌
```

Report ne broad definition regex ko error bataya aur stronger syntactic validation recommend ki.

filecite\turn9file0\L89-L92

## Error 8 — Wrong reading order

Especially:

```
Multi-column PDF
Tables
Floating text
Sidebars
```

Wrong order:

```
A → C → B → D
```

instead of:

```
A → B → C → D
```

Impact:

```
Context corruption
↓
Wrong concept extraction
↓
Wrong relations
↓
Wrong hierarchy
```

## 41. Layer 2 me confidence kyu hai?

Suppose:

```
Block:
"Emergency Diagnosis"
```

System:

```
HEADING
confidence = 0.90
```

Another:

```
"High | Medium | Low"
```

System unsure:

```
TABLE
confidence = 0.55
```

Confidence future safeguards ke liye useful metadata hai.

Important:

*Block type prediction ko absolute truth treat nahi karna chahiye.*

Report `DocumentBlock` me confidence field explicitly store karti hai. `filecite\turn6file2\L192-L203`

---

## 42. Exception handling — Report me kya explicitly defined hai?

Yahan mujhe source aur technical recommendation ko separate rakhna hoga.

### Report explicitly Layer 2 pipeline me

Shown pseudocode primarily:

```
segment
→ split
→ classify
→ create block
→ return
```

dikhata hai. `filecite\turn6file2\L205-L234`

### Lekin shown Layer 2 section me

Har failure ke liye detailed explicit exceptions, jaise:

```
RegionSegmentationError
BlockClassificationError
ReadingOrderError
```

defined nahi dikhte.

Isliye ye exact exception classes main report ke naam se claim nahi karunga.

---

## 43. Recommended exception handling

Ye mera technical design recommendation hai, report ka explicitly defined code nahi.

### Case A — Region segmentation fail

```
try:
 regions = segment_into_regions(page)
except:
 regions = [entire_page_as_single_region]
 log_warning()
```

Why?

Page completely drop nahi hoga.

### Case B — Block classification fail

```
try:
 type = classify(block)
except:
 type = PARAGRAPH
 confidence = low
```

Safe fallback:

Unknown content  
→ preserve as paragraph

Better than:

Delete block ❌

### Case C — Table reconstruction fail

```
Try:
pdfplumber

Fail:
Camelot

Still fail:
Preserve raw text
+
low confidence
```

**Note:** pdfplumber / Camelot table reconstruction report me mentioned hai; this exact fallback chain is my recommended handling. [filecite.com/turn6file5/L478-L486](https://filecite.com/turn6file5/L478-L486)

### Case D — Empty block

```
raw_block.strip() == ""
```

Recommended:

Skip

Otherwise meaningless blocks:



```
BLK_000123 = ""
```

banenge.

## 44. Layer 2 ke safeguards

Layer 2 ka core principle hona chahiye:

```
Never silently lose content.
```

Recommended safe behavior:

```
Classification uncertain
→ Preserve text
→ Lower confidence

Table parse fails
→ Preserve raw table text

Region detection fails
→ Preserve whole page

Unknown type
→ PARAGRAPH fallback
```

Ye **technical safeguards** hain. Report me `DocumentBlock` confidence aur structural preservation ka design defined hai, lekin in exact fallback policies ka complete implementation shown Layer 2 pseudocode me detailed nahi hai. `filecite\turn6file2\L192-L234`

## 45. Layer 2 me report ki missing/unclear parts

Ab important technical review.

### Missing Part 1 — `segment_into_regions()` ka exact algorithm

Report call karti hai:

```
segment_into_regions(page_text)
```

Lekin shown pseudocode me exact algorithm define nahi hai:

```
Whitespace-based?
Coordinates?
XY-cut?
ML layout model?
PDF geometry?
```

Isliye exact region segmentation method currently unclear hai. `filecite\turn6file2\L210-L216`

### Missing Part 2 — `split_into_blocks()` ka exact rule

Function present hai:

```
split_into_blocks(region)
```

But exact splitting logic shown snippet me fully specified nahi hai.

Possible ambiguity:

```
Blank line?
Font change?
Indentation?
Coordinates?
Semantic boundaries?
```

Report snippet is point ko exact algorithmically complete define nahi karti. [filecite\turn6file2\L214-L220](#)

---

### Missing Part 3 — Complex reading-order algorithm

Global counter:

```
0
1
2
3
```

assign karna easy hai.

Hard problem:

*Correct order determine kaise hua?*

Especially:

```
Two columns
Sidebars
Footnotes
Floating tables
Figures
```

Report reading order ko requirement/output ke roop me clearly specify karti hai, but shown Layer 2 pseudocode me complex layout ordering algorithm detail nahi hai. [filecite\turn6file0\L38-L44](#)

---

### Missing Part 4 — Figure/image semantic content

`FIGURE_CAPTION` type hai. [filecite\turn6file2\L181-L190](#)

Lekin figure ke actual image content:

```
Medical architecture diagram
Flowchart
Graph
Chart
```

ko Layer 2 visual understanding se kaise represent karegi, ye shown parsing pseudocode me explicitly defined nahi hai.

---

### Missing Part 5 — Confidence calibration

Report:

```
confidence = float
```

store karti hai. `filecite\turn6file2\L192-L203`

Lekin:

```
0.90 ka exact meaning?
0.60 threshold?
Low confidence par retry?
Manual review?
```

ye shown Layer 2 implementation me fully defined nahi hai.

---

## 46. Layer 2 ka sabse dangerous failure cascade

Suppose:

```
Layer 2
TABLE ko PARAGRAPH bana deta hai
```

Then:

```
Layer 3
Wrong concepts extract karega

Layer 4
Wrong relations validate karega

Layer 5
Wrong hierarchy build ho sakti hai

Layer 6
Wrong retrieval

Layer 7
Wrong answer
```

Isliye Layer 2:

*Knowledge extraction se pehle ka structural foundation hai.*

Report ke overall architecture me Layer 2 ka output directly Layer 3 ko jata hai, jahan `DocumentBlock[]` se concepts/definitions/relations extract hote hain. `filecite\turn6file0\L41-L50`

---

## 47. Layer 2 output Layer 3 ko exactly kaise jata hai?

Layer 2:

```
DocumentBlock[]
```

produce karta hai.

Example:

```
[
 BLK_000120 → HEADING,
 BLK_000121 → PARAGRAPH,
 BLK_000122 → LIST,
 BLK_000123 → TABLE,
 BLK_000124 → ALGORITHM
]
```

Layer 3:

```
for block in blocks:
```

process karega.

Report ke Layer 3 pipeline me kuch non-content blocks, including `HEADING` aur `FIGURE_CAPTION`, skip kiye gaye hain; remaining relevant blocks knowledge extraction ke liye use hote hain. `filecite_turn6file7L640-L655`

So:

```
Layer 2
 ↓
DocumentBlock[]
 ↓
Layer 3
 ↓
KnowledgeUnit extraction
```

## 48. Hamare example ka exact Layer 1 → Layer 2 → Layer 3 connection

### Layer 1 output

```
NormalizedDocument

DOC_000004
120 pages
Cleaned text
Metadata
OCR information
```

### Layer 2 processing

```
Page 15
 ↓
Region 0
 ↓
Heading Block

Region 1
 ↓
Paragraph Blocks

Region 2
 ↓
List Block

Region 3
 ↓
Table Block

Region 4
 ↓
Algorithm Block
```

### Layer 2 output

```
DocumentBlock[]
```

### Layer 3 input

Example:

```
BLK_000121
Type = PARAGRAPH

"The AI diagnostic engine analyzes patient
symptoms to estimate emergency risk."
```

Layer 3 is text se later concepts nikaalegi:

```
AI diagnostic engine
Patient symptoms
Emergency risk
```

But ye Layer 3 ka kaam hai, Layer 2 ka nahi.

### 49. Layer 2 kya nahi karta?

Ye bahut important distinction hai.

Layer 2:

```
"AI diagnosis"
```

ko dekhkar officially ye decide nahi karta:

```
Concept
Domain
Parent concept
Relation
Knowledge graph node
```

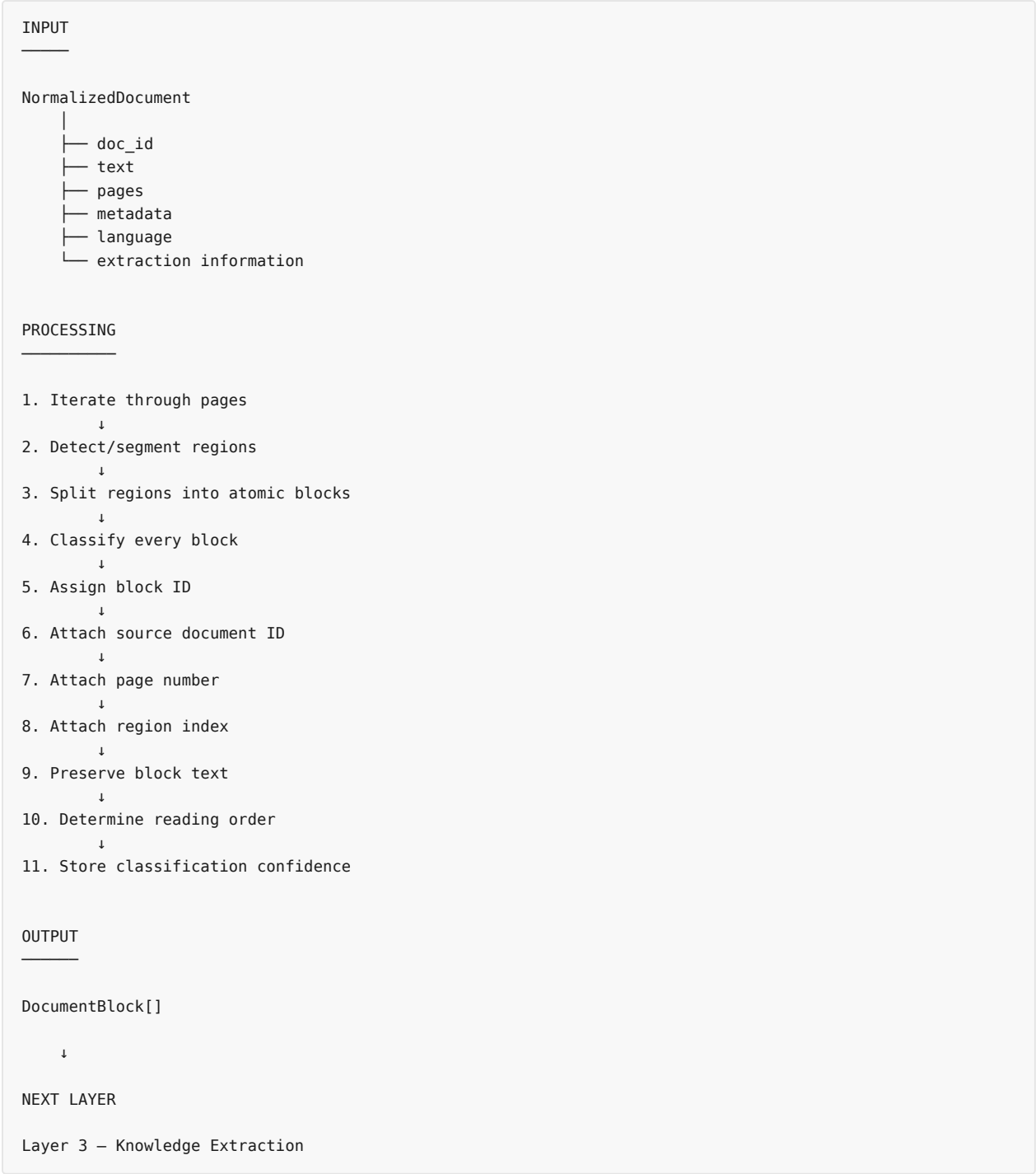
Ye sab abhi nahi.

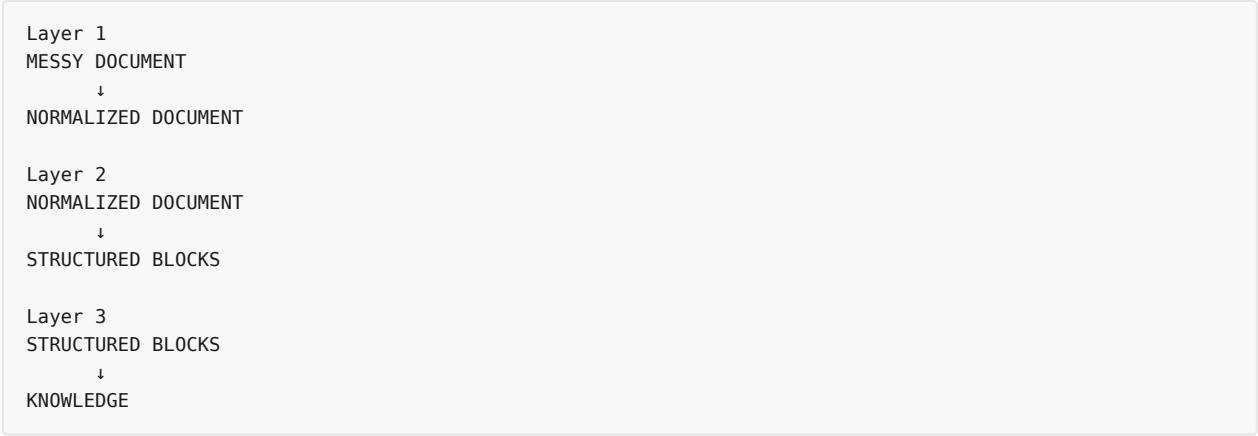
Layer 2 sirf:

```
Is content ka structure kya hai?
Heading?
Paragraph?
List?
Table?
Code?
Algorithm?
Definition?
```

decide karta hai.

### 50. Layer 2 ka final end-to-end summary





52. Final distinction between Layer 0, 1 and 2

| Layer | Main Question | Output |

---|---|---

| **Layer 0** | Data ko represent kaise karenge? | Schemas |

| **Layer 1** | Raw document ko safely kaise ingest/extract karenge? | `NormalizedDocument` |

| **Layer 2** | Document ke content ka structure kya hai? | `DocumentBlock[]` |

Layer 2 complete.

you 20/08/2026, 14:28:46

layer 3 smjhao

CHATGPT 20/08/2026, 14:28:52

```
{"paths":["files"],"query":"search"}
```

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite`turn10file0

- [L1] metadata:
- [L2] query\_result\_index: 0
- [L3] file\_id: file\_000000003c8081fab658b1752549d64
- [L4] version\_id: 1
- [L5] name: SHIA\_RAG\_Complete\_Report(5).md
- [L6] mime\_type: text/markdown
- [L7] surface: library
- [L8] score: 0.01821162444113264
- [L9] document\_chunk\_id: file\_000000003c8081fab658b1752549d64--1--34
- [L10] snippet:
- [L11] > [!IMPORTANT]
- [L12] > **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology, resolves conflicts, and assigns confidence scores. Layer 3 outputs raw `KnowledgeUnit` objects; Layer 4 produces scored `KnowledgeEdge` objects.
- [L13]

```

[L14] Input: List of DocumentBlock from Layer 2
[L15] Output: List of KnowledgeNode objects (canonical, deduplicated)
[L16]
[L17] ```python
[L18] from dataclasses import dataclass
[L19]
[L20] @dataclass
[L21] class KnowledgeUnit:
[L22] """Atomic fact extracted from a single document block.
[L23] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L24] unit_id: str
[L25] concept_name: str # Raw extracted concept name
[L26] aliases: list # Alternative names found in text
[L27] definitions: list # Definition sentences
[L28] properties: dict # Key-value properties (typed)
[L29] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]
[L30] source_doc: str
[L31] source_block: str
[L32] page: int
[L33] extraction_confidence: float # LLM extraction confidence
[L34] domain_hint: str # LLM-suggested domain
[L35]
[L36] # --- Extraction Pipeline ---
[L37]
[L38] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L39]
[L40] 1. Concepts: Named entities, technical terms, or ideas discussed
[L41] 2. Definitions: If the text defines any concept, extract the definition
[L42] 3. Properties: Key attributes with types and values
[L43] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L44] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")
[L45]
[L46] Respond in JSON format:
[L47] {
[L48] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."], "properties": {"key": {"type": "...", "value": "..."}}}],
[L49] "relations": [{"source": "...", "type": "IS_A|PART_OF|USES|CAUSES|...", "target": "...", "evidence": "..."}],
[L50] "domain": "...
[L51] }
[L52]
[L53] TEXT BLOCK:
[L54] {text}
[L55] """
[L56]
[L57] def layer3_extract(blocks: list, llm_client) -> list:
[L58] """Layer 3: Knowledge Extraction pipeline."""

```



```

[L59] knowledge_units = []
[L60]
[L61] for block in blocks:
[L62] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L63] continue # Skip non-content blocks
[L64]
[L65] # Step 1: LLM-based extraction
[L66] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L67] try:
[L68] response = llm_client.query(prompt, response_format="json")
[L69] extracted = json.loads(response)
[L70] except (json.JSONDecodeError, LLMLTimeout) as e:
[L71] # Fallback: rule-based extraction
[L72] extracted = rule_based_extract(block.text)
[L73]
[L74] # Step 2: Create KnowledgeUnit per concept
[L75] for concept in extracted.get('concepts', []):
[L76] unit = KnowledgeUnit(
[L77] unit_id=f"KU_{len(knowledge_units):06d}",
[L78] concept_name=concept['name'],
[L79] aliases=concept.get('aliases', []),
[L80] definitions=concept.get('definitions', []),
[L81] properties=concept.get('properties', {}),
[L82] raw_relations=[r for r in extracted.get('relations', [])
[L83] if r['source'] == concept['name']],
[L84] source_doc=block.doc_id,
[L85] source_block=block.block_id,
[L86] page=block.page_number,

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite[turn10file1]`

```

[L87] metadata:
[L88] query_result_index: 1
[L89] file_id: file_0000000033088207b45d2ae58cbd406c
[L90] version_id: 1
[L91] name: SHIA_RAG_Complete_Report(8).md
[L92] mime_type: text/markdown
[L93] surface: library
[L94] score: 0.017947214076246335
[L95] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--34
[L96] snippet:
[L97] > [!IMPORTANT]
[L98] > Layer 3 / Layer 4 boundary: Layer 3 performs candidate extraction — it identifies entities, definitions, and
raw relation triples from text. Layer 4 performs validation and scoring — it validates extracted relations against the
ontology, resolves conflicts, and assigns confidence scores. Layer 3 outputs raw KnowledgeUnit objects; Layer 4

```

produces scored `KnowledgeEdge` objects.

[L99]

[L100] **Input:** List of `DocumentBlock` from Layer 2

[L101] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)

[L102]

[L103] ```python

[L104] from dataclasses import dataclass

[L105]

[L106] @dataclass

[L107] class KnowledgeUnit:

[L108] """Atomic fact extracted from a single document block.

[L109] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""

[L110] unit\_id: str

[L111] concept\_name: str # Raw extracted concept name

[L112] aliases: list # Alternative names found in text

[L113] definitions: list # Definition sentences

[L114] properties: dict # Key-value properties (typed)

[L115] raw\_relations: list # Candidate relations: [(relation\_type, target\_concept, evidence\_text)]

[L116] source\_doc: str

[L117] source\_block: str

[L118] page: int

[L119] extraction\_confidence: float # LLM extraction confidence

[L120] domain\_hint: str # LLM-suggested domain

[L121]

[L122] # --- Extraction Pipeline ---

[L123]

[L124] EXTRACTION\_PROMPT = """You are a knowledge extraction system. Given a text block from a document, extract:

[L125]

[L126] 1. **Concepts:** Named entities, technical terms, or ideas discussed

[L127] 2. **Definitions:** If the text defines any concept, extract the definition

[L128] 3. **Properties:** Key attributes with types and values

[L129] 4. **Relations:** How concepts relate (IS\_A, PART\_OF, USES, CAUSES, etc.)

[L130] 5. **Domain:** The subject area (e.g., "Operating Systems", "Networking")

[L131]

[L132] Respond in JSON format:

[L133] {

[L134] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."], "properties": {"key": {"type": "...", "value": "..."}},

[L135] "relations": [{"source": "...", "type": "IS\_A|PART\_OF|USES|CAUSES|...", "target": "...", "evidence": "..."}],

[L136] "domain": "..."

[L137] }

[L138]

[L139] TEXT BLOCK:

[L140] {text}

[L141] """

[L142]

```

[L143] def layer3_extract(blocks: list, llm_client) -> list:
[L144] """Layer 3: Knowledge Extraction pipeline."""
[L145] knowledge_units = []
[L146]
[L147] for block in blocks:
[L148] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L149] continue # Skip non-content blocks
[L150]
[L151] # Step 1: LLM-based extraction
[L152] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L153] try:
[L154] response = llm_client.query(prompt, response_format="json")
[L155] extracted = json.loads(response)
[L156] except (json.JSONDecodeError, LLMLTimeout) as e:
[L157] # Fallback: rule-based extraction
[L158] extracted = rule_based_extract(block.text)
[L159]
[L160] # Step 2: Create KnowledgeUnit per concept
[L161] for concept in extracted.get('concepts', []):
[L162] unit = KnowledgeUnit(
[L163] unit_id=f"KU_{len(knowledge_units):06d}",
[L164] concept_name=concept['name'],
[L165] aliases=concept.get('aliases', []),
[L166] definitions=concept.get('definitions', []),
[L167] properties=concept.get('properties', {}),
[L168] raw_relations=[r for r in extracted.get('relations', [])
[L169] if r['source'] == concept['name']],
[L170] source_doc=block.doc_id,
[L171] source_block=block.block_id,
[L172] page=block.page_number,

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: [filecite]turn10file2[

```

[L173] metadata:
[L174] query_result_index: 2
[L175] file_id: file_00000000a0a481fab4c739c73d87551b
[L176] version_id: 1
[L177] name: SHIA_RAG_Complete_Report(7).md
[L178] mime_type: text/markdown
[L179] surface: library
[L180] score: 0.01769119769119769
[L181] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--34
[L182] snippet:
[L183] > [!IMPORTANT]
[L184] > Layer 3 / Layer 4 boundary: Layer 3 performs candidate extraction — it identifies entities, definitions, and

```

raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology, resolves conflicts, and assigns confidence scores. Layer 3 outputs raw `KnowledgeUnit` objects; Layer 4 produces scored `KnowledgeEdge` objects.

[L185]

[L186] **Input:** List of `DocumentBlock` from Layer 2

[L187] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)

[L188]

[L189] ```python

[L190] from dataclasses import dataclass

[L191]

[L192] @dataclass

[L193] class KnowledgeUnit:

[L194] """Atomic fact extracted from a single document block.

[L195] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""

[L196] unit\_id: str

[L197] concept\_name: str # Raw extracted concept name

[L198] aliases: list # Alternative names found in text

[L199] definitions: list # Definition sentences

[L200] properties: dict # Key-value properties (typed)

[L201] raw\_relations: list # Candidate relations: [(relation\_type, target\_concept, evidence\_text)]

[L202] source\_doc: str

[L203] source\_block: str

[L204] page: int

[L205] extraction\_confidence: float # LLM extraction confidence

[L206] domain\_hint: str # LLM-suggested domain

[L207]

[L208] # --- Extraction Pipeline ---

[L209]

[L210] EXTRACTION\_PROMPT = """You are a knowledge extraction system. Given a text block from a document, extract:

[L211]

[L212] 1. **Concepts:** Named entities, technical terms, or ideas discussed

[L213] 2. **Definitions:** If the text defines any concept, extract the definition

[L214] 3. **Properties:** Key attributes with types and values

[L215] 4. **Relations:** How concepts relate (IS\_A, PART\_OF, USES, CAUSES, etc.)

[L216] 5. **Domain:** The subject area (e.g., "Operating Systems", "Networking")

[L217]

[L218] Respond in JSON format:

[L219] {

[L220] "concepts": [{"name": "...", "aliases": [...], "definitions": [...], "properties": {"key": {"type": "...", "value": "..."}}}],

[L221] "relations": [{"source": "...", "type": "IS\_A|PART\_OF|USES|CAUSES|...", "target": "...", "evidence": "..."}],

[L222] "domain": "..."

[L223] }

[L224]

[L225] TEXT BLOCK:

[L226] {text}

```

[L227] """
[L228]
[L229] def layer3_extract(blocks: list, llm_client) -> list:
[L230] """Layer 3: Knowledge Extraction pipeline."""
[L231] knowledge_units = []
[L232]
[L233] for block in blocks:
[L234] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L235] continue # Skip non-content blocks
[L236]
[L237] # Step 1: LLM-based extraction
[L238] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L239] try:
[L240] response = llm_client.query(prompt, response_format="json")
[L241] extracted = json.loads(response)
[L242] except (json.JSONDecodeError, LLMTimeout) as e:
[L243] # Fallback: rule-based extraction
[L244] extracted = rule_based_extract(block.text)
[L245]
[L246] # Step 2: Create KnowledgeUnit per concept
[L247] for concept in extracted.get('concepts', []):
[L248] unit = KnowledgeUnit(
[L249] unit_id=f"KU_{len(knowledge_units):06d}",
[L250] concept_name=concept['name'],
[L251] aliases=concept.get('aliases', []),
[L252] definitions=concept.get('definitions', []),
[L253] properties=concept.get('properties', {}),
[L254] raw_relations=[r for r in extracted.get('relations', [])
[L255] if r['source'] == concept['name']],
[L256] source_doc=block.doc_id,
[L257] source_block=block.block_id,
[L258] page=block.page_number,

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite[turn10file3]`

```

[L259] metadata:
[L260] query_result_index: 3
[L261] file_id: file_000000001dd481fa9855f2731dbc53d5
[L262] version_id: 1
[L263] name: SHIA_RAG_Complete_Report(6).md
[L264] mime_type: text/markdown
[L265] surface: library
[L266] score: 0.01744318181818182
[L267] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--33
[L268] snippet:

```

```

[L269] ', '2.', 'a)', 'b)']):
[L270] return BlockType.LIST, 0.85
[L271]
[L272] # Table detection (contains tab-separated or pipe-separated data)
[L273] if '|' in text_stripped and text_stripped.count('|') >= 3:
[L274] return BlockType.TABLE, 0.8
[L275]
[L276] # Code/algorithm detection
[L277] if any(kw in text_stripped for kw in ['def ', 'class ', 'import ', 'for ', 'while ', 'if ']):
[L278] return BlockType.CODE, 0.75
[L279]
[L280] # Definition detection (contains "is a", "is defined as", "refers to")
[L281] if any(p in text_stripped.lower() for p in ['is defined as', 'refers to', 'is a type of']):
[L282] return BlockType.DEFINITION, 0.8
[L283]
[L284] # Default: paragraph
[L285] return BlockType.PARAGRAPH, 0.7
[L286]
[L287] def segment_into_regions(page_text: str) -> list:
[L288] """Split a page into logical regions based on whitespace gaps."""
[L289] regions = []
[L290] current_region = []
[L291]
[L292] for line in page_text.split("\n"):
[L293] if line.strip() == "" and current_region:
[L294] regions.append("\n".join(current_region))
[L295] current_region = []
[L296] else:
[L297] current_region.append(line)
[L298]
[L299] if current_region:
[L300] regions.append("\n".join(current_region))
[L301]
[L302] return regions
[L303]

```

```

[L304]
[L305] ---
[L306]
[L307] ### 7.4 Layer 3: Knowledge Extraction
[L308]
[L309] Responsibility: Extract concepts, definitions, and relations from document blocks to produce `KnowledgeNode` objects.
[L310]
[L311] > [!IMPORTANT]
[L312] > Layer 3 / Layer 4 boundary: Layer 3 performs candidate extraction – it identifies entities,
[L313]
[L314] Input: List of `DocumentBlock` from Layer 2
[L315] Output: List of `KnowledgeNode` objects (canonical, deduplicated)
[L316]
[L317]

```

```

python
[L318] from dataclasses import dataclass
[L319]
[L320] @dataclass
[L321] class KnowledgeUnit:
[L322] """Atomic fact extracted from a single document block.
[L323] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L324] unit_id: str
[L325] concept_name: str # Raw extracted concept name
[L326] aliases: list # Alternative names found in text
[L327] definitions: list # Definition sentences
[L328] properties: dict # Key-value properties (typed)
[L329] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]
[L330] source_doc: str
[L331] source_block: str
[L332] page: int
[L333] extraction_confidence: float # LLM extraction confidence
[L334] domain_hint: str # LLM-suggested domain
[L335]
[L336] # --- Extraction Pipeline ---
[L337]
[L338] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L339]
[L340] 1. Concepts: Named entities, technical terms, or ideas discussed
[L341] 2. Definitions: If the text defines any concept, extract the definition
[L342] 3. Properties: Key attributes with types and values
[L343] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L344] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite[turn10file4]`

```

[L345] metadata:
[L346] query_result_index: 4
[L347] file_id: file_000000001dd481fa9855f2731dbc53d5
[L348] version_id: 1
[L349] name: SHIA_RAG_Complete_Report(6).md
[L350] mime_type: text/markdown
[L351] surface: library
[L352] score: 0.017202797202797205
[L353] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--34
[L354] snippet:
[L355] > [!IMPORTANT]
[L356] > Layer 3 / Layer 4 boundary: Layer 3 performs candidate extraction — it identifies entities, definitions, and
raw relation triples from text. Layer 4 performs validation and scoring — it validates extracted relations against the

```

ontology, resolves conflicts, and assigns confidence scores. Layer 3 outputs raw `KnowledgeUnit` objects; Layer 4 produces scored `KnowledgeEdge` objects.

[L357]

[L358] **Input:** List of `DocumentBlock` from Layer 2

[L359] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)

[L360]

[L361] ```python

[L362] from dataclasses import dataclass

[L363]

[L364] @dataclass

[L365] class KnowledgeUnit:

[L366] """Atomic fact extracted from a single document block.

[L367] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""

[L368] unit\_id: str

[L369] concept\_name: str # Raw extracted concept name

[L370] aliases: list # Alternative names found in text

[L371] definitions: list # Definition sentences

[L372] properties: dict # Key-value properties (typed)

[L373] raw\_relations: list # Candidate relations: [(relation\_type, target\_concept, evidence\_text)]

[L374] source\_doc: str

[L375] source\_block: str

[L376] page: int

[L377] extraction\_confidence: float # LLM extraction confidence

[L378] domain\_hint: str # LLM-suggested domain

[L379]

[L380] # --- Extraction Pipeline ---

[L381]

[L382] EXTRACTION\_PROMPT = """You are a knowledge extraction system. Given a text block from a document, extract:

[L383]

[L384] 1. **Concepts:** Named entities, technical terms, or ideas discussed

[L385] 2. **Definitions:** If the text defines any concept, extract the definition

[L386] 3. **Properties:** Key attributes with types and values

[L387] 4. **Relations:** How concepts relate (IS\_A, PART\_OF, USES, CAUSES, etc.)

[L388] 5. **Domain:** The subject area (e.g., "Operating Systems", "Networking")

[L389]

[L390] Respond in JSON format:

[L391] {

[L392] "concepts": [{"name": "...", "aliases": [...], "definitions": [...], "properties": {"key": {"type": "...", "value": "..."}}}],

[L393] "relations": [{"source": "...", "type": "IS\_A|PART\_OF|USES|CAUSES|...", "target": "...", "evidence": "..."}],

[L394] "domain": "..."

[L395] }

[L396]

[L397] TEXT BLOCK:

[L398] {text}

[L399] """



```

[L400]
[L401] def layer3_extract(blocks: list, llm_client) -> list:
[L402] """Layer 3: Knowledge Extraction pipeline."""
[L403] knowledge_units = []
[L404]
[L405] for block in blocks:
[L406] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L407] continue # Skip non-content blocks
[L408]
[L409] # Step 1: LLM-based extraction
[L410] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L411] try:
[L412] response = llm_client.query(prompt, response_format="json")
[L413] extracted = json.loads(response)
[L414] except (json.JSONDecodeError, LLMLTimeout) as e:
[L415] # Fallback: rule-based extraction
[L416] extracted = rule_based_extract(block.text)
[L417]
[L418] # Step 2: Create KnowledgeUnit per concept
[L419] for concept in extracted.get('concepts', []):
[L420] unit = KnowledgeUnit(
[L421] unit_id=f"KU_{len(knowledge_units):06d}",
[L422] concept_name=concept['name'],
[L423] aliases=concept.get('aliases', []),
[L424] definitions=concept.get('definitions', []),
[L425] properties=concept.get('properties', {}),
[L426] raw_relations=[r for r in extracted.get('relations', [])
[L427] if r['source'] == concept['name']],
[L428] source_doc=block.doc_id,
[L429] source_block=block.block_id,
[L430] page=block.page_number,

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite[turn10file5]`

```

[L431] metadata:
[L432] query_result_index: 5
[L433] file_id: file_0000000033088207b45d2ae58cbd406c
[L434] version_id: 1
[L435] name: SHIA_RAG_Complete_Report(8).md
[L436] mime_type: text/markdown
[L437] surface: library
[L438] score: 0.01696969696969697
[L439] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--33
[L440] snippet:
[L441] ', '2.', 'a)', 'b)']):

```

```

[L442] return BlockType.LIST, 0.85
[L443]
[L444] # Table detection (contains tab-separated or pipe-separated data)
[L445] if '|' in text_stripped and text_stripped.count('|') >= 3:
[L446] return BlockType.TABLE, 0.8
[L447]
[L448] # Code/algorithm detection
[L449] if any(kw in text_stripped for kw in ['def ', 'class ', 'import ', 'for ', 'while ', 'if ']):
[L450] return BlockType.CODE, 0.75
[L451]
[L452] # Definition detection (contains "is a", "is defined as", "refers to")
[L453] if any(p in text_stripped.lower() for p in ['is defined as', 'refers to', 'is a type of']):
[L454] return BlockType.DEFINITION, 0.8
[L455]
[L456] # Default: paragraph
[L457] return BlockType.PARAGRAPH, 0.7
[L458]
[L459] def segment_into_regions(page_text: str) -> list:
[L460] """Split a page into logical regions based on whitespace gaps."""
[L461] regions = []
[L462] current_region = []
[L463]
[L464] for line in page_text.split("\n"):
[L465] if line.strip() == "" and current_region:
[L466] regions.append('\n'.join(current_region))
[L467] current_region = []
[L468] else:
[L469] current_region.append(line)
[L470]
[L471] if current_region:
[L472] regions.append('\n'.join(current_region))
[L473]
[L474] return regions
[L475]

```

```

[L476]
[L477] ---
[L478]
[L479] ### 7.4 Layer 3: Knowledge Extraction
[L480]
[L481] **Responsibility:** Extract concepts, definitions, and relations from document blocks to produce `KnowledgeNode` objects
[L482]
[L483] > [!IMPORTANT]
[L484] > **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, relations, and concepts from the input text.
[L485]
[L486] **Input:** List of `DocumentBlock` from Layer 2
[L487] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)
[L488]
[L489]

```

```

python
[L490] from dataclasses import dataclass
[L491]
[L492] @dataclass
[L493] class KnowledgeUnit:
[L494] """Atomic fact extracted from a single document block.
[L495] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L496] unit_id: str
[L497] concept_name: str # Raw extracted concept name
[L498] aliases: list # Alternative names found in text
[L499] definitions: list # Definition sentences
[L500] properties: dict # Key-value properties (typed)
[L501] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]
[L502] source_doc: str
[L503] source_block: str
[L504] page: int
[L505] extraction_confidence: float # LLM extraction confidence
[L506] domain_hint: str # LLM-suggested domain
[L507]
[L508] # --- Extraction Pipeline ---
[L509]
[L510] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L511]
[L512] 1. Concepts: Named entities, technical terms, or ideas discussed
[L513] 2. Definitions: If the text defines any concept, extract the definition
[L514] 3. Properties: Key attributes with types and values
[L515] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L516] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite[turn10file6]`

```

[L517] metadata:
[L518] query_result_index: 6
[L519] file_id: file_000000003c8081fab658b1752549d64
[L520] version_id: 1
[L521] name: SHIA_RAG_Complete_Report(5).md
[L522] mime_type: text/markdown
[L523] surface: library
[L524] score: 0.01696969696969697
[L525] document_chunk_id: file_000000003c8081fab658b1752549d64--1--33
[L526] snippet:
[L527] ', '2.', 'a)', 'b)']):
[L528] return BlockType.LIST, 0.85
[L529]

```

```

[L530] # Table detection (contains tab-separated or pipe-separated data)
[L531] if '|' in text_stripped and text_stripped.count('|') >= 3:
[L532] return BlockType.TABLE, 0.8
[L533]
[L534] # Code/algorithm detection
[L535] if any(kw in text_stripped for kw in ['def ', 'class ', 'import ', 'for ', 'while ', 'if ']):
[L536] return BlockType.CODE, 0.75
[L537]
[L538] # Definition detection (contains "is a", "is defined as", "refers to")
[L539] if any(p in text_stripped.lower() for p in ['is defined as', 'refers to', 'is a type of']):
[L540] return BlockType.DEFINITION, 0.8
[L541]
[L542] # Default: paragraph
[L543] return BlockType.PARAGRAPH, 0.7
[L544]
[L545] def segment_into_regions(page_text: str) -> list:
[L546] """Split a page into logical regions based on whitespace gaps."""
[L547] regions = []
[L548] current_region = []
[L549]
[L550] for line in page_text.split('\n'):
[L551] if line.strip() == "" and current_region:
[L552] regions.append('\n'.join(current_region))
[L553] current_region = []
[L554] else:
[L555] current_region.append(line)
[L556]
[L557] if current_region:
[L558] regions.append('\n'.join(current_region))
[L559]
[L560] return regions
[L561]

```

```

[L562]
[L563] ---
[L564]
[L565] ### 7.4 Layer 3: Knowledge Extraction
[L566]
[L567] **Responsibility:** Extract concepts, definitions, and relations from document blocks to produce `KnowledgeNode` objects
[L568]
[L569] > [!IMPORTANT]
[L570] > **Layer 3 / Layer 4 boundary:** Layer 3 performs candidate extraction – it identifies entities, relations, and concepts from the input text.
[L571]
[L572] **Input:** List of `DocumentBlock` from Layer 2
[L573] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)
[L574]
[L575]

```

python

```

[L576] from dataclasses import dataclass
[L577]

```

```

[L578] @dataclass
[L579] class KnowledgeUnit:
[L580] """Atomic fact extracted from a single document block.
[L581] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L582] unit_id: str
[L583] concept_name: str # Raw extracted concept name
[L584] aliases: list # Alternative names found in text
[L585] definitions: list # Definition sentences
[L586] properties: dict # Key-value properties (typed)
[L587] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]
[L588] source_doc: str
[L589] source_block: str
[L590] page: int
[L591] extraction_confidence: float # LLM extraction confidence
[L592] domain_hint: str # LLM-suggested domain
[L593]
[L594] # --- Extraction Pipeline ---
[L595]
[L596] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L597]
[L598] 1. Concepts: Named entities, technical terms, or ideas discussed
[L599] 2. Definitions: If the text defines any concept, extract the definition
[L600] 3. Properties: Key attributes with types and values
[L601] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L602] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")

```

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `filecite{turn10file7}`

```

[L603] metadata:
[L604] query_result_index: 7
[L605] file_id: file_00000000a0a481fab4c739c73d87551b
[L606] version_id: 1
[L607] name: SHIA_RAG_Complete_Report(7).md
[L608] mime_type: text/markdown
[L609] surface: library
[L610] score: 0.01696969696969697
[L611] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--33
[L612] snippet:
[L613] ', '2.', 'a)', 'b)']):
[L614] return BlockType.LIST, 0.85
[L615]
[L616] # Table detection (contains tab-separated or pipe-separated data)
[L617] if '|' in text_stripped and text_stripped.count('|') >= 3:
[L618] return BlockType.TABLE, 0.8

```

```

[L619]
[L620] # Code/algorithm detection
[L621] if any(kw in text_stripped for kw in ['def ', 'class ', 'import ', 'for ', 'while ', 'if ']):
[L622] return BlockType.CODE, 0.75
[L623]
[L624] # Definition detection (contains "is a", "is defined as", "refers to")
[L625] if any(p in text_stripped.lower() for p in ['is defined as', 'refers to', 'is a type of']):
[L626] return BlockType.DEFINITION, 0.8
[L627]
[L628] # Default: paragraph
[L629] return BlockType.PARAGRAPH, 0.7
[L630]
[L631] def segment_into_regions(page_text: str) -> list:
[L632] """Split a page into logical regions based on whitespace gaps."""
[L633] regions = []
[L634] current_region = []
[L635]
[L636] for line in page_text.split("\n"):
[L637] if line.strip() == "" and current_region:
[L638] regions.append("\n".join(current_region))
[L639] current_region = []
[L640] else:
[L641] current_region.append(line)
[L642]
[L643] if current_region:
[L644] regions.append("\n".join(current_region))
[L645]
[L646] return regions
[L647]

```

```

[L648]
[L649] ---
[L650]
[L651] ### 7.4 Layer 3: Knowledge Extraction
[L652]
[L653] **Responsibility:** Extract concepts, definitions, and relations from document blocks to produce `KnowledgeUnit` objects.
[L654]
[L655] > [!IMPORTANT]
[L656] > **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, relations, and concepts from the input text.
[L657]
[L658] **Input:** List of `DocumentBlock` from Layer 2
[L659] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)
[L660]
[L661]

```

python

```

[L662] from dataclasses import dataclass
[L663]
[L664] @dataclass
[L665] class KnowledgeUnit:
[L666] """Atomic fact extracted from a single document block.

```

[L667] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""

[L668] unit\_id: str

[L669] concept\_name: str # Raw extracted concept name

[L670] aliases: list # Alternative names found in text

[L671] definitions: list # Definition sentences

[L672] properties: dict # Key-value properties (typed)

[L673] raw\_relations: list # Candidate relations: [(relation\_type, target\_concept, evidence\_text)]

[L674] source\_doc: str

[L675] source\_block: str

[L676] page: int

[L677] extraction\_confidence: float # LLM extraction confidence

[L678] domain\_hint: str # LLM-suggested domain

[L679]

[L680] # --- Extraction Pipeline ---

[L681]

[L682] EXTRACTION\_PROMPT = """"You are a knowledge extraction system. Given a text block from a document, extract:

[L683]

[L684] 1. **Concepts:** Named entities, technical terms, or ideas discussed

[L685] 2. **Definitions:** If the text defines any concept, extract the definition

[L686] 3. **Properties:** Key attributes with types and values

[L687] 4. **Relations:** How concepts relate (IS\_A, PART\_OF, USES, CAUSES, etc.)

[L688] 5. **Domain:** The subject area (e.g., "Operating Systems", "Networking")

---

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `[[filecite]]turn10file8[[`

[L689] metadata:

[L690] query\_result\_index: 8

[L691] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L692] version\_id: 1

[L693] name: SHIA\_RAG\_Final\_Report.md

[L694] mime\_type: text/markdown

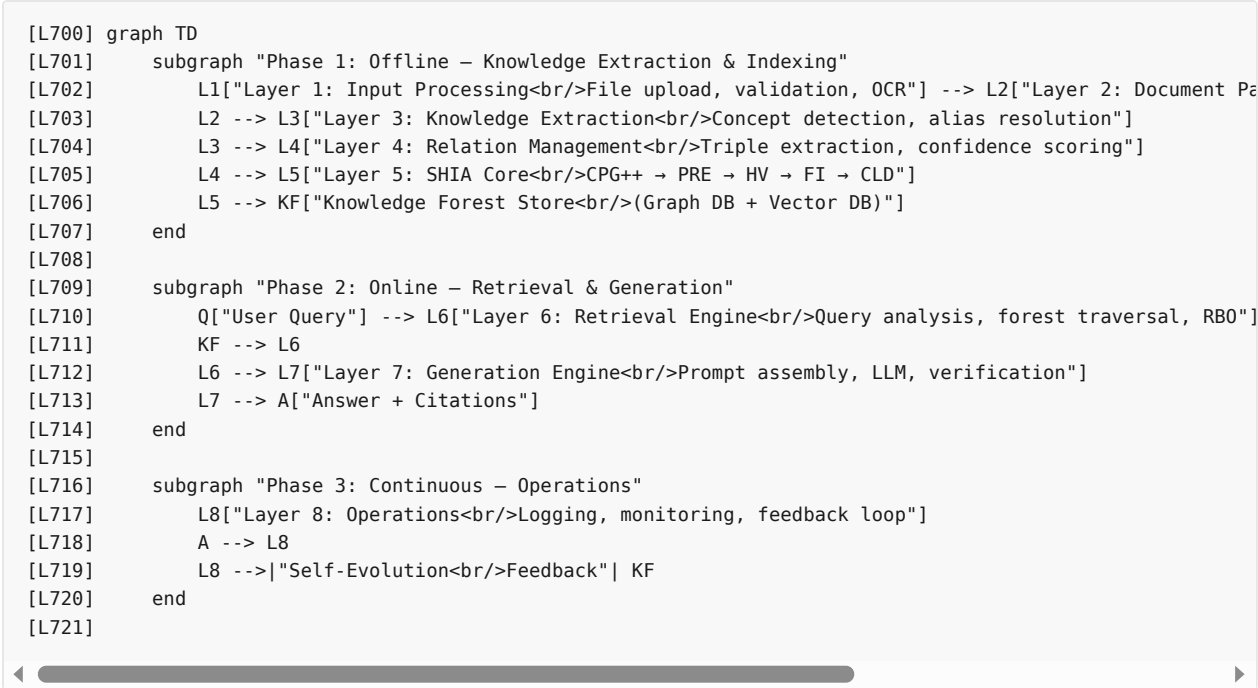
[L695] surface: library

[L696] score: 0.016310935441370224

[L697] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--6

[L698] snippet:

[L699]

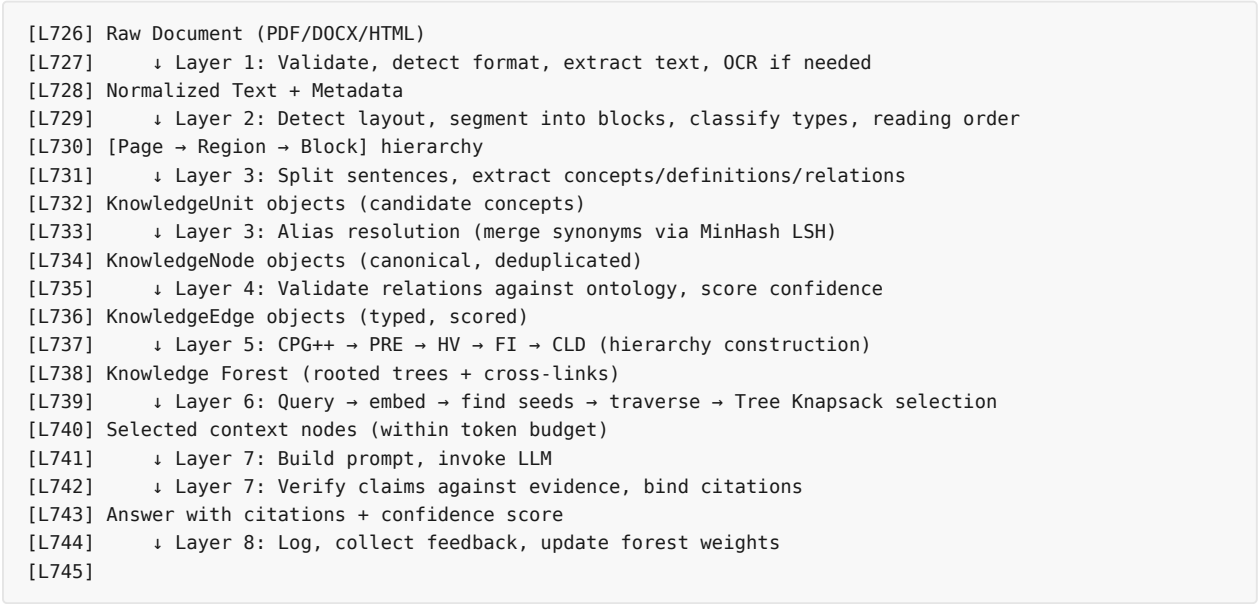


[L722]

[L723] ### 3.2 Data Flow

[L724]

[L725]



[L746]

[L747] ### 3.3 Layer Summary

[L748]

[L749] | Layer | Name | Input | Output | Key Algorithms |

[L750] |-----|-----|-----|-----|-----|

[L751] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L752] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L753] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

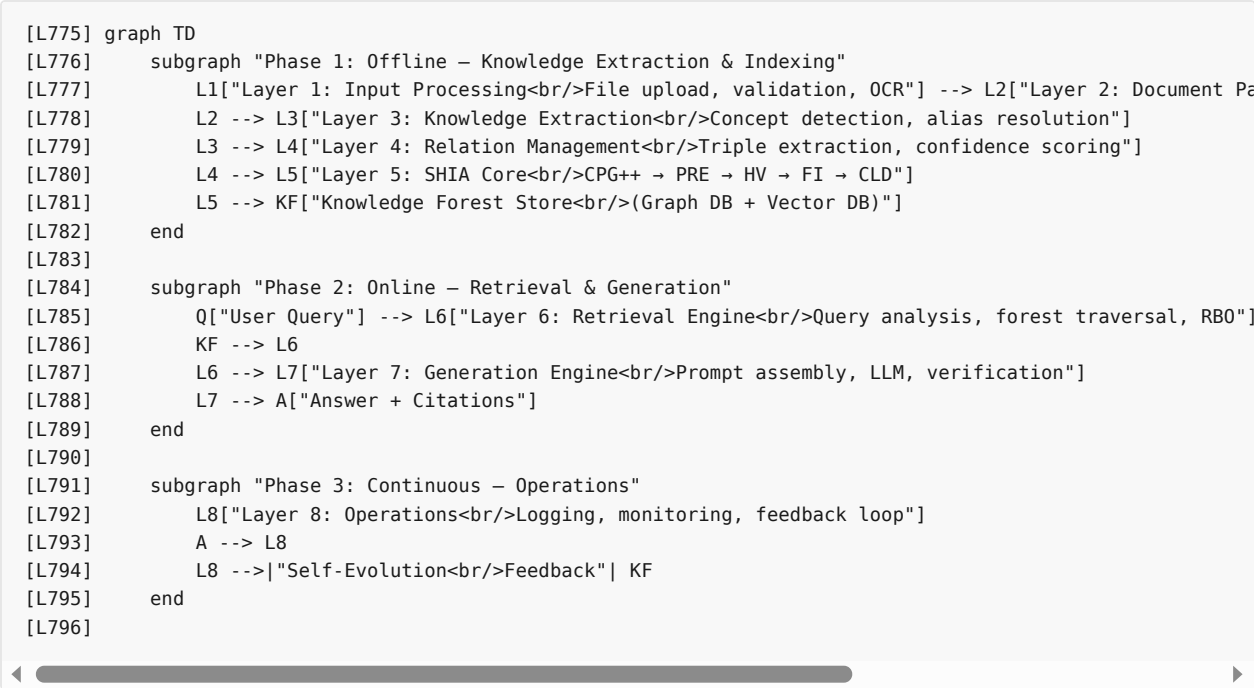


[L754] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L755] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L756] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L757] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L758] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L759] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L760]  
[L761] ---  
[L762]  
[L763] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:28:59

Citation Marker: `[[filecite]]turn10file9`

[L764] metadata:  
[L765] query\_result\_index: 9  
[L766] file\_id: file\_00000000f620821186e54e582d59731e  
[L767] version\_id: 1  
[L768] name: SHIA\_RAG\_Final\_Report(3).md  
[L769] mime\_type: text/markdown  
[L770] surface: library  
[L771] score: 0.016310935441370224  
[L772] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--6  
[L773] snippet:  
[L774]



[L797]  
[L798] ### 3.2 Data Flow

[L799]  
[L800]

```
[L801] Raw Document (PDF/DOCX/HTML)
[L802] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L803] Normalized Text + Metadata
[L804] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L805] [Page → Region → Block] hierarchy
[L806] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L807] KnowledgeUnit objects (candidate concepts)
[L808] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L809] KnowledgeNode objects (canonical, deduplicated)
[L810] ↓ Layer 4: Validate relations against ontology, score confidence
[L811] KnowledgeEdge objects (typed, scored)
[L812] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L813] Knowledge Forest (rooted trees + cross-links)
[L814] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L815] Selected context nodes (within token budget)
[L816] ↓ Layer 7: Build prompt, invoke LLM
[L817] ↓ Layer 7: Verify claims against evidence, bind citations
[L818] Answer with citations + confidence score
[L819] ↓ Layer 8: Log, collect feedback, update forest weights
[L820]
```

[L821]  
[L822] ### 3.3 Layer Summary  
[L823]  
[L824] | Layer | Name | Input | Output | Key Algorithms |  
[L825] |-----|-----|-----|-----|-----|  
[L826] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |  
[L827] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |  
[L828] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |  
[L829] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L830] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L831] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L832] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L833] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L834] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L835]  
[L836] ---  
[L837]  
[L838] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 14:29:03  
Citation Marker: fileciteturn11file0

[L1] metadata:  
[L2] ref\_id: turn10file0  
[L3] file\_id: file\_000000003c8081fab658b1752549d64  
[L4] version\_id: 1  
[L5] name: SHIA\_RAG\_Complete\_Report(5).md  
[L6] lines\_shown: 57-236

[L7] total\_file\_lines: 3316  
[L8] has\_more: True  
[L9] next\_start\_line: 237  
[L10] next\_read: {"ref\_id":"turn10file0","mode":"full","start\_line":237,"max\_lines":180}  
[L11] content:  
[L12]  
[L13] **SHIA-RAG** instead:  
[L14] 1. **Extracts knowledge** (concepts, definitions, relations) from documents  
[L15] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes (not text chunks)  
[L16] 3. **Retrieves knowledge nodes** by traversing the hierarchy (parents, children, siblings)  
[L17] 4. **Assembles context** intelligently within a token budget  
[L18] 5. **Generates answers** with citation-backed evidence  
[L19]  
[L20] **### 1.2 Why It Matters**  
[L21]  
[L22] | Problem with Traditional RAG | How SHIA-RAG Solves It |  
[L23] |---|---|  
[L24] | Chunks break semantic continuity | Knowledge nodes preserve complete concepts |  
[L25] | Duplicate/fragmented information | Entity resolution merges synonyms into single nodes |  
[L26] | Fixed chunk granularity | Hierarchy allows multi-level retrieval (broad → specific) |  
[L27] | Weak multi-hop reasoning | Parent-child paths encode reasoning chains |  
[L28] | No cross-document fusion | Knowledge Forest merges concepts across all documents |  
[L29]  
[L30] **### 1.3 Core Innovation**  
[L31]  
[L32] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "CS → OS → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.  
[L33]  
[L34] ---  
[L35]  
[L36] **## 2. Document Inventory & Summary**  
[L37]  
[L38] **### 2.1 All 6 PDFs Analyzed**  
[L39]  
[L40] | # | File Name | Pages | Content Type | What It Contains |  
[L41] |---|-----|-----|-----|-----|  
[L42] | 1 | **first Pdf.pdf** | 24 | Original MDS (Academic) | Executive Summary, Problem Statement, Literature Review, System Architecture (13 modules), SHIA Core algorithms (CPG++, PRE, HV, FI, CLD), Evaluation Plan, Implementation Roadmap |  
[L43] | 2 | **third Pdf.pdf** | 10 | Algorithms & Scoring | Detailed scoring formulas for all SHIA modules, pseudocode for 6 algorithms, retrieval policies (RL-based adaptive depth), uncertainty resolution, security considerations |  
[L44] | 3 | **4h pdf.pdf** | 22 | Formal Specification | Formal mathematical definitions, Energy Minimization objective, C++ interfaces, SHEF evaluation framework, complexity analysis, experimental methodology, reproducibility checklist |  
[L45] | 4 | **Master Design Specification (SHIA-RAG).pdf** | 46 | Production MDS | Complete 8-layer architecture with JSON schemas, SQL/Cypher DB design, OpenAPI contracts, microservice folder structure, Layer-by-layer

algorithms with pseudocode |

[L46] | 5 | PDF Comparison Summary.pdf | 236 | Research Comparison | Deep comparison of all PDFs, literature positioning against TreeRAG/HiChunk/RAPTOR/GraphRAG, gap analysis, combined architecture walkthrough |

[L47] | 6 | PDFs discussion and analysis.pdf | 483 | Extended Analysis | Collaborative transcript refining SHIA-RAG, detailed layer-by-layer walkthroughs, critical review identifying gaps, research contribution proposals |

[L48]

[L49] ### 2.2 How Documents Relate

[L50]

[L51]

```
[L52] graph TD
[L53] A["PDF 1: first Pdf.pdf
(24p) Original Vision"] --> D["PDF 5: Comparison Summary
(236p) Re
[L54] B["PDF 2: third Pdf.pdf
(10p) Algorithms"] --> D
[L55] C["PDF 3: 4h pdf.pdf
(22p) Formal Spec"] --> D
[L56] E["PDF 4: Master Design Spec
(46p) Production Blueprint"] --> D
[L57] D --> F["PDF 6: Discussion & Analysis
(483p) Critical Refinement"]
[L58]
[L59] A -->|"Evolved into"| E
[L60] B -->|"Adds formulas to"| E
[L61] C -->|"Adds rigor to"| E
[L62] F -->|"Identifies gaps in"| A
[L63] F -->|"Identifies gaps in"| E
[L64]
```

[L65]

[L66] ---

[L67]

[L68] ## 3. Critical Errors & Fixes

[L69]

[L70] > [!CAUTION]

[L71] > **24+ issues identified across all 6 PDFs.** Below are the most critical ones with proposed fixes.

[L72]

[L73] ### 3.1 FATAL: Broken Cycle Detection (PDF 2 — third Pdf.pdf )

[L74]

[L75] **The Bug:** The DFS cycle detection algorithm traverses the **wrong direction**.

[L76]

[L77]

```

[L78] # BROKEN (current code)
[L79] - def is_valid(N, P, forest):
[L80] - stack = [P] # Start from Parent
[L81] - while stack:
[L82] - cur = stack.pop()
[L83] - for child in forest.children(cur): # Go DOWN from P
[L84] - if child == N: # Found N as descendant of P
[L85] - return False # This detects redundancy, NOT cycles
[L86] - stack.append(child)
[L87] - return True
[L88]
[L89] # FIXED
[L90] + def is_valid(N, P, forest):
[L91] + """Check: would adding edge P-N create a cycle?
[L92] + A cycle exists if N is already an ANCESTOR of P.
[L93] + So traverse UP from P toward roots, looking for N."""
[L94] + visited = set()
[L95] + stack = [P]
[L96] + while stack:
[L97] + cur = stack.pop()
[L98] + if cur == N:
[L99] + return False # N is ancestor of P → cycle!
[L100] + if cur not in visited:
[L101] + visited.add(cur)
[L102] + for parent in forest.parents(cur): # Go UP
[L103] + stack.append(parent)
[L104] + return True
[L105]

```

[L106]

[L107] **Impact:** Without this fix, cycles in the Knowledge Forest will go undetected, violating the acyclic invariant and causing infinite loops during traversal.

[L108]

[L109] ---

[L110]

[L111] ### 3.2 FATAL: No Feedback Loop Between PRE and HV (PDF 3 — 4h pdf.pdf)

[L112]

[L113] **The Bug:** PRE selects the single best parent → HV validates it. If HV rejects (e.g., cycle detected), the node is **orphaned** with no fallback.

[L114]

[L115]

```

[L116] # BROKEN (current pipeline)
[L117] - best_parent = PRE.rank(candidates) # Pick ONE best
[L118] - if HV.validate(best_parent, node):
[L119] - FI.insert(node, best_parent)
[L120] - else:
[L121] - ??? # Node is lost!
[L122]
[L123] # FIXED: PRE returns ranked list, HV iterates
[L124] + ranked_parents = PRE.rank_all(candidates) # Return SORTED list
[L125] + inserted = False
[L126] + for parent in ranked_parents:
[L127] + if HV.validate(parent, node):
[L128] + FI.insert(node, parent)
[L129] + inserted = True
[L130] + break
[L131] + if not inserted:
[L132] + FI.insert_as_root(node) # Create new tree
[L133]

```

[L134]

[L135] ---

[L136]

[L137] ### 3.3 FATAL: Confidence Propagation Without Topological Sort (PDF 2)

[L138]

[L139] **The Bug:** `propagate_confidence()` iterates over nodes in arbitrary order. A child's confidence depends on its parent's updated confidence, but the parent may not have been processed yet. Additionally, the original formula only uses parent confidence with decay, ignoring the node's own extraction confidence and evidence confidence.

[L140]

[L141]

```
[L142] # BROKEN
[L143] - for node in forest.nodes(): # Arbitrary order
[L144] - if node.parents:
[L145] - node.confidence = avg(p.confidence for p in node.parents) * decay
[L146]
[L147] # FIXED: Use topological sort (parents always before children)
[L148] # FIXED: Include node's extraction confidence and evidence confidence
[L149] + from collections import deque
[L150] +
[L151] + # Confidence weighting factors (must sum to 1.0)
[L152] + ALPHA = 0.4 # Node's own extraction confidence weight
[L153] + BETA = 0.4 # Parent confidence weight
[L154] + GAMMA = 0.2 # Evidence confidence weight
[L155] + DECAY = 0.95 # Per-level decay factor
[L156] +
[L157] + def propagate_confidence(forest):
[L158] + # Kahn's algorithm for topological ordering
[L159] + in_degree = {n: len(forest.parents(n)) for n in forest.nodes()}
[L160] + queue = deque([n for n, d in in_degree.items() if d == 0]) # Roots first
[L161] + while queue:
[L162] + node = queue.popleft()
[L163] + if forest.parents(node):
[L164] + parent_confs = [p.confidence for p in forest.parents(node)]
[L165] + avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L166] + evidence_conf = avg([e.confidence for e in node.evidence]) if node.evidence else 0.5
[L167] + node.confidence = (
[L168] + ALPHA * node.extraction_confidence +
[L169] + BETA * avg_parent_conf +
[L170] + GAMMA * evidence_conf
[L171] +) * DECAY
[L172] + else:
[L173] + node.confidence = node.base_confidence # Root nodes (default: 0.5)
[L174] + for child in forest.children(node):
[L175] + in_degree[child] -= 1
[L176] + if in_degree[child] == 0:
[L177] + queue.append(child)
[L178]
```

[L179]

[L180] &gt; [NOTE]

[L181] > **base\_confidence** is the standardized term used throughout (default: 0.5 , configurable). Previously referred to as `baseline_conf` in some PDFs.

[L182]

[L183] ---

[L184]

[L185] ### 3.4 CRITICAL: RBO Knapsack Ignores Node Dependencies (PDF 6)

[L186]

[L187] **The Bug:** The Retrieval Budget Optimizer treats nodes as independent items in a 0-1 Knapsack. But in a hierarchy, a child node is meaningless without its parent context.

[L188]

[L189] > [!WARNING]

[L190] > **Implementation Note:** The algorithm below is a **greedy approximation** for the Precedence-Constrained Tree Knapsack problem, NOT an exact dynamic programming solution. Exact DP for tree knapsack is  $\mathcal{O}(nB)$  where  $B$  is the token budget (potentially very large). The greedy approach uses utility-density sorting and provides a practical near-optimal solution.

---

DEEP RESEARCH 20/08/2026, 14:29:03

Citation Marker: `filecite[turn11file1]`

[L191] metadata:

[L192] read\_index: 1

[L193] ref\_id: turn10file8

[L194] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L195] version\_id: 1

[L196] name: SHIA\_RAG\_Final\_Report.md

[L197] lines\_shown: 723-767

[L198] total\_file\_lines: 1527

[L199] has\_more: True

[L200] next\_start\_line: 768

[L201] next\_read: {"ref\_id":"turn10file8","mode":"full","start\_line":768,"max\_lines":45}

[L202] content:

[L203] context\_text: str # Assembled context string

[L204] traversal\_path\_edges: list # Edges traversed (for Layer 8 feedback)

[L205] query\_complexity: str # SIMPLE, MODERATE, COMPLEX

[L206] query\_intent: str # DEFINITION, COMPARISON, etc.

[L207] seed\_node\_ids: list # Initial seed nodes from vector search

[L208] token\_count: int # Estimated tokens in context

[L209]

[L210] class RetrievalEngine:

[L211] def \_\_init\_\_(self, forest, vector\_index, cache\_manager, config):

[L212] self.forest = forest

[L213] self.vector\_index = vector\_index

[L214] self.cache = cache\_manager

[L215] self.config = config

[L216]

[L217] def retrieve(self, query) -> RetrievalResult:

[L218] # Step 0: Cache check

[L219] cached = self.cache.get\_query\_result(hash\_query(query))

[L220] if cached:

[L221] return cached

[L222]

[L223] # Step 1: Classify query intent and complexity

```
[L224] intent = detect_intent(query)
[L225] complexity = classify_complexity(intent)
[L226]
[L227] # Step 2: Embed query and find seed nodes
[L228] query_emb = embed_query(query) # Asymmetric embedding model
[L229] seed_nodes = self.vector_index.search(query_emb, k=5)
[L230]
[L231] # Step 3: Adaptive depth traversal
[L232] depth_config = {
[L233] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L234] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L235] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L236] }
[L237] traversal = depth_config[complexity]
[L238]
[L239] # Step 4: BFS traversal from all seeds
[L240] candidate_nodes = set()
[L241] for seed in seed_nodes:
[L242] candidate_nodes |= self.traverse(seed, traversal)
[L243]
[L244] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L245] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L246]
[L247] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
```

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file0]`

```
[L1] metadata:
[L2] query_result_index: 0
[L3] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(2).md
[L6] mime_type: text/markdown
[L7] surface: library
[L8] score: 0.01848549283215584
[L9] document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--13
[L10] snippet:
[L11] The parser uses rule-based heuristics for block classification:
[L12] - Heading: Short line (< 100 chars), no ending punctuation, starts with uppercase
[L13] - List: Starts with -, *, •, or 1.
[L14] - Table: Contains 3+ pipe characters (|)
[L15] - Code: Contains Python keywords (def, class, import)
[L16] - Definition: Contains phrases like "is defined as", "refers to"
[L17] - Default: Paragraph
[L18]
```



[L19] ---

[L20]

[L21] ### 5.3 Layer 3: Knowledge Extraction

[L22]

[L23] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical `KnowledgeNode` objects.

[L24]

[L25] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)

[L26]

[L27] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.

[L28]

[L29] ##### Extraction Prompt

[L30]

[L31]

```
[L32] You are a knowledge extraction system. Given a text block, extract:
[L33] 1. Concepts: Named entities, technical terms
[L34] 2. Definitions: If the text defines any concept, extract the definition
[L35] 3. Properties: Key attributes with types and values
[L36] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L37] 5. Domain: Subject area (e.g., "Operating Systems")
[L38]
[L39] Respond in JSON format:
[L40] {
[L41] "concepts": [{"name": "...", "aliases": [...], "definitions": [...],
[L42] "properties": {"key": {"type": "...", "value": "...}}}],
[L43] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L44] "evidence": "...}],
[L45] "domain": "...
[L46] }
[L47]
```

[L48]

[L49] ##### Alias Resolution

[L50]

[L51] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L52]

[L53] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L54] 2. Find connected components of near-duplicate units

[L55] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L56]

[L57]

```
[L58] def alias_resolution(units: list) -> list:
[L59] """Merge KnowledgeUnits referring to the same concept.
[L60]
[L61] Merge strategy:
[L62] - Canonical name: most frequent name across units
[L63] - Aliases: union of all names and aliases
[L64] - Definitions: union of all definitions
[L65] - Confidence: average of unit extraction confidences
[L66] - Properties: merged (latest value wins, conflicts logged)
[L67] """
[L68] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L69] # ... build LSH index from concept names + aliases ...
[L70] merge_groups = find_connected_components(units, lsh)
[L71] return [create_canonical_node(group) for group in merge_groups]
[L72]
```

[L73]

[L74] ---

[L75]

[L76] ### 5.4 Layer 4: Relation Management

[L77]

[L78] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L79]

[L80] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file1]`

[L81] metadata:

[L82] query\_result\_index: 1

[L83] file\_id: file\_000000003218820b918ee3d86ec345ae

[L84] version\_id: 1

[L85] name: SHIA\_RAG\_Final\_Report(1).md

[L86] mime\_type: text/markdown

[L87] surface: library

[L88] score: 0.018221082467269535

[L89] document\_chunk\_id: file\_000000003218820b918ee3d86ec345ae--1--13

[L90] snippet:

[L91] The parser uses rule-based heuristics for block classification:

[L92] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase

[L93] - **List:** Starts with `-`, `*`, `•`, or `1.`

[L94] - **Table:** Contains 3+ pipe characters (`|`)

[L95] - **Code:** Contains Python keywords (`def`, `class`, `import`)

[L96] - **Definition:** Contains phrases like "is defined as", "refers to"

[L97] - **Default:** Paragraph

[L98]

[L99] ---

[L100]

[L101] ### 5.3 Layer 3: Knowledge Extraction

[L102]

[L103] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical `KnowledgeNode` objects.

[L104]

[L105] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)

[L106]

[L107] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.

[L108]

[L109] ##### Extraction Prompt

[L110]

[L111]

```
[L112] You are a knowledge extraction system. Given a text block, extract:
[L113] 1. Concepts: Named entities, technical terms
[L114] 2. Definitions: If the text defines any concept, extract the definition
[L115] 3. Properties: Key attributes with types and values
[L116] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L117] 5. Domain: Subject area (e.g., "Operating Systems")
[L118]
[L119] Respond in JSON format:
[L120] {
[L121] "concepts": [{ "name": "...", "aliases": [...], "definitions": [...],
[L122] "properties": { "key": { "type": "...", "value": "..." } } },
[L123] "relations": [{ "source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L124] "evidence": "... }],
[L125] "domain": "..."
[L126] }
[L127]
```

[L128]

[L129] ##### Alias Resolution

[L130]

[L131] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L132]

[L133] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L134] 2. Find connected components of near-duplicate units

[L135] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L136]

[L137]

```

[L138] def alias_resolution(units: list) -> list:
[L139] """Merge KnowledgeUnits referring to the same concept.
[L140]
[L141] Merge strategy:
[L142] - Canonical name: most frequent name across units
[L143] - Aliases: union of all names and aliases
[L144] - Definitions: union of all definitions
[L145] - Confidence: average of unit extraction confidences
[L146] - Properties: merged (latest value wins, conflicts logged)
[L147] """
[L148] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L149] # ... build LSH index from concept names + aliases ...
[L150] merge_groups = find_connected_components(units, lsh)
[L151] return [create_canonical_node(group) for group in merge_groups]
[L152]

```

[L153]

[L154] ---

[L155]

[L156] ### 5.4 Layer 4: Relation Management

[L157]

[L158] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L159]

[L160] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file2]`

[L161] metadata:

[L162] query\_result\_index: 2

[L163] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L164] version\_id: 1

[L165] name: SHIA\_RAG\_Final\_Report.md

[L166] mime\_type: text/markdown

[L167] surface: library

[L168] score: 0.01796506608222089

[L169] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--14

[L170] snippet:

[L171] ##### Alias Resolution

[L172]

[L173] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L174]

[L175] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L176] 2. Find connected components of near-duplicate units

[L177] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L178]

[L179]

```
[L180] def alias_resolution(units: list) -> list:
[L181] """Merge KnowledgeUnits referring to the same concept.
[L182]
[L183] Merge strategy:
[L184] - Canonical name: most frequent name across units
[L185] - Aliases: union of all names and aliases
[L186] - Definitions: union of all definitions
[L187] - Confidence: average of unit extraction confidences
[L188] - Properties: merged (latest value wins, conflicts logged)
[L189] """
[L190] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L191] # ... build LSH index from concept names + aliases ...
[L192] merge_groups = find_connected_components(units, lsh)
[L193] return [create_canonical_node(group) for group in merge_groups]
[L194]
```

[L195]

[L196] ---

[L197]

[L198] ### 5.4 Layer 4: Relation Management

[L199]

[L200] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L201]

[L202] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L203]

[L204] **Pipeline:**

[L205] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L206] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L207] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L208] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L209] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L210]

[L211]

```
[L212] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L213] """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L214] conf = 0.5
[L215] conf += 0.15 * source.confidence
[L216] conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L217] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L218] return min(conf, 1.0)
[L219]
```

[L220]

[L221] ---

[L222]

[L223] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L224]

[L225] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L226]

[L227] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L228]

[L229] ##### Pipeline Overview

[L230]

[L231]

```
[L232] graph LR
[L233] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L234] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L235] PRE --> HV{"HV
Validate #1?"}
[L236] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L237] HV -->|"Fail"| HV2{"HV
Validate #2?"}
[L238] HV2 -->|"Pass"| FI
[L239] HV2 -->|"Fail"| HV3{"HV
Validate #3?"}
[L240] HV3 -->|"Pass"| FI
[L241] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L242] FI --> CLD["CLD
Discover Cross-Links"]
[L243] CLD --> DONE["Node Placed ✓"]
[L244]
```

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file3]`

[L245] metadata:

[L246] query\_result\_index: 3

[L247] file\_id: file\_00000000f620821186e54e582d59731e

[L248] version\_id: 1

[L249] name: SHIA\_RAG\_Final\_Report(3).md

[L250] mime\_type: text/markdown

[L251] surface: library

[L252] score: 0.01796506608222089

[L253] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--14

[L254] snippet:

[L255] ##### Alias Resolution

[L256]

[L257] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L258]

[L259] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L260] 2. Find connected components of near-duplicate units

[L261] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L262]

[L263]

```
[L264] def alias_resolution(units: list) -> list:
[L265] """Merge KnowledgeUnits referring to the same concept.
[L266]
[L267] Merge strategy:
[L268] - Canonical name: most frequent name across units
[L269] - Aliases: union of all names and aliases
[L270] - Definitions: union of all definitions
[L271] - Confidence: average of unit extraction confidences
[L272] - Properties: merged (latest value wins, conflicts logged)
[L273] """
[L274] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L275] # ... build LSH index from concept names + aliases ...
[L276] merge_groups = find_connected_components(units, lsh)
[L277] return [create_canonical_node(group) for group in merge_groups]
[L278]
```

[L279]

[L280] ---

[L281]

[L282] ### 5.4 Layer 4: Relation Management

[L283]

[L284] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L285]

[L286] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L287]

[L288] **Pipeline:**

[L289] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L290] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L291] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L292] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L293] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L294]

[L295]

```
[L296] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L297] """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L298] conf = 0.5
[L299] conf += 0.15 * source.confidence
[L300] conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L301] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L302] return min(conf, 1.0)
[L303]
```

[L304]

[L305] ---

[L306]

[L307] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L308]

[L309] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L310]

[L311] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L312]

[L313] ##### Pipeline Overview

[L314]

[L315]

```
[L316] graph LR
[L317] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L318] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L319] PRE --> HV{"HV
Validate #1?"}
[L320] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L321] HV -->|"Fail"| HV2{"HV
Validate #2?"}
[L322] HV2 -->|"Pass"| FI
[L323] HV2 -->|"Fail"| HV3{"HV
Validate #3?"}
[L324] HV3 -->|"Pass"| FI
[L325] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L326] FI --> CLD["CLD
Discover Cross-Links"]
[L327] CLD --> DONE["Node Placed ✓"]
[L328]
```

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file4]`

[L329] metadata:

[L330] query\_result\_index: 4

[L331] file\_id: file\_000000004bc481fdbf9e949ec3b7544b

[L332] version\_id: 1

[L333] name: SHIA\_RAG\_Final\_Report(2).md

[L334] mime\_type: text/markdown

[L335] surface: library

[L336] score: 0.017476665593820405

[L337] document\_chunk\_id: file\_000000004bc481fdbf9e949ec3b7544b--1--14

[L338] snippet:

[L339] ##### Alias Resolution

[L340]

[L341] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L342]

[L343] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L344] 2. Find connected components of near-duplicate units

[L345] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L346]

[L347]



```
[L348] def alias_resolution(units: list) -> list:
[L349] """Merge KnowledgeUnits referring to the same concept.
[L350]
[L351] Merge strategy:
[L352] - Canonical name: most frequent name across units
[L353] - Aliases: union of all names and aliases
[L354] - Definitions: union of all definitions
[L355] - Confidence: average of unit extraction confidences
[L356] - Properties: merged (latest value wins, conflicts logged)
[L357] """
[L358] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L359] # ... build LSH index from concept names + aliases ...
[L360] merge_groups = find_connected_components(units, lsh)
[L361] return [create_canonical_node(group) for group in merge_groups]
[L362]
```

[L363]

[L364] ---

[L365]

[L366] ### 5.4 Layer 4: Relation Management

[L367]

[L368] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L369]

[L370] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L371]

[L372] **Pipeline:**

[L373] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L374] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L375] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L376] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L377] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L378]

[L379]

```
[L380] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L381] """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L382] conf = 0.5
[L383] conf += 0.15 * source.confidence
[L384] conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L385] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L386] return min(conf, 1.0)
[L387]
```

[L388]

[L389] ---

[L390]

[L391] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L392]

[L393] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L394]

[L395] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L396]

[L397] ##### Pipeline Overview

[L398]

[L399]

```
[L400] graph LR
[L401] N["New Node N"] --> CPG["CPG++
Generate ≤
parent candidates"]
[L402] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L403] PRE --> HV{"HV
Validate #1?"}
[L404] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L405] HV -->|"Fail"| HV2{"HV
Validate #2?"}
[L406] HV2 -->|"Pass"| FI
[L407] HV2 -->|"Fail"| HV3{"HV
Validate #3?"}
[L408] HV3 -->|"Pass"| FI
[L409] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L410] FI --> CLD["CLD
Discover Cross-Links"]
[L411] CLD --> DONE["Node Placed ✓"]
[L412]
```

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite[turn12file5]`

[L413] metadata:

[L414] query\_result\_index: 5

[L415] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L416] version\_id: 1

[L417] name: SHIA\_RAG\_Final\_Report.md

[L418] mime\_type: text/markdown

[L419] surface: library

[L420] score: 0.01724356536072017

[L421] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--13

[L422] snippet:

[L423] The parser uses rule-based heuristics for block classification:

[L424] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase

[L425] - **List:** Starts with `-`, `*`, `•`, or `1.`

[L426] - **Table:** Contains 3+ pipe characters ( `|` )

[L427] - **Code:** Contains Python keywords ( `def` , `class` , `import` )

[L428] - **Definition:** Contains phrases like "is defined as", "refers to"

[L429] - **Default:** Paragraph

[L430]

[L431] ---

[L432]

[L433] ### 5.3 Layer 3: Knowledge Extraction

[L434]

[L435] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical KnowledgeNode objects.

[L436]

[L437] **Input:** DocumentBlock[] → **Output:** KnowledgeNode[] (canonical, deduplicated)

[L438]

[L439] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.

[L440]

[L441] ##### Extraction Prompt

[L442]

[L443]

```
[L444] You are a knowledge extraction system. Given a text block, extract:
[L445] 1. Concepts: Named entities, technical terms
[L446] 2. Definitions: If the text defines any concept, extract the definition
[L447] 3. Properties: Key attributes with types and values
[L448] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L449] 5. Domain: Subject area (e.g., "Operating Systems")
[L450]
[L451] Respond in JSON format:
[L452] {
[L453] "concepts": [{"name": "...", "aliases": [...], "definitions": [...],
[L454] "properties": {"key": {"type": "...", "value": "...}}}],
[L455] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L456] "evidence": "...}],
[L457] "domain": "...
[L458] }
[L459]
```

[L460]

[L461] ##### Alias Resolution

[L462]

[L463] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L464]

[L465] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L466] 2. Find connected components of near-duplicate units

[L467] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L468]

[L469]

```
[L470] def alias_resolution(units: list) -> list:
[L471] """Merge KnowledgeUnits referring to the same concept.
[L472]
[L473] Merge strategy:
[L474] - Canonical name: most frequent name across units
[L475] - Aliases: union of all names and aliases
[L476] - Definitions: union of all definitions
[L477] - Confidence: average of unit extraction confidences
[L478] - Properties: merged (latest value wins, conflicts logged)
[L479] """
[L480] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L481] # ... build LSH index from concept names + aliases ...
[L482] merge_groups = find_connected_components(units, lsh)
[L483] return [create_canonical_node(group) for group in merge_groups]
[L484]
```

[L485]

[L486] ---  
[L487]  
[L488] ### 5.4 Layer 4: Relation Management  
[L489]  
[L490] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.  
[L491]  
[L492] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

---

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: `filecite{turn12file6}`

[L493] metadata:  
[L494] query\_result\_index: 6  
[L495] file\_id: file\_00000000f620821186e54e582d59731e  
[L496] version\_id: 1  
[L497] name: SHIA\_RAG\_Final\_Report(3).md  
[L498] mime\_type: text/markdown  
[L499] surface: library  
[L500] score: 0.01724356536072017  
[L501] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--13  
[L502] snippet:  
[L503] The parser uses rule-based heuristics for block classification:  
[L504] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase  
[L505] - **List:** Starts with `-`, `*`, `•`, or `1.`  
[L506] - **Table:** Contains 3+ pipe characters (`|`)  
[L507] - **Code:** Contains Python keywords (`def`, `class`, `import`)  
[L508] - **Definition:** Contains phrases like "is defined as", "refers to"  
[L509] - **Default:** Paragraph  
[L510]  
[L511] ---

[L512]  
[L513] ### 5.3 Layer 3: Knowledge Extraction  
[L514]  
[L515] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical `KnowledgeNode` objects.  
[L516]  
[L517] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)  
[L518]  
[L519] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.  
[L520]  
[L521] #### Extraction Prompt  
[L522]  
[L523]

```

[L524] You are a knowledge extraction system. Given a text block, extract:
[L525] 1. Concepts: Named entities, technical terms
[L526] 2. Definitions: If the text defines any concept, extract the definition
[L527] 3. Properties: Key attributes with types and values
[L528] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L529] 5. Domain: Subject area (e.g., "Operating Systems")
[L530]
[L531] Respond in JSON format:
[L532] {
[L533] "concepts": [{"name": "...", "aliases": [...], "definitions": [...],
[L534] "properties": {"key": {"type": "...", "value": "...}}}],
[L535] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L536] "evidence": "...}],
[L537] "domain": "...
[L538] }
[L539]

```

[L540]

[L541] ##### Alias Resolution

[L542]

[L543] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L544]

[L545] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L546] 2. Find connected components of near-duplicate units

[L547] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L548]

[L549]

```

[L550] def alias_resolution(units: list) -> list:
[L551] """Merge KnowledgeUnits referring to the same concept.
[L552]
[L553] Merge strategy:
[L554] - Canonical name: most frequent name across units
[L555] - Aliases: union of all names and aliases
[L556] - Definitions: union of all definitions
[L557] - Confidence: average of unit extraction confidences
[L558] - Properties: merged (latest value wins, conflicts logged)
[L559] """
[L560] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L561] # ... build LSH index from concept names + aliases ...
[L562] merge_groups = find_connected_components(units, lsh)
[L563] return [create_canonical_node(group) for group in merge_groups]
[L564]

```

[L565]

[L566] ---

[L567]

[L568] ### 5.4 Layer 4: Relation Management

[L569]

[L570] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L571]

[L572] **Input:** KnowledgeNode[] (with raw relations) → **Output:** KnowledgeEdge[] (validated, scored)

DEEP RESEARCH 20/08/2026, 14:29:07

Citation Marker: [filecite]turn12file7[

[L573] metadata:

[L574] query\_result\_index: 7

[L575] file\_id: file\_00000000f620821186e54e582d59731e

[L576] version\_id: 1

[L577] name: SHIA\_RAG\_Final\_Report(3).md

[L578] mime\_type: text/markdown

[L579] surface: library

[L580] score: 0.016797932562146196

[L581] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--6

[L582] snippet:

[L583]

```
[L584] graph TD
[L585] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L586] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L587] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L588] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L589] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L590] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L591] end
[L592]
[L593] subgraph "Phase 2: Online – Retrieval & Generation"
[L594] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L595] KF --> L6
[L596] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L597] L7 --> A["Answer + Citations"]
[L598] end
[L599]
[L600] subgraph "Phase 3: Continuous – Operations"
[L601] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L602] A --> L8
[L603] L8 -->|"Self-Evolution
Feedback"| KF
[L604] end
[L605]
```

[L606]

[L607] ### 3.2 Data Flow

[L608]

[L609]

```
[L610] Raw Document (PDF/DOCX/HTML)
[L611] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L612] Normalized Text + Metadata
[L613] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L614] [Page → Region → Block] hierarchy
[L615] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L616] KnowledgeUnit objects (candidate concepts)
[L617] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L618] KnowledgeNode objects (canonical, deduplicated)
[L619] ↓ Layer 4: Validate relations against ontology, score confidence
[L620] KnowledgeEdge objects (typed, scored)
[L621] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L622] Knowledge Forest (rooted trees + cross-links)
[L623] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L624] Selected context nodes (within token budget)
[L625] ↓ Layer 7: Build prompt, invoke LLM
[L626] ↓ Layer 7: Verify claims against evidence, bind citations
[L627] Answer with citations + confidence score
[L628] ↓ Layer 8: Log, collect feedback, update forest weights
[L629]
```

```
[L630]
[L631] ### 3.3 Layer Summary
[L632]
[L633] | Layer | Name | Input | Output | Key Algorithms |
[L634] |-----|-----|-----|-----|-----|
[L635] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L636] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L637] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L638] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L639] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L640] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L641] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L642] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L643] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L644]
[L645] ---
[L646]
[L647] ## 4. Data Model — Layer 0
```

CHATGPT 20/08/2026, 14:29:07

Layer 3 — Knowledge Extraction

Ab hum **same Smart Hospital example** continue karenge jo Layer 0, Layer 1 aur Layer 2 me use kiya tha.

Layer 3 ko samajhne ka sabse important point:

*Layer 2 document ka structure samajhta hai. Layer 3 document ke andar chhupa actual knowledge nikalta hai.*

Report ke according Layer 3 DocumentBlock[] se **concepts, definitions, properties aur candidate relations** extract karta hai; phir synonyms/aliases merge karke canonical, deduplicated KnowledgeNode objects banata hai.

📄filecite🔄turn12file0📄L21-L27📄

## 1. Layer 0 → Layer 1 → Layer 2 → Layer 3 connection

Pehle complete chain yaad karo:

```

Raw Smart Hospital PDF
↓
Layer 1
↓
NormalizedDocument
↓
Layer 2
↓
DocumentBlock[]
↓
Layer 3
↓
KnowledgeUnit[]
↓
Alias Resolution
↓
KnowledgeNode[]
↓
Layer 4

```

Report ke overall data flow me Layer 3 ke do major phases clearly hain:

1. **Concept/definition/relation extraction → KnowledgeUnit**
2. **Alias resolution using MinHash LSH → canonical KnowledgeNode** [filecite]turn12file7[L607-L620]

## 2. Layer 3 actually karta kya hai?

Maan lo Layer 2 ne ye paragraph diya:

```

The AI diagnostic engine analyzes patient symptoms,
medical history, and diagnostic images to estimate
emergency risk. The system uses a machine learning
model to generate a preliminary risk score.

```

Layer 2 ke liye ye sirf:

```

DocumentBlock
type = PARAGRAPH

```

tha.

Lekin Layer 3 isko dekhlega aur boleگا:



**Concepts:**

- AI diagnostic engine
- patient symptoms
- medical history
- diagnostic images
- emergency risk
- machine learning model
- preliminary risk score

**Relations:**

AI diagnostic engine  
USES  
machine learning model

AI diagnostic engine  
ANALYZES  
patient symptoms

AI diagnostic engine  
GENERATES  
preliminary risk score

**Yahi Layer 3 ka main transformation hai:**

STRUCTURED TEXT  
↓  
STRUCTURED KNOWLEDGE

### 3. Layer 3 ka exact input kya hai?

#### Input

DocumentBlock[]

Layer 2 se aane wale blocks.

Example:

```
[
 BLK_000120 → HEADING,
 BLK_000121 → PARAGRAPH,
 BLK_000122 → LIST,
 BLK_000123 → TABLE,
 BLK_000124 → ALGORITHM
]
```

Report ke according Layer 3 ka input exactly `DocumentBlock[]` hai. [filecite.it/turn12file0/L21-L27](https://filecite.it/turn12file0/L21-L27)

Ek individual block kuch aisa hoga:

```

DocumentBlock(
 block_id = "BLK_000121",
 doc_id = "DOC_000004",
 page_number = 15,
 block_type = PARAGRAPH,

 text = """
 The AI diagnostic engine analyzes patient symptoms,
 medical history, and diagnostic images to estimate
 emergency risk.
 """,

 reading_order = 121,
 confidence = 0.91
)

```

#### 4. Layer 3 ka high-level flow

```

DocumentBlock[]
 |
 ▼
Select relevant blocks
 |
 ▼
Extract concepts
 |
 ▼
Extract definitions
 |
 ▼
Extract properties
 |
 ▼
Extract raw/candidate relations
 |
 ▼
Assign source provenance
 |
 ▼
KnowledgeUnit[]
 |
 ▼
Alias / Synonym Resolution
 |
 ▼
Near-duplicate grouping
 |
 ▼
Canonical KnowledgeNode[]
 |
 ▼
Layer 4

```

#### 5. Layer 3 ka sabse important boundary: Layer 3 vs Layer 4

Ye bahut important hai.

##### Layer 3

Layer 3 ka kaam hai:

**Candidate extraction**

Meaning:

*Text me kya-kya knowledge dikh raha hai, wo extract karo.*

Example:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Layer 3 ye relation **candidate** ke roop me nikalta hai.

**Layer 4**

Layer 4 puchega:

```
Kya relation valid hai?
Kya USES ontology me allowed hai?
Kya target correct node hai?
Kya duplicate relation already exist karta hai?
Confidence kitna hai?
```

Report explicitly kehti hai:

*Layer 3 identifies entities, definitions, and raw relation triples, while Layer 4 validates them, resolves conflicts, and assigns confidence scores. [filecite\turn10file0\L11-L15](#)*

**Simple formula:**

```
Layer 3 = "Mujhe ye knowledge mila"
Layer 4 = "Kya ye knowledge valid aur trustworthy hai?"
```

**6. Layer 3 ka first step — kaunse blocks process honge?**

Report ke shown pipeline me:

```
for block in blocks:

 if block.block_type in (
 HEADING,
 FIGURE_CAPTION
):
 continue
```

Meaning:

```
HEADING → skip
FIGURE_CAPTION → skip
```

बाकी relevant blocks process honge. [filecite\turn10file0\L57-L73](#)

**Hamare example me**

```
BLK_000120
"3. EMERGENCY DIAGNOSIS"

Type = HEADING
 ↓
SKIP
```

```
BLK_000121
"The AI diagnostic engine analyzes..."

Type = PARAGRAPH
 ↓
PROCESS
```

```
BLK_000122
"1. Collect symptoms
 2. Extract features
 3. Predict risk"

Type = LIST
 ↓
PROCESS
```

```
BLK_000123
Risk table

Type = TABLE
 ↓
PROCESS
```

## 7. HEADING skip kyun kar rahe hain?

Heading:

```
EMERGENCY DIAGNOSIS
```

khud usually complete factual statement nahi hoti.

Agar har heading ko concept extraction ke liye directly bheja:

```
Concept = Emergency Diagnosis
```

possible hai.

Lekin report ke shown implementation me heading primary extraction input nahi hai. `filecite\turn10file0\L61-L63`

### Yahan ek important observation

**Potential limitation:** Heading context valuable ho sakta hai.

Example:

```
Heading:
AI-Assisted Emergency Diagnosis

Paragraph:
The system estimates patient risk...
```

Agar heading skip ho jaye, paragraph ka extraction us section context se lose ho sakta hai.

Isliye **report ke current shown logic me heading skip hota hai**, but future improvement me heading ko context metadata ke roop me pass karna useful ho sakta hai.

---

## 8. Layer 3 ka core data structure — KnowledgeUnit

Layer 3 pehle directly final KnowledgeNode nahi banata.

Pehle ek intermediate object banta hai:

```
KnowledgeUnit
```

Report ke according:

```
KnowledgeUnit = Atomic fact extracted from a single document block
```

aur ye Layer 3 extraction aur later hierarchy-related processing ke beech intermediate representation hai.

filecite[turn10file0]L21-L34

---

## 9. KnowledgeUnit ke saare fields

```
KnowledgeUnit
├── unit_id
├── concept_name
├── aliases
├── definitions
├── properties
├── raw_relations
├── source_doc
├── source_block
├── page
├── extraction_confidence
└── domain_hint
```

Report me ye fields explicitly defined hain. filecite[turn10file0]L21-L34

Ab ek-ek karke samjhte hain.

---

### 9.1 unit\_id

Example:

```
KU_000001
```

Ye individual extracted knowledge unit ki identity hai.

---

### 9.2 concept\_name

Example:

```
AI Diagnostic Engine
```

Ye actual concept hai jo text se extract hua.

---

### 9.3 aliases

Same concept ke alternative names.

Example document me:

```
AI Diagnostic Engine
AI Diagnosis System
Diagnostic AI
```

ho sakte hain.

Initial unit:

```
aliases = [
 "AI Diagnosis System",
 "Diagnostic AI"
]
```

### 9.4 definitions

Agar document ne concept define kiya hai:

```
AI-assisted diagnosis is the use of artificial
intelligence to support clinical decision-making.
```

To:

```
definitions = [
 "AI-assisted diagnosis is the use of artificial intelligence
 to support clinical decision-making."
]
```

### 9.5 properties

Concept ki key attributes.

Example:

```
Machine Learning Model
```

Properties:

```
{
 "model_type": {
 "type": "string",
 "value": "classification"
 },
 "output": {
 "type": "string",
 "value": "risk score"
 }
}
```

Report extraction prompt explicitly key attributes with types and values extract karne ko kehta hai.

filecite\turn12file0\L29-L46

## 9.6 raw\_relations

Ye bahut important hai.

Example:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Stored conceptually as:

```
(
 "USES",
 "Machine Learning Model",
 "The AI diagnostic engine uses a machine learning model..."
)
```

Ye final validated relation nahi hai.

Isliye naam:

```
raw_relations
```

hai.

Layer 4 later verify karega.

---

## 9.7 source\_doc

```
DOC_000004
```

Knowledge kis document se nikla?

---

## 9.8 source\_block

```
BLK_000121
```

Knowledge exactly kis block se nikla?

---

## 9.9 page

```
15
```

Later citation/evidence tracking ke liye.

Complete provenance:

```
Concept
 ↓
KnowledgeUnit KU_000001
 ↓
Document DOC_000004
 ↓
Block BLK_000121
 ↓
Page 15
```

### 9.10 extraction\_confidence

Example:

0.87

Meaning:

*Extractor ko apni extraction par kitna confidence hai.*

**Important:** Report ke data model me field present hai. But exact confidence-calculation method shown Layer 3 snippet me fully defined nahi hai. `filecite\turn10file0\L30-L34`

### 9.11 domain\_hint

Example:

Healthcare AI

Ya:

Medical Informatics

Ye LLM-suggested subject/domain hint hai. `filecite\turn10file0\L30-L34`

## 10. Hamare Smart Hospital example ka actual input

Layer 2 se:

```
BLK_000121

Type:
PARAGRAPH

Text:
"The AI diagnostic engine analyzes patient symptoms,
medical history, and diagnostic images to estimate
emergency risk. The system uses a machine learning
model to generate a preliminary risk score."
```

## 11. Step 1 — LLM-based Knowledge Extraction

Layer 3 report ke according ek extraction prompt banata hai.

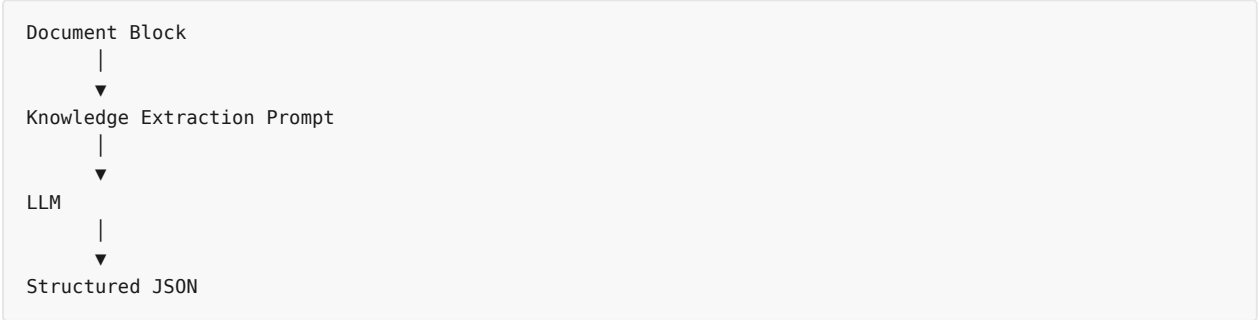
LLM se specifically ye 5 things maangi jaati hain:



1. Concepts
  2. Definitions
  3. Properties
  4. Relations
  5. Domain

Output JSON format me expected hai. `[{"filecite": "turn12file0", "L29-L46"}]`

Conceptually:



12. Hamare example ke liye LLM output

Input:

"The AI diagnostic engine analyzes patient symptoms, medical history, and diagnostic images to estimate emergency risk. The system uses a machine learning model to generate a preliminary risk score."

Potential extraction:

```

{
 "concepts": [
 {
 "name": "AI Diagnostic Engine",
 "aliases": [
 "AI Diagnosis System"
],
 "definitions": [],
 "properties": {
 "purpose": {
 "type": "string",
 "value": "estimate emergency risk"
 }
 }
 },
 {
 "name": "Machine Learning Model",
 "aliases": [],
 "definitions": [],
 "properties": {
 "output": {
 "type": "string",
 "value": "preliminary risk score"
 }
 }
 },
 {
 "name": "Emergency Risk",
 "aliases": [
 "patient risk"
],
 "definitions": [],
 "properties": {}
 }
],
 "relations": [
 {
 "source": "AI Diagnostic Engine",
 "type": "USES",
 "target": "Machine Learning Model",
 "evidence": "The system uses a machine learning model."
 },
 {
 "source": "AI Diagnostic Engine",
 "type": "ESTIMATES",
 "target": "Emergency Risk",
 "evidence": "The AI diagnostic engine ... estimates emergency risk."
 }
],
 "domain": "Healthcare AI"
}

```

**Note:** Ye hamare common example ka illustrative extraction hai; exact report ka example nahi.

### 13. Concepts exactly kya hote hain?

Layer 3 har noun ko blindly concept nahi banata.

Target:

Named entities  
 Technical terms  
 Important ideas  
 Domain concepts

Report extraction prompt me exactly isi type ki extraction defined hai. `filecite\turn12file0\L31-L37`

Example:

Sentence:

The AI diagnostic engine analyzes patient symptoms.

Possible concepts:

AI Diagnostic Engine  
 Patient Symptoms

But generally:

the  
 a  
 system  
 data  
 process

jaise generic words ideally concepts nahi banne chahiye.

## 14. Concept extraction me possible error

LLM ye output de:

Concepts:  
 AI  
 System  
 Data  
 Patient  
 Risk  
 Model

Problem:

Too generic

Agar har word node ban gaya:

Knowledge Forest  
 ↓  
 Thousands of meaningless nodes

Example:

System  
 Data  
 Information  
 Process  
 Method

ye har document me aa sakte hain.

**Result:**

```
Knowledge pollution
```

Isliye concept extraction quality Layer 3 ki critical dependency hai.

---

## 15. Definition extraction

Example block:

```
"AI-assisted diagnosis is defined as the use of
artificial intelligence techniques to support clinical
decision-making."
```

Layer 3 should produce:

```
Concept:
AI-assisted diagnosis

Definition:
the use of artificial intelligence techniques
to support clinical decision-making
```

---

## 16. Definition ka importance

Later user pooche:

| *What is AI-assisted diagnosis?*

Traditional RAG me system relevant text chunk dhundhega.

SHIA-RAG ka target structure:

```
KnowledgeNode:
AI-Assisted Diagnosis

Definition:
...
```

Isliye retrieval more concept-centric ho sakta hai.

---

## 17. Properties extraction

Example:

```
"The emergency risk model produces a score from 0 to 100."
```

Extract:

```
Concept:
Emergency Risk Model

Property:
score_range

Value:
0 to 100
```

Structured form:

```
{
 "score_range": {
 "type": "numeric_range",
 "value": "0-100"
 }
}
```

## 18. Properties aur relations ka difference

### Property

```
Risk Model
 score_range = 0-100
```

Ye concept ki internal attribute hai.

### Relation

```
Risk Model
 USES
 Patient Symptoms
```

Ye do concepts ke beech connection hai.

Simple:

```
Property
= Concept ke andar ki information

Relation
= Concept aur doosre concept ka connection
```

## 19. Raw Relation Extraction

Layer 3 relation ko directly final truth nahi maanta.

Example:

```
AI Diagnostic Engine
 USES
 Machine Learning Model
```

Raw relation:

```
source = AI Diagnostic Engine
type = USES
target = Machine Learning Model
evidence = original sentence
```

Report me relation types ke examples include hain:

```
IS_A
PART_OF
USES
CAUSES
```

filecite turn12file0 L33-L45

## 20. Evidence text kyun store karna important hai?

Example:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Evidence:

"The system uses a machine learning model to generate a preliminary risk score."

Later agar Layer 4 puche:

*Tumne ye relation kahan se nikala?*

To evidence available hai.

Without evidence:

```
Relation exists
But source unclear
```

This harms traceability.

## 21. Layer 3 ka first intermediate output — KnowledgeUnit[]

Ab hamare ek block se:

```
KU_000001
concept_name = AI Diagnostic Engine
```

```
KU_000002
concept_name = Machine Learning Model
```

```
KU_000003
concept_name = Emergency Risk
```

ban sakte hain.

So:

```
One DocumentBlock
 ↓
Multiple KnowledgeUnits
```

## 22. Same document ke doosre page ka example

Page 27:

```
"Diagnostic AI continuously evaluates patient data
and assigns a preliminary emergency score."
```

Layer 3 extract karega:

```
KU_000045

concept_name:
Diagnostic AI

aliases:
[]

properties:
{
 "output": "preliminary emergency score"
}
```

## 23. Ab problem — Duplicate concepts

Ab system ke paas:

```
KU_000001
AI Diagnostic Engine
```

Aur:

```
KU_000045
Diagnostic AI
```

Aur kisi aur document me:

```
KU_000089
AI Diagnosis System
```

Question:

```
3 different concepts?
OR
Same concept ke 3 names?
```

Yahi Layer 3 ka **second major phase** solve karta hai:

## Alias Resolution

## 24. Alias Resolution kya hai?

Report ka example:

```
RR
Round Robin
Round-Robin Scheduling
```

same concept ko refer kar sakte hain. `filecite\turn12file0\L49-L55`

Hamare example me:

```
AI Diagnostic Engine
Diagnostic AI
AI Diagnosis System
```

Possible same concept.

Without merging:

```
KnowledgeNode 1
AI Diagnostic Engine

KnowledgeNode 2
Diagnostic AI

KnowledgeNode 3
AI Diagnosis System
```

Retrieval confused ho sakta hai.

## 25. Alias Resolution ka exact flow

Report:

1. MinHash LSH index build karo
2. Near-duplicate units ke connected components find karo
3. Har group ko merge karo

Threshold:

```
0.7
```

aur `num_perm=128` shown hai. `filecite\turn12file0\L49-L71`

Flow:

```
KnowledgeUnit A
"AI Diagnostic Engine"
|
KnowledgeUnit B
"Diagnostic AI"
|
KnowledgeUnit C
"AI Diagnosis System"
↓
MinHash LSH
↓
Similarity / candidate grouping
↓
Connected Component
↓
One Canonical KnowledgeNode
```

## 26. MinHash LSH yahan kyun?

Suppose 10 lakh concepts hain.



Har concept ko har doosre concept se compare karna:

$O(n^2)$

expensive ho sakta hai.

MinHash LSH ka idea:

*Pehle likely similar candidates quickly find karo.*

Then:

```

All concepts
 ↓
Likely similar groups
 ↓
Merge candidates

```

Report specifically alias resolution ke liye `MinHashLSH(threshold=0.7, num_perm=128)` specify karti hai.

`filecite\turn12file0\L57-L71`

## 27. Connected Components kya role play karta hai?

Example:

```

A ↔ B
B ↔ C

```

Even if:

A directly C se match nahi karta

Still:

```

A
|
B
|
C

```

ek connected group ban sakta hai.

Then:

```

[A, B, C]
 ↓
One merged concept

```

## 28. Canonical Name kaise choose hota hai?

Report ke merge strategy ke according:

Most frequent name

canonical name banta hai. `filecite\turn12file0\L61-L67`

Suppose:

```
AI Diagnostic Engine → 12 occurrences
Diagnostic AI → 7 occurrences
AI Diagnosis System → 2 occurrences
```

Canonical:

```
AI Diagnostic Engine
```

## 29. Merge ke baad aliases kya honge?

Before:

```
KU_1
Name = AI Diagnostic Engine

KU_2
Name = Diagnostic AI

KU_3
Name = AI Diagnosis System
```

After:

```
KnowledgeNode

Canonical Name:
AI Diagnostic Engine

Aliases:
[
 Diagnostic AI,
 AI Diagnosis System
]
```

Report strategy me aliases ke liye all names + aliases ka union use hota hai. `filecite\turn12file0\L61-L67`

## 30. Definitions merge kaise hoti hain?

Suppose:

```
Definition 1:
AI diagnostic engine analyzes medical information.

Definition 2:
Diagnostic AI supports clinical risk assessment.
```

Report:

```
Definitions = union
```

karti hai. `filecite\turn12file0\L61-L67`

Meaning:

```
KnowledgeNode
├─ Definition 1
└─ Definition 2
```

## 31. Properties merge kaise hoti hain?

Report ke according:

```
Properties merged
Latest value wins
Conflicts logged
```

filecite turn12file0 L61-L67

Example:

```
Unit A:
model_type = classification

Unit B:
model_type = deep neural network
```

Agar conflict resolution simply:

```
latest wins
```

hai, to:

```
Final = latest value
```

But:

```
Conflict logged
```

hona important hai.

---

## 32. Ye ek potential weakness bhi hai

Technical observation:

"Latest value wins" always semantically correct nahi hai.

Example:

Document A:

```
model_version = v1
```

Document B:

```
model_version = v2
```

Latest wins → reasonable.

But:

```
Property A:
supports X = true

Property B:
supports X = false
```

Yahan blindly latest choose karna dangerous ho sakta hai.

Report Layer 4 ko relation validation/conflict handling ke liye use karti hai, lekin property-conflict semantics ke liye shown Layer 3 merge strategy limited hai. `filecite\turn12file0\L61-L67`

### 33. Extraction Confidence merge

Report ke according:

```
Final confidence
=
average of unit extraction confidences
```

`filecite\turn12file0\L61-L67`

Example:

```
KU_1 = 0.90
KU_2 = 0.80
KU_3 = 0.85
```

Final:

```
(0.90 + 0.80 + 0.85) / 3
= 0.85
```

### 34. Hamare Smart Hospital example ka complete Layer 3 walkthrough

#### Input from Layer 2

```
BLK_000121

The AI diagnostic engine analyzes patient symptoms,
medical history, and diagnostic images to estimate
emergency risk. The system uses a machine learning
model to generate a preliminary risk score.
```

#### Step 1 — Relevant block check

```
Block type = PARAGRAPH
```

Not:

```
HEADING
FIGURE_CAPTION
```

Therefore:

```
PROCESS ✓
```

#### Step 2 — Extraction prompt

System asks LLM:

Extract:

Concepts  
Definitions  
Properties  
Relations  
Domain

---

### Step 3 — LLM response

Concepts:  
AI Diagnostic Engine  
Machine Learning Model  
Emergency Risk  
Patient Symptoms  
Diagnostic Images  
Preliminary Risk Score

Relations:  
AI Diagnostic Engine  
    USES  
Machine Learning Model

AI Diagnostic Engine  
    ANALYZES  
Patient Symptoms

AI Diagnostic Engine  
    ESTIMATES  
Emergency Risk

Machine Learning Model  
    GENERATES  
Preliminary Risk Score

Domain:  
Healthcare AI

---

### Step 4 — KnowledgeUnits create karo

KU\_000001  
Concept:  
AI Diagnostic Engine

KU\_000002  
Concept:  
Machine Learning Model

KU\_000003  
Concept:  
Emergency Risk

Har unit ke saath:

source\_doc  
source\_block  
page  
confidence  
domain\_hint

attach honge.

---

## Step 5 — Next blocks process karo

Page 27:

```
"Diagnostic AI continuously evaluates patient data."
```

Extract:

```
KU_000045
Concept:
Diagnostic AI
```

Page 39:

```
"The AI Diagnosis System provides risk predictions."
```

Extract:

```
KU_000089
Concept:
AI Diagnosis System
```

---

## Step 6 — Alias Resolution

System sees:

```
AI Diagnostic Engine
Diagnostic AI
AI Diagnosis System
```

MinHash LSH candidate grouping:

```
Group 1:
[
 AI Diagnostic Engine,
 Diagnostic AI,
 AI Diagnosis System
]
```

---

## Step 7 — Canonical node

Output:

```
KnowledgeNode

Canonical Name:
AI Diagnostic Engine

Aliases:
[
 Diagnostic AI,
 AI Diagnosis System
]

Definitions:
[
 ...
]

Properties:
{
 ...
}

Evidence:
Multiple source blocks

Confidence:
Merged average

Domain:
Healthcare AI
```

### 35. Layer 3 ka complete output

Final output:

```
KnowledgeNode[]
```

Example:

```
[
 KnowledgeNode(
 name = "AI Diagnostic Engine",
 aliases = [...],
 definitions = [...],
 properties = {...}
),

 KnowledgeNode(
 name = "Machine Learning Model",
 aliases = [...],
 definitions = [...],
 properties = {...}
),

 KnowledgeNode(
 name = "Emergency Risk",
 aliases = [...],
 definitions = [...],
 properties = {...}
)
]
```

Report ke layer summary me bhi Layer 3 ka output canonical, deduplicated `KnowledgeNode[]` diya gaya hai.

📄filecite🔄turn1file0📄L21-L27📄

### 36. Layer 3 me exactly kaunse algorithms/techniques use hote hain?

| Step | Algorithm / Technique |  
 |---|---|  
 | Knowledge extraction | LLM-based structured extraction |  
 | Output formatting | JSON |  
 | Failure fallback | Rule-based extraction |  
 | Concept creation | One `KnowledgeUnit` per extracted concept |  
 | Relation extraction | Candidate/raw triples |  
 | Synonym detection | MinHash LSH |  
 | Group formation | Connected Components |  
 | Canonicalization | Most frequent name |  
 | Alias merging | Set/union |  
 | Definition merging | Set/union |  
 | Confidence merging | Average |  
 | Property conflict | Latest value wins + logging |

Ye techniques report ke Layer 3 extraction aur alias-resolution sections me defined hain. `filecite\turn10file0\L36-L86` `filecite\turn12file0\L49-L71`

### 37. Layer 3 ka pseudocode — simple form



```

knowledge_units = []

FOR each DocumentBlock:

 IF block is HEADING or FIGURE_CAPTION:
 skip

 TRY:
 extract structured JSON using LLM

 EXCEPT:
 use rule-based extraction

 FOR each extracted concept:

 create KnowledgeUnit

 attach:
 aliases
 definitions
 properties
 raw relations
 source document
 source block
 page
 extraction confidence
 domain

 add KnowledgeUnit

AFTER all blocks:

 perform alias resolution

 find similar/synonymous units

 group connected units

 merge each group

 create canonical KnowledgeNodes

RETURN KnowledgeNode[]

```

Ye report ke shown extraction pipeline aur alias-resolution flow ka combined representation hai.

📄filecite🔄turn10file0📄L57-L86📄 📄filecite🔄turn12file0📄L49-L71📄

### 38. Exception Handling — Layer 3 me explicitly kya diya hai?

Ye Layer 3 me clearly defined hai.

Report code:

```

TRY:
 LLM se JSON response lo
 JSON parse karo

EXCEPT:
 JSONDecodeError
 LLMLTimeout

↓

Rule-based extraction fallback

```

filecite\turn10file0\L65-L73

### 39. Exception 1 — LLM Timeout

Suppose:

```
Request sent
 ↓
LLM overloaded
 ↓
Timeout
```

System:

```
LLMTimeout
 ↓
rule_based_extract(block.text)
```

So:

```
Pipeline stops nahi hoti ✓
Block silently lost nahi hota ✓
Fallback extraction hoti hai ✓
```

### 40. Exception 2 — Invalid JSON

LLM expected:

```
{
 "concepts": [...],
 "relations": [...]
}
```

But returns:

```
Sure! Here is the extracted information:

{
 "concepts": ...
}
```

Or malformed JSON:

```
{
 "concepts": [
 ...
]
}
```

Then:

```
json.loads()
 ↓
JSONDecodeError
 ↓
Fallback
```

Report explicitly ye fallback define karti hai. filecite\turn10file0\L65-L73

## 41. Rule-based fallback ka limitation

Important:

LLM extraction

and

rule\_based extraction

same quality guarantee nahi dete.

LLM:

Complex semantic relation samajh sakta hai

Rule-based:

Patterns detect karega

So fallback ke baad extraction quality reduce ho sakti hai.

Report shown snippet fallback function ko call karta hai, lekin `rule_based_extract()` ka detailed algorithm us extracted snippet me fully specified nahi hai. `filecite\turn10file0\L65-L73`

Isliye exact fallback logic ko assume nahi karna chahiye.

---

## 42. Layer 3 me possible errors — Error 1: Hallucinated Concept

Input:

The system analyzes patient symptoms.

Correct concepts:

System  
Patient Symptoms

LLM hallucinate kar de:

Disease Prediction Model ❌

Jab text me model mention hi nahi tha.

Impact:

Fake KnowledgeUnit  
↓  
Alias merge  
↓  
KnowledgeNode  
↓  
Layer 4  
↓  
Potential graph pollution

### 43. Error 2 — Concept miss ho jana

Input:

The system uses federated learning for privacy-preserving model training.

Expected:

Federated Learning  
Privacy-Preserving Model Training

LLM extract kare:

Federated Learning only

Then second concept lost.

---

### 44. Error 3 — Over-extraction

Input:

The model processes patient data.

LLM:

The  
Model  
Processes  
Patient  
Data

sab concepts bana de.

Result:

Too many noisy nodes

---

### 45. Error 4 — Wrong Relation Direction

Sentence:

Machine Learning Model is used by AI Diagnostic Engine.

Correct:

AI Diagnostic Engine  
USES  
Machine Learning Model

Possible wrong extraction:

Machine Learning Model  
USES  
AI Diagnostic Engine ❌

Yahi reason hai ki Layer 3 ka relation **raw candidate** hai aur Layer 4 validation karta hai. `filecite\turn10file0\L11-L15`

---

## 46. Error 5 — Wrong Relation Type

Sentence:

The diagnostic module is part of the Smart Hospital platform.

Correct candidate:

Diagnostic Module  
PART\_OF  
Smart Hospital Platform

Possible extraction:

Diagnostic Module  
IS\_A  
Smart Hospital Platform ❌

Again:

Layer 3 extracts  
Layer 4 validates

---

## 47. Error 6 — False Alias Merge

Suppose:

Risk Score

and:

Risk Model

Similarity high ho sakti hai.

But:

Risk Score  $\neq$  Risk Model

Agar MinHash LSH wrongly same group bana de:

False Merge

Then two different concepts permanently-ish same canonical node me merge ho sakte hain.

---

## 48. Error 7 — Missed Alias

AI Diagnostic Engine

and:

Clinical Intelligence Module

Actually same system ho sakte hain.

But lexical overlap low:

Similarity < 0.7

Then:

No merge

Duplicate nodes remain.

## 49. Error 8 — Chain merge problem

Suppose:

A ~ B  
B ~ C

Connected component merge karega:

[A, B, C]

But possibility:

A and C directly actually same concept na hon.

This is a known transitivity risk in similarity-based connected-component clustering.

Report connected components use karti hai, so ye potential technical risk hai. [filecite\turn12file0\L51-L55](#)

## 50. Error 9 — Property conflict

Example:

Document A:  
Risk threshold = 70  
  
Document B:  
Risk threshold = 80

Report strategy:

Latest value wins  
Conflicts logged

[filecite\turn12file0\L61-L67](#)

Problem:

*Latest document necessarily correct nahi hota.*

Maybe:

A = global threshold  
B = one department threshold

Context missing.

## 51. Error 10 — Wrong confidence

LLM says:

0.95 confidence

But extraction incorrect.

Therefore:

LLM confidence  
≠  
Ground-truth correctness

Ye Layer 3 confidence ka fundamental limitation hai.

Report field store karti hai, but Layer 4 relation-level scoring/validation ka additional layer rakhti hai.

`filecite\turn10file0\L11-L15`

## 52. Layer 3 safeguards

### Report-defined safeguards

#### 1. Structured JSON output

Free text ke instead expected structure:

concepts  
relations  
domain

`filecite\turn12file0\L39-L46`

#### 2. Exception fallback

JSON error  
OR  
LLM timeout  
  
↓  
Rule-based extraction

`filecite\turn10file0\L65-L73`

#### 3. Provenance preservation

Every unit gets:

source\_doc  
source\_block  
page

filecite[turn10file0L29-L34]

#### 4. Raw relation preservation

Layer 3 relation ko final truth nahi banata.

Layer 4 validation ke liye forward karta hai. filecite[turn10file0L11-L15]

#### 5. Conflict logging

Property merge me conflicts log karne ka strategy diya gaya hai. filecite[turn12file0L61-L67]

---

### 53. Report me Layer 3 ki missing/unclear parts

Ab critical analysis.

#### Missing Part 1 — Sentence splitting detail

Overall data flow me:

Layer 3: Split sentences, extract concepts/definitions/relations

likha hai. filecite[turn12file7L613-L618]

Lekin shown extraction pseudocode primarily:

```
block.text
↓
directly LLM prompt
```

bhejta hai. filecite[turn10file0L65-L69]

#### Question:

Exact sentence splitting:

```
Kaunsa tokenizer?
spaCy?
NLTK?
Rule-based?
Multilingual handling?
```

Shown code me fully defined nahi hai.

---

#### Missing Part 2 — Exact `rule_based_extract()` implementation

Fallback call hai:

```
rule_based_extract(block.text)
```

But shown Layer 3 section me iska full algorithm visible nahi hai.

So currently exact details unclear:



Concept extraction rule?  
Definition rule?  
Relation patterns?  
NER?  
Dependency parsing?

### Missing Part 3 — LLM extraction confidence calculation

Field:

extraction\_confidence

hai.

But:

0.87 exactly kaise calculate hua?  
Self-reported?  
Probability?  
Schema confidence?  
Multiple runs?

shown implementation me fully specified nahi hai.

### Missing Part 4 — False merge recovery

Suppose:

Risk Score

and:

Risk Model

wrongly merge ho gaye.

Report shown alias-resolution flow me explicit:

automatic unmerge mechanism

detail nahi dikhata.

### Missing Part 5 — Alias similarity beyond lexical similarity

MinHash LSH concept names + aliases ke liye useful hai. `filecite\turn12file0\L49-L71`

But:

AI Diagnostic Engine  
Clinical Intelligence Module

same entity ho sakte hain but word overlap very low.

So lexical/near-duplicate approach alone sufficient nahi ho sakta.

## 54. Layer 3 ka complete end-to-end example

### Input

BLK\_000121

"The AI diagnostic engine analyzes patient symptoms, medical history, and diagnostic images to estimate emergency risk. The system uses a machine learning model to generate a preliminary risk score."

### LLM Extraction

Concepts:

- AI Diagnostic Engine
- Patient Symptoms
- Medical History
- Diagnostic Images
- Emergency Risk
- Machine Learning Model
- Preliminary Risk Score

Relations:

AI Diagnostic Engine  
ANALYZES

Patient Symptoms

AI Diagnostic Engine  
ANALYZES

Medical History

AI Diagnostic Engine  
ANALYZES

Diagnostic Images

AI Diagnostic Engine  
USES

Machine Learning Model

AI Diagnostic Engine  
ESTIMATES

Emergency Risk

Machine Learning Model  
GENERATES

Preliminary Risk Score

### Create KnowledgeUnits

|                        |
|------------------------|
| KU_001                 |
| AI Diagnostic Engine   |
| KU_002                 |
| Patient Symptoms       |
| KU_003                 |
| Medical History        |
| KU_004                 |
| Diagnostic Images      |
| KU_005                 |
| Emergency Risk         |
| KU_006                 |
| Machine Learning Model |
| KU_007                 |
| Preliminary Risk Score |

More documents processed

Later:

|                     |
|---------------------|
| Diagnostic AI       |
| AI Diagnosis System |

appear.

Alias Resolution

|                      |
|----------------------|
| AI Diagnostic Engine |
| ↕                    |
| Diagnostic AI        |
| ↕                    |
| AI Diagnosis System  |

Canonical Node

```

KnowledgeNode

Name:
AI Diagnostic Engine

Aliases:
- Diagnostic AI
- AI Diagnosis System

Definitions:
[...]

Properties:
{
 purpose:
 "estimate emergency risk"
}

Evidence:
Multiple source blocks

Confidence:
Merged average

Domain:
Healthcare AI

```

## 55. Layer 3 output Layer 4 ko kaise jata hai?

Layer 3 ke baad:

```
KnowledgeNode[]
```

milta hai.

Conceptually:

```

Layer 3
 ↓
AI Diagnostic Engine
Machine Learning Model
Emergency Risk
Patient Symptoms
...
 ↓
KnowledgeNode[]
 ↓
Layer 4

```

Layer 4 phir candidate relations ko ontology ke against validate karega aur scored `KnowledgeEdge` objects banayega.

filecite[turn12file2][L198-L209]

## 56. Layer 3 kya nahi karta?

Layer 3:

- × final relation validity decide nahi karta
- × relation confidence ka final scoring nahi karta
- × ontology validation nahi karta

- × Knowledge Forest hierarchy nahi banata
- × parent-child placement nahi karta

Layer 3 ka scope:

```

TEXT
↓
KNOWLEDGE CANDIDATES
↓
DE-DUPLICATED CONCEPT NODES

```

## 57. Layer 2 vs Layer 3 — sabse important difference

| Layer 2 | Layer 3 |

|---|---|

| Structure samajhta hai | Meaning/knowledge extract karta hai |

| Heading identify karta hai | Concept identify karta hai |

| Table identify karta hai | Table se knowledge extract karta hai |

| Paragraph identify karta hai | Paragraph se concepts/relations nikalta hai |

| `DocumentBlock[]` output | `KnowledgeUnit[]` → `KnowledgeNode[]` |

| “Ye content kya type hai?” | “Is content ka actual knowledge kya hai?” |

## 58. Final memory trick

```

Layer 1
DOCUMENT KO CLEAN KARTA HAI
↓
Layer 2
DOCUMENT KO STRUCTURE KARTA HAI
↓
Layer 3
DOCUMENT KO KNOWLEDGE ME CONVERT KARTA HAI
↓
Layer 4
KNOWLEDGE KO VALIDATE KARTA HAI

```

Ek line me:

*Layer 3 SHIA-RAG ka woh layer hai jahan normal document content pehli baar actual structured knowledge me convert hota hai.*

## Layer 3 ka final pipeline

```

DocumentBlock[]
 ↓
Skip non-content blocks
 ↓
LLM-based structured extraction
 ↓
Concepts
Definitions
Properties
Raw Relations
Domain
 ↓
KnowledgeUnit[]
 ↓
MinHash LSH
 ↓
Connected Components
 ↓
Alias/Synonym Merge
 ↓
Canonical Name Selection
 ↓
KnowledgeNode[]
 ↓
Layer 4

```

Next logical step Layer 4 hai, jahan hum in `KnowledgeNode` aur unki raw relations ko **validate, resolve aur confidence score** karenge.

YOU 20/08/2026, 15:06:12

layer 4 smjhao

CHATGPT 20/08/2026, 15:06:18

```
{
 "paths": ["files"],
 "query": "search"
}
```

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: `filecite`turn13file0

[L1] metadata:

[L2] query\_result\_index: 0

[L3] file\_id: file\_00000000beeb4821185577e08118db227

[L4] version\_id: 1

[L5] name: Executive Summary (1)(6).pdf

[L6] mime\_type: application/pdf

[L7] surface: library

[L8] score: 0.02153443766346992

[L9] document\_chunk\_id: file\_00000000beeb4821185577e08118db227:0--1--18

[L10] snippet:

[L11] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L12] Use knowledge bases: e.g. Wikipedia links.

[L13] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L14] Context Assembler may include one-hop neighbors via CLD).

[L15] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

- [L16] additional edges are “related-to.”
- [L17] Data Structures and Indexing
- [L18] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent
- [L19] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),
- [L20] embedding vector, and a version/timestamp.
- [L21] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)
- [L22] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.
- [L23] NetworkX, Neo4j).
- [L24] •
- [L25] •
- [L26] •
- [L27] •
- [L28] •
- [L29] •
- [L30] •
- [L31] •
- [L32] •
- [L33] •
- [L34] •
- [L35] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node
- [L36] embeddings. This supports CPG++ search and semantic neighbor queries.
- [L37] Ancillary Indexes:
- [L38] Entity-to-Node map: For quick lookups by name/alias.
- [L39] Relation Index: If needed, a mapping from relation type to node pairs.
- [L40] Metadata store: (document IDs, source text) for evidence retrieval.
- [L41] All indices are updated incrementally as nodes are added.
- [L42] Retrieval and Context Assembly
- [L43] 10. Query Analyzer
- [L44] Input: Natural language user query.
- [L45] Output: Query intent and candidate target concepts.
- [L46] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. “Explain X”
- [L47] suggests retrieving up the hierarchy) or specific (e.g. “Symptoms of disease Y” suggests looking at children).
- [L48] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept
- [L49] matching (e.g. if query mentions “Round Robin”, we directly target that node).
- [L50] 11. Semantic Retrieval Engine
- [L51] Input: Query intent, key terms.
- [L52] Output: Set of relevant KnowledgeNodes.
- [L53] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with
- [L54] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).
- [L55] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:
- [L56] Parent Expansion: If the query is general, include some ancestors of the matched nodes.
- [L57] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For
- [L58] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related
- [L59] knowledge.)
- [L60] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG’s
- [L61] semantic paths【29†L37-L44】).

[L62] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example  
 [L63] sentences, and optionally a context snippet from source documents.  
 [L64] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a  
 [L65] subset of these nodes and their evidence that maximizes informational coverage under a token limit.  
 [L66] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: □filecite□turn13file1□

[L67] metadata:

[L68] query\_result\_index: 1

[L69] file\_id: file\_00000000377082068d7078ad6d943179

[L70] version\_id: 1

[L71] name: Executive Summary (1)(5).pdf

[L72] mime\_type: application/pdf

[L73] surface: library

[L74] score: 0.02115415788620522

[L75] document\_chunk\_id: file\_00000000377082068d7078ad6d943179:0--1--18

[L76] snippet:

[L77] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L78] Use knowledge bases: e.g. Wikipedia links.

[L79] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L80] Context Assembler may include one-hop neighbors via CLD).

[L81] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L82] additional edges are "related-to."

[L83] Data Structures and Indexing

[L84] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L85] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L86] embedding vector, and a version/timestamp.

[L87] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L88] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L89] NetworkX, Neo4j).

[L90] •

[L91] •

[L92] •

[L93] •

[L94] •

[L95] •

[L96] •

[L97] •

[L98] •

[L99] •

[L100] •

[L101] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L102] embeddings. This supports CPG++ search and semantic neighbor queries.

[L103] Ancillary Indexes:



- [L104] Entity-to-Node map: For quick lookups by name/alias.
- [L105] Relation Index: If needed, a mapping from relation type to node pairs.
- [L106] Metadata store: (document IDs, source text) for evidence retrieval.
- [L107] All indices are updated incrementally as nodes are added.
- [L108] Retrieval and Context Assembly
- [L109] 10. Query Analyzer
- [L110] Input: Natural language user query.
- [L111] Output: Query intent and candidate target concepts.
- [L112] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"
- [L113] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).
- [L114] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept
- [L115] matching (e.g. if query mentions "Round Robin", we directly target that node).
- [L116] 11. Semantic Retrieval Engine
- [L117] Input: Query intent, key terms.
- [L118] Output: Set of relevant KnowledgeNodes.
- [L119] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with
- [L120] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).
- [L121] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:
- [L122] Parent Expansion: If the query is general, include some ancestors of the matched nodes.
- [L123] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For
- [L124] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related
- [L125] knowledge.)
- [L126] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's
- [L127] semantic paths【29†L37-L44】).
- [L128] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example
- [L129] sentences, and optionally a context snippet from source documents.
- [L130] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a
- [L131] subset of these nodes and their evidence that maximizes informational coverage under a token limit.
- [L132] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: □filecite□turn13file2□

- [L133] metadata:
- [L134] query\_result\_index: 2
- [L135] file\_id: file\_00000000a31c81fa97d0727ba9c7d681
- [L136] version\_id: 1
- [L137] name: Executive Summary (1)(4).pdf
- [L138] mime\_type: application/pdf
- [L139] surface: library
- [L140] score: 0.021104156636173967
- [L141] document\_chunk\_id: file\_00000000a31c81fa97d0727ba9c7d681:0--1--18
- [L142] snippet:
- [L143] Use relation extraction: if n appears in a relation with m outside the tree, link them.
- [L144] Use knowledge bases: e.g. Wikipedia links.
- [L145] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L146] Context Assembler may include one-hop neighbors via CLD).

[L147] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L148] additional edges are "related-to."

[L149] Data Structures and Indexing

[L150] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L151] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L152] embedding vector, and a version/timestamp.

[L153] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L154] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L155] NetworkX, Neo4j).

[L156] •

[L157] •

[L158] •

[L159] •

[L160] •

[L161] •

[L162] •

[L163] •

[L164] •

[L165] •

[L166] •

[L167] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L168] embeddings. This supports CPG++ search and semantic neighbor queries.

[L169] Ancillary Indexes:

[L170] Entity-to-Node map: For quick lookups by name/alias.

[L171] Relation Index: If needed, a mapping from relation type to node pairs.

[L172] Metadata store: (document IDs, source text) for evidence retrieval.

[L173] All indices are updated incrementally as nodes are added.

[L174] Retrieval and Context Assembly

[L175] 10. Query Analyzer

[L176] Input: Natural language user query.

[L177] Output: Query intent and candidate target concepts.

[L178] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L179] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L180] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L181] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L182] 11. Semantic Retrieval Engine

[L183] Input: Query intent, key terms.

[L184] Output: Set of relevant KnowledgeNodes.

[L185] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L186] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L187] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L188] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L189] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L190] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L191] knowledge.)

[L192] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's [L193] semantic paths [29+L37-L44]).

[L194] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example [L195] sentences, and optionally a context snippet from source documents.

[L196] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a [L197] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L198] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: [filecite]turn13file3[

[L199] metadata:

[L200] query\_result\_index: 3

[L201] file\_id: file\_00000000f7d08207bca62408a5680f95

[L202] version\_id: 1

[L203] name: Executive Summary (1)(1).pdf

[L204] mime\_type: application/pdf

[L205] surface: library

[L206] score: 0.021104156636173967

[L207] document\_chunk\_id: file\_00000000f7d08207bca62408a5680f95:0--1--18

[L208] snippet:

[L209] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L210] Use knowledge bases: e.g. Wikipedia links.

[L211] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g. [L212] Context Assembler may include one-hop neighbors via CLD).

[L213] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and [L214] additional edges are "related-to."

[L215] Data Structures and Indexing

[L216] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent [L217] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences), [L218] embedding vector, and a version/timestamp.

[L219] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges) [L220] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g. [L221] NetworkX, Neo4j).

[L222] •

[L223] •

[L224] •

[L225] •

[L226] •

[L227] •

[L228] •

[L229] •

[L230] •

[L231] •

[L232] •

[L233] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L234] embeddings. This supports CPG++ search and semantic neighbor queries.

[L235] Ancillary Indexes:

[L236] Entity-to-Node map: For quick lookups by name/alias.

[L237] Relation Index: If needed, a mapping from relation type to node pairs.

[L238] Metadata store: (document IDs, source text) for evidence retrieval.

[L239] All indices are updated incrementally as nodes are added.

[L240] Retrieval and Context Assembly

[L241] 10. Query Analyzer

[L242] Input: Natural language user query.

[L243] Output: Query intent and candidate target concepts.

[L244] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L245] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L246] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L247] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L248] 11. Semantic Retrieval Engine

[L249] Input: Query intent, key terms.

[L250] Output: Set of relevant KnowledgeNodes.

[L251] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L252] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L253] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L254] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L255] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L256] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L257] knowledge.)

[L258] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG's

[L259] semantic paths【29†L37-L44】).

[L260] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L261] sentences, and optionally a context snippet from source documents.

[L262] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L263] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L264] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: □filecite□turn13file4□

[L265] metadata:

[L266] query\_result\_index: 4

[L267] file\_id: file\_00000000c708820bba7bf4726d6b4174

[L268] version\_id: 1

[L269] name: Executive Summary (1)(3).pdf

[L270] mime\_type: application/pdf

[L271] surface: library

[L272] score: 0.021104156636173967

[L273] document\_chunk\_id: file\_00000000c708820bba7bf4726d6b4174:0--1--18

[L274] snippet:

[L275] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L276] Use knowledge bases: e.g. Wikipedia links.

[L277] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L278] Context Assembler may include one-hop neighbors via CLD).

[L279] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L280] additional edges are "related-to."

[L281] Data Structures and Indexing

[L282] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L283] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L284] embedding vector, and a version/timestamp.

[L285] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L286] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L287] NetworkX, Neo4j).

[L288] •

[L289] •

[L290] •

[L291] •

[L292] •

[L293] •

[L294] •

[L295] •

[L296] •

[L297] •

[L298] •

[L299] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L300] embeddings. This supports CPG++ search and semantic neighbor queries.

[L301] Ancillary Indexes:

[L302] Entity-to-Node map: For quick lookups by name/alias.

[L303] Relation Index: If needed, a mapping from relation type to node pairs.

[L304] Metadata store: (document IDs, source text) for evidence retrieval.

[L305] All indices are updated incrementally as nodes are added.

[L306] Retrieval and Context Assembly

[L307] 10. Query Analyzer

[L308] Input: Natural language user query.

[L309] Output: Query intent and candidate target concepts.

[L310] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L311] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L312] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L313] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L314] 11. Semantic Retrieval Engine

[L315] Input: Query intent, key terms.

[L316] Output: Set of relevant KnowledgeNodes.

[L317] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L318] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L319] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L320] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L321] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L322] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related  
 [L323] knowledge.)  
 [L324] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG’s  
 [L325] semantic paths【29†L37-L44】).  
 [L326] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example  
 [L327] sentences, and optionally a context snippet from source documents.  
 [L328] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a  
 [L329] subset of these nodes and their evidence that maximizes informational coverage under a token limit.  
 [L330] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: □filecite□turn13file5□

[L331] metadata:  
 [L332] query\_result\_index: 5  
 [L333] file\_id: file\_00000000068482079b26e665931694b4  
 [L334] version\_id: 1  
 [L335] name: Executive Summary (1).pdf  
 [L336] mime\_type: application/pdf  
 [L337] surface: library  
 [L338] score: 0.02098340119010335  
 [L339] document\_chunk\_id: file\_00000000068482079b26e665931694b4:0--1--18  
 [L340] snippet:  
 [L341] Use relation extraction: if n appears in a relation with m outside the tree, link them.  
 [L342] Use knowledge bases: e.g. Wikipedia links.  
 [L343] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.  
 [L344] Context Assembler may include one-hop neighbors via CLD).  
 [L345] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are “is-a/part-of” and  
 [L346] additional edges are “related-to.”  
 [L347] Data Structures and Indexing  
 [L348] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent  
 [L349] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),  
 [L350] embedding vector, and a version/timestamp.  
 [L351] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)  
 [L352] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.  
 [L353] NetworkX, Neo4j).  
 [L354] •  
 [L355] •  
 [L356] •  
 [L357] •  
 [L358] •  
 [L359] •  
 [L360] •  
 [L361] •  
 [L362] •  
 [L363] •

[L364] •

[L365] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L366] embeddings. This supports CPG++ search and semantic neighbor queries.

[L367] Ancillary Indexes:

[L368] Entity-to-Node map: For quick lookups by name/alias.

[L369] Relation Index: If needed, a mapping from relation type to node pairs.

[L370] Metadata store: (document IDs, source text) for evidence retrieval.

[L371] All indices are updated incrementally as nodes are added.

[L372] Retrieval and Context Assembly

[L373] 10. Query Analyzer

[L374] Input: Natural language user query.

[L375] Output: Query intent and candidate target concepts.

[L376] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. “Explain X”

[L377] suggests retrieving up the hierarchy) or specific (e.g. “Symptoms of disease Y” suggests looking at children).

[L378] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L379] matching (e.g. if query mentions “Round Robin”, we directly target that node).

[L380] 11. Semantic Retrieval Engine

[L381] Input: Query intent, key terms.

[L382] Output: Set of relevant KnowledgeNodes.

[L383] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L384] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L385] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:

[L386] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L387] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L388] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L389] knowledge.)

[L390] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG’s

[L391] semantic paths【29†L37-L44】).

[L392] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L393] sentences, and optionally a context snippet from source documents.

[L394] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L395] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L396] This process yields a Context Graph of nodes and edges relevant to the query.

---

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: □filecite□turn13file6□

[L397] metadata:

[L398] query\_result\_index: 6

[L399] file\_id: file\_000000006128822fa187fd31b769ca47

[L400] version\_id: 1

[L401] name: Executive Summary (1)(2).pdf

[L402] mime\_type: application/pdf

[L403] surface: library

[L404] score: 0.019109632670394224

[L405] document\_chunk\_id: file\_000000006128822fa187fd31b769ca47:0--1--18

[L406] snippet:

[L407] Use relation extraction: if n appears in a relation with m outside the tree, link them.

[L408] Use knowledge bases: e.g. Wikipedia links.

[L409] These cross-links do not affect parent selection but enrich retrieval graph. They are used at query time (e.g.

[L410] Context Assembler may include one-hop neighbors via CLD).

[L411] Data Structure: The forest becomes a directed acyclic graph (DAG) where tree edges are "is-a/part-of" and

[L412] additional edges are "related-to."

[L413] Data Structures and Indexing

[L414] Knowledge Node Object: Stores concept name, aliases, definition text, children list, parent

[L415] pointer(s), relation lists (with types and confidence), evidence pointers (doc IDs, sentences),

[L416] embedding vector, and a version/timestamp.

[L417] Knowledge Forest: A collection of nodes. Represented as either an adjacency list (for tree edges)

[L418] plus extra edges for related links. We can use a graph database or an in-memory graph (e.g.

[L419] NetworkX, Neo4j).

[L420] •

[L421] •

[L422] •

[L423] •

[L424] •

[L425] •

[L426] •

[L427] •

[L428] •

[L429] •

[L430] •

[L431] 12Vector Index: We maintain an ANN index (e.g. HNSW via FAISS or Qdrant) over the node

[L432] embeddings. This supports CPG++ search and semantic neighbor queries.

[L433] Ancillary Indexes:

[L434] Entity-to-Node map: For quick lookups by name/alias.

[L435] Relation Index: If needed, a mapping from relation type to node pairs.

[L436] Metadata store: (document IDs, source text) for evidence retrieval.

[L437] All indices are updated incrementally as nodes are added.

[L438] Retrieval and Context Assembly

[L439] 10. Query Analyzer

[L440] Input: Natural language user query.

[L441] Output: Query intent and candidate target concepts.

[L442] We classify the query to guide retrieval depth. For instance, detect if the query is broad (e.g. "Explain X"

[L443] suggests retrieving up the hierarchy) or specific (e.g. "Symptoms of disease Y" suggests looking at children).

[L444] We may use simple rules or a small intent classifier. We also identify key terms in the query to seed concept

[L445] matching (e.g. if query mentions "Round Robin", we directly target that node).

[L446] 11. Semantic Retrieval Engine

[L447] Input: Query intent, key terms.

[L448] Output: Set of relevant KnowledgeNodes.

[L449] Algorithm: 1. Concept Matching: Use the vector index to find nodes similar to query. Combine with

[L450] keyword matching: find nodes whose concept or aliases appear in the query. This yields initial node(s).

[L451] 2. Hierarchical Expansion: Based on intent, decide which directions to explore in the hierarchy:



[L452] Parent Expansion: If the query is general, include some ancestors of the matched nodes.

[L453] Child/Sibling Expansion: If specific, include children and siblings of the matched node. (For

[L454] example, TagRAG-style retrieval of tag chains【10†L185-L194】 suggests siblings may hold related

[L455] knowledge.)

[L456] Cross-links: Also follow CLD-related edges to capture semantically linked concepts (like LeanRAG’s

[L457] semantic paths【29†L37-L44】).

[L458] Context Packing: Collect the text evidence for each selected node: e.g. definitions, example

[L459] sentences, and optionally a context snippet from source documents.

[L460] Budget Optimization: Solve the Retrieval Budget Optimization problem (detailed next) to pick a

[L461] subset of these nodes and their evidence that maximizes informational coverage under a token limit.

[L462] This process yields a Context Graph of nodes and edges relevant to the query.

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: `filecite[turn13file7]`

[L463] metadata:

[L464] query\_result\_index: 7

[L465] file\_id: file\_00000000f620821186e54e582d59731e

[L466] version\_id: 1

[L467] name: SHIA\_RAG\_Final\_Report(3).md

[L468] mime\_type: text/markdown

[L469] surface: library

[L470] score: 0.01821162444113264

[L471] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--6

[L472] snippet:

[L473]

```
[L474] graph TD
[L475] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L476] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L477] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L478] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L479] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L480] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L481] end
[L482]
[L483] subgraph "Phase 2: Online – Retrieval & Generation"
[L484] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L485] KF --> L6
[L486] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L487] L7 --> A["Answer + Citations"]
[L488] end
[L489]
[L490] subgraph "Phase 3: Continuous – Operations"
[L491] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L492] A --> L8
[L493] L8 -->| "Self-Evolution
Feedback" | KF
[L494] end
[L495]
```

[L496]

[L497] ### 3.2 Data Flow

[L498]  
[L499]

```
[L500] Raw Document (PDF/DOCX/HTML)
[L501] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L502] Normalized Text + Metadata
[L503] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L504] [Page → Region → Block] hierarchy
[L505] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L506] KnowledgeUnit objects (candidate concepts)
[L507] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L508] KnowledgeNode objects (canonical, deduplicated)
[L509] ↓ Layer 4: Validate relations against ontology, score confidence
[L510] KnowledgeEdge objects (typed, scored)
[L511] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L512] Knowledge Forest (rooted trees + cross-links)
[L513] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L514] Selected context nodes (within token budget)
[L515] ↓ Layer 7: Build prompt, invoke LLM
[L516] ↓ Layer 7: Verify claims against evidence, bind citations
[L517] Answer with citations + confidence score
[L518] ↓ Layer 8: Log, collect feedback, update forest weights
[L519]
```

[L520]  
[L521] ### 3.3 Layer Summary  
[L522]  
[L523] | Layer | Name | Input | Output | Key Algorithms |  
[L524] |-----|-----|-----|-----|-----|  
[L525] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |  
[L526] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |  
[L527] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |  
[L528] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L529] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L530] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L531] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L532] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L533] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L534]  
[L535] ---  
[L536]  
[L537] ## 4. Data Model — Layer 0

Citation Marker: `[[filecite]]turn13file8`

[L538] metadata:  
[L539] query\_result\_index: 8  
[L540] file\_id: file\_00000000fb2c81fd82f14a7a48993554  
[L541] version\_id: 1  
[L542] name: SHIA\_RAG\_Final\_Report.md  
[L543] mime\_type: text/markdown

[L544] surface: library

[L545] score: 0.01821162444113264

[L546] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--6

[L547] snippet:

[L548]

```
[L549] graph TD
[L550] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L551] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L552] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L553] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L554] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L555] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L556] end
[L557]
[L558] subgraph "Phase 2: Online – Retrieval & Generation"
[L559] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L560] KF --> L6
[L561] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L562] L7 --> A["Answer + Citations"]
[L563] end
[L564]
[L565] subgraph "Phase 3: Continuous – Operations"
[L566] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L567] A --> L8
[L568] L8 -->|"Self-Evolution
Feedback"| KF
[L569] end
[L570]
```

[L571]

[L572] ### 3.2 Data Flow

[L573]

[L574]

```
[L575] Raw Document (PDF/DOCX/HTML)
[L576] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L577] Normalized Text + Metadata
[L578] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L579] [Page → Region → Block] hierarchy
[L580] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L581] KnowledgeUnit objects (candidate concepts)
[L582] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L583] KnowledgeNode objects (canonical, deduplicated)
[L584] ↓ Layer 4: Validate relations against ontology, score confidence
[L585] KnowledgeEdge objects (typed, scored)
[L586] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L587] Knowledge Forest (rooted trees + cross-links)
[L588] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L589] Selected context nodes (within token budget)
[L590] ↓ Layer 7: Build prompt, invoke LLM
[L591] ↓ Layer 7: Verify claims against evidence, bind citations
[L592] Answer with citations + confidence score
[L593] ↓ Layer 8: Log, collect feedback, update forest weights
[L594]
```

[L595]

[L596] ### 3.3 Layer Summary

[L597]

|        |  |       |  |                      |  |                    |  |                    |  |                                         |  |
|--------|--|-------|--|----------------------|--|--------------------|--|--------------------|--|-----------------------------------------|--|
| [L598] |  | Layer |  | Name                 |  | Input              |  | Output             |  | Key Algorithms                          |  |
| [L599] |  | ----- |  | -----                |  | -----              |  | -----              |  | -----                                   |  |
| [L600] |  | 0     |  | Data Model           |  | —                  |  | Schemas            |  | KnowledgeNode, KnowledgeEdge            |  |
| [L601] |  | 1     |  | Input Processing     |  | Raw files          |  | NormalizedDocument |  | Format detection, OCR fallback          |  |
| [L602] |  | 2     |  | Document Parsing     |  | NormalizedDocument |  | DocumentBlock[]    |  | Block classification, reading order     |  |
| [L603] |  | 3     |  | Knowledge Extraction |  | DocumentBlock[]    |  | KnowledgeNode[]    |  | LLM extraction, alias resolution        |  |
| [L604] |  | 4     |  | Relation Management  |  | KnowledgeNode[]    |  | KnowledgeEdge[]    |  | Ontology validation, confidence scoring |  |
| [L605] |  | 5     |  | SHIA Core            |  | Nodes + Edges      |  | Knowledge Forest   |  | CPG++, PRE, HV, FI, CLD                 |  |
| [L606] |  | 6     |  | Retrieval Engine     |  | User query         |  | RetrievalResult    |  | Adaptive depth, Tree Knapsack           |  |
| [L607] |  | 7     |  | Generation Engine    |  | RetrievalResult    |  | GenerationResult   |  | Prompt assembly, claim verification     |  |
| [L608] |  | 8     |  | Operations           |  | GenerationResult   |  | Updated Forest     |  | Feedback, self-evolution, monitoring    |  |
| [L609] |  |       |  |                      |  |                    |  |                    |  |                                         |  |
| [L610] |  | ---   |  |                      |  |                    |  |                    |  |                                         |  |
| [L611] |  |       |  |                      |  |                    |  |                    |  |                                         |  |
| [L612] |  | ###   |  | 4. Data Model        |  | —                  |  | Layer 0            |  |                                         |  |

DEEP RESEARCH 20/08/2026, 15:06:23

Citation Marker: `filecite[turn13file9]`

|        |                                                                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [L613] | metadata:                                                                                                                                                                       |
| [L614] | query_result_index: 9                                                                                                                                                           |
| [L615] | file_id: file_000000004bc481fdbf9e949ec3b7544b                                                                                                                                  |
| [L616] | version_id: 1                                                                                                                                                                   |
| [L617] | name: SHIA_RAG_Final_Report(2).md                                                                                                                                               |
| [L618] | mime_type: text/markdown                                                                                                                                                        |
| [L619] | surface: library                                                                                                                                                                |
| [L620] | score: 0.01821162444113264                                                                                                                                                      |
| [L621] | document_chunk_id: file_000000004bc481fdbf9e949ec3b7544b--1--13                                                                                                                 |
| [L622] | snippet:                                                                                                                                                                        |
| [L623] | The parser uses rule-based heuristics for block classification:                                                                                                                 |
| [L624] | - <b>Heading:</b> Short line (< 100 chars), no ending punctuation, starts with uppercase                                                                                        |
| [L625] | - <b>List:</b> Starts with <code>-</code> , <code>*</code> , <code>•</code> , or <code>1.</code>                                                                                |
| [L626] | - <b>Table:</b> Contains 3+ pipe characters ( <code> </code> )                                                                                                                  |
| [L627] | - <b>Code:</b> Contains Python keywords ( <code>def</code> , <code>class</code> , <code>import</code> )                                                                         |
| [L628] | - <b>Definition:</b> Contains phrases like "is defined as", "refers to"                                                                                                         |
| [L629] | - <b>Default:</b> Paragraph                                                                                                                                                     |
| [L630] |                                                                                                                                                                                 |
| [L631] | ---                                                                                                                                                                             |
| [L632] |                                                                                                                                                                                 |
| [L633] | ### 5.3 Layer 3: Knowledge Extraction                                                                                                                                           |
| [L634] |                                                                                                                                                                                 |
| [L635] | <b>Responsibility:</b> Extract concepts, definitions, and candidate relations from document blocks, then merge synonyms to create canonical <code>KnowledgeNode</code> objects. |
| [L636] |                                                                                                                                                                                 |
| [L637] | <b>Input:</b> <code>DocumentBlock[]</code> → <b>Output:</b> <code>KnowledgeNode[]</code> (canonical, deduplicated)                                                              |
| [L638] |                                                                                                                                                                                 |

[L639] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** — it identifies entities, definitions, and raw relation triples from text. Layer 4 performs **validation and scoring** — it validates extracted relations against the ontology and assigns confidence scores.

[L640]

[L641] ##### Extraction Prompt

[L642]

[L643]

```
[L644] You are a knowledge extraction system. Given a text block, extract:
[L645] 1. Concepts: Named entities, technical terms
[L646] 2. Definitions: If the text defines any concept, extract the definition
[L647] 3. Properties: Key attributes with types and values
[L648] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L649] 5. Domain: Subject area (e.g., "Operating Systems")
[L650]
[L651] Respond in JSON format:
[L652] {
[L653] "concepts": [{"name": "...", "aliases": [...], "definitions": [...],
[L654] "properties": {"key": {"type": "...", "value": "...}}}],
[L655] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L656] "evidence": "...}],
[L657] "domain": "...
[L658] }
[L659]
```

[L660]

[L661] ##### Alias Resolution

[L662]

[L663] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L664]

[L665] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L666] 2. Find connected components of near-duplicate units

[L667] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L668]

[L669]

```
[L670] def alias_resolution(units: list) -> list:
[L671] """Merge KnowledgeUnits referring to the same concept.
[L672]
[L673] Merge strategy:
[L674] - Canonical name: most frequent name across units
[L675] - Aliases: union of all names and aliases
[L676] - Definitions: union of all definitions
[L677] - Confidence: average of unit extraction confidences
[L678] - Properties: merged (latest value wins, conflicts logged)
[L679] """
[L680] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L681] # ... build LSH index from concept names + aliases ...
[L682] merge_groups = find_connected_components(units, lsh)
[L683] return [create_canonical_node(group) for group in merge_groups]
[L684]
```

[L685]

[L686] ---

[L687]

[L688] ### 5.4 Layer 4: Relation Management

[L689]

[L690] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L691]

[L692] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

---

DEEP RESEARCH 20/08/2026, 15:06:27

Citation Marker: `filecite{turn14file0}`

[L1] metadata:

[L2] `ref_id`: turn13file9

[L3] `file_id`: file\_000000004bc481fdbf9e949ec3b7544b

[L4] `version_id`: 1

[L5] `name`: SHIA\_RAG\_Final\_Report(2).md

[L6] `lines_shown`: 688-927

[L7] `total_file_lines`: 1527

[L8] `has_more`: True

[L9] `next_start_line`: 928

[L10] `next_read`: {"ref\_id":"turn13file9","mode":"full","start\_line":928,"max\_lines":240}

[L11] `content`:

[L12] `forest.add_edge(node.id, neighbor.id,`

[L13] `edge_class='SEMANTIC', relation_type=relation_type,`

[L14] `confidence=sim, weight=1.0)`

[L15] `cross_links.append((neighbor.id, relation_type, sim))`

[L16]

[L17] `return cross_links`

[L18]

```

[L19]
[L20] ---
[L21]
[L22] ## 6. Online Pipeline – Layers 6-7
[L23]
[L24] ### 6.1 Layer 6: Retrieval Engine
[L25]
[L26] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes
[L27]
[L28] **Input:** User query string → **Output:** `RetrievalResult`
[L29]
[L30] #### Query Complexity Classification
[L31]
[L32] | Intent | Example | Complexity | Traversal Depth |
[L33] |-----|-----|-----|-----|
[L34] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |
[L35] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |
[L36] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |
[L37] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |
[L38] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |
[L39]
[L40] #### Retrieval Pipeline
[L41]
[L42]

```

python

```

[L43] @dataclass
[L44] class RetrievalResult:
[L45] """Output of Layer 6 — input to Layer 7."""
[L46] selected_nodes: list # Nodes selected by Tree Knapsack
[L47] context_text: str # Assembled context string
[L48] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L49] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L50] query_intent: str # DEFINITION, COMPARISON, etc.
[L51] seed_node_ids: list # Initial seed nodes from vector search
[L52] token_count: int # Estimated tokens in context
[L53]
[L54] class RetrievalEngine:
[L55] def __init__(self, forest, vector_index, cache_manager, config):
[L56] self.forest = forest
[L57] self.vector_index = vector_index
[L58] self.cache = cache_manager
[L59] self.config = config
[L60]
[L61] def retrieve(self, query) -> RetrievalResult:
[L62] # Step 0: Cache check
[L63] cached = self.cache.get_query_result(hash_query(query))
[L64] if cached:
[L65] return cached
[L66]
[L67] # Step 1: Classify query intent and complexity
[L68] intent = detect_intent(query)
[L69] complexity = classify_complexity(intent)

```

```

[L70]
[L71] # Step 2: Embed query and find seed nodes
[L72] query_emb = embed_query(query) # Asymmetric embedding model
[L73] seed_nodes = self.vector_index.search(query_emb, k=5)
[L74]
[L75] # Step 3: Adaptive depth traversal
[L76] depth_config = {
[L77] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L78] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L79] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L80] }
[L81] traversal = depth_config[complexity]
[L82]
[L83] # Step 4: BFS traversal from all seeds
[L84] candidate_nodes = set()
[L85] for seed in seed_nodes:
[L86] candidate_nodes |= self.traverse(seed, traversal)
[L87]
[L88] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L89] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L90]
[L91] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L92] context_text = self.assemble_context(selected, intent)
[L93]
[L94] return RetrievalResult(
[L95] selected_nodes=selected, context_text=context_text,
[L96] traversal_path_edges=self._get_path_edges(selected),
[L97] query_complexity=complexity, query_intent=intent,
[L98] seed_node_ids=[s.id for s in seed_nodes],
[L99] token_count=estimate_tokens(context_text)
[L100])
[L101]

```

```

[L102]
[L103] ##### Multi-Tree Knapsack Selection
[L104]
[L105] For multi-seed queries, candidates may span multiple trees. The algorithm:
[L106]
[L107] 1. **Group** candidates by tree root
[L108] 2. **Allocate** token budget proportionally to each tree
[L109] 3. **Run** Tree Knapsack on each tree independently
[L110] 4. **Merge** results and trim to total budget
[L111]
[L112]

```

python

```

[L113] def multi_tree_knapsack(self, candidate_nodes, budget):
[L114] tree_groups = {}
[L115] for node in candidate_nodes:
[L116] root = self.forest.find_root(node)

```



```

[L117] tree_groups.setdefault(root.id, []).append(node)
[L118]
[L119] all_selected = []
[L120] for root_id, nodes in tree_groups.items():
[L121] tree_budget = int(budget len(nodes) / max(len(candidate_nodes), 1))
[L122] _, selected = tree_knapsack(self.forest.get_node(root_id), tree_budget, self.forest)
[L123] all_selected.extend([n for n in selected if n in nodes])
[L124]
[L125] # Final trim to total budget
[L126] all_selected.sort(key=lambda n: utility(n, self.forest), reverse=True)
[L127] final, total_tokens = [], 0
[L128] for node in all_selected:
[L129] cost = token_cost(node)
[L130] if total_tokens + cost <= budget:
[L131] final.append(node)
[L132] total_tokens += cost
[L133] return final
[L134]

[L135]
[L136] ##### Utility Function (Self-Evolution Integrated)
[L137]
[L138]

```

python

```

[L139] def utility(node, forest):
[L140] """Utility for Tree Knapsack — uses edge weights from self-evolution feedback."""
[L141] if not node.parent:
[L142] return node.confidence len(node.evidence)
[L143]
[L144] parent_edge = forest.get_edge(node.parent, node.id)
[L145] edge_weight = parent_edge.weight if parent_edge else 1.0
[L146]
[L147] return (
[L148] node.confidence # Node quality
[L149] edge_weight # Self-evolution signal
[L150] len(node.evidence) * # Evidence richness
[L151] (1.0 / max(token_cost(node), 1)) # Cost efficiency
[L152])
[L153]

```

```

[L154]
[L155] ---
[L156]
[L157] ### 6.2 Layer 7: Generation Engine
[L158]
[L159] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each
[L160]
[L161] **Input:** `RetrievalResult` → **Output:** `GenerationResult`
[L162]
[L163] #### LLM Configuration
[L164]
[L165] | Parameter | Value | Rationale |
[L166] |-----|-----|-----|
[L167] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |
[L168] | Temperature | 0.3 | Low for factual answers |
[L169] | Max output tokens | 2048 | Sufficient for detailed answers |
[L170] | Response format | JSON | Force structured output for claim extraction |
[L171] | Timeout | 30 seconds | With retry on timeout |
[L172]
[L173] #### Prompt Template
[L174]
[L175]

```

python

```

[L176] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L177] RULES:
[L178] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L179] 2. For each claim, cite the source using [NODE_ID] format.
[L180] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L181] 4. If evidence conflicts, present both views with citations.
[L182]
[L183] REQUIRED OUTPUT FORMAT:
[L184] {
[L185] "answer": "Your answer with [NODE_ID] citations inline.",
[L186] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L187] "overall_confidence": 0.0-1.0,
[L188] "insufficient_evidence": false
[L189] }"""
[L190]

```

```

[L191]
[L192] #### Claim Verification
[L193]
[L194] After the LLM generates an answer, each claim is verified against its cited evidence:
[L195]
[L196]

```

python

```

[L197] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L198] """Returns (status, confidence).
[L199] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L200] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L201] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L202] - CONFLICTING: evidence items contradict each other
[L203] """

```

```

[L204] scores = []
[L205] for eid in evidence_ids:
[L206] node = node_map.get(eid)
[L207] if node and node.evidence:
[L208] evidence_text = ' '.join(e.text for e in node.evidence)
[L209] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L210]
[L211] if not scores:
[L212] return 'UNSUPPORTED', 0.0
[L213]
[L214] avg_score = sum(scores) / len(scores)
[L215]
[L216] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L217] return 'CONFLICTING', avg_score
[L218] if avg_score > 0.7:
[L219] return 'SUPPORTED', avg_score
[L220] elif avg_score > 0.4:
[L221] return 'PARTIAL', avg_score
[L222] else:
[L223] return 'UNSUPPORTED', avg_score
[L224]

```

```

[L225]
[L226] ##### Unsupported Claim Policy
[L227]
[L228] | Status | Action |
[L229] |-----|-----|
[L230] | SUPPORTED | Keep as-is with citation |
[L231] | PARTIAL | Keep but mark as partially supported |
[L232] | UNSUPPORTED | Remove from answer, note at end |
[L233] | CONFLICTING | Present both views with note |
[L234]
[L235] ##### Answer Confidence Formula
[L236]
[L237] $\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \text{overall confidence}$
[L238]
[L239] ##### Output Schema
[L240]
[L241]

```

python

```

[L242] @dataclass
[L243] class GenerationResult:
[L244] """Output of Layer 7 — ready for user and Layer 8."""
[L245] answer: str # Answer text with citations
[L246] claims: list # List of VerifiedClaim
[L247] citations: list # Formatted citation list
[L248] overall_confidence: float # 0.0-1.0
[L249] retrieval_result: object # Preserved for Layer 8
[L250] generation_metadata: dict # Model used, claim counts, etc.
[L251] ``

```

DEEP RESEARCH 20/08/2026, 15:06:27

Citation Marker: □filecite□turn14file1□

[L252] metadata:

[L253] read\_index: 1

[L254] ref\_id: turn13file7

[L255] file\_id: file\_00000000f620821186e54e582d59731e

[L256] version\_id: 1

[L257] name: SHIA\_RAG\_Final\_Report(3).md

[L258] lines\_shown: 495-534

[L259] total\_file\_lines: 1527

[L260] has\_more: True

[L261] next\_start\_line: 535

[L262] next\_read: {"ref\_id":"turn13file7","mode":"full","start\_line":535,"max\_lines":40}

[L263] content:

[L264] ##### Alias Resolution

[L265]

[L266] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round Robin", "Round-Robin Scheduling"). Alias resolution merges them:

[L267]

[L268] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L269] 2. Find connected components of near-duplicate units

[L270] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L271]

[L272]

```
[L273] def alias_resolution(units: list) -> list:
[L274] """Merge KnowledgeUnits referring to the same concept.
[L275]
[L276] Merge strategy:
[L277] - Canonical name: most frequent name across units
[L278] - Aliases: union of all names and aliases
[L279] - Definitions: union of all definitions
[L280] - Confidence: average of unit extraction confidences
[L281] - Properties: merged (latest value wins, conflicts logged)
[L282] """
[L283] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L284] # ... build LSH index from concept names + aliases ...
[L285] merge_groups = find_connected_components(units, lsh)
[L286] return [create_canonical_node(group) for group in merge_groups]
[L287]
```

[L288]

[L289] ---

[L290]

[L291] ### 5.4 Layer 4: Relation Management

[L292]

[L293] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L294]

[L295] **Input:** KnowledgeNode[] (with raw relations) → **Output:** KnowledgeEdge[] (validated, scored)

[L296]

[L297] **Pipeline:**

[L298] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L299] 2. **Resolve target node** — map target concept name to a KnowledgeNode

[L300] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L301] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L302] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

DEEP RESEARCH 20/08/2026, 15:06:30

Citation Marker: □filecite□turn15file0□

[L1] metadata:

[L2] ref\_id: turn13file7

[L3] file\_id: file\_00000000f620821186e54e582d59731e

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Final\_Report(3).md

[L6] lines\_shown: 789-1008

[L7] total\_file\_lines: 1527

[L8] has\_more: True

[L9] next\_start\_line: 1009

[L10] next\_read: {"ref\_id":"turn13file7","mode":"full","start\_line":1009,"max\_lines":220}

[L11] content:

[L12] def multi\_tree\_knapsack(self, candidate\_nodes, budget):

[L13] tree\_groups = {}

[L14] for node in candidate\_nodes:

[L15] root = self.forest.find\_root(node)

[L16] tree\_groups.setdefault(root.id, []).append(node)

[L17]

[L18] all\_selected = []

[L19] for root\_id, nodes in tree\_groups.items():

[L20] tree\_budget = int(budget len(nodes) / max(len(candidate\_nodes), 1))

[L21] \_, selected = tree\_knapsack(self.forest.get\_node(root\_id), tree\_budget, self.forest)

[L22] all\_selected.extend([n for n in selected if n in nodes])

[L23]

[L24] # Final trim to total budget

[L25] all\_selected.sort(key=lambda n: utility(n, self.forest), reverse=True)

[L26] final, total\_tokens = [], 0

[L27] for node in all\_selected:

[L28] cost = token\_cost(node)

[L29] if total\_tokens + cost <= budget:

[L30] final.append(node)

[L31] *total\_tokens += cost*

[L32] *return final*

[L33]

```
[L34]
[L35] ##### Utility Function (Self-Evolution Integrated)
[L36]
[L37]
```

*python*

[L38] *def utility(node, forest):*

[L39] *"""Utility for Tree Knapsack — uses edge weights from self-evolution feedback."""*

[L40] *if not node.parent:*

[L41] *return node.confidence len(node.evidence)*

[L42]

[L43] *parent\_edge = forest.get\_edge(node.parent, node.id)*

[L44] *edge\_weight = parent\_edge.weight if parent\_edge else 1.0*

[L45]

[L46] *return (*

[L47] *node.confidence # Node quality*

[L48] *edge\_weight # Self-evolution signal*

[L49] *len(node.evidence) # Evidence richness*

[L50] *(1.0 / max(token\_cost(node), 1)) # Cost efficiency*

[L51] *)*

[L52]

```
[L53]
[L54] ---
[L55]
[L56] ### 6.2 Layer 7: Generation Engine
[L57]
[L58] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each
[L59]
[L60] **Input:** `RetrievalResult` → **Output:** `GenerationResult`
[L61]
[L62] ##### LLM Configuration
[L63]
[L64] | Parameter | Value | Rationale |
[L65] |-----|-----|-----|
[L66] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |
[L67] | Temperature | 0.3 | Low for factual answers |
[L68] | Max output tokens | 2048 | Sufficient for detailed answers |
[L69] | Response format | JSON | Force structured output for claim extraction |
[L70] | Timeout | 30 seconds | With retry on timeout |
[L71]
[L72] ##### Prompt Template
[L73]
[L74]
```

*python*

[L75] *SYSTEM\_PROMPT = """You are a knowledge-grounded question answering system.*

[L76] *RULES:*

[L77] *1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.*

[L78] *2. For each claim, cite the source using [NODE\_ID] format.*

[L79] *3. If evidence is insufficient, say "Insufficient evidence to answer."*

[L80] 4. If evidence conflicts, present both views with citations.

[L81]

[L82] REQUIRED OUTPUT FORMAT:

[L83] {

[L84] "answer": "Your answer with [NODE\_ID] citations inline.",

[L85] "claims": [{"text": "Factual claim", "evidence\_ids": ["NODE\_ID1"], "confidence": 0.0-1.0}],

[L86] "overall\_confidence": 0.0-1.0,

[L87] "insufficient\_evidence": false

[L88] }""

[L89]

[L90]

[L91] ##### Claim Verification

[L92]

[L93] After the LLM generates an answer, each claim is verified against its cited evidence:

[L94]

[L95]

python

[L96] def verify\_claim(claim\_text, evidence\_ids, node\_map) -> tuple:

[L97] """Returns (status, confidence).

[L98] - SUPPORTED: evidence entails claim (confidence > 0.7)

[L99] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)

[L100] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)

[L101] - CONFLICTING: evidence items contradict each other

[L102] """

[L103] scores = []

[L104] for eid in evidence\_ids:

[L105] node = node\_map.get(eid)

[L106] if node and node.evidence:

[L107] evidence\_text = ''.join(e.text for e in node.evidence)

[L108] scores.append(cosine\_similarity(embed(claim\_text), embed(evidence\_text)))

[L109]

[L110] if not scores:

[L111] return 'UNSUPPORTED', 0.0

[L112]

[L113] avg\_score = sum(scores) / len(scores)

[L114]

[L115] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:

[L116] return 'CONFLICTING', avg\_score

[L117] if avg\_score > 0.7:

[L118] return 'SUPPORTED', avg\_score

[L119] elif avg\_score > 0.4:

[L120] return 'PARTIAL', avg\_score

[L121] else:

[L122] return 'UNSUPPORTED', avg\_score

[L123]

```

[L124]
[L125] ##### Unsupported Claim Policy
[L126]
[L127] | Status | Action |
[L128] |-----|-----|
[L129] | SUPPORTED | Keep as-is with citation |
[L130] | PARTIAL | Keep but mark as partially supported |
[L131] | UNSUPPORTED | Remove from answer, note at end |
[L132] | CONFLICTING | Present both views with note |
[L133]
[L134] ##### Answer Confidence Formula
[L135]
[L136] $\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \text{overall confidence}$
[L137]
[L138] ##### Output Schema
[L139]
[L140]

```

python

```

[L141] @dataclass
[L142] class GenerationResult:
[L143] """Output of Layer 7 — ready for user and Layer 8."""
[L144] answer: str # Answer text with citations
[L145] claims: list # List of VerifiedClaim
[L146] citations: list # Formatted citation list
[L147] overall_confidence: float # 0.0-1.0
[L148] retrieval_result: object # Preserved for Layer 8
[L149] generation_metadata: dict # Model used, claim counts, etc.
[L150]

```

```

[L151]
[L152] ---
[L153]
[L154] ## 7. Operations Layer – Layer 8
[L155]
[L156] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution)
[L157]
[L158] ### 7.1 Logging
[L159]
[L160] Every query processed through the system generates a structured `QueryLog` entry:
[L161]
[L162]

```

python

```

[L163] @dataclass
[L164] class QueryLog:
[L165] event_id: str
[L166] timestamp: str
[L167] query: str
[L168] intent: str # DEFINITION, COMPARISON, etc.
[L169] complexity: str # SIMPLE, MODERATE, COMPLEX
[L170] seed_nodes: list
[L171] candidate_count: int

```



```
[L172] selected_count: int
[L173] traversal_edges: list # For feedback attribution
[L174] answer_length: int
[L175] claim_count: int
[L176] supported_claims: int
[L177] unsupported_claims: int
[L178] overall_confidence: float
[L179] model_used: str
[L180] latency_ms: float
[L181] user_feedback: str = None # HELPFUL / NOT_HELPFUL (filled later)
[L182]
```

```
[L183]
[L184] ### 7.2 Feedback Acquisition & Attribution
[L185]
[L186] User feedback is collected via API:
[L187]
[L188]
```

```
[L189] POST /api/feedback {"event_id": "...", "feedback": "HELPFUL" | "NOT_HELPFUL"}
[L190]
```

```
[L191]
[L192] **Attribution strategy:** Feedback applies ONLY to edges in the recorded `traversal_path`. This isolates
[L193]
[L194] **Safeguards:**
[L195] - Edge weight updates use a dampened learning rate ($\eta=0.02$, small increments)
[L196] - Layer 7 verification metrics (supported/unsupported claims) are tracked separately
[L197] - Anomaly detection: if `confidence > 0.8` AND `feedback == "NOT_HELPFUL"`, log a warning (possible edge
[L198]
[L199] ### 7.3 Self-Evolution Update Rule
[L200]
[L201] After each query with feedback signal $f \in \{+1, -1\}$:
[L202]
[L203]
$$w_e^{(t+1)} = w_e^{(t)} \cdot \exp(\eta \cdot f) \quad \text{forall } e \in \text{traversal_path}$$

[L204]
[L205] Where $\eta = 0.02$. Edge weights are clipped to $[0.1, 10.0]$.
[L206]
[L207]
```

python

```
[L208] import math
[L209]
[L210] ETA = 0.02
[L211] WEIGHT_MIN, WEIGHT_MAX = 0.1, 10.0
[L212] RESTRUCTURE_THRESHOLD = 0.3
[L213]
[L214] def update_forest_from_feedback(selected_nodes, user_feedback, forest):
[L215] f = +1 if user_feedback == "HELPFUL" else -1
[L216] path_edges = get_traversal_path_edges(selected_nodes, forest)
[L217]
[L218] for edge in path_edges:
[L219] edge.weight = edge.weight * math.exp(ETA * f)
```

```

[L220] edge.weight = max(WEIGHT_MIN, min(edge.weight, WEIGHT_MAX))
[L221] edge.access_count += 1
[L222]
[L223] if f == -1 and edge.weight < RESTRUCTURE_THRESHOLD:
[L224] flag_for_restructure(edge, forest)
[L225]
[L226] if forest.total_queries % 100 == 0:
[L227] rebalance_forest(forest)
[L228] forest.total_queries += 1
[L229] ```
[L230]
[L231] ### 7.4 Closing the Self-Evolution Loop

```

---

DEEP RESEARCH 20/08/2026, 15:06:33

Citation Marker: `filecite[turn16file0]`

```

[L1] metadata:
[L2] ref_id: turn13file7
[L3] file_id: file_00000000f620821186e54e582d59731e
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(3).md
[L6] lines_shown: 535-714
[L7] total_file_lines: 1527
[L8] has_more: True
[L9] next_start_line: 715
[L10] next_read: {"ref_id":"turn13file7","mode":"full","start_line":715,"max_lines":180}
[L11] content:
[L12]

```

```

[L13] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L14] """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L15] conf = 0.5
[L16] conf += 0.15 * source.confidence
[L17] conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L18] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L19] return min(conf, 1.0)
[L20]

```

```

[L21]
[L22] ---
[L23]
[L24] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction
[L25]
[L26] Responsibility: Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.
[L27]
[L28] Input: KnowledgeNode[] + KnowledgeEdge[] → Output: Knowledge Forest
[L29]
[L30] #### Pipeline Overview

```

[L31]

[L32]

```

[L33] graph LR
[L34] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L35] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L36] PRE --> HV{"HV
Validate #1?"}
[L37] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L38] HV -->|"Fail"| HV2{"HV
Validate #2?"}
[L39] HV2 -->|"Pass"| FI
[L40] HV2 -->|"Fail"| HV3{"HV
Validate #3?"}
[L41] HV3 -->|"Pass"| FI
[L42] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L43] FI --> CLD["CLD
Discover Cross-Links"]
[L44] CLD --> DONE["Node Placed ✓"]
[L45]

```

[L46]

[L47] ##### 5.5.1 CPG++ — Candidate Parent Generator

[L48]

[L49] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L50]

[L51]

```

[L52] def cpg_plus_plus(node_q, forest, config):
[L53] """Generate ≤k candidate parents for node q."""
[L54] # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L55] domain = classify_domain(node_q)
[L56] domain_nodes = forest.get_nodes_by_domain(domain)
[L57]
[L58] # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L59] if len(domain_nodes) > config.ann_k:
[L60] ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L61] candidates = [r for r in ann_results if r in domain_nodes]
[L62] else:
[L63] candidates = domain_nodes
[L64]
[L65] # Stage 3: Relation Filter – keep only nodes with plausible relations
[L66] candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L67]
[L68] # Stage 4: Abstraction Filter – parent must be MORE abstract
[L69] q_level = estimate_abstraction_level(node_q)
[L70] candidates = [c for c in candidates
[L71] if c.abstraction_level > q_level
[L72] and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L73]
[L74] # Stage 5: Evidence Filter – prioritize cross-document evidence
[L75] candidates = sorted(candidates,
[L76] key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L77]
[L78] return candidates[:config.max_candidates] # Return top 5
[L79]

```

[L80]

[L81] **Overall complexity:**  $O(\log N)$  dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L82]

[L83] ##### 5.5.2 PRE — Parent Ranking Engine

[L84]

[L85] Ranks all candidates using a weighted scoring formula:

[L86]

$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}(P)}{D_{\max}} + w_4 \cdot S_{\text{evi}}(P, N) + w_5 \cdot \text{Conf}(P)$$

[L88]

[L89] Where:

[L90] -  $w_1=0.30$  — Embedding cosine similarity[L91] -  $w_2=0.25$  — Hypernym score (Hearst patterns + LLM fallback)[L92] -  $w_3=0.15$  — Depth penalty (shallower parents preferred;  $D_{\max}=8$ )[L93] -  $w_4=0.20$  — Evidence support[L94] -  $w_5=0.10$  — Parent confidence

[L95]

[L96]

```

[L97] def pre_rank_all(node_q, candidates, config):
[L98] """Rank ALL candidates, return sorted list."""
[L99] scored = []
[L100] for p in candidates:
[L101] sim = cosine_similarity(node_q.embedding, p.embedding)
[L102] hyp = hypernym_score(node_q, p)
[L103] depth_score = (config.d_max - p.depth) / config.d_max
[L104] evi = evidence_support(node_q, p)
[L105] conf = p.confidence
[L106]
[L107] score = config.w1*sim + config.w2*hyp + config.w3*depth_score + config.w4*evi + config.w5*conf
[L108] scored.append(CandidateResult(parent=p, pre_score=score,
[L109] components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))
[L110]
[L111] scored.sort(key=lambda x: x.pre_score, reverse=True)
[L112]
[L113] if scored and scored[0].pre_score < config.new_tree_threshold:
[L114] return [] # No acceptable candidate → create new root
[L115] return scored
[L116]

```

[L117]

[L118] ##### 5.5.3 HV — Hierarchy Validator

[L119]

[L120] Validates that adding a parent → child edge maintains forest invariants:

[L121]

[L122] 1. **Self-loop check** — parent ≠ child[L123] 2. **Cycle detection** — traverse UP from parent looking for child; if found, reject (would create cycle)[L124] 3. **Depth limit** — `parent.depth + 1 < config.max_depth`[L125] 4. **Sibling coherence** — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized

[L126]

[L127] ##### 5.5.4 FI — Forest Inserter

[L128]

[L129] Inserts a node into the forest by trying HV-validated candidates in ranked order:

[L130]

[L131]

```
[L132] def fi_insert(node, ranked_candidates, forest, config):
[L133] """Insert node into forest. Try candidates in order; fall back to new root."""
[L134] for candidate in ranked_candidates:
[L135] if hv_validate(candidate.parent, node, forest, config):
[L136] forest.add_edge(candidate.parent.id, node.id, edge_class='HIERARCHICAL')
[L137] node.parent = candidate.parent.id
[L138] node.depth = candidate.parent.depth + 1
[L139] propagate_confidence(forest) # Re-propagate confidence scores
[L140] return InsertResult(status='PLACED', parent=candidate.parent.id)
[L141]
[L142] # All candidates failed validation – create new root
[L143] forest.add_root(node)
[L144] return InsertResult(status='NEW_ROOT', reason='NO_VALID_CANDIDATE')
[L145]
```

[L146]

[L147] ##### 5.5.5 CLD — Cross-Link Discoverer

[L148]

[L149] After hierarchical placement, discover semantic (non-hierarchical) links:

[L150]

[L151]

```
[L152] def cld_discover(node, forest, config):
[L153] """Find semantic cross-links for a newly placed node."""
[L154] cross_links = []
[L155] neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L156]
[L157] for neighbor in neighbors:
[L158] if neighbor.id == node.id or neighbor.id == node.parent:
[L159] continue
[L160]
[L161] sim = cosine_similarity(node.embedding, neighbor.embedding)
[L162] if sim > config.cross_link_threshold:
[L163] relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L164] if relation_type:
[L165] forest.add_edge(node.id, neighbor.id,
[L166] edge_class='SEMANTIC', relation_type=relation_type,
[L167] confidence=sim, weight=1.0)
[L168] cross_links.append((neighbor.id, relation_type, sim))
[L169]
[L170] return cross_links
[L171]
```

[L172]

[L173] ---

[L174]

[L175] ## 6. Online Pipeline — Layers 6-7

[L176]

[L177] ### 6.1 Layer 6: Retrieval Engine

[L178]

[L179] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L180]

[L181] **Input:** User query string → **Output:** RetrievalResult

[L182]

[L183] ##### Query Complexity Classification

[L184]

[L185] | Intent | Example | Complexity | Traversal Depth |

[L186] |-----|-----|-----|-----|

[L187] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |

[L188] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L189] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L190] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L191] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

---

DEEP RESEARCH 20/08/2026, 15:06:35

Citation Marker: `filecite[turn17file0]`

[L1] metadata:

[L2] ref\_id: turn13file7

[L3] file\_id: file\_00000000f620821186e54e582d59731e

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Final\_Report(3).md

[L6] lines\_shown: 500-574

[L7] total\_file\_lines: 1527

[L8] has\_more: True

[L9] next\_start\_line: 575

[L10] next\_read: {"ref\_id":"turn13file7","mode":"full","start\_line":575,"max\_lines":75}

[L11] content:

[L12] 2. Find connected components of near-duplicate units

[L13] 3. For each group: select canonical name (most frequent), merge all aliases, definitions, properties, and evidence

[L14]

[L15]

```
[L16] def alias_resolution(units: list) -> list:
[L17] """Merge KnowledgeUnits referring to the same concept.
[L18]
[L19] Merge strategy:
[L20] - Canonical name: most frequent name across units
[L21] - Aliases: union of all names and aliases
[L22] - Definitions: union of all definitions
[L23] - Confidence: average of unit extraction confidences
[L24] - Properties: merged (latest value wins, conflicts logged)
[L25] """
[L26] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L27] # ... build LSH index from concept names + aliases ...
[L28] merge_groups = find_connected_components(units, lsh)
[L29] return [create_canonical_node(group) for group in merge_groups]
[L30]
```

[L31]

[L32] ---

[L33]

[L34] ##### 5.4 Layer 4: Relation Management

[L35]

[L36] **Responsibility:** Validate candidate relations from Layer 3 against the relation ontology, score confidence, resolve conflicts, and produce final `KnowledgeEdge` objects.

[L37]

[L38] **Input:** `KnowledgeNode[]` (with raw relations) → **Output:** `KnowledgeEdge[]` (validated, scored)

[L39]

[L40] **Pipeline:**

[L41] 1. **Validate relation type** — check against ontology; if unknown type, attempt inference via LLM

[L42] 2. **Resolve target node** — map target concept name to a `KnowledgeNode`

[L43] 3. **Check for duplicates** — merge evidence into existing edge if duplicate

[L44] 4. **Determine edge class** — hierarchical or semantic based on ontology mapping

[L45] 5. **Calculate confidence** — weighted combination of source confidence, evidence length, and semantic similarity

[L46]

[L47]

```
[L48] def calculate_edge_confidence(raw_rel, source, target) -> float:
[L49] """Confidence = 0.5 (base) + 0.15 * source_conf + 0.10 * evidence_quality + 0.10 * sim"""
[L50] conf = 0.5
[L51] conf += 0.15 * source.confidence
[L52] conf += 0.10 * min(len(raw_rel.get('evidence', '')) / 200, 1.0)
[L53] conf += 0.10 * cosine_similarity(source.embedding, embed(raw_rel['evidence']))
[L54] return min(conf, 1.0)
[L55]
```

[L56]

[L57] ---

[L58]

[L59] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L60]

[L61] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L62]

[L63] **Input:** `KnowledgeNode[]` + `KnowledgeEdge[]` → **Output:** Knowledge Forest

[L64]

[L65] ##### Pipeline Overview

[L66]

[L67]

```
[L68] graph LR
[L69] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L70] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L71] PRE --> HV["HV
Validate #1?"]
[L72] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L73] HV -->|"Fail"| HV2["HV
Validate #2?"]
[L74] HV2 -->|"Pass"| FI
[L75] HV2 -->|"Fail"| HV3["HV
Validate #3?"]
[L76] HV3 -->|"Pass"| FI
[L77] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L78] FI --> CLD["CLD
Discover Cross-Links"]
[L79] CLD --> DONE["Node Placed ✓"]
[L80]
```

[L81]

```
[L82] ##### 5.5.1 CPG++ — Candidate Parent Generator
[L83]
[L84] Generates up to 5 candidate parents for a new node using a multi-stage filter:
[L85]
[L86] ```python
```

CHATGPT 20/08/2026, 15:06:35

Layer 4 — Relation Management

Ab hum wahi same Smart Hospital example continue karenge jo Layer 0, 1, 2 aur 3 me use kiya tha.

Layer 4 ko samajhne ka sabse important point:

Layer 3 relation ko extract karta hai. Layer 4 decide karta hai ki relation valid hai ya nahi, duplicate hai ya nahi, kis node se connect hoga, aur us par kitna confidence hoga.

Report ke according Layer 4 ka:

- **Input:** KnowledgeNode[] with raw relations
  - **Output:** KnowledgeEdge[]
  - **Main work:** ontology validation, target resolution, duplicate handling, edge classification, confidence scoring.
- fileciteurn17file0L34-L55

1. Previous layers se Layer 4 tak kya aaya?

Layer 3 ka final output

Hamare Smart Hospital example me Layer 3 ne canonical nodes banaye:

```
KnowledgeNode[]
|
├─ AI Diagnostic Engine
├─ Machine Learning Model
├─ Patient Symptoms
├─ Medical History
├─ Diagnostic Images
├─ Emergency Risk
└─ Preliminary Risk Score
```

Aur AI Diagnostic Engine node ke andar raw relations hain:

```
AI Diagnostic Engine
 USES
 Machine Learning Model

AI Diagnostic Engine
 ANALYZES
 Patient Symptoms

AI Diagnostic Engine
 ESTIMATES
 Emergency Risk
```



Lekin yaad rakho:

RAW RELATION  
≠  
FINAL KNOWLEDGE EDGE

Yahi Layer 4 ka kaam hai.

2. Complete Layer 4 flow

KnowledgeNode[]  
with raw relations  
↓  
Step 1: Relation Type Validation  
↓  
Step 2: Target Node Resolution  
↓  
Step 3: Duplicate Check  
↓  
Step 4: Edge Class Determination  
↓  
Step 5: Confidence Calculation  
↓  
Conflict Handling  
↓  
KnowledgeEdge[]  
↓  
Layer 5

Ye exact high-level pipeline report me defined hai. `filecite{turn17file0}{L40-L45}`

3. Layer 3 vs Layer 4 — exact difference

| Layer 3 | Layer 4 |  
|---|---|  
| Relation **extract** karta hai | Relation **validate** karta hai |  
| Raw/candidate relation | Final/scored relation |  
| "USES" detect karta hai | Check karta hai "USES" ontology me valid hai ya nahi |  
| Target name text hota hai | Actual KnowledgeNode resolve karta hai |  
| Duplicate relation create ho sakta hai | Duplicates merge karta hai |  
| Confidence extraction level ka hota hai | Edge confidence calculate karta hai |  
| Output: KnowledgeNode[] | Output: KnowledgeEdge[] |

Report bhi Layer 3 ko **candidate extraction** aur Layer 4 ko **validation and scoring** boundary ke roop me define karti hai. `filecite{turn13file9}{L633-L639}`

4. Layer 4 ka exact input

KnowledgeNode[]

Lekin har node ke paas uske raw relations hone chahiye.

Example:

```
KnowledgeNode

node_id:
NODE_001

name:
AI Diagnostic Engine

confidence:
0.85

embedding:
[...]

raw_relations:
[
 {
 source: "AI Diagnostic Engine",
 type: "USES",
 target: "Machine Learning Model",
 evidence:
 "The system uses a machine learning model
 to generate a preliminary risk score."
 }
]
```

Phir doosra node:

```
KnowledgeNode

node_id:
NODE_002

name:
Machine Learning Model

confidence:
0.90

embedding:
[...]
```

---

## 5. Layer 4 ka final output kya hai?

```
KnowledgeEdge[]
```

Conceptually:

```
KnowledgeEdge

source_node_id
target_node_id
relation_type
edge_class
confidence
evidence
```

Is layer ka final result:

```
NODE_001
 |
 | USES
 | confidence = 0.78
 ▼
NODE_002
```

Overall pipeline me Layer 4 `KnowledgeNode[]` ko validated, typed aur scored `KnowledgeEdge[]` me convert karta hai. `filecite\turn13file7\L497-L533`

## 6. Step 1 — Relation Type Validation

Ab hamara raw relation lo:

```
AI Diagnostic Engine
 |
 | USES
 |
Machine Learning Model
```

Layer 4 sabse pehle puchega:

`"USES"` valid relation type hai ya nahi?

Report ke according relation type ko ontology ke against check kiya jata hai. Agar relation type unknown ho, to LLM inference attempt kiya jata hai. `filecite\turn17file0\L40-L45`

## Relation ontology ko simple language me samjho

Ontology ko tum allowed relation dictionary samajh sakte ho.

Example:

```
IS_A
PART_OF
USES
CAUSES
...
```

Layer 3 ne agar relation diya:

```
AI Diagnostic Engine
 |
 | USES
 |
Machine Learning Model
```

Then:

`USES ∈ Relation Ontology`

Therefore:

`VALID ✓`

## 7. Unknown relation type ka case

Maan lo Layer 3 LLM ne extract kiya:

```

AI Diagnostic Engine
 DEPENDS_ON
Machine Learning Model

```

Lekin:

```
DEPENDS_ON
```

ontology me directly defined nahi hai.

Then report ke according:

```

Unknown relation type
 ↓
Attempt inference via LLM

```

filecite[turn17file0L40-L45]

Conceptually LLM infer kar sakta hai:

```

DEPENDS_ON
 ↓
Possible semantic mapping
 ↓
USES

```

Ya:

```

Cannot safely map
 ↓
Unresolved / rejected

```

### Important

Report me **LLM inference attempt** clearly mentioned hai, lekin exact inference prompt, fallback behavior, ya unknown relation ko reject/store karne ka detailed pseudocode shown section me specified nahi hai.

## 8. Step 1 me possible errors

### Error A — Wrong relation type

Text:

```
The diagnostic module is part of the hospital platform.
```

Correct:

```

Diagnostic Module
 PART_OF
Hospital Platform

```

Layer 3 accidentally:

```

Diagnostic Module
 IS_A
Hospital Platform ❌

```

Layer 4 ontology validation karega, lekin **sirf type ontology me valid hone se semantic correctness guarantee nahi hoti.**

Because:

```
IS_A
```

ontology me valid ho sakta hai, but is specific pair ke liye wrong hai.

**Important observation**

*Report says relation type is validated against ontology, but the shown Layer 4 section does not specify a separate semantic entailment test for whether this exact source-target pair truly supports that relation.*

Ye ek potential missing validation detail hai.

**9. Step 2 — Target Node Resolution**

Raw relation initially text-based ho sakta hai:

```
source:
AI Diagnostic Engine

target:
Machine Learning Model
```

Layer 4 ko actual target node dhoondhna hai:

```
"Machine Learning Model"
 ↓
NODE_002
```

Report explicitly kehti hai:

*Target concept name ko KnowledgeNode se map karo. [filecite turn 17file0 L40-L44](#)*

**Example**

System ke paas:

```
NODE_002
Name:
Machine Learning Model

Aliases:
[
 ML Model,
 Predictive Model
]
```

Raw relation target:

```
"ML Model "
```

Layer 4:

```

"ML Model"
 ↓
Check canonical names
 ↓
Check aliases
 ↓
NODE_002 found ✓

```

Final edge text se ID-based connection me convert ho sakta hai:

```

NODE_001
 USES
NODE_002

```

## 10. Target node resolution kyun important hai?

Without this:

```

AI Diagnostic Engine
 USES
"Machine Learning Model"

```

aur:

```

AI Diagnostic Engine
 USES
"ML Model"

```

system ke liye do different targets lag sakte hain.

Layer 3 alias resolution ke baad ideally:

```

Machine Learning Model
Aliases:
ML Model

```

already merged honge.

Layer 4 us canonical node ko resolve karta hai.

## 11. Target Resolution me possible errors

### Error A — Target missing

Raw relation:

```

AI Diagnostic Engine
 USES
Deep Neural Risk Model

```

Lekin `Deep Neural Risk Model` ka node exist hi nahi karta.

Possible reasons:

1. Layer 3 ne target concept extract nahi kiya  
2. Typo hai  
3. Alias resolution fail hua  
4. LLM hallucinated target

Then:

Target resolution failure

Report ke shown Layer 4 pipeline me target resolve karna defined hai, but unresolved target ke exact exception/fallback handling ka detailed behavior explicitly specified nahi hai. [filecite\turn17file0\L40-L45](#)

12. Step 3 — Duplicate Check

Ab maan lo same relation multiple documents me mila.

Document 1

The AI diagnostic engine uses a machine learning model.

Extracted:

AI Diagnostic Engine  
USES  
Machine Learning Model

Evidence 1 stored.

Document 2

The diagnostic AI relies on an ML model for risk prediction.

After alias resolution:

Diagnostic AI  
→ AI Diagnostic Engine  
  
ML Model  
→ Machine Learning Model

Relation again becomes:

AI Diagnostic Engine  
USES  
Machine Learning Model

Layer 4 duplicate check karega.

Report ke according:

Duplicate edge?  
↓ YES  
Merge evidence into existing edge

[filecite\turn17file0\L40-L44](#)

### 13. Duplicate merge ke baad

Instead of:

```
Edge 1:
NODE_001 → USES → NODE_002

Edge 2:
NODE_001 → USES → NODE_002
```

System:

```
ONE EDGE

NODE_001
 |
 | USES
 ▼
NODE_002

Evidence:
[
 Evidence from Document 1,
 Evidence from Document 2
]
```

Ye knowledge graph ko clean rakhta hai.

### 14. Duplicate check me relation direction important hai

Ye dono same nahi hain:

```
A USES B
```

versus:

```
B USES A
```

Example:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Reverse:

```
Machine Learning Model
 USES
AI Diagnostic Engine
```

semantic meaning completely change ho gaya.

Isliye edge identity conceptually:

```
(Source, Relation Type, Target)
```

order-sensitive honi chahiye.



Report duplicate check mention karti hai, lekin exact edge-key implementation shown Layer 4 section me explicitly defined nahi hai.

## 15. Step 4 — Edge Class Determination

Ab relation type validate ho gaya:

USES

Target resolve ho gaya:

NODE\_002

Ab Layer 4 decide karega:

*Ye relation **hierarchical** hai ya **semantic**?*

Report:

```
Determine edge class
=
hierarchical or semantic
based on ontology mapping
```

filecite\turn17file0\L40-L45

## Hierarchical relation kya hota hai?

Example:

```
Machine Learning Model
 IS_A
Artificial Intelligence Model
```

Ya:

```
Diagnostic Module
 PART_OF
Smart Hospital Platform
```

Ye hierarchy build karne me useful hain.

## Semantic relation kya hota hai?

Example:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Ya:

```
Patient Symptoms
 CAUSES
Risk Alert
```

Ye concepts ko connect karte hain, but necessarily parent-child hierarchy nahi banate.

## 16. Edge Class ka Layer 5 se connection

Layer 5 Knowledge Forest construct karega.

Report ke Layer 5 description me input explicitly:

```
KnowledgeNode[] + KnowledgeEdge[]
```

hai. `filecite{turn17file0}{L59-L63}`

So Layer 4:

```
validated relation
 ↓
HIERARCHICAL?
 ↓
Layer 5 hierarchy construction me useful
```

Ya:

```
SEMANTIC?
 ↓
Later cross-link / semantic connection ke liye useful
```

## 17. Step 5 — Confidence Calculation

Ye Layer 4 ka most mathematical part hai.

Report exact formula deti hai:

```
Confidence
=
0.5
+ 0.15 × Source Confidence
+ 0.10 × Evidence Quality
+ 0.10 × Semantic Similarity
```

Evidence quality ko evidence length se calculate kiya gaya hai:

```
min(
 evidence_length / 200,
 1.0
)
```

Aur similarity:

```
cosine_similarity(
 source.embedding,
 embedding(evidence)
)
```

Finally confidence maximum `1.0` tak capped hai. `filecite{turn17file0}{L47-L55}`

## 18. Formula ko hamare example par apply karte hain

Raw relation:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Suppose:

```
Source confidence = 0.85
```

Evidence:

```
"The system uses a machine learning model to generate
a preliminary risk score."
```

Assume:

```
Evidence length = 100 characters
```

Then:

```
Evidence quality
=
min(100 / 200, 1)
=
0.5
```

Assume:

```
Semantic similarity = 0.80
```

Now:

```
Confidence
=
0.5
+ (0.15 × 0.85)
+ (0.10 × 0.50)
+ (0.10 × 0.80)
```

```
=
0.5
+ 0.1275
+ 0.05
+ 0.08
```

```
= 0.7575
```

Final:

```
Edge Confidence = 0.7575
```

## 19. Formula ko deeply samjho

### Part 1 — Base confidence

```
0.5
```

Har edge 0.5 se start hota hai.

Meaning:

```
No relation starts from zero.
```

---

## Part 2 — Source confidence

```
0.15 × source.confidence
```

Agar source node:

```
confidence = 0.90
```

then contribution:

```
0.135
```

Agar:

```
confidence = 0.40
```

then contribution:

```
0.06
```

Meaning:

*Better-quality source node se extracted relation ko higher confidence contribution milta hai.*

---

## Part 3 — Evidence quality

```
0.10 × min(evidence_length / 200, 1.0)
```

Agar evidence length:

```
50
```

Then:

```
50 / 200 = 0.25
Contribution = 0.025
```

Agar:

```
200
```

Then:

```
200 / 200 = 1
Contribution = 0.10
```

Agar:

500

Still:

```
min(500/200, 1)
= 1
```

Maximum contribution:

0.10

## 20. Evidence length ko "evidence quality" kehna ek limitation hai

Ye **bahut important critical observation** hai.

Long evidence:

500 characters

necessarily:

High-quality evidence

nahi hota.

Example:

```
"This is a hospital system. It has many modules.
Many modules perform many functions..."
```

long hai but relation support weak ho sakta hai.

Aur:

```
"The engine uses the ML model."
```

short hai but direct evidence hai.

Isliye:

*Evidence length ≠ Evidence correctness*

Report ke formula me evidence quality length proxy ke through measured hai. Exact semantic entailment check Layer 4 ke shown formula me included nahi hai. [filecite\turn17file0\L47-L55](https://chatgpt.com/e2b31603-f9df-4055-82cd-e524a077a7c4)

## 21. Part 4 — Semantic Similarity

Formula:

```
cosine_similarity(
 source.embedding,
 embed(evidence)
)
```

Example:

Source:  
AI Diagnostic Engine

Evidence:  
The AI diagnostic engine uses a machine learning model.

Similarity high ho sakti hai.

Suppose:

0.90

Contribution:

$0.10 \times 0.90$   
= 0.09

## 22. Ek important issue: similarity source aur evidence ke beech hai

Notice formula:

source.embedding  
vs  
evidence.embedding

Target relation:

Machine Learning Model

directly similarity calculation me shown formula ke andar include nahi hai. [filecite\turn17file0\L47-L55](#)

Potential risk:

Evidence source concept ke baare me highly relevant ho sakta hai, but extracted relation target ke saath wrong ho sakta hai.

Example:

Evidence:  
AI Diagnostic Engine processes medical information.

Source similarity:

high

But LLM accidentally relation banaye:

AI Diagnostic Engine  
USES  
Quantum Computer

Source-evidence similarity high ho sakti hai despite target unsupported hona.

**This is a genuine limitation of the shown confidence formula.**

## 23. Complete confidence range

Formula ka maximum:

```
0.5
+ 0.15
+ 0.10
+ 0.10
=
0.85
```

Then code:

```
return min(conf, 1.0)
```

karta hai. `filecite\turn17file0\L47-L55`

### Critical observation

Shown formula ke according, agar:

```
source confidence = 1
evidence quality = 1
similarity = 1
```

tab bhi:

```
maximum = 0.85
```

So normal shown inputs ke under `1.0` cap actually practically reach nahi hota.

Ye formula design me investigate karne layak point hai.

---

## 24. Conflict Resolution

Layer 4 ki responsibility me explicitly:

```
resolve conflicts
```

included hai. `filecite\turn17file0\L34-L38`

Lekin important distinction:

### Report kya clearly kehti hai?

```
Layer 4 conflicts resolve karega.
```

### Report ke shown Layer 4 snippet me kya missing hai?

Exact algorithm:

```
IF conflicting relation:
 then what?
```

Fully specified nahi hai.

For example:

Document A:  
  
AI Diagnostic Engine  
    USES  
Machine Learning Model

Aur:

Document B:  
  
AI Diagnostic Engine  
    DOES\_NOT\_USE  
Machine Learning Model

Ya conceptual conflict:

Module A  
    PART\_OF  
System X

and:

Module A  
    PART\_OF  
System Y

Shown Layer 4 section me exact conflict-resolution rules, priorities, voting, timestamps, evidence counts, ya contradiction policy explicitly defined nahi hai.

So is point par **main assumption nahi kar raha**.

---

25. Layer 4 ka complete walkthrough — ek raw relation

Chalo ek relation ko end-to-end process karte hain.

Input from Layer 3

Source:  
AI Diagnostic Engine  
  
Type:  
USES  
  
Target:  
ML Model  
  
Evidence:  
"The AI diagnostic engine uses an ML model  
to generate emergency risk predictions."

Step 1 — Source resolve

AI Diagnostic Engine  
    ↓  
NODE\_001



## Step 2 — Relation type validation

```
USES
 ↓
Ontology lookup
 ↓
Valid ✓
```

## Step 3 — Target resolution

Target:

ML Model

Canonical node:

NODE\_002  
Machine Learning Model

Aliases:

ML Model ✓

Therefore:

Target resolved ✓

## Step 4 — Duplicate check

Existing graph me search:

NODE\_001  
USES  
NODE\_002

### Case A: Already exists

YES

Then:

New evidence  
↓  
Merge into existing edge

### Case B: Does not exist

NO

Then new edge candidate.

## Step 5 — Edge class

USES

Based on ontology mapping:

SEMANTIC\_EDGE

## Step 6 — Confidence

Suppose:

Source confidence = 0.85  
Evidence quality = 0.80  
Semantic similarity = 0.90

Calculation:

0.5  
+ 0.15(0.85)  
+ 0.10(0.80)  
+ 0.10(0.90)

= 0.7975

## Final output

```
KnowledgeEdge

edge_id:
EDGE_001

source:
NODE_001

target:
NODE_002

relation_type:
USES

edge_class:
SEMANTIC

confidence:
0.7975

evidence:
[
 "The AI diagnostic engine uses an ML model..."
]
```

This is the final Layer 4 product.

## 26. Multiple evidence ka example

Suppose 3 documents same relation support karte hain.

### Evidence 1

The engine uses an ML model for prediction.

### Evidence 2

Diagnostic AI relies on machine learning algorithms.

### Evidence 3

The risk prediction module is based on a trained ML model.

After alias and duplicate handling:

ONE KnowledgeEdge

AI Diagnostic Engine  
USES  
Machine Learning Model

with:

```
Evidence:
[
 Evidence 1,
 Evidence 2,
 Evidence 3
]
```

Report explicitly duplicate relation milne par evidence existing edge me merge karne ko kehti hai.

filecite\turn17file0\L40-L45

## 27. Layer 4 me possible errors

### Error 1 — Unknown relation type

AI Diagnostic Engine  
COOPERATES\_WITH  
Machine Learning Model

Ontology me nahi hai.

Action:

Attempt LLM inference

But LLM:

USES

map kar sakta hai incorrectly.

### Risk

```

Unknown relation
 ↓
LLM inference
 ↓
Wrong mapping

```

## 28. Error 2 — Wrong target resolution

Raw:

```
Target = Risk Model
```

System me:

```

NODE_A = Emergency Risk Model
NODE_B = Financial Risk Model

```

Question:

```
"Risk Model" kis node ko refer karta hai?
```

Wrong mapping:

```

AI Diagnostic Engine
 USES
 Financial Risk Model ❌

```

Report target mapping mention karti hai, but ambiguous target disambiguation ka detailed algorithm shown Layer 4 section me specified nahi hai.

## 29. Error 3 — Duplicate relation not detected

Same meaning:

```
USES
```

and:

```
RELIES_ON
```

After ontology inference, maybe both:

```
USES
```

ban jaye.

Agar normalization properly nahi hui:

```
Duplicate semantic edges
```

create ho sakte hain.

## 30. Error 4 — Wrong edge class

Relation:

PART\_OF

should be:

HIERARCHICAL

But mistakenly:

SEMANTIC

classified.

Then Layer 5 hierarchy construction miss kar sakta hai.

Reverse case:

USES

wrongly hierarchical bana diya.

Then hierarchy artificial aur incorrect ho sakti hai.

---

### 31. Error 5 — High confidence for wrong relation

Because formula starts at:

0.5 base

even a weak relation ko significant starting confidence milta hai.

Suppose:

```
Source confidence = 0.80
Evidence length quality = 0.80
Similarity = 0.90
```

High score mil sakta hai.

But extracted target wrong ho sakta hai.

So:

```
High confidence
≠
Guaranteed factual correctness
```

---

### 32. Error 6 — Evidence too short

Evidence:

Uses ML.

Meaning clear ho sakta hai, but length low:

Evidence quality contribution low

So confidence artificially lower ho sakta hai.

### 33. Error 7 — Evidence too long

Long irrelevant paragraph:

200+ characters

Formula maximum evidence contribution de sakta hai:

0.10

Even if most of paragraph relation ko directly support nahi karta.

### 34. Error 8 — Source-only semantic similarity

Shown formula:

`similarity(source, evidence)`

not explicitly:

`similarity(source + relation + target, evidence)`

Therefore target hallucination potentially detect nahi hogi through this term alone.

Ye report ke shown implementation se directly derived critical observation hai. [filecite\turn17file0\L47-L55](#)

### 35. Exception Handling — Layer 4 me kya clearly defined hai?

Yahan mujhe strict rehna hoga.

#### Explicitly mentioned

##### 1. Unknown relation type

Unknown  
↓  
Attempt inference via LLM

[filecite\turn17file0\L40-L45](#)

##### 2. Duplicate relation

Duplicate  
↓  
Merge evidence

[filecite\turn17file0\L40-L45](#)

##### 3. Confidence cap

```
min(conf, 1.0)
```

Confidence overflow prevent karta hai. `filecite\turn17file0\L47-L55`

---

### 36. Exception handling me missing parts

Shown Layer 4 section me explicitly defined nahi hai:

```
LLM inference timeout ho gaya to?
```

```
Target node resolve nahi hua to?
```

```
Embedding generation fail hui to?
```

```
Evidence missing ho to?
```

```
Cosine similarity NaN ho to?
```

```
Conflicting edges ko exactly kaise resolve karein?
```

Isliye inke exact behaviors ke baare me **"report me specified hai"** nahi bola ja sakta.

---

### 37. Mere according missing implementation points — clearly separate

#### Suggestion / Critical Analysis — Not explicitly specified in the report

A robust implementation ideally ye kare:

```
Target not found
↓
Try alias lookup
↓
Try semantic lookup
↓
If still unresolved:
 mark edge UNRESOLVED
 do not silently create wrong edge
```

Unknown relation:

```
LLM inference
↓
Validate inferred type again
↓
If still uncertain:
 QUARANTINE relation
```

Embedding failure:

```

Retry
↓
Fallback embedding model
↓
If still failed:
 mark similarity contribution unavailable

```

Ye suggestions hain, report ke explicit implementation ka part nahi.

### 38. Layer 4 ka complete pseudocode — report-based logic

```

knowledge_edges = []

FOR each KnowledgeNode source:

 FOR each raw_relation in source.raw_relations:

 relation_type = raw_relation.type

 # STEP 1
 IF relation_type not in ontology:
 relation_type = infer_relation_via_LLM()

 # STEP 2
 target = resolve_target_node(
 raw_relation.target
)

 # STEP 3
 IF edge already exists:
 merge evidence
 CONTINUE

 # STEP 4
 edge_class = ontology_mapping(
 relation_type
)

 # STEP 5
 confidence =
 0.5
 + 0.15 × source_confidence
 + 0.10 × evidence_quality
 + 0.10 × semantic_similarity

 create KnowledgeEdge

RETURN KnowledgeEdge[]

```

Ye report ke stated pipeline aur shown confidence function ka simplified integrated representation hai.

filecite\turn17file0\L34-L55

### 39. Hamare Smart Hospital project ka complete Layer 4 output

Input relations:

```

AI Diagnostic Engine
 USES
Machine Learning Model

```



AI Diagnostic Engine  
ANALYZES  
Patient Symptoms

AI Diagnostic Engine  
ANALYZES  
Medical History

AI Diagnostic Engine  
ANALYZES  
Diagnostic Images

AI Diagnostic Engine  
ESTIMATES  
Emergency Risk

Layer 4 process ke baad:

EDGE\_001

AI Diagnostic Engine  
— USES —>  
Machine Learning Model

Class:  
SEMANTIC

Confidence:  
0.79

EDGE\_002

AI Diagnostic Engine  
— ANALYZES —>  
Patient Symptoms

Class:  
SEMANTIC

Confidence:  
0.81

EDGE\_003

AI Diagnostic Engine  
— ANALYZES —>  
Medical History

Class:  
SEMANTIC

Confidence:  
0.77

EDGE\_004

AI Diagnostic Engine

— ANALYZES —>

Diagnostic Images

Class:

SEMANTIC

Confidence:

0.82

EDGE\_005

AI Diagnostic Engine

— ESTIMATES —>

Emergency Risk

Class:

SEMANTIC

Confidence:

0.80

## 40. Ek hierarchical example bhi dekhte hain

New raw relation:

AI Diagnostic Engine

PART\_OF

Smart Hospital AI Platform

Layer 4:

### Relation validation

PART\_OF

→ Valid ✓

### Target resolution

Smart Hospital AI Platform

→ NODE\_100 ✓

### Edge class

PART\_OF

→ HIERARCHICAL

### Confidence calculate

0.76

Final:

```
EDGE_006

AI Diagnostic Engine
 PART_OF
Smart Hospital AI Platform

Class:
HIERARCHICAL

Confidence:
0.76
```

Ye edge Layer 5 ke hierarchy construction me important hoga.

Layer 5 ka input exactly nodes aur validated edges hai. `filecite\turn17file0\L59-L63`

---

## 41. Layer 4 ke baad data exactly kaisa dikhega?

### Before Layer 4

```
KnowledgeNode

AI Diagnostic Engine

raw_relations:
[
 ("USES", "ML Model", evidence),
 ("PART_OF", "Hospital AI Platform", evidence),
 ("ANALYZES", "Patient Symptoms", evidence)
]
```

---

### After Layer 4

```
KnowledgeEdge[]
```

```
EDGE_001:
NODE_001 — USES —> NODE_002
confidence = 0.79
class = SEMANTIC
```

```
EDGE_002:
NODE_001 — PART_OF —> NODE_100
confidence = 0.76
class = HIERARCHICAL
```

```
EDGE_003:
NODE_001 — ANALYZES —> NODE_005
confidence = 0.81
class = SEMANTIC
```

---

## 42. Layer 4 ka Layer 5 ko exact handoff

```
Layer 3
KnowledgeNode[]
+
Raw Relations
↓
Layer 4
↓
Validate
Resolve
Deduplicate
Classify
Score
↓
KnowledgeEdge[]
↓
Layer 5
↓
Knowledge Forest Construction
```

Overall architecture me bhi Layer 4 ke baad Layer 5 SHIA Core hierarchy construction start hota hai.

filecite[turn13file7][L475-L511]

### 43. Layer 4 ka biggest purpose

Agar Layer 4 nahi hota:

```
LLM extracted relation
↓
Directly Knowledge Graph
```

Problem:

```
Wrong relation
Hallucinated target
Duplicate edge
Unknown relation type
Wrong classification
Uncontrolled confidence
```

sab directly graph me chale jate.

Layer 4 ek:

### Quality Gate / Relation Firewall

ki tarah kaam karta hai.

Layer 3:

"I found this relation."

↓

Layer 4:

"Wait. Is this relation valid?

Does this target exist?

Is this duplicate?

What type of edge is it?

How confident should we be?"

↓

Layer 5:

"Now use validated edges to build the Knowledge Forest."

---

## 44. Final memory trick

### Layer 3

WHO is connected to WHOM?

### Layer 4

IS THAT CONNECTION VALID?

### Layer 5

WHERE SHOULD EACH NODE LIVE  
IN THE KNOWLEDGE FOREST?

---

## Layer 4 in one complete diagram

```

RAW RELATION
from Layer 3

AI Diagnostic Engine
 USES
ML Model

 ↓

1. VALIDATE TYPE

USES valid?

 ↓ YES

2. RESOLVE TARGET

ML Model
 ↓
Machine Learning Model
NODE_002

 ↓

3. CHECK DUPLICATE

Existing edge?

YES → merge evidence
NO → create candidate

 ↓

4. DETERMINE CLASS

USES
 ↓
SEMANTIC

 ↓

5. CALCULATE CONFIDENCE

Base
+ Source Confidence
+ Evidence Quality
+ Semantic Similarity

 ↓

FINAL

KnowledgeEdge

NODE_001
| USES
| confidence = 0.79
▼
NODE_002

 ↓
Layer 5

```

#### Sabse important one-line understanding:

*Layer 4 raw relations ko blindly accept nahi karta; unhe ontology ke against validate, actual nodes se resolve, duplicates se clean, hierarchical/semantic class me divide, aur confidence score karke final KnowledgeEdge objects*

*banata hai.*

YOU 20/08/2026, 15:15:45

layer 5 smjhao

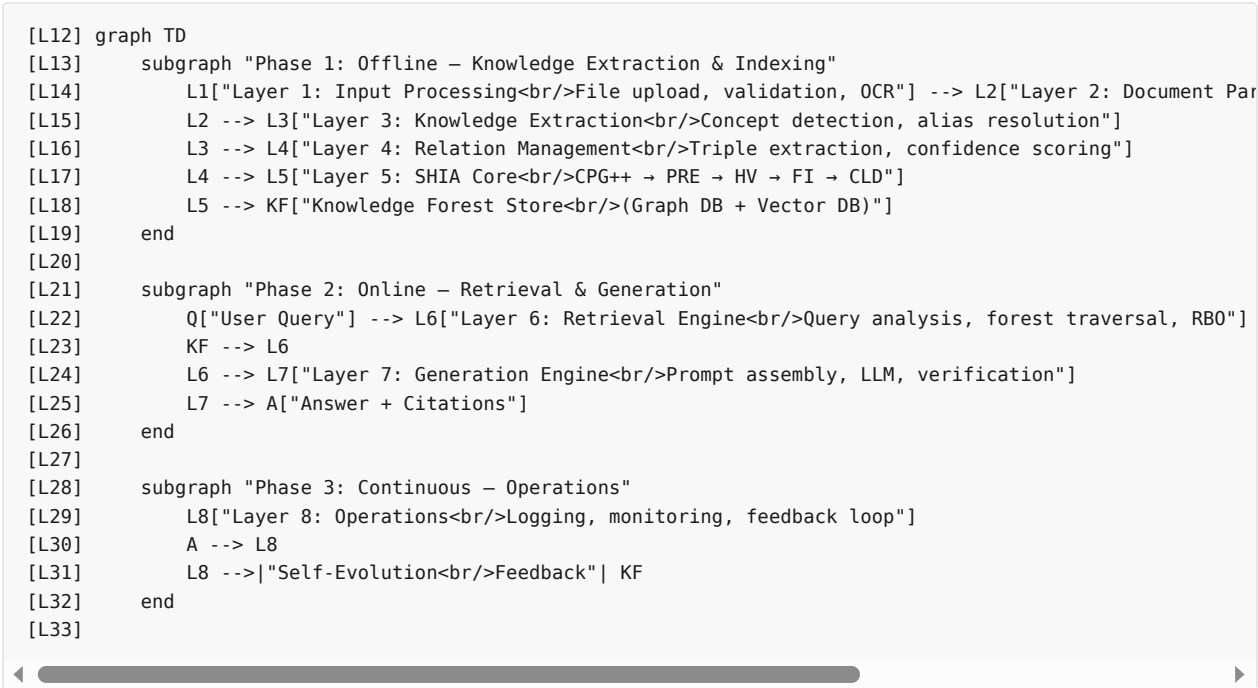
CHATGPT 20/08/2026, 15:15:51

{"paths":["files"]}

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite`turn18file0

- [L1] metadata:
- [L2] query\_result\_index: 0
- [L3] file\_id: file\_0000000012688211b4f57f5b8bcb8b42
- [L4] version\_id: 1
- [L5] name: SHIA\_RAG\_Final\_Report(4).md
- [L6] mime\_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.03225806451612903
- [L9] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--6
- [L10] snippet:
- [L11]



- [L34]
- [L35] ### 3.2 Data Flow
- [L36]
- [L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47] ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53] ↓ Layer 7: Build prompt, invoke LLM
[L54] ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56] ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

```
[L58]
[L59] ### 3.3 Layer Summary
[L60]
[L61] | Layer | Name | Input | Output | Key Algorithms |
[L62] |-----|-----|-----|-----|-----|
[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0
```

Citation Marker: `filecite`turn18file1

```
[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(4).md
[L81] mime_type: text/markdown
[L82] surface: conversation
[L83] score: 0.032018442622950824
[L84] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--0
```



[L85] snippet:

[L86] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation

[L87]

[L88] > **Project Type:** Minor Project (M.Tech)

[L89] > **Date:** August 2026

[L90] > **Author:** [Your Name]

[L91]

[L92] ---

[L93]

[L94] ## Abstract

[L95]

[L96] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.

[L97]

[L98] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS\_A, PART\_OF) and semantic (USES, CAUSES) relationships.

[L99]

[L100] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L101]

[L102] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L103]

[L104] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L105]

[L106] ---

[L107]

[L108] ## Table of Contents

[L109]

[L110] 1. Introduction

[L111] 2. Literature Review

[L112] 3. System Architecture

[L113] 4. Data Model — Layer 0

[L114] 5. Offline Pipeline — Layers 1-5

[L115] 6. Online Pipeline — Layers 6-7

[L116] 7. Operations Layer — Layer 8

[L117] 8. Novel Contributions

[L118] 9. Mathematical Framework

[L119] 10. Implementation

[L120] 11. Evaluation Framework

[L121] 12. Risk Analysis

[L122] 13. Conclusion & Future Work

[L123]

[L124] ---

[L125]

[L126] ## 1. Introduction

[L127]

[L128] ### 1.1 Problem Statement

[L129]

[L130] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L131]

[L132] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L133] 2. Embed each chunk into a vector space

[L134] 3. At query time, retrieve the top-k most similar chunks

[L135] 4. Feed these chunks as context to an LLM for answer generation

[L136]

[L137] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: □filecite□turn18file2□

[L138] metadata:

[L139] query\_result\_index: 2

[L140] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L141] version\_id: 1

[L142] name: SHIA\_RAG\_Final\_Report(4).md

[L143] mime\_type: text/markdown

[L144] surface: conversation

[L145] score: 0.03200204813108039

[L146] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--7

[L147] snippet:

[L148] deduplicated)

[L149] ↓ Layer 4: Validate relations against ontology, score confidence

[L150] KnowledgeEdge objects (typed, scored)

[L151] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)

[L152] Knowledge Forest (rooted trees + cross-links)

[L153] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection

[L154] Selected context nodes (within token budget)

[L155] ↓ Layer 7: Build prompt, invoke LLM

[L156] ↓ Layer 7: Verify claims against evidence, bind citations

[L157] Answer with citations + confidence score

[L158] ↓ Layer 8: Log, collect feedback, update forest weights

[L159]

```
[L160]
[L161] ### 3.3 Layer Summary
[L162]
[L163] | Layer | Name | Input | Output | Key Algorithms |
[L164] |-----|-----|-----|-----|-----|
[L165] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L166] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L167] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L168] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L169] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L170] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L171] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L172] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L173] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L174]
[L175] ---
[L176]
[L177] ## 4. Data Model – Layer 0
[L178]
[L179] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L180]
[L181] ### 4.1 KnowledgeNode Schema
[L182]
[L183] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L184]
[L185]
```

json

```
[L186] {
[L187] "node_id": "NODE_000789",
[L188] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L189] "canonical_name": "Round Robin",
[L190] "aliases": ["RR", "Round-Robin Scheduling"],
[L191] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L192] "summaries": ["CPU scheduling method using time slices"],
[L193] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L194] "domain": "Operating Systems",
[L195] "properties": {
[L196] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L197] "preemptive": {"type": "boolean", "value": true}
[L198] },
[L199] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L200] "parent": "NODE_000456",
[L201] "children": ["NODE_001001", "NODE_001002"],
[L202] "related": ["NODE_000890"],
[L203] "incoming_edges": ["EDGE_001"],
[L204] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L205] "evidence": [
```

[L206] {  
[L207] "evidence\_type": "DEFINITION",  
[L208] "text": "Round Robin is a preemptive scheduling algorithm...",  
[L209] "source\_doc": "DOC\_000001",  
[L210] "source\_block": "BLK\_000234",  
[L211] "page": 42  
[L212] }  
[L213] ],

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `[[filecite]]turn18file3[[`

[L214] metadata:  
[L215] query\_result\_index: 3  
[L216] file\_id: file\_0000000012688211b4f57f5b8bcb8b42  
[L217] version\_id: 1  
[L218] name: SHIA\_RAG\_Final\_Report(4).md  
[L219] mime\_type: text/markdown  
[L220] surface: conversation  
[L221] score: 0.031024531024531024  
[L222] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--42  
[L223] snippet:  
[L224] **Novelty level:** Moderately Novel — the combined formulation for RAG hierarchy induction is new.  
[L225]  
[L226] ---  
[L227]  
[L228] ### 8.4 Novelty 4: Adaptive Depth Traversal via Query Complexity Classification  
[L229]  
[L230] **Existing approach:** All RAG systems retrieve a fixed number of chunks/nodes regardless of query complexity.  
[L231]  
[L232] **SHIA-RAG approach:** A query complexity classifier that dynamically adjusts traversal depth:  
[L233]  
[L234] | Query Type | Example | Traversal Strategy |  
[L235] |-----|-----|-----|  
[L236] | DEFINITION | "What is OSPF?" | Shallow: Target node + parent only (depth=1) |  
[L237] | COMPARISON | "Compare OSPF vs RIP" | Medium: Multiple subtrees, sibling links (depth=2) |  
[L238] | MULTI-HOP | "How does OSPF convergence affect reliability?" | Deep: Full path traversal + cross-links (depth=3+) |  
[L239] | SYNTHESIS | "Summarize all routing protocols" | Breadth: Root-level retrieval, expand selectively |  
[L240]  
[L241] **Novelty level:** Incremental — novel in the specific application to hierarchical RAG traversal depth.  
[L242]  
[L243] ---  
[L244]  
[L245] ### 8.5 Novelty 5: SHEF — SHIA-RAG Evaluation Framework  
[L246]

[L247] **Existing approach:** Standard RAG benchmarks (MS MARCO, NaturalQuestions) only measure flat retrieval. No benchmark tests hierarchical retrieval quality.

[L248]

[L249] **SHIA-RAG approach:** A novel evaluation framework with 3 metric categories:

[L250]

[L251] ##### A. Retrieval Metrics

[L252] | Metric | Formula | What It Measures |

[L253] |-----|-----|-----|

[L254] | Hop Accuracy | correct multi-hop answers / total multi-hop queries | Multi-hop reasoning quality |

[L255] | Ancestor Recall | retrieved ancestors  $\cap$  gold ancestors / gold ancestors | Correct concept chain retrieval |

[L256] | Context Coherence | Avg semantic similarity between consecutive context nodes | Logical coherence of assembled context |

[L257]

[L258] ##### B. Hierarchy Metrics

[L259] | Metric | Formula | What It Measures |

[L260] |-----|-----|-----|

[L261] | Parent Assignment Accuracy | correctly placed nodes / total nodes | Quality of hierarchy induction |

[L262] | Tree Edit Distance | Normalized TED vs gold taxonomy | Structural similarity to human taxonomy |

[L263] | Depth Distribution Score | KL-divergence from ideal depth distribution | Tree balance |

[L264]

[L265] ##### C. Forest Quality Metrics

[L266] | Metric | Formula | What It Measures |

[L267] |-----|-----|-----|

[L268] | Cross-Doc Fusion Accuracy | correctly merged nodes / total mergeable pairs | Entity resolution quality |

[L269] | Orphan Rate | root nodes without children / total root nodes | Isolated concept rate |

[L270] | Forest Density | edges / nodes | Knowledge forest connectivity |

[L271]

[L272] **Novelty level:** Incremental (Novel Combination) — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L273]

[L274] ### 8.6 Novelty Level Assessment Summary

[L275]

[L276] | # | Novelty | Level | Justification |

[L277] |---|-----|-----|-----|

[L278] | 1 | Tree Knapsack for Context Selection | **Incremental (Novel Application)** | Adaptation of existing OR technique to a new domain (RAG context selection) |

[L279] | 2 | Self-Evolving Knowledge Forest | **Moderately Novel** | Novel combination of retrieval feedback + hierarchy restructuring |

[L280] | 3 | Multi-Objective Energy Minimization | **Moderately Novel** | Combined formulation for RAG hierarchy induction is new |

[L281] | 4 | Adaptive Depth Traversal | **Incremental** | Novel application to hierarchical RAG traversal depth |

[L282] | 5 | SHEF Evaluation Framework | **Incremental (Novel Combination)** | Standard metrics adapted as a unified framework for hierarchical RAG |

[L283]

[L284] ---

[L285]

[L286] ## 9. Mathematical Framework

[L287]

[L288] ### 9.1 Hierarchy Energy (Global Objective)

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite{turn18file4}`

[L289] metadata:

[L290] query\_result\_index: 4

[L291] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L292] version\_id: 1

[L293] name: SHIA\_RAG\_Final\_Report(4).md

[L294] mime\_type: text/markdown

[L295] surface: conversation

[L296] score: 0.030330882352941176

[L297] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--5

[L298] snippet:

[L299] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.

[L300]

[L301] ### 2.3 Literature Search Methodology

[L302]

[L303] | Parameter | Details |

[L304] |-----|-----|

[L305] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |

[L306] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |

[L307] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |

[L308] | **Time Range** | 2020–2026 |

[L309] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |

[L310]

[L311] ---

[L312]

[L313] ## 3. System Architecture

[L314]

[L315] ### 3.1 Overview

[L316]

[L317] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L318]

[L319] | Phase | Layers | Mode | Purpose |

[L320] |-----|-----|-----|-----|

[L321] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L322] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L323] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L324]

[L325] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L326]

[L327]

```
[L328] graph TD
[L329] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L330] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L331] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L332] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L333] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L334] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L335] end
[L336]
[L337] subgraph "Phase 2: Online – Retrieval & Generation"
[L338] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L339] KF --> L6
[L340] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L341] L7 --> A["Answer + Citations"]
[L342] end
[L343]
[L344] subgraph "Phase 3: Continuous – Operations"
[L345] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L346] A --> L8
[L347] L8 -->|"Self-Evolution
Feedback"| KF
[L348] end
[L349]
```

[L350]

[L351] ### 3.2 Data Flow

[L352]

[L353] ```

[L354] Raw Document (PDF/DOCX/HTML)

[L355] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L356] Normalized Text + Metadata

[L357] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L358] [Page → Region → Block] hierarchy

[L359] ↓ Layer 3: Split sentences,

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite`turn18file5

[L360] metadata:

[L361] query\_result\_index: 5

[L362] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L363] version\_id: 1

[L364] name: SHIA\_RAG\_Final\_Report(4).md

[L365] mime\_type: text/markdown

[L366] surface: conversation

[L367] score: 0.030158730158730156

[L368] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--31

[L369] snippet:

[L370] ##### 5.5.5 CLD — Cross-Link Discoverer

[L371]

[L372] After hierarchical placement, discover semantic (non-hierarchical) links:

[L373]

[L374]

```
[L375] def cld_discover(node, forest, config):
[L376] """Find semantic cross-links for a newly placed node."""
[L377] cross_links = []
[L378] neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L379]
[L380] for neighbor in neighbors:
[L381] if neighbor.id == node.id or neighbor.id == node.parent:
[L382] continue
[L383]
[L384] sim = cosine_similarity(node.embedding, neighbor.embedding)
[L385] if sim > config.cross_link_threshold:
[L386] relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L387] if relation_type:
[L388] forest.add_edge(node.id, neighbor.id,
[L389] edge_class='SEMANTIC', relation_type=relation_type,
[L390] confidence=sim, weight=1.0)
[L391] cross_links.append((neighbor.id, relation_type, sim))
[L392]
[L393] return cross_links
[L394]
```

[L395]

[L396] ---

[L397]

[L398] ## 6. Online Pipeline — Layers 6-7

[L399]

[L400] ### 6.1 Layer 6: Retrieval Engine

[L401]

[L402] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L403]

[L404] **Input:** User query string → **Output:** RetrievalResult

[L405]

[L406] ##### Query Complexity Classification

[L407]

[L408] | Intent | Example | Complexity | Traversal Depth |

[L409] |-----|-----|-----|-----|

[L410] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |

[L411] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L412] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L413] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L414] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

[L415]

[L416] ##### Retrieval Pipeline

[L417]



```

[L418] ```python
[L419] @dataclass
[L420] class RetrievalResult:
[L421] """Output of Layer 6 — input to Layer 7."""
[L422] selected_nodes: list # Nodes selected by Tree Knapsack
[L423] context_text: str # Assembled context string
[L424] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L425] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L426] query_intent: str # DEFINITION, COMPARISON, etc.
[L427] seed_node_ids: list # Initial seed nodes from vector search
[L428] token_count: int # Estimated tokens in context
[L429]
[L430] class RetrievalEngine:
[L431] def __init__(self, forest, vector_index, cache_manager, config):
[L432] self.forest = forest
[L433] self.vector_index = vector_index
[L434] self.cache = cache_manager
[L435] self.config = config
[L436]
[L437] def retrieve(self, query) -> RetrievalResult:
[L438] # Step 0: Cache check
[L439] cached = self.cache.get_query_result(hash_query(query))
[L440] if cached:
[L441] return cached
[L442]
[L443] # Step 1: Classify query intent and complexity
[L444] intent = detect_intent(query)
[L445] complexity = classify_complexity(intent)
[L446]
[L447] # Step 2: Embed query and find seed nodes
[L448] query_emb = embed_query(query) # Asymmetric embedding model
[L449] seed_nodes = self.vector_index.search(query_emb, k=5)
[L450]
[L451] # Step 3: Adaptive depth traversal
[L452] depth_config = {
[L453] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L454] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L455] 'COMPLEX': {'max_depth': 3, 'parents': True,

```

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite[turn18file6]`

```

[L456] metadata:
[L457] query_result_index: 6
[L458] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L459] version_id: 1

```

```
[L460] name: SHIA_RAG_Final_Report(4).md
[L461] mime_type: text/markdown
[L462] surface: conversation
[L463] score: 0.029211087420042643
[L464] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--25
[L465] snippet:
[L466] Select the relation_type with highest weighted vote
[L467] 4. Log conflicts for manual review
[L468] ""
[L469] from collections import defaultdict
[L470]
[L471] # Group by node pair
[L472] pair_groups = defaultdict(list)
[L473] for edge in candidate_edges:
[L474] key = (edge.source_node, edge.target_node)
[L475] pair_groups[key].append(edge)
[L476]
[L477] resolved = []
[L478] for (src, tgt), edges in pair_groups.items():
[L479] if len(edges) == 1:
[L480] resolved.append(edges[0])
[L481] continue
[L482]
[L483] # Tally weighted votes per relation type
[L484] votes = defaultdict(float)
[L485] evidence_pool = defaultdict(list)
[L486] for edge in edges:
[L487] weight = edge.confidence * len(edge.evidence)
[L488] votes[edge.relation_type] += weight
[L489] evidence_pool[edge.relation_type].extend(edge.evidence)
[L490]
[L491] # Select winning relation type
[L492] winner = max(votes, key=votes.get)
[L493]
[L494] # Log conflict if close contest
[L495] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L496] if len(sorted_votes) > 1:
[L497] margin = sorted_votes[0][1] - sorted_votes[1][1]
[L498] if margin < 0.2: # Close contest
[L499] log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L500]
[L501] winning_edge = edges[0]
[L502] winning_edge.relation_type = winner
[L503] winning_edge.evidence = evidence_pool[winner]
[L504] winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L505] winning_edge.metadata['conflict_resolved'] = True
```

```
[L506] winning_edge.metadata['alternative_relations'] = [
[L507] (rt, score) for rt, score in sorted_votes if rt != winner
[L508]]
[L509] resolved.append(winning_edge)
[L510]
[L511] return resolved
[L512]
```

```
[L513]
[L514] ---
[L515]
[L516] ### 5.5 Layer 5: SHIA Core – Hierarchy Construction
[L517]
[L518] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage
[L519]
[L520] **Input:** `KnowledgeNode[]` + `KnowledgeEdge[]` → **Output:** Knowledge Forest
[L521]
[L522] ### Pipeline Overview
[L523]
[L524]
```

mermaid

```
[L525] graph LR
[L526] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L527] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L528] PRE --> HV["HV
Validate #1?"]
[L529] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L530] HV -->|"Fail"| HV2["HV
Validate #2?"]
[L531] HV2 -->|"Pass"| FI
[L532] HV2 -->|"Fail"| HV3["HV
Validate #3?"]
[L533] HV3 -->|"Pass"| FI
[L534] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L535] FI --> CLD["CLD
Discover Cross-Links"]
[L536] CLD --> DONE["Node Placed ✓"]
[L537]
```

```
[L538]
[L539] ### 5.5.1 CPG++ – Candidate Parent Generator
[L540]
[L541] Generates up to 5 candidate parents for a new node using a multi-stage filter:
[L542]
[L543]
```

python

```
[L544] def cpg_plus_plus(node_q, forest, config):
[L545] """Generate ≤k candidate parents for node q."""
[L546] # Stage 1: Domain Routing (O(1)) — filter to same-domain nodes
[L547] domain = classify_domain(node_q)
[L548] domain_nodes = forest.get_nodes_by_domain(domain)
[L549]
[L550] # Stage 2: ANN Retrieval (O(log N)) — find semantically similar nodes
```

```
[L551] if len(domain_nodes) > config.ann_k:
[L552] ann_results = vector_index.search(node_q.embedding, k=config.
```

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: [filecite]turn18file7[

```
[L553] metadata:
[L554] query_result_index: 7
[L555] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L556] version_id: 1
[L557] name: SHIA_RAG_Final_Report(4).md
[L558] mime_type: text/markdown
[L559] surface: conversation
[L560] score: 0.028985507246376812
[L561] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--32
[L562] snippet:
[L563]
```

```

[L564] @dataclass
[L565] class RetrievalResult:
[L566] """Output of Layer 6 – input to Layer 7."""
[L567] selected_nodes: list # Nodes selected by Tree Knapsack
[L568] context_text: str # Assembled context string
[L569] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L570] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L571] query_intent: str # DEFINITION, COMPARISON, etc.
[L572] seed_node_ids: list # Initial seed nodes from vector search
[L573] token_count: int # Estimated tokens in context
[L574]
[L575] class RetrievalEngine:
[L576] def __init__(self, forest, vector_index, cache_manager, config):
[L577] self.forest = forest
[L578] self.vector_index = vector_index
[L579] self.cache = cache_manager
[L580] self.config = config
[L581]
[L582] def retrieve(self, query) -> RetrievalResult:
[L583] # Step 0: Cache check
[L584] cached = self.cache.get_query_result(hash_query(query))
[L585] if cached:
[L586] return cached
[L587]
[L588] # Step 1: Classify query intent and complexity
[L589] intent = detect_intent(query)
[L590] complexity = classify_complexity(intent)
[L591]
[L592] # Step 2: Embed query and find seed nodes
[L593] query_emb = embed_query(query) # Asymmetric embedding model
[L594] seed_nodes = self.vector_index.search(query_emb, k=5)
[L595]
[L596] # Step 3: Adaptive depth traversal
[L597] depth_config = {
[L598] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L599] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L600] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L601] }
[L602] traversal = depth_config[complexity]
[L603]
[L604] # Step 4: BFS traversal from all seeds
[L605] candidate_nodes = set()
[L606] for seed in seed_nodes:
[L607] candidate_nodes |= self.traverse(seed, traversal)
[L608]
[L609] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L610] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L611]
[L612] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L613] context_text = self.assemble_context(selected, intent)
[L614]
[L615] return RetrievalResult(
[L616] selected_nodes=selected, context_text=context_text,
[L617] traversal_path_edges=self._get_path_edges(selected),
[L618] query_complexity=complexity, query_intent=intent,
[L619] seed_node_ids=[s.id for s in seed_nodes],
[L620] token_count=estimate_tokens(context_text)
[L621])
[L622]

```

[L623]

[L624] ##### Multi-Tree Knapsack Selection

[L625]

[L626] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L627]

[L628] 1. **Group** candidates by tree root

[L629] 2. **Allocate** token budget proportionally to each tree

[L630] 3. **Run** Tree Knapsack on each tree independently

[L631] 4. **Merge** results and trim to total budget

[L632]

[L633] ```python

[L634] def multi\_tree\_knapsack(self, candidate\_nodes, budget):

[L635] tree\_groups = {}

[L636] for node in candidate\_nodes:

[L637] root = self.forest.find\_root(node)

[L638] tree\_groups.setdefault(root.id, []).append(node)

[L639]

[L640] all\_selected = []

[L641] for root\_id, nodes in tree\_groups.items():

[L642] tree\_budget = int(budget \* len(nodes) / max(len(candidate\_nodes), 1))

[L643] \_, selected = tree\_knapsack(self.forest.get\_node(root\_id), tree\_budget, self.

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite[turn18file8]`

[L644] metadata:

[L645] query\_result\_index: 8

[L646] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L647] version\_id: 1

[L648] name: SHIA\_RAG\_Final\_Report(4).md

[L649] mime\_type: text/markdown

[L650] surface: conversation

[L651] score: 0.028782894736842105

[L652] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--2

[L653] snippet:

[L654] | Problem | Description | Impact |

[L655] |-----|-----|-----|

[L656] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L657] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L658] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L659] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,  $A \rightarrow B \rightarrow C$ ) |

[L660] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L661]

[L662] ### 1.2 Proposed Solution: SHIA-RAG

[L663]

[L664] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L665]

[L666]

|                         |                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------|
| [L667] Traditional RAG: | Document → [Chunk <sub>1</sub> , Chunk <sub>2</sub> , ..., Chunk <sub>n</sub> ] → Top-k → LLM |
| [L668] SHIA-RAG:        | Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM                 |
| [L669]                  |                                                                                               |

[L670]

[L671] The system:

[L672] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L673] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L674] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L675] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L676] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L677] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L678]

[L679] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L680]

[L681] ### 1.3 Objectives

[L682]

[L683] 1. Design a complete 8-layer architecture for hierarchical RAG

[L684] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L685] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L686] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L687] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L688] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L689]

[L690] ### 1.4 Contributions

[L691]

[L692] This work makes five novel contributions to the RAG literature:

[L693]

[L694] | # | Contribution | Novelty Level |

[L695] |---|-----|-----|

[L696] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L697] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L698] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L699] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L700] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L701]

[L702] ---

[L703]  
[L704] ## 2. Literature Review  
[L705]  
[L706] ### 2.1 Related Work  
[L707]  
[L708] #### 2.1.1 Flat-Chunk RAG Systems

---

DEEP RESEARCH 20/08/2026, 15:15:56

Citation Marker: `filecite[turn18file9]`

[L709] metadata:  
[L710] query\_result\_index: 9  
[L711] file\_id: file\_0000000012688211b4f57f5b8bcb8b42  
[L712] version\_id: 1  
[L713] name: SHIA\_RAG\_Final\_Report(4).md  
[L714] mime\_type: text/markdown  
[L715] surface: conversation  
[L716] score: 0.028665028665028666  
[L717] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--54  
[L718] snippet:  
[L719] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem  
[L720] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure  
[L721] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence  
[L722] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG  
[L723] 5. **SHF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality  
[L724]  
[L725] The complete system specification covers:  
[L726] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts  
[L727] - **12 defined functions** with complete implementations  
[L728] - **12 relation types** across 4 hierarchical and 8 semantic categories  
[L729] - **8 schemas** covering all data objects from document blocks to query logs  
[L730] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline  
[L731] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing  
[L732]  
[L733] ### 13.2 Key Novelties  
[L734]  
[L735] | # | Novelty | Level | Why It's New |  
[L736] |---|-----|-----|-----|  
[L737] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |  
[L738] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback



mechanism that modifies RAG knowledge structures |

[L739] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L740] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L741] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L742]

[L743] ### 13.3 Next Steps

[L744]

[L745] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L746] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L747] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L748] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L749] 5. **Build Layer 4** (Relation management with ontology validation)

[L750] 6. **Build Layer 5** (SHIA Core with algorithms)

[L751] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L752] 8. **Build Layer 7** (Generation with verification)

[L753] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L754] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L755] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L756] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L757]

[L758] ### 13.4 Future Work

---

DEEP RESEARCH 20/08/2026, 15:15:58

Citation Marker: `filecite[turn19file0]`

[L1] metadata:

[L2] ref\_id: turn18file6

[L3] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Final\_Report(4).md

[L6] lines\_shown: 516-935

[L7] total\_file\_lines: 2066

[L8] has\_more: True

[L9] next\_start\_line: 936

[L10] next\_read: {"ref\_id":"turn18file6","mode":"full","start\_line":936,"max\_lines":420,"include\_images":false}

[L11] content:

[L12] 2. Filter out words with confidence < 10 (noise)

[L13] 3. Compute weighted average by word length (longer words = more reliable signal)

[L14] 4. Normalize to [0.0, 1.0]

[L15] """

[L16] word\_data = ocr\_result.get('words', [])

[L17] if not word\_data:

[L18] return 0.0

```
[L19]
[L20] total_weight, weighted_sum = 0, 0
[L21] for word in word_data:
[L22] if word['confidence'] >= 10: # Filter noise
[L23] weight = len(word['text']) # Weight by word length
[L24] weighted_sum += word['confidence'] * weight
[L25] total_weight += weight
[L26]
[L27] return (weighted_sum / max(total_weight, 1)) / 100.0
[L28]
```

```
[L29]
[L30] ##### Text Cleaning (Safe)
[L31]
[L32]
```

*python*

```
[L33] def clean_text(text: str, pages: list) -> str:
[L34] """Clean extracted text while preserving legitimate content.
[L35]
[L36] Safeguards against false header/footer removal:
[L37] 1. Identify candidate headers/footers by frequency across pages
[L38] 2. Only remove lines that appear on 60%+ of pages (true headers/footers)
[L39] 3. Never remove lines that appear on fewer than 3 pages
[L40] 4. Normalize whitespace but preserve paragraph boundaries
[L41] """
[L42] if len(pages) >= 3:
[L43] # Frequency-based header/footer detection
[L44] first_lines = [p.split("\n")[0].strip() for p in pages if p.strip()]
[L45] last_lines = [p.split("\n")[-1].strip() for p in pages if p.strip()]
[L46]
[L47] from collections import Counter
[L48] header_counts = Counter(first_lines)
[L49] footer_counts = Counter(last_lines)
[L50]
[L51] threshold = max(3, int(len(pages) * 0.6)) # 60% of pages OR at least 3
[L52] headers_to_remove = {h for h, c in header_counts.items() if c >= threshold}
[L53] footers_to_remove = {f for f, c in footer_counts.items() if c >= threshold}
[L54]
[L55] lines = text.split("\n")
[L56] lines = [l for l in lines
[L57] if l.strip() not in headers_to_remove
[L58] and l.strip() not in footers_to_remove]
[L59] text = "\n".join(lines)
[L60]
[L61] # Normalize whitespace (preserve paragraph breaks)
[L62] import re
[L63] text = re.sub(r'[\t]+' + ' ', text) # Collapse horizontal space
```

```
[L64] text = re.sub(r'\n{3,}', '\n\n', text) # Max 2 consecutive newlines
[L65] return text.strip()
[L66]
```

```
[L67]
[L68] ##### Language Detection
[L69]
[L70]
```

```
python
[L71] from langdetect import detect
[L72]
[L73] def detect_language(text: str) -> str:
[L74] """Detect document language for OCR configuration.
[L75]
[L76] Used to configure Tesseract's language parameter.
[L77] Falls back to 'eng' if detection fails."""
[L78] try:
[L79] lang = detect(text[:5000]) # Sample first 5000 chars
[L80] # Map langdetect codes to Tesseract codes
[L81] lang_map = {'en': 'eng', 'hi': 'hin', 'de': 'deu', 'fr': 'fra', 'es': 'spa'}
[L82] return lang_map.get(lang, 'eng')
[L83] except Exception:
[L84] return 'eng'
[L85]
```

```
[L86]
[L87] ##### Partial Extraction Policy
[L88]
[L89] If some pages fail during extraction, the system uses a best-effort policy:
[L90]
[L91] | Condition | Action |
[L92] |-----|-----|
[L93] | ≥80% pages succeed | Accept document, flag `partial_extraction=True`, record `failed_pages` |
[L94] | <80% pages succeed | Reject entire document, queue for manual review |
[L95] | OCR confidence < 0.6 on any page | Flag page for manual review, still include in document |
[L96]
[L97]
```

```
python
[L98] def layer1_process(file_path: str, doc_registry: dict) -> NormalizedDocument:
[L99] """Layer 1: Input Processing pipeline with all safeguards."""
[L100] # Step 0: Duplicate check
[L101] content_hash = check_duplicate(file_path, doc_registry)
[L102]
[L103] fmt = detect_format(file_path)
[L104] validate_file(file_path, fmt) # Format check, size check (max 100MB, 5000 pages)
[L105]
[L106] failed_pages = []
[L107] if fmt == 'pdf':
[L108] text, pages, is_ocr, ocr_conf, failed_pages = extract_pdf_safe(file_path)
[L109] # Partial extraction policy
```

```

[L110] success_rate = (len(pages) - len(failed_pages)) / max(len(pages), 1)
[L111] if success_rate < 0.8:
[L112] raise ExtractionError(f"Only {success_rate:.0%} pages extracted; below 80% threshold")
[L113] elif fmt == 'docx':
[L114] text, pages = extract_docx(file_path), []
[L115] is_ocr, ocr_conf = False, 1.0
[L116] elif fmt == 'html':
[L117] text, pages = extract_html(file_path), []
[L118] is_ocr, ocr_conf = False, 1.0
[L119] else: # txt
[L120] text = open(file_path, 'r', encoding='utf-8').read()
[L121] pages, is_ocr, ocr_conf = [], False, 1.0
[L122]
[L123] language = detect_language(text)
[L124] text = clean_text(text, pages) # Safe cleaning with frequency-based header removal
[L125] metadata = extract_metadata(file_path, fmt)
[L126]
[L127] doc = NormalizedDocument(
[L128] doc_id=generate_doc_id(), original_filename=os.path.basename(file_path),
[L129] format=DocumentFormat(fmt), text=text, pages=pages,
[L130] metadata=metadata, is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L131] char_count=len(text), page_count=len(pages) or 1,
[L132] content_hash=content_hash, language=language,
[L133] partial_extraction=bool(failed_pages), failed_pages=failed_pages
[L134])
[L135] doc_registry[content_hash] = doc.doc_id # Register for dedup
[L136] return doc
[L137]

```

```

[L138]
[L139] ---
[L140]
[L141] ### 5.2 Layer 2: Document Parsing
[L142]
[L143] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L144]
[L145] **Input:** `NormalizedDocument` → **Output:** List of `DocumentBlock`
[L146]
[L147]

```

python

```

[L148] class BlockType(Enum):
[L149] HEADING = "heading"
[L150] PARAGRAPH = "paragraph"
[L151] LIST = "list"
[L152] TABLE = "table"
[L153] CODE = "code"
[L154] ALGORITHM = "algorithm"
[L155] FIGURE_CAPTION = "figure_caption"

```

```

[L156] EQUATION = "equation"
[L157] DEFINITION = "definition"
[L158]
[L159] @dataclass
[L160] class DocumentBlock:
[L161] """Atomic unit of document structure."""
[L162] block_id: str
[L163] doc_id: str
[L164] page_number: int
[L165] region_index: int # Which region on the page
[L166] block_type: BlockType
[L167] text: str
[L168] reading_order: int # Global ordering across the document
[L169] confidence: float = 1.0 # Classification confidence
[L170] table_cells: list = None # Parsed table cells (for TABLE blocks only)
[L171]

```

```

[L172]
[L173] The parser uses rule-based heuristics for block classification:
[L174] - **Heading:** Short line (< 100 chars), no ending punctuation, starts with uppercase
[L175] - **List:** Starts with ` - `, `* `, `• `, or `1.`
[L176] - **Table:** Contains 3+ pipe characters (`|`) – uses regex `re.match(r'^(def |class |import |from)'
[L177] - **Code:** Uses regex pattern matching (NOT `"def " in text`, which falsely matches "definitely")
[L178] - **Definition:** Contains phrases like "is defined as", "refers to"
[L179] - **Default:** Paragraph
[L180]
[L181] ##### Table Cell-Level Parsing
[L182]
[L183] When a block is classified as `TABLE`, the system uses `pdfplumber` or `Camelot` to reconstruct cell-
[L184]
[L185]

```

python

```

[L186] def parse_table_cells(page, table_region) -> list:
[L187] """Extract table structure with cell-level detail.
[L188]
[L189] Uses pdfplumber for text-based tables, Camelot for image-based.
[L190] Returns list of rows, each row is a list of cell strings."""
[L191] try:
[L192] # Primary: pdfplumber table extraction
[L193] table = page.extract_table(table_region)
[L194] if table and len(table) > 1: # At least header + 1 data row
[L195] return table
[L196] except Exception:
[L197] pass
[L198]
[L199] # Fallback: Camelot for image-based tables
[L200] try:
[L201] import camelot
[L202] tables = camelot.read_pdf(page.source_path, pages=str(page.number))
[L203] if tables:

```

```
[L204] return tables[0].df.values.tolist()
[L205] except Exception:
[L206] pass
[L207]
[L208] return None # Table structure could not be parsed
[L209]
```

```
[L210]
[L211] ##### Table Detection Fallback
[L212]
[L213] If table detection confidence is low, the block is flagged:
[L214]
[L215]
```

python

```
[L216] def classify_block_with_fallback(text, page) -> tuple:
[L217] """Returns (BlockType, confidence). Low-confidence tables get fallback."""
[L218] block_type, confidence = classify_block(text)
[L219]
[L220] if block_type == BlockType.TABLE and confidence < 0.6:
[L221] # Attempt cell-level parsing to confirm
[L222] cells = parse_table_cells(page, text)
[L223] if cells is None:
[L224] return BlockType.PARAGRAPH, 0.5 # Downgrade to paragraph
[L225] else:
[L226] return BlockType.TABLE, 0.8 # Confirmed by parser
[L227]
[L228] return block_type, confidence
[L229]
```

```
[L230]
[L231] ##### Reading Order Validation
[L232]
[L233] Reading order is assigned left-to-right, top-to-bottom within each page. A confidence score tracks re
[L234]
[L235]
```

python

```
[L236] def assign_reading_order(blocks: list) -> list:
[L237] """Assign global reading order with confidence.
[L238]
[L239] Multi-column layouts can produce incorrect reading order.
[L240] We detect potential issues and flag low confidence."""
[L241] # Sort by page, then y-coordinate, then x-coordinate
[L242] sorted_blocks = sorted(blocks, key=lambda b: (b.page_number, b.y_pos, b.x_pos))
[L243]
[L244] for i, block in enumerate(sorted_blocks):
[L245] block.reading_order = i
[L246]
[L247] # Detect potential multi-column issues
```

```
[L248] if i > 0:
[L249] prev = sorted_blocks[i - 1]
[L250] if (prev.page_number == block.page_number and
[L251] abs(prev.y_pos - block.y_pos) < 20 and # Same vertical band
[L252] prev.x_pos > block.x_pos): # But previous is to the right
[L253] block.confidence *= 0.7 # Reduce confidence — possible column issue
[L254]
[L255] return sorted_blocks
[L256]
```

```
[L257]
[L258] ---
[L259]
[L260] ### 5.3 Layer 3: Knowledge Extraction
[L261]
[L262] **Responsibility:** Extract concepts, definitions, and candidate relations from document blocks, then
[L263]
[L264] **Input:** `DocumentBlock[]` → **Output:** `KnowledgeNode[]` (canonical, deduplicated)
[L265]
[L266] **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities, c
[L267]
[L268] ##### Extraction Prompt
[L269]
[L270]
```

```
[L271] You are a knowledge extraction system. Given a text block, extract:
[L272] 1. Concepts: Named entities, technical terms
[L273] 2. Definitions: If the text defines any concept, extract the definition
[L274] 3. Properties: Key attributes with types and values
[L275] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L276] 5. Domain: Subject area (e.g., "Operating Systems")
[L277]
[L278] Respond in JSON format:
[L279] {
[L280] "concepts": [{"name": "...", "aliases": [...], "definitions": ["..."]},
[L281] "properties": {"key": {"type": "...", "value": "..."}},
[L282] "relations": [{"source": "...", "type": "IS_A|PART_OF|...", "target": "...",
[L283] "evidence": "..."}],
[L284] "domain": "..."}
[L285] }
[L286]
```

```
[L287]
[L288] ##### Definition Extraction (Dependency-Parse Based)
[L289]
[L290] Simple regex patterns like `^(.*) is (a|an|the)? (.*)` are too broad – they match non-definitions lik
[L291]
[L292]
```

python

```
[L293] def is_definition(text: str) -> tuple:
```

[L294] """Detect definitions using dependency parse + pattern matching.

[L295]

[L296] Returns (is\_def: bool, concept: str, definition: str).

[L297]

[L298] Strategy:

[L299] 1. Check for explicit definitional phrases (high precision)

[L300] 2. Use dependency parse to find copular definitions (medium precision)

[L301] 3. Fallback to LLM classification (expensive, high recall)

[L302] """

[L303] import re

[L304]

[L305] # Strategy 1: Explicit definitional phrases (high precision)

[L306] explicit\_patterns = [

[L307] r'^(.+)\s+is defined as\s+(.+)',

[L308] r'^(.+)\s+refers to\s+(.+)',

[L309] r'^(.+)\s+:\s+A-Z', # "Concept: A description..."

[L310] r'^(.+)\s+is a\s+(type|kind|form|category|class)\s+of\s+(.+)',

[L311] ]

[L312] for pattern in explicit\_patterns:

[L313] match = re.match(pattern, text, re.IGNORECASE)

[L314] if match:

[L315] return True, match.group(1).strip(), match.group(2).strip()

[L316]

[L317] # Strategy 2: Copular structure via dependency parse

[L318] # (requires spaCy or similar — use LLM as fallback if not available)

[L319]

[L320] # Strategy 3: LLM fallback (only for DEFINITION-classified blocks)

[L321] return False, None, None

[L322]

[L323]

[L324] ##### Alias Resolution (with Antonym Protection)

[L325]

[L326] After extraction, multiple `KnowledgeUnit` objects may refer to the same concept (e.g., "RR", "Round

[L327]

[L328] 1. Build a MinHash LSH index (threshold=0.7) from concept names + aliases

[L329] 2. \*\*Antonym check\*\*: Before merging, verify candidates do not have a CONTRASTS\_WITH relation

[L330] 3. Find connected components of near-duplicate units

[L331] 4. For each group: select canonical name (most frequent), merge all aliases, definitions, properties,

[L332]

[L333]

python

[L334] def alias\_resolution(units: list, relation\_cache: dict) -> list:

[L335] """Merge KnowledgeUnits referring to the same concept.

[L336]

[L337] Merge strategy:

[L338] - Canonical name: most frequent name across units

[L339] - Aliases: union of all names and aliases

[L340] - Definitions: union of all definitions



```

[L341] - Confidence: average of unit extraction confidences
[L342] - Properties: merged (latest value wins, conflicts logged)
[L343]
[L344] Antonym protection:
[L345] - Before merging, check if candidates have CONTRASTS_WITH relation
[L346] - Also check for negation prefixes (non-, un-, in-, dis-)
[L347] """
[L348] import re
[L349] NEGATION_PREFIXES = re.compile(r'^(non-?|un|in|dis)', re.IGNORECASE)
[L350]
[L351] lsh = MinHashLSH(threshold=0.7, num_perm=128)
[L352] # ... build LSH index from concept names + aliases ...
[L353]
[L354] raw_groups = find_connected_components(units, lsh)
[L355]
[L356] # Antonym protection: split groups containing antonym pairs
[L357] safe_groups = []
[L358] for group in raw_groups:
[L359] names = [u.name.lower() for u in group]
[L360] has_antonym = False
[L361] for i, n1 in enumerate(names):
[L362] for n2 in names[i+1:]:
[L363] # Check 1: Negation prefix
[L364] if NEGATION_PREFIXES.sub("", n1) == NEGATION_PREFIXES.sub("", n2):
[L365] has_antonym = True
[L366] # Check 2: Known CONTRASTS_WITH relation
[L367] if relation_cache.get((n1, n2)) == 'CONTRASTS_WITH':
[L368] has_antonym = True
[L369]
[L370] if has_antonym:
[

```

---

DEEP RESEARCH 20/08/2026, 15:16:01

Citation Marker: □filecite□turn20file0□

```

[L1] metadata:
[L2] ref_id: turn18file0
[L3] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L4] version_id: 1
[L5] name: SHIA_RAG_Final_Report(4).md
[L6] lines_shown: 930-1349
[L7] total_file_lines: 2066
[L8] has_more: True
[L9] next_start_line: 1350
[L10] next_read: {"ref_id":"turn18file0","mode":"full","start_line":1350,"max_lines":420,"include_images":false}
[L11] content:

```

[L12] When multiple documents provide **conflicting relations** for the same concept pair (e.g., Document A says "Round Robin IS\_A Scheduling" but Document B says "Round Robin PART\_OF OS"), the system resolves conflicts using evidence-weighted voting:

[L13]

[L14]

```
[L15] def resolve_cross_doc_conflicts(candidate_edges: list) -> KnowledgeEdge:
[L16] """Resolve conflicting relations from different documents.
[L17]
[L18] Strategy: Evidence-weighted voting
[L19] 1. Group edges by (source_node, target_node) pair
[L20] 2. For each group, tally votes per relation_type weighted by evidence confidence
[L21] 3. Select the relation_type with highest weighted vote
[L22] 4. Log conflicts for manual review
[L23] """
[L24] from collections import defaultdict
[L25]
[L26] # Group by node pair
[L27] pair_groups = defaultdict(list)
[L28] for edge in candidate_edges:
[L29] key = (edge.source_node, edge.target_node)
[L30] pair_groups[key].append(edge)
[L31]
[L32] resolved = []
[L33] for (src, tgt), edges in pair_groups.items():
[L34] if len(edges) == 1:
[L35] resolved.append(edges[0])
[L36] continue
[L37]
[L38] # Tally weighted votes per relation type
[L39] votes = defaultdict(float)
[L40] evidence_pool = defaultdict(list)
[L41] for edge in edges:
[L42] weight = edge.confidence * len(edge.evidence)
[L43] votes[edge.relation_type] += weight
[L44] evidence_pool[edge.relation_type].extend(edge.evidence)
[L45]
[L46] # Select winning relation type
[L47] winner = max(votes, key=votes.get)
[L48]
[L49] # Log conflict if close contest
[L50] sorted_votes = sorted(votes.items(), key=lambda x: x[1], reverse=True)
[L51] if len(sorted_votes) > 1:
[L52] margin = sorted_votes[0][1] - sorted_votes[1][1]
[L53] if margin < 0.2: # Close contest
[L54] log_conflict(src, tgt, sorted_votes) # Queue for manual review
[L55]
[L56] winning_edge = edges[0]
[L57] winning_edge.relation_type = winner
[L58] winning_edge.evidence = evidence_pool[winner]
[L59] winning_edge.confidence = calculate_edge_confidence_from_evidence(evidence_pool[winner])
[L60] winning_edge.metadata['conflict_resolved'] = True
[L61] winning_edge.metadata['alternative_relations'] = [
[L62] (rt, score) for rt, score in sorted_votes if rt != winner
[L63]]
[L64] resolved.append(winning_edge)
[L65]
[L66] return resolved
[L67]
```

[L68]

[L69] ---

[L70]

[L71] ### 5.5 Layer 5: SHIA Core — Hierarchy Construction

[L72]

[L73] **Responsibility:** Take extracted nodes and edges and construct the Knowledge Forest using a 5-stage pipeline.

[L74]

[L75] **Input:** KnowledgeNode[] + KnowledgeEdge[] → **Output:** Knowledge Forest

[L76]

[L77] #### Pipeline Overview

[L78]

[L79]

```
[L80] graph LR
[L81] N["New Node N"] --> CPG["CPG++
Generate ≤5
parent candidates"]
[L82] CPG --> PRE["PRE
Rank ALL candidates
(sorted list)"]
[L83] PRE --> HV{"HV
Validate #1?"}
[L84] HV -->|"Pass"| FI["FI
Insert into Forest"]
[L85] HV -->|"Fail"| HV2{"HV
Validate #2?"}
[L86] HV2 -->|"Pass"| FI
[L87] HV2 -->|"Fail"| HV3{"HV
Validate #3?"}
[L88] HV3 -->|"Pass"| FI
[L89] HV3 -->|"All Fail"| ROOT["FI
Insert as New Root"]
[L90] FI --> CLD["CLD
Discover Cross-Links"]
[L91] CLD --> DONE["Node Placed ✓"]
[L92]
```

[L93]

[L94] ##### 5.5.1 CPG++ — Candidate Parent Generator

[L95]

[L96] Generates up to 5 candidate parents for a new node using a multi-stage filter:

[L97]

[L98]

```

[L199] def cpg_plus_plus(node_q, forest, config):
[L100] """Generate ≤k candidate parents for node q."""
[L101] # Stage 1: Domain Routing (O(1)) – filter to same-domain nodes
[L102] domain = classify_domain(node_q)
[L103] domain_nodes = forest.get_nodes_by_domain(domain)
[L104]
[L105] # Stage 2: ANN Retrieval (O(log N)) – find semantically similar nodes
[L106] if len(domain_nodes) > config.ann_k:
[L107] ann_results = vector_index.search(node_q.embedding, k=config.ann_k)
[L108] candidates = [r for r in ann_results if r in domain_nodes]
[L109] else:
[L110] candidates = domain_nodes
[L111]
[L112] # Stage 3: Relation Filter – keep only nodes with plausible relations
[L113] candidates = [c for c in candidates if has_plausible_relation(node_q, c)]
[L114]
[L115] # Stage 4: Abstraction Filter – parent must be MORE abstract
[L116] # estimate_abstraction_level uses multi-factor heuristic + optional LLM fallback
[L117] q_level = estimate_abstraction_level(node_q)
[L118] candidates = [c for c in candidates
[L119] if c.abstraction_level > q_level
[L120] and abs(q_level - c.abstraction_level) < config.max_level_gap]
[L121]
[L122] # Stage 5: Evidence Filter – prioritize cross-document evidence
[L123] candidates = sorted(candidates,
[L124] key=lambda c: cross_doc_evidence_count(node_q, c), reverse=True)
[L125]
[L126] return candidates[:config.max_candidates] # Return top 5
[L127]

```

[L128]

[L129] **Overall complexity:**  $O(\log N)$  dominated by the ANN search (Stage 2). All other stages are linear filters over the ANN results.

[L130]

[L131] **Abstraction Level Estimation:**

[L132]

[L133]

```

[L134] def estimate_abstraction_level(node) -> float:
[L135] """Multi-factor heuristic for estimating abstraction level of unplaced nodes.
[L136] Returns a score in [0.0, 1.0] where higher = more abstract.
[L137]
[L138] NOTE: Simple name-length alone is insufficient (e.g., 'CPU' is short but
[L139] context-dependent). We combine multiple signals for robustness."""
[L140] score = 0.0
[L141]
[L142] # Factor 1: Name word count (shorter multi-word names tend to be more abstract)
[L143] word_count = len(node.name.split())
[L144] if word_count <= 2:
[L145] score += 0.15
[L146] elif word_count >= 4:
[L147] score -= 0.05
[L148]
[L149] # Factor 2: Number of source documents (more sources = more general)
[L150] score += min(len(node.sources) * 0.05, 0.20)
[L151]
[L152] # Factor 3: Number of sub-concept hints (more sub-concepts = more abstract)
[L153] score += min(len(node.children_hints) * 0.05, 0.25)
[L154]
[L155] # Factor 4: Definition keywords indicating abstraction level
[L156] abstract_keywords = ['general', 'category', 'type', 'class', 'family',
[L157] 'framework', 'paradigm', 'approach', 'method']
[L158] specific_keywords = ['specific', 'particular', 'instance', 'example',
[L159] 'implementation', 'variant', 'version']
[L160] defn_text = ' '.join(node.definitions).lower() if node.definitions else ''
[L161] score += sum(0.05 for kw in abstract_keywords if kw in defn_text)
[L162] score -= sum(0.05 for kw in specific_keywords if kw in defn_text)
[L163]
[L164] # Factor 5 (Optional): LLM-based hypernym detection
[L165] # If available and budget permits, use LLM to classify abstraction level
[L166] # This is expensive (~1 API call per node) so use only when factors 1-4
[L167] # produce ambiguous results (score in [0.3, 0.6] range)
[L168] if 0.3 <= score <= 0.6 and config.use_llm_abstraction:
[L169] llm_score = llm_classify_abstraction(node.name, node.definitions)
[L170] score = 0.5 * score + 0.5 * llm_score # Blend heuristic + LLM
[L171]
[L172] return max(0.0, min(score, 1.0)) # Clamp to [0, 1]
[L173]

```

[L174]

[L175] > **Note:** This heuristic requires empirical validation on domain-specific datasets. The LLM fallback (Factor 5) significantly improves accuracy but increases cost. Use sparingly.

[L176]

[L177] ##### 5.5.2 PRE — Parent Ranking Engine

[L178]

[L179] Ranks all candidates using a weighted scoring formula:

[L180]

[L181] 
$$\text{Score}(P, N) = w_1 \cdot \cos(E_P, E_N) + w_2 \cdot H(P, N) + w_3 \cdot \frac{D_{\max} - \text{depth}(P)}{D_{\max}} + w_4 \cdot S_{\text{evi}}(P, N) + w_5 \cdot \text{Conf}(P)$$

[L182]

[L183] Where:

[L184] -  $w_1=0.30$  — Embedding cosine similarity

[L185] -  $w_2=0.25$  — Hypernym score (Hearst patterns + LLM fallback)

[L186] -  $w_3=0.15$  — Depth penalty (shallower parents preferred;  $D_{\max}=8$ )

[L187] -  $w_4=0.20$  — Evidence support

[L188] -  $w_5=0.10$  — Parent confidence

[L189]

[L190]

```

[L191] def pre_rank_all(node_q, candidates, config):
[L192] """Rank ALL candidates, return sorted list."""
[L193] scored = []
[L194] for p in candidates:
[L195] sim = cosine_similarity(node_q.embedding, p.embedding)
[L196] hyp = hypernym_score(node_q, p)
[L197] depth_score = (config.d_max - p.depth) / config.d_max
[L198] evi = evidence_support(node_q, p)
[L199] conf = p.confidence
[L200]
[L201] score = config.w1*sim + config.w2*hyp + config.w3*depth_score + config.w4*evi + config.w5*cor
[L202] scored.append(CandidateResult(parent=p, pre_score=score,
[L203] components={'sim': sim, 'hyp': hyp, 'depth': depth_score, 'evi': evi, 'conf': conf}))
[L204]
[L205] scored.sort(key=lambda x: x.pre_score, reverse=True)
[L206]
[L207] if scored and scored[0].pre_score < config.new_tree_threshold:
[L208] return [] # No acceptable candidate → create new root
[L209] return scored
[L210]

```

[L211]

[L212] ##### 5.5.3 HV — Hierarchy Validator

[L213]

[L214] Validates that adding a parent → child edge maintains forest invariants:

[L215]

[L216] 1. **Self-loop check** — parent ≠ child[L217] 2. **Cycle detection** — traverse UP from parent looking for child; if found, reject (would create cycle)[L218] 3. **Depth limit** — `parent.depth + 1 < config.max_depth`[L219] 4. **Sibling coherence** — soft penalty (not a hard gate); dissimilar siblings are allowed but penalized

[L220]

[L221] ##### 5.5.4 FI — Forest Inserter

[L222]

[L223] Inserts a node into the forest by trying HV-validated candidates in ranked order:

[L224]

[L225]

```

[L226] def fi_insert(node, ranked_candidates, forest, config):
[L227] """Insert node into forest. Try candidates in order; fall back to new root."""
[L228] for candidate in ranked_candidates:
[L229] if hv_validate(candidate.parent, node, forest, config):
[L230] forest.add_edge(candidate.parent.id, node.id, edge_class='HIERARCHICAL')
[L231] node.parent = candidate.parent.id
[L232] node.depth = candidate.parent.depth + 1
[L233] propagate_confidence(forest) # Re-propagate confidence scores
[L234] return InsertResult(status='PLACED', parent=candidate.parent.id)
[L235]
[L236] # All candidates failed validation — create new root
[L237] forest.add_root(node)
[L238] return InsertResult(status='NEW_ROOT', reason='NO_VALID_CANDIDATE')
[L239]

```

[L240]

[L241] ##### 5.5.5 CLD — Cross-Link Discoverer

[L242]

[L243] After hierarchical placement, discover semantic (non-hierarchical) links:

[L244]

[L245]

```
[L246] def cld_discover(node, forest, config):
[L247] """Find semantic cross-links for a newly placed node."""
[L248] cross_links = []
[L249] neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L250]
[L251] for neighbor in neighbors:
[L252] if neighbor.id == node.id or neighbor.id == node.parent:
[L253] continue
[L254]
[L255] sim = cosine_similarity(node.embedding, neighbor.embedding)
[L256] if sim > config.cross_link_threshold:
[L257] relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L258] if relation_type:
[L259] forest.add_edge(node.id, neighbor.id,
[L260] edge_class='SEMANTIC', relation_type=relation_type,
[L261] confidence=sim, weight=1.0)
[L262] cross_links.append((neighbor.id, relation_type, sim))
[L263]
[L264] return cross_links
[L265]
```

[L266]

[L267] ---

[L268]

[L269] ## 6. Online Pipeline — Layers 6-7

[L270]

[L271] ### 6.1 Layer 6: Retrieval Engine

[L272]

[L273] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.

[L274]

[L275] **Input:** User query string → **Output:** RetrievalResult

[L276]

[L277] ##### Query Complexity Classification

[L278]

[L279] | Intent | Example | Complexity | Traversal Depth |

[L280] |-----|-----|-----|-----|

[L281] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |

[L282] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L283] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L284] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L285] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

[L286]

[L287] ##### Retrieval Pipeline

[L288]

[L289]

```

[L290] @dataclass
[L291] class RetrievalResult:
[L292] """Output of Layer 6 – input to Layer 7."""
[L293] selected_nodes: list # Nodes selected by Tree Knapsack
[L294] context_text: str # Assembled context string
[L295] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L296] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L297] query_intent: str # DEFINITION, COMPARISON, etc.
[L298] seed_node_ids: list # Initial seed nodes from vector search
[L299] token_count: int # Estimated tokens in context
[L300]
[L301] class RetrievalEngine:
[L302] def __init__(self, forest, vector_index, cache_manager, config):
[L303] self.forest = forest
[L304] self.vector_index = vector_index
[L305] self.cache = cache_manager
[L306] self.config = config
[L307]
[L308] def retrieve(self, query) -> RetrievalResult:
[L309] # Step 0: Cache check
[L310] cached = self.cache.get_query_result(hash_query(query))
[L311] if cached:
[L312] return cached
[L313]
[L314] # Step 1: Classify query intent and complexity
[L315] intent = detect_intent(query)
[L316] complexity = classify_complexity(intent)
[L317]
[L318] # Step 2: Embed query and find seed nodes
[L319] query_emb = embed_query(query) # Asymmetric embedding model
[L320] seed_nodes = self.vector_index.search(query_emb, k=5)
[L321]
[L322] # Step 3: Adaptive depth traversal
[L323] depth_config = {
[L324] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L325] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L326] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L327] }
[L328] traversal = depth_config[complexity]
[L329]
[L330] # Step 4: BFS traversal from all seeds
[L331] candidate_nodes = set()
[L332] for seed in seed_nodes:
[L333] candidate_nodes |= self.traverse(seed, traversal)
[L334]
[L335] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L336] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L337]
[L338] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L339] context_text = self.assemble_context(selected, intent)
[L340]
[L341] return RetrievalResult(
[L342] selected_nodes=selected, context_text=context_text,
[L343] traversal_path_edges=self._get_path_edges(selected),
[L344] query_complexity=complexity, query_intent=intent,
[L345] seed_node_ids=[s.id for s in seed_nodes],
[L346] token_count=estimate_tokens(context_text)
[L347])
[L348]

```

[L349]



[L350] ##### Multi-Tree Knapsack Selection

[L351]

[L352] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L353]

[L354] 1.

CHATGPT 20/08/2026, 15:16:01

Layer 5 — SHIA Core: Hierarchy Construction

Ab hum **same large Smart Hospital example** continue karenge jo Layer 0, 1, 2, 3 aur 4 me use kiya tha.

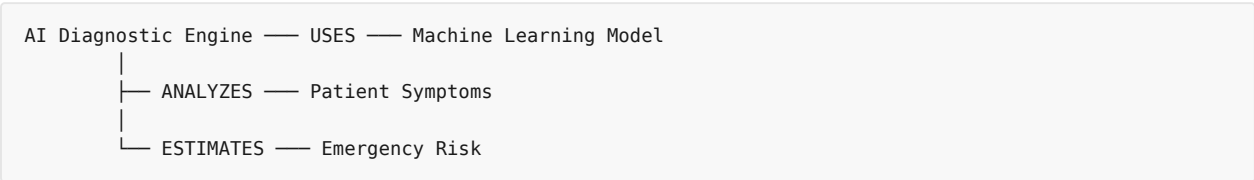
Layer 5 is project ka **core layer** hai, kyunki pehli baar yahan individual nodes aur edges ko ek meaningful **Knowledge Forest** me organize kiya jata hai.

- Layer 3: Concepts nikale
- Layer 4: Relations validate kiye
- Layer 5: Decide karta hai — *kaunsa concept kis concept ke neeche hierarchy me jayega.*

Report ke according Layer 5 ka responsibility hai extracted nodes aur edges se **5-stage pipeline ke through Knowledge Forest construct karna**. Input `KnowledgeNode[] + KnowledgeEdge[]` hai aur output `Knowledge Forest` hai.

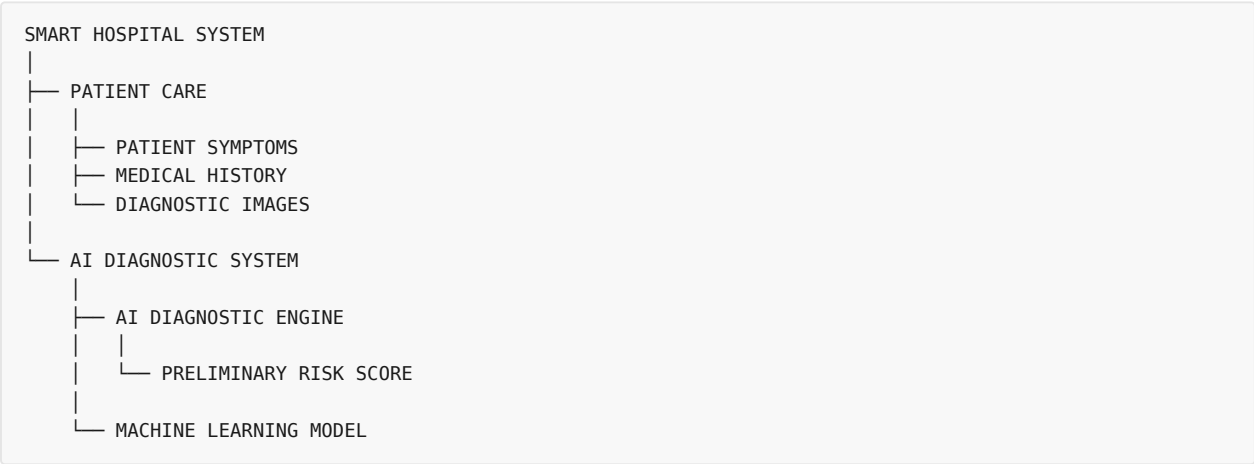
1. Sabse pehle: Knowledge Forest kya hota hai?

Normal graph kuch aisa ho sakta hai:

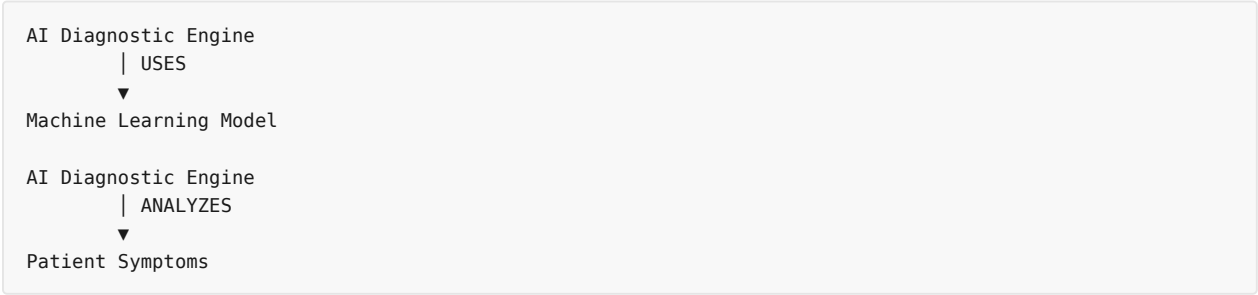


Isme koi clear hierarchy nahi hai.

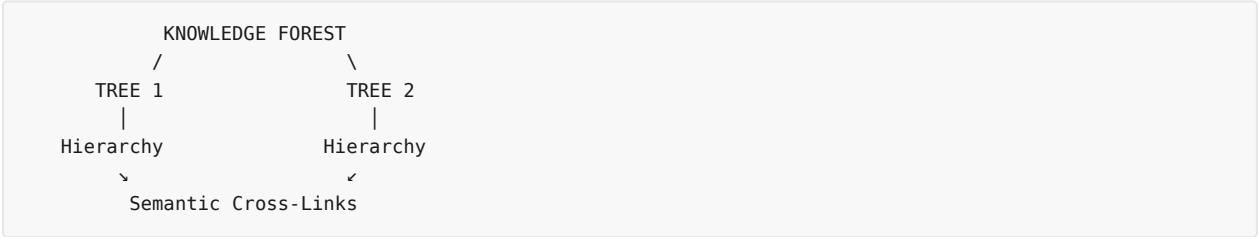
Layer 5 ke baad system try karta hai knowledge ko aise organize karne ka:



Aur hierarchy ke alawa semantic cross-links:



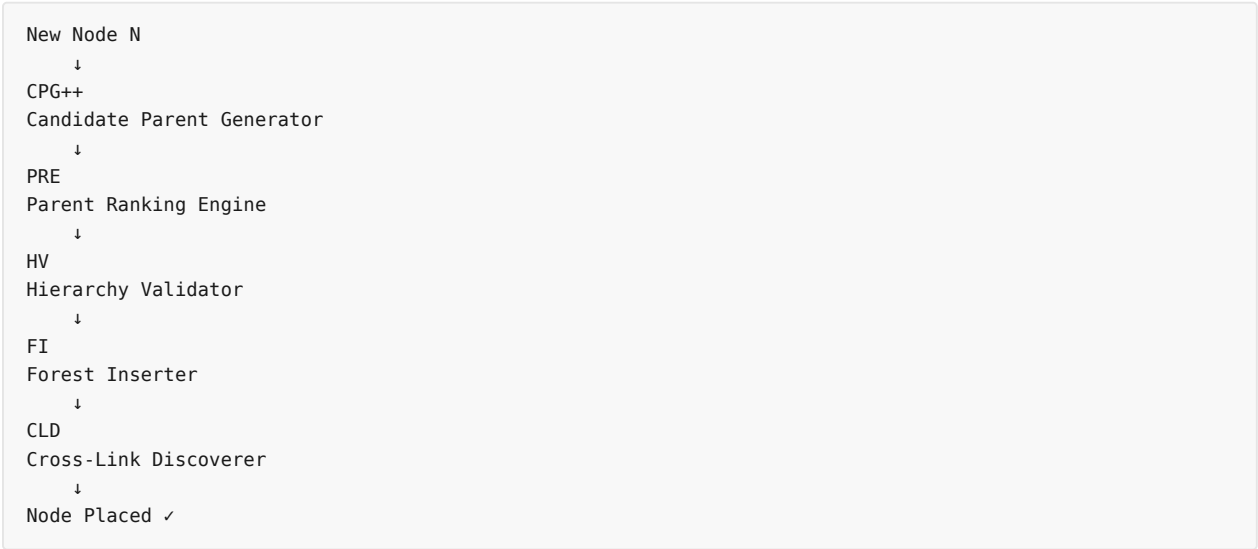
So final structure:



SHIA-RAG flat chunks ki jagah concepts aur relationships ko rooted trees aur cross-links wale **Knowledge Forest** me organize karne ka approach propose karta hai. [filecite turn18file1 L94-L104](#)

## 2. Layer 5 ka exact pipeline

Report me exact 5 stages hain:



Agar highest-ranked parent validation fail kare:

```
Candidate #1
↓
HV fail
↓
Candidate #2
↓
HV fail
↓
Candidate #3
↓
...
↓
All candidates fail
↓
Insert as NEW ROOT
```

Ye Layer 5 ka exact high-level control flow report me diya gaya hai. `filecite\turn20file0\L77-L91`

### 3. Hamare example ka Layer 5 input

Layer 4 ke baad hamare paas canonical nodes aur validated edges hain.

#### **KnowledgeNode[]**

```
NODE_001 → Smart Hospital System
NODE_002 → Patient Care
NODE_003 → AI Diagnostic System
NODE_004 → AI Diagnostic Engine
NODE_005 → Machine Learning Model
NODE_006 → Patient Symptoms
NODE_007 → Medical History
NODE_008 → Diagnostic Images
NODE_009 → Emergency Risk
NODE_010 → Preliminary Risk Score
```

Aur har node ke paas generally information hai:

```
KnowledgeNode
|
├─ node_id
├─ canonical_name
├─ aliases
├─ definitions
├─ domain
├─ evidence
├─ confidence
├─ embedding
├─ parent
├─ children
└─ ...
```

#### **KnowledgeEdge[]**

Layer 4 se validated edges aaye:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

AI Diagnostic Engine  
ANALYZES  
Patient Symptoms

AI Diagnostic Engine  
ANALYZES  
Medical History

AI Diagnostic Engine  
ANALYZES  
Diagnostic Images

AI Diagnostic Engine  
ESTIMATES  
Emergency Risk

Important:

*Layer 4 ka har edge automatically hierarchy edge nahi banega.*

Example:

USES  
ANALYZES  
ESTIMATES

semantic relations hain.

Layer 5 ka main difficult kaam hai:

*Existing concepts ko parent-child hierarchy me intelligently place karna.*

## 4. Layer 5 ka final output

Final output:

### Knowledge Forest

Conceptually:

```
KnowledgeForest
|
├─ Roots
| ├── ROOT_1
| ├── ROOT_2
| └── ROOT_3
|
├─ Hierarchical Edges
| ├── parent → child
| └── parent → child
|
└─ Semantic Cross-Links
 ├── USES
 ├── CAUSES
 └── ...
```

Report ke data flow me Layer 5 ka output explicitly:

Knowledge Forest  
=  
rooted trees + cross-links

hai. `filecite`turn18file0L43-L50

## 5. Layer 5 ko deeply samajhne ka main question

Suppose naya node hai:

AI Diagnostic Engine

Question:

*Iska parent kaun hona chahiye?*

Possible candidates:

1. Smart Hospital System
2. Patient Care
3. AI Diagnostic System
4. Machine Learning Model
5. Emergency Risk

Obviously:

AI Diagnostic System

sabse logical parent lagta hai.

Lekin system **sirf manually guess nahi karta**.

Iske liye poori pipeline chalti hai:

```
CPG++
→ possible parents generate

PRE
→ unko score karke rank

HV
→ check ki hierarchy valid hai

FI
→ actual forest me insert

CLD
→ additional semantic links discover
```

Ab hum **ek-ek stage ko complete depth me** samjhenge.

## PART A — CPG++

## 6. CPG++ kya karta hai?

CPG++ =

Candidate Parent Generator

Iska kaam:

New node ke liye maximum 5 possible parent candidates generate karna.

Report ke according CPG++ multi-stage filtering use karta hai aur final me maximum 5 candidates return karta hai.

fileciteurn20file0L94-L129

7. Example: New node = AI Diagnostic Engine

Input:

node\_q = AI Diagnostic Engine

Forest me already nodes:

Smart Hospital System  
Patient Care  
AI Diagnostic System  
Machine Learning Model  
Emergency Risk  
Medical History  
Patient Symptoms

Ab CPG++ 5 stages chalata hai.

Stage 1 — Domain Routing

Report:

Domain classify karo  
↓  
Same-domain nodes lo

Complexity:

O(1)

fileciteurn20file0L101-L104

Hamara example

New node:

AI Diagnostic Engine

Domain classify:

Healthcare AI

Forest me same domain ke nodes:

```

Healthcare AI
|
├─ Smart Hospital System
├─ Patient Care
├─ AI Diagnostic System
├─ Machine Learning Model
├─ Emergency Risk
└─ Patient Symptoms

```

Suppose kisi dusre domain me node hai:

```
CPU Scheduling
```

Then:

```
CPU Scheduling ❌
```

initially filter out ho jayega.

**Why?**

Kyuki:

```
AI Diagnostic Engine
```

ka parent:

```
Round Robin Scheduling
```

hone ka chance extremely low hai.

So:

```

All Forest Nodes
 ↓
Domain Filter
 ↓
Relevant Domain Nodes

```

## 8. Stage 1 me possible error

### Domain classification wrong

Suppose:

```
AI Diagnostic Engine
```

galti se classify hua:

```
Medical Devices
```

instead of:

```
Healthcare AI
```

Then correct parent:

### AI Diagnostic System

candidate pool me aa hi nahi sakta.

This is a serious error propagation problem:

```
Wrong Domain
 ↓
Wrong Candidate Pool
 ↓
Correct Parent Removed
 ↓
PRE can never select it
```

### Important

CPG++ me downstream algorithm kitna bhi intelligent ho, agar correct parent pehle hi filter out ho gaya:

*PRE usko recover nahi kar sakta.*

## 9. Stage 2 — ANN Retrieval

ANN = Approximate Nearest Neighbor.

Ab same-domain nodes bahut zyada ho sakte hain.

Example:

```
Healthcare AI domain
=
50,000 nodes
```

Har node ko compare karna expensive hoga.

Isliye report ke according vector index se semantic similarity ke basis par nearest candidates retrieve kiye jate hain.

```
AI Diagnostic Engine embedding
 ↓
Vector Index
 ↓
Nearest Nodes
```

Complexity:

$O(\log N)$

overall CPG++ complexity ka dominant part ANN search hai. [filecite.it/turn20file0/L105-L129](https://filecite.it/turn20file0/L105-L129)

### Example

ANN search result:

|                              |                   |
|------------------------------|-------------------|
| 1. AI Diagnostic System      | similarity = 0.91 |
| 2. Medical Decision System   | similarity = 0.84 |
| 3. Machine Learning Model    | similarity = 0.81 |
| 4. Patient Monitoring System | similarity = 0.76 |
| 5. Emergency Risk            | similarity = 0.72 |

Important:



*ANN result ka matlab ye nahi ki sab parent ban sakte hain.*

Abhi ye sirf **semantically nearby nodes** hain.

## 10. ANN me possible mistake

Similarity:

```
AI Diagnostic Engine
↔
Machine Learning Model
```

high ho sakti hai.

But:

```
Machine Learning Model
```

necessarily parent nahi hai.

Actually possible structure:

```
AI Diagnostic System
├── AI Diagnostic Engine
└── Machine Learning Model
```

So ANN:

```
semantic similarity
```

find karta hai.

But hierarchy:

```
parent-child relation
```

determine nahi karta.

Isliye next filters required hain.

## 11. Stage 3 — Relation Filter

Report:

*Sirf un candidates ko rakho jinke saath plausible relation ho. [filecite turn20file0 L 112-L 114](#)*

Example candidates:

```
AI Diagnostic System
Machine Learning Model
Emergency Risk
Patient Symptoms
```

Question:

Can "AI Diagnostic System"  
plausibly be parent of  
"AI Diagnostic Engine"?

YES ✓

Question:

Can "Emergency Risk"  
plausibly be parent of  
"AI Diagnostic Engine"?

Probably:

NO ✗

Because:

AI Diagnostic Engine  
ESTIMATES  
Emergency Risk

This is semantic relation, not parent-child hierarchy.

So:

Candidate Pool  
↓  
Plausible Relation Filter  
↓  
AI Diagnostic System ✓  
Machine Learning Model ?  
Emergency Risk ✗  
Patient Symptoms ✗

## 12. Relation filter ka important limitation

Report function:

```
has_plausible_relation(node_q, c)
```

call karta hai.

Lekin shown section me exact:

- relation plausibility model
- threshold
- LLM prompt
- rule set

fully defined nahi hai.

So:

*Report ye specify karti hai ki plausible-relation filter exist karta hai, lekin us function ka exact internal implementation shown section me specified nahi hai.*

Ye bahut important missing implementation detail hai.

---

### 13. Stage 4 — Abstraction Filter

Ye Layer 5 ka **bahut important concept** hai.

Parent should be:

#### More abstract than child

Report:

```
parent.abstraction_level > child.abstraction_level
```

Aur level gap bhi maximum allowed limit ke andar hona chahiye. `0 ≤ L115-L120`

---

#### Hamara example

```
AI Diagnostic System
```

More abstract hai.

```
AI Diagnostic Engine
```

more specific hai.

So:

```
AI Diagnostic System
 ↓
AI Diagnostic Engine
```

valid abstraction direction.

---

#### Invalid case

```
AI Diagnostic Engine
 ↓
Machine Learning Model
```

Question:

Kya ML Model AI Diagnostic Engine ka parent hai?

Potentially wrong, kyunki ye dono:

```
system component / method
```

relationship me ho sakte hain, hierarchy me nahi.

---

### 14. Abstraction level kaise estimate hota hai?

Report ek multi-factor heuristic define karti hai.

Score:

0.0 → more specific  
1.0 → more abstract

5 factors use hote hain. `filecite\turn20file0\L131-L175`

---

### Factor 1 — Name word count

Short names ko kuch abstraction score milta hai.

Example:

System

more abstract ho sakta hai than:

AI Emergency Diagnostic Prediction Engine

Code:

$\leq 2$  words  
→ +0.15  
  
 $\geq 4$  words  
→ -0.05

---

### Problem

Name length reliable nahi hai.

Example:

CPU

short hai.

But:

CPU

har context me necessarily very abstract parent nahi hai.

Report bhi note karti hai ki simple name-length alone insufficient hai. `filecite\turn20file0\L131-L175`

---

### Factor 2 — Number of source documents

Report assumption:

More source documents  
→ concept more general

Contribution:

`min(number_of_sources × 0.05, 0.20)`

Example:

```
AI Diagnostic System
appears in 10 documents
```

Then high contribution.

But:

```
Preliminary Risk Score
appears in 1 document
```

then lower contribution.

---

### Factor 3 — Sub-concept hints

More child/sub-concept hints:

```
→ concept probably more abstract
```

Maximum contribution:

```
0.25
```

Example:

```
AI Diagnostic System
```

has hints:

```
AI Diagnostic Engine
Image Analysis Module
Risk Prediction Module
```

Then abstraction score increase ho sakta hai.

---

### Factor 4 — Definition keywords

Abstract keywords:

```
general
category
type
class
family
framework
paradigm
approach
method
```

Specific keywords:

specific  
particular  
instance  
example  
implementation  
variant  
version

Definition me abstract terms:

score ↑

Specific terms:

score ↓

Factor 5 — Optional LLM fallback

Agar heuristic ambiguous result de:

$0.3 \leq \text{score} \leq 0.6$

then optionally LLM abstraction classify kar sakta hai.

Final:

50% heuristic  
+  
50% LLM score

Report clearly kehti hai ki LLM fallback accuracy improve kar sakta hai but cost badhata hai, aur heuristic ko domain-specific empirical validation ki zarurat hai. [filecite turn20file0 L164-L175](#)

15. Abstraction calculation example

Suppose:

AI Diagnostic System

Score roughly:

|                     |       |
|---------------------|-------|
| Name factor         | +0.15 |
| Source documents    | +0.20 |
| Children hints      | +0.20 |
| Definition keywords | +0.10 |
| -----               |       |
| Raw Score           | 0.65  |

Suppose:

AI Diagnostic Engine

Score:

|                        |       |
|------------------------|-------|
| Name factor            | -0.05 |
| Source documents       | +0.10 |
| Children hints         | +0.05 |
| Specificity indicators | -0.05 |
| -----                  |       |
| Raw Score              | 0.05  |

Therefore:

```
AI Diagnostic System
Abstraction = 0.65

AI Diagnostic Engine
Abstraction = 0.05
```

Thus:

$0.65 > 0.05$

Parent direction plausible:

```
AI Diagnostic System
 |
 ▼
AI Diagnostic Engine
```

## 16. CPG++ ka final output

After all 5 filters:

Candidate Parents

Example:

```
[
 AI Diagnostic System,
 Smart Hospital System,
 Medical Decision System
]
```

Maximum:

$\leq 5$  candidates

Now ye PRE ko jayenge.

## PART B — PRE

### 17. PRE kya hai?

PRE =

### Parent Ranking Engine

CPG++ bolta hai:

"Ye possible parents hain."

PRE bolta hai:

"Inme se best parent kaun hai?"

Report ke according PRE ALL candidates ko rank karta hai aur sorted list return karta hai.

fileciteurn20file0L177-L210

### 18. PRE ka exact scoring formula

Report formula:

$$\begin{aligned} \text{Score}(P,N)= & w\_1 \cdot \text{Similarity} \\ & + \\ & w\_2 \cdot \text{Hypernym} \\ & + \\ & w\_3 \cdot \text{DepthScore} \\ & + \\ & w\_4 \cdot \text{EvidenceSupport} \\ & + \\ & w\_5 \cdot \text{ParentConfidence} \end{aligned}$$

Weights:

| Factor               | Weight |
|----------------------|--------|
| Embedding similarity | 0.30   |
| Hypernym score       | 0.25   |
| Depth preference     | 0.15   |
| Evidence support     | 0.20   |
| Parent confidence    | 0.10   |

Total:

1.00

Report me ye exact weights aur formula defined hain. fileciteurn20file0L177-L209

### 19. Har PRE component deeply samjho

#### A. Semantic Similarity — weight 0.30

```
cosine_similarity(
 child.embedding,
 parent.embedding
)
```

Example:



AI Diagnostic Engine  
↔ AI Diagnostic System  
  
similarity = 0.92

Contribution:

$0.30 \times 0.92$   
=  
0.276

## B. Hypernym Score — weight 0.25

Hypernym basically:

*Parent concept zyada general hai kya?*

Example:

AI Diagnostic System

is a more general category/system than:

AI Diagnostic Engine

Suppose:

Hypernym Score = 0.90

Contribution:

$0.25 \times 0.90$   
=  
0.225

Report hypernym score ke liye:

Hearst patterns + LLM fallback

mention karti hai. `filecite\turn20file0\L181-L188`

## C. Depth Score — weight 0.15

Formula:

$(D_{\max} - \text{parent.depth}) / D_{\max}$

Report:

$D_{\max} = 8$

Shallower parents preferred.

Example:

Parent depth:

2

Then:

$$(8 - 2) / 8 \\ = \\ 0.75$$

Contribution:

$$0.15 \times 0.75 \\ = \\ 0.1125$$

**Why?**

System unnecessarily deep hierarchy avoid karna chahta hai.

---

### D. Evidence Support — weight 0.20

Question:

*Kya parent-child relation ke liye actual evidence support available hai?*

Suppose:

AI Diagnostic Engine

and:

AI Diagnostic System

multiple documents me related appear hote hain.

Then evidence score higher ho sakta hai.

---

### E. Parent Confidence — weight 0.10

Agar candidate parent itself reliable node hai:

$$\text{confidence} = 0.95$$

Then:

$$0.10 \times 0.95 \\ = \\ 0.095$$

---

## 20. Complete PRE calculation

Candidates:

P1 = AI Diagnostic System  
P2 = Smart Hospital System  
P3 = Medical Decision System

New node:

N = AI Diagnostic Engine

Suppose values:

| Factor | P1 | P2 | P3 |  
|---|---|---|---|  
| Similarity | 0.92 | 0.70 | 0.78 |  
| Hypernym | 0.90 | 0.60 | 0.75 |  
| Depth score | 0.75 | 1.00 | 0.875 |  
| Evidence | 0.90 | 0.65 | 0.60 |  
| Confidence | 0.95 | 0.95 | 0.80 |

P1

0.30(0.92)  
+  
0.25(0.90)  
+  
0.15(0.75)  
+  
0.20(0.90)  
+  
0.10(0.95)

=  
0.276  
+ 0.225  
+ 0.1125  
+ 0.18  
+ 0.095

= 0.8885

P2

=  
0.30(0.70)  
+ 0.25(0.60)  
+ 0.15(1.0)  
+ 0.20(0.65)  
+ 0.10(0.95)

= 0.735

P3

≈ 0.746

Final ranking:

- 1. AI Diagnostic System → 0.8885
- 2. Medical Decision System → 0.746
- 3. Smart Hospital System → 0.735

So PRE output:

SORTED CANDIDATE LIST

```
#1 AI Diagnostic System
#2 Medical Decision System
#3 Smart Hospital System
```

## 21. New Tree Threshold

Report me important condition hai:

```
if best_score < new_tree_threshold:
 return []
```

Meaning:

*Agar best candidate bhi sufficiently good nahi hai, node ko zabardasti kisi parent ke neeche mat daalo.*

Instead:

NEW TREE

create hoga. [filecite\turn20file0\L205-L209](#)

**Ye bahut important design decision hai.**

Wrong:

Every node MUST have a parent

Correct SHIA design:

```
No good parent
 ↓
New Root
```

## PART C — HV

### 22. HV kya hai?

HV =

### Hierarchy Validator

PRE bolta hai:

Candidate #1 best hai

Lekin score high hone ka matlab automatically hierarchy valid nahi.

HV check karta hai:

*Agar parent → child edge add kiya, kya forest ke rules break honge?*

Report ke according HV 4 main checks use karta hai. [filecite\turn20file0\L212-L220](#)

## 23. HV Check 1 — Self Loop

Check:

```
parent ≠ child
```

Invalid:

```
AI Diagnostic Engine
 |
 ▼
AI Diagnostic Engine
```

This creates:

```
SELF LOOP ❌
```

Reject.

## 24. HV Check 2 — Cycle Detection

This is extremely important.

Suppose current hierarchy:

```
AI System
 ↓
Diagnostic System
 ↓
AI Diagnostic Engine
```

Now galti se system tries:

```
AI Diagnostic Engine
 ↓
AI System
```

Then:

```
AI System
 ↓
Diagnostic System
 ↓
AI Diagnostic Engine
 ↓
AI System
```

Infinite cycle:

```
A → B → C → A
```

HV parent se upward traverse karke check karta hai ki child ancestor chain me already exist toh nahi karta.

Agar child mil jaye:

```
Cycle detected ❌
```

Report explicitly ye cycle detection strategy define karti hai. `filecite{turn20file0}{L214-L220}`

---

## 25. Cycle kyun dangerous hai?

Layer 6 retrieval me traversal hoga.

Agar:

```
A → B → C → A
```

then BFS/DFS:

```
A → B → C → A → B → ...
```

potentially loop kar sakta hai.

Aur Tree Knapsack ka fundamental assumption bhi break ho sakta hai:

*Tree must actually be a tree.*

Isliye HV essential hai.

---

## 26. HV Check 3 — Depth Limit

Report condition:

```
parent.depth + 1 < config.max_depth
```

Depth unlimited nahi hone di jayegi. `filecite{turn20file0}{L214-L220}`

Suppose:

```
max_depth = 8
```

Already:

```
Root
 ↓ 1
A
 ↓ 2
B
 ↓ 3
C
 ...
```

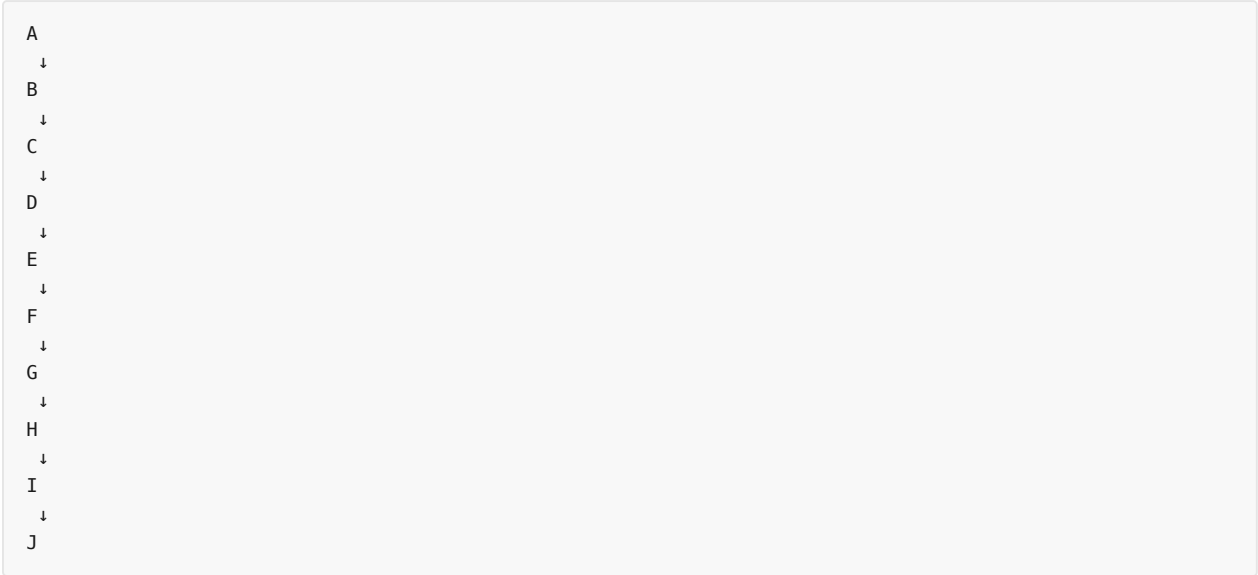
Agar new child add karne se configured depth limit cross hoti hai:

Reject ✗

---

## Why depth limit?

Otherwise hierarchy:



bahut deep ho sakti hai.

Retrieval expensive ho sakta hai.

Context chain unnecessarily long ho sakti hai.

27. HV Check 4 — Sibling Coherence

Suppose parent:

AI Diagnostic System

Existing children:

Image Analysis Module  
Risk Prediction Module  
Symptom Analysis Module

New child:

Hospital Cafeteria Menu

Semantically unrelated.

Sibling coherence low hogi.

But important:

Report ke according:

Sibling coherence *soft penalty* hai, *hard rejection gate* nahi.

Meaning:

Low sibling coherence  
≠  
Automatically reject

Instead ranking/validation quality affect ho sakti hai. `filecite turn20file0 L214-L220`

## 28. HV output

HV:

PASS

or:

FAIL

Hamare example me:

Parent = AI Diagnostic System  
Child = AI Diagnostic Engine

Check:

Self-loop?            No ✓  
Cycle?                No ✓  
Depth valid?        Yes ✓  
Sibling coherence?   Good ✓

Final:

HV = PASS

---

## PART D — FI

### 29. FI kya hai?

FI =

#### Forest Inserter

Ye actual insertion karta hai.

PRE ranked list deta hai:

#1 AI Diagnostic System  
#2 Medical Decision System  
#3 Smart Hospital System

FI:

Try #1  
↓  
HV Pass?

Agar yes:

Insert

Agar fail:

Try #2



Phir:

Try #3

Aur sab fail:

NEW ROOT

Report ka exact insertion flow isi fallback behavior ko implement karta hai. [filecite\turn20file0\L221-L239](#)

---

### 30. Hamare example me FI

Candidate #1:

AI Diagnostic System

HV:

PASS ✓

Then:

```
forest.add_edge(
 AI Diagnostic System,
 AI Diagnostic Engine,
 edge_class = HIERARCHICAL
)
```

Then:

```
node.parent =
AI Diagnostic System
```

Then:

```
node.depth =
parent.depth + 1
```

Example:

```
AI Diagnostic System
depth = 2

AI Diagnostic Engine
depth = 3
```

Report insertion ke baad confidence propagation bhi call karti hai. [filecite\turn20file0\L226-L238](#)

---

### 31. Agar Candidate #1 fail ho jaye?

Suppose cycle detected.

```
#1 AI Diagnostic System
HV → FAIL ❌
```

Then:

```
#2 Medical Decision System
HV → PASS ✓
```

Then node wahan insert ho jayega.

So:

*PRE ka #1 candidate final answer guaranteed nahi hai.*

**HV is the final structural gate.**

---

### 32. Agar sab candidates fail?

```
Candidate #1 → FAIL
Candidate #2 → FAIL
Candidate #3 → FAIL
Candidate #4 → FAIL
Candidate #5 → FAIL
```

Then:

```
forest.add_root(node)
```

Output:

```
NEW ROOT
```

Report result:

```
InsertResult(
 status='NEW_ROOT',
 reason='NO_VALID_CANDIDATE'
)
```

filecite turn20file0L236-L239

---

### 33. New root example

Suppose system already mostly Healthcare domain me hai.

New concept:

```
Hospital Cybersecurity Architecture
```

CPG++ candidates:

```
AI Diagnostic System
Patient Care
Machine Learning Model
```

PRE scores low.

No candidate crosses acceptable threshold.

Then:

```
Hospital Cybersecurity Architecture
```

becomes:

NEW ROOT

A new tree:

FOREST

Tree 1:

Smart Hospital System

|

├─ Patient Care

└─ AI Diagnostic System

Tree 2:

Hospital Cybersecurity Architecture

This is exactly why system is called:

## Knowledge Forest

not one single Knowledge Tree.

## PART E — CLD

### 34. CLD kya hai?

CLD =

### Cross-Link Discoverer

Ab node hierarchy me place ho chuka hai.

But real-world knowledge sirf parent-child relationship nahi hota.

Example:

AI Diagnostic Engine

hierarchically:

AI Diagnostic System

↓

AI Diagnostic Engine

But semantically:

AI Diagnostic Engine

USES

Machine Learning Model

So CLD additional:

### non-hierarchical semantic links

discover karta hai. <https://chatgpt.com/e2b31603-f9df-4055-82cd-e524a077a7c4>

### 35. CLD step-by-step

Newly placed node:

```
AI Diagnostic Engine
```

#### Step 1 — Vector neighbors search

```
AI Diagnostic Engine embedding
↓
Vector Index
↓
Top k neighbors
```

Suppose:

```
Machine Learning Model
Emergency Risk
Patient Symptoms
Medical History
AI Diagnostic System
```

#### Step 2 — Self skip

```
neighbor.id == node.id
```

Then skip.

Obviously:

```
AI Diagnostic Engine
↔
AI Diagnostic Engine
```

cross-link nahi banana.

#### Step 3 — Parent skip

Agar neighbor:

```
AI Diagnostic System
```

hai aur already parent hai:

```
AI Diagnostic System
↓
AI Diagnostic Engine
```

Then semantic cross-link duplicate/unnecessary avoid kiya jata hai.

Report code self aur parent ko skip karta hai. `filecite\turn20file0\L249-L253`

#### Step 4 — Similarity threshold

Suppose:

```

AI Diagnostic Engine
↔
Machine Learning Model

similarity = 0.88

```

Agar:

```
0.88 > cross_link_threshold
```

then next step.

## Step 5 — Relation classification

System:

```

classify_relation(
 AI Diagnostic Engine,
 Machine Learning Model
)

```

Suppose output:

```
USES
```

Then:

```

AI Diagnostic Engine
 — USES —>
Machine Learning Model

```

Semantic edge create hota hai.

Report CLD me **USES** , **CAUSES** jaise relation types classify karne ka step explicitly diya gaya hai.

filecite[turn20file0][L255-L264]

## 36. CLD ke baad final structure

```

SMART HOSPITAL SYSTEM
├── AI DIAGNOSTIC SYSTEM
│ ├── AI DIAGNOSTIC ENGINE
│ │ ├── USES —> MACHINE LEARNING MODEL
│ │ ├── ANALYZES —> PATIENT SYMPTOMS
│ │ └── ESTIMATES —> EMERGENCY RISK
│ └── RISK PREDICTION MODULE

```

Solid hierarchy:

```
parent → child
```

Cross-links:

semantic → semantic

Dono ko separate maintain karna Layer 5 ka major design principle hai.

### 37. Ab poora Layer 5 ek example par end-to-end

Let's process:

**New Node = AI Diagnostic Engine**

#### INPUT

KnowledgeNode:

ID:

NODE\_004

Name:

AI Diagnostic Engine

Domain:

Healthcare AI

Confidence:

0.91

Embedding:

[...]

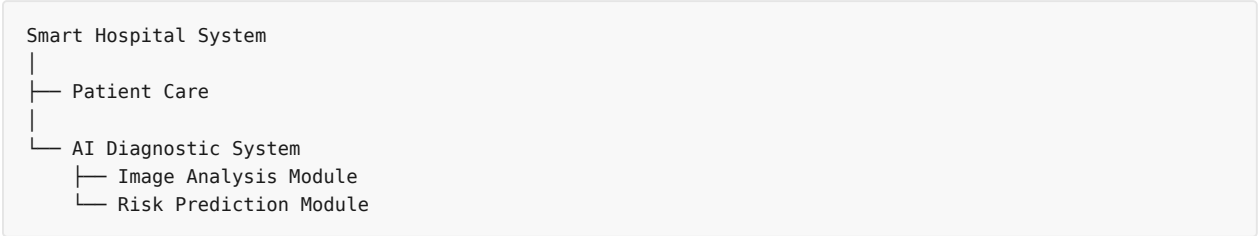
Definitions:

[  
"An AI component that analyzes clinical data  
to assist in diagnostic risk assessment."  
]

Evidence:

[  
Document A,  
Document B,  
Document C  
]

Existing forest:



#### Step 1 — CPG++

Domain

Healthcare AI

Candidates after domain filtering:

```
Smart Hospital System
AI Diagnostic System
Image Analysis Module
Risk Prediction Module
Machine Learning Model
Emergency Risk
```

## ANN

Top semantic matches:

```
AI Diagnostic System
Machine Learning Model
Risk Prediction Module
Smart Hospital System
Emergency Risk
```

## Relation filter

|                        |                   |
|------------------------|-------------------|
| AI Diagnostic System   | ✓ plausible       |
| Machine Learning Model | ? weak            |
| Risk Prediction Module | ✓ possible        |
| Smart Hospital System  | ✓ possible        |
| Emergency Risk         | x semantic object |

## Abstraction filter

Suppose:

```
AI Diagnostic System
 abstraction = 0.72

Risk Prediction Module
 abstraction = 0.40

Smart Hospital System
 abstraction = 0.85

AI Diagnostic Engine
 abstraction = 0.25
```

Keep more abstract candidates.

Final CPG++:

```
[
 AI Diagnostic System,
 Risk Prediction Module,
 Smart Hospital System
]
```

## Step 2 — PRE

Scores:

```
AI Diagnostic System
→ 0.89

Risk Prediction Module
→ 0.71

Smart Hospital System
→ 0.73
```

Sorted:

```
#1 AI Diagnostic System
#2 Smart Hospital System
#3 Risk Prediction Module
```

### Step 3 — HV

Candidate #1:

```
AI Diagnostic System
 ↓
AI Diagnostic Engine
```

Checks:

```
Self loop? NO
Cycle? NO
Depth valid? YES
Sibling coherence? YES
```

Result:

```
PASS ✓
```

### Step 4 — FI

Actual insertion:

```
AI Diagnostic System
 |
 ├── Image Analysis Module
 ├── Risk Prediction Module
 |
 └── AI Diagnostic Engine ← NEW
```

Node metadata:

```
parent =
AI Diagnostic System

depth =
parent.depth + 1
```

Confidence propagation called.



## Step 5 — CLD

Nearest semantic neighbors:

```
Machine Learning Model
Patient Symptoms
Medical History
Emergency Risk
```

Discover:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

```
AI Diagnostic Engine
 ANALYZES
Patient Symptoms
```

```
AI Diagnostic Engine
 ANALYZES
Medical History
```

```
AI Diagnostic Engine
 ESTIMATES
Emergency Risk
```

## 38. Layer 5 ka final output for this node

Knowledge Forest

```
Smart Hospital System
|
├── Patient Care
|
├── AI Diagnostic System
| |
| ├── Image Analysis Module
| |
| ├── Risk Prediction Module
| |
| └── AI Diagnostic Engine
| |
| ├── USES ———> Machine Learning Model
| |
| ├── ANALYZES ———> Patient Symptoms
| |
| ├── ANALYZES ———> Medical History
| |
| └── ESTIMATES ———> Emergency Risk
```

## 39. Layer 5 me error cases

Ab hum **errors aur mistakes** ko deeply dekhenge.

## Error 1 — Wrong domain routing

AI Diagnostic Engine

Wrong domain:

Hospital Administration

Correct parent:

AI Diagnostic System

different candidate pool me chala gaya.

Result:

Correct parent never reaches PRE ❌

## Error 2 — ANN correct parent miss kar de

ANN approximate hai.

Possible:

True Parent:  
AI Diagnostic System

ANN Result:  
not in top-k

Then:

CPG++ cannot rank it

PRE perfect hone ke baad bhi useless.

This is **candidate recall problem**.

## Error 3 — High semantic similarity but wrong parent

Example:

AI Diagnostic Engine  
↔  
Machine Learning Model

high similarity.

But actual relationship:

USES

not:

IS\_A / parent-child

If relation filter weak hua:

Wrong hierarchy

ban sakti hai.

Error 4 — Abstraction estimation wrong

Suppose:

Machine Learning Model

incorrectly more abstract score receive kare.

Then:

Machine Learning Model  
↓  
AI Diagnostic System

jaisi wrong hierarchy ban sakti hai.

Report khud abstraction heuristic ke liye domain-specific empirical validation ki need mention karti hai.

filecite{turn20file0}{L164-L175}

Error 5 — PRE weights poorly tuned

Current weights:

Similarity = 0.30  
Hypernym = 0.25  
Depth = 0.15  
Evidence = 0.20  
Confidence = 0.10

Suppose semantic similarity bahut dominant ho jaye.

Then:

highly similar sibling

galti se parent ban sakta hai.

Example:

Image Analysis Module  
Risk Prediction Module

semantically related hain, but both may be siblings.

PRE ko distinguish karna hoga:

Similarity  
≠  
Parenthood

## Error 6 — Hypernym score error

Hearst patterns:

```
X is a type of Y
```

kabhi source text me available nahi honge.

LLM fallback wrong classify kar sakta hai.

Result:

```
Wrong general-specific direction
```

## Error 7 — Cycle

Without HV:

```
A → B
B → C
C → A
```

possible.

HV isko prevent karta hai. `filecite[turn20file0]L214-L220`

## Error 8 — Maximum depth issue

Agar node logically deep hierarchy me belong karta hai, but:

```
max_depth
```

limit reached hai:

```
Valid parent
```

HV reject kar sakta hai.

Then FI:

```
another parent
```

try karega.

Worst case:

```
New root
```

ban sakta hai, although concept logically existing tree me belong karta tha.

## Error 9 — Cross-link false positive

CLD:

```
High embedding similarity
```

find karta hai.

But:

```
High similarity
≠
Actual relation
```

Example:

```
Emergency Risk
```

and:

```
Preliminary Risk Score
```

highly similar.

LLM relation classifier galti se:

```
CAUSES
```

predict kar sakta hai.

False semantic edge create ho sakta hai.

## 40. Exception handling — report me clearly kya hai?

Main yahan strict distinction kar raha hoon.

### Report me explicitly defined behavior

#### A. No acceptable candidate

```
best PRE score
<
new_tree_threshold
```

Then:

```
No candidate
→ New Root
```

📄filecite📄turn20file0📄L205-L209📄

#### B. Candidate validation fail

```
Try next ranked candidate
```

All fail:

```
New Root
```

📄filecite📄turn20file0📄L221-L239📄

### C. Cycle

Reject edge

### D. Self-loop

Reject edge

### E. Depth violation

Reject candidate

### F. CLD me no valid relation

No cross-link created

Because relation classifier output valid relation nahi deta. `filecite\turn20file0\L241-L264`

## 41. Missing exception handling — report me explicitly specified nahi hai

Ye points report ke shown Layer 5 implementation me **fully specified nahi hain**:

### 1. ANN/vector database unavailable ho gaya toh?

Exact fallback:

Not specified in shown Layer 5 section.

### 2. Embedding contains invalid values/NaN?

Exact handling:

Not specified.

### 3. Domain classification itself fails?

Exact retry/fallback:

Not specified.

### 4. LLM abstraction fallback timeout?

Exact exception policy:

Not specified.

### 5. Multiple parent candidates ka exactly same PRE score?

Tie-breaking rule:

Not specified.

## 6. `propagate_confidence(forest)` internally exactly kya karta hai?

FI usko call karta hai, but shown Layer 5 section me uska complete implementation:

Not specified.

## 42. Important missing implementation dependencies

Layer 5 pseudocode kuch functions use karta hai jinke exact implementations is shown section me nahi diye gaye:

```
classify_domain()
has_plausible_relation()
cross_doc_evidence_count()
hypernym_score()
evidence_support()
hv_validate()
propagate_confidence()
classify_relation()
```

Isliye unke internal behavior ke baare me exact claim nahi kar sakte.

*Report function ko use karti hai, lekin shown Layer 5 section me har helper function ka complete algorithm provide nahi hai.*

Ye implementation ke time fill karna padega.

## 43. Layer 5 ki sabse badi conceptual challenge

Mere analysis ke according sabse difficult distinction hai:

Semantic Similarity

vs

Hierarchical Parent-Child Relationship

Example:

```
AI Diagnostic Engine
↔
Machine Learning Model
```

Highly related.

But hierarchy:

```
AI Diagnostic System
↓
AI Diagnostic Engine
```

Isliye:

Similar ≠ Parent

Layer 5 isi problem ko solve karne ke liye multiple signals combine karta hai:

```
Domain
+ ANN Similarity
+ Plausible Relation
+ Abstraction
+ Hypernym
+ Depth
+ Evidence
+ Confidence
+ Structural Validation
```

Ye ek single cosine similarity decision se much stronger design hai.

## 44. Layer 5 ka complete pseudocode

Report ke described components ko combine karke complete flow:

```
FOR each new node N:

 # 1. CPG++
 candidates = generate_candidate_parents(N)

 # 2. PRE
 ranked_candidates = rank_all_candidates(N, candidates)

 # If no candidate reaches threshold
 IF ranked_candidates is empty:
 insert N as NEW ROOT

 ELSE:

 # 3 + 4
 FOR candidate P in ranked_candidates:

 # HV
 IF hierarchy_is_valid(P, N):

 # FI
 insert P → N
 assign parent
 assign depth
 propagate confidence

 BREAK

 # No candidate passed
 IF N not inserted:
 insert N as NEW ROOT

 # 5. CLD
 discover semantic cross-links for N

RETURN Knowledge Forest
```

Ye report ke 5-stage pipeline aur FI fallback logic ka integrated representation hai. [filecite\turn20file0\L77-L91](#)  
[filecite\turn20file0\L221-L264](#)

## 45. Layer 5 ke baad data ka transformation

### Before Layer 5



```
Nodes
+
Edges
```

Flat collection:

```
NODE_001
NODE_002
NODE_003
NODE_004
...
```

Relations:

```
A USES B
A ANALYZES C
D CAUSES E
```

But no complete navigable hierarchy.

## After Layer 5

Knowledge Forest

```
ROOT
|
├── Topic A
│ ├── Subtopic A1
│ └── Subtopic A2
└── Topic B
 ├── Subtopic B1
 └── Subtopic B2
```

+ semantic cross-links

Report ke architecture me Layer 5 ke baad ye Knowledge Forest retrieval layer ko input structure provide karta hai.

filecite[turn18file0][L47-L55]

## 46. Layer 5 → Layer 6 handoff

Layer 5 final output:

Knowledge Forest

Ye Layer 6 ko jata hai.

Layer 6 phir query ke according:

```
User Query
↓
Find Seed Nodes
↓
Traverse Parents / Children / Cross-links
↓
Tree Knapsack
↓
Selected Context
```

Isliye Layer 5 ki hierarchy directly Layer 6 retrieval ko affect karegi. Report Layer 6 me adaptive traversal aur multi-tree grouping use karti hai. `filecite\turn20file0\L322-L380`

**Bahut important:**

Agar Layer 5 me hierarchy wrong:

```
Wrong Parent
 ↓
Wrong Tree
 ↓
Wrong Traversal Path
 ↓
Wrong Context
 ↓
Potentially Wrong LLM Answer
```

So Layer 5 is not just storage organization.

**Layer 5 directly retrieval quality control karta hai.**

---

**47. Layer 5 ko ek line me yaad rakho**

**CPG++**

Possible parents dhoondo

**PRE**

Possible parents ko rank karo

**HV**

Check karo hierarchy valid hai ya nahi

**FI**

Node ko forest me insert karo

**CLD**

Additional semantic cross-links discover karo

---

**Final complete mental model**



Sabse important understanding

Layer 4 ke paas knowledge relationships hain, lekin Layer 5 un relationships aur concept properties ka use karke knowledge ko ek navigable structure me organize karta hai — pehle possible parent generate karta hai, phir best parent rank karta hai, hierarchy validate karta hai, node insert karta hai, aur finally non-hierarchical semantic cross-links discover karta hai.

memcite

YOU 20/08/2026, 15:26:06

ab mujhe layer 6 ache se smjhao

CHATGPT 20/08/2026, 15:26:12

```
{"paths":["files"],"query":"search"}
```

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file0]`

[L1] metadata:

[L2] query\_result\_index: 0

[L3] file\_id: file\_00000000a0a481fab4c739c73d87551b

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(7).md

[L6] mime\_type: text/markdown

[L7] surface: library

[L8] score: 0.032018442622950824

[L9] document\_chunk\_id: file\_00000000a0a481fab4c739c73d87551b--1--45

[L10] snippet:

[L11] @dataclass

[L12] class RetrievalResult:

[L13] """Output of Layer 6 — input to Layer 7."""

[L14] selected\_nodes: list # Nodes selected by Tree Knapsack

[L15] context\_text: str # Assembled context string

[L16] traversal\_path\_edges: list # Edges traversed (for Layer 8 feedback)

[L17] query\_complexity: str # SIMPLE, MODERATE, COMPLEX

[L18] query\_intent: str # DEFINITION, COMPARISON, etc.

[L19] seed\_node\_ids: list # Initial seed nodes from vector search

[L20] token\_count: int # Estimated tokens in context

[L21]

[L22] class RetrievalEngine:

[L23] def \_\_init\_\_(self, forest, vector\_index, cache\_manager, config):

[L24] self.forest = forest

[L25] self.vector\_index = vector\_index

[L26] self.cache = cache\_manager # Redis cache integration

[L27] self.config = config

[L28] self.seed\_k = config.get('seed\_k', 5) # Default: 5 seed nodes

[L29]

[L30] def retrieve(self, query) -> RetrievalResult:

[L31] # Step 0: Cache check

[L32] cache\_key = hash\_query(query)

[L33] cached = self.cache.get\_query\_result(cache\_key)

[L34] if cached:

[L35] return cached

[L36]

[L37] # Step 1: Analyze query

```
[L38] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L39] complexity = classify_complexity(intent) # Rule-based mapping
[L40]
[L41] # Step 2: Embed and find seed nodes
[L42] query_emb = embed_query(query) # Asymmetric embedding model
[L43] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L44]
[L45] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L46] depth_config = {
[L47] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L48] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L49] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L50] }
[L51] traversal = depth_config[complexity]
[L52]
[L53] # Step 4: Traverse forest from all seeds
[L54] candidate_nodes = set()
[L55] for seed in seed_nodes:
[L56] candidate_nodes |= self.traverse(seed, traversal)
[L57]
[L58] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L59] selected = self.multi_tree_knapsack(
[L60] candidate_nodes,
[L61] budget=self.config.token_budget
[L62])
[L63]
[L64] # Step 6: Assemble context in semantic order
[L65] context_text = self.assemble_context(selected, intent)
[L66]
[L67] # Build result with traversal path for Layer 8 feedback
[L68] result = RetrievalResult(
[L69] selected_nodes=selected,
[L70] context_text=context_text,
[L71] traversal_path_edges=self._get_path_edges(selected),
[L72] query_complexity=complexity,
[L73] query_intent=intent,
[L74] seed_node_ids=[s.id for s in seed_nodes],
[L75] token_count=estimate_tokens(context_text)
[L76])
[L77]
[L78] # Cache result
[L79] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L80]
[L81] return result
[L82]
[L83] def multi_tree_knapsack(self, candidate_nodes, budget):
```

[L84] """Apply Tree Knapsack per distinct tree, then merge results.

[L85]

[L86] For multi-seed queries, candidates may span multiple trees.

[L87] We optimize each tree independently, then merge within budget.

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite`turn21file1

[L88] metadata:

[L89] query\_result\_index: 1

[L90] file\_id: file\_000000001dd481fa9855f2731dbc53d5

[L91] version\_id: 1

[L92] name: SHIA\_RAG\_Complete\_Report(6).md

[L93] mime\_type: text/markdown

[L94] surface: library

[L95] score: 0.03177805800756621

[L96] document\_chunk\_id: file\_000000001dd481fa9855f2731dbc53d5--1--45

[L97] snippet:

[L98] @dataclass

[L99] class RetrievalResult:

[L100] """Output of Layer 6 — input to Layer 7."""

[L101] selected\_nodes: list # Nodes selected by Tree Knapsack

[L102] context\_text: str # Assembled context string

[L103] traversal\_path\_edges: list # Edges traversed (for Layer 8 feedback)

[L104] query\_complexity: str # SIMPLE, MODERATE, COMPLEX

[L105] query\_intent: str # DEFINITION, COMPARISON, etc.

[L106] seed\_node\_ids: list # Initial seed nodes from vector search

[L107] token\_count: int # Estimated tokens in context

[L108]

[L109] class RetrievalEngine:

[L110] def \_\_init\_\_(self, forest, vector\_index, cache\_manager, config):

[L111] self.forest = forest

[L112] self.vector\_index = vector\_index

[L113] self.cache = cache\_manager # Redis cache integration

[L114] self.config = config

[L115] self.seed\_k = config.get('seed\_k', 5) # Default: 5 seed nodes

[L116]

[L117] def retrieve(self, query) -> RetrievalResult:

[L118] # Step 0: Cache check

[L119] cache\_key = hash\_query(query)

[L120] cached = self.cache.get\_query\_result(cache\_key)

[L121] if cached:

[L122] return cached

[L123]

[L124] # Step 1: Analyze query

[L125] intent = detect\_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS

```
[L126] complexity = classify_complexity(intent) # Rule-based mapping
[L127]
[L128] # Step 2: Embed and find seed nodes
[L129] query_emb = embed_query(query) # Asymmetric embedding model
[L130] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L131]
[L132] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L133] depth_config = {
[L134] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L135] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L136] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L137] }
[L138] traversal = depth_config[complexity]
[L139]
[L140] # Step 4: Traverse forest from all seeds
[L141] candidate_nodes = set()
[L142] for seed in seed_nodes:
[L143] candidate_nodes |= self.traverse(seed, traversal)
[L144]
[L145] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L146] selected = self.multi_tree_knapsack(
[L147] candidate_nodes,
[L148] budget=self.config.token_budget
[L149])
[L150]
[L151] # Step 6: Assemble context in semantic order
[L152] context_text = self.assemble_context(selected, intent)
[L153]
[L154] # Build result with traversal path for Layer 8 feedback
[L155] result = RetrievalResult(
[L156] selected_nodes=selected,
[L157] context_text=context_text,
[L158] traversal_path_edges=self._get_path_edges(selected),
[L159] query_complexity=complexity,
[L160] query_intent=intent,
[L161] seed_node_ids=[s.id for s in seed_nodes],
[L162] token_count=estimate_tokens(context_text)
[L163])
[L164]
[L165] # Cache result
[L166] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L167]
[L168] return result
[L169]
[L170] def multi_tree_knapsack(self, candidate_nodes, budget):
[L171] """Apply Tree Knapsack per distinct tree, then merge results.
```

[L172]

[L173] For multi-seed queries, candidates may span multiple trees.

[L174] We optimize each tree independently, then merge within budget.

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file2]`

[L175] metadata:

[L176] query\_result\_index: 2

[L177] file\_id: file\_000000003c8081fab658b1752549d64

[L178] version\_id: 1

[L179] name: SHIA\_RAG\_Complete\_Report(5).md

[L180] mime\_type: text/markdown

[L181] surface: library

[L182] score: 0.03177805800756621

[L183] document\_chunk\_id: file\_000000003c8081fab658b1752549d64--1--45

[L184] snippet:

[L185] @dataclass

[L186] class RetrievalResult:

[L187] """Output of Layer 6 — input to Layer 7."""

[L188] selected\_nodes: list # Nodes selected by Tree Knapsack

[L189] context\_text: str # Assembled context string

[L190] traversal\_path\_edges: list # Edges traversed (for Layer 8 feedback)

[L191] query\_complexity: str # SIMPLE, MODERATE, COMPLEX

[L192] query\_intent: str # DEFINITION, COMPARISON, etc.

[L193] seed\_node\_ids: list # Initial seed nodes from vector search

[L194] token\_count: int # Estimated tokens in context

[L195]

[L196] class RetrievalEngine:

[L197] def \_\_init\_\_(self, forest, vector\_index, cache\_manager, config):

[L198] self.forest = forest

[L199] self.vector\_index = vector\_index

[L200] self.cache = cache\_manager # Redis cache integration

[L201] self.config = config

[L202] self.seed\_k = config.get('seed\_k', 5) # Default: 5 seed nodes

[L203]

[L204] def retrieve(self, query) -> RetrievalResult:

[L205] # Step 0: Cache check

[L206] cache\_key = hash\_query(query)

[L207] cached = self.cache.get\_query\_result(cache\_key)

[L208] if cached:

[L209] return cached

[L210]

[L211] # Step 1: Analyze query

[L212] intent = detect\_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS

[L213] complexity = classify\_complexity(intent) # Rule-based mapping



```
[L214]
[L215] # Step 2: Embed and find seed nodes
[L216] query_emb = embed_query(query) # Asymmetric embedding model
[L217] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L218]
[L219] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L220] depth_config = {
[L221] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L222] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L223] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L224] }
[L225] traversal = depth_config[complexity]
[L226]
[L227] # Step 4: Traverse forest from all seeds
[L228] candidate_nodes = set()
[L229] for seed in seed_nodes:
[L230] candidate_nodes |= self.traverse(seed, traversal)
[L231]
[L232] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L233] selected = self.multi_tree_knapsack(
[L234] candidate_nodes,
[L235] budget=self.config.token_budget
[L236])
[L237]
[L238] # Step 6: Assemble context in semantic order
[L239] context_text = self.assemble_context(selected, intent)
[L240]
[L241] # Build result with traversal path for Layer 8 feedback
[L242] result = RetrievalResult(
[L243] selected_nodes=selected,
[L244] context_text=context_text,
[L245] traversal_path_edges=self._get_path_edges(selected),
[L246] query_complexity=complexity,
[L247] query_intent=intent,
[L248] seed_node_ids=[s.id for s in seed_nodes],
[L249] token_count=estimate_tokens(context_text)
[L250])
[L251]
[L252] # Cache result
[L253] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L254]
[L255] return result
[L256]
[L257] def multi_tree_knapsack(self, candidate_nodes, budget):
[L258] """Apply Tree Knapsack per distinct tree, then merge results.
```

[L259]

[L260] For multi-seed queries, candidates may span multiple trees.

[L261] We optimize each tree independently, then merge within budget.

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file3]`

[L262] metadata:

[L263] query\_result\_index: 3

[L264] file\_id: file\_0000000033088207b45d2ae58cbd406c

[L265] version\_id: 1

[L266] name: SHIA\_RAG\_Complete\_Report(8).md

[L267] mime\_type: text/markdown

[L268] surface: library

[L269] score: 0.030117753623188408

[L270] document\_chunk\_id: file\_0000000033088207b45d2ae58cbd406c--1--45

[L271] snippet:

[L272] @dataclass

[L273] class RetrievalResult:

[L274] """Output of Layer 6 — input to Layer 7."""

[L275] selected\_nodes: list # Nodes selected by Tree Knapsack

[L276] context\_text: str # Assembled context string

[L277] traversal\_path\_edges: list # Edges traversed (for Layer 8 feedback)

[L278] query\_complexity: str # SIMPLE, MODERATE, COMPLEX

[L279] query\_intent: str # DEFINITION, COMPARISON, etc.

[L280] seed\_node\_ids: list # Initial seed nodes from vector search

[L281] token\_count: int # Estimated tokens in context

[L282]

[L283] class RetrievalEngine:

[L284] def \_\_init\_\_(self, forest, vector\_index, cache\_manager, config):

[L285] self.forest = forest

[L286] self.vector\_index = vector\_index

[L287] self.cache = cache\_manager # Redis cache integration

[L288] self.config = config

[L289] self.seed\_k = config.get('seed\_k', 5) # Default: 5 seed nodes

[L290]

[L291] def retrieve(self, query) -> RetrievalResult:

[L292] # Step 0: Cache check

[L293] cache\_key = hash\_query(query)

[L294] cached = self.cache.get\_query\_result(cache\_key)

[L295] if cached:

[L296] return cached

[L297]

[L298] # Step 1: Analyze query

[L299] intent = detect\_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS

[L300] complexity = classify\_complexity(intent) # Rule-based mapping

```
[L301]
[L302] # Step 2: Embed and find seed nodes
[L303] query_emb = embed_query(query) # Asymmetric embedding model
[L304] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L305]
[L306] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L307] depth_config = {
[L308] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L309] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L310] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L311] }
[L312] traversal = depth_config[complexity]
[L313]
[L314] # Step 4: Traverse forest from all seeds
[L315] candidate_nodes = set()
[L316] for seed in seed_nodes:
[L317] candidate_nodes |= self.traverse(seed, traversal)
[L318]
[L319] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L320] selected = self.multi_tree_knapsack(
[L321] candidate_nodes,
[L322] budget=self.config.token_budget
[L323])
[L324]
[L325] # Step 6: Assemble context in semantic order
[L326] context_text = self.assemble_context(selected, intent)
[L327]
[L328] # Build result with traversal path for Layer 8 feedback
[L329] result = RetrievalResult(
[L330] selected_nodes=selected,
[L331] context_text=context_text,
[L332] traversal_path_edges=self._get_path_edges(selected),
[L333] query_complexity=complexity,
[L334] query_intent=intent,
[L335] seed_node_ids=[s.id for s in seed_nodes],
[L336] token_count=estimate_tokens(context_text)
[L337])
[L338]
[L339] # Cache result
[L340] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L341]
[L342] return result
[L343]
[L344] def multi_tree_knapsack(self, candidate_nodes, budget):
[L345] """Apply Tree Knapsack per distinct tree, then merge results.
```

[L346]

[L347] For multi-seed queries, candidates may span multiple trees.

[L348] We optimize each tree independently, then merge within budget.

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `[[filecite]]turn21file4`

[L349] metadata:

[L350] query\_result\_index: 4

[L351] file\_id: file\_00000000f620821186e54e582d59731e

[L352] version\_id: 1

[L353] name: SHIA\_RAG\_Final\_Report(3).md

[L354] mime\_type: text/markdown

[L355] surface: library

[L356] score: 0.01871774687308668

[L357] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--40

[L358] snippet:

[L359] temporal versioning |

[L360] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L361] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L362] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L363]

[L364] ---

[L365]

[L366] ## 13. Conclusion & Future Work

[L367]

[L368] ### 13.1 Summary

[L369]

[L370] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L371]

[L372] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L373] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L374] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L375] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L376] 5. **SHEF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L377]

[L378] The complete system specification covers:

[L379] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L380] - **12 defined functions** with complete implementations

[L381] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L382] - **8 schemas** covering all data objects from document blocks to query logs

[L383] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L384] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L385]

[L386] ### 13.2 Key Novelties

[L387]

[L388] | # | Novelty | Level | Why It's New |

[L389] |---|-----|-----|-----|

[L390] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L391] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L392] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L393] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L394] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L395]

[L396] ### 13.3 Next Steps

[L397]

[L398] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L399] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L400] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L401] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L402] 5. **Build Layer 4** (Relation management with ontology validation)

[L403] 6. **Build Layer 5** (SHIA Core with algorithms)

[L404] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L405] 8. **Build Layer 7** (Generation with verification)

[L406] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L407] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L408] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L409] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L410]

[L411] ### 13.4 Future Work

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file5]`

[L412] metadata:

[L413] query\_result\_index: 5

[L414] file\_id: file\_000000004bc481fdbf9e949ec3b7544b

[L415] version\_id: 1

[L416] name: SHIA\_RAG\_Final\_Report(2).md

[L417] mime\_type: text/markdown

[L418] surface: library

[L419] score: 0.01871774687308668

[L420] document\_chunk\_id: file\_000000004bc481fdbf9e949ec3b7544b--1--40

[L421] snippet:

[L422] temporal versioning |

[L423] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L424] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L425] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L426]

[L427] ---

[L428]

[L429] ## 13. Conclusion & Future Work

[L430]

[L431] ### 13.1 Summary

[L432]

[L433] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L434]

[L435] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem

[L436] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L437] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction

[L438] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation

[L439] 5. **SHF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality

[L440]

[L441] The complete system specification covers:

[L442] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L443] - **12 defined functions** with complete implementations

[L444] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L445] - **8 schemas** covering all data objects from document blocks to query logs

[L446] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L447] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L448]

[L449] ### 13.2 Key Novelties

[L450]

[L451] | # | Novelty | Level | Why It's New |

[L452] |---|-----|-----|-----|

[L453] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |

[L454] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |

[L455] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |

[L456] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |

[L457] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |

[L458]

[L459] ### 13.3 Next Steps

[L460]

[L461] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L462] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L463] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L464] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L465] 5. **Build Layer 4** (Relation management with ontology validation)

[L466] 6. **Build Layer 5** (SHIA Core with algorithms)

[L467] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L468] 8. **Build Layer 7** (Generation with verification)

[L469] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L470] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L471] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L472] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L473]

[L474] ### 13.4 Future Work

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file6]`

[L475] metadata:

[L476] query\_result\_index: 6

[L477] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L478] version\_id: 1

[L479] name: SHIA\_RAG\_Final\_Report(4).md

[L480] mime\_type: text/markdown

[L481] surface: conversation

[L482] score: 0.0186932215234102

[L483] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--54

[L484] snippet:

[L485] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L486] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L487] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L488] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L489] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L490]

[L491] The complete system specification covers:

[L492] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L493] - **12 defined functions** with complete implementations

[L494] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L495] - **8 schemas** covering all data objects from document blocks to query logs

[L496] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L497] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L498]

[L499] ### 13.2 Key Novelties

[L500]

[L501] | # | Novelty | Level | Why It's New |

[L502] |---|-----|-----|-----|

[L503] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L504] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L505] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L506] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L507] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L508]

[L509] ### 13.3 Next Steps

[L510]

[L511] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L512] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L513] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L514] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L515] 5. **Build Layer 4** (Relation management with ontology validation)

[L516] 6. **Build Layer 5** (SHIA Core with algorithms)

[L517] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L518] 8. **Build Layer 7** (Generation with verification)

[L519] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L520] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L521] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L522] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L523]

[L524] ### 13.4 Future Work



[L525] metadata:  
[L526] query\_result\_index: 7  
[L527] file\_id: file\_00000000fb2c81fd82f14a7a48993554  
[L528] version\_id: 1  
[L529] name: SHIA\_RAG\_Final\_Report.md  
[L530] mime\_type: text/markdown  
[L531] surface: library  
[L532] score: 0.018354803401532373  
[L533] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--40  
[L534] snippet:  
[L535] temporal versioning |  
[L536] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |  
[L537] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |  
[L538] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |  
[L539]  
[L540] ---  
[L541]  
[L542] ## 13. Conclusion & Future Work  
[L543]  
[L544] ### 13.1 Summary  
[L545]  
[L546] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:  
[L547]  
[L548] 1. **Precedence-Constrained Tree Knapsack** — the first formulation of RAG context selection as a precedence-constrained optimization problem  
[L549] 2. **Self-Evolving Knowledge Forest** — the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure  
[L550] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction  
[L551] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation  
[L552] 5. **SHF Evaluation Framework** — the first benchmark specifically designed for hierarchical RAG quality  
[L553]  
[L554] The complete system specification covers:  
[L555] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts  
[L556] - **12 defined functions** with complete implementations  
[L557] - **12 relation types** across 4 hierarchical and 8 semantic categories  
[L558] - **8 schemas** covering all data objects from document blocks to query logs  
[L559] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline  
[L560] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing  
[L561]  
[L562] ### 13.2 Key Novelties  
[L563]

[L564] | # | Novelty | Level | Why It's New |  
[L565] |---|-----|-----|-----|  
[L566] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | First RAG system to formulate context selection as a precedence-constrained optimization |  
[L567] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | First closed-loop feedback mechanism that modifies RAG knowledge structures |  
[L568] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | First formal multi-objective optimization for hierarchy induction in RAG |  
[L569] | 4 | **Adaptive Depth Traversal** | Incremental | First query-complexity-aware retrieval depth adaptation in hierarchical RAG |  
[L570] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | First benchmark specifically designed for hierarchical RAG quality |  
[L571]  
[L572] ### 13.3 Next Steps  
[L573]  
[L574] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)  
[L575] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)  
[L576] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)  
[L577] 4. **Build Layer 3** (Knowledge extraction with LLM)  
[L578] 5. **Build Layer 4** (Relation management with ontology validation)  
[L579] 6. **Build Layer 5** (SHIA Core with algorithms)  
[L580] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)  
[L581] 8. **Build Layer 7** (Generation with verification)  
[L582] 9. **Build Layer 8** (Operations, monitoring, self-evolution)  
[L583] 10. **Run first evaluation** on OS textbook dataset using SHEF  
[L584] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)  
[L585] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027  
[L586]  
[L587] ### 13.4 Future Work

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file8]`

[L588] metadata:  
[L589] query\_result\_index: 8  
[L590] file\_id: file\_000000003c8081fab658b1752549d64  
[L591] version\_id: 1  
[L592] name: SHIA\_RAG\_Complete\_Report(5).md  
[L593] mime\_type: text/markdown  
[L594] surface: library  
[L595] score: 0.01830794536222556  
[L596] document\_chunk\_id: file\_000000003c8081fab658b1752549d64--1--44  
[L597] snippet:  
[L598] id)  
[L599] for p in forest.get\_parents(cur):  
[L600] stack.append(p)

```

[L601]
[L602] # Check 3: Depth limit (FIXED: use parent's depth + 1)
[L603] if parent.depth + 1 >= config.max_depth:
[L604] return False
[L605]
[L606] # Check 4: Sibling coherence (soft penalty, not hard rejection)
[L607] # NOTE: Siblings can be semantically different but still valid
[L608] # (e.g., "Time Quantum" and "Preemption" are both children of Round Robin)
[L609] sibling_penalty = 0.0
[L610] siblings = forest.get_children(parent)
[L611] if siblings:
[L612] avg_sim = mean([cosine_similarity(child.embedding, s.embedding) for s in siblings])
[L613] if avg_sim < config.min_sibling_coherence:
[L614] sibling_penalty = config.sibling_penalty # e.g., 0.1
[L615] # Store penalty for PRE to use in scoring, but do NOT reject
[L616] child._sibling_penalty = sibling_penalty
[L617]
[L618] return True # Sibling coherence is a soft signal, not a gate
[L619]

```

```

[L620]
[L621] ---
[L622]
[L623] ### 7.7 Layer 6: Retrieval Engine (with Adaptive Depth)
[L624]
[L625] > [!NOTE]
[L626] > **Query Intent → Complexity Mapping:**
[L627] > | Intent | Example | Complexity | Depth |
[L628] > |-----|-----|-----|-----|
[L629] > | DEFINITION | "What is OSPF?" | SIMPLE | 1 |
[L630] > | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |
[L631] > | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |
[L632] > | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |
[L633] > | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |
[L634]
[L635]

```

python

```

[L636] from dataclasses import dataclass
[L637]
[L638] @dataclass
[L639] class RetrievalResult:
[L640] """Output of Layer 6 — input to Layer 7."""
[L641] selected_nodes: list # Nodes selected by Tree Knapsack
[L642] context_text: str # Assembled context string
[L643] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L644] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L645] query_intent: str # DEFINITION, COMPARISON, etc.
[L646] seed_node_ids: list # Initial seed nodes from vector search
[L647] token_count: int # Estimated tokens in context
[L648]
[L649] class RetrievalEngine:

```

```
[L650] def __init__(self, forest, vector_index, cache_manager, config):
[L651] self.forest = forest
[L652] self.vector_index = vector_index
[L653] self.cache = cache_manager # Redis cache integration
[L654] self.config = config
[L655] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L656]
[L657] def retrieve(self, query) -> RetrievalResult:
[L658] # Step 0: Cache check
[L659] cache_key = hash_query(query)
[L660] cached = self.cache.get_query_result(cache_key)
[L661] if cached:
[L662] return cached
[L663]
[L664] # Step 1: Analyze query
[L665] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L666] complexity = classify_complexity(intent) # Rule-based mapping
[L667]
[L668] # Step 2: Embed and find seed nodes
[L669] query_emb = embed_query(query) # Asymmetric embedding model
[L670] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L671]
[L672] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L673] depth_config = {
[L674] 'SIMPLE': {'max_depth': 1, 'include_parents': True,
```

---

DEEP RESEARCH 20/08/2026, 15:26:17

Citation Marker: `filecite[turn21file9]`

```
[L675] metadata:
[L676] query_result_index: 9
[L677] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L678] version_id: 1
[L679] name: SHIA_RAG_Final_Report(4).md
[L680] mime_type: text/markdown
[L681] surface: conversation
[L682] score: 0.01821162444113264
[L683] document_chunk_id: file_0000000012688211b4f57f5b8bcb8b42--1--32
[L684] snippet:
[L685]
```

```

[L686] @dataclass
[L687] class RetrievalResult:
[L688] """Output of Layer 6 – input to Layer 7."""
[L689] selected_nodes: list # Nodes selected by Tree Knapsack
[L690] context_text: str # Assembled context string
[L691] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L692] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L693] query_intent: str # DEFINITION, COMPARISON, etc.
[L694] seed_node_ids: list # Initial seed nodes from vector search
[L695] token_count: int # Estimated tokens in context
[L696]
[L697] class RetrievalEngine:
[L698] def __init__(self, forest, vector_index, cache_manager, config):
[L699] self.forest = forest
[L700] self.vector_index = vector_index
[L701] self.cache = cache_manager
[L702] self.config = config
[L703]
[L704] def retrieve(self, query) -> RetrievalResult:
[L705] # Step 0: Cache check
[L706] cached = self.cache.get_query_result(hash_query(query))
[L707] if cached:
[L708] return cached
[L709]
[L710] # Step 1: Classify query intent and complexity
[L711] intent = detect_intent(query)
[L712] complexity = classify_complexity(intent)
[L713]
[L714] # Step 2: Embed query and find seed nodes
[L715] query_emb = embed_query(query) # Asymmetric embedding model
[L716] seed_nodes = self.vector_index.search(query_emb, k=5)
[L717]
[L718] # Step 3: Adaptive depth traversal
[L719] depth_config = {
[L720] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L721] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L722] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L723] }
[L724] traversal = depth_config[complexity]
[L725]
[L726] # Step 4: BFS traversal from all seeds
[L727] candidate_nodes = set()
[L728] for seed in seed_nodes:
[L729] candidate_nodes |= self.traverse(seed, traversal)
[L730]
[L731] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L732] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L733]
[L734] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L735] context_text = self.assemble_context(selected, intent)
[L736]
[L737] return RetrievalResult(
[L738] selected_nodes=selected, context_text=context_text,
[L739] traversal_path_edges=self._get_path_edges(selected),
[L740] query_complexity=complexity, query_intent=intent,
[L741] seed_node_ids=[s.id for s in seed_nodes],
[L742] token_count=estimate_tokens(context_text)
[L743])
[L744]

```

[L745]

[L746] ##### Multi-Tree Knapsack Selection

[L747]

[L748] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L749]

[L750] 1. **Group** candidates by tree root

[L751] 2. **Allocate** token budget proportionally to each tree

[L752] 3. **Run** Tree Knapsack on each tree independently

[L753] 4. **Merge** results and trim to total budget

[L754]

[L755] ```python

[L756] def multi\_tree\_knapsack(self, candidate\_nodes, budget):

[L757] tree\_groups = {}

[L758] for node in candidate\_nodes:

[L759] root = self.forest.find\_root(node)

[L760] tree\_groups.setdefault(root.id, []).append(node)

[L761]

[L762] all\_selected = []

[L763] for root\_id, nodes in tree\_groups.items():

[L764] tree\_budget = int(budget \* len(nodes) / max(len(candidate\_nodes), 1))

[L765] \_, selected = tree\_knapsack(self.forest.get\_node(root\_id), tree\_budget, self.

---

DEEP RESEARCH 20/08/2026, 15:26:21

Citation Marker: `filecite[turn22file0]`

[L1] metadata:

[L2] ref\_id: turn21file8

[L3] file\_id: file\_000000003c8081fab658b1752549d64

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(5).md

[L6] mode: chunk\_context

[L7] lines\_shown: 1-120

[L8] total\_file\_lines: 281

[L9] has\_more: True

[L10] next\_start\_line: 121

[L11] next\_read:

{"ref\_id":"turn21file8","mode":"chunk\_context","start\_line":121,"context\_before\_lines":20,"context\_after\_lines":260}

[L12] content:

[L13] > | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |

[L14] > | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |

[L15] > | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |

[L16] > | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |

[L17]

[L18] ```python

[L19] from dataclasses import dataclass

[L20]

[L21] @dataclass

[L22] class RetrievalResult:

[L23] """Output of Layer 6 — input to Layer 7."""

```

[L24] selected_nodes: list # Nodes selected by Tree Knapsack
[L25] context_text: str # Assembled context string
[L26] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L27] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L28] query_intent: str # DEFINITION, COMPARISON, etc.
[L29] seed_node_ids: list # Initial seed nodes from vector search
[L30] token_count: int # Estimated tokens in context
[L31]
[L32] class RetrievalEngine:
[L33] def __init__(self, forest, vector_index, cache_manager, config):
[L34] self.forest = forest
[L35] self.vector_index = vector_index
[L36] self.cache = cache_manager # Redis cache integration
[L37] self.config = config
[L38] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L39]
[L40] def retrieve(self, query) -> RetrievalResult:
[L41] # Step 0: Cache check
[L42] cache_key = hash_query(query)
[L43] cached = self.cache.get_query_result(cache_key)
[L44] if cached:
[L45] return cached
[L46]
[L47] # Step 1: Analyze query
[L48] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L49] complexity = classify_complexity(intent) # Rule-based mapping
[L50]
[L51] # Step 2: Embed and find seed nodes
[L52] query_emb = embed_query(query) # Asymmetric embedding model
[L53] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L54]
[L55] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L56] depth_config = {
[L57] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L58] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L59] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L60] }
[L61] traversal = depth_config[complexity]
[L62]
[L63] # Step 4: Traverse forest from all seeds
[L64] candidate_nodes = set()
[L65] for seed in seed_nodes:
[L66] candidate_nodes |= self.traverse(seed, traversal)
[L67]
[L68] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L69] selected = self.multi_tree_knapsack(

```

```
[L70] candidate_nodes,
[L71] budget=self.config.token_budget
[L72])
[L73]
[L74] # Step 6: Assemble context in semantic order
[L75] context_text = self.assemble_context(selected, intent)
[L76]
[L77] # Build result with traversal path for Layer 8 feedback
[L78] result = RetrievalResult(
[L79] selected_nodes=selected,
[L80] context_text=context_text,
[L81] traversal_path_edges=self._get_path_edges(selected),
[L82] query_complexity=complexity,
[L83] query_intent=intent,
[L84] seed_node_ids=[s.id for s in seed_nodes],
[L85] token_count=estimate_tokens(context_text)
[L86])
[L87]
[L88] # Cache result
[L89] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L90]
[L91] return result
[L92]
[L93] def multi_tree_knapsack(self, candidate_nodes, budget):
[L94] """Apply Tree Knapsack per distinct tree, then merge results.
[L95]
[L96] For multi-seed queries, candidates may span multiple trees.
[L97] We optimize each tree independently, then merge within budget.
[L98]
[L99] Note: Semantic cross-linked nodes do NOT require their
[L100] semantic predecessors to be selected (only hierarchical
[L101] parent constraints apply).
[L102] """
[L103] # Group candidates by their tree root
[L104] tree_groups = {}
[L105] for node in candidate_nodes:
[L106] root = self.forest.find_root(node)
[L107] tree_groups.setdefault(root.id, []).append(node)
[L108]
[L109] # Budget allocation: proportional to number of candidates per tree
[L110] total_candidates = len(candidate_nodes)
[L111] all_selected = []
[L112]
[L113] for root_id, nodes in tree_groups.items():
[L114] # Allocate budget proportionally
[L115] tree_budget = int(budget * len(nodes) / max(total_candidates, 1))
```



```

[L116] root_node = self.forest.get_node(root_id)
[L117]
[L118] # Run Tree Knapsack on this tree
[L119] _, selected_set = tree_knapsack(root_node, tree_budget, self.forest)
[L120]
[L121] # Filter: only keep nodes that were in our candidate set
[L122] selected = [n for n in selected_set if n in nodes]
[L123] all_selected.extend(selected)
[L124]
[L125] # Final trim if over total budget
[L126] all_selected.sort(key=lambda n: utility(n, self.forest), reverse=True)
[L127] total_tokens = 0
[L128] final = []
[L129] for node in all_selected:
[L130] cost = token_cost(node)
[L131] if total_tokens + cost <= budget:
[L132] final.append(node)

```

---

DEEP RESEARCH 20/08/2026, 15:26:25

Citation Marker: `filecite[turn23file0]`

```

[L1] metadata:
[L2] ref_id: turn21file8
[L3] file_id: file_000000003c8081fab658b1752549d64
[L4] version_id: 1
[L5] name: SHIA_RAG_Complete_Report(5).md
[L6] mode: chunk_context
[L7] lines_shown: 121-240
[L8] total_file_lines: 281
[L9] has_more: True
[L10] next_start_line: 241
[L11] next_read:
{"ref_id":"turn21file8","mode":"chunk_context","start_line":241,"context_before_lines":20,"context_after_lines":260}
[L12] content:
[L13] total_tokens += cost
[L14]
[L15] return final
[L16]
[L17] def traverse(self, node, config):
[L18] """BFS traversal from seed node based on depth config."""
[L19] visited = set()
[L20] queue = [(node, 0)]
[L21]
[L22] while queue:
[L23] current, depth = queue.pop(0)
[L24] if current.id in visited or depth > config['max_depth']:

```

```

[L25] continue
[L26] visited.add(current.id)
[L27]
[L28] if config.get('include_parents'):
[L29] for parent in self.forest.get_parents(current):
[L30] queue.append((parent, depth + 1))
[L31]
[L32] if config.get('include_children'):
[L33] for child in self.forest.get_children(current):
[L34] queue.append((child, depth + 1))
[L35]
[L36] if config.get('include_crosslinks'):
[L37] for linked in self.forest.get_crosslinks(current):
[L38] queue.append((linked, depth + 1))
[L39]
[L40] return {self.forest.get_node(nid) for nid in visited}
[L41]
[L42] def assemble_context(self, selected_nodes, intent):
[L43] """Assemble selected nodes into ordered context string.
[L44]
[L45] Order: definitions first, then core concepts, then related evidence.
[L46] Each node rendered as: [NODE_ID] Concept: Definition\nEvidence: ...
[L47] """
[L48] # Separate by role
[L49] definitions = [n for n in selected_nodes if n.definitions]
[L50] core = [n for n in selected_nodes if n not in definitions]
[L51]
[L52] # Sort within each group by confidence (highest first)
[L53] definitions.sort(key=lambda n: n.confidence, reverse=True)
[L54] core.sort(key=lambda n: n.confidence, reverse=True)
[L55]
[L56] parts = []
[L57] for node in definitions + core:
[L58] block = f"[{node.node_id}] {node.canonical_name}"
[L59] if node.definitions:
[L60] block += f"\nDefinition: {node.definitions[0]}"
[L61] if node.evidence:
[L62] block += f"\nEvidence: {node.evidence[0].text}"
[L63] block += f"\nSource: {node.evidence[0].source_doc if node.evidence else 'N/A'}, Page {node.evidence[0].page if
node.evidence else 'N/A'}"
[L64] parts.append(block)
[L65]
[L66] return "\n\n--\n\n".join(parts)
[L67]
[L68] def _get_path_edges(self, selected_nodes):
[L69] """Get traversal path edges for Layer 8 feedback."""

```

```

[L70] edges = []
[L71] for node in selected_nodes:
[L72] current = node
[L73] while current.parent:
[L74] edge = self.forest.get_edge(current.parent, current.id)
[L75] if edge:
[L76] edges.append(edge)
[L77] current = self.forest.get_node(current.parent)
[L78] return edges
[L79]
[L80] def classify_complexity(intent: str) -> str:
[L81] """Map query intent to traversal complexity."""
[L82] mapping = {
[L83] 'DEFINITION': 'SIMPLE',
[L84] 'COMPARISON': 'MODERATE',
[L85] 'EXPLANATION': 'MODERATE',
[L86] 'MULTI_HOP': 'COMPLEX',
[L87] 'SYNTHESIS': 'COMPLEX',
[L88] }
[L89] return mapping.get(intent, 'MODERATE')
[L90]

```

```

[L91]
[L92] ---
[L93]
[L94] ### 7.8 Layer 7: Generation Engine
[L95]
[L96] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims from the res
[L97]
[L98] **Input:** `RetrievalResult` from Layer 6
[L99] **Output:** `GenerationResult` with verified answer, citations, and confidence
[L100]
[L101] #### LLM Configuration
[L102]
[L103]

```

python

```

[L104] GENERATION_CONFIG = {
[L105] 'model': 'gpt-4o-mini', # Primary model (configurable)
[L106] 'fallback_model': 'llama-3.1-8b', # Fallback for cost/availability
[L107] 'temperature': 0.3, # Low temperature for factual answers
[L108] 'max_output_tokens': 2048,
[L109] 'top_p': 0.9,
[L110] 'timeout_seconds': 30,
[L111] 'max_retries': 3,
[L112] 'response_format': 'json', # Force structured output
[L113] }
[L114]

```

```
[L115]
[L116] ##### Prompt Template
[L117]
[L118]
```

python

[L119] SYSTEM\_PROMPT = """"You are a knowledge-grounded question answering system.

[L120] RULES:

[L121] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.

[L122] 2. For each claim in your answer, cite the source using [NODE\_ID] format.

[L123] 3. If evidence is insufficient, say "Insufficient evidence to answer."

[L124] 4. If evidence conflicts, present both views with citations.

[L125] 5. Structure your response as JSON with the schema below.

[L126]

[L127] REQUIRED OUTPUT FORMAT:

[L128] {

[L129] "answer": "Your complete answer text with [NODE\_ID] citations inline.",

[L130] "claims": [

[L131] {"text": "Individual factual claim", "evidence\_ids": ["NODE\_ID1"], "confidence": 0.0-1.0}

[L132] ],

---

DEEP RESEARCH 20/08/2026, 15:26:28

Citation Marker: `filecite`turn24file0

[L1] metadata:

[L2] ref\_id: turn21file8

[L3] file\_id: file\_000000003c8081fab7e320a5ee7ca1ad

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(5).md

[L6] mode: chunk\_context

[L7] lines\_shown: 141-141

[L8] total\_file\_lines: 141

[L9] has\_more: False

[L10] content:

[L11] for child in self.forest.get\_children(current):

---

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite`turn25file0

[L1] metadata:

[L2] query\_result\_index: 0

[L3] file\_id: file\_000000004a3881fab7e320a5ee7ca1ad

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report.md

[L6] mime\_type: text/markdown

[L7] surface: library

[L8] score: 0.027417840375586856

[L9] document\_chunk\_id: file\_000000004a3881fab7e320a5ee7ca1ad--1--5

```

[L10] snippet:
[L11] # FIXED: Tree Knapsack (precedence-constrained)
[L12] + def tree_knapsack(root, budget, forest):
[L13] + """Select nodes maximizing utility, but a child can only be
[L14] + selected if its parent is also selected."""
[L15] + dp = {} # dp[(node, budget)] = (max_utility, selected_set)
[L16] +
[L17] + def solve(node, remaining_budget):
[L18] + if (node.id, remaining_budget) in dp:
[L19] + return dp[(node.id, remaining_budget)]
[L20] +
[L21] + # Option 1: Don't include this node (or any descendants)
[L22] + best = (0, set())
[L23] +
[L24] + # Option 2: Include this node, then greedily pick children
[L25] + if remaining_budget >= node.token_cost:
[L26] + child_budget = remaining_budget - node.token_cost
[L27] + total_util = node.utility
[L28] + selected = {node.id}
[L29] +
[L30] + # Greedily allocate remaining budget to children
[L31] + children = sorted(forest.children(node),
[L32] + key=lambda c: c.utility / max(c.token_cost, 1),
[L33] + reverse=True)
[L34] + for child in children:
[L35] + if child_budget >= child.token_cost:
[L36] + cu, cs = solve(child, child_budget)
[L37] + total_util += cu
[L38] + selected |= cs
[L39] + child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L40] +
[L41] + if total_util > best[0]:
[L42] + best = (total_util, selected)
[L43] +
[L44] + dp[(node.id, remaining_budget)] = best
[L45] + return best
[L46] +
[L47] + return solve(root, budget)
[L48]

```

```

[L49]
[L50] ---
[L51]
[L52] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)
[L53]
[L54] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q` i
[L55]
[L56] **Fix:** Estimate abstraction level from textual features before placement:
[L57]
[L58]

```

python

```

[L59] def estimate_abstraction_level(node):
[L60] """Heuristic: more abstract concepts have shorter names,
[L61] broader definitions, and appear in more documents."""
[L62] score = 0.0
[L63] score += 1.0 / (len(node.name.split()) + 1) # Shorter = more abstract
[L64] score += len(node.sources) 0.1 # More sources = more general
[L65] score += len(node.children_hints) 0.05 # More sub-concepts = more abstract
[L66] if any(kw in node.definition.lower()
[L67] for kw in ['general', 'category', 'type', 'class', 'family']):
[L68] score += 0.3
[L69] return score
[L70]

```

```

[L71]
[L72] ---
[L73]
[L74] ### 3.6 CRITICAL: $O(N^2)$ Canonicalization (PDF 2)
[L75]
[L76] **The Bug:** Alias merging compares every node pair: `for A in nodes: for B in nodes: → $O(N^2)$. For mi
[L77]
[L78] **Fix:** Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L79]
[L80]

```

python

```

[L81] from datasketch import MinHashLSH

```

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: fileciteturn25file1

```

[L82] metadata:
[L83] query_result_index: 1
[L84] file_id: file_00000000a4bc81faba1d49e4f811b447
[L85] version_id: 1
[L86] name: SHIA_RAG_Complete_Report(1).md
[L87] mime_type: text/markdown
[L88] surface: library
[L89] score: 0.027417840375586856
[L90] document_chunk_id: file_00000000a4bc81faba1d49e4f811b447--1--5
[L91] snippet:

```

```

[L92] # FIXED: Tree Knapsack (precedence-constrained)
[L93] + def tree_knapsack(root, budget, forest):
[L94] + """Select nodes maximizing utility, but a child can only be
[L95] + selected if its parent is also selected."""
[L96] + dp = {} # dp[(node, budget)] = (max_utility, selected_set)
[L97] +
[L98] + def solve(node, remaining_budget):
[L99] + if (node.id, remaining_budget) in dp:
[L100] + return dp[(node.id, remaining_budget)]
[L101] +
[L102] + # Option 1: Don't include this node (or any descendants)
[L103] + best = (0, set())
[L104] +
[L105] + # Option 2: Include this node, then greedily pick children
[L106] + if remaining_budget >= node.token_cost:
[L107] + child_budget = remaining_budget - node.token_cost
[L108] + total_util = node.utility
[L109] + selected = {node.id}
[L110] +
[L111] + # Greedily allocate remaining budget to children
[L112] + children = sorted(forest.children(node),
[L113] + key=lambda c: c.utility / max(c.token_cost, 1),
[L114] + reverse=True)
[L115] + for child in children:
[L116] + if child_budget >= child.token_cost:
[L117] + cu, cs = solve(child, child_budget)
[L118] + total_util += cu
[L119] + selected |= cs
[L120] + child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L121] +
[L122] + if total_util > best[0]:
[L123] + best = (total_util, selected)
[L124] +
[L125] + dp[(node.id, remaining_budget)] = best
[L126] + return best
[L127] +
[L128] + return solve(root, budget)
[L129]

```

```

[L130]
[L131] ---
[L132]
[L133] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)
[L134]
[L135] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q`
[L136]
[L137] **Fix:** Estimate abstraction level from textual features before placement:
[L138]
[L139]

```

python

```

[L140] def estimate_abstraction_level(node):
[L141] """Heuristic: more abstract concepts have shorter names,
[L142] broader definitions, and appear in more documents."""
[L143] score = 0.0
[L144] score += 1.0 / (len(node.name.split()) + 1) # Shorter = more abstract
[L145] score += len(node.sources) * 0.1 # More sources = more general
[L146] score += len(node.children_hints) * 0.05 # More sub-concepts = more abstract
[L147] if any(kw in node.definition.lower()
[L148] for kw in ['general', 'category', 'type', 'class', 'family']):
[L149] score += 0.3
[L150] return score
[L151]

```

```

[L152]
[L153] ---
[L154]
[L155] ### 3.6 CRITICAL: $O(N^2)$ Canonicalization (PDF 2)
[L156]
[L157] **The Bug:** Alias merging compares every node pair: `for A in nodes: for B in nodes:` $\rightarrow O(N^2)$. For n
[L158]
[L159] **Fix:** Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L160]
[L161]

```

python

```

[L162] from datasketch import MinHashLSH

```

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite`turn25file2

```

[L163] metadata:
[L164] query_result_index: 2
[L165] file_id: file_00000000b004820b99e50da306d8422a
[L166] version_id: 1
[L167] name: SHIA_RAG_Complete_Report.md
[L168] mime_type: text/markdown
[L169] surface: library
[L170] score: 0.027417840375586856
[L171] document_chunk_id: file_00000000b004820b99e50da306d8422a--1--5
[L172] snippet:
[L173] # FIXED: Tree Knapsack (precedence-constrained)
[L174] + def tree_knapsack(root, budget, forest):
[L175] + """Select nodes maximizing utility, but a child can only be
[L176] + selected if its parent is also selected."""
[L177] + dp = {} # dp[(node, budget)] = (max_utility, selected_set)
[L178] +
[L179] + def solve(node, remaining_budget):
[L180] + if (node.id, remaining_budget) in dp:

```



```

[L181] + return dp[(node.id, remaining_budget)]
[L182] +
[L183] + # Option 1: Don't include this node (or any descendants)
[L184] + best = (0, set())
[L185] +
[L186] + # Option 2: Include this node, then greedily pick children
[L187] + if remaining_budget >= node.token_cost:
[L188] + child_budget = remaining_budget - node.token_cost
[L189] + total_util = node.utility
[L190] + selected = {node.id}
[L191] +
[L192] + # Greedily allocate remaining budget to children
[L193] + children = sorted(forest.children(node),
[L194] + key=lambda c: c.utility / max(c.token_cost, 1),
[L195] + reverse=True)
[L196] + for child in children:
[L197] + if child_budget >= child.token_cost:
[L198] + cu, cs = solve(child, child_budget)
[L199] + total_util += cu
[L200] + selected |= cs
[L201] + child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L202] +
[L203] + if total_util > best[0]:
[L204] + best = (total_util, selected)
[L205] +
[L206] + dp[(node.id, remaining_budget)] = best
[L207] + return best
[L208] +
[L209] + return solve(root, budget)
[L210]

```

```

[L211]
[L212] ---
[L213]
[L214] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)
[L215]
[L216] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q`
[L217]
[L218] **Fix:** Estimate abstraction level from textual features before placement:
[L219]
[L220]

```

python

```

[L221] def estimate_abstraction_level(node):
[L222] """Heuristic: more abstract concepts have shorter names,
[L223] broader definitions, and appear in more documents."""
[L224] score = 0.0
[L225] score += 1.0 / (len(node.name.split()) + 1) # Shorter = more abstract
[L226] score += len(node.sources) * 0.1 # More sources = more general

```

```
[L227] score += len(node.children_hints) 0.05 # More sub-concepts = more abstract
[L228] if any(kw in node.definition.lower()
[L229] for kw in ['general', 'category', 'type', 'class', 'family']):
[L230] score += 0.3
[L231] return score
[L232]
```

```
[L233]
[L234] ---
[L235]
[L236] ### 3.6 CRITICAL: $O(N^2)$ Canonicalization (PDF 2)
[L237]
[L238] **The Bug:** Alias merging compares every node pair: `for A in nodes: for B in nodes:` $\rightarrow O(N^2)$. For n
[L239]
[L240] **Fix:** Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L241]
[L242]
```

python

```
[L243] from datasketch import MinHashLSH
```

---

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite[turn25file3]`

```
[L244] metadata:
[L245] query_result_index: 3
[L246] file_id: file_00000000bb588211b69ffa1cf13ab86d
[L247] version_id: 1
[L248] name: SHIA_RAG_Complete_Report(2).md
[L249] mime_type: text/markdown
[L250] surface: library
[L251] score: 0.02684684684684685
[L252] document_chunk_id: file_00000000bb588211b69ffa1cf13ab86d--1--5
[L253] snippet:
[L254] # FIXED: Tree Knapsack (precedence-constrained)
[L255] + def tree_knapsack(root, budget, forest):
[L256] + """Select nodes maximizing utility, but a child can only be
[L257] + selected if its parent is also selected."""
[L258] + dp = {} # dp[(node, budget)] = (max_utility, selected_set)
[L259] +
[L260] + def solve(node, remaining_budget):
[L261] + if (node.id, remaining_budget) in dp:
[L262] + return dp[(node.id, remaining_budget)]
[L263] +
[L264] + # Option 1: Don't include this node (or any descendants)
[L265] + best = (0, set())
[L266] +
[L267] + # Option 2: Include this node, then greedily pick children
[L268] + if remaining_budget >= node.token_cost:
```

```

[L269] + child_budget = remaining_budget - node.token_cost
[L270] + total_util = node.utility
[L271] + selected = {node.id}
[L272] +
[L273] + # Greedily allocate remaining budget to children
[L274] + children = sorted(forest.children(node),
[L275] + key=lambda c: c.utility / max(c.token_cost, 1),
[L276] + reverse=True)
[L277] + for child in children:
[L278] + if child_budget >= child.token_cost:
[L279] + cu, cs = solve(child, child_budget)
[L280] + total_util += cu
[L281] + selected |= cs
[L282] + child_budget -= sum(forest.get_node(n).token_cost for n in cs)
[L283] +
[L284] + if total_util > best[0]:
[L285] + best = (total_util, selected)
[L286] +
[L287] + dp[(node.id, remaining_budget)] = best
[L288] + return best
[L289] +
[L290] + return solve(root, budget)
[L291]

```

```

[L292]
[L293] ---
[L294]
[L295] ### 3.5 CRITICAL: Undefined `node_level` for Unplaced Nodes (PDFs 1, 3)
[L296]
[L297] **The Bug:** CPG++ and PRE use `node_level(q)` to filter candidates by abstraction distance. But `q`
[L298]
[L299] **Fix:** Estimate abstraction level from textual features before placement:
[L300]
[L301]

```

python

```

[L302] def estimate_abstraction_level(node):
[L303] """Heuristic: more abstract concepts have shorter names,
[L304] broader definitions, and appear in more documents."""
[L305] score = 0.0
[L306] score += 1.0 / (len(node.name.split()) + 1) # Shorter = more abstract
[L307] score += len(node.sources) * 0.1 # More sources = more general
[L308] score += len(node.children_hints) * 0.05 # More sub-concepts = more abstract
[L309] if any(kw in node.definition.lower()
[L310] for kw in ['general', 'category', 'type', 'class', 'family']):
[L311] score += 0.3
[L312] return score
[L313]

```

```

[L314]
[L315] ---
[L316]
[L317] ### 3.6 CRITICAL: $O(N^2)$ Canonicalization (PDF 2)
[L318]
[L319] The Bug: Alias merging compares every node pair: `for A in nodes: for B in nodes:` $\rightarrow O(N^2)$. For n
[L320]
[L321] Fix: Use Locality-Sensitive Hashing (LSH) for approximate nearest neighbors:
[L322]
[L323]

```

python

```
[L324] from datasketch import MinHashLSH
```

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite[turn25file4]`

[L325] metadata:

[L326] query\_result\_index: 4

[L327] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L328] version\_id: 1

[L329] name: SHIA\_RAG\_Final\_Report.md

[L330] mime\_type: text/markdown

[L331] surface: library

[L332] score: 0.025849212924606464

[L333] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--2

[L334] snippet:

[L335] | Problem | Description | Impact |

[L336] |-----|-----|-----|

[L337] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L338] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L339] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L340] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,  $A \rightarrow B \rightarrow C$ ) |

[L341] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L342]

[L343] ### 1.2 Proposed Solution: SHIA-RAG

[L344]

[L345] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L346]

[L347]

[L348] Traditional RAG: Document → [Chunk<sub>1</sub>, Chunk<sub>2</sub>, ..., Chunk<sub>n</sub>] → Top-k → LLM  
 [L349] SHIA-RAG: Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM  
 [L350]

[L351]

[L352] The system:

[L353] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L354] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L355] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L356] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L357] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L358] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L359]

[L360] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L361]

[L362] ### 1.3 Objectives

[L363]

[L364] 1. Design a complete 8-layer architecture for hierarchical RAG

[L365] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L366] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L367] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L368] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L369] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L370]

[L371] ### 1.4 Contributions

[L372]

[L373] This work makes five novel contributions to the RAG literature:

[L374]

[L375] | # | Contribution | Novelty Level |

[L376] |---|-----|-----|

[L377] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L378] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L379] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L380] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L381] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L382]

[L383] ---

[L384]

[L385] ## 2. Literature Review

[L386]

[L387] ### 2.1 Related Work

[L388]

[L389] ##### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: [filecite]turn25file5[

[L390] metadata:

[L391] query\_result\_index: 5

[L392] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L393] version\_id: 1

[L394] name: SHIA\_RAG\_Final\_Report(4).md

[L395] mime\_type: text/markdown

[L396] surface: conversation

[L397] score: 0.02571241401899771

[L398] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--2

[L399] snippet:

[L400] | Problem | Description | Impact |

[L401] |-----|-----|-----|

[L402] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L403] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L404] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L405] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g., A → B → C) |

[L406] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L407]

[L408] ### 1.2 Proposed Solution: SHIA-RAG

[L409]

[L410] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L411]

[L412]

|                         |                                                                                               |
|-------------------------|-----------------------------------------------------------------------------------------------|
| [L413] Traditional RAG: | Document → [Chunk <sub>1</sub> , Chunk <sub>2</sub> , ..., Chunk <sub>n</sub> ] → Top-k → LLM |
| [L414] SHIA-RAG:        | Document → Knowledge Forest (concepts + relations) → Adaptive Traversal → LLM                 |
| [L415]                  |                                                                                               |

[L416]

[L417] The system:

[L418] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L419] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L420] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L421] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence

constraints

[L422] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L423] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L424]

[L425] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L426]

[L427] ### 1.3 Objectives

[L428]

[L429] 1. Design a complete 8-layer architecture for hierarchical RAG

[L430] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L431] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L432] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L433] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L434] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L435]

[L436] ### 1.4 Contributions

[L437]

[L438] This work makes five novel contributions to the RAG literature:

[L439]

[L440] | # | Contribution | Novelty Level |

[L441] |---|-----|-----|

[L442] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L443] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L444] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L445] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L446] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L447]

[L448] ---

[L449]

[L450] ## 2. Literature Review

[L451]

[L452] ### 2.1 Related Work

[L453]

[L454] ##### 2.1.1 Flat-Chunk RAG Systems

---

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite[turn25file6]`

[L455] metadata:

[L456] query\_result\_index: 6

[L457] file\_id: file\_000000003218820b918ee3d86ec345ae

[L458] version\_id: 1

[L459] name: SHIA\_RAG\_Final\_Report(1).md

[L460] mime\_type: text/markdown

[L461] surface: library

[L462] score: 0.02557875991581674

[L463] document\_chunk\_id: file\_000000003218820b918ee3d86ec345ae--1--2

[L464] snippet:

[L465] | Problem | Description | Impact |

[L466] |-----|-----|-----|

[L467] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L468] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L469] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L470] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,  $A \rightarrow B \rightarrow C$ ) |

[L471] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L472]

[L473] ### 1.2 Proposed Solution: SHIA-RAG

[L474]

[L475] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L476]

[L477]

[L478] Traditional RAG: Document  $\rightarrow$  [Chunk<sub>1</sub>, Chunk<sub>2</sub>, ..., Chunk<sub>n</sub>]  $\rightarrow$  Top-k  $\rightarrow$  LLM

[L479] SHIA-RAG: Document  $\rightarrow$  Knowledge Forest (concepts + relations)  $\rightarrow$  Adaptive Traversal  $\rightarrow$  LLM

[L480]

[L481]

[L482] The system:

[L483] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L484] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L485] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L486] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L487] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L488] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L489]

[L490] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science  $\rightarrow$  Operating Systems  $\rightarrow$  Scheduling  $\rightarrow$  Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L491]

[L492] ### 1.3 Objectives

[L493]

[L494] 1. Design a complete 8-layer architecture for hierarchical RAG

[L495] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)



[L496] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L497] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L498] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L499] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L500]

[L501] ### 1.4 Contributions

[L502]

[L503] This work makes five novel contributions to the RAG literature:

[L504]

[L505] | # | Contribution | Novelty Level |

[L506] |---|-----|-----|

[L507] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L508] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L509] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L510] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L511] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L512]

[L513] ---

[L514]

[L515] ## 2. Literature Review

[L516]

[L517] ### 2.1 Related Work

[L518]

[L519] #### 2.1.1 Flat-Chunk RAG Systems

DEEP RESEARCH 20/08/2026, 15:26:34

Citation Marker: `filecite[turn25file7]`

[L520] metadata:

[L521] query\_result\_index: 7

[L522] file\_id: file\_000000004bc481fdbf9e949ec3b7544b

[L523] version\_id: 1

[L524] name: SHIA\_RAG\_Final\_Report(2).md

[L525] mime\_type: text/markdown

[L526] surface: library

[L527] score: 0.02557875991581674

[L528] document\_chunk\_id: file\_000000004bc481fdbf9e949ec3b7544b--1--2

[L529] snippet:

[L530] | Problem | Description | Impact |

[L531] |-----|-----|-----|

[L532] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L533] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L534] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L535] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,  $A \rightarrow B \rightarrow C$ ) |

[L536] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L537]

[L538] ### 1.2 Proposed Solution: SHIA-RAG

[L539]

[L540] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L541]

[L542]

|                         |                                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| [L543] Traditional RAG: | Document $\rightarrow$ [Chunk <sub>1</sub> , Chunk <sub>2</sub> , ..., Chunk <sub>n</sub> ] $\rightarrow$ Top-k $\rightarrow$ LLM |
| [L544] SHIA-RAG:        | Document $\rightarrow$ Knowledge Forest (concepts + relations) $\rightarrow$ Adaptive Traversal $\rightarrow$ LLM                 |
| [L545]                  |                                                                                                                                   |

[L546]

[L547] The system:

[L548] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L549] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L550] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L551] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L552] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L553] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L554]

[L555] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science  $\rightarrow$  Operating Systems  $\rightarrow$  Scheduling  $\rightarrow$  Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L556]

[L557] ### 1.3 Objectives

[L558]

[L559] 1. Design a complete 8-layer architecture for hierarchical RAG

[L560] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L561] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L562] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L563] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L564] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L565]

[L566] ### 1.4 Contributions

[L567]

[L568] This work makes five novel contributions to the RAG literature:

[L569]

[L570] | # | Contribution | Novelty Level |

[L571] |---|-----|-----|

[L572] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L573] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |  
[L574] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |  
[L575] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |  
[L576] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |  
[L577]  
[L578] ---  
[L579]  
[L580] ## 2. Literature Review  
[L581]  
[L582] ### 2.1 Related Work  
[L583]  
[L584] #### 2.1.1 Flat-Chunk RAG Systems

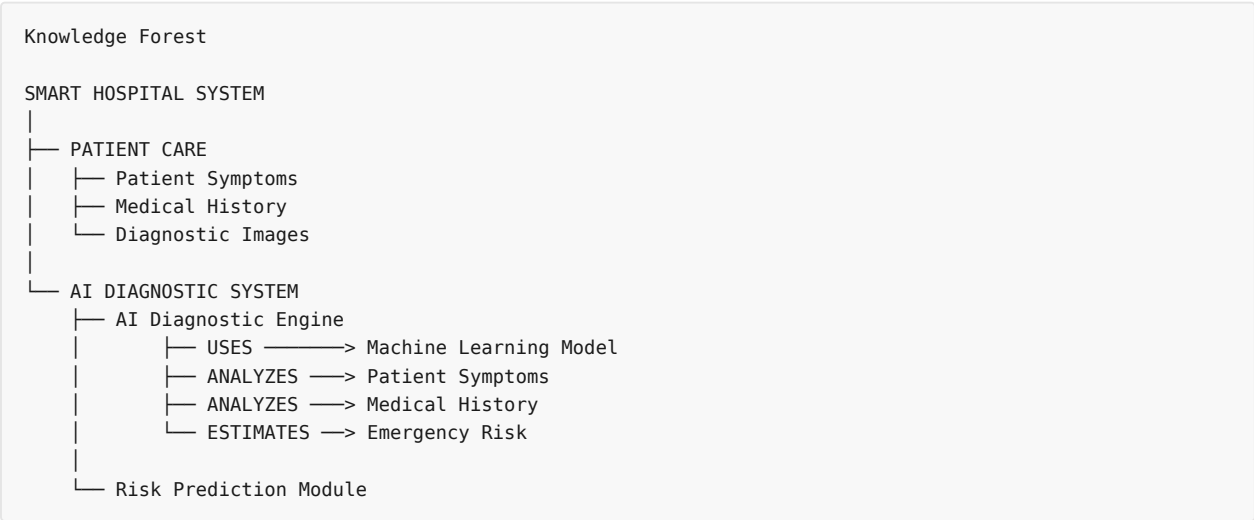
CHATGPT 20/08/2026, 15:26:33

Layer 6 — Retrieval Engine

Ab SHIA-RAG ka Knowledge Forest actual query ke liye use hona start hota hai

Hum same Smart Hospital example continue karenge.

Layer 5 tak humne ye banaya tha:



Layer 6 ka kaam hai:

*User ke query ke according Knowledge Forest me relevant knowledge dhoondhna, hierarchy traverse karna, token budget ke andar best context select karna, aur Layer 7 ko structured retrieval result dena.*

Report ke according Layer 6 ka output RetrievalResult hota hai, jisme selected nodes, assembled context, traversal path, query complexity, intent, seed nodes aur token count store hote hain. filecite turn22file0 L21-L30

1. Sabse pehle Layer 6 ka complete mental model

```
USER QUERY
↓
[Step 0] Cache Check
↓
[Step 1] Query Intent + Complexity
↓
[Step 2] Query Embedding + Seed Nodes
↓
[Step 3] Adaptive Depth Configuration
↓
[Step 4] BFS Forest Traversal
↓
Candidate Nodes
↓
[Step 5] Multi-Tree Knapsack
↓
Selected Nodes within Token Budget
↓
[Step 6] Context Assembly
↓
RetrievalResult
↓
LAYER 7
```

Ye exact retrieval flow report ke `retrieve()` function me defined hai. `filecite\turn22file0\L40-L91`

## 2. Layer 6 ka input exactly kya hai?

Layer 6 ke paas multiple inputs hain.

### Input 1: User Query

Example ke liye hum ek **bada complex query** lenge:

*"How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?"*

Is query ko hum poore Layer 6 me use karenge.

### Input 2: Knowledge Forest

Ye Layer 5 se aata hai:

Knowledge Forest

Isme:

- hierarchical parent-child edges
- semantic cross-links
- node confidence
- definitions
- evidence
- embeddings
- token cost

available honi chahiye.

### Input 3: Vector Index

Query ko directly har node se compare karna inefficient hoga.

Isliye:

```
Query Embedding
 ↓
Vector Index
 ↓
Top-k Seed Nodes
```

Report me `vector_index.search(query_emb, k=self.seed_k)` use hua hai aur default `seed_k = 5` hai.  
filecite[turn22file0L32-L53]

---

### Input 4: Cache Manager

Report retrieval results ko cache se retrieve aur baad me store karti hai, with `ttl=3600`. filecite[turn22file0L40-L45] filecite[turn22file0L88-L90]

---

### Input 5: Configuration

Example:

```
seed_k = 5
token_budget = configured budget
max_depth = complexity dependent
```

## 3. Layer 6 ka output exactly kya hota hai?

```
RetrievalResult:
 selected_nodes
 context_text
 traversal_path_edges
 query_complexity
 query_intent
 seed_node_ids
 token_count
```

Report isko explicitly Layer 7 ka input bolti hai. filecite[turn22file0L21-L30]

Hamare example me final output conceptual form me:

```

RetrievalResult
├── selected_nodes
│ ├── AI Diagnostic Engine
│ ├── AI Diagnostic System
│ ├── Patient Symptoms
│ ├── Medical History
│ ├── Machine Learning Model
│ └── Emergency Risk
├── query_intent
│ └── MULTI_HOP
├── query_complexity
│ └── COMPLEX
├── seed_node_ids
│ └── [...]
├── traversal_path_edges
│ └── all hierarchy paths used
├── context_text
│ └── formatted evidence
└── token_count
 └── final context tokens

```

#### 4. Step 0 — Cache Check

Sabse pehle:

```

User Query
 ↓
hash_query(query)
 ↓
cache_key
 ↓
Cache me search

```

Report ka logic:

```

cache_key = hash_query(query)
cached = cache.get_query_result(cache_key)

if cached:
 return cached

```

filecite\turn22file0\L40-L45

#### Hamara example

Query:

```

How does the AI Diagnostic Engine analyze patient symptoms
and medical history to estimate emergency risk, and what role
does the Machine Learning Model play?

```

Suppose hash:

```
query_hash = X8F72...
```

System check karega:

```
Redis Cache
|
|— X8F72...? YES → return cached result
|
|— NO → continue retrieval
```

## Cache hit case

Agar same query pehle already process ho chuki hai:

```
Query
↓
Cache HIT
↓
Previous RetrievalResult
↓
Directly Layer 7
```

No:

- query embedding
- vector search
- traversal
- Tree Knapsack

required.

## Cache miss

```
Query
↓
Cache MISS
↓
Continue to Step 1
```

## 5. Step 0 me possible errors

### Error A — Cache unavailable

Report ke shown Layer 6 code me Redis/cache failure ke liye explicit:

```
try/except fallback policy
```

**Not specified.**

Meaning, shown implementation me ye exact behavior defined nahi hai:

```
Redis down
↓
What exactly happens?
```

Suggestion (Not from the report)

Safer implementation:

```
try cache
↓
if failure:
 log warning
 continue without cache
```

Cache failure ki wajah se complete retrieval fail nahi hona chahiye.

Error B — Query hash issue

Exact collision-handling strategy:

```
Not specified in the shown Layer 6 section.
```

Agar do different queries theoretically same key produce karein, wrong cached result ka risk ho sakta hai, depending on hash strategy.

6. Step 1 — Query Intent Detection

Ab system query ko samajhne ki try karta hai.

Report intents:

```
DEFINITION
COMPARISON
EXPLANATION
MULTI_HOP
SYNTHESIS
```

Phir intent ko complexity me map kiya jata hai. `filecite{turn23file0L80-L89}`

7. Hamari query ka intent kya hai?

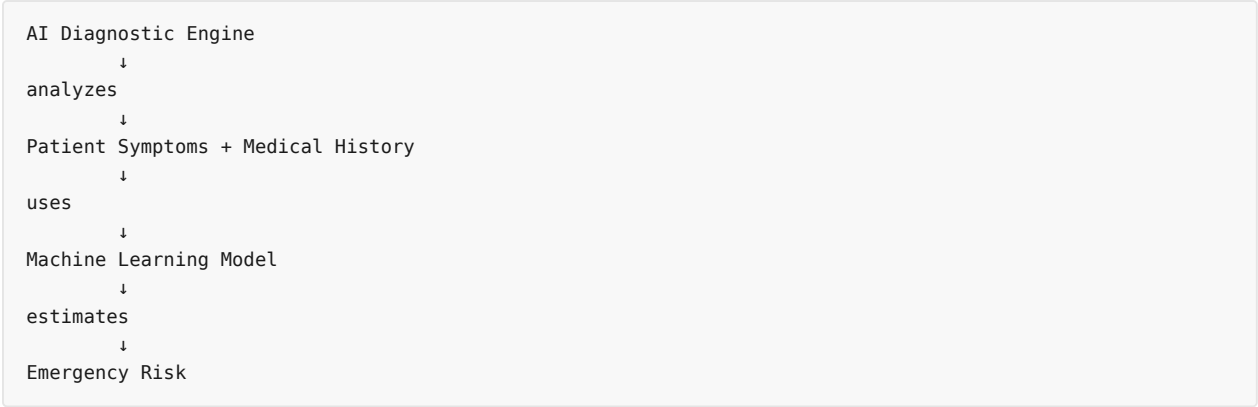
Query:

```
"How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?"
```

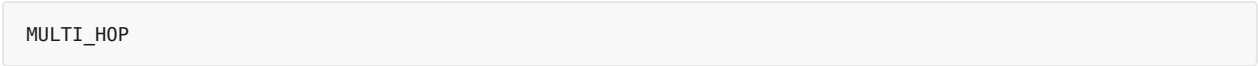
Isme sirf ek definition nahi puchi gayi.

Reasoning chain hai:

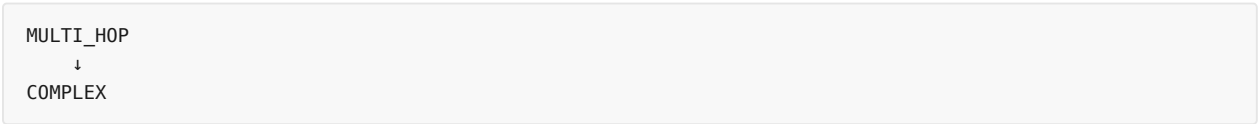




Isliye suitable intent:



Then report mapping:



Report me MULTI\_HOP → COMPLEX defined hai. [filecite\turn23file0\L80-L89](#)

8. Intent → Complexity mapping

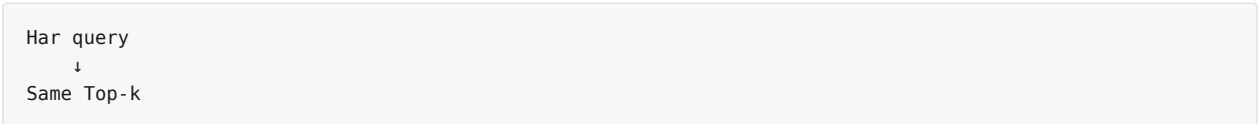
| Intent   Complexity   Traversal Depth |
|---------------------------------------|
| --- --- ---                           |
| DEFINITION   SIMPLE   1               |
| COMPARISON   MODERATE   2             |
| EXPLANATION   MODERATE   2            |
| MULTI_HOP   COMPLEX   3               |
| SYNTHESIS   COMPLEX   3               |

Report me examples bhi diye gaye hain, jaise definition query ko depth 1, comparison/explanation ko depth 2, aur multi-hop/synthesis ko depth 3 map kiya gaya hai. [filecite\turn22file0\L13-L16](#) [filecite\turn23file0\L80-L89](#)

9. Important: Complexity directly retrieval behavior change karti hai

Ye Layer 6 ka ek major point hai.

Traditional fixed retrieval:



SHIA-RAG:

```

Query type
↓
Required reasoning complexity
↓
Traversal strategy
↓
Different amount/direction of knowledge

```

Example:

### Query 1

"What is Emergency Risk?"

```

DEFINITION
↓
SIMPLE
↓
Depth = 1

```

### Query 2

"How does AI Diagnostic Engine estimate Emergency Risk?"

```

EXPLANATION
↓
MODERATE
↓
Depth = 2

```

### Query 3 — Hamari query

```

AI Engine
↓
Symptoms + History
↓
ML Model
↓
Emergency Risk

```

```

MULTI_HOP
↓
COMPLEX
↓
Depth = 3

```

## 10. Step 1 me possible mistake

### Wrong intent classification

Suppose hamari complex query classify ho gayi:

```

EXPLANATION

```

instead of:

```
MULTI_HOP
```

Then:

```
MODERATE
Depth = 2
Cross-links = OFF
```

But hamari query ko Machine Learning Model tak semantic cross-link se jaana useful ho sakta hai.

So:

```
Wrong Intent
 ↓
Wrong Complexity
 ↓
Wrong Traversal Configuration
 ↓
Relevant Knowledge Miss
```

Ye error propagation hai.

## 11. Step 2 — Query Embedding

Ab raw query ko vector me convert kiya jata hai.

Report:

```
query_emb = embed_query(query)
```

Aur comment:

```
Asymmetric embedding model
```

filecite\turn22file0\L51-L53

Conceptually:

```
"How does AI Diagnostic Engine..."
 ↓
Embedding Model
 ↓
[0.21, -0.34, 0.88, ...]
```

## 12. Query embedding ka purpose kya hai?

Query ko Knowledge Forest ke har node se direct text matching nahi karna.

Instead:

```
Query Vector
 ↓
Vector Space
 ↓
Semantically nearest Knowledge Nodes
```

## 13. Step 2 — Seed Node Retrieval

Report:

```
seed_nodes =
vector_index.search(
 query_emb,
 k = self.seed_k
)
```

Default:

```
seed_k = 5
```

filecite\turn22file0\L32-L53

### Hamare example me

Query:

```
AI Diagnostic Engine
+ Patient Symptoms
+ Medical History
+ Emergency Risk
+ Machine Learning Model
```

Suppose vector search result:

| Rank | Seed Node              | Similarity |
|------|------------------------|------------|
| 1    | AI Diagnostic Engine   | 0.94       |
| 2    | Emergency Risk         | 0.89       |
| 3    | Machine Learning Model | 0.86       |
| 4    | Patient Symptoms       | 0.82       |
| 5    | Medical History        | 0.79       |

These are now:

### Seed Nodes

```
Seeds =
{
 AI Diagnostic Engine,
 Emergency Risk,
 Machine Learning Model,
 Patient Symptoms,
 Medical History
}
```

Important:

*Ye final context nahi hai.*

Ye sirf:

```
STARTING POINTS
```

hain.

## 14. Seed node aur final selected node me difference

Bahut important.

Seed Nodes

= vector search ne directly relevant bataye.

Candidate Nodes

= seeds se traversal ke baad discovered nodes.

Selected Nodes

= Tree Knapsack ne budget ke andar final choose kiye.

Flow:

```

Vector Search
 ↓
Seed Nodes
 ↓
Traversal
 ↓
Candidate Nodes
 ↓
Tree Knapsack
 ↓
Selected Nodes

```

## 15. Step 2 me possible errors

### Error A — Correct node Top-5 me nahi aaya

Suppose actual most useful node:

AI Diagnostic System

Top-5 seeds me nahi aaya.

Koi problem nahi agar:

AI Diagnostic Engine

seed aaya aur parent traversal enabled hai.

Then:

```

AI Diagnostic Engine
 ↑
AI Diagnostic System

```

traversal se recover ho sakta hai.

**Lekin**

Agar correct concept:

- seed me nahi
- kisi reachable traversal path me nahi

then retrieval miss ho sakta hai.

## Error B — Approximate vector search false positive

Vector search semantically similar but irrelevant node return kar sakta hai:

Hospital Management Dashboard

because it is semantically close to "hospital system".

Then traversal:

```
Wrong Seed
 ↓
Wrong Neighborhood
 ↓
Token Budget Waste
```

## 16. Step 3 — Adaptive Depth Traversal

Ab complexity ke basis par traversal config choose hoti hai.

Report:

```
SIMPLE:
 max_depth = 1
 parents = True
 children = False
 crosslinks = False

MODERATE:
 max_depth = 2
 parents = True
 children = True
 crosslinks = False

COMPLEX:
 max_depth = 3
 parents = True
 children = True
 crosslinks = True
```

filecite\turn22file0\L55-L61

## 17. Hamari query ke liye traversal configuration

Hamari query:

```
MULTI_HOP
 ↓
COMPLEX
```

So:

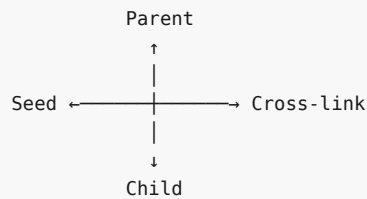
```

max_depth = 3

include_parents = True
include_children = True
include_crosslinks = True

```

Matlab system teen directions me knowledge explore kar sakta hai:



## 18. Parent traversal kyun?

Seed:

AI Diagnostic Engine

Parent:

AI Diagnostic System

Aur uska parent:

Smart Hospital System

Question answer karte time kabhi specific concept ka broader context chahiye hota hai.

Example:

AI Diagnostic Engine

samajhne ke liye:

AI Diagnostic System

useful context de sakta hai.

## 19. Child traversal kyun?

Suppose seed:

AI Diagnostic System

Children:

AI Diagnostic Engine  
Risk Prediction Module

Query:

"How does the AI Diagnostic System work?"

Sirf parent definition sufficient nahi ho sakti.

System uske internal/specific concepts bhi retrieve kar sakta hai.

---

## 20. Cross-link traversal kyun?

Ye especially Layer 5 ke CLD ka benefit hai.

Hierarchy:

```
AI Diagnostic System
 ↓
AI Diagnostic Engine
```

Lekin semantic relation:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Aur:

```
AI Diagnostic Engine
 ANALYZES
Patient Symptoms
```

Agar query multi-hop hai:

```
AI Engine
 ↓ USES
ML Model
```

cross-link highly relevant ho sakta hai.

---

## 21. SIMPLE vs MODERATE vs COMPLEX — same forest par

### SIMPLE

Query:

*"What is AI Diagnostic Engine?"*

```
AI Diagnostic Engine
 ↑
AI Diagnostic System
```

Only depth 1.

No children.

No cross-links.

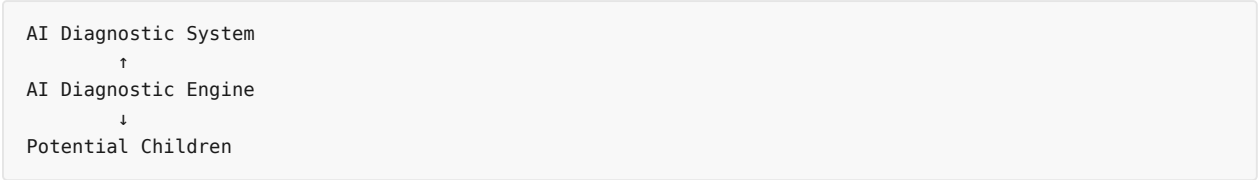
---

### MODERATE

Query:

*"How does AI Diagnostic Engine work?"*



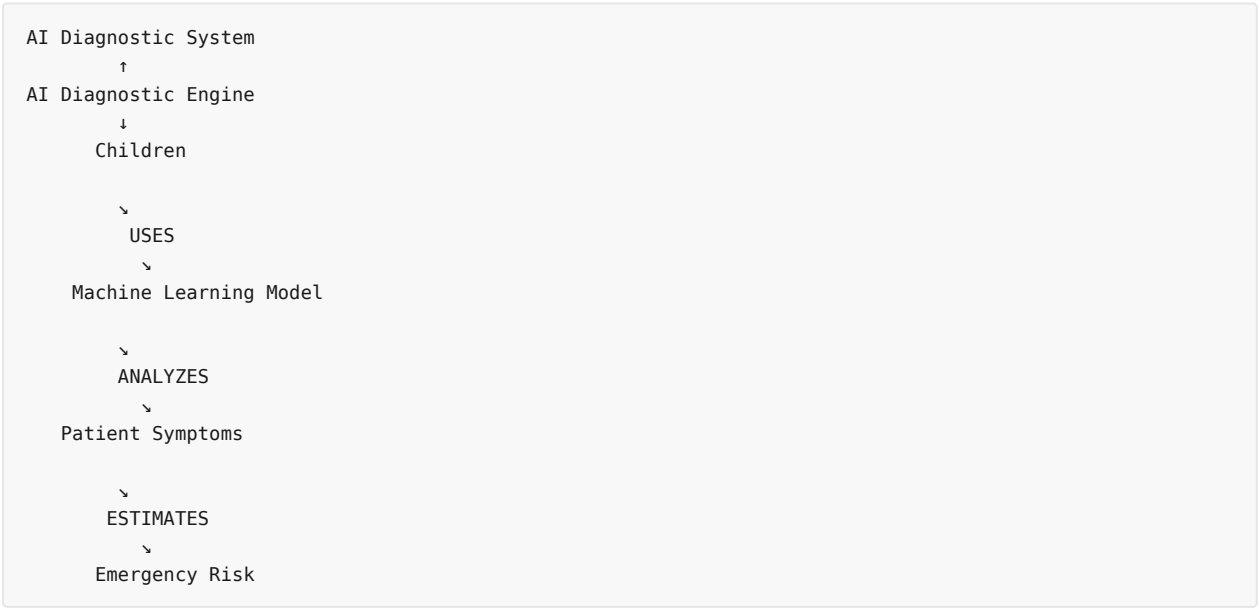


Parents + children.

Depth 2.

No cross-links.

COMPLEX — Hamari query

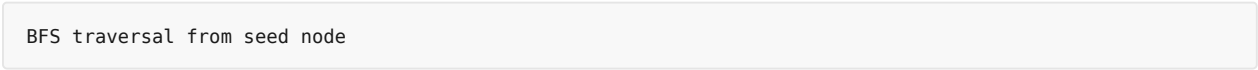


Depth up to 3.

Parents + children + cross-links.

22. Step 4 — BFS Traversal

Report ke according:



use hota hai. `filecite{turn23file0}L17-L40`

Pseudo-flow:

```
queue = [(seed, 0)]

while queue:
 current, depth = queue.pop()

 if already visited:
 skip

 if depth > max_depth:
 skip

 mark visited

 if parents enabled:
 add parents

 if children enabled:
 add children

 if cross-links enabled:
 add cross-links
```

## 23. Hamare example ka exact traversal

### Seed 1

AI Diagnostic Engine

Start:

Queue:  
[(AI Diagnostic Engine, depth 0)]

### Depth 0

Current:

AI Diagnostic Engine

Visited:

{AI Diagnostic Engine}

COMPLEX config:

Parents = YES  
Children = YES  
Cross-links = YES

Queue me add:

### Parent

AI Diagnostic System, depth 1

### Cross-links

```
Machine Learning Model, depth 1
Patient Symptoms, depth 1
Medical History, depth 1
Emergency Risk, depth 1
```

## Depth 1

Now queue:

```
AI Diagnostic System
Machine Learning Model
Patient Symptoms
Medical History
Emergency Risk
```

Har node ke:

- parent
- child
- cross-link

explore honge, subject to:

```
max_depth = 3
```

## 24. ~~visited~~ set kyun important hai?

Suppose graph:

```
AI Diagnostic Engine
↕
Machine Learning Model
```

Cross-links ke wajah se graph cycle-like navigation possible hai.

Without ~~visited~~:

```
A → B → A → B → A ...
```

BFS repeatedly same nodes visit kar sakta hai.

Report:

```
if current.id in visited:
 continue
```

use karti hai. `filecite{turn23file0}{L17-L40}`

So:

```
Visited Set
=
Duplicate traversal prevention
+
Cycle-safe traversal behavior
```

## 25. Multiple seeds ka traversal

Hamare paas 5 seeds hain.

System:

```
candidate_nodes = set()

for seed in seed_nodes:
 candidate_nodes |= traverse(seed)
```

filecite\turn22file0\L63-L66

Example:

```
Seed 1 traversal
→ {A, B, C, D}

Seed 2 traversal
→ {C, D, E, F}

Union
→ {A, B, C, D, E, F}
```

Duplicate nodes automatically set me merge ho jate hain.

## 26. Candidate nodes kya ban sakte hain?

Suppose all traversal ke baad:

Candidate Nodes

1. Smart Hospital System
2. AI Diagnostic System
3. AI Diagnostic Engine
4. Risk Prediction Module
5. Machine Learning Model
6. Patient Symptoms
7. Medical History
8. Emergency Risk
9. Preliminary Risk Score
10. Diagnostic Images

Important:

*Abhi bhi ye sab LLM ko nahi jayenge.*

Why?

Token budget.

Agar sab nodes:

```
10 nodes × average 200 tokens
=
2000 tokens
```

Aur budget:

```
1000 tokens
```

To selection required hai.

Yahan aata hai Layer 6 ka major component:

## Tree Knapsack

### 27. Step 5 — Tree Knapsack

Normal RAG kya karta hai?

```
Retrieve Top-k
↓
Sab LLM ko bhejo
```

SHIA-RAG:

```
Candidate Nodes
↓
Utility
+
Token Cost
+
Hierarchy Constraint
↓
Optimal / constrained selection
```

Project ka main idea:

*Child node select karne ke liye uska required hierarchical parent context bhi respect hona chahiye.*

Retrieved report me Tree Knapsack ko precedence-constrained selection ke form me define kiya gaya hai: child tabhi selected ho sakta hai jab parent bhi selected ho. `filecite[turn25file0][L11-L47]`

### 28. Simple Knapsack aur Tree Knapsack ka difference

Suppose:

```
AI Diagnostic System
Utility = 30
Cost = 100

AI Diagnostic Engine
Utility = 95
Cost = 150
```

Normal knapsack keh sakta hai:

```
Budget = 150

Select:
AI Diagnostic Engine only ✓
```

But hierarchy says:

```
AI Diagnostic System
↓
AI Diagnostic Engine
```

Tree Knapsack constraint:

```
Child selected
 ↓
Parent context constraint respected
```

Retrieved Tree Knapsack pseudocode explicitly states this parent-precedence condition. [filecite\turn25file0\L11-L47](#)

## 29. Precedence Constraint ko clearly samjho

Valid:

```
AI Diagnostic System
 ↓
AI Diagnostic Engine
```

Selection:

```
Selected = {Parent, Child}
```

Invalid conceptual selection:

```
Selected = {Child}
Parent not selected
```

Why?

Kyuki specific node ka context missing ho sakta hai.

## 30. Tree Knapsack ka internal idea

Retrieved implementation me:

```
Option 1:
Don't include node
→ descendants bhi select nahi honge

Option 2:
Include node
→ remaining budget me children consider karo
```

Child candidates ko utility/token-cost ratio ke basis par sort karke remaining budget allocate kiya jata hai.

Memoization `dp[(node.id, remaining_budget)]` bhi use hoti hai. [filecite\turn25file0\L15-L47](#)

Conceptually:

```
solve(Node, Budget)
```

Question:

Is node ko select karna worth it hai?

NO

→ utility = 0

YES

→ node cost subtract

→ children ko remaining budget

→ best valid selection

### 31. Hamare example me Tree Knapsack

Suppose:

| Node                  | Utility | Token Cost |
|-----------------------|---------|------------|
| AI Diagnostic System  | 80      | 120        |
| AI Diagnostic Engine  | 100     | 150        |
| Patient Symptoms      | 70      | 90         |
| Medical History       | 65      | 100        |
| ML Model              | 75      | 110        |
| Emergency Risk        | 90      | 100        |
| Smart Hospital System | 30      | 150        |

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

Suppose budget:

500 tokens

Total sabka cost:

820 tokens

So all cannot be selected.

Tree Knapsack ko decide karna hai:

*Kaunse nodes maximum useful knowledge denge while hierarchy constraints aur budget respect honge?*

Possible result:

|                      |     |
|----------------------|-----|
| AI Diagnostic System | 120 |
| AI Diagnostic Engine | 150 |
| Emergency Risk       | 100 |
| ML Model             | 110 |
| -----                |     |
| Total                | 480 |

Remaining:

20 tokens

Patient Symptoms aur Medical History fit nahi hote.

Then another combination check karna pad sakta hai:

```

AI Diagnostic System
AI Diagnostic Engine
Patient Symptoms
Emergency Risk

```

Cost:

```

120 + 150 + 90 + 100
=
460

```

Ab query-specific utility decide karegi ki kaunsa combination better hai.

---

## 32. Multi-Tree Knapsack kya hai?

Layer 6 sirf ek tree par kaam nahi karta.

Query ke multiple seeds alag trees se aa sakte hain.

Report algorithm:

1. Candidates ko tree root ke basis par group karo
2. Har tree ko proportional token budget do
3. Har tree par Tree Knapsack independently run karo
4. Results merge karo
5. Total budget ke andar final trim karo

`filecite\turn22file0\L93-L119` `filecite\turn22file0\L125-L132`

---

## 33. Example of multiple trees

Suppose:

```

TREE A – Patient Care

Patient Care
├─ Patient Symptoms
└─ Medical History

```

And:

```

TREE B – AI Diagnostic System

AI Diagnostic System
├─ AI Diagnostic Engine
└─ ML Model

```

Candidate count:

```

Tree A = 4 candidates
Tree B = 6 candidates

```

Total:

```

10

```

Budget:



1000 tokens

Report proportional allocation logic:

Tree A:  
 $1000 \times 4/10$   
 =  
 400 tokens

Tree B:  
 $1000 \times 6/10$   
 =  
 600 tokens

[filecite\turn22file0\L103-L119](#)

Then:

Tree A  
 → Tree Knapsack with 400

Tree B  
 → Tree Knapsack with 600

Then merge.

### 34. Important: Semantic cross-links par same parent constraint nahi

Report explicitly note karti hai:

*Semantic cross-linked nodes ke semantic predecessors ko necessarily select karna required nahi hai; hierarchical parent constraints apply hote hain.* [filecite\turn22file0\L93-L102](#)

Example:

AI Diagnostic Engine  
 USES  
 Machine Learning Model

Machine Learning Model select karne ke liye necessarily:

AI Diagnostic Engine

ko uska hierarchical parent treat nahi kiya jayega.

Because:

USES  
 ≠  
 parent-child hierarchy

Ye Layer 5 aur Layer 6 ke beech important consistency rule hai.

### 35. Multi-tree selection ke baad final trim

Report:

```
all_selected.sort(
 key=utility,
 reverse=True
)
```

Then:

```
for node:
 if total_tokens + cost <= budget:
 final.append(node)
```

filecite\turn22file0\L125-L132

So second-level budget enforcement hota hai.

### 36. Yahan ek important technical issue hai

Main yahan **report ke implementation ko exactly follow karke critical observation** kar raha hoon.

Per-tree Tree Knapsack ke baad report ye filter karti hai:

```
selected = [
 n for n in selected_set
 if n in nodes
]
```

Phir merge karti hai. filecite\turn22file0\L121-L123

#### Potential issue

Tree Knapsack hierarchical parent ko select kar sakta hai as required context.

Lekin agar parent original `candidate_nodes` me nahi tha aur sirf traversal/Tree Knapsack logic se include hua, to final filter:

```
if n in nodes
```

us parent ko remove kar sakta hai.

Then conceptual risk:

```
Tree Knapsack:
Parent + Child selected ✓

Candidate-only filter:
Parent removed ✗
Child remains ✓
```

Isse precedence constraint final output me break ho sakta hai.

**This is a critical implementation point.**

Report ka intention:

```
Child requires hierarchical parent
```

hai. filecite\turn25file0\L11-L47

Lekin shown multi-tree filtering logic ke saath edge case carefully test karna padega. [filecite\turn22file0\L121-L123](#)

Suggestion (Not from the report)

Final filtering ko:

```
selected node
+
all required ancestors
```

closure preserve karna chahiye.

37. Step 6 — Context Assembly

Ab final selected nodes ko LLM ke liye readable context me convert kiya jata hai.

Report:

```
Definitions first
↓
Core concepts
↓
Related evidence
```

Aur groups ke andar confidence descending order me sort kiya jata hai. [filecite\turn23file0\L42-L66](#)

38. Hamare example ka selected set

Suppose final:

```
AI Diagnostic System
AI Diagnostic Engine
Machine Learning Model
Patient Symptoms
Medical History
Emergency Risk
```

System inhe raw graph format me nahi bhejta.

It creates structured text.

39. Node rendering format

Report me har node approximately:

```
[NODE_ID] Concept Name

Definition:
...

Evidence:
...

Source:
...
```

format me render hota hai. `filecite\turn23file0\L42-L66`

Hamare example:

```
[NODE_003] AI Diagnostic System

Definition:
A healthcare AI system designed to support diagnostic analysis.

Evidence:
...

Source:
Hospital_AI_Report, Page 12
```

Then:

```

[NODE_004] AI Diagnostic Engine

Definition:
An AI component that analyzes clinical information to support
diagnostic risk assessment.

Evidence:
...

Source:
Hospital_AI_Report, Page 15
```

And so on.

## 40. Definitions first kyun?

Suppose Layer 7 ko ye context mile:

```
AI Diagnostic Engine
Emergency Risk
Machine Learning Model
Patient Symptoms
```

Agar definitions pehle hain:

```
Concept kya hai?
↓
Uska evidence kya hai?
↓
Related concept kya hai?
```

LLM ko semantic grounding better mil sakti hai.

Report ka assembly order explicitly definitions-first aur confidence-based sorting use karta hai.

`filecite\turn23file0\L42-L66`

## 41. Context assembly me possible issue

Shown code:

```

definitions =
all nodes with definitions

core =
all remaining nodes

```

Then definitions-first.

### Potential issue

Query-specific causal order automatically guarantee nahi hota.

Hamari query ka natural order:

```

Patient Symptoms
+
Medical History
 ↓
AI Diagnostic Engine
 ↓ uses
Machine Learning Model
 ↓
Emergency Risk

```

But confidence-based sorting result de sakta hai:

```

Emergency Risk
AI Diagnostic Engine
Medical History
ML Model
Patient Symptoms

```

Meaning:

*Definitions-first + confidence sorting ≠ guaranteed reasoning-chain order.*

Report me intent-aware semantic ordering ka exact algorithm beyond shown `assemble_context()` logic fully specified nahi hai.

## 42. `_get_path_edges()` — Layer 8 ke liye feedback trail

Layer 6 sirf context nahi deta.

Report selected nodes ke parent path edges bhi collect karti hai. `filecite\turn23file0\L68-L78`

Example:

```

AI Diagnostic Engine
 ↑
AI Diagnostic System
 ↑
Smart Hospital System

```

Collected:

```

Edge 1:
Smart Hospital System
→ AI Diagnostic System

Edge 2:
AI Diagnostic System
→ AI Diagnostic Engine

```

Why?

Layer 8 later retrieval success/failure ko structure se connect kar sakta hai.

Conceptually:

```

Selected Node
↓
Which hierarchy path led here?
↓
Store path edges
↓
Layer 8 Feedback

```

### 43. Final RetrievalResult

Hamari example query ke liye:

```

RetrievalResult
├── selected_nodes
│ ├── AI Diagnostic System
│ ├── AI Diagnostic Engine
│ ├── Machine Learning Model
│ ├── Patient Symptoms
│ ├── Medical History
│ └── Emergency Risk
├── context_text
│ └── Structured evidence blocks
├── traversal_path_edges
│ ├── Smart Hospital System
│ │ └── AI Diagnostic System
│ └── AI Diagnostic System
│ └── AI Diagnostic Engine
├── query_complexity
│ └── COMPLEX
├── query_intent
│ └── MULTI_HOP
├── seed_node_ids
│ └── [5 initial seed IDs]
└── token_count
 └── final estimated context size

```

Ye directly Layer 7 ko jayega. [filecite\turn22file0\L77-L91](https://chatgpt.com/e2b31603-f9df-4055-82cd-e524a077a7c4)

## 44. Ab complete Layer 6 ko ek baar end-to-end chalao

### Query

How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?

### Step 0 — Cache

Cache lookup  
↓  
MISS

Continue.

### Step 1 — Intent

MULTI\_HOP

### Complexity

COMPLEX

### Step 2 — Embedding

Query  
↓  
Asymmetric Embedding Model  
↓  
Query Vector

### Seed Search

Top 5:

1. AI Diagnostic Engine
2. Emergency Risk
3. Machine Learning Model
4. Patient Symptoms
5. Medical History

### Step 3 — Traversal Config

COMPLEX

Depth = 3

Parents      ✓  
Children     ✓  
Cross-links ✓

## Step 4 — BFS

From:

AI Diagnostic Engine

Explore:

Parent:  
AI Diagnostic System

Cross-links:  
Machine Learning Model  
Patient Symptoms  
Medical History  
Emergency Risk

Other seeds bhi independently traverse honge.

Union:

```
Candidate Nodes
=
{
 Smart Hospital System,
 AI Diagnostic System,
 AI Diagnostic Engine,
 ML Model,
 Patient Symptoms,
 Medical History,
 Emergency Risk,
 ...
}
```

## Step 5 — Multi-Tree Knapsack

```
Candidates
↓
Group by Tree Root
↓
Allocate proportional budget
↓
Tree Knapsack per Tree
↓
Merge
↓
Final Budget Trim
```

Final:



```
Selected Nodes
=
{
 AI Diagnostic System,
 AI Diagnostic Engine,
 ML Model,
 Patient Symptoms,
 Medical History,
 Emergency Risk
}
```

Step 6 — Context Assembly

```
Definitions
↓
Confidence sorting
↓
Evidence blocks
↓
One context string
```

Output

```
RetrievalResult
```

45. Layer 6 me major error scenarios

Ab hum specifically errors dekhenge.

Error 1 — Wrong intent

```
MULTI_HOP
```

galti se:

```
DEFINITION
```

classify ho gaya.

Result:

```
Depth = 1
Cross-links = OFF
```

Machine Learning Model miss ho sakta hai.

Error 2 — Wrong seed nodes

Correct seed:

```
AI Diagnostic Engine
```

miss.

Instead:

```
Hospital Administration System
```

retrieve.

Result:

```
Wrong starting point
↓
Wrong traversal
↓
Wrong candidates
```

### Error 3 — `seed_k = 5` insufficient

Query bahut broad hai:

```
"Explain the complete Smart Hospital AI workflow
from patient input to final emergency decision."
```

5 seeds insufficient ho sakte hain.

Exact adaptive `seed_k` policy shown Layer 6 code me:

```
Not specified.
```

### Error 4 — Fixed depth limit

Complex query ko actual 4-hop reasoning chahiye:

```
A → B → C → D → E
```

But:

```
COMPLEX max_depth = 3
```

Then E reach nahi hoga.

### Error 5 — Wrong cross-link

Layer 5 ne false semantic edge banaya:

```
AI Diagnostic Engine
CAUSES
Hospital Cafeteria Delay
```

COMPLEX traversal me:

```
Cross-links = ON
```

to irrelevant node candidate pool me aa sakta hai.

### Error 6 — Candidate explosion

Multiple seeds × depth 3 ×:

- parents
- children
- cross-links

Result:

5 seeds  
↓  
Hundreds of candidates

Tree Knapsack workload increase.

## Error 7 — Proportional budget allocation problem

Report budget candidates ke count ke proportional allocate karti hai. [filecite\turn22file0\L103-L119](#)

Potential issue:

Tree A:  
100 candidates  
mostly low relevance

Tree B:  
5 candidates  
extremely relevant

Candidate-count proportional allocation:

Tree A gets more budget

But relevance-based allocation better ho sakta tha.

## Suggestion (Not from the report)

Budget allocation consider kar sakta hai:

Candidate Count  
+  
Average Relevance  
+  
Seed Similarity  
+  
Query Coverage

## Error 8 — Tree Knapsack utility definition dependency

Multi-tree final trim:

`utility(node, forest)`

use karta hai. [filecite\turn22file0\L125-L132](#)

Lekin shown Layer 6 section me exact `utility()` formula fully shown nahi hai.

So:

*Utility ka exact internal scoring formula is retrieved Layer 6 section me not specified.*

Ye important missing dependency hai.

### Error 9 — `token_cost(node)` exact estimation

Shown code:

```
token_cost(node)
```

use karta hai.

Lekin exact tokenizer/model-specific implementation:

```
Not specified in the shown Layer 6 section.
```

Different tokenizers:

```
same text
→ different token counts
```

de sakte hain.

### Error 10 — Context order breaks reasoning chain

As discussed:

```
Confidence sorting
```

high-confidence node ko pehle la sakti hai, chahe logical order baad me hona chahiye.

## 46. Exception handling — report me explicitly kya hai?

Strictly report ke shown logic ke according:

### A. Cache Hit

```
Return cached result
```

```
filecite[turn22file0][L40-L45]
```

### B. Unknown Intent

`classify_complexity()` me:

```
mapping.get(intent, 'MODERATE')
```

Meaning unknown intent ka fallback:

```
MODERATE
```

```
filecite[turn23file0][L80-L89]
```

Ye Layer 6 ka explicit fallback behavior hai.

---

### C. Traversal Duplicate

```
visited
```

duplicate traversal avoid karta hai. `filecite\turn23file0\L17-L40`

---

### D. Depth exceeded

```
if depth > max_depth:
 continue
```

Traversal stop.

`filecite\turn23file0\L22-L26`

---

### E. Budget exceeded

Final trim:

```
if total_tokens + cost <= budget:
 select
```

Otherwise node skip. `filecite\turn22file0\L125-L132`

---

## 47. Exception handling jo shown report me specified nahi hai

Main kuch invent nahi kar raha — ye exact behaviors shown section me missing hain:

### Vector database down

```
Not specified.
```

### Query embedding API/model timeout

```
Not specified.
```

### Empty seed result

```
Not explicitly handled.
```

### Forest unavailable

```
Not specified.
```

### Invalid embedding / NaN

```
Not specified.
```

### Cache write failure

Not specified.

### Tree Knapsack internal failure

Not specified.

### Context exceeds budget after assembly due to estimation mismatch

Not explicitly handled.

## 48. Layer 6 ki sabse important dependency

Layer 6 khud se knowledge invent nahi karta.

Iski quality heavily depend karti hai:

```
Layer 5 Forest Quality
+
Vector Search Quality
+
Intent Classification Quality
+
Traversal Policy
+
Tree Knapsack Utility
```

## 49. Error propagation from Layer 5

Suppose Layer 5 ne wrong hierarchy banayi:

```
Machine Learning Model
↓
AI Diagnostic Engine
```

instead of semantic:

```
AI Diagnostic Engine
 USES
Machine Learning Model
```

Then Layer 6:

Parent traversal

ko galat structure milta hai.

Result:

```
Wrong Layer 5
↓
Wrong traversal path
↓
Wrong candidate set
↓
Wrong context
```

So Layer 6 ka algorithm correct hone ke baad bhi Layer 5 structure error affect karega.

## 50. Layer 6 ki core novelty ko simple language me samjho

### Traditional RAG

```
Question
↓
Vector Search
↓
Top-k Chunks
↓
LLM
```

Problem:

```
Top-k chunks
=
flat isolated pieces
```

### SHIA-RAG Layer 6

```
Question
↓
Understand Query Type
↓
Find Relevant Seed Concepts
↓
Explore Knowledge Structure
↓
Different Depth based on Complexity
↓
Respect Hierarchical Context
↓
Optimize under Token Budget
↓
Structured Evidence
↓
LLM
```

## 51. Ek line me har step yaad rakho

| Step | Component | Simple Meaning |

|---|---|---|

| 0 | Cache | Kya result already available hai? |

| 1 | Intent | User actually kya puch raha hai? |

| 2 | Seed Search | Query se directly relevant concepts kaunse hain? |

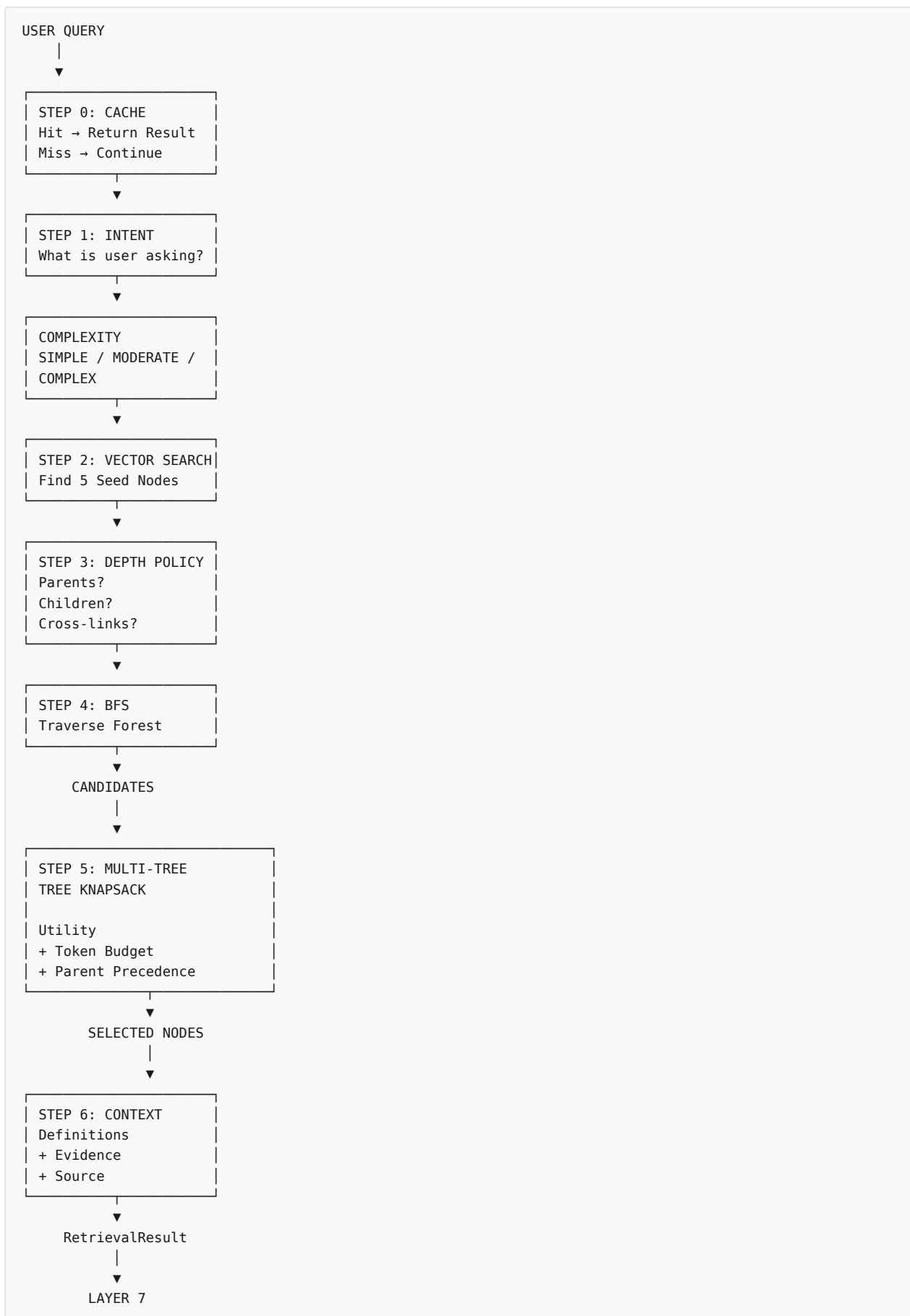
| 3 | Adaptive Depth | Kitna deeply explore karna hai? |

| 4 | BFS | Forest me relevant directions me chalo |

| 5 | Tree Knapsack | Limited tokens me best valid context select karo |

| 6 | Context Assembly | Selected knowledge ko LLM-readable format me banao |

## 52. Final complete mental model



Sabse important understanding



*Layer 5 knowledge ko organize karta hai, lekin Layer 6 us organized knowledge ko query ke according dynamically navigate karta hai.*

Aur Layer 6 me do major decisions hote hain:

Decision 1

Forest me kitna aur kis direction me search karna hai?

→ Adaptive Depth Traversal

Decision 2

Bahut saare relevant nodes me se limited token budget me kaunse final context me jayenge?

→ Precedence-Constrained Tree Knapsack

Isi wajah se Layer 6 SHIA-RAG ka **actual retrieval intelligence layer** hai. Layer 5 ne forest banaya; **Layer 6 decide karta hai ki user ke current question ke liye us forest ka kaunsa exact part retrieve karna hai.**

fileciteurn25file4L343-L368

memcite

you 20/08/2026, 15:58:24

ab mujhe layer 7 smjhao jese baki layer smjhayee the

CHATGPT 20/08/2026, 15:58:29

{ "paths": [ "files" ], "query": "search" }

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: fileciteurn26file0

- [L1] metadata:
- [L2] query\_result\_index: 0
- [L3] file\_id: file\_000000004bc481fdbf9e949ec3b7544b
- [L4] version\_id: 1
- [L5] name: SHIA\_RAG\_Final\_Report(2).md
- [L6] mime\_type: text/markdown
- [L7] surface: library
- [L8] score: 0.028594771241830064
- [L9] document\_chunk\_id: file\_000000004bc481fdbf9e949ec3b7544b--1--23
- [L10] snippet:
- [L11] ##### Prompt Template
- [L12]
- [L13]

```

[L14] SYSTEM_PROMPT = """"You are a knowledge-grounded question answering system.
[L15] RULES:
[L16] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L17] 2. For each claim, cite the source using [NODE_ID] format.
[L18] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L19] 4. If evidence conflicts, present both views with citations.
[L20]
[L21] REQUIRED OUTPUT FORMAT:
[L22] {
[L23] "answer": "Your answer with [NODE_ID] citations inline.",
[L24] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L25] "overall_confidence": 0.0-1.0,
[L26] "insufficient_evidence": false
[L27] }""""
[L28]

```

[L29]

[L30] ##### Claim Verification

[L31]

[L32] After the LLM generates an answer, each claim is verified against its cited evidence:

[L33]

[L34]

```

[L35] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L36] """Returns (status, confidence).
[L37] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L38] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L39] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L40] - CONFLICTING: evidence items contradict each other
[L41] """
[L42] scores = []
[L43] for eid in evidence_ids:
[L44] node = node_map.get(eid)
[L45] if node and node.evidence:
[L46] evidence_text = ' '.join(e.text for e in node.evidence)
[L47] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L48]
[L49] if not scores:
[L50] return 'UNSUPPORTED', 0.0
[L51]
[L52] avg_score = sum(scores) / len(scores)
[L53]
[L54] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L55] return 'CONFLICTING', avg_score
[L56] if avg_score > 0.7:
[L57] return 'SUPPORTED', avg_score
[L58] elif avg_score > 0.4:
[L59] return 'PARTIAL', avg_score
[L60] else:
[L61] return 'UNSUPPORTED', avg_score
[L62]

```

[L63]

[L64] ##### Unsupported Claim Policy

[L65]

[L66] | Status | Action |

[L67] |-----|-----|

[L68] | SUPPORTED | Keep as-is with citation |

[L69] | PARTIAL | Keep but mark as partially supported |

[L70] | UNSUPPORTED | Remove from answer, note at end |

[L71] | CONFLICTING | Present both views with note |

[L72]

[L73] ##### Answer Confidence Formula

[L74]

[L75] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{node\_conf}} + 0.2 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L76]

[L77] ##### Output Schema

[L78]

[L79]

```
[L80] @dataclass
[L81] class GenerationResult:
[L82] """Output of Layer 7 – ready for user and Layer 8."""
[L83] answer: str # Answer text with citations
[L84] claims: list # List of VerifiedClaim
[L85] citations: list # Formatted citation list
[L86] overall_confidence: float # 0.0-1.0
[L87] retrieval_result: object # Preserved for Layer 8
[L88] generation_metadata: dict # Model used, claim counts, etc.
[L89]
```

[L90]

[L91] ---

[L92]

[L93] ## 7. Operations Layer — Layer 8

[L94]

[L95] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L96]

[L97] ### 7.1 Logging

[L98]

[L99] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite`turn26file1

[L100] metadata:

[L101] query\_result\_index: 1

[L102] file\_id: file\_00000000a0a481fab4c739c73d87551b

[L103] version\_id: 1

[L104] name: SHIA\_RAG\_Complete\_Report(7).md

[L105] mime\_type: text/markdown

[L106] surface: library

[L107] score: 0.02844163539403516

[L108] document\_chunk\_id: file\_00000000a0a481fab4c739c73d87551b--1--45

```

[L109] snippet:
[L110] @dataclass
[L111] class RetrievalResult:
[L112] """Output of Layer 6 — input to Layer 7."""
[L113] selected_nodes: list # Nodes selected by Tree Knapsack
[L114] context_text: str # Assembled context string
[L115] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L116] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L117] query_intent: str # DEFINITION, COMPARISON, etc.
[L118] seed_node_ids: list # Initial seed nodes from vector search
[L119] token_count: int # Estimated tokens in context
[L120]
[L121] class RetrievalEngine:
[L122] def __init__(self, forest, vector_index, cache_manager, config):
[L123] self.forest = forest
[L124] self.vector_index = vector_index
[L125] self.cache = cache_manager # Redis cache integration
[L126] self.config = config
[L127] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L128]
[L129] def retrieve(self, query) -> RetrievalResult:
[L130] # Step 0: Cache check
[L131] cache_key = hash_query(query)
[L132] cached = self.cache.get_query_result(cache_key)
[L133] if cached:
[L134] return cached
[L135]
[L136] # Step 1: Analyze query
[L137] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L138] complexity = classify_complexity(intent) # Rule-based mapping
[L139]
[L140] # Step 2: Embed and find seed nodes
[L141] query_emb = embed_query(query) # Asymmetric embedding model
[L142] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L143]
[L144] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L145] depth_config = {
[L146] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L147] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L148] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L149] }
[L150] traversal = depth_config[complexity]
[L151]
[L152] # Step 4: Traverse forest from all seeds
[L153] candidate_nodes = set()
[L154] for seed in seed_nodes:

```

```

[L155] candidate_nodes |= self.traverse(seed, traversal)
[L156]
[L157] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L158] selected = self.multi_tree_knapsack(
[L159] candidate_nodes,
[L160] budget=self.config.token_budget
[L161])
[L162]
[L163] # Step 6: Assemble context in semantic order
[L164] context_text = self.assemble_context(selected, intent)
[L165]
[L166] # Build result with traversal path for Layer 8 feedback
[L167] result = RetrievalResult(
[L168] selected_nodes=selected,
[L169] context_text=context_text,
[L170] traversal_path_edges=self._get_path_edges(selected),
[L171] query_complexity=complexity,
[L172] query_intent=intent,
[L173] seed_node_ids=[s.id for s in seed_nodes],
[L174] token_count=estimate_tokens(context_text)
[L175])
[L176]
[L177] # Cache result
[L178] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L179]
[L180] return result
[L181]
[L182] def multi_tree_knapsack(self, candidate_nodes, budget):
[L183] """Apply Tree Knapsack per distinct tree, then merge results.
[L184]
[L185] For multi-seed queries, candidates may span multiple trees.
[L186] We optimize each tree independently, then merge within budget.

```

---

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite[turn26file2]`

```

[L187] metadata:
[L188] query_result_index: 2
[L189] file_id: file_000000003218820b918ee3d86ec345ae
[L190] version_id: 1
[L191] name: SHIA_RAG_Final_Report(1).md
[L192] mime_type: text/markdown
[L193] surface: library
[L194] score: 0.028404512489927477
[L195] document_chunk_id: file_000000003218820b918ee3d86ec345ae--1--23
[L196] snippet:

```

[L197] ##### Prompt Template

[L198]

[L199]

```
[L200] SYSTEM_PROMPT = """"You are a knowledge-grounded question answering system.
[L201] RULES:
[L202] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L203] 2. For each claim, cite the source using [NODE_ID] format.
[L204] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L205] 4. If evidence conflicts, present both views with citations.
[L206]
[L207] REQUIRED OUTPUT FORMAT:
[L208] {
[L209] "answer": "Your answer with [NODE_ID] citations inline.",
[L210] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L211] "overall_confidence": 0.0-1.0,
[L212] "insufficient_evidence": false
[L213] }""""
[L214]
```

[L215]

[L216] ##### Claim Verification

[L217]

[L218] After the LLM generates an answer, each claim is verified against its cited evidence:

[L219]

[L220]

```
[L221] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L222] """Returns (status, confidence).
[L223] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L224] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L225] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L226] - CONFLICTING: evidence items contradict each other
[L227] """
[L228] scores = []
[L229] for eid in evidence_ids:
[L230] node = node_map.get(eid)
[L231] if node and node.evidence:
[L232] evidence_text = ' '.join(e.text for e in node.evidence)
[L233] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L234]
[L235] if not scores:
[L236] return 'UNSUPPORTED', 0.0
[L237]
[L238] avg_score = sum(scores) / len(scores)
[L239]
[L240] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L241] return 'CONFLICTING', avg_score
[L242] if avg_score > 0.7:
[L243] return 'SUPPORTED', avg_score
[L244] elif avg_score > 0.4:
[L245] return 'PARTIAL', avg_score
[L246] else:
[L247] return 'UNSUPPORTED', avg_score
[L248]
```

[L249]

[L250] ##### Unsupported Claim Policy

[L251]

[L252] | Status | Action |

[L253] |-----|-----|

[L254] | SUPPORTED | Keep as-is with citation |

[L255] | PARTIAL | Keep but mark as partially supported |

[L256] | UNSUPPORTED | Remove from answer, note at end |

[L257] | CONFLICTING | Present both views with note |

[L258]

[L259] ##### Answer Confidence Formula

[L260]

[L261] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{node\_conf}} + 0.2 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L262]

[L263] ##### Output Schema

[L264]

[L265]

```
[L266] @dataclass
[L267] class GenerationResult:
[L268] """Output of Layer 7 – ready for user and Layer 8."""
[L269] answer: str # Answer text with citations
[L270] claims: list # List of VerifiedClaim
[L271] citations: list # Formatted citation list
[L272] overall_confidence: float # 0.0-1.0
[L273] retrieval_result: object # Preserved for Layer 8
[L274] generation_metadata: dict # Model used, claim counts, etc.
[L275]
```

[L276]

[L277] ---

[L278]

[L279] ## 7. Operations Layer — Layer 8

[L280]

[L281] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L282]

[L283] ### 7.1 Logging

[L284]

[L285] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `[[filecite]]turn26file3[[/filecite]]`

[L286] metadata:

[L287] query\_result\_index: 3

[L288] file\_id: file\_000000003c8081fab658b1752549d64

[L289] version\_id: 1

[L290] name: SHIA\_RAG\_Complete\_Report(5).md

[L291] mime\_type: text/markdown

```

[L292] surface: library
[L293] score: 0.028177225029148854
[L294] document_chunk_id: file_000000003c8081fab658b1752549d64--1--45
[L295] snippet:
[L296] @dataclass
[L297] class RetrievalResult:
[L298] """Output of Layer 6 — input to Layer 7."""
[L299] selected_nodes: list # Nodes selected by Tree Knapsack
[L300] context_text: str # Assembled context string
[L301] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L302] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L303] query_intent: str # DEFINITION, COMPARISON, etc.
[L304] seed_node_ids: list # Initial seed nodes from vector search
[L305] token_count: int # Estimated tokens in context
[L306]
[L307] class RetrievalEngine:
[L308] def __init__(self, forest, vector_index, cache_manager, config):
[L309] self.forest = forest
[L310] self.vector_index = vector_index
[L311] self.cache = cache_manager # Redis cache integration
[L312] self.config = config
[L313] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L314]
[L315] def retrieve(self, query) -> RetrievalResult:
[L316] # Step 0: Cache check
[L317] cache_key = hash_query(query)
[L318] cached = self.cache.get_query_result(cache_key)
[L319] if cached:
[L320] return cached
[L321]
[L322] # Step 1: Analyze query
[L323] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L324] complexity = classify_complexity(intent) # Rule-based mapping
[L325]
[L326] # Step 2: Embed and find seed nodes
[L327] query_emb = embed_query(query) # Asymmetric embedding model
[L328] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L329]
[L330] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L331] depth_config = {
[L332] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L333] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L334] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L335] }
[L336] traversal = depth_config[complexity]
[L337]

```



```

[L338] # Step 4: Traverse forest from all seeds
[L339] candidate_nodes = set()
[L340] for seed in seed_nodes:
[L341] candidate_nodes |= self.traverse(seed, traversal)
[L342]
[L343] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L344] selected = self.multi_tree_knapsack(
[L345] candidate_nodes,
[L346] budget=self.config.token_budget
[L347])
[L348]
[L349] # Step 6: Assemble context in semantic order
[L350] context_text = self.assemble_context(selected, intent)
[L351]
[L352] # Build result with traversal path for Layer 8 feedback
[L353] result = RetrievalResult(
[L354] selected_nodes=selected,
[L355] context_text=context_text,
[L356] traversal_path_edges=self._get_path_edges(selected),
[L357] query_complexity=complexity,
[L358] query_intent=intent,
[L359] seed_node_ids=[s.id for s in seed_nodes],
[L360] token_count=estimate_tokens(context_text)
[L361])
[L362]
[L363] # Cache result
[L364] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L365]
[L366] return result
[L367]
[L368] def multi_tree_knapsack(self, candidate_nodes, budget):
[L369] """Apply Tree Knapsack per distinct tree, then merge results.
[L370]
[L371] For multi-seed queries, candidates may span multiple trees.
[L372] We optimize each tree independently, then merge within budget.

```

---

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite[turn26file4]`

```

[L373] metadata:
[L374] query_result_index: 4
[L375] file_id: file_000000001dd481fa9855f2731dbc53d5
[L376] version_id: 1
[L377] name: SHIA_RAG_Complete_Report(6).md
[L378] mime_type: text/markdown
[L379] surface: library

```

```

[L380] score: 0.028177225029148854
[L381] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--45
[L382] snippet:
[L383] @dataclass
[L384] class RetrievalResult:
[L385] """Output of Layer 6 — input to Layer 7."""
[L386] selected_nodes: list # Nodes selected by Tree Knapsack
[L387] context_text: str # Assembled context string
[L388] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L389] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L390] query_intent: str # DEFINITION, COMPARISON, etc.
[L391] seed_node_ids: list # Initial seed nodes from vector search
[L392] token_count: int # Estimated tokens in context
[L393]
[L394] class RetrievalEngine:
[L395] def __init__(self, forest, vector_index, cache_manager, config):
[L396] self.forest = forest
[L397] self.vector_index = vector_index
[L398] self.cache = cache_manager # Redis cache integration
[L399] self.config = config
[L400] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L401]
[L402] def retrieve(self, query) -> RetrievalResult:
[L403] # Step 0: Cache check
[L404] cache_key = hash_query(query)
[L405] cached = self.cache.get_query_result(cache_key)
[L406] if cached:
[L407] return cached
[L408]
[L409] # Step 1: Analyze query
[L410] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L411] complexity = classify_complexity(intent) # Rule-based mapping
[L412]
[L413] # Step 2: Embed and find seed nodes
[L414] query_emb = embed_query(query) # Asymmetric embedding model
[L415] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L416]
[L417] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L418] depth_config = {
[L419] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L420] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L421] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L422] }
[L423] traversal = depth_config[complexity]
[L424]
[L425] # Step 4: Traverse forest from all seeds

```

```

[L426] candidate_nodes = set()
[L427] for seed in seed_nodes:
[L428] candidate_nodes |= self.traverse(seed, traversal)
[L429]
[L430] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L431] selected = self.multi_tree_knapsack(
[L432] candidate_nodes,
[L433] budget=self.config.token_budget
[L434])
[L435]
[L436] # Step 6: Assemble context in semantic order
[L437] context_text = self.assemble_context(selected, intent)
[L438]
[L439] # Build result with traversal path for Layer 8 feedback
[L440] result = RetrievalResult(
[L441] selected_nodes=selected,
[L442] context_text=context_text,
[L443] traversal_path_edges=self._get_path_edges(selected),
[L444] query_complexity=complexity,
[L445] query_intent=intent,
[L446] seed_node_ids=[s.id for s in seed_nodes],
[L447] token_count=estimate_tokens(context_text)
[L448])
[L449]
[L450] # Cache result
[L451] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L452]
[L453] return result
[L454]
[L455] def multi_tree_knapsack(self, candidate_nodes, budget):
[L456] """Apply Tree Knapsack per distinct tree, then merge results.
[L457]
[L458] For multi-seed queries, candidates may span multiple trees.
[L459] We optimize each tree independently, then merge within budget.

```

---

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite`turn26file5

```

[L460] metadata:
[L461] query_result_index: 5
[L462] file_id: file_0000000012688211b4f57f5b8bcb8b42
[L463] version_id: 1
[L464] name: SHIA_RAG_Final_Report(4).md
[L465] mime_type: text/markdown
[L466] surface: conversation
[L467] score: 0.02786377708978328

```

[L468] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--35

[L469] snippet:

[L470] ##### Prompt Template

[L471]

[L472]

```
[L473] SYSTEM_PROMPT = """"You are a knowledge-grounded question answering system.
[L474] RULES:
[L475] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L476] 2. For each claim, cite the source using [NODE_ID] format.
[L477] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L478] 4. If evidence conflicts, present both views with citations.
[L479]
[L480] REQUIRED OUTPUT FORMAT:
[L481] {
[L482] "answer": "Your answer with [NODE_ID] citations inline.",
[L483] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L484] "overall_confidence": 0.0-1.0,
[L485] "insufficient_evidence": false
[L486] }""""
[L487]
```

[L488]

[L489] ##### Claim Verification

[L490]

[L491] After the LLM generates an answer, each claim is verified against its cited evidence:

[L492]

[L493]

```
[L494] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L495] """Returns (status, confidence).
[L496] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L497] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L498] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L499] - CONFLICTING: evidence items contradict each other
[L500] """
[L501] scores = []
[L502] for eid in evidence_ids:
[L503] node = node_map.get(eid)
[L504] if node and node.evidence:
[L505] evidence_text = ' '.join(e.text for e in node.evidence)
[L506] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L507]
[L508] if not scores:
[L509] return 'UNSUPPORTED', 0.0
[L510]
[L511] avg_score = sum(scores) / len(scores)
[L512]
[L513] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L514] return 'CONFLICTING', avg_score
[L515] if avg_score > 0.7:
[L516] return 'SUPPORTED', avg_score
[L517] elif avg_score > 0.4:
[L518] return 'PARTIAL', avg_score
[L519] else:
[L520] return 'UNSUPPORTED', avg_score
[L521]
```

[L522]

[L523] ##### Unsupported Claim Policy

[L524]

[L525] | Status | Action |

[L526] |-----|-----|

[L527] | SUPPORTED | Keep as-is with citation |

[L528] | PARTIAL | Keep but mark as partially supported |

[L529] | UNSUPPORTED | Remove from answer, note at end |

[L530] | CONFLICTING | Present both views with note |

[L531]

[L532] ##### Answer Confidence Formula

[L533]

[L534] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{node\_conf}} + 0.2 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L535]

[L536] ##### Output Schema

[L537]

[L538]

```
[L539] @dataclass
[L540] class GenerationResult:
[L541] """Output of Layer 7 – ready for user and Layer 8."""
[L542] answer: str # Answer text with citations
[L543] claims: list # List of VerifiedClaim
[L544] citations: list # Formatted citation list
[L545] overall_confidence: float # 0.0-1.0
[L546] retrieval_result: object # Preserved for Layer 8
[L547] generation_metadata: dict # Model used, claim counts, etc.
[L548]
```

[L549]

[L550] ---

[L551]

[L552] ## 7. Operations Layer — Layer 8

[L553]

[L554] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L555]

[L556] ### 7.1 Logging

[L557]

[L558] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite`turn26file6

[L559] metadata:

[L560] query\_result\_index: 6

[L561] file\_id: file\_00000000f620821186e54e582d59731e

[L562] version\_id: 1

[L563] name: SHIA\_RAG\_Final\_Report(3).md  
[L564] mime\_type: text/markdown  
[L565] surface: library  
[L566] score: 0.027692895339954164  
[L567] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--23  
[L568] snippet:  
[L569] ##### Prompt Template  
[L570]  
[L571]

```
[L572] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L573] RULES:
[L574] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L575] 2. For each claim, cite the source using [NODE_ID] format.
[L576] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L577] 4. If evidence conflicts, present both views with citations.
[L578]
[L579] REQUIRED OUTPUT FORMAT:
[L580] {
[L581] "answer": "Your answer with [NODE_ID] citations inline.",
[L582] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L583] "overall_confidence": 0.0-1.0,
[L584] "insufficient_evidence": false
[L585] }"""
[L586]
```

[L587]  
[L588] ##### Claim Verification  
[L589]  
[L590] After the LLM generates an answer, each claim is verified against its cited evidence:  
[L591]  
[L592]

```

[L593] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L594] """Returns (status, confidence).
[L595] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L596] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L597] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L598] - CONFLICTING: evidence items contradict each other
[L599] """
[L600] scores = []
[L601] for eid in evidence_ids:
[L602] node = node_map.get(eid)
[L603] if node and node.evidence:
[L604] evidence_text = ' '.join(e.text for e in node.evidence)
[L605] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L606]
[L607] if not scores:
[L608] return 'UNSUPPORTED', 0.0
[L609]
[L610] avg_score = sum(scores) / len(scores)
[L611]
[L612] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L613] return 'CONFLICTING', avg_score
[L614] if avg_score > 0.7:
[L615] return 'SUPPORTED', avg_score
[L616] elif avg_score > 0.4:
[L617] return 'PARTIAL', avg_score
[L618] else:
[L619] return 'UNSUPPORTED', avg_score
[L620]

```

[L621]

[L622] ##### Unsupported Claim Policy

[L623]

[L624] | Status | Action |

[L625] |-----|-----|

[L626] | SUPPORTED | Keep as-is with citation |

[L627] | PARTIAL | Keep but mark as partially supported |

[L628] | UNSUPPORTED | Remove from answer, note at end |

[L629] | CONFLICTING | Present both views with note |

[L630]

[L631] ##### Answer Confidence Formula

[L632]

[L633] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{node\_conf}} + 0.2 \cdot \frac{\text{node\_conf}}{\text{cited nodes}} + 0.1 \cdot \frac{\text{selected nodes}}{\text{selected nodes}}$$

[L634]

[L635] ##### Output Schema

[L636]

[L637]

```
[L638] @dataclass
[L639] class GenerationResult:
[L640] """Output of Layer 7 – ready for user and Layer 8."""
[L641] answer: str # Answer text with citations
[L642] claims: list # List of VerifiedClaim
[L643] citations: list # Formatted citation list
[L644] overall_confidence: float # 0.0-1.0
[L645] retrieval_result: object # Preserved for Layer 8
[L646] generation_metadata: dict # Model used, claim counts, etc.
[L647]
```

[L648]

[L649] ---

[L650]

[L651] ## 7. Operations Layer — Layer 8

[L652]

[L653] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L654]

[L655] ### 7.1 Logging

[L656]

[L657] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite`turn26file7

[L658] metadata:

[L659] query\_result\_index: 7

[L660] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L661] version\_id: 1

[L662] name: SHIA\_RAG\_Final\_Report.md

[L663] mime\_type: text/markdown

[L664] surface: library

[L665] score: 0.02661064425770308

[L666] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--23

[L667] snippet:

[L668] ##### Prompt Template

[L669]

[L670]



```

[L671] SYSTEM_PROMPT = """"You are a knowledge-grounded question answering system.
[L672] RULES:
[L673] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L674] 2. For each claim, cite the source using [NODE_ID] format.
[L675] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L676] 4. If evidence conflicts, present both views with citations.
[L677]
[L678] REQUIRED OUTPUT FORMAT:
[L679] {
[L680] "answer": "Your answer with [NODE_ID] citations inline.",
[L681] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L682] "overall_confidence": 0.0-1.0,
[L683] "insufficient_evidence": false
[L684] }""""
[L685]

```

[L686]

[L687] ##### Claim Verification

[L688]

[L689] After the LLM generates an answer, each claim is verified against its cited evidence:

[L690]

[L691]

```

[L692] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L693] """Returns (status, confidence).
[L694] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L695] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L696] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L697] - CONFLICTING: evidence items contradict each other
[L698] """
[L699] scores = []
[L700] for eid in evidence_ids:
[L701] node = node_map.get(eid)
[L702] if node and node.evidence:
[L703] evidence_text = ' '.join(e.text for e in node.evidence)
[L704] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L705]
[L706] if not scores:
[L707] return 'UNSUPPORTED', 0.0
[L708]
[L709] avg_score = sum(scores) / len(scores)
[L710]
[L711] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L712] return 'CONFLICTING', avg_score
[L713] if avg_score > 0.7:
[L714] return 'SUPPORTED', avg_score
[L715] elif avg_score > 0.4:
[L716] return 'PARTIAL', avg_score
[L717] else:
[L718] return 'UNSUPPORTED', avg_score
[L719]

```

[L720]

[L721] ##### Unsupported Claim Policy

[L722]

[L723] | Status | Action |

[L724] |-----|-----|

[L725] | SUPPORTED | Keep as-is with citation |

[L726] | PARTIAL | Keep but mark as partially supported |

[L727] | UNSUPPORTED | Remove from answer, note at end |

[L728] | CONFLICTING | Present both views with note |

[L729]

[L730] ##### Answer Confidence Formula

[L731]

[L732] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{node\_conf}} + 0.2 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L733]

[L734] ##### Output Schema

[L735]

[L736]

```
[L737] @dataclass
[L738] class GenerationResult:
[L739] """Output of Layer 7 – ready for user and Layer 8."""
[L740] answer: str # Answer text with citations
[L741] claims: list # List of VerifiedClaim
[L742] citations: list # Formatted citation list
[L743] overall_confidence: float # 0.0-1.0
[L744] retrieval_result: object # Preserved for Layer 8
[L745] generation_metadata: dict # Model used, claim counts, etc.
[L746]
```

[L747]

[L748] ---

[L749]

[L750] ## 7. Operations Layer — Layer 8

[L751]

[L752] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L753]

[L754] ### 7.1 Logging

[L755]

[L756] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite`turn26file8

[L757] metadata:

[L758] query\_result\_index: 8

[L759] file\_id: file\_0000000033088207b45d2ae58cbd406c

[L760] version\_id: 1

[L761] name: SHIA\_RAG\_Complete\_Report(8).md

[L762] mime\_type: text/markdown

[L763] surface: library

[L764] score: 0.026419626007891578

[L765] document\_chunk\_id: file\_0000000033088207b45d2ae58cbd406c--1--45

```

[L766] snippet:
[L767] @dataclass
[L768] class RetrievalResult:
[L769] """Output of Layer 6 — input to Layer 7."""
[L770] selected_nodes: list # Nodes selected by Tree Knapsack
[L771] context_text: str # Assembled context string
[L772] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L773] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L774] query_intent: str # DEFINITION, COMPARISON, etc.
[L775] seed_node_ids: list # Initial seed nodes from vector search
[L776] token_count: int # Estimated tokens in context
[L777]
[L778] class RetrievalEngine:
[L779] def __init__(self, forest, vector_index, cache_manager, config):
[L780] self.forest = forest
[L781] self.vector_index = vector_index
[L782] self.cache = cache_manager # Redis cache integration
[L783] self.config = config
[L784] self.seed_k = config.get('seed_k', 5) # Default: 5 seed nodes
[L785]
[L786] def retrieve(self, query) -> RetrievalResult:
[L787] # Step 0: Cache check
[L788] cache_key = hash_query(query)
[L789] cached = self.cache.get_query_result(cache_key)
[L790] if cached:
[L791] return cached
[L792]
[L793] # Step 1: Analyze query
[L794] intent = detect_intent(query) # DEFINITION, COMPARISON, WHY, HOW, SYNTHESIS
[L795] complexity = classify_complexity(intent) # Rule-based mapping
[L796]
[L797] # Step 2: Embed and find seed nodes
[L798] query_emb = embed_query(query) # Asymmetric embedding model
[L799] seed_nodes = self.vector_index.search(query_emb, k=self.seed_k)
[L800]
[L801] # Step 3: Adaptive traversal depth (NOVELTY 4)
[L802] depth_config = {
[L803] 'SIMPLE': {'max_depth': 1, 'include_parents': True, 'include_children': False, 'include_crosslinks': False},
[L804] 'MODERATE': {'max_depth': 2, 'include_parents': True, 'include_children': True, 'include_crosslinks': False},
[L805] 'COMPLEX': {'max_depth': 3, 'include_parents': True, 'include_children': True, 'include_crosslinks': True},
[L806] }
[L807] traversal = depth_config[complexity]
[L808]
[L809] # Step 4: Traverse forest from all seeds
[L810] candidate_nodes = set()
[L811] for seed in seed_nodes:

```

```
[L812] candidate_nodes |= self.traverse(seed, traversal)
[L813]
[L814] # Step 5: Tree Knapsack selection — per-tree, then merge (NOVELTY 1)
[L815] selected = self.multi_tree_knapsack(
[L816] candidate_nodes,
[L817] budget=self.config.token_budget
[L818])
[L819]
[L820] # Step 6: Assemble context in semantic order
[L821] context_text = self.assemble_context(selected, intent)
[L822]
[L823] # Build result with traversal path for Layer 8 feedback
[L824] result = RetrievalResult(
[L825] selected_nodes=selected,
[L826] context_text=context_text,
[L827] traversal_path_edges=self._get_path_edges(selected),
[L828] query_complexity=complexity,
[L829] query_intent=intent,
[L830] seed_node_ids=[s.id for s in seed_nodes],
[L831] token_count=estimate_tokens(context_text)
[L832])
[L833]
[L834] # Cache result
[L835] self.cache.cache_query_result(cache_key, result, ttl=3600)
[L836]
[L837] return result
[L838]
[L839] def multi_tree_knapsack(self, candidate_nodes, budget):
[L840] """Apply Tree Knapsack per distinct tree, then merge results.
[L841]
[L842] For multi-seed queries, candidates may span multiple trees.
[L843] We optimize each tree independently, then merge within budget.
```

---

DEEP RESEARCH 20/08/2026, 15:58:35

Citation Marker: `filecite[turn26file9]`

```
[L844] metadata:
[L845] query_result_index: 9
[L846] file_id: file_0000000033088207b45d2ae58cbd406c
[L847] version_id: 1
[L848] name: SHIA_RAG_Complete_Report(8).md
[L849] mime_type: text/markdown
[L850] surface: library
[L851] score: 0.01821162444113264
[L852] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--49
[L853] snippet:
```

```

[L854] join(parts)
[L855]
[L856] def _get_path_edges(self, selected_nodes):
[L857] """Get traversal path edges for Layer 8 feedback."""
[L858] edges = []
[L859] for node in selected_nodes:
[L860] current = node
[L861] while current.parent:
[L862] edge = self.forest.get_edge(current.parent, current.id)
[L863] if edge:
[L864] edges.append(edge)
[L865] current = self.forest.get_node(current.parent)
[L866] return edges
[L867]
[L868] def classify_complexity(intent: str) -> str:
[L869] """Map query intent to traversal complexity."""
[L870] mapping = {
[L871] 'DEFINITION': 'SIMPLE',
[L872] 'COMPARISON': 'MODERATE',
[L873] 'EXPLANATION': 'MODERATE',
[L874] 'MULTI_HOP': 'COMPLEX',
[L875] 'SYNTHESIS': 'COMPLEX',
[L876] }
[L877] return mapping.get(intent, 'MODERATE')
[L878]

```

```

[L879]
[L880] ---
[L881]
[L882] ### 7.8 Layer 7: Generation Engine
[L883]
[L884] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims from the re
[L885]
[L886] **Input:** `RetrievalResult` from Layer 6
[L887] **Output:** `GenerationResult` with verified answer, citations, and confidence
[L888]
[L889] #### LLM Configuration
[L890]
[L891]

```

python

```

[L892] GENERATION_CONFIG = {
[L893] 'model': 'gpt-4o-mini', # Primary model (configurable)
[L894] 'fallback_model': 'llama-3.1-8b', # Fallback for cost/availability
[L895] 'temperature': 0.3, # Low temperature for factual answers
[L896] 'max_output_tokens': 2048,
[L897] 'top_p': 0.9,
[L898] 'timeout_seconds': 30,
[L899] 'max_retries': 3,
[L900] 'response_format': 'json', # Force structured output

```

[L901] }

[L902]

```
[L903]
[L904] ##### Prompt Template
[L905]
[L906]
```

python

[L907] SYSTEM\_PROMPT = """You are a knowledge-grounded question answering system.

[L908] RULES:

[L909] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.

[L910] 2. For each claim in your answer, cite the source using [NODE\_ID] format.

[L911] 3. If evidence is insufficient, say "Insufficient evidence to answer."

[L912] 4. If evidence conflicts, present both views with citations.

[L913] 5. Structure your response as JSON with the schema below.

[L914]

[L915] REQUIRED OUTPUT FORMAT:

[L916] {

[L917] "answer": "Your complete answer text with [NODE\_ID] citations inline.",

[L918] "claims": [

[L919] {"text": "Individual factual claim", "evidence\_ids": ["NODE\_ID1"], "confidence": 0.0-1.0}

[L920] ],

[L921] "overall\_confidence": 0.0-1.0,

[L922] "insufficient\_evidence": false

[L923] }

[L924] """

[L925]

[L926] def build\_prompt(query: str, retrieval\_result) -> str:

[L927] """Build the complete LLM prompt from query + retrieved context."""

[L928] context\_section = retrieval\_result.context\_text

[L929]

[L930] return f"""{SYSTEM\_PROMPT}

[L931]

[L932] QUESTION: {query}

[L933]

[L934] EVIDENCE (use ONLY this to answer):

[L935] {context\_section}

[L936]

[L937] Respond with valid JSON matching the schema above."""

[L938]

```
[L939]
[L940] ##### Generation Pipeline
[L941]
[L942]
```

python

[L943] import json

[L944] from dataclasses import dataclass

[L945]  
[L946] @dataclass  
[L947] class VerifiedClaim:  
[L948] """A single claim from the LLM answer, verified against evidence."""  
[L949] text: str  
[L950] evidence\_ids: list # NODE\_IDs supporting this claim  
[L951] verification\_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING  
[L952] confidence: float # Claim-level confidence  
[L953] citation: str # Formatted citation string

---

DEEP RESEARCH 20/08/2026, 15:58:39

Citation Marker: `filecite[turn27file0]`

[L1] metadata:  
[L2] ref\_id: turn26file9  
[L3] file\_id: file\_0000000033088207b45d2ae58cbd406c  
[L4] version\_id: 1  
[L5] name: SHIA\_RAG\_Complete\_Report(8).md  
[L6] lines\_shown: 882-1301  
[L7] total\_file\_lines: 3316  
[L8] has\_more: True  
[L9] next\_start\_line: 1302  
[L10] next\_read: {"ref\_id":"turn26file9","mode":"full","start\_line":1302,"max\_lines":420}  
[L11] content:  
[L12] "dimension": 768,  
[L13] "vector": [0.123, -0.456, "..."],  
[L14] "created\_at": "2026-08-17T10:00:00Z"  
[L15] }  
[L16] ],  
[L17] "primary\_embedding\_model": "bge-base-en-v1.5",  
[L18] "confidence": 0.92,  
[L19] "base\_confidence": 0.5,  
[L20] "extraction\_confidence": 0.88,  
[L21] "abstraction\_level": 0.35,  
[L22] "statistics": {  
[L23] "access\_count": 0,  
[L24] "update\_count": 1,  
[L25] "merge\_count": 0,  
[L26] "last\_access": null  
[L27] },  
[L28] "version": {  
[L29] "current": 1,  
[L30] "revisions": []  
[L31] }  
[L32] }  
[L33]

```
[L34]
[L35] > [!WARNING]
[L36] > **Schema Design Note – `parent` and `children` redundancy:** The `parent` and `children` fields are
[L37] >
```

python

```
[L38] > @property
[L39] > def parent(self) -> Optional[str]:
[L40] > hierarchical_in = [e for e in self.incoming_edges if e.edge_class == "HIERARCHICAL"]
[L41] > return hierarchical_in[0].source_node if hierarchical_in else None
[L42] >
```

```
[L43] > Consider removing `parent`/`children` fields in a future schema version.
[L44]
[L45] ##### KnowledgeEdge (Two Types)
[L46]
[L47]
```

json

```
[L48] {
[L49] "edge_id": "EDGE_001",
[L50] "source_node": "NODE_000789",
[L51] "target_node": "NODE_000456",
[L52] "edge_class": "HIERARCHICAL",
[L53] "relation_type": "IS_A",
[L54] "confidence": 0.95,
[L55] "weight": 1.0,
[L56] "evidence": [...],
[L57] "metadata": {"source_docs": ["DOC_001"]}
[L58] }
[L59]
```

```
[L60]
[L61]
```

json

```
[L62] {
[L63] "edge_id": "EDGE_002",
[L64] "source_node": "NODE_000789",
[L65] "target_node": "NODE_000890",
[L66] "edge_class": "SEMANTIC",
[L67] "relation_type": "USES",
[L68] "confidence": 0.87,
[L69] "weight": 1.0,
[L70] "evidence": [...],
[L71] "metadata": {"source_docs": ["DOC_001", "DOC_003"]}
[L72] }
[L73]
```



```

[L74]
[L75] > [!IMPORTANT]
[L76] > **Two edge classes solve the Tree-vs-Graph contradiction:**
[L77] > - `HIERARCHICAL` edges form an acyclic forest (enforced by HV)
[L78] > - `SEMANTIC` edges form an unrestricted graph (cycles allowed)
[L79]
[L80]

```

python

```

[L81] # Edge class enforcement — applied consistently in all modules
[L82] def validate_edge(edge, forest):
[L83] """Validate edge based on its class."""
[L84] if edge.edge_class == "HIERARCHICAL":
[L85] # MUST be acyclic — check for cycles
[L86] if creates_cycle(edge.source_node, edge.target_node, forest):
[L87] raise CycleError(f"HIERARCHICAL edge {edge.edge_id} would create a cycle")
[L88] # MUST respect single-parent constraint
[L89] validate_single_parent(forest.get_node(edge.target_node), edge.source_node, forest)
[L90] elif edge.edge_class == "SEMANTIC":
[L91] # Cycles are ALLOWED — no acyclicity check needed
[L92] # Just validate that both nodes exist
[L93] assert forest.has_node(edge.source_node)
[L94] assert forest.has_node(edge.target_node)
[L95] else:
[L96] raise ValueError(f"Unknown edge_class: {edge.edge_class}. Must be HIERARCHICAL or SEMANTIC.")
[L97]

```

```

[L98]
[L99] ---
[L100]
[L101] ### 7.2 Layer 1: Input Processing
[L102]
[L103] **Responsibility:** Accept raw documents (PDF, DOCX, HTML, TXT), validate format, extract text, and p
[L104]
[L105] **Input:** Raw document file
[L106] **Output:** `NormalizedDocument` object
[L107]
[L108]

```

python

```

[L109] from dataclasses import dataclass, field
[L110] from enum import Enum
[L111] from typing import Optional
[L112]
[L113] class DocumentFormat(Enum):
[L114] PDF = "pdf"
[L115] DOCX = "docx"
[L116] HTML = "html"
[L117] TXT = "txt"
[L118]
[L119] @dataclass
[L120] class NormalizedDocument:

```

```

[L121] """Output of Layer 1 — ready for Layer 2 parsing."""
[L122] doc_id: str
[L123] original_filename: str
[L124] format: DocumentFormat
[L125] text: str # Full extracted text
[L126] pages: list # List of page texts (for PDFs)
[L127] metadata: dict # Author, title, date, etc.
[L128] is_ocr: bool = False # True if OCR was used
[L129] ocr_confidence: float = 1.0 # Average OCR confidence score
[L130] char_count: int = 0
[L131] page_count: int = 0
[L132]
[L133] # Technology choices
[L134] EXTRACTION_CONFIG = {
[L135] 'pdf_library': 'PyMuPDF (fitz)', # Primary PDF text extraction
[L136] 'pdf_fallback': 'pdfplumber', # Fallback for complex layouts
[L137] 'ocr_engine': 'Tesseract OCR', # OCR for scanned documents
[L138] 'ocr_trigger': 'auto', # 'auto' = detect scanned pages
[L139] 'ocr_min_text_ratio': 0.1, # If extracted_chars / page_area < 0.1, trigger OCR
[L140] 'ocr_min_confidence': 0.6, # Below this, flag for manual review
[L141] 'docx_library': 'python-docx',
[L142] 'html_library': 'BeautifulSoup4',
[L143] 'max_document_size_mb': 100,
[L144] 'max_pages': 5000,
[L145] 'supported_formats': ['pdf', 'docx', 'html', 'txt'],
[L146] }
[L147]
[L148] def layer1_process(file_path: str) -> NormalizedDocument:
[L149] """Layer 1: Input Processing pipeline."""
[L150] # Step 1: Validate file
[L151] fmt = detect_format(file_path)
[L152] if fmt not in EXTRACTION_CONFIG['supported_formats']:
[L153] raise UnsupportedFormatError(f"Format {fmt} not supported")
[L154]
[L155] file_size_mb = os.path.getsize(file_path) / (1024 * 1024)
[L156] if file_size_mb > EXTRACTION_CONFIG['max_document_size_mb']:
[L157] raise FileTooLargeError(f"File size {file_size_mb}MB exceeds limit")
[L158]
[L159] # Step 2: Extract text based on format
[L160] if fmt == 'pdf':
[L161] text, pages, is_ocr, ocr_conf = extract_pdf(file_path)
[L162] elif fmt == 'docx':
[L163] text, pages = extract_docx(file_path), []
[L164] is_ocr, ocr_conf = False, 1.0
[L165] elif fmt == 'html':
[L166] text, pages = extract_html(file_path), []

```

```
[L 167] is_ocr, ocr_conf = False, 1.0
[L 168] else: # txt
[L 169] text = open(file_path, 'r', encoding='utf-8').read()
[L 170] pages, is_ocr, ocr_conf = [], False, 1.0
[L 171]
[L 172] # Step 3: Clean text (remove headers/footers, normalize whitespace)
[L 173] text = clean_text(text)
[L 174]
[L 175] # Step 4: Extract metadata
[L 176] metadata = extract_metadata(file_path, fmt)
[L 177]
[L 178] return NormalizedDocument(
[L 179] doc_id=generate_doc_id(),
[L 180] original_filename=os.path.basename(file_path),
[L 181] format=DocumentFormat(fmt),
[L 182] text=text, pages=pages,
[L 183] metadata=metadata,
[L 184] is_ocr=is_ocr, ocr_confidence=ocr_conf,
[L 185] char_count=len(text), page_count=len(pages) or 1
[L 186])
[L 187]
[L 188] def extract_pdf(file_path: str):
[L 189] """Extract text from PDF, with automatic OCR fallback."""
[L 190] import fitz # PyMuPDF
[L 191] doc = fitz.open(file_path)
[L 192] pages = []
[L 193] is_ocr = False
[L 194]
[L 195] for page in doc:
[L 196] text = page.get_text()
[L 197] # Detect scanned page: low text-to-area ratio
[L 198] if len(text.strip()) / max(page.rect.width, page.rect.height, 1) < EXTRACTION_CONFIG['ocr_min_text_ratio']:
[L 199] # OCR fallback
[L 200] pix = page.get_pixmap()
[L 201] text = pytesseract.image_to_string(pix)
[L 202] is_ocr = True
[L 203] pages.append(text)
[L 204]
[L 205] full_text = '\n\n'.join(pages)
[L 206] ocr_conf = 1.0 if not is_ocr else estimate_ocr_confidence(pages)
[L 207]
[L 208] if is_ocr and ocr_conf < EXTRACTION_CONFIG['ocr_min_confidence']:
[L 209] raise OCRLowConfidenceError(f"OCR confidence {ocr_conf:.2f} below threshold")
[L 210]
[L 211] return full_text, pages, is_ocr, ocr_conf
[L 212]
```

```

[L213]
[L214] **Error handling:** See Section 7.7 for `OCRFailure`, `UnsupportedFormatError`, and retry logic.
[L215]
[L216] ---
[L217]
[L218] ### 7.3 Layer 2: Document Parsing
[L219]
[L220] **Responsibility:** Parse normalized text into a structured `[Page → Region → Block]` hierarchy with
[L221]
[L222] **Input:** `NormalizedDocument` from Layer 1
[L223] **Output:** List of `DocumentBlock` objects
[L224]
[L225]

```

python

```

[L226] from enum import Enum
[L227] from dataclasses import dataclass
[L228]
[L229] class BlockType(Enum):
[L230] HEADING = "heading"
[L231] PARAGRAPH = "paragraph"
[L232] LIST = "list"
[L233] TABLE = "table"
[L234] CODE = "code"
[L235] ALGORITHM = "algorithm"
[L236] FIGURE_CAPTION = "figure_caption"
[L237] EQUATION = "equation"
[L238] DEFINITION = "definition"
[L239]
[L240] @dataclass
[L241] class DocumentBlock:
[L242] """Atomic unit of document structure."""
[L243] block_id: str
[L244] doc_id: str
[L245] page_number: int
[L246] region_index: int # Which region on the page
[L247] block_type: BlockType
[L248] text: str
[L249] reading_order: int # Global ordering across the document
[L250] bounding_box: Optional[dict] = None # For PDF: {x0, y0, x1, y1}
[L251] confidence: float = 1.0 # Classification confidence
[L252]
[L253] def layer2_parse(doc: NormalizedDocument) -> list:
[L254] """Layer 2: Document Parsing pipeline."""
[L255] blocks = []
[L256] global_order = 0
[L257]
[L258] for page_num, page_text in enumerate(doc.pages or [doc.text]):
[L259] # Step 1: Segment page into regions (contiguous content areas)
[L260] regions = segment_into_regions(page_text)

```

```
[L261]
[L262] for region_idx, region in enumerate(regions):
[L263] # Step 2: Split region into blocks
[L264] raw_blocks = split_into_blocks(region)
[L265]
[L266] for raw_block in raw_blocks:
[L267] # Step 3: Classify block type
[L268] block_type, conf = classify_block(raw_block)
[L269]
[L270] blocks.append(DocumentBlock(
[L271] block_id=f"BLK_{len(blocks):06d}",
[L272] doc_id=doc.doc_id,
[L273] page_number=page_num + 1,
[L274] region_index=region_idx,
[L275] block_type=block_type,
[L276] text=raw_block.strip(),
[L277] reading_order=global_order,
[L278] confidence=conf
[L279]))
[L280] global_order += 1
[L281]
[L282] return blocks
[L283]
[L284] def classify_block(text: str) -> tuple:
[L285] """Classify a text block into a BlockType using rule-based heuristics."""
[L286] text_stripped = text.strip()
[L287]
[L288] # Heading detection: short line, no ending punctuation, often capitalized
[L289] if len(text_stripped) < 100 and not text_stripped.endswith('.') and text_stripped[0].isupper():
[L290] if text_stripped.startswith('#') or text_stripped.isupper():
[L291] return BlockType.HEADING, 0.9
[L292]
[L293] # List detection
[L294] if any(text_stripped.startswith(p) for p in ['- ', '* ', '1.', '2.', 'a)', 'b)']):
[L295] return BlockType.LIST, 0.85
[L296]
[L297] # Table detection (contains tab-separated or pipe-separated data)
[L298] if '|' in text_stripped and text_stripped.count('|') >= 3:
[L299] return BlockType.TABLE, 0.8
[L300]
[L301] # Code/algorithm detection
[L302] if any(kw in text_stripped for kw in ['def ', 'class ', 'import ', 'for ', 'while ', 'if ']):
[L303] return BlockType.CODE, 0.75
[L304]
[L305] # Definition detection (contains "is a", "is defined as", "refers to")
[L306] if any(p in text_stripped.lower() for p in ['is defined as', 'refers to', 'is a type of']):
```

```

[L307] return BlockType.DEFINITION, 0.8
[L308]
[L309] # Default: paragraph
[L310] return BlockType.PARAGRAPH, 0.7
[L311]
[L312] def segment_into_regions(page_text: str) -> list:
[L313] """Split a page into logical regions based on whitespace gaps."""
[L314] regions = []
[L315] current_region = []
[L316]
[L317] for line in page_text.split('\n'):
[L318] if line.strip() == "" and current_region:
[L319] regions.append('\n'.join(current_region))
[L320] current_region = []
[L321] else:
[L322] current_region.append(line)
[L323]
[L324] if current_region:
[L325] regions.append('\n'.join(current_region))
[L326]
[L327] return regions
[L328]

```

```

[L329]
[L330] ---
[L331]
[L332] ### 7.4 Layer 3: Knowledge Extraction
[L333]
[L334] **Responsibility:** Extract concepts, definitions, and relations from document blocks to produce `KnowledgeUnit` objects.
[L335]
[L336] > [!IMPORTANT]
[L337] > **Layer 3 / Layer 4 boundary:** Layer 3 performs **candidate extraction** – it identifies entities,
[L338]
[L339] **Input:** List of `DocumentBlock` from Layer 2
[L340] **Output:** List of `KnowledgeNode` objects (canonical, deduplicated)
[L341]
[L342]

```

python

```

[L343] from dataclasses import dataclass
[L344]
[L345] @dataclass
[L346] class KnowledgeUnit:
[L347] """Atomic fact extracted from a single document block.
[L348] Intermediate representation between Layer 3 extraction and Layer 5 hierarchy."""
[L349] unit_id: str
[L350] concept_name: str # Raw extracted concept name
[L351] aliases: list # Alternative names found in text
[L352] definitions: list # Definition sentences
[L353] properties: dict # Key-value properties (typed)
[L354] raw_relations: list # Candidate relations: [(relation_type, target_concept, evidence_text)]

```

```

[L355] source_doc: str
[L356] source_block: str
[L357] page: int
[L358] extraction_confidence: float # LLM extraction confidence
[L359] domain_hint: str # LLM-suggested domain
[L360]
[L361] # --- Extraction Pipeline ---
[L362]
[L363] EXTRACTION_PROMPT = """You are a knowledge extraction system. Given a text block from a document,
extract:
[L364]
[L365] 1. Concepts: Named entities, technical terms, or ideas discussed
[L366] 2. Definitions: If the text defines any concept, extract the definition
[L367] 3. Properties: Key attributes with types and values
[L368] 4. Relations: How concepts relate (IS_A, PART_OF, USES, CAUSES, etc.)
[L369] 5. Domain: The subject area (e.g., "Operating Systems", "Networking")
[L370]
[L371] Respond in JSON format:
[L372] {
[L373] "concepts": [{"name": "...", "aliases": [...], "definitions": [...], "properties": {"key": {"type": "...", "value": "..."}}}],
[L374] "relations": [{"source": "...", "type": "IS_A|PART_OF|USES|CAUSES|...", "target": "...", "evidence": "...}],
[L375] "domain": "...
[L376] }
[L377]
[L378] TEXT BLOCK:
[L379] {text}
[L380] """
[L381]
[L382] def layer3_extract(blocks: list, llm_client) -> list:
[L383] """Layer 3: Knowledge Extraction pipeline."""
[L384] knowledge_units = []
[L385]
[L386] for block in blocks:
[L387] if block.block_type in (BlockType.HEADING, BlockType.FIGURE_CAPTION):
[L388] continue # Skip non-content blocks
[L389]
[L390] # Step 1: LLM-based extraction
[L391] prompt = EXTRACTION_PROMPT.format(text=block.text)
[L392] try:
[L393] response = llm_client.query(prompt, response_format="json")
[L394] extracted = json.loads(response)
[L395] except (json.JSONDecodeE

```

[L1] metadata:  
 [L2] query\_result\_index: 0  
 [L3] file\_id: file\_000000003c8081fab658b1752549d64  
 [L4] version\_id: 1  
 [L5] name: SHIA\_RAG\_Complete\_Report(5).md  
 [L6] mime\_type: text/markdown  
 [L7] surface: library  
 [L8] score: 0.01848549283215584  
 [L9] document\_chunk\_id: file\_000000003c8081fab658b1752549d64--1--50  
 [L10] snippet:  
 [L11] ##### Prompt Template  
 [L12]  
 [L13]

```
[L14] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L15] RULES:
[L16] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L17] 2. For each claim in your answer, cite the source using [NODE_ID] format.
[L18] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L19] 4. If evidence conflicts, present both views with citations.
[L20] 5. Structure your response as JSON with the schema below.
[L21]
[L22] REQUIRED OUTPUT FORMAT:
[L23] {
[L24] "answer": "Your complete answer text with [NODE_ID] citations inline.",
[L25] "claims": [
[L26] {"text": "Individual factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}
[L27]],
[L28] "overall_confidence": 0.0-1.0,
[L29] "insufficient_evidence": false
[L30] }
[L31] """
[L32]
[L33] def build_prompt(query: str, retrieval_result) -> str:
[L34] """Build the complete LLM prompt from query + retrieved context."""
[L35] context_section = retrieval_result.context_text
[L36]
[L37] return f"""{SYSTEM_PROMPT}
[L38]
[L39] QUESTION: {query}
[L40]
[L41] EVIDENCE (use ONLY this to answer):
[L42] {context_section}
[L43]
[L44] Respond with valid JSON matching the schema above."""
[L45]
```

[L46]  
 [L47] ##### Generation Pipeline  
 [L48]  
 [L49] ```python  
 [L50] import json  
 [L51] from dataclasses import dataclass  
 [L52]  
 [L53] @dataclass  
 [L54] class VerifiedClaim:



```

[L55] """A single claim from the LLM answer, verified against evidence."""
[L56] text: str
[L57] evidence_ids: list # NODE_IDs supporting this claim
[L58] verification_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING
[L59] confidence: float # Claim-level confidence
[L60] citation: str # Formatted citation string
[L61]
[L62] @dataclass
[L63] class GenerationResult:
[L64] """Output of Layer 7 — ready for user and Layer 8."""
[L65] answer: str # Final answer text with citations
[L66] claims: list # List of VerifiedClaim
[L67] citations: list # Formatted citation list
[L68] overall_confidence: float # Answer-level confidence
[L69] retrieval_result: object # Preserved for Layer 8 feedback
[L70] generation_metadata: dict # Model used, tokens, latency
[L71]
[L72] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L73] """Layer 7: Generation Engine — complete pipeline."""
[L74]
[L75] # --- Edge case: Empty context ---
[L76] if not retrieval_result.selected_nodes:
[L77] return GenerationResult(
[L78] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L79] claims=[], citations=[],
[L80] overall_confidence=0.0,
[L81] retrieval_result=retrieval_result,
[L82] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L83])
[L84]
[L85] # Step 1: Build prompt
[L86] prompt = build_prompt(query, retrieval_result)
[L87]
[L88] # Step 2: Invoke LLM with structured output
[L89] try:
[L90] raw_response = llm_client.generate(
[L91] prompt,
[L92] model=GENERATION_CONFIG['model'],
[L93] temperature=GENERATION_CONFIG['temperature'],
[L94] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L95] response_format='json',
[L96] timeout=GENERATION_CONFIG['timeout_seconds']
[L97])
[L98] parsed = json.loads(raw_response)
[L99] except json.JSONDecodeError:
[L100] # Retry with explicit JSON instruction

```

```
[L101] raw_response = llm_client.generate(
[L102] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L103] model=GENERATION_CONFIG['model'],
[L104] temperature=0.1, # Lower temperature for retry
[L105] response_format='json'
[L106])
[L107] parsed = json.loads(raw_response)
[L108] except LLMLTimeout:
[L109] # Fallback to smaller model
[L110] raw_response = llm_client.generate(
[L111] prompt,
```

---

DEEP RESEARCH 20/08/2026, 15:58:43

Citation Marker: `filecite`turn28file1

```
[L112] metadata:
[L113] query_result_index: 1
[L114] file_id: file_0000000033088207b45d2ae58cbd406c
[L115] version_id: 1
[L116] name: SHIA_RAG_Complete_Report(8).md
[L117] mime_type: text/markdown
[L118] surface: library
[L119] score: 0.018221082467269535
[L120] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--50
[L121] snippet:
[L122] ##### Prompt Template
[L123]
[L124]
```

```

[L125] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L126] RULES:
[L127] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L128] 2. For each claim in your answer, cite the source using [NODE_ID] format.
[L129] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L130] 4. If evidence conflicts, present both views with citations.
[L131] 5. Structure your response as JSON with the schema below.
[L132]
[L133] REQUIRED OUTPUT FORMAT:
[L134] {
[L135] "answer": "Your complete answer text with [NODE_ID] citations inline.",
[L136] "claims": [
[L137] {"text": "Individual factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}
[L138]],
[L139] "overall_confidence": 0.0-1.0,
[L140] "insufficient_evidence": false
[L141] }
[L142] """
[L143]
[L144] def build_prompt(query: str, retrieval_result) -> str:
[L145] """Build the complete LLM prompt from query + retrieved context."""
[L146] context_section = retrieval_result.context_text
[L147]
[L148] return f"""{SYSTEM_PROMPT}
[L149]
[L150] QUESTION: {query}
[L151]
[L152] EVIDENCE (use ONLY this to answer):
[L153] {context_section}
[L154]
[L155] Respond with valid JSON matching the schema above."""
[L156]

```

[L157]

[L158] ##### Generation Pipeline

[L159]

[L160] ```python

[L161] import json

[L162] from dataclasses import dataclass

[L163]

[L164] @dataclass

[L165] class VerifiedClaim:

[L166] """A single claim from the LLM answer, verified against evidence."""

[L167] text: str

[L168] evidence\_ids: list # NODE\_IDs supporting this claim

[L169] verification\_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING

[L170] confidence: float # Claim-level confidence

[L171] citation: str # Formatted citation string

[L172]

[L173] @dataclass

[L174] class GenerationResult:

[L175] """Output of Layer 7 — ready for user and Layer 8."""

[L176] answer: str # Final answer text with citations

[L177] claims: list # List of VerifiedClaim

[L178] citations: list # Formatted citation list

```
[L179] overall_confidence: float # Answer-level confidence
[L180] retrieval_result: object # Preserved for Layer 8 feedback
[L181] generation_metadata: dict # Model used, tokens, latency
[L182]
[L183] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L184] """Layer 7: Generation Engine — complete pipeline."""
[L185]
[L186] # --- Edge case: Empty context ---
[L187] if not retrieval_result.selected_nodes:
[L188] return GenerationResult(
[L189] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L190] claims=[], citations=[],
[L191] overall_confidence=0.0,
[L192] retrieval_result=retrieval_result,
[L193] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L194])
[L195]
[L196] # Step 1: Build prompt
[L197] prompt = build_prompt(query, retrieval_result)
[L198]
[L199] # Step 2: Invoke LLM with structured output
[L200] try:
[L201] raw_response = llm_client.generate(
[L202] prompt,
[L203] model=GENERATION_CONFIG['model'],
[L204] temperature=GENERATION_CONFIG['temperature'],
[L205] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L206] response_format='json',
[L207] timeout=GENERATION_CONFIG['timeout_seconds']
[L208])
[L209] parsed = json.loads(raw_response)
[L210] except json.JSONDecodeError:
[L211] # Retry with explicit JSON instruction
[L212] raw_response = llm_client.generate(
[L213] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L214] model=GENERATION_CONFIG['model'],
[L215] temperature=0.1, # Lower temperature for retry
[L216] response_format='json'
[L217])
[L218] parsed = json.loads(raw_response)
[L219] except LLMTimeout:
[L220] # Fallback to smaller model
[L221] raw_response = llm_client.generate(
[L222] prompt,
```

Citation Marker: `[[filecite]]turn28file2[[`

[L223] metadata:

[L224] query\_result\_index: 2

[L225] file\_id: file\_00000000a0a481fab4c739c73d87551b

[L226] version\_id: 1

[L227] name: SHIA\_RAG\_Complete\_Report(7).md

[L228] mime\_type: text/markdown

[L229] surface: library

[L230] score: 0.018221082467269535

[L231] document\_chunk\_id: file\_00000000a0a481fab4c739c73d87551b--1--50

[L232] snippet:

[L233] ##### Prompt Template

[L234]

[L235]

```
[L236] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L237] RULES:
[L238] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L239] 2. For each claim in your answer, cite the source using [NODE_ID] format.
[L240] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L241] 4. If evidence conflicts, present both views with citations.
[L242] 5. Structure your response as JSON with the schema below.
[L243]
[L244] REQUIRED OUTPUT FORMAT:
[L245] {
[L246] "answer": "Your complete answer text with [NODE_ID] citations inline.",
[L247] "claims": [
[L248] {"text": "Individual factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}
[L249]],
[L250] "overall_confidence": 0.0-1.0,
[L251] "insufficient_evidence": false
[L252] }
[L253] """
[L254]
[L255] def build_prompt(query: str, retrieval_result) -> str:
[L256] """Build the complete LLM prompt from query + retrieved context."""
[L257] context_section = retrieval_result.context_text
[L258]
[L259] return f"""{SYSTEM_PROMPT}
[L260]
[L261] QUESTION: {query}
[L262]
[L263] EVIDENCE (use ONLY this to answer):
[L264] {context_section}
[L265]
[L266] Respond with valid JSON matching the schema above."""
[L267]
```

[L268]

[L269] ##### Generation Pipeline

[L270]

[L271] ```python

[L272] import json

[L273] from dataclasses import dataclass

[L274]

[L275] @dataclass

```

[L276] class VerifiedClaim:
[L277] """A single claim from the LLM answer, verified against evidence."""
[L278] text: str
[L279] evidence_ids: list # NODE_IDs supporting this claim
[L280] verification_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING
[L281] confidence: float # Claim-level confidence
[L282] citation: str # Formatted citation string
[L283]
[L284] @dataclass
[L285] class GenerationResult:
[L286] """Output of Layer 7 — ready for user and Layer 8."""
[L287] answer: str # Final answer text with citations
[L288] claims: list # List of VerifiedClaim
[L289] citations: list # Formatted citation list
[L290] overall_confidence: float # Answer-level confidence
[L291] retrieval_result: object # Preserved for Layer 8 feedback
[L292] generation_metadata: dict # Model used, tokens, latency
[L293]
[L294] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L295] """Layer 7: Generation Engine — complete pipeline."""
[L296]
[L297] # --- Edge case: Empty context ---
[L298] if not retrieval_result.selected_nodes:
[L299] return GenerationResult(
[L300] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L301] claims=[], citations=[],
[L302] overall_confidence=0.0,
[L303] retrieval_result=retrieval_result,
[L304] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L305])
[L306]
[L307] # Step 1: Build prompt
[L308] prompt = build_prompt(query, retrieval_result)
[L309]
[L310] # Step 2: Invoke LLM with structured output
[L311] try:
[L312] raw_response = llm_client.generate(
[L313] prompt,
[L314] model=GENERATION_CONFIG['model'],
[L315] temperature=GENERATION_CONFIG['temperature'],
[L316] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L317] response_format='json',
[L318] timeout=GENERATION_CONFIG['timeout_seconds']
[L319])
[L320] parsed = json.loads(raw_response)
[L321] except json.JSONDecodeError:

```

```
[L322] # Retry with explicit JSON instruction
[L323] raw_response = llm_client.generate(
[L324] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L325] model=GENERATION_CONFIG['model'],
[L326] temperature=0.1, # Lower temperature for retry
[L327] response_format='json'
[L328])
[L329] parsed = json.loads(raw_response)
[L330] except LLMLTimeout:
[L331] # Fallback to smaller model
[L332] raw_response = llm_client.generate(
[L333] prompt,
```

---

DEEP RESEARCH 20/08/2026, 15:58:43

Citation Marker: □filecite□turn28file3□

```
[L334] metadata:
[L335] query_result_index: 3
[L336] file_id: file_000000001dd481fa9855f2731dbc53d5
[L337] version_id: 1
[L338] name: SHIA_RAG_Complete_Report(6).md
[L339] mime_type: text/markdown
[L340] surface: library
[L341] score: 0.01771705020920502
[L342] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--50
[L343] snippet:
[L344] ##### Prompt Template
[L345]
[L346]
```

```

[L347] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L348] RULES:
[L349] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L350] 2. For each claim in your answer, cite the source using [NODE_ID] format.
[L351] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L352] 4. If evidence conflicts, present both views with citations.
[L353] 5. Structure your response as JSON with the schema below.
[L354]
[L355] REQUIRED OUTPUT FORMAT:
[L356] {
[L357] "answer": "Your complete answer text with [NODE_ID] citations inline.",
[L358] "claims": [
[L359] {"text": "Individual factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}
[L360]],
[L361] "overall_confidence": 0.0-1.0,
[L362] "insufficient_evidence": false
[L363] }
[L364] """
[L365]
[L366] def build_prompt(query: str, retrieval_result) -> str:
[L367] """Build the complete LLM prompt from query + retrieved context."""
[L368] context_section = retrieval_result.context_text
[L369]
[L370] return f"""{SYSTEM_PROMPT}
[L371]
[L372] QUESTION: {query}
[L373]
[L374] EVIDENCE (use ONLY this to answer):
[L375] {context_section}
[L376]
[L377] Respond with valid JSON matching the schema above."""
[L378]

```

```

[L379]
[L380] ##### Generation Pipeline
[L381]
[L382] ```python
[L383] import json
[L384] from dataclasses import dataclass
[L385]
[L386] @dataclass
[L387] class VerifiedClaim:
[L388] """A single claim from the LLM answer, verified against evidence."""
[L389] text: str
[L390] evidence_ids: list # NODE_IDs supporting this claim
[L391] verification_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING
[L392] confidence: float # Claim-level confidence
[L393] citation: str # Formatted citation string
[L394]
[L395] @dataclass
[L396] class GenerationResult:
[L397] """Output of Layer 7 — ready for user and Layer 8."""
[L398] answer: str # Final answer text with citations
[L399] claims: list # List of VerifiedClaim
[L400] citations: list # Formatted citation list

```



```
[L401] overall_confidence: float # Answer-level confidence
[L402] retrieval_result: object # Preserved for Layer 8 feedback
[L403] generation_metadata: dict # Model used, tokens, latency
[L404]
[L405] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L406] """Layer 7: Generation Engine — complete pipeline."""
[L407]
[L408] # --- Edge case: Empty context ---
[L409] if not retrieval_result.selected_nodes:
[L410] return GenerationResult(
[L411] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L412] claims=[], citations=[],
[L413] overall_confidence=0.0,
[L414] retrieval_result=retrieval_result,
[L415] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L416])
[L417]
[L418] # Step 1: Build prompt
[L419] prompt = build_prompt(query, retrieval_result)
[L420]
[L421] # Step 2: Invoke LLM with structured output
[L422] try:
[L423] raw_response = llm_client.generate(
[L424] prompt,
[L425] model=GENERATION_CONFIG['model'],
[L426] temperature=GENERATION_CONFIG['temperature'],
[L427] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L428] response_format='json',
[L429] timeout=GENERATION_CONFIG['timeout_seconds']
[L430])
[L431] parsed = json.loads(raw_response)
[L432] except json.JSONDecodeError:
[L433] # Retry with explicit JSON instruction
[L434] raw_response = llm_client.generate(
[L435] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L436] model=GENERATION_CONFIG['model'],
[L437] temperature=0.1, # Lower temperature for retry
[L438] response_format='json'
[L439])
[L440] parsed = json.loads(raw_response)
[L441] except LLMTimeout:
[L442] # Fallback to smaller model
[L443] raw_response = llm_client.generate(
[L444] prompt,
```

Citation Marker: □filecite□turn28file4□

[L445] metadata:

[L446] query\_result\_index: 4

[L447] file\_id: file\_0000000033088207b45d2ae58cbd406c

[L448] version\_id: 1

[L449] name: SHIA\_RAG\_Complete\_Report(8).md

[L450] mime\_type: text/markdown

[L451] surface: library

[L452] score: 0.017476665593820405

[L453] document\_chunk\_id: file\_0000000033088207b45d2ae58cbd406c--1--51

[L454] snippet:

[L455] def layer7\_generate(query: str, retrieval\_result, llm\_client) -> GenerationResult:

[L456] """Layer 7: Generation Engine — complete pipeline."""

[L457]

[L458] # --- Edge case: Empty context ---

[L459] if not retrieval\_result.selected\_nodes:

[L460] return GenerationResult(

[L461] answer="I don't have sufficient evidence in the knowledge base to answer this question.",

[L462] claims=[], citations=[],

[L463] overall\_confidence=0.0,

[L464] retrieval\_result=retrieval\_result,

[L465] generation\_metadata={'reason': 'EMPTY\_CONTEXT'})

[L466]

[L467]

[L468] # Step 1: Build prompt

[L469] prompt = build\_prompt(query, retrieval\_result)

[L470]

[L471] # Step 2: Invoke LLM with structured output

[L472] try:

[L473] raw\_response = llm\_client.generate(

[L474] prompt,

[L475] model=GENERATION\_CONFIG['model'],

[L476] temperature=GENERATION\_CONFIG['temperature'],

[L477] max\_tokens=GENERATION\_CONFIG['max\_output\_tokens'],

[L478] response\_format='json',

[L479] timeout=GENERATION\_CONFIG['timeout\_seconds'])

[L480]

[L481] parsed = json.loads(raw\_response)

[L482] except json.JSONDecodeError:

[L483] # Retry with explicit JSON instruction

[L484] raw\_response = llm\_client.generate(

[L485] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",

[L486] model=GENERATION\_CONFIG['model'],

[L487] temperature=0.1, # Lower temperature for retry

[L488] response\_format='json')

[L489]

```
[L490] parsed = json.loads(raw_response)
[L491] except LLMLTimeout:
[L492] # Fallback to smaller model
[L493] raw_response = llm_client.generate(
[L494] prompt,
[L495] model=GENERATION_CONFIG['fallback_model'],
[L496] temperature=GENERATION_CONFIG['temperature'],
[L497] response_format='json'
[L498])
[L499] parsed = json.loads(raw_response)
[L500]
[L501] # Step 3: Extract and verify claims
[L502] node_map = {n.node_id: n for n in retrieval_result.selected_nodes}
[L503] verified_claims = []
[L504]
[L505] for claim_data in parsed.get('claims', []):
[L506] claim_text = claim_data['text']
[L507] evidence_ids = claim_data.get('evidence_ids', [])
[L508]
[L509] # Verify: does evidence actually support this claim?
[L510] status, conf = verify_claim(claim_text, evidence_ids, node_map)
[L511]
[L512] # Build citation
[L513] citation = format_citation(evidence_ids, node_map)
[L514]
[L515] verified_claims.append(VerifiedClaim(
[L516] text=claim_text,
[L517] evidence_ids=evidence_ids,
[L518] verification_status=status,
[L519] confidence=conf,
[L520] citation=citation
[L521]))
[L522]
[L523] # Step 4: Apply unsupported claim policy
[L524] answer_text = apply_claim_policy(parsed.get('answer', ''), verified_claims)
[L525]
[L526] # Step 5: Compute answer confidence
[L527] overall_confidence = compute_answer_confidence(verified_claims, retrieval_result)
[L528]
[L529] return GenerationResult(
[L530] answer=answer_text,
[L531] claims=verified_claims,
[L532] citations=[c.citation for c in verified_claims if c.verification_status != 'UNSUPPORTED'],
[L533] overall_confidence=overall_confidence,
[L534] retrieval_result=retrieval_result,
[L535] generation_metadata={
```

```

[L536] 'model': GENERATION_CONFIG['model'],
[L537] 'claim_count': len(verified_claims),
[L538] 'supported_count': sum(1 for c in verified_claims if c.verification_status == 'SUPPORTED'),
[L539] }
[L540])
[L541]
[L542] def verify_claim(claim_text: str, evidence_ids: list, node_map: dict) -> tuple:
[L543] """Verify a claim against its cited evidence using semantic entailment.
[L544]
[L545] Returns: (status, confidence)
[L546] - SUPPORTED: evidence semantically entails the claim (confidence > 0.7)
[L547] - PARTIAL: evidence partially supports (0.

```

---

DEEP RESEARCH 20/08/2026, 15:58:43

Citation Marker: `filecite`turn28file5

```

[L548] metadata:
[L549] query_result_index: 5
[L550] file_id: file_00000000a0a481fab4c739c73d87551b
[L551] version_id: 1
[L552] name: SHIA_RAG_Complete_Report(7).md
[L553] mime_type: text/markdown
[L554] surface: library
[L555] score: 0.01724356536072017
[L556] document_chunk_id: file_00000000a0a481fab4c739c73d87551b--1--51
[L557] snippet:
[L558] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L559] """Layer 7: Generation Engine — complete pipeline."""
[L560]
[L561] # --- Edge case: Empty context ---
[L562] if not retrieval_result.selected_nodes:
[L563] return GenerationResult(
[L564] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L565] claims=[], citations=[],
[L566] overall_confidence=0.0,
[L567] retrieval_result=retrieval_result,
[L568] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L569])
[L570]
[L571] # Step 1: Build prompt
[L572] prompt = build_prompt(query, retrieval_result)
[L573]
[L574] # Step 2: Invoke LLM with structured output
[L575] try:
[L576] raw_response = llm_client.generate(
[L577] prompt,

```

```
[L578] model=GENERATION_CONFIG['model'],
[L579] temperature=GENERATION_CONFIG['temperature'],
[L580] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L581] response_format='json',
[L582] timeout=GENERATION_CONFIG['timeout_seconds']
[L583])
[L584] parsed = json.loads(raw_response)
[L585] except json.JSONDecodeError:
[L586] # Retry with explicit JSON instruction
[L587] raw_response = llm_client.generate(
[L588] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L589] model=GENERATION_CONFIG['model'],
[L590] temperature=0.1, # Lower temperature for retry
[L591] response_format='json'
[L592])
[L593] parsed = json.loads(raw_response)
[L594] except LLMTimeout:
[L595] # Fallback to smaller model
[L596] raw_response = llm_client.generate(
[L597] prompt,
[L598] model=GENERATION_CONFIG['fallback_model'],
[L599] temperature=GENERATION_CONFIG['temperature'],
[L600] response_format='json'
[L601])
[L602] parsed = json.loads(raw_response)
[L603]
[L604] # Step 3: Extract and verify claims
[L605] node_map = {n.node_id: n for n in retrieval_result.selected_nodes}
[L606] verified_claims = []
[L607]
[L608] for claim_data in parsed.get('claims', []):
[L609] claim_text = claim_data['text']
[L610] evidence_ids = claim_data.get('evidence_ids', [])
[L611]
[L612] # Verify: does evidence actually support this claim?
[L613] status, conf = verify_claim(claim_text, evidence_ids, node_map)
[L614]
[L615] # Build citation
[L616] citation = format_citation(evidence_ids, node_map)
[L617]
[L618] verified_claims.append(VerifiedClaim(
[L619] text=claim_text,
[L620] evidence_ids=evidence_ids,
[L621] verification_status=status,
[L622] confidence=conf,
[L623] citation=citation
```

```

[L624]))
[L625]
[L626] # Step 4: Apply unsupported claim policy
[L627] answer_text = apply_claim_policy(parsed.get('answer', ''), verified_claims)
[L628]
[L629] # Step 5: Compute answer confidence
[L630] overall_confidence = compute_answer_confidence(verified_claims, retrieval_result)
[L631]
[L632] return GenerationResult(
[L633] answer=answer_text,
[L634] claims=verified_claims,
[L635] citations=[c.citation for c in verified_claims if c.verification_status != 'UNSUPPORTED'],
[L636] overall_confidence=overall_confidence,
[L637] retrieval_result=retrieval_result,
[L638] generation_metadata={
[L639] 'model': GENERATION_CONFIG['model'],
[L640] 'claim_count': len(verified_claims),
[L641] 'supported_count': sum(1 for c in verified_claims if c.verification_status == 'SUPPORTED'),
[L642] }
[L643])
[L644]
[L645] def verify_claim(claim_text: str, evidence_ids: list, node_map: dict) -> tuple:
[L646] """Verify a claim against its cited evidence using semantic entailment.
[L647]
[L648] Returns: (status, confidence)
[L649] - SUPPORTED: evidence semantically entails the claim (confidence > 0.7)
[L650] - PARTIAL: evidence partially supports (0.

```

---

DEEP RESEARCH 20/08/2026, 15:58:43

Citation Marker: `filecite[turn28file6]`

```

[L651] metadata:
[L652] query_result_index: 6
[L653] file_id: file_000000003c8081fab658b1752549d64
[L654] version_id: 1
[L655] name: SHIA_RAG_Complete_Report(5).md
[L656] mime_type: text/markdown
[L657] surface: library
[L658] score: 0.01701742334353338
[L659] document_chunk_id: file_000000003c8081fab658b1752549d64--1--51
[L660] snippet:
[L661] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L662] """Layer 7: Generation Engine — complete pipeline."""
[L663]
[L664] # --- Edge case: Empty context ---
[L665] if not retrieval_result.selected_nodes:

```

```
[L666] return GenerationResult(
[L667] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L668] claims=[], citations=[],
[L669] overall_confidence=0.0,
[L670] retrieval_result=retrieval_result,
[L671] generation_metadata={'reason': 'EMPTY_CONTEXT'})
[L672])
[L673]
[L674] # Step 1: Build prompt
[L675] prompt = build_prompt(query, retrieval_result)
[L676]
[L677] # Step 2: Invoke LLM with structured output
[L678] try:
[L679] raw_response = llm_client.generate(
[L680] prompt,
[L681] model=GENERATION_CONFIG['model'],
[L682] temperature=GENERATION_CONFIG['temperature'],
[L683] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L684] response_format='json',
[L685] timeout=GENERATION_CONFIG['timeout_seconds']
[L686])
[L687] parsed = json.loads(raw_response)
[L688] except json.JSONDecodeError:
[L689] # Retry with explicit JSON instruction
[L690] raw_response = llm_client.generate(
[L691] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L692] model=GENERATION_CONFIG['model'],
[L693] temperature=0.1, # Lower temperature for retry
[L694] response_format='json'
[L695])
[L696] parsed = json.loads(raw_response)
[L697] except LLMLLMTimeout:
[L698] # Fallback to smaller model
[L699] raw_response = llm_client.generate(
[L700] prompt,
[L701] model=GENERATION_CONFIG['fallback_model'],
[L702] temperature=GENERATION_CONFIG['temperature'],
[L703] response_format='json'
[L704])
[L705] parsed = json.loads(raw_response)
[L706]
[L707] # Step 3: Extract and verify claims
[L708] node_map = {n.node_id: n for n in retrieval_result.selected_nodes}
[L709] verified_claims = []
[L710]
[L711] for claim_data in parsed.get('claims', []):
```

```
[L712] claim_text = claim_data['text']
[L713] evidence_ids = claim_data.get('evidence_ids', [])
[L714]
[L715] # Verify: does evidence actually support this claim?
[L716] status, conf = verify_claim(claim_text, evidence_ids, node_map)
[L717]
[L718] # Build citation
[L719] citation = format_citation(evidence_ids, node_map)
[L720]
[L721] verified_claims.append(VerifiedClaim(
[L722] text=claim_text,
[L723] evidence_ids=evidence_ids,
[L724] verification_status=status,
[L725] confidence=conf,
[L726] citation=citation
[L727]))
[L728]
[L729] # Step 4: Apply unsupported claim policy
[L730] answer_text = apply_claim_policy(parsed.get('answer', ''), verified_claims)
[L731]
[L732] # Step 5: Compute answer confidence
[L733] overall_confidence = compute_answer_confidence(verified_claims, retrieval_result)
[L734]
[L735] return GenerationResult(
[L736] answer=answer_text,
[L737] claims=verified_claims,
[L738] citations=[c.citation for c in verified_claims if c.verification_status != 'UNSUPPORTED'],
[L739] overall_confidence=overall_confidence,
[L740] retrieval_result=retrieval_result,
[L741] generation_metadata={
[L742] 'model': GENERATION_CONFIG['model'],
[L743] 'claim_count': len(verified_claims),
[L744] 'supported_count': sum(1 for c in verified_claims if c.verification_status == 'SUPPORTED'),
[L745] }
[L746])
[L747]
[L748] def verify_claim(claim_text: str, evidence_ids: list, node_map: dict) -> tuple:
[L749] """Verify a claim against its cited evidence using semantic entailment.
[L750]
[L751] Returns: (status, confidence)
[L752] - SUPPORTED: evidence semantically entails the claim (confidence > 0.7)
[L753] - PARTIAL: evidence partially supports (0.
```



```
[L754] metadata:
[L755] query_result_index: 7
[L756] file_id: file_000000001dd481fa9855f2731dbc53d5
[L757] version_id: 1
[L758] name: SHIA_RAG_Complete_Report(6).md
[L759] mime_type: text/markdown
[L760] surface: library
[L761] score: 0.016797932562146196
[L762] document_chunk_id: file_000000001dd481fa9855f2731dbc53d5--1--51
[L763] snippet:
[L764] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L765] """Layer 7: Generation Engine — complete pipeline."""
[L766]
[L767] # --- Edge case: Empty context ---
[L768] if not retrieval_result.selected_nodes:
[L769] return GenerationResult(
[L770] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L771] claims=[], citations=[],
[L772] overall_confidence=0.0,
[L773] retrieval_result=retrieval_result,
[L774] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L775])
[L776]
[L777] # Step 1: Build prompt
[L778] prompt = build_prompt(query, retrieval_result)
[L779]
[L780] # Step 2: Invoke LLM with structured output
[L781] try:
[L782] raw_response = llm_client.generate(
[L783] prompt,
[L784] model=GENERATION_CONFIG['model'],
[L785] temperature=GENERATION_CONFIG['temperature'],
[L786] max_tokens=GENERATION_CONFIG['max_output_tokens'],
[L787] response_format='json',
[L788] timeout=GENERATION_CONFIG['timeout_seconds']
[L789])
[L790] parsed = json.loads(raw_response)
[L791] except json.JSONDecodeError:
[L792] # Retry with explicit JSON instruction
[L793] raw_response = llm_client.generate(
[L794] prompt + "\n\nIMPORTANT: Respond with valid JSON only.",
[L795] model=GENERATION_CONFIG['model'],
[L796] temperature=0.1, # Lower temperature for retry
[L797] response_format='json'
[L798])
[L799] parsed = json.loads(raw_response)
```

```
[L800] except LLMTimeout:
[L801] # Fallback to smaller model
[L802] raw_response = llm_client.generate(
[L803] prompt,
[L804] model=GENERATION_CONFIG['fallback_model'],
[L805] temperature=GENERATION_CONFIG['temperature'],
[L806] response_format='json'
[L807])
[L808] parsed = json.loads(raw_response)
[L809]
[L810] # Step 3: Extract and verify claims
[L811] node_map = {n.node_id: n for n in retrieval_result.selected_nodes}
[L812] verified_claims = []
[L813]
[L814] for claim_data in parsed.get('claims', []):
[L815] claim_text = claim_data['text']
[L816] evidence_ids = claim_data.get('evidence_ids', [])
[L817]
[L818] # Verify: does evidence actually support this claim?
[L819] status, conf = verify_claim(claim_text, evidence_ids, node_map)
[L820]
[L821] # Build citation
[L822] citation = format_citation(evidence_ids, node_map)
[L823]
[L824] verified_claims.append(VerifiedClaim(
[L825] text=claim_text,
[L826] evidence_ids=evidence_ids,
[L827] verification_status=status,
[L828] confidence=conf,
[L829] citation=citation
[L830]))
[L831]
[L832] # Step 4: Apply unsupported claim policy
[L833] answer_text = apply_claim_policy(parsed.get('answer', ''), verified_claims)
[L834]
[L835] # Step 5: Compute answer confidence
[L836] overall_confidence = compute_answer_confidence(verified_claims, retrieval_result)
[L837]
[L838] return GenerationResult(
[L839] answer=answer_text,
[L840] claims=verified_claims,
[L841] citations=[c.citation for c in verified_claims if c.verification_status != 'UNSUPPORTED'],
[L842] overall_confidence=overall_confidence,
[L843] retrieval_result=retrieval_result,
[L844] generation_metadata={
[L845] 'model': GENERATION_CONFIG['model'],
```

```

[L846] 'claim_count': len(verified_claims),
[L847] 'supported_count': sum(1 for c in verified_claims if c.verification_status == 'SUPPORTED'),
[L848] }
[L849])
[L850]
[L851] def verify_claim(claim_text: str, evidence_ids: list, node_map: dict) -> tuple:
[L852] """Verify a claim against its cited evidence using semantic entailment.
[L853]
[L854] Returns: (status, confidence)
[L855] - SUPPORTED: evidence semantically entails the claim (confidence > 0.7)
[L856] - PARTIAL: evidence partially supports (0.

```

---

DEEP RESEARCH 20/08/2026, 15:58:47

Citation Marker: `filecite[turn29file0]`

```

[L1] metadata:
[L2] ref_id: turn28file4
[L3] file_id: file_0000000033088207b45d2ae58cbd406c
[L4] version_id: 1
[L5] name: SHIA_RAG_Complete_Report(8).md
[L6] mode: chunk_context
[L7] lines_shown: 1-120
[L8] total_file_lines: 241
[L9] has_more: True
[L10] next_start_line: 121
[L11] next_read:
{"ref_id":"turn28file4","mode":"chunk_context","start_line":121,"context_before_lines":20,"context_after_lines":220}
[L12] content:
[L13] edges.append(edge)
[L14] current = self.forest.get_node(current.parent)
[L15] return edges
[L16]
[L17] def classify_complexity(intent: str) -> str:
[L18] """Map query intent to traversal complexity."""
[L19] mapping = {
[L20] 'DEFINITION': 'SIMPLE',
[L21] 'COMPARISON': 'MODERATE',
[L22] 'EXPLANATION': 'MODERATE',
[L23] 'MULTI_HOP': 'COMPLEX',
[L24] 'SYNTHESIS': 'COMPLEX',
[L25] }
[L26] return mapping.get(intent, 'MODERATE')
[L27]

```

```

[L28]
[L29] ---
[L30]
[L31] ### 7.8 Layer 7: Generation Engine
[L32]
[L33] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims from the res
[L34]
[L35] **Input:** `RetrievalResult` from Layer 6
[L36] **Output:** `GenerationResult` with verified answer, citations, and confidence
[L37]
[L38] #### LLM Configuration
[L39]
[L40]

```

python

```

[L41] GENERATION_CONFIG = {
[L42] 'model': 'gpt-4o-mini', # Primary model (configurable)
[L43] 'fallback_model': 'llama-3.1-8b', # Fallback for cost/availability
[L44] 'temperature': 0.3, # Low temperature for factual answers
[L45] 'max_output_tokens': 2048,
[L46] 'top_p': 0.9,
[L47] 'timeout_seconds': 30,
[L48] 'max_retries': 3,
[L49] 'response_format': 'json', # Force structured output
[L50] }
[L51]

```

```

[L52]
[L53] #### Prompt Template
[L54]
[L55]

```

python

```

[L56] SYSTEM_PROMPT = """You are a knowledge-grounded question answering system.
[L57] RULES:
[L58] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L59] 2. For each claim in your answer, cite the source using [NODE_ID] format.
[L60] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L61] 4. If evidence conflicts, present both views with citations.
[L62] 5. Structure your response as JSON with the schema below.
[L63]
[L64] REQUIRED OUTPUT FORMAT:
[L65] {
[L66] "answer": "Your complete answer text with [NODE_ID] citations inline.",
[L67] "claims": [
[L68] {"text": "Individual factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}
[L69]],
[L70] "overall_confidence": 0.0-1.0,
[L71] "insufficient_evidence": false
[L72] }
[L73] """

```

```

[L74]
[L75] def build_prompt(query: str, retrieval_result) -> str:
[L76] """Build the complete LLM prompt from query + retrieved context."""
[L77] context_section = retrieval_result.context_text
[L78]
[L79] return f"""{SYSTEM_PROMPT}
[L80]
[L81] QUESTION: {query}
[L82]
[L83] EVIDENCE (use ONLY this to answer):
[L84] {context_section}
[L85]
[L86] Respond with valid JSON matching the schema above."""
[L87]

```

```

[L88]
[L89] ##### Generation Pipeline
[L90]
[L91]

```

```

python
[L92] import json
[L93] from dataclasses import dataclass
[L94]
[L95] @dataclass
[L96] class VerifiedClaim:
[L97] """A single claim from the LLM answer, verified against evidence."""
[L98] text: str
[L99] evidence_ids: list # NODE_IDs supporting this claim
[L100] verification_status: str # SUPPORTED, UNSUPPORTED, PARTIAL, CONFLICTING
[L101] confidence: float # Claim-level confidence
[L102] citation: str # Formatted citation string
[L103]
[L104] @dataclass
[L105] class GenerationResult:
[L106] """Output of Layer 7 — ready for user and Layer 8."""
[L107] answer: str # Final answer text with citations
[L108] claims: list # List of VerifiedClaim
[L109] citations: list # Formatted citation list
[L110] overall_confidence: float # Answer-level confidence
[L111] retrieval_result: object # Preserved for Layer 8 feedback
[L112] generation_metadata: dict # Model used, tokens, latency
[L113]
[L114] def layer7_generate(query: str, retrieval_result, llm_client) -> GenerationResult:
[L115] """Layer 7: Generation Engine — complete pipeline."""
[L116]
[L117] # --- Edge case: Empty context ---
[L118] if not retrieval_result.selected_nodes:

```

```
[L119] return GenerationResult(
[L120] answer="I don't have sufficient evidence in the knowledge base to answer this question.",
[L121] claims=[], citations=[],
[L122] overall_confidence=0.0,
[L123] retrieval_result=retrieval_result,
[L124] generation_metadata={'reason': 'EMPTY_CONTEXT'}
[L125])
[L126]
[L127] # Step 1: Build prompt
[L128] prompt = build_prompt(query, retrieval_result)
[L129]
[L130] # Step 2: Invoke LLM with structured output
[L131] try:
[L132] raw_response = llm_client.generate(
```

---

DEEP RESEARCH 20/08/2026, 15:58:47

Citation Marker: `filecite`turn29file1

```
[L133] metadata:
[L134] read_index: 1
[L135] ref_id: turn26file0
[L136] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L137] version_id: 1
[L138] name: SHIA_RAG_Final_Report(2).md
[L139] mode: chunk_context
[L140] lines_shown: 1-120
[L141] total_file_lines: 181
[L142] has_more: True
[L143] next_start_line: 121
[L144] next_read:
{"ref_id": "turn26file0", "mode": "chunk_context", "start_line": 121, "context_before_lines": 20, "context_after_lines": 160}
[L145] content:
[L146] def utility(node, forest):
[L147] """Utility for Tree Knapsack — uses edge weights from self-evolution feedback."""
[L148] if not node.parent:
[L149] return node.confidence len(node.evidence)
[L150]
[L151] parent_edge = forest.get_edge(node.parent, node.id)
[L152] edge_weight = parent_edge.weight if parent_edge else 1.0
[L153]
[L154] return (
[L155] node.confidence # Node quality
[L156] edge_weight # Self-evolution signal
[L157] len(node.evidence) # Evidence richness
```

[L158] (1.0 / max(token\_cost(node), 1)) # Cost efficiency

[L159])

[L160]

```
[L161]
[L162] ---
[L163]
[L164] ### 6.2 Layer 7: Generation Engine
[L165]
[L166] **Responsibility:** Build a prompt from retrieved context, invoke the LLM, extract claims, verify each
[L167]
[L168] **Input:** `RetrievalResult` → **Output:** `GenerationResult`
[L169]
[L170] ##### LLM Configuration
[L171]
[L172] | Parameter | Value | Rationale |
[L173] |-----|-----|-----|
[L174] | Model | GPT-4o-mini (primary), Llama 3.1 8B (fallback) | Cost-effective with good quality |
[L175] | Temperature | 0.3 | Low for factual answers |
[L176] | Max output tokens | 2048 | Sufficient for detailed answers |
[L177] | Response format | JSON | Force structured output for claim extraction |
[L178] | Timeout | 30 seconds | With retry on timeout |
[L179]
[L180] ##### Prompt Template
[L181]
[L182]
```

python

[L183] SYSTEM\_PROMPT = """You are a knowledge-grounded question answering system.

[L184] RULES:

[L185] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.

[L186] 2. For each claim, cite the source using [NODE\_ID] format.

[L187] 3. If evidence is insufficient, say "Insufficient evidence to answer."

[L188] 4. If evidence conflicts, present both views with citations.

[L189]

[L190] REQUIRED OUTPUT FORMAT:

[L191] {

[L192] "answer": "Your answer with [NODE\_ID] citations inline.",

[L193] "claims": [{"text": "Factual claim", "evidence\_ids": ["NODE\_ID1"], "confidence": 0.0-1.0}],

[L194] "overall\_confidence": 0.0-1.0,

[L195] "insufficient\_evidence": false

[L196] }"""

[L197]

```
[L198]
[L199] ##### Claim Verification
[L200]
[L201] After the LLM generates an answer, each claim is verified against its cited evidence:
[L202]
[L203]
```

python

[L204] def verify\_claim(claim\_text, evidence\_ids, node\_map) -> tuple:

[L205] """Returns (status, confidence).

[L206] - SUPPORTED: evidence entails claim (confidence > 0.7)

```

[L207] - PARTIAL: evidence partially supports ($0.4 < \text{confidence} \leq 0.7$)
[L208] - UNSUPPORTED: no evidence supports ($\text{confidence} \leq 0.4$)
[L209] - CONFLICTING: evidence items contradict each other
[L210] """
[L211] scores = []
[L212] for eid in evidence_ids:
[L213] node = node_map.get(eid)
[L214] if node and node.evidence:
[L215] evidence_text = ' '.join(e.text for e in node.evidence)
[L216] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L217]
[L218] if not scores:
[L219] return 'UNSUPPORTED', 0.0
[L220]
[L221] avg_score = sum(scores) / len(scores)
[L222]
[L223] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L224] return 'CONFLICTING', avg_score
[L225] if avg_score > 0.7:
[L226] return 'SUPPORTED', avg_score
[L227] elif avg_score > 0.4:
[L228] return 'PARTIAL', avg_score
[L229] else:
[L230] return 'UNSUPPORTED', avg_score
[L231]

```

```

[L232]
[L233] ##### Unsupported Claim Policy
[L234]
[L235] | Status | Action |
[L236] |-----|-----|
[L237] | SUPPORTED | Keep as-is with citation |
[L238] | PARTIAL | Keep but mark as partially supported |
[L239] | UNSUPPORTED | Remove from answer, note at end |
[L240] | CONFLICTING | Present both views with note |
[L241]
[L242] ##### Answer Confidence Formula
[L243]
[L244] $\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \text{over}$
[L245]
[L246] ##### Output Schema
[L247]
[L248]

```

python

```

[L249] @dataclass
[L250] class GenerationResult:
[L251] """Output of Layer 7 — ready for user and Layer 8."""
[L252] answer: str # Answer text with citations
[L253] claims: list # List of VerifiedClaim
[L254] citations: list # Formatted citation list
[L255] overall_confidence: float # 0.0-1.0

```



[L256] retrieval\_result: object # Preserved for Layer 8

[L257] generation\_metadata: dict # Model used, claim counts, etc.

[L258] ```

[L259]

[L260] ---

[L261]

[L262] ## 7. Operations Layer — Layer 8

[L263]

[L264] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

---

DEEP RESEARCH 20/08/2026, 15:58:50

Citation Marker: `filecite[turn30file0]`

[L1] metadata:

[L2] ref\_id: turn29file0

[L3] file\_id: file\_0000000033088207b45d2ae58cbd406c

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(8).md

[L6] mode: chunk\_context

[L7] lines\_shown: 121-240

[L8] total\_file\_lines: 3316

[L9] has\_more: True

[L10] next\_start\_line: 241

[L11] next\_read:

{"ref\_id":"turn29file0","mode":"chunk\_context","start\_line":241,"context\_before\_lines":0,"context\_after\_lines":180}

[L12] warnings: ["chunk\_context requested without anchor\_text; returned the beginning of the file."]

[L13] content:

[L14]

[L15]

```

[L16] # BROKEN (current code)
[L17] - def is_valid(N, P, forest):
[L18] - stack = [P] # Start from Parent
[L19] - while stack:
[L20] - cur = stack.pop()
[L21] - for child in forest.children(cur): # Go DOWN from P
[L22] - if child == N: # Found N as descendant of P
[L23] - return False # This detects redundancy, NOT cycles
[L24] - stack.append(child)
[L25] - return True
[L26]
[L27] # FIXED
[L28] + def is_valid(N, P, forest):
[L29] + """Check: would adding edge P-N create a cycle?
[L30] + A cycle exists if N is already an ANCESTOR of P.
[L31] + So traverse UP from P toward roots, looking for N."""
[L32] + visited = set()
[L33] + stack = [P]
[L34] + while stack:
[L35] + cur = stack.pop()
[L36] + if cur == N:
[L37] + return False # N is ancestor of P → cycle!
[L38] + if cur not in visited:
[L39] + visited.add(cur)
[L40] + for parent in forest.parents(cur): # Go UP
[L41] + stack.append(parent)
[L42] + return True
[L43]

```

[L44]

[L45] **Impact:** Without this fix, cycles in the Knowledge Forest will go undetected, violating the acyclic invariant and causing infinite loops during traversal.

[L46]

[L47] ---

[L48]

[L49] ### 3.2 FATAL: No Feedback Loop Between PRE and HV (PDF 3 — [4h pdf.pdf](#))

[L50]

[L51] **The Bug:** PRE selects the single best parent → HV validates it. If HV rejects (e.g., cycle detected), the node is **orphaned** with no fallback.

[L52]

[L53]

```

[L54] # BROKEN (current pipeline)
[L55] - best_parent = PRE.rank(candidates) # Pick ONE best
[L56] - if HV.validate(best_parent, node):
[L57] - FI.insert(node, best_parent)
[L58] - else:
[L59] - ??? # Node is lost!
[L60]
[L61] # FIXED: PRE returns ranked list, HV iterates
[L62] + ranked_parents = PRE.rank_all(candidates) # Return SORTED list
[L63] + inserted = False
[L64] + for parent in ranked_parents:
[L65] + if HV.validate(parent, node):
[L66] + FI.insert(node, parent)
[L67] + inserted = True
[L68] + break
[L69] + if not inserted:
[L70] + FI.insert_as_root(node) # Create new tree
[L71]

```

[L72]

[L73] ---

[L74]

[L75] ### 3.3 FATAL: Confidence Propagation Without Topological Sort (PDF 2)

[L76]

[L77] **The Bug:** `propagate_confidence()` iterates over nodes in arbitrary order. A child's confidence depends on its parent's updated confidence, but the parent may not have been processed yet. Additionally, the original formula only uses parent confidence with decay, ignoring the node's own extraction confidence and evidence confidence.

[L78]

[L79]

```
[L80] # BROKEN
[L81] - for node in forest.nodes(): # Arbitrary order
[L82] - if node.parents:
[L83] - node.confidence = avg(p.confidence for p in node.parents) * decay
[L84]
[L85] # FIXED: Use topological sort (parents always before children)
[L86] # FIXED: Include node's extraction confidence and evidence confidence
[L87] + from collections import deque
[L88] +
[L89] + # Confidence weighting factors (must sum to 1.0)
[L90] + ALPHA = 0.4 # Node's own extraction confidence weight
[L91] + BETA = 0.4 # Parent confidence weight
[L92] + GAMMA = 0.2 # Evidence confidence weight
[L93] + DECAY = 0.95 # Per-level decay factor
[L94] +
[L95] + def propagate_confidence(forest):
[L96] + # Kahn's algorithm for topological ordering
[L97] + in_degree = {n: len(forest.parents(n)) for n in forest.nodes()}
[L98] + queue = deque([n for n, d in in_degree.items() if d == 0]) # Roots first
[L99] + while queue:
[L100] + node = queue.popleft()
[L101] + if forest.parents(node):
[L102] + parent_confs = [p.confidence for p in forest.parents(node)]
[L103] + avg_parent_conf = sum(parent_confs) / len(parent_confs)
[L104] + evidence_conf = avg([e.confidence for e in node.evidence]) if node.evidence else 0.5
[L105] + node.confidence = (
[L106] + ALPHA * node.extraction_confidence +
[L107] + BETA * avg_parent_conf +
[L108] + GAMMA * evidence_conf
[L109] +) * DECAY
[L110] + else:
[L111] + node.confidence = node.base_confidence # Root nodes (default: 0.5)
[L112] + for child in forest.children(node):
[L113] + in_degree[child] -= 1
[L114] + if in_degree[child] == 0:
[L115] + queue.append(child)
[L116]
```

[L117]

[L118] &gt; [NOTE]

[L119] > **base\_confidence** is the standardized term used throughout (default: 0.5, configurable). Previously referred to as `baseline_conf` in some PDFs.

[L120]

[L121] ---

[L122]

[L123] ### 3.4 CRITICAL: RBO Knapsack Ignores Node Dependencies (PDF 6)

[L124]

[L125] **The Bug:** The Retrieval Budget Optimizer treats nodes as independent items in a 0-1 Knapsack. But in a hierarchy, a child node is meaningless without its parent context.

[L126]

[L127] > [!WARNING]

[L128] > **Implementation Note:** The algorithm below is a **greedy approximation** for the Precedence-Constrained Tree Knapsack problem, NOT an exact dynamic programming solution. Exact DP for tree knapsack is  $\$O(nB)\$$  where  $\$B\$$  is the token budget (potentially very large). The greedy approach uses utility-density sorting and provides a practical near-optimal solution.

[L129]

[L130] ```diff

[L131] # BROKEN: Standard Knapsack (independent items)

[L132] - nodes.sort(key=lambda n: n.utility / n.token\_cost, reverse=True)

[L133] - for n in nodes:

---

CHATGPT 20/08/2026, 15:58:50

## Layer 7 — Generation Engine

### Ab SHIA-RAG retrieved knowledge ko final verified answer me convert karta hai

Hum **same Smart Hospital example** continue karenge, exactly jaise Layer 0–6 me kiya tha.

**Hamari same query:**

*“How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?”*

Layer 6 tak system answer **likh nahi raha tha**. Uska kaam sirf relevant knowledge retrieve karna tha.

**Layer 7 ka kaam hai:**

*Retrieved knowledge se answer generate karna, answer ko individual claims me todna, har claim ko evidence ke against verify karna, citations bind karna, aur final answer confidence calculate karna. [filecite\turn29file0\L31-L36](#)*

---

## 1. Sabse pehle Layer 6 aur Layer 7 ka exact difference

### Layer 6

```
Question
 ↓
Knowledge Forest me search
 ↓
Relevant Nodes
 ↓
Structured Context
```

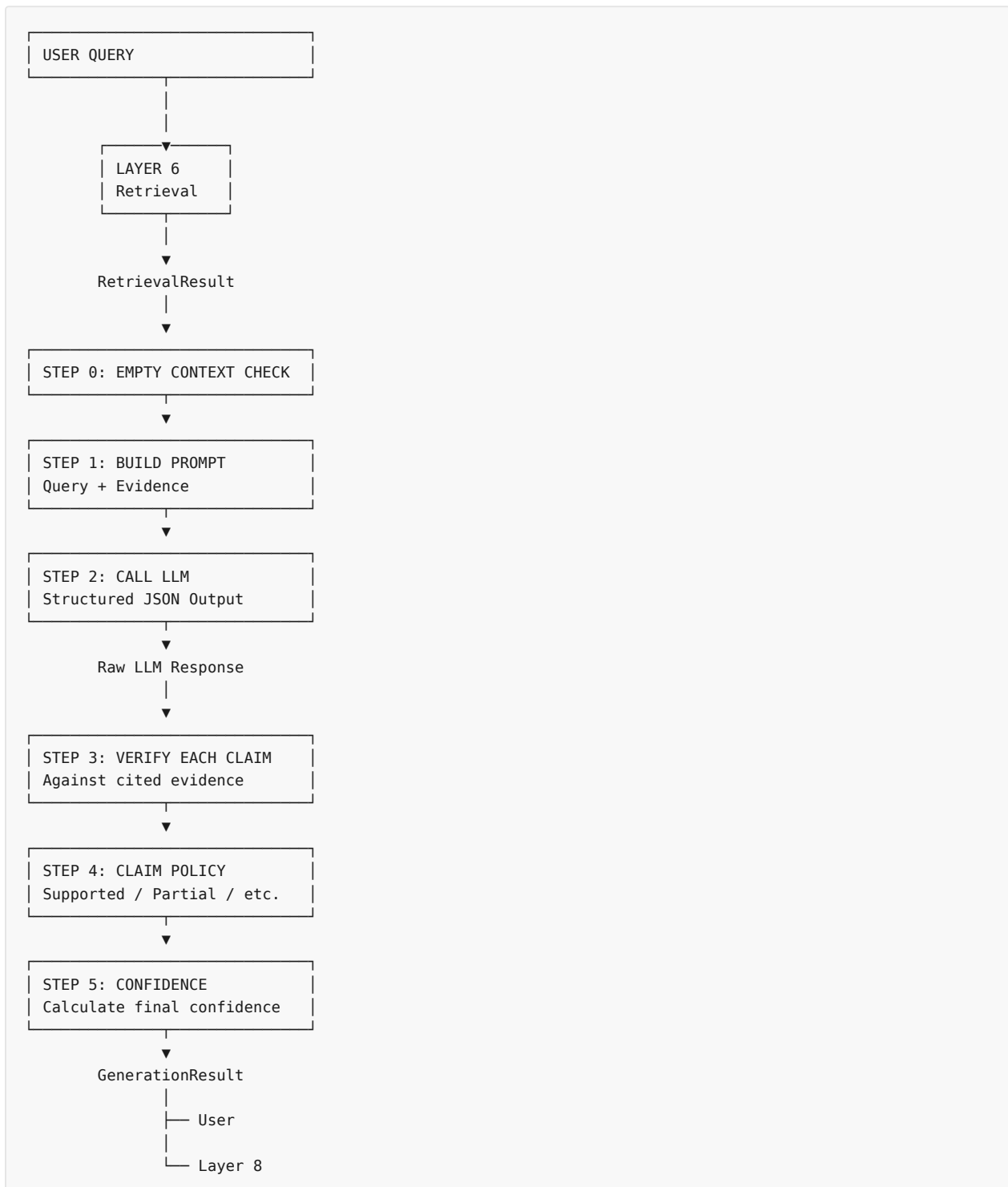
### Layer 7

```
Structured Context
↓
Prompt
↓
LLM
↓
Draft Answer
↓
Claims Extraction
↓
Claim Verification
↓
Unsupported Claims Remove/Mark
↓
Citations
↓
Confidence Calculation
↓
Final Answer
```

Ek line me:

*Layer 6 decide karta hai “kya knowledge deni hai”, Layer 7 decide karta hai “us knowledge se safe aur evidence-grounded answer kaise banana hai.”*

## 2. Layer 7 ka complete pipeline



Ye pipeline report ke Layer 7 implementation me explicitly defined hai. [filecite\turn29file0\L89-L130](#)  
[filecite\turn28file4\L501-L540](#)

### 3. Layer 7 ka input exactly kya hota hai?

Layer 7 ka primary input hai:

RetrievalResult

Ye Layer 6 se aata hai.

Isme important fields:

```
RetrievalResult
|
├─ selected_nodes
├─ context_text
├─ traversal_path_edges
├─ query_complexity
├─ query_intent
├─ seed_node_ids
└─ token_count
```

Report me `RetrievalResult` ko explicitly Layer 6 ka output aur Layer 7 ka input define kiya gaya hai.

📄filecite📄turn26file1📄L110-L119📄

---

#### 4. Hamare example me Layer 7 ko actual input kya milega?

Layer 6 se suppose:

```
selected_nodes =
[
 NODE_001: AI Diagnostic System,
 NODE_002: AI Diagnostic Engine,
 NODE_003: Machine Learning Model,
 NODE_004: Patient Symptoms,
 NODE_005: Medical History,
 NODE_006: Emergency Risk
]
```

Aur `context_text` something like:

**[NODE\_001] AI Diagnostic System****Definition:**

A healthcare AI system designed to support diagnostic analysis.

**Evidence:**

The system contains an AI Diagnostic Engine and a Risk Prediction Module.

-----

**[NODE\_002] AI Diagnostic Engine****Definition:**

A component that analyzes clinical information for diagnostic risk assessment.

**Evidence:**

The engine analyzes patient symptoms and medical history.

-----

**[NODE\_003] Machine Learning Model****Definition:**

A computational model used by the AI Diagnostic Engine.

**Evidence:**

The model processes relevant clinical information to support risk estimation.

-----

**[NODE\_004] Patient Symptoms****Evidence:**

Patient symptoms are analyzed by the AI Diagnostic Engine.

-----

**[NODE\_005] Medical History****Evidence:**

Medical history is analyzed by the AI Diagnostic Engine.

-----

**[NODE\_006] Emergency Risk****Evidence:**

The AI Diagnostic Engine estimates emergency risk using clinical information.

**Important:** Ye illustrative Smart Hospital example hai jo hum explanation ke liye use kar rahe hain; project report me actual example nahi diya gaya.

## 5. Layer 7 ka output exactly kya hota hai?

Layer 7 output:

GenerationResult

Report ke according:



```
GenerationResult
|
├─ answer
├─ claims
├─ citations
├─ overall_confidence
├─ retrieval_result
└─ generation_metadata
```

`retrieval_result` ko preserve kiya jata hai, jo Layer 8 feedback ke liye useful hota hai. [filecite\turn29file0\L104-L112](#)

## 6. Hamare example me final GenerationResult

Conceptually:

```
GenerationResult
|
├─ answer
│ └─ "The AI Diagnostic Engine analyzes patient symptoms
│ and medical history to estimate emergency risk...
│ [NODE_002][NODE_004][NODE_005] ..."
├─ claims
│ ├── Claim 1 → SUPPORTED
│ ├── Claim 2 → SUPPORTED
│ ├── Claim 3 → PARTIAL
│ └─ Claim 4 → UNSUPPORTED
├─ citations
│ ├── [NODE_002]
│ ├── [NODE_003]
│ └─ [NODE_006]
├─ overall_confidence
│ └─ e.g. calculated value
├─ retrieval_result
│ └─ Original Layer 6 result
└─ generation_metadata
 ├── model
 ├── claim_count
 └─ supported_count
```

## 7. LLM configuration — Layer 7 kis tarah ka answer generate karta hai?

Report configuration:

```
Primary Model = gpt-4o-mini
Fallback Model = llama-3.1-8b
Temperature = 0.3
Max Output Tokens = 2048
Top-p = 0.9
Timeout = 30 seconds
Max Retries = 3
Response Format = JSON
```

[filecite\turn29file0\L38-L50](#)

## Temperature = 0.3 kyun?

Low temperature ka purpose report me:

Factual answers

ke liye diya gaya hai. `filecite\turn29file1\L170-L179`

Simple language me:

High Temperature  
→ More creative variation

Low Temperature  
→ More controlled / factual behavior

Layer 7 creative story nahi bana raha.

Isko chahiye:

Evidence  
↓  
Controlled answer

## 8. Step 0 — Empty Context Check

Sabse pehle system check karta hai:

```
if not retrieval_result.selected_nodes:
```

Agar Layer 6 ko koi relevant node hi nahi mila:

```
selected_nodes = []
```

To LLM ko unnecessary call nahi kiya jata.

System directly return karta hai:

```
"I don't have sufficient evidence in the knowledge base
to answer this question."
```

Aur:

```
claims = []
citations = []
overall_confidence = 0.0
reason = EMPTY_CONTEXT
```

`filecite\turn29file0\L114-L125`

## Hamare example me empty context hota to?

Suppose user asks:

*"What is the legal salary structure of doctors in another country?"*

But Knowledge Forest me sirf:

```
Smart Hospital AI
AI Diagnostic Engine
Patient Symptoms
Medical History
Emergency Risk
```

available hai.

Layer 6:

```
selected_nodes = []
```

Then Layer 7:

```
✗ Don't ask LLM to guess
✗ Don't generate from prior knowledge
✗ Don't hallucinate
```

Instead:

```
"I don't have sufficient evidence..."
```

Ye important hallucination-control mechanism hai.

## 9. Lekin ek subtle issue hai

Code only checks:

```
selected_nodes empty hai ya nahi
```

It does **not** explicitly check:

```
selected_nodes ≠ empty
BUT
context_text = ""
```

**Example:**

```
selected_nodes = [NODE_001, NODE_002]
context_text = ""
```

Shown Layer 7 code me is exact inconsistency ke liye separate handling:

*Not explicitly specified.*

### Suggestion (Not from the report)

Check should ideally be:

```
if selected_nodes empty
OR
context_text empty/invalid:
 insufficient evidence path
```

## 10. Step 1 — Prompt Building

Ab Layer 7 ke paas do major cheezein hain:

1. User Question
2. Layer 6 Evidence

In dono ko combine karke LLM prompt banaya jata hai.

Report ka prompt structure:

#### SYSTEM INSTRUCTIONS

QUESTION:

{user query}

EVIDENCE (use ONLY this to answer):

{retrieved context}

Respond with valid JSON.

filecite\turn29file0\L53-L87

## 11. System Prompt ki sabse important rules

Layer 7 LLM ko basically ye rules diye jate hain:

### Rule 1

Answer ONLY from provided evidence.

Do NOT use prior knowledge.

### Rule 2

Every factual claim must have NODE\_ID evidence.

### Rule 3

Evidence insufficient hai

→ clearly say insufficient evidence.

### Rule 4

Evidence conflict karta hai

→ dono views present karo.

### Rule 5

Output structured JSON hona chahiye.

Ye rules report ke Layer 7 prompt template me explicitly defined hain. filecite\turn29file0\L55-L86

## 12. Hamare example ka actual conceptual prompt

Layer 7 internally something like:

You are a knowledge-grounded question answering system.

RULES:

1. Answer ONLY from the provided evidence.
2. Do NOT use prior knowledge.
3. Cite every claim using NODE\_ID.
4. If evidence is insufficient, say so.
5. If evidence conflicts, present both views.
6. Return valid JSON.

QUESTION:

How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?

EVIDENCE:

[NODE\_002]

The AI Diagnostic Engine analyzes patient symptoms and medical history.

[NODE\_003]

The Machine Learning Model is used by the AI Diagnostic Engine.

[NODE\_006]

The AI Diagnostic Engine estimates emergency risk.

Return valid JSON.

### 13. Ye prompt hallucination ko kaise control karta hai?

Suppose LLM ke paas general knowledge hai:

Doctors use CT scans  
+  
Blood tests  
+  
X-rays  
+  
Neural networks

But retrieved evidence me ye nahi hai.

Prompt ke according:

ONLY PROVIDED EVIDENCE

So ideally LLM should not say:

*"The engine performs CT scan analysis."*

Unless evidence me actual CT scan information present ho.

### 14. Step 2 — LLM ko call karna

System LLM ko call karta hai with:

```
prompt
model
temperature = 0.3
max_tokens = 2048
response_format = JSON
timeout = 30 seconds
```

Phir:

```
raw_response
↓
json.loads()
↓
parsed structured response
```

filecite\turn28file0\L85-L110

## 15. LLM se expected raw output kya hai?

Expected JSON:

```
{
 "answer": "The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk.",
 "claims": [
 {
 "text": "The AI Diagnostic Engine analyzes patient symptoms.",
 "evidence_ids": ["NODE_002", "NODE_004"],
 "confidence": 0.94
 },
 {
 "text": "The AI Diagnostic Engine analyzes medical history.",
 "evidence_ids": ["NODE_002", "NODE_005"],
 "confidence": 0.91
 },
 {
 "text": "The engine estimates emergency risk.",
 "evidence_ids": ["NODE_006"],
 "confidence": 0.90
 },
 {
 "text": "The Machine Learning Model supports the diagnostic process.",
 "evidence_ids": ["NODE_003"],
 "confidence": 0.87
 }
],
 "overall_confidence": 0.90,
 "insufficient_evidence": false
}
```

**Important:** Ye Smart Hospital example ke liye hamara illustrative output hai, actual report output nahi.

## 16. Sabse important concept — Answer aur Claims alag hain

Maan lo answer:

*"The AI Diagnostic Engine analyzes symptoms and medical history, uses a Machine Learning Model, and estimates emergency risk."*

Ye ek paragraph hai.

Layer 7 isko internally factual units me treat karta hai:

```
CLAIM 1:
AI Diagnostic Engine analyzes symptoms

CLAIM 2:
AI Diagnostic Engine analyzes medical history

CLAIM 3:
Machine Learning Model is involved

CLAIM 4:
Engine estimates emergency risk
```

Har claim separately verify hota hai.

Yahi Layer 7 ka important design point hai.

---

## 17. Step 3 — Claim Verification

Ab LLM ne khud claim ke saath evidence IDs diye:

```
Claim
 +
Evidence IDs
 ↓
Verification Engine
```

`node_map` create hota hai:

```
NODE_ID
 ↓
Actual KnowledgeNode
```

Then each claim verify hota hai. `filecite\turn28file4\L501-L521`

---

## 18. Verification ka actual logic

For each evidence ID:

```
Claim Text
 ↓
Embed Claim

Evidence Text
 ↓
Embed Evidence

 ↓
Cosine Similarity
```

Phir evidence scores aggregate kiye jate hain.

Report verification function ke according:

No valid evidence  
→ UNSUPPORTED

Average score > 0.7  
→ SUPPORTED

$0.4 < \text{score} \leq 0.7$   
→ PARTIAL

score  $\leq 0.4$   
→ UNSUPPORTED

Multiple evidence scores me large difference  
→ CONFLICTING

filecite turn29file1 L199-L230

## 19. Hamare example ka Claim 1 verify karo

LLM claim:

"The AI Diagnostic Engine analyzes patient symptoms."

Evidence IDs:

[NODE\_002, NODE\_004]

NODE\_002 evidence:

"The AI Diagnostic Engine analyzes patient symptoms and medical history."

NODE\_004 evidence:

"Patient symptoms are analyzed by the AI Diagnostic Engine."

Verification:

Claim embedding  
↓  
Compare with NODE\_002 evidence  
↓  
High similarity

Claim embedding  
↓  
Compare with NODE\_004 evidence  
↓  
High similarity

Suppose:

Score 1 = 0.95  
Score 2 = 0.93

Average = 0.94

Since:



0.94 > 0.7

Result:

SUPPORTED

## 20. Claim 2 verify karo

Claim:

"The AI Diagnostic Engine analyzes medical history."

Evidence:

NODE\_002  
NODE\_005

Suppose average:

0.88

Result:

SUPPORTED

## 21. Ab PARTIAL case samjho

LLM claim:

*"The Machine Learning Model independently makes the final emergency decision."*

Evidence:

NODE\_003:  
Machine Learning Model supports the diagnostic process.

Evidence directly **final independent decision** support nahi karta.

Suppose semantic score:

0.58

Then:

$0.4 < 0.58 \leq 0.7$

Result:

PARTIAL

Meaning:

*Claim ka kuch part evidence se related hai, but complete claim fully supported nahi hai.*

## 22. UNSUPPORTED case

LLM claim:

*"The Machine Learning Model sends an automatic ambulance."*

Evidence IDs:

[ ]

or:

NODE\_003

but NODE\_003 me ambulance ka koi evidence nahi.

Then:

No supporting evidence  
OR  
Similarity  $\leq 0.4$

Result:

UNSUPPORTED

Report me no valid evidence directly **UNSUPPORTED**, 0.0 return karta hai. `filecite\turn29file1\L211-L230`

---

## 23. CONFLICTING case

Suppose do retrieved evidence nodes hain.

**NODE\_A**

The system automatically escalates all high-risk patients.

**NODE\_B**

The system does not automatically escalate patients;  
human approval is required.

LLM claim:

*"The system automatically escalates high-risk patients."*

Agar evidence scores me significant disagreement ho aur score spread:

`max_score - min_score > 0.4`

then report logic:

CONFLICTING

use karta hai. `filecite\turn29file1\L221-L230`

---

## 24. Four verification statuses

| Status | Meaning |

|---|---|

| **SUPPORTED** | Evidence claim ko sufficiently support karta hai |

| **PARTIAL** | Evidence claim ko partially support karta hai |

| **UNSUPPORTED** | Evidence claim support nahi karta |

| **CONFLICTING** | Retrieved evidence items me conflict detected hai |

Ye four statuses report me explicitly defined hain. `filecite\turn29file1\L203-L230`

---

## 25. Step 4 — Unsupported Claim Policy

Verification ke baad har claim ko same treatment nahi milta.

Report policy:

| Verification Status | Action |

|---|---|

| **SUPPORTED** | Citation ke saath keep |

| **PARTIAL** | Keep, but partially supported mark |

| **UNSUPPORTED** | Answer se remove, note |

| **CONFLICTING** | Dono views present karo with note |

`filecite\turn29file1\L233-L240`

---

## 26. Hamare example me raw LLM answer

Suppose LLM ne generate kiya:

The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk.

The Machine Learning Model supports the diagnostic process.

The model independently makes final treatment decisions and automatically dispatches an ambulance.

Verification:

Claim 1 → SUPPORTED  
Claim 2 → SUPPORTED  
Claim 3 → UNSUPPORTED  
Claim 4 → UNSUPPORTED

Then final policy:

The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk.

The Machine Learning Model supports the diagnostic process.

[Unsupported claims removed]

---

## 27. Important point: LLM ka self-confidence enough nahi hai

LLM JSON me already ye de sakta hai:

```
"confidence": 0.99
```

But Layer 7 blindly trust nahi karta.

It separately calculates:

```
status, conf = verify_claim(...)
```

Then verified confidence use karta hai. `filecite\turn28file4\L501-L524`

So:

```
LLM says:
"I am 99% confident"

≠

System accepts 99%
```

System evidence ke against independently verify karta hai.

## 28. Citation Binding

Har claim ke evidence IDs:

```
["NODE_002", "NODE_004"]
```

se citation create hoti hai.

Conceptually:

```
Claim
↓
Evidence IDs
↓
node_map
↓
format_citation()
↓
Formatted Citation
```

Layer 7 code verification ke baad citation format karta hai. `filecite\turn28file4\L509-L521`

## 29. Hamare final answer me citations

Conceptually:

```
The AI Diagnostic Engine analyzes patient symptoms and
medical history [NODE_002][NODE_004][NODE_005].

It estimates emergency risk [NODE_006].

The Machine Learning Model supports the diagnostic process
[NODE_003].
```

Ye [NODE\_ID] citations user ke final answer ko evidence traceability dete hain.

### 30. Step 5 — Overall Answer Confidence

Ab poore answer ka ek confidence calculate hota hai.

Report formula:

$$\begin{aligned} & \left[ \right. \\ & \text{\text{confidence}} \\ & = \\ & 0.4 \\ & \times \\ & \frac{\text{\text{supported claims}}}{\text{\text{total claims}}} \\ & + \\ & 0.3 \\ & \times \\ & \overline{\text{\text{claim confidence}}} \\ & + \\ & 0.2 \\ & \times \\ & \overline{\text{\text{node confidence}}} \\ & + \\ & 0.1 \\ & \times \\ & \frac{\text{\text{cited nodes}}}{\text{\text{selected nodes}}} \\ & \left. \right] \end{aligned}$$

fileciteurn29file1L242-L244

### 31. Formula ko simple language me samjho

Final confidence 4 things dekhta hai:

#### Part A — 40%

Supported Claims / Total Claims

Question:

Generated answer me kitne claims actually evidence-supported nikle?

#### Part B — 30%

Average Claim Confidence

Question:

Har verified claim individually kitna strongly supported hai?

**Part C — 20%**

Average Node Confidence

Question:

*Layer 6 ne jo knowledge nodes retrieve kiye, wo khud kitne reliable/confident hain?***Part D — 10%**

Cited Nodes / Selected Nodes

Question:

*Retrieved knowledge ka kitna portion answer me actually use aur cite hua?***32. Hamare example se confidence calculate karo**

Suppose:

Total Claims = 5  
Supported Claims = 4

Then:

Supported Ratio =  $4/5 = 0.8$ 

Contribution:

 $0.4 \times 0.8 = 0.32$ 

Suppose average claim confidence:

0.85

Contribution:

 $0.3 \times 0.85 = 0.255$ 

Suppose average node confidence:

0.90

Contribution:

 $0.2 \times 0.90 = 0.18$ 

Suppose:

Selected Nodes = 6  
Cited Nodes = 5

Coverage:

$$5/6 = 0.833$$

Contribution:

$$0.1 \times 0.833 \\ \approx 0.083$$

Final:

$$\begin{array}{r} 0.32 \\ + 0.255 \\ + 0.18 \\ + 0.083 \\ \hline = 0.838 \end{array}$$

So approximately:

$$\text{Overall Confidence} = 0.838$$

Ye calculation hamara illustrative example hai; formula report se hai.

### 33. Important: High confidence $\neq$ answer guaranteed true

Ye bahut important critical point hai.

System ka confidence depend karta hai:

```
Claim verification
+
Node confidence
+
Citation coverage
```

But agar:

```
Layer 3 ne wrong fact extract kiya
 ↓
Layer 4 ne validate kar diya
 ↓
Layer 5 me wrong knowledge stored ho gayi
 ↓
Layer 6 ne retrieve kar diya
 ↓
Layer 7 perfectly verify kar raha hai
```

To:

System may still produce a wrong answer

because verification **retrieved evidence ke against** ho rahi hai.

#### Logical interpretation

*Layer 7 mainly checks **evidence-grounding consistency**, not absolute real-world truth.*

Ye distinction bahut important hai.

---

### 34. Step-by-step exception handling

Ab dekhte hain Layer 7 errors ko kaise handle karta hai.

---

#### Exception 1 — Empty Context

Handled explicitly.

```
selected_nodes = []
```

Action:

```
Don't call LLM
```

Return:

```
Insufficient evidence answer
confidence = 0.0
reason = EMPTY_CONTEXT
```

filecite\turn29file0\L114-L125

---

#### Exception 2 — LLM invalid JSON return karta hai

Suppose LLM returns:

```
Sure! Here is your answer:

{
 "answer": ...
```

JSON parser fail:

```
json.JSONDecodeError
```

System:

```
Retry
```

with stronger instruction:

```
IMPORTANT:
Respond with valid JSON only.
```

Aur retry me temperature:

```
0.1
```

kar diya jata hai. filecite\turn28file0\L88-L107

---

#### Why lower temperature?



Retry me system aur controlled output chahta hai:

```
First attempt:
temperature = 0.3

JSON failure
 ↓

Retry:
temperature = 0.1
```

Less variation, hopefully more structured JSON.

### 35. Important missing issue in JSON retry

Config me:

```
max_retries = 3
```

defined hai. `filecite[turn29file0][L41-L50]`

Lekin shown generation code me `json.JSONDecodeError` ke liye:

```
explicit retry loop
```

3 times ka visible implementation nahi dikh raha.

Shown code ek explicit retry dikhata hai. `filecite[turn28file0][L99-L107]`

**Therefore:**

*Configuration aur shown code ke beech `max_retries = 3` ka complete enforcement explicitly visible nahi hai.*

Ye implementation inconsistency / missing detail hai.

### 36. Exception 3 — LLM Timeout

Primary LLM timeout:

```
LLMTimeout
```

Then:

```
Primary Model
 ↓ timeout
Fallback Model
```

Report code fallback model call karta hai. `filecite[turn28file4][L491-L499]`

Flow:

```

gpt-4o-mini
├── Success → continue
└── Timeout
 └── llama-3.1-8b
 └── Generate response

```

### 37. Timeout handling me possible issue

Fallback model bhi:

```
Timeout
```

ho jaye to?

Shown Layer 7 code me:

*Fallback model ke second failure ke liye explicit handling not shown.*

Similarly:

```

NetworkError
AuthenticationError
RateLimitError
Malformed fallback JSON

```

ke exact handling shown snippet me explicitly specified nahi hai.

### Suggestion (Not from the report)

Production implementation:

```

Primary LLM
└── failure
 Retry 1
 └── failure
 Retry 2
 └── failure
 Fallback Model
 └── failure
 Return controlled GenerationError

```

### 38. Exception 4 — Evidence ID galat hai

Suppose LLM says:

```

{
 "text": "The engine uses a deep neural network.",
 "evidence_ids": ["NODE_99999"]
}

```

But:

```
NODE_99999
```

selected node map me exist hi nahi karta.

Then verification:

```
node = node_map.get(eid)
```

No valid evidence score add hota hai.

Finally:

```
if not scores:
 return UNSUPPORTED, 0.0
```

filecite turn29file1 L211-L219

So invalid evidence ID automatically unsupported path me ja sakta hai.

### 39. Exception 5 — LLM claim list hi na de

Expected:

```
"claims": [...]
```

But response:

```
{
 "answer": "Some answer"
}
```

Code:

```
parsed.get('claims', [])
```

use karta hai.

So:

```
claims = []
```

Then answer policy aur confidence calculation aage run honge. filecite turn28file4 L501-L527

#### Potential risk

Answer text generated ho sakta hai but uske verify karne ke liye:

```
0 claims
```

hon.

Shown code me answer text ko independently claim-extract karke verify karne ka second mechanism explicitly shown nahi hai.

#### Critical observation

*Layer 7 verification depends on the LLM-provided claim list.*

Agar LLM answer me factual sentence likhe but `claims` list me include na kare, shown pipeline us sentence ko automatically discover karke verify karta hua nahi dikhaya gaya.

### Suggestion (Not from the report)

System should ideally:

```

Answer Text
 ↓
Independent Claim Extractor
 ↓
Compare with LLM-provided claims
 ↓
Missing claims?
 ↓
Verify them too

```

Ye Layer 7 ko stronger banayega.

## 40. Sabse important potential weakness — “verification” actually kya verify kar rahi hai?

Report verification:

```

Embed(claim)
 vs
Embed(evidence text)
 ↓
Cosine Similarity

```

use karti hai. `filecite\turn29file1\L203-L230`

Problem:

```

Semantic similarity
≠
Logical entailment

```

### Example

Evidence:

*“The AI Diagnostic Engine analyzes patient symptoms.”*

Claim:

*“The AI Diagnostic Engine automatically treats the patient.”*

Dono semantically healthcare domain me related ho sakte hain.

Embedding similarity moderate/high aa sakti hai.

But:

```

Analyzes
≠
Automatically treats

```

So pure cosine similarity logical proof nahi hai.

### Critical technical observation

Report comment verification ko semantic entailment bolta hai, lekin shown implementation actual entailment model ke bajay:

```
embedding + cosine similarity
```

use karti hai. `filecite\turn29file1\L203-L230`

### Suggestion (Not from the report)

Stronger verification:

```
Claim
 ↓
NLI / Entailment Model
 ↓
Entailed?
Contradicted?
Neutral?
```

This would be more rigorous than semantic similarity alone.

## 41. Conflict detection me bhi ek subtle issue

Report conflict detect karti hai if:

```
max(score) - min(score) > 0.4
```

`filecite\turn29file1\L221-L230`

But score difference ka matlab always factual contradiction nahi hota.

Example:

```
Evidence A strongly supports claim = 0.95
Evidence B only loosely related = 0.40
```

Difference:

```
0.95 - 0.40 = 0.55
```

System:

```
CONFLICTING
```

classify kar sakta hai.

But Evidence B necessarily contradictory nahi hai — maybe sirf irrelevant hai.

**Therefore:**

*High similarity-score variance ≠ guaranteed contradiction.*

Ye report ke shown implementation ka critical limitation hai.

## 42. Step 4 — `apply_claim_policy()` actually kya karta hai?

Pipeline:

```

Parsed Answer
 +
Verified Claims
 ↓
apply_claim_policy()
 ↓
Filtered / Annotated Answer

```

Report policy defines expected behavior:

```

SUPPORTED
→ Keep

PARTIAL
→ Keep but mark partial

UNSUPPORTED
→ Remove

CONFLICTING
→ Present both views

```

filecite\turn29file1\L233-L240

Lekin shown snippets me `apply_claim_policy()` ka **exact implementation body** visible nahi hai.

Therefore:

*Exact text-removal mechanism, claim-to-answer sentence mapping, aur partial/conflicting annotation format **not specified** in the shown code.*

### 43. Ye issue kyun important hai?

Suppose answer:

```

Sentence:
"The AI Diagnostic Engine analyzes symptoms and automatically
dispatches an ambulance."

```

Internally:

```

Claim A:
Analyzes symptoms
→ SUPPORTED

Claim B:
Dispatches ambulance
→ UNSUPPORTED

```

`apply_claim_policy()` ko sentence ko accurately edit karna padega:

Correct:

```
"The AI Diagnostic Engine analyzes symptoms."
```

But exact unsupported substring identify karna difficult ho sakta hai.

Report me policy hai, but shown implementation me exact rewriting mechanism visible nahi hai.

## 44. Generation Metadata

Final output me metadata bhi hota hai:

```
generation_metadata
```

Shown code stores:

```
model
claim_count
supported_count
```

filecite\turn28file4\L529-L539

Example:

```
generation_metadata = {
 "model": "gpt-4o-mini",
 "claim_count": 5,
 "supported_count": 4
}
```

## 45. Ek aur important implementation inconsistency

Config:

```
Primary model = gpt-4o-mini
Fallback = llama-3.1-8b
```

Timeout hone par fallback model use hota hai. filecite\turn29file0\L41-L50 filecite\turn28file4\L491-L499

Lekin final metadata shown code me:

```
'model': GENERATION_CONFIG['model']
```

store karta hai. filecite\turn28file4\L529-L539

### Potential problem

Suppose actual generation:

```
Primary timed out
↓
Fallback generated answer
```

But metadata could still say:

```
model = primary model
```

because it directly uses configured primary model name.

### Suggestion (Not from the report)

Track:

```
actual_model_used
```

and metadata me wahi store karo.

46. Ab poora example Layer 7 me end-to-end chalo

Input Query

How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?

Layer 6 Output

```
RetrievalResult
├── selected_nodes
│ ├── NODE_002 AI Diagnostic Engine
│ ├── NODE_003 Machine Learning Model
│ ├── NODE_004 Patient Symptoms
│ ├── NODE_005 Medical History
│ └── NODE_006 Emergency Risk
├── context_text
│ └── Evidence from these nodes
├── query_intent
│ └── MULTI_HOP
└── query_complexity
 └── COMPLEX
```

Step 0 — Empty Check

selected\_nodes = 5

Not empty:

Continue ✓

Step 1 — Build Prompt

```
SYSTEM RULES
+
USER QUESTION
+
LAYER 6 CONTEXT
```

Step 2 — LLM Generate

LLM returns:



```
{
 "answer": "The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk.",
 "claims": [
 {
 "text": "The AI Diagnostic Engine analyzes patient symptoms.",
 "evidence_ids": ["NODE_002", "NODE_004"]
 },
 {
 "text": "The AI Diagnostic Engine analyzes medical history.",
 "evidence_ids": ["NODE_002", "NODE_005"]
 },
 {
 "text": "The engine estimates emergency risk.",
 "evidence_ids": ["NODE_006"]
 },
 {
 "text": "The Machine Learning Model supports the process.",
 "evidence_ids": ["NODE_003"]
 }
]
}
```

Step 3 — Verify Claim 1

Claim:  
AI Engine analyzes symptoms

Evidence:  
NODE\_002 + NODE\_004

Similarity:  
High

Status:  
SUPPORTED

Verify Claim 2

Claim:  
AI Engine analyzes medical history

Evidence:  
NODE\_002 + NODE\_005

Status:  
SUPPORTED

Verify Claim 3

Claim:  
Engine estimates emergency risk

Evidence:  
NODE\_006

Status:  
SUPPORTED

---

## Verify Claim 4

Claim:  
ML Model supports process

Evidence:  
NODE\_003

Status:  
SUPPORTED

---

## Step 4 — Claim Policy

SUPPORTED → Keep all 4

---

## Step 5 — Confidence

Suppose:

4 supported / 4 total  
High average claim confidence  
High node confidence  
Most selected nodes cited

Then:

Overall Confidence  
→ High

Exact value formula se calculate hoga. `filecite\turn29file1\L242-L244`

---

## 47. Final output user ko kya milega?

Conceptually:

ANSWER:

The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk [NODE\_002] [NODE\_004] [NODE\_005] [NODE\_006].

The Machine Learning Model supports the diagnostic process [NODE\_003].

CITATIONS:

[NODE\_002]  
[NODE\_003]  
[NODE\_004]  
[NODE\_005]  
[NODE\_006]

OVERALL CONFIDENCE:

0.XX

Internally:

GenerationResult

Layer 8 ko bhi diya jayega.

48. Layer 7 me major errors aur unka impact

Error 1 — Empty Retrieval

Layer 6  
↓  
No nodes

Handled:

Insufficient evidence  
Confidence = 0

Error 2 — Invalid JSON

LLM  
↓  
Malformed JSON

Handled:

Retry  
+  
Explicit JSON-only instruction  
+  
Lower temperature

fileciteurn28file0L99-L107

### Error 3 — Primary LLM timeout

```
Primary
↓ timeout
Fallback model
```

Handled. `filecite\turn28file4\L491-L499`

---

### Error 4 — Wrong evidence ID

```
Claim
↓
NODE_ID doesn't exist
```

Result:

```
No valid evidence
↓
UNSUPPORTED
```

`filecite\turn29file1\L211-L219`

---

### Error 5 — Unsupported hallucinated claim

```
LLM invents fact
```

Verification:

```
Similarity low / no evidence
↓
UNSUPPORTED
↓
Remove according to policy
```

`filecite\turn29file1\L233-L240`

---

### Error 6 — Conflicting evidence

```
Evidence A
↔
Evidence B
```

System policy:

```
Present both views
```

with note/citations. `filecite\turn29file1\L233-L240`

---

## 49. Layer 7 me missing / weak parts

Ab main report ke basis par technically critical points clearly bata raha hoon.

### Missing 1 — Exact `apply_claim_policy()` implementation

Policy defined hai, but shown code snippets me function body nahi.

So exact sentence editing:

```
Not specified in the shown implementation.
```

### Missing 2 — Full `max_retries = 3` enforcement

Config me 3 retries hain, but shown code me complete retry loop clearly visible nahi.

```
Configuration: 3
Shown explicit JSON retry: one retry
```

Potential inconsistency.

### Missing 3 — Fallback failure handling

Fallback model bhi fail ho:

```
Not explicitly specified.
```

### Missing 4 — Answer aur claims consistency validation

LLM answer me extra factual claim likh de but `claims[]` me na de:

```
Potentially unverified.
```

Shown code claims ko primarily LLM ke provided list se verify karta hai. `filecite[turn28file4][L501-L524]`

### Weak 5 — Cosine similarity as entailment

```
Semantic Similarity
≠
True Logical Entailment
```

Ye Layer 7 ka most important technical weakness hai.

### Weak 6 — Conflict detection by score spread

```
max(score) - min(score) > 0.4
```

means score disagreement, but not necessarily actual contradiction.

### Weak 7 — Empty nodes but malformed context

Separate check:

```
selected_nodes non-empty
but context invalid
```

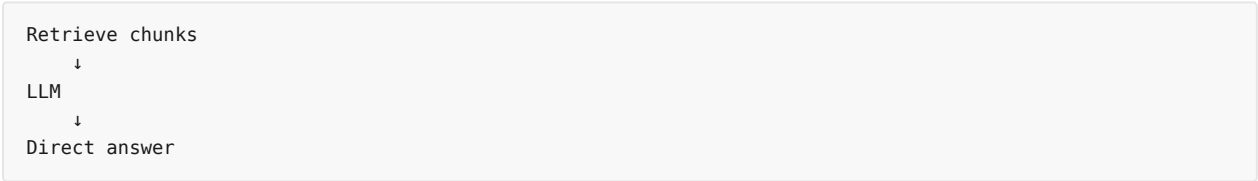
explicitly shown nahi.

Weak 8 — Metadata may record wrong model

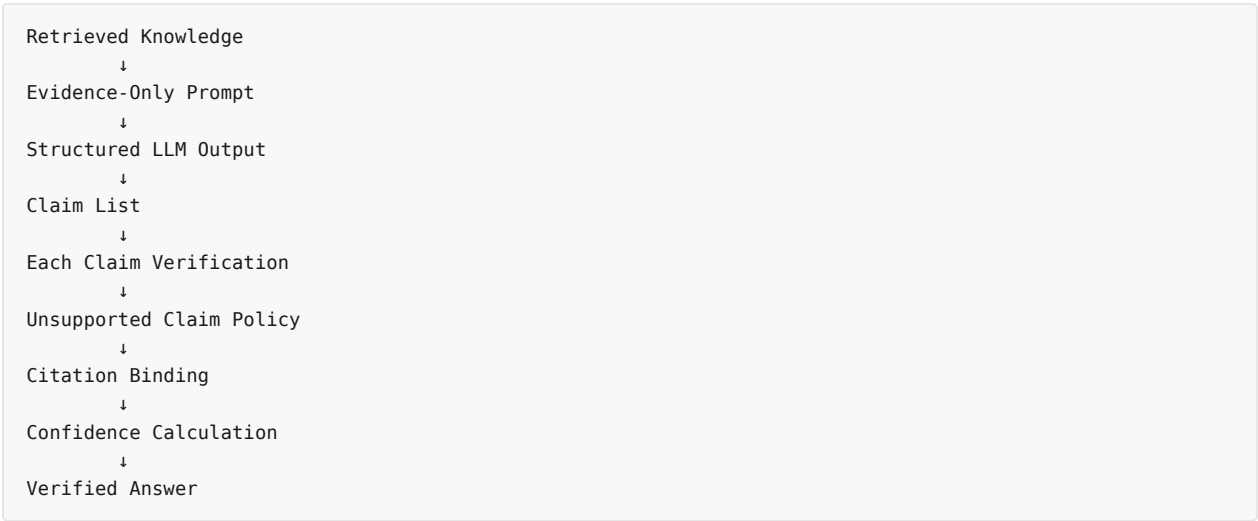
Fallback use hone ke baad bhi final metadata shown code me primary model config value store kar sakta hai.

50. Layer 7 ka actual core contribution kya hai?

Agar normal RAG hota:



SHIA-RAG Layer 7:



Isliye Layer 7 sirf:

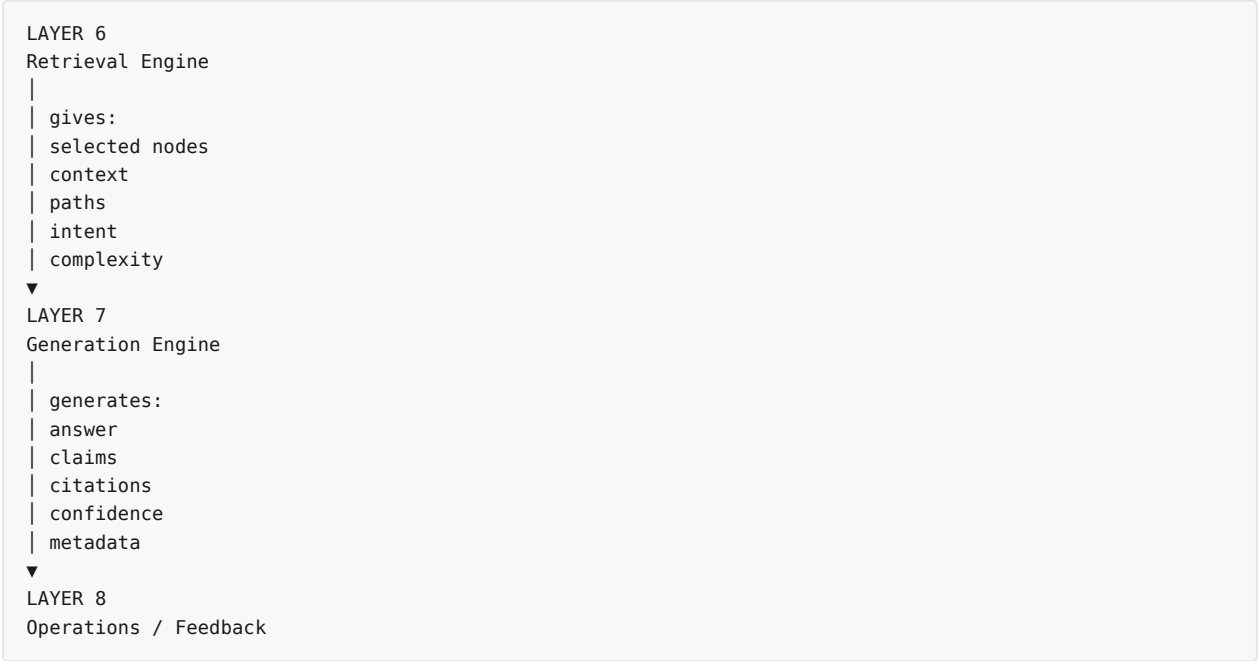
“LLM Answer Generator”

nahi hai.

Better description:

Evidence-Grounded Generation + Claim Verification Layer

51. Layer 6 → Layer 7 → Layer 8 connection



Layer 7 original RetrievalResult preserve karta hai, isliye next layer retrieval aur generation outcome ko connect karke analyze kar sakti hai. [filecite\turn29file0\L104-L112](#)

## 52. Sabse important mental model

Layer 6 poochta hai:

*“Question answer karne ke liye kaunsi knowledge chahiye?”*

Layer 7 poochta hai:

*“Kya main retrieved knowledge ke basis par answer likh sakta hoon?”*

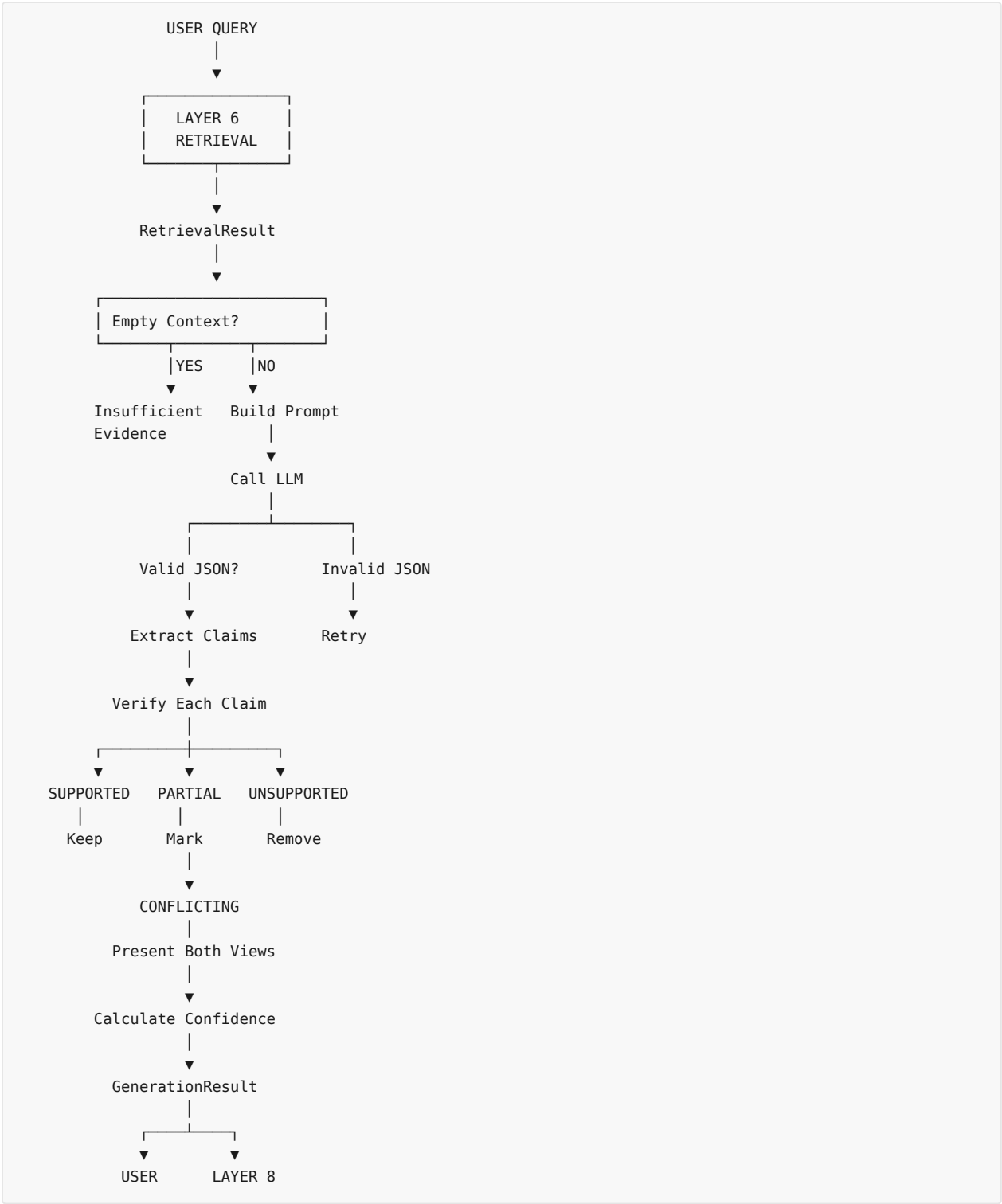
Phir:

*“Answer ke har factual claim ka evidence hai ya nahi?”*

## 53. One-line summary of every step

| Step | Component           | Simple Meaning                                    |
|------|---------------------|---------------------------------------------------|
| 0    | Empty Context Check | Knowledge mila bhi hai ya nahi?                   |
| 1    | Prompt Builder      | Question + evidence combine karo                  |
| 2    | LLM Generation      | Structured draft answer banao                     |
| 3    | Claim Verification  | Har claim evidence se check karo                  |
| 4    | Claim Policy        | Unsupported/conflicting claims handle karo        |
| 5    | Confidence          | Answer kitna evidence-grounded hai calculate karo |
| 6    | GenerationResult    | Final verified result user + Layer 8 ko do        |

## 54. Final complete mental model



Layer 7 ko ek sentence me yaad rakho

*Layer 6 relevant knowledge retrieve karta hai, aur Layer 7 us knowledge ko blindly answer me convert nahi karta—pehle structured answer generate karta hai, phir har claim ko evidence ke against verify karke citations aur confidence ke saath final answer banata hai. filecite turn29file0 L31-L36*

**Next natural step: Layer 8**, jahan system logging, feedback, self-evolution, edge-weight updates aur performance monitoring karega.



layer 8 smjhao

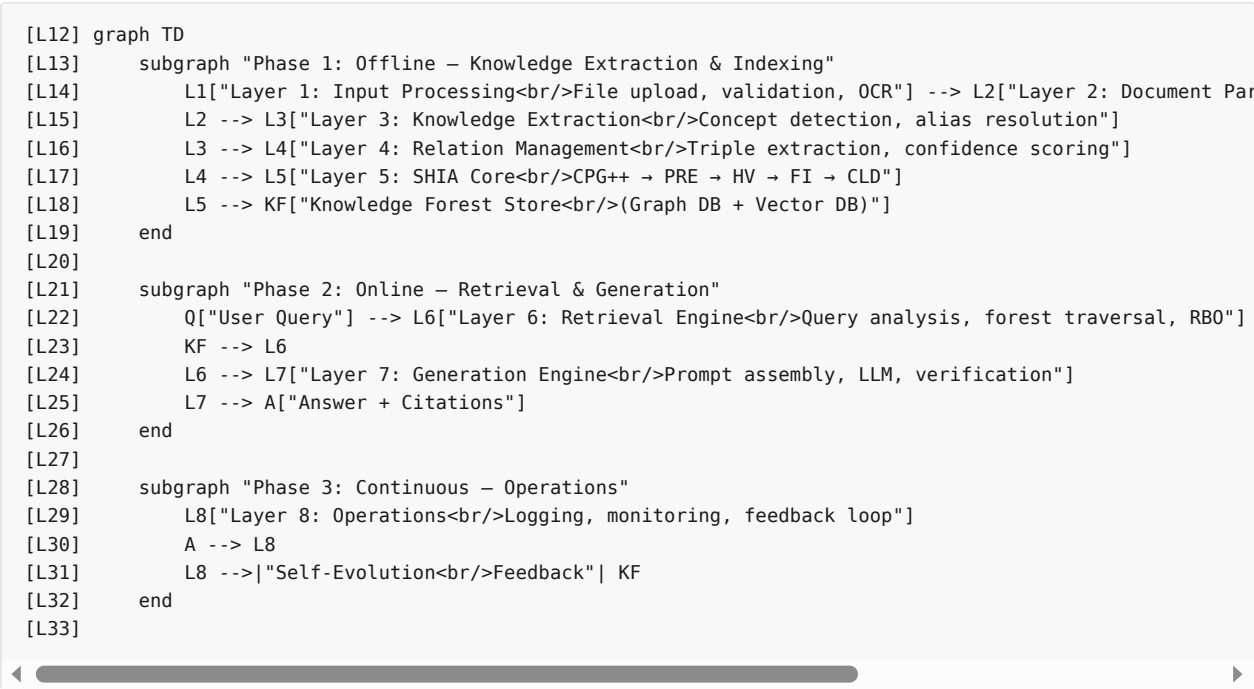
CHATGPT 20/08/2026, 16:04:56

{"paths":["files"],"query":"search"}

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `[[filecite]]turn31file0[[`

- [L1] metadata:
- [L2] query\_result\_index: 0
- [L3] file\_id: file\_0000000012688211b4f57f5b8bcb8b42
- [L4] version\_id: 1
- [L5] name: SHIA\_RAG\_Final\_Report(4).md
- [L6] mime\_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.03131881575727918
- [L9] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--6
- [L10] snippet:
- [L11]



- [L34]
- [L35] ### 3.2 Data Flow
- [L36]
- [L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47] ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53] ↓ Layer 7: Build prompt, invoke LLM
[L54] ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56] ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

```
[L58]
[L59] ### 3.3 Layer Summary
[L60]
[L61] | Layer | Name | Input | Output | Key Algorithms |
[L62] |-----|-----|-----|-----|-----|
[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L72]
[L73] ---
[L74]
[L75] ## 4. Data Model — Layer 0
```

Citation Marker: `filecite`turn31file1

```
[L76] metadata:
[L77] query_result_index: 1
[L78] file_id: file_000000004bc481fdbf9e949ec3b7544b
[L79] version_id: 1
[L80] name: SHIA_RAG_Final_Report(2).md
[L81] mime_type: text/markdown
[L82] surface: library
[L83] score: 0.031054405392392875
```

[L84] document\_chunk\_id: file\_000000004bc481fdbf9e949ec3b7544b--1--6

[L85] snippet:

[L86]

```
[L87] graph TD
[L88] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L89] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Par
[L90] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L91] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L92] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L93] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L94] end
[L95]
[L96] subgraph "Phase 2: Online – Retrieval & Generation"
[L97] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RBO"]
[L98] KF --> L6
[L99] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L100] L7 --> A["Answer + Citations"]
[L101] end
[L102]
[L103] subgraph "Phase 3: Continuous – Operations"
[L104] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L105] A --> L8
[L106] L8 -->| "Self-Evolution
Feedback" | KF
[L107] end
[L108]
```

[L109]

[L110] ### 3.2 Data Flow

[L111]

[L112]

```
[L113] Raw Document (PDF/DOCX/HTML)
[L114] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L115] Normalized Text + Metadata
[L116] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L117] [Page → Region → Block] hierarchy
[L118] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L119] KnowledgeUnit objects (candidate concepts)
[L120] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L121] KnowledgeNode objects (canonical, deduplicated)
[L122] ↓ Layer 4: Validate relations against ontology, score confidence
[L123] KnowledgeEdge objects (typed, scored)
[L124] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L125] Knowledge Forest (rooted trees + cross-links)
[L126] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L127] Selected context nodes (within token budget)
[L128] ↓ Layer 7: Build prompt, invoke LLM
[L129] ↓ Layer 7: Verify claims against evidence, bind citations
[L130] Answer with citations + confidence score
[L131] ↓ Layer 8: Log, collect feedback, update forest weights
[L132]
```

[L133]

[L134] ### 3.3 Layer Summary

[L135]

[L136] | Layer | Name | Input | Output | Key Algorithms |

[L137] |-----|-----|-----|-----|-----|

[L138] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |  
[L139] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |  
[L140] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |  
[L141] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L142] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L143] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L144] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L145] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L146] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L147]  
[L148] ---  
[L149]  
[L150] ## 4. Data Model — Layer 0

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `[[filecite]]turn31file2`

[L151] metadata:  
[L152] query\_result\_index: 2  
[L153] file\_id: file\_000000003218820b918ee3d86ec345ae  
[L154] version\_id: 1  
[L155] name: SHIA\_RAG\_Final\_Report(1).md  
[L156] mime\_type: text/markdown  
[L157] surface: library  
[L158] score: 0.030798389007344232  
[L159] document\_chunk\_id: file\_000000003218820b918ee3d86ec345ae--1--6  
[L160] snippet:  
[L161]

```
[L162] graph TD
[L163] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L164] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L165] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L166] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L167] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L168] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L169] end
[L170]
[L171] subgraph "Phase 2: Online – Retrieval & Generation"
[L172] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L173] KF --> L6
[L174] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L175] L7 --> A["Answer + Citations"]
[L176] end
[L177]
[L178] subgraph "Phase 3: Continuous – Operations"
[L179] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L180] A --> L8
[L181] L8 -->| "Self-Evolution
Feedback" | KF
[L182] end
[L183]
```

[L184]  
[L185] ### 3.2 Data Flow  
[L186]  
[L187]

```
[L188] Raw Document (PDF/DOCX/HTML)
[L189] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L190] Normalized Text + Metadata
[L191] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L192] [Page → Region → Block] hierarchy
[L193] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L194] KnowledgeUnit objects (candidate concepts)
[L195] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L196] KnowledgeNode objects (canonical, deduplicated)
[L197] ↓ Layer 4: Validate relations against ontology, score confidence
[L198] KnowledgeEdge objects (typed, scored)
[L199] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L200] Knowledge Forest (rooted trees + cross-links)
[L201] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L202] Selected context nodes (within token budget)
[L203] ↓ Layer 7: Build prompt, invoke LLM
[L204] ↓ Layer 7: Verify claims against evidence, bind citations
[L205] Answer with citations + confidence score
[L206] ↓ Layer 8: Log, collect feedback, update forest weights
[L207]
```

[L208]  
[L209] ### 3.3 Layer Summary  
[L210]  
[L211] | Layer | Name | Input | Output | Key Algorithms |  
[L212] |-----|-----|-----|-----|-----|  
[L213] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |  
[L214] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |  
[L215] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |  
[L216] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L217] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L218] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L219] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L220] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L221] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L222]  
[L223] ---  
[L224]  
[L225] ## 4. Data Model — Layer 0

Citation Marker: `[[filecite]]turn31file3[[/filecite]]`

[L226] metadata:  
[L227] query\_result\_index: 3  
[L228] file\_id: file\_00000000a0a481fab4c739c73d87551b

[L229] version\_id: 1

[L230] name: SHIA\_RAG\_Complete\_Report(7).md

[L231] mime\_type: text/markdown

[L232] surface: library

[L233] score: 0.030679156908665108

[L234] document\_chunk\_id: file\_00000000a0a481fab4c739c73d87551b--1--85

[L235] snippet:

[L236] ### 12.3 Next Steps

[L237]

[L238] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L239] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L240] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)

[L241] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L242] 5. **Build Layer 4** (Relation management with ontology validation)

[L243] 6. **Build Layer 5** (SHIA Core with corrected algorithms)

[L244] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L245] 8. **Build Layer 7** (Generation with verification)

[L246] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L247] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L248] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L249] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L250]

[L251] ### 12.4 Future Work

[L252]

[L253] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)

[L254] - **Multi-Tenancy**: Implement access control and document-level permissions

[L255] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L256] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies

[L257] - **Incremental Updates**: Support adding new documents without full forest reconstruction

[L258] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L259] - **Exploration in Self-Evolution**: Add  $\epsilon$ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias

[L260]

[L261] ---

[L262]

[L263] > [!TIP]

[L264] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0–8) are now fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

---

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `[[filecite]]turn31file4[[/filecite]]`

[L265] metadata:  
[L266] query\_result\_index: 4  
[L267] file\_id: file\_00000000f620821186e54e582d59731e  
[L268] version\_id: 1  
[L269] name: SHIA\_RAG\_Final\_Report(3).md  
[L270] mime\_type: text/markdown  
[L271] surface: library  
[L272] score: 0.03055037313432836  
[L273] document\_chunk\_id: file\_00000000f620821186e54e582d59731e--1--6  
[L274] snippet:  
[L275]

```
[L276] graph TD
[L277] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L278] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L279] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L280] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L281] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L282] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L283] end
[L284]
[L285] subgraph "Phase 2: Online – Retrieval & Generation"
[L286] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L287] KF --> L6
[L288] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L289] L7 --> A["Answer + Citations"]
[L290] end
[L291]
[L292] subgraph "Phase 3: Continuous – Operations"
[L293] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L294] A --> L8
[L295] L8 -->|"Self-Evolution
Feedback"| KF
[L296] end
[L297]
```

[L298]  
[L299] ### 3.2 Data Flow  
[L300]  
[L301]

```
[L302] Raw Document (PDF/DOCX/HTML)
[L303] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L304] Normalized Text + Metadata
[L305] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L306] [Page → Region → Block] hierarchy
[L307] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L308] KnowledgeUnit objects (candidate concepts)
[L309] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L310] KnowledgeNode objects (canonical, deduplicated)
[L311] ↓ Layer 4: Validate relations against ontology, score confidence
[L312] KnowledgeEdge objects (typed, scored)
[L313] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L314] Knowledge Forest (rooted trees + cross-links)
[L315] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L316] Selected context nodes (within token budget)
[L317] ↓ Layer 7: Build prompt, invoke LLM
[L318] ↓ Layer 7: Verify claims against evidence, bind citations
[L319] Answer with citations + confidence score
[L320] ↓ Layer 8: Log, collect feedback, update forest weights
[L321]
```

```
[L322]
[L323] ### 3.3 Layer Summary
[L324]
[L325] | Layer | Name | Input | Output | Key Algorithms |
[L326] |-----|-----|-----|-----|-----|
[L327] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |
[L328] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L329] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L330] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L331] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L332] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L333] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L334] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L335] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L336]
[L337] ---
[L338]
[L339] ## 4. Data Model — Layer 0
```

Citation Marker: `filecite[turn31file5]`

```
[L340] metadata:
[L341] query_result_index: 5
[L342] file_id: file_0000000033088207b45d2ae58cbd406c
[L343] version_id: 1
[L344] name: SHIA_RAG_Complete_Report(8).md
[L345] mime_type: text/markdown
[L346] surface: library
[L347] score: 0.0304147465437788
[L348] document_chunk_id: file_0000000033088207b45d2ae58cbd406c--1--85
```



[L349] snippet:

[L350] ### 12.3 Next Steps

[L351]

[L352] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L353] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L354] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)

[L355] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L356] 5. **Build Layer 4** (Relation management with ontology validation)

[L357] 6. **Build Layer 5** (SHIA Core with corrected algorithms)

[L358] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L359] 8. **Build Layer 7** (Generation with verification)

[L360] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L361] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L362] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L363] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L364]

[L365] ### 12.4 Future Work

[L366]

[L367] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)

[L368] - **Multi-Tenancy**: Implement access control and document-level permissions

[L369] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L370] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies

[L371] - **Incremental Updates**: Support adding new documents without full forest reconstruction

[L372] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L373] - **Exploration in Self-Evolution**: Add  $\epsilon$ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias

[L374]

[L375] ---

[L376]

[L377] > [!TIP]

[L378] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0–8) are now fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

---

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `filecite[turn31file6]`

[L379] metadata:

[L380] query\_result\_index: 6

[L381] file\_id: file\_00000000fb2c81fd82f14a7a48993554

[L382] version\_id: 1

[L383] name: SHIA\_RAG\_Final\_Report.md

[L384] mime\_type: text/markdown

[L385] surface: library

[L386] score: 0.030309988518943745

[L387] document\_chunk\_id: file\_00000000fb2c81fd82f14a7a48993554--1--6

[L388] snippet:

[L389]

```
[L390] graph TD
[L391] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L392] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L393] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L394] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L395] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L396] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L397] end
[L398]
[L399] subgraph "Phase 2: Online – Retrieval & Generation"
[L400] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L401] KF --> L6
[L402] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L403] L7 --> A["Answer + Citations"]
[L404] end
[L405]
[L406] subgraph "Phase 3: Continuous – Operations"
[L407] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L408] A --> L8
[L409] L8 -->|"Self-Evolution
Feedback"| KF
[L410] end
[L411]
```

[L412]

[L413] ### 3.2 Data Flow

[L414]

[L415]

```
[L416] Raw Document (PDF/DOCX/HTML)
[L417] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L418] Normalized Text + Metadata
[L419] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L420] [Page → Region → Block] hierarchy
[L421] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L422] KnowledgeUnit objects (candidate concepts)
[L423] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L424] KnowledgeNode objects (canonical, deduplicated)
[L425] ↓ Layer 4: Validate relations against ontology, score confidence
[L426] KnowledgeEdge objects (typed, scored)
[L427] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L428] Knowledge Forest (rooted trees + cross-links)
[L429] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L430] Selected context nodes (within token budget)
[L431] ↓ Layer 7: Build prompt, invoke LLM
[L432] ↓ Layer 7: Verify claims against evidence, bind citations
[L433] Answer with citations + confidence score
[L434] ↓ Layer 8: Log, collect feedback, update forest weights
[L435]
```

[L436]

[L437] ### 3.3 Layer Summary

[L438]

[L439] | Layer | Name | Input | Output | Key Algorithms |

[L440] |-----|-----|-----|-----|-----|

[L441] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |  
[L442] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |  
[L443] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |  
[L444] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L445] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L446] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L447] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L448] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L449] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L450]  
[L451] ---  
[L452]  
[L453] ## 4. Data Model — Layer 0

---

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `filecite[turn31file7]`

[L454] metadata:  
[L455] query\_result\_index: 7  
[L456] file\_id: file\_000000001dd481fa9855f2731dbc53d5  
[L457] version\_id: 1  
[L458] name: SHIA\_RAG\_Complete\_Report(6).md  
[L459] mime\_type: text/markdown  
[L460] surface: library  
[L461] score: 0.02967032967032967  
[L462] document\_chunk\_id: file\_000000001dd481fa9855f2731dbc53d5--1--85  
[L463] snippet:  
[L464] ### 12.3 Next Steps  
[L465]  
[L466] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)  
[L467] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)  
[L468] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)  
[L469] 4. **Build Layer 3** (Knowledge extraction with LLM)  
[L470] 5. **Build Layer 4** (Relation management with ontology validation)  
[L471] 6. **Build Layer 5** (SHIA Core with corrected algorithms)  
[L472] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)  
[L473] 8. **Build Layer 7** (Generation with verification)  
[L474] 9. **Build Layer 8** (Operations, monitoring, self-evolution)  
[L475] 10. **Run first evaluation** on OS textbook dataset using SHEF  
[L476] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)  
[L477] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027  
[L478]  
[L479] ### 12.4 Future Work  
[L480]  
[L481] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)  
[L482] - **Multi-Tenancy**: Implement access control and document-level permissions

[L483] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L484] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies

[L485] - **Incremental Updates**: Support adding new documents without full forest reconstruction

[L486] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L487] - **Exploration in Self-Evolution**: Add  $\epsilon$ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias

[L488]

[L489] ---

[L490]

[L491] > [!TIP]

[L492] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0–8) are now fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

---

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: `filecite[turn31file8]`

[L493] metadata:

[L494] query\_result\_index: 8

[L495] file\_id: file\_000000003c8081fab658b1752549d64

[L496] version\_id: 1

[L497] name: SHIA\_RAG\_Complete\_Report(5).md

[L498] mime\_type: text/markdown

[L499] surface: library

[L500] score: 0.029437229437229435

[L501] document\_chunk\_id: file\_000000003c8081fab658b1752549d64--1--85

[L502] snippet:

[L503] ### 12.3 Next Steps

[L504]

[L505] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L506] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L507] 3. **Build Layers 1–2** (Document ingestion and parsing pipeline)

[L508] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L509] 5. **Build Layer 4** (Relation management with ontology validation)

[L510] 6. **Build Layer 5** (SHIA Core with corrected algorithms)

[L511] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L512] 8. **Build Layer 7** (Generation with verification)

[L513] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L514] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L515] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L516] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L517]

[L518] ### 12.4 Future Work

[L519]

[L520] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)

- [L521] - **Multi-Tenancy**: Implement access control and document-level permissions
- [L522] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets
- [L523] - **Cross-Lingual Support**: Extend to multilingual documents and concept hierarchies
- [L524] - **Incremental Updates**: Support adding new documents without full forest reconstruction
- [L525] - **Advanced Verification**: Replace semantic similarity with NLI (Natural Language Inference) model for claim verification
- [L526] - **Exploration in Self-Evolution**: Add  $\epsilon$ -greedy or Thompson sampling to edge weight updates to avoid exploitation-only bias
- [L527]
- [L528] ---
- [L529]
- [L530] > [!TIP]
- [L531] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0–8) are now fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

DEEP RESEARCH 20/08/2026, 16:05:03

Citation Marker: [filecite]turn31file9[

- [L532] metadata:
- [L533] query\_result\_index: 9
- [L534] file\_id: file\_000000001dd481fa9855f2731dbc53d5
- [L535] version\_id: 1
- [L536] name: SHIA\_RAG\_Complete\_Report(6).md
- [L537] mime\_type: text/markdown
- [L538] surface: library
- [L539] score: 0.021635138073494237
- [L540] document\_chunk\_id: file\_000000001dd481fa9855f2731dbc53d5--1--55
- [L541] snippet:
- [L542] ##### Layer 7 → Layer 8 Data Contract
- [L543]
- [L544]

```

[L545] @dataclass
[L546] class QueryEvent:
[L547] """Complete event record passed from Layer 7 to Layer 8.
[L548] Contains everything needed for logging, monitoring, and self-evolution."""
[L549] event_id: str
[L550] timestamp: str
[L551] query: str
[L552] query_intent: str
[L553] query_complexity: str
[L554] seed_node_ids: list
[L555] retrieved_node_ids: list
[L556] selected_node_ids: list
[L557] traversal_path_edges: list # Edge IDs for feedback update
[L558] answer: str
[L559] claims: list # VerifiedClaim objects
[L560] citations: list
[L561] overall_confidence: float
[L562] model_used: str
[L563] latency_ms: float
[L564] token_count_context: int
[L565] token_count_answer: int
[L566] user_feedback: str = None # Set later: "HELPFUL" or "NOT_HELPFUL"
[L567]

```

[L568]

[L569] ---

[L570]

[L571] ### 7.9 Layer 8: Operations, Monitoring & Self-Evolution

[L572]

[L573] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L574]

[L575] **Input:** `QueryEvent` from Layer 7, user feedback

[L576] **Output:** Updated forest weights, monitoring metrics, alerts

[L577]

[L578] ##### Logging Schema

[L579]

[L580]

```

[L581] from dataclasses import dataclass, field
[L582] from datetime import datetime
[L583] import json
[L584]
[L585] @dataclass
[L586] class QueryLog:
[L587] """Structured log entry for every query processed."""
[L588] event_id: str
[L589] timestamp: str
[L590] # Query info
[L591] query: str
[L592] intent: str
[L593] complexity: str
[L594] # Retrieval info
[L595] seed_nodes: list
[L596] candidate_count: int
[L597] selected_count: int
[L598] traversal_edges: list # For feedback attribution
[L599] # Generation info
[L600] answer_length: int
[L601] claim_count: int
[L602] supported_claims: int
[L603] unsupported_claims: int
[L604] overall_confidence: float
[L605] model_used: str
[L606] latency_ms: float
[L607] # Feedback (filled later)
[L608] user_feedback: str = None # HELPFUL / NOT_HELPFUL
[L609] feedback_timestamp: str = None
[L610]
[L611] def log_query_event(event: 'QueryEvent', db):
[L612] """Persist a complete query event to the logging database."""
[L613] log = QueryLog(
[L614] event_id=event.event_id,
[L615] timestamp=event.timestamp,
[L616] query=event.query,
[L617] intent=event.query_intent,
[L618] complexity=event.query_complexity,
[L619] seed_nodes=event.seed_node_ids,
[L620] candidate_count=len(event.retrieved_node_ids),
[L621] selected_count=len(event.selected_node_ids),
[L622] traversal_edges=[e.edge_id for e in event.traversal_path_edges],
[L623] answer_length=len(event.answer),
[L624] claim_count=len(event.claims),
[L625] supported_claims=sum(1 for c in event.claims if c.verification_status == 'SUPPORTED'),
[L626] unsupported_claims=sum(1 for c in event.claims if c.verification_status == 'UNSUPPORTED'),
[L627] overall_confidence=event.overall_confidence,
[L628] model_used=event.model_used,
[L629] latency_ms=event.latency_ms,
[L630])
[L631] db.insert('query_logs', log)
[L632] return log
[L633]

```

```

[L634]
[L635] ##### Feedback Acquisition & Attribution
[L636]
[L637] ```python
[L638] def collect_feedback(event_id: str, feedback: str, forest, db):
[L639] """Collect user feedback and trigger self-evolution update.
[L640]

```

[L641] Feedback is collected via API endpoint:

[L642] POST /api/feedback { "event\_id": "...",

---

DEEP RESEARCH 20/08/2026, 16:05:07

Citation Marker: □filecite□turn32file0□

[L1] metadata:

[L2] ref\_id: turn31file9

[L3] file\_id: file\_000000001dd481fa9855f2731dbc53d5

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(6).md

[L6] mode: chunk\_context

[L7] lines\_shown: 1-120

[L8] total\_file\_lines: 421

[L9] has\_more: True

[L10] next\_start\_line: 121

[L11] next\_read:

{"ref\_id":"turn31file9","mode":"chunk\_context","start\_line":121,"context\_before\_lines":20,"context\_after\_lines":400}

[L12] content:

[L13] query\_complexity: str

[L14] seed\_node\_ids: list

[L15] retrieved\_node\_ids: list

[L16] selected\_node\_ids: list

[L17] traversal\_path\_edges: list # Edge IDs for feedback update

[L18] answer: str

[L19] claims: list # VerifiedClaim objects

[L20] citations: list

[L21] overall\_confidence: float

[L22] model\_used: str

[L23] latency\_ms: float

[L24] token\_count\_context: int

[L25] token\_count\_answer: int

[L26] user\_feedback: str = None # Set later: "HELPFUL" or "NOT\_HELPFUL"

[L27]

```
[L28]
[L29] ---
[L30]
[L31] ### 7.9 Layer 8: Operations, Monitoring & Self-Evolution
[L32]
[L33] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution),
[L34]
[L35] **Input:** `QueryEvent` from Layer 7, user feedback
[L36] **Output:** Updated forest weights, monitoring metrics, alerts
[L37]
[L38] #### Logging Schema
[L39]
[L40]
```



```
python
[L41] from dataclasses import dataclass, field
[L42] from datetime import datetime
[L43] import json
[L44]
[L45] @dataclass
[L46] class QueryLog:
[L47] """Structured log entry for every query processed."""
[L48] event_id: str
[L49] timestamp: str
[L50] # Query info
[L51] query: str
[L52] intent: str
[L53] complexity: str
[L54] # Retrieval info
[L55] seed_nodes: list
[L56] candidate_count: int
[L57] selected_count: int
[L58] traversal_edges: list # For feedback attribution
[L59] # Generation info
[L60] answer_length: int
[L61] claim_count: int
[L62] supported_claims: int
[L63] unsupported_claims: int
[L64] overall_confidence: float
[L65] model_used: str
[L66] latency_ms: float
[L67] # Feedback (filled later)
[L68] user_feedback: str = None # HELPFUL / NOT_HELPFUL
[L69] feedback_timestamp: str = None
[L70]
[L71] def log_query_event(event: 'QueryEvent', db):
[L72] """Persist a complete query event to the logging database."""
[L73] log = QueryLog(
[L74] event_id=event.event_id,
[L75] timestamp=event.timestamp,
[L76] query=event.query,
[L77] intent=event.query_intent,
[L78] complexity=event.query_complexity,
[L79] seed_nodes=event.seed_node_ids,
[L80] candidate_count=len(event.retrieved_node_ids),
[L81] selected_count=len(event.selected_node_ids),
[L82] traversal_edges=[e.edge_id for e in event.traversal_path_edges],
[L83] answer_length=len(event.answer),
[L84] claim_count=len(event.claims),
[L85] supported_claims=sum(1 for c in event.claims if c.verification_status == 'SUPPORTED'),
```

```

[L86] unsupported_claims=sum(1 for c in event.claims if c.verification_status == 'UNSUPPORTED'),
[L87] overall_confidence=event.overall_confidence,
[L88] model_used=event.model_used,
[L89] latency_ms=event.latency_ms,
[L90])
[L91] db.insert('query_logs', log)
[L92] return log
[L93]

```

```

[L94]
[L95] ##### Feedback Acquisition & Attribution
[L96]
[L97]

```

python

```

[L98] def collect_feedback(event_id: str, feedback: str, forest, db):
[L99] """Collect user feedback and trigger self-evolution update.
[L100]
[L101] Feedback is collected via API endpoint:
[L102] POST /api/feedback { "event_id": "...", "feedback": "HELPFUL" | "NOT_HELPFUL" }
[L103]
[L104] Attribution strategy:
[L105] - Feedback applies ONLY to edges in the recorded traversal_path
[L106] - This isolates retrieval quality from answer quality
[L107] - Layer 7 verification quality is tracked separately via claim metrics
[L108] """
[L109] # Retrieve the original query event
[L110] log = db.get('query_logs', event_id)
[L111] if not log:
[L112] raise ValueError(f"Event {event_id} not found")
[L113]
[L114] # Update log with feedback
[L115] log.user_feedback = feedback
[L116] log.feedback_timestamp = datetime.now().isoformat()
[L117] db.update('query_logs', log)
[L118]
[L119] # Self-evolution: update edge weights along traversal path
[L120] update_forest_from_feedback(
[L121] query=log.query,
[L122] retrieved_nodes=None, # Not needed; path edges are stored
[L123] selected_nodes=None,
[L124] user_feedback=feedback,
[L125] forest=forest,
[L126] path_edges=log.traversal_edges # Pre-recorded edges
[L127])
[L128]
[L129] # NOTE: update_forest_from_feedback() is defined in Section 5 (Novelty 2)

```

[L130] # with exponential update rule:  $w = \exp(\eta f)$ ,  $\eta=0.02$

[L131] ```

---

DEEP RESEARCH 20/08/2026, 16:05:12

Citation Marker: [filecite[turn33file0]

[L1] metadata:

[L2] ref\_id: turn31file9

[L3] file\_id: file\_000000001dd481fa9855f2731dbc53d5

[L4] version\_id: 1

[L5] name: SHIA\_RAG\_Complete\_Report(6).md

[L6] mode: chunk\_context

[L7] lines\_shown: 121-240

[L8] total\_file\_lines: 421

[L9] has\_more: True

[L10] next\_start\_line: 241

[L11] next\_read:

{"ref\_id":"turn31file9","mode":"chunk\_context","start\_line":241,"context\_before\_lines":20,"context\_after\_lines":400}

[L12] content:

[L13] > [!IMPORTANT]

[L14] > **Feedback attribution isolation:** User feedback like "HELPFUL" reflects **overall answer quality**, which includes both retrieval AND generation. To avoid incorrectly crediting/blaming retrieval edges:

[L15] > - Edge weight updates use a **dampened learning rate**  $\eta=0.02$  (small increments)

[L16] > - Layer 7 verification metrics (supported/unsupported claims) are tracked **separately**

[L17] > - If `overall_confidence < 0.5 AND user_feedback == "HELPFUL"`, the system logs a warning (possible false positive)

[L18] > - If `overall_confidence > 0.8 AND user_feedback == "NOT_HELPFUL"`, the system logs a warning (possible answer format issue, not retrieval issue)

[L19]

[L20] ##### Edge Weight → Retrieval Integration (Closing the Loop)

[L21]

[L22]

```
[L23] def utility(node, forest):
[L24] """Utility function for Tree Knapsack – NOW uses edge weights from self-evolution.
[L25]
[L26] This is how the self-evolution loop is CLOSED:
[L27] Feedback → edge.weight update → utility() uses edge.weight →
[L28] Tree Knapsack selects differently → future retrieval improves.
[L29] """
[L30] if not node.parent:
[L31] return node.confidence * len(node.evidence)
[L32]
[L33] # Get the edge connecting this node to its parent
[L34] parent_edge = forest.get_edge(node.parent, node.id)
[L35] edge_weight = parent_edge.weight if parent_edge else 1.0
[L36]
[L37] return (
[L38] node.confidence * # Node quality
[L39] edge_weight * # Self-evolution signal (NOVELTY 2)
[L40] len(node.evidence) * # Evidence richness
[L41] (1.0 / max(token_cost(node), 1)) # Cost efficiency
[L42])
[L43]
```

[L44]

[L45] ##### Monitoring Metrics

[L46]

[L47]

```

[L48] MONITORING_CONFIG = {
[L49] 'evaluation_window': '1h', # Rolling window for metrics
[L50] 'alert_thresholds': {
[L51] 'avg_confidence': 0.4, # Alert if avg confidence drops below
[L52] 'feedback_ratio': 0.6, # Alert if helpful/total < 60%
[L53] 'orphan_rate': 0.1, # Alert if orphan nodes > 10%
[L54] 'latency_p95_ms': 5000, # Alert if P95 latency > 5s
[L55] 'error_rate': 0.05, # Alert if errors > 5%
[L56] }
[L57] }
[L58]
[L59] def compute_monitoring_metrics(db, forest, window='1h') -> dict:
[L60] """Compute system health metrics over a time window.
[L61]
[L62] These metrics are logged to the monitoring dashboard and trigger alerts.
[L63] Monitoring = real-time health. Evaluation = periodic quality assessment (SHEF).
[L64] """
[L65] logs = db.get_recent_logs(window)
[L66]
[L67] if not logs:
[L68] return {'status': 'NO_DATA'}
[L69]
[L70] total = len(logs)
[L71] with_feedback = [l for l in logs if l.user_feedback is not None]
[L72]
[L73] metrics = {
[L74] # Retrieval metrics
[L75] 'avg_confidence': sum(l.overall_confidence for l in logs) / total,
[L76] 'avg_selected_nodes': sum(l.selected_count for l in logs) / total,
[L77] 'avg_latency_ms': sum(l.latency_ms for l in logs) / total,
[L78]
[L79] # Feedback metrics
[L80] 'feedback_ratio': (
[L81] sum(1 for l in with_feedback if l.user_feedback == 'HELPFUL')
[L82] / max(len(with_feedback), 1) # Denominator: queries WITH feedback
[L83]),
[L84] 'feedback_coverage': len(with_feedback) / total, # % of queries that got feedback
[L85]
[L86] # Generation metrics
[L87] 'avg_claim_support_ratio': sum(
[L88] l.supported_claims / max(l.claim_count, 1) for l in logs
[L89]) / total,
[L90]
[L91] # Forest health
[L92] 'total_nodes': forest.node_count(),
[L93] 'total_edges': forest.edge_count(),
[L94] 'orphan_rate': forest.orphan_count() / max(forest.node_count(), 1), # Nodes with no parent are orphans
[L95] 'avg_edge_weight': forest.avg_edge_weight(),
[L96] 'flagged_edges': len(forest.restructure_queue),
[L97]
[L98] # Error rate
[L99] 'error_rate': sum(1 for l in logs if l.overall_confidence == 0) / total,
[L100] }
[L101]
[L102] # Check alert thresholds
[L103] alerts = []
[L104] for metric, threshold in MONITORING_CONFIG['alert_thresholds'].items():
[L105] if metric in metrics:
[L106] if metric == 'orphan_rate' or metric == 'error_rate':
[L107] if metrics[metric] > threshold:
[L108] alerts.append(f"⚠️ {metric} = {metrics[metric]:.3f} (threshold: {threshold})")
[L109] else:
[L110] if metrics[metric] < threshold:
[L111] alerts.append(f"⚠️ {metric} = {metrics[metric]:.3f} (threshold: {threshold})")
[L112]
[L113] metrics['alerts'] = alerts

```

```
[L114] return metrics
[L115]
```

```
[L116]
[L117] ##### Edge Weight Dynamics
[L118]
[L119] ```python
[L120] # Edge weight management to prevent long-term drift
[L121]
[L122] def normalize_edge_weights(forest, target_mean=1.0):
[L123] """Periodically normalize edge weights to prevent global drift.
[L124]
[L125] Run after every REBALANCE_INTERVAL queries.
[L126] Preserves relative ordering while keeping weights in a manageable range.
[L127] """
[L128] all_weights = [e.weight for e in forest.get_all_edges(edge_class='HIERARCHICAL')]
[L129] if not all_weights:
[L130] return
[L131]
[L132] current_mean = sum(all_weights) / len(all_weights)
```

---

DEEP RESEARCH 20/08/2026, 16:05:15

Citation Marker: `filecite[turn34file0]`

```
[L1] metadata:
[L2] ref_id: turn31file9
[L3] file_id: file_000000001dd481fa9855f2731dbc53d5
[L4] version_id: 1
[L5] name: SHIA_RAG_Complete_Report(6).md
[L6] mode: chunk_context
[L7] lines_shown: 241-360
[L8] total_file_lines: 421
[L9] has_more: True
[L10] next_start_line: 361
[L11] next_read:
{"ref_id":"turn31file9","mode":"chunk_context","start_line":361,"context_before_lines":20,"context_after_lines":400}
[L12] content:
[L13] scale = target_mean / current_mean
[L14]
[L15] for edge in forest.get_all_edges(edge_class='HIERARCHICAL'):
[L16] edge.weight = max(WEIGHT_MIN, min(edge.weight scale, WEIGHT_MAX))
[L17]
[L18] def apply_weight_decay(forest, decay_rate=0.999):
[L19] """Apply slow decay to all edge weights — recent feedback matters more.
[L20]
[L21] Called periodically (e.g., daily) to ensure old feedback gradually fades.
```

[L22] """

[L23] for edge in forest.get\_all\_edges(edge\_class='HIERARCHICAL'):

[L24] edge.weight = max(WEIGHT\_MIN, edge.weight decay\_rate)

[L25]

[L26]

[L27] ---

[L28]

[L29] ### 7.10 Schema Validation (Pydantic Models)

[L30]

[L31]

python

[L32] from pydantic import BaseModel, Field, validator

[L33] from typing import List, Optional, Dict, Any

[L34] from datetime import datetime

[L35]

[L36] class EvidenceEntry(BaseModel):

[L37] evidence\_type: str = Field(..., regex="^(DEFINITION|RELATION|PROPERTY)\$")

[L38] text: str

[L39] source\_doc: str = Field(..., regex="^DOC\_\\d+\$")

[L40] source\_block: str = Field(..., regex="^BLK\_\\d+\$")

[L41] page: int = Field(..., ge=1)

[L42] relation\_type: Optional[str] = None

[L43] target\_node: Optional[str] = None

[L44] property\_key: Optional[str] = None

[L45]

[L46] class EmbeddingEntry(BaseModel):

[L47] model: str

[L48] dimension: int = Field(..., ge=1)

[L49] vector: List[float]

[L50] created\_at: datetime

[L51]

[L52] @validator('vector')

[L53] def check\_dimension(cls, v, values):

[L54] if 'dimension' in values and len(v) != values['dimension']:

[L55] raise ValueError(f"Vector length {len(v)} != declared dimension {values['dimension']}")

[L56] return v

[L57]

[L58] class KnowledgeNode(BaseModel):

[L59] node\_id: str = Field(..., regex="^NODE\_\\d+\$")

[L60] uuid: str

[L61] canonical\_name: str = Field(..., min\_length=1)

[L62] aliases: List[str] = []

[L63] definitions: List[str] = Field(..., min\_items=1)

[L64] domain: str = "General"

[L65] properties: Dict[str, Any] = {}

[L66] evidence: List[EvidenceEntry] = []

[L67] embeddings: List[EmbeddingEntry] = []

```
[L68] confidence: float = Field(0.5, ge=0.0, le=1.0)
[L69] base_confidence: float = Field(0.5, ge=0.0, le=1.0)
[L70] extraction_confidence: float = Field(0.5, ge=0.0, le=1.0)
[L71] abstraction_level: float = Field(0.5, ge=0.0, le=1.0)
[L72]
[L73] class KnowledgeEdge(BaseModel):
[L74] edge_id: str = Field(..., regex="^EDGE_\\d+$")
[L75] source_node: str = Field(..., regex="^NODE_\\d+$")
[L76] target_node: str = Field(..., regex="^NODE_\\d+$")
[L77] edge_class: str = Field(..., regex="^(HIERARCHICAL|SEMANTIC)$")
[L78] relation_type: str
[L79] confidence: float = Field(..., ge=0.0, le=1.0)
[L80] weight: float = Field(1.0, ge=0.1, le=10.0)
[L81]
```

```
[L82]
[L83] ---
[L84]
[L85] ### 7.11 Database Indexing Strategy
[L86]
[L87]
```

sql

```
[L88] -- PostgreSQL: Relational data (documents, pages, blocks)
[L89] CREATE INDEX idx_node_id ON knowledge_nodes(node_id);
[L90] CREATE INDEX idx_canonical_name ON knowledge_nodes(canonical_name);
[L91] CREATE INDEX idx_domain ON knowledge_nodes(domain);
[L92] CREATE INDEX idx_confidence ON knowledge_nodes(confidence);
[L93]
[L94] -- Neo4j: Graph traversal indexes
[L95] CREATE INDEX node_id_index FOR (n:KnowledgeNode) ON (n.node_id);
[L96] CREATE INDEX canonical_name_index FOR (n:KnowledgeNode) ON (n.canonical_name);
[L97] CREATE INDEX domain_index FOR (n:KnowledgeNode) ON (n.domain);
[L98]
[L99] -- Vector DB (Milvus): Embedding search
[L100] -- IVF_FLAT index for approximate nearest neighbor search
[L101] -- nlist=128 for datasets up to 1M vectors
[L102] CREATE INDEX embedding_index ON knowledge_embeddings
[L103] USING IVF_FLAT (embedding vector_cosine_ops)
[L104] WITH (nlist = 128);
[L105]
```

```
[L106]
[L107] ---
[L108]
[L109] ### 7.12 Migration Strategy
[L110]
[L111]
```

python

```
[L112] MIGRATIONS = [
```



```

[L113] {
[L114] "version": 1,
[L115] "description": "Initial schema",
[L116] "up": ""CREATE TABLE knowledge_nodes (
[L117] node_id VARCHAR PRIMARY KEY,
[L118] canonical_name VARCHAR NOT NULL,
[L119] domain VARCHAR DEFAULT 'General',
[L120] confidence FLOAT DEFAULT 0.5,
[L121] data JSONB
[L122])" "",
[L123] "down": "DROP TABLE knowledge_nodes"
[L124] },
[L125] {
[L126] "version": 2,
[L127] "description": "Add abstraction level and base confidence",
[L128] "up": ""ALTER TABLE knowledge_nodes
[L129] ADD COLUMN abstraction_level FLOAT DEFAULT 0.5,
[L130] ADD COLUMN base_confidence FLOAT DEFAULT 0.5,
[L131] ADD COLUMN extraction_confidence FLOAT DEFAULT 0.5""",
[L132] "down": ""ALTER TABLE knowledge_nodes

```

---

CHATGPT 20/08/2026, 16:05:15

## Layer 8 — Operations, Monitoring & Self-Evolution

Ab hum **same Smart Hospital example** continue karenge, taaki Layer 0 se Layer 8 tak ek hi example se complete project samajh aaye.

### Same query:

*"How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?"*

Layer 7 tak system ne:

```

Question
↓
Knowledge retrieve ki
↓
Answer generate kiya
↓
Claims verify kiye
↓
Citations bind ki
↓
Confidence calculate ki

```

### Ab Layer 8 ka kaam shuru hota hai

Layer 8 ka responsibility hai:

*Har query event ko log karna, user feedback collect karna, us feedback ke basis par Knowledge Forest ke edge weights update karna, system health monitor karna, aur performance problems par alerts generate karna.*

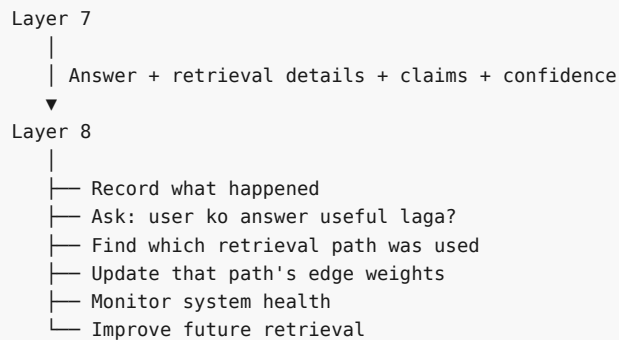
filecite turn32file0 L31-L36

## 1. Layer 8 ko simple language me samjho

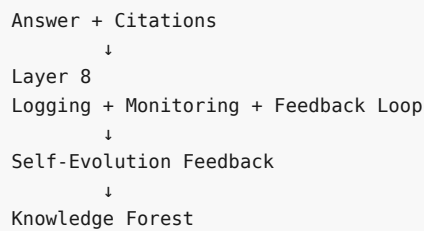
Agar Layer 7 system ka answer dene wala brain hai, to Layer 8 system ka:

### Memory + Performance Monitor + Learning Loop

hai.



Project architecture me Layer 8 continuous operations phase hai:



Isliye Layer 8 answer generate nahi karta; future retrieval ko improve karne ke liye past query outcomes se system ko learn karwata hai. filecite turn31file0 L21-L31

## 2. Layer 8 ka exact input kya hai?

Layer 8 ka input:

QueryEvent

plus later:

User Feedback

Report ke according:

```
Input:
QueryEvent from Layer 7
+
User feedback

Output:
Updated forest weights
+
Monitoring metrics
+
Alerts
```

filecite\turn32file0\L31-L36

---

### 3. QueryEvent kya hota hai?

Ye bahut important object hai.

Layer 7 ka sirf final answer Layer 8 ko nahi milta.

Layer 8 ko **poori query journey ka record** milta hai.

```
QueryEvent
|
├─ event_id
├─ timestamp
|
├─ query
├─ query_intent
├─ query_complexity
|
├─ seed_node_ids
├─ retrieved_node_ids
├─ selected_node_ids
├─ traversal_path_edges
|
├─ answer
├─ claims
├─ citations
├─ overall_confidence
|
├─ model_used
├─ latency_ms
|
├─ token_count_context
├─ token_count_answer
|
└─ user_feedback
```

Report me ye Layer 7 → Layer 8 data contract ke roop me defined hai. filecite\turn31file9\L542-L567

---

### 4. Hamare Smart Hospital example ka QueryEvent

Suppose:

```
event_id = "EVT_000001"
timestamp = "2026-08-20T10:30:00"

query =
"How does the AI Diagnostic Engine analyze patient symptoms
and medical history to estimate emergency risk, and what role
does the Machine Learning Model play?"

query_intent = "EXPLANATORY"
query_complexity = "COMPLEX"
```

## Retrieval information

Layer 6 ne seed nodes find kiye:

```
seed_node_ids =
[
 NODE_001: Smart Hospital AI System,
 NODE_002: AI Diagnostic Engine
]
```

Traversal path:

```
Smart Hospital AI System
|
EDGE_101
▼
AI Diagnostic Engine
|
EDGE_102
▼
Patient Symptoms
|
EDGE_103
▼
Medical History
|
AI Diagnostic Engine
|
EDGE_104
▼
Machine Learning Model
|
AI Diagnostic Engine
|
EDGE_105
▼
Emergency Risk
```

Therefore:

```
traversal_path_edges =
[
 EDGE_101,
 EDGE_102,
 EDGE_103,
 EDGE_104,
 EDGE_105
]
```

Ye edges Layer 8 ke self-evolution ke liye extremely important hain.

## 5. Layer 8 ko `traversal_path_edges` kyun chahiye?

Maan lo user answer ke baad click karta hai:

👍 HELPFUL

Ab system ko pata hona chahiye:

*Is successful answer tak pahunchne ke liye kaunsa retrieval path use hua tha?*

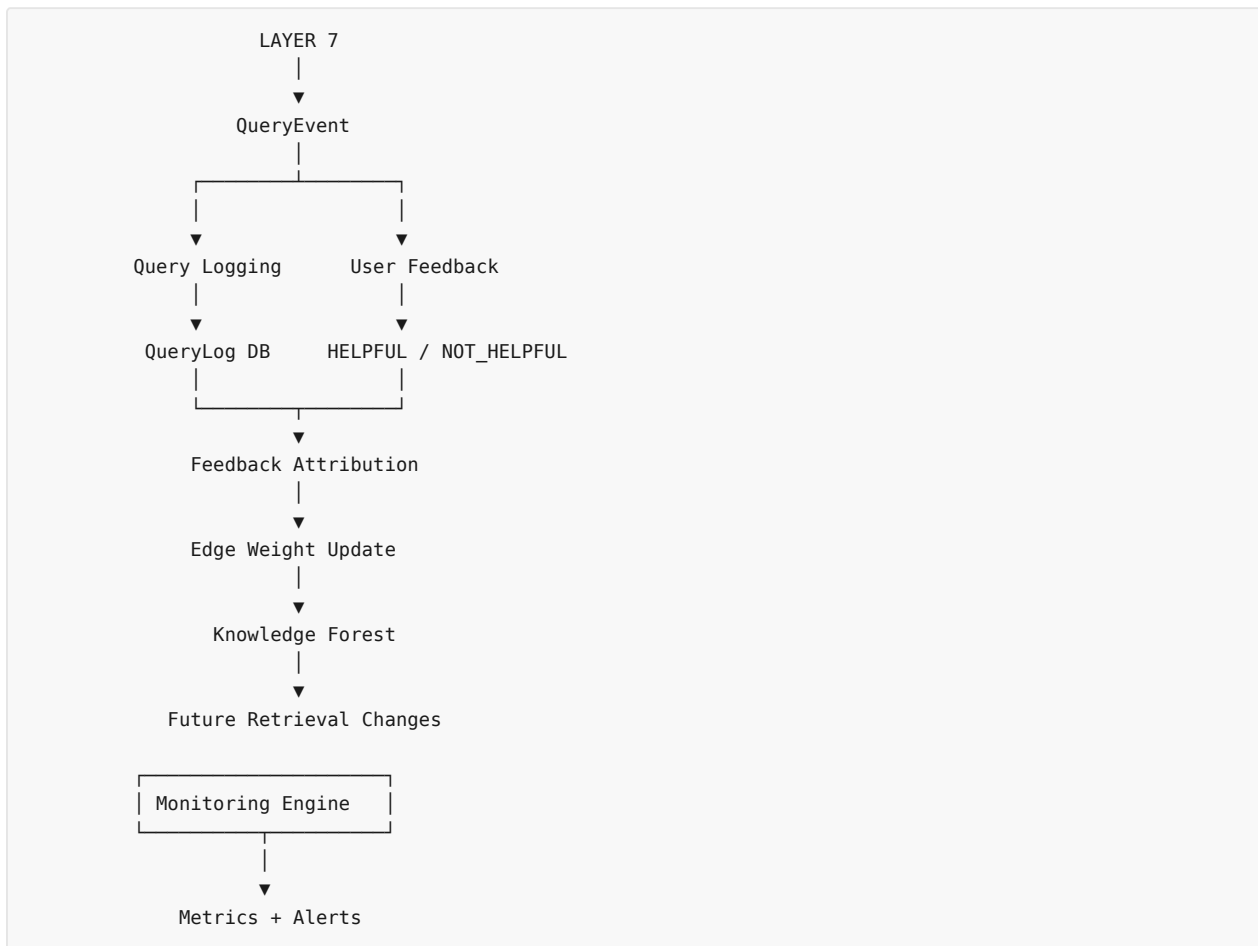
Answer alone se system ye nahi jaan sakta.

Isliye Layer 6 se exact path store kiya gaya:

```
Query
↓
EDGE_101
↓
EDGE_102
↓
EDGE_103
↓
EDGE_104
↓
EDGE_105
↓
Final Answer
```

Phir Layer 8 in edges ko feedback attribution ke liye use karta hai. `filecite\turn32file0\L95-L107`

## 6. Layer 8 ka complete pipeline



## 7. Step 1 — Query Logging

Jab Layer 7 answer complete karta hai, Layer 8 sabse pehle event ko log karta hai.

Logging schema me information teen major groups me divided hai:

1. Query Information
2. Retrieval Information
3. Generation Information

plus later:

4. User Feedback

filecite\turn32file0\L38-L69

## 8. Query information me kya log hota hai?

```

event_id
timestamp
query
intent
complexity

```

Example:

```
event_id = EVT_000001

query =
How does the AI Diagnostic Engine...

intent =
EXPLANATORY

complexity =
COMPLEX
```

### Ye kyun important hai?

Later system analyze kar sakta hai:

*Kya COMPLEX queries par confidence low aa raha hai?*

Ya:

*Kya EXPLANATORY queries ka latency ज्यादा hai?*

## 9. Retrieval information me kya log hota hai?

```
seed_nodes
candidate_count
selected_count
traversal_edges
```

Report ke logging schema me ye explicitly stored hai. `filecite\turn32file0\L54-L58`

Hamare example me:

```
seed_nodes =
[
 NODE_001,
 NODE_002
]

candidate_count = 15

selected_count = 6

traversal_edges =
[
 EDGE_101,
 EDGE_102,
 EDGE_103,
 EDGE_104,
 EDGE_105
]
```

## 10. candidate\_count aur selected\_count me difference

Ye important hai.

Suppose Layer 6 ne initially:

```
15 candidate nodes
```

find kiye.

But token budget ke andar Tree Knapsack ke baad:

6 nodes

final context me select hue.

So:

```
Candidate Nodes = 15
 ↓
Selection
 ↓
Selected Nodes = 6
```

Layer 8 dono ko log karta hai.

### Why?

Because future monitoring me system dekh sakta hai:

Candidates bahut zyada  
but selected nodes bahut kam

ya:

Candidates hi bahut kam

Ye retrieval behavior ko diagnose karne me useful hai.

## 11. Generation information me kya log hota hai?

```
answer_length
claim_count
supported_claims
unsupported_claims
overall_confidence
model_used
latency_ms
```

filecite\turn32file0\L59-L66

Hamare example me suppose:

```
answer_length = 420 characters

claim_count = 4

supported_claims = 4

unsupported_claims = 0

overall_confidence = 0.89

model_used = gpt-4o-mini

latency_ms = 1850
```

## 12. Complete QueryLog hamare example me



Conceptually:

```
QueryLog
├── EVT_000001
├── QUERY
│ ├── intent = EXPLANATORY
│ └── complexity = COMPLEX
├── RETRIEVAL
│ ├── seed_nodes = [NODE_001, NODE_002]
│ ├── candidate_count = 15
│ ├── selected_count = 6
│ └── traversal_edges =
│ └── [EDGE_101...EDGE_105]
├── GENERATION
│ ├── answer_length = 420
│ ├── claim_count = 4
│ ├── supported_claims = 4
│ ├── unsupported_claims = 0
│ ├── confidence = 0.89
│ ├── model = gpt-4o-mini
│ └── latency = 1850 ms
└── FEEDBACK
 └── None initially
```

### 13. Step 2 — Database me event save karna

Conceptually:

```
QueryEvent
 ↓
log_query_event()
 ↓
QueryLog object
 ↓
db.insert('query_logs', log)
```

Report ke code me structured log database me persist hota hai. `filecite\turn32file0\L71-L92`

### 14. Ab user ko answer diya jata hai

User ko Layer 7 ka answer dikh raha hai.

For example:

*The AI Diagnostic Engine analyzes patient symptoms and medical history to estimate emergency risk...*

Ab user feedback de sakta hai:

👍 HELPFUL

or:

👎 NOT\_HELPFUL

Feedback API conceptually:

```
POST /api/feedback
```

```
{
 "event_id": "EVT_000001",
 "feedback": "HELPFUL"
}
```

Report me exactly ye two feedback states defined hain:

```
HELPFUL
NOT_HELPFUL
```

filecite\turn32file0\L95-L117

## 15. Feedback aane par Layer 8 kya karta hai?

```
event_id
+
feedback
↓
Original QueryLog fetch
↓
Feedback update
↓
Timestamp add
↓
Traversal edges retrieve
↓
Self-evolution update
```

Code conceptually:

```
db.get('query_logs', event_id)
```

se original event fetch hota hai.

Phir:

```
log.user_feedback = feedback
log.feedback_timestamp = now
```

Then:

```
db.update(...)
```

Finally traversal path ke edges update ke liye use hote hain. filecite\turn32file0\L98-L130

## 16. Sabse important concept — Feedback kis par apply hota hai?

Report me clearly:

*Feedback **ONLY** recorded traversal path ke edges par apply hota hai. filecite\turn32file0\L104-L107*

Hamare example me:

HELPFUL

↓

Only these edges:

EDGE\_101 ✓

EDGE\_102 ✓

EDGE\_103 ✓

EDGE\_104 ✓

EDGE\_105 ✓

Random forest ke baaki edges:

EDGE\_200 ✗

EDGE\_201 ✗

EDGE\_999 ✗

update nahi honge.

## 17. Ye “feedback attribution” kya hota hai?

Suppose Knowledge Forest me:

100,000 edges

hain.

User ne ek answer ko helpful bola.

Agar system sab edges ko reward kar de:

✗ Completely wrong

System ko specifically pata hai:

Is answer ke retrieval ke time  
kaunsa path actually traverse hua tha?

Therefore:

Feedback

↓

Recorded traversal path

↓

Only those edges

Yahi **feedback attribution** hai.

## 18. Step 3 — Self-Evolution

Ab Layer 8 ka most interesting part.

User feedback:

HELPFUL

System ka interpretation:

| *Is retrieval path ka use karke answer mila aur user ne useful bola.*

To path ki importance increase hoti hai.

```
Future retrieval
me
ye path relatively more attractive ho jayega
```

## Conceptually

Before feedback:

```
EDGE_101 weight = 1.00
EDGE_102 weight = 1.00
EDGE_103 weight = 1.00
```

User:

👍 HELPFUL

Then:

```
Feedback
↓
Weight update
↓
EDGE_101 = increased
EDGE_102 = increased
EDGE_103 = increased
```

Exact update function report me Section 5 ke `update_forest_from_feedback()` se connected hai aur exponential update rule:

$$w \leftarrow w \times e^{\eta F}$$

with:

$$\eta = 0.02$$

use karta hai. `turn32file0L119-L130`

## 19. $\eta = 0.02$ itna small kyun hai?

Report explicitly small/dampened learning rate use karta hai.

Reason:

```
User feedback
≠
Pure retrieval quality
```

User answer ko helpful bolta hai, but answer quality me:

```

Retrieval quality
+
LLM generation quality
+
Answer format
+
User expectation

```

sab include hote hain.

Isliye system directly huge edge update nahi karta.

Report specifically feedback attribution isolation ke liye:

```

η = 0.02

```

small increment use karta hai. `filecite\turn33file0\13-L18`

## 20. Example: HELPFUL feedback

Suppose:

```

Initial weight = 1.0
η = 0.02
feedback signal f = +1

```

Formula:

```

\[\]
w_{new}=w_{old}\times e^{\eta f}
\[\]

```

So:

```

\[\]
w_{new}=1.0\times e^{\{0.02\}}
\[\]

```

Approximately:

```

\[\]
w_{new}\approx1.0202
\[\]

```

So weight:

```

1.0000
↓
1.0202

```

Yahan numerical example formula ka application hai.

## 21. NOT\_HELPFUL feedback me?

Agar feedback signal negative ho:

```

f = -1

```

Then:

$$w_{\text{new}} = w_{\text{old}} \times e^{-0.02}$$

For:

$$w_{\text{old}} = 1$$

approximately:

$$\begin{array}{c} 1.0000 \\ \downarrow \\ 0.9802 \end{array}$$

Meaning:

*Path ko destroy nahi kiya jata. Bas slightly less preferred banaya jata hai.*

Yahi reason hai small learning rate ka.

## 22. Lekin Layer 8 directly future answer ko change kaise karta hai?

Bahut important question.

Suppose Layer 8 ne:

EDGE\_104

ka weight increase kiya.

Future query me Layer 6 utility calculate karta hai.

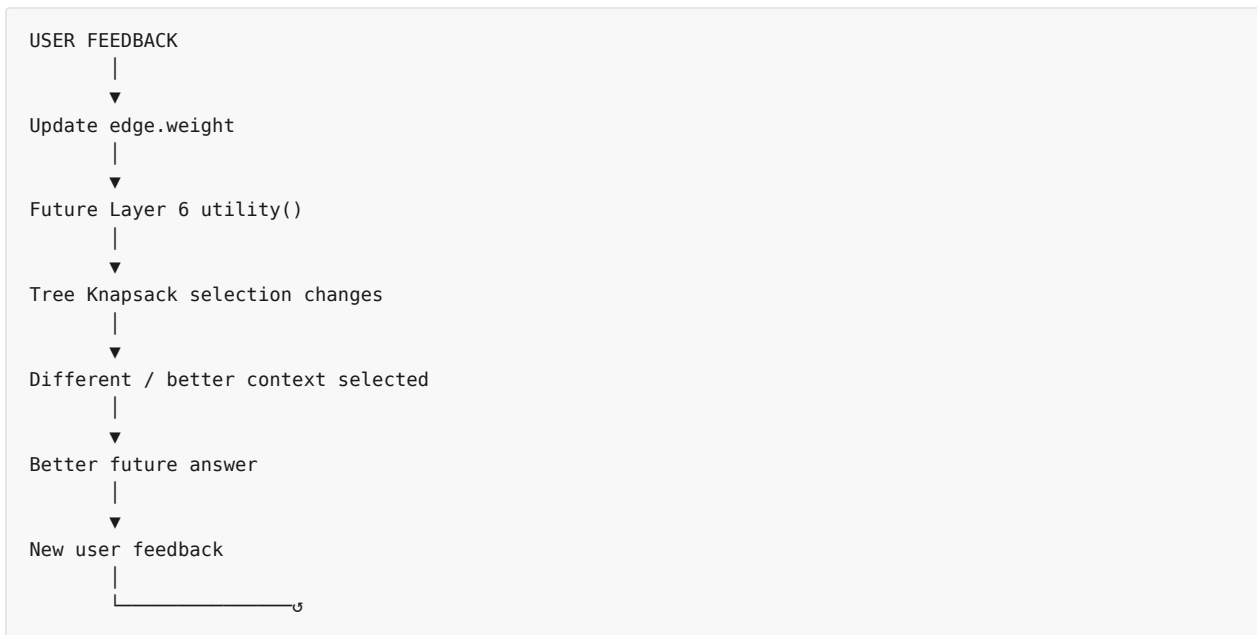
Report ka utility function:

$$\text{Utility} = \text{NodeConfidence} \times \text{EdgeWeight} \times \text{EvidenceRichness} \times \text{CostEfficiency}$$

Hierarchical child nodes ke liye parent edge weight utility calculation me directly use hota hai.

filecite[turn33file0L20-L42]

## 23. Self-evolution loop actually close kaise hota hai?



Report khud is loop ko:

*Feedback → edge.weight update → utility() uses edge.weight → Tree Knapsack selects differently → future retrieval improves*

ke flow me define karta hai. [filecite\turn33file0\L20-L29](#)

## 24. Same Smart Hospital example se future retrieval

Initially:

```

AI Diagnostic Engine
├── EDGE_104 → Machine Learning Model
└── EDGE_105 → Emergency Risk

```

Suppose users repeatedly questions poochte hain:

*"How does the ML Model affect emergency risk?"*

Aur is path ke answers repeatedly:

HELPFUL

receive karte hain.

Then over time:

```

EDGE_104 weight ↑
EDGE_105 weight ↑

```

Future similar queries me:

```

utility(Machine Learning Model) ↑
utility(Emergency Risk) ↑

```

So Tree Knapsack in nodes ko select karne ki higher preference develop kar sakta hai.

This is the core self-evolution idea.

---

## 25. Lekin ek major danger hai

Suppose:

```
First 50 users
→ same path ko HELPFUL bolte hain
```

Weight continuously increase hota raha:

```
1.0
→ 1.2
→ 1.8
→ 3.5
→ 8.0
```

Then ek path system ko dominate karne lag sakta hai.

Isko kehte hain:

## Weight Drift

Layer 8 is problem ko bhi handle karta hai.

---

## 26. Edge Weight Normalization

Report me periodic normalization function hai.

Goal:

*Global weight values ko drift hone se bachana aur relative ordering preserve karna. [filecite turn34file0 L 13-L 17](#)*

Concept:

```
Current Mean Weight = 1.5
Target Mean Weight = 1.0

scale = 1.0 / 1.5
```

Then every hierarchical edge:

```
new_weight = old_weight × scale
```

subject to:

```
WEIGHT_MIN
WEIGHT_MAX
```

---

## 27. Simple normalization example

Before:



```
EDGE_A = 2.0
EDGE_B = 1.5
EDGE_C = 1.0

Mean = 1.5
```

Target:

```
Mean = 1.0
```

Scale:

```
1.0 / 1.5 = 0.666...
```

After:

```
EDGE_A ≈ 1.33
EDGE_B = 1.00
EDGE_C ≈ 0.67
```

Important:

```
A > B > C
```

ordering preserve hui.

But global scale control me aa gaya.

## 28. Weight Decay

Report me old feedback ko permanently dominant nahi rehne diya jata.

```
edge.weight = edge.weight × decay_rate
```

Default conceptual decay:

```
0.999
```

☐filecite☐turn34file0☐L18-L25☐

Meaning:

*Recent feedback matters more; very old feedback gradually fades.*

## 29. Example: Weight Decay

Suppose:

```
EDGE_104 = 2.0
```

Daily:

```
2.0 × 0.999
```

After one decay:

```
1.998
```

Small change.

But repeated long-term decay:

```
Old historical feedback
gradually loses influence
```

## 30. Step 4 — Monitoring

Ab Layer 8 ka second major responsibility:

### System Health Monitoring

Report monitoring aur evaluation ko alag rakhta hai:

```
Monitoring
=
Real-time system health

Evaluation
=
Periodic quality assessment
```

filecite\turn33file0\L59-L64

## 31. Monitoring ka rolling window

Default:

```
evaluation_window = 1h
```

Meaning:

*Last 1 hour ke logs analyze karo.*

Report configuration me defined thresholds:

```
Metric	Threshold
avg_confidence	0.4
feedback_ratio	0.6
orphan_rate	0.1
latency_p95_ms	5000 ms
error_rate	0.05
```

filecite\turn33file0\L45-L57

## 32. Metric 1 — Average Confidence

Formula conceptually:

$$\frac{\text{Average Confidence}}{\text{Total Queries}}$$

Report code recent logs se ye calculate karta hai. `turn33file0\L73-L77`

Example:

```
Query 1 = 0.90
Query 2 = 0.80
Query 3 = 0.30
```

Average:

```
0.666...
```

Threshold:

```
0.4
```

So:

```
0.66 > 0.4
→ No confidence alert
```

### 33. Metric 2 — Feedback Ratio

Formula:

$$\frac{\text{Helpful Feedback}}{\text{Queries With Feedback}}$$

Notice:

*Total queries denominator nahi hai.*

Only those queries considered jinke paas actual feedback hai. `turn33file0\L79-L84`

#### Example

```
Total queries = 100

Only 40 users gave feedback

HELPFUL = 28
NOT_HELPFUL = 12
```

Then:

$$\text{feedback ratio} = 28/40 = 0.70$$

Not:

\[  
28/100  
\]

This is correct according to the report's shown formula.

---

### 34. Metric 3 — Feedback Coverage

Queries with feedback  
/  
Total queries

Same example:

40 / 100 = 0.40

Meaning:

| 40% queries par user feedback available hai.

Feedback ratio aur feedback coverage **different metrics** hain.

---

### 35. Metric 4 — Average Claim Support Ratio

Formula:

\[  
\frac{SupportedClaims}{TotalClaims}  
\]

Har query ke liye calculate karke average liya jata hai.

Example:

Query A:  
4/4 supported = 1.0  
  
Query B:  
3/5 supported = 0.6  
  
Average:  
0.8

Report ise generation metric ke roop me track karta hai. `filecite\turn33file0\L86-L89`

---

### 36. Metric 5 — Forest Health

Layer 8 sirf queries nahi monitor karta.

Ye Knowledge Forest ki condition bhi check karta hai:

```
total_nodes
total_edges
orphan_rate
avg_edge_weight
flagged_edges
```

filecite\turn33file0\L91-L96

### 37. Orphan Rate kya hai?

Report me orphan node:

*Node jiska no parent and no children hai.* filecite\turn33file0\L91-L95

Formula:

$$\frac{\text{orphan\_nodes}}{\text{total\_nodes}}$$

#### Example

```
Total Nodes = 1000
Orphan Nodes = 120
```

Then:

$$\frac{120}{1000} = 0.12$$

Threshold:

0.10

Therefore:

0.12 > 0.10  
⚠ Alert

### 38. Metric 6 — Average Edge Weight

```
avg_edge_weight
```

monitoring useful hai because self-evolution continuously weights update kar raha hai.

Agar:

Average = 1.0

Normal baseline.

Agar suddenly:

```
Average = 8.5
```

Possible drift problem.

Normalization and decay isi long-term stability ke liye useful hain.

### 39. Metric 7 — Flagged Edges

```
flagged_edges =
len(forest.restructure_queue)
```

Report monitoring me ye track hota hai. `filecite\turn33file0\L91-L96`

Meaning:

*Forest ke kuch edges/structures future restructuring ke liye flag hue hain.*

Exact restructuring mechanism ka complete implementation **Layer 8 shown snippet me specified nahi hai.**

### 40. Metric 8 — Error Rate

Shown monitoring code:

```
overall_confidence == 0
```

wale logs ko error rate ke liye count karta hai.

```
\[
error\ rate
=
\frac{queries\ with\ confidence=0}{total\ queries}
\]
```

`filecite\turn33file0\L98-L100`

### 41. Yahan ek critical problem hai

Report code:

```
overall_confidence == 0
```

ko error count karta hai.

But confidence 0 hone ka matlab har baar:

```
Technical system error
```

nahi hota.

Layer 7 me valid insufficient-evidence case me bhi:

```
confidence = 0
```

ho sakta hai.

To:

No evidence

ko:

System Error

count kiya ja sakta hai.

### Critical observation

*Shown implementation me **error\_rate** true technical errors aur valid "insufficient evidence" responses ko clearly distinguish nahi karta.*

Ye project implementation me improve karna chahiye.

### Suggestion (Not from the report)

Separate fields:

```
status = SUCCESS
status = INSUFFICIENT_EVIDENCE
status = LLM_ERROR
status = PARSING_ERROR
status = TIMEOUT
```

Then:

technical\_error\_rate

sirf actual errors se calculate karo.

## 42. Alerts kaise trigger hote hain?

Layer 8:

```
metrics
 ↓
threshold comparison
 ↓
alerts[]
```

For:

orphan\_rate  
error\_rate

alert if:

metric > threshold

For other configured metrics:

metric < threshold

shown code ke according. `filecite\turn33file0\L102-L114`

### 43. Ek important inconsistency — `latency_p95_ms`

Configuration me threshold hai:

```
latency_p95_ms = 5000
```

But shown `compute_monitoring_metrics()` code metrics dictionary me:

```
avg_latency_ms
```

calculate karta hai.

```
latency_p95_ms
```

explicitly calculate hota hua shown snippet me nahi hai. `filecite\turn33file0\L47-L56` `filecite\turn33file0\L73-L100`

Therefore:

*Alert configuration me P95 latency threshold hai, but shown metric calculation me corresponding `latency_p95_ms` key explicitly present nahi hai.*

Isliye shown code ke basis par:

```
P95 latency alert
```

effectively trigger nahi hoga.

Ye **clear implementation mismatch** hai.

### 44. Step 5 — Feedback Attribution Isolation

Ye Layer 8 ka bahut important design principle hai.

Suppose user says:

 NOT\_HELPFUL

Question:

*Kya problem retrieval me thi?*

Maybe.

But maybe:

```
Retrieval was perfect
+
LLM answer poorly written
```

Or:

```
Answer correct tha
+
User wanted shorter answer
```



Isliye user feedback ko directly:

```
Retrieval = bad
```

assume karna incorrect hoga.

Report explicitly is problem ko identify karta hai. `filecite\turn33file0\L13-L18`

---

## 45. Is problem ko project kaise handle karta hai?

Three protections:

### 1. Small learning rate

```
 $\eta = 0.02$
```

Large update nahi.

---

### 2. Claim metrics separately tracked

```
supported_claims
unsupported_claims
```

Layer 7 verification quality alag monitor hoti hai.

---

### 3. Confidence-feedback mismatch warnings

Case A:

```
overall_confidence < 0.5
+
user says HELPFUL
```

System warning:

*Possible false positive.*

Case B:

```
overall_confidence > 0.8
+
user says NOT_HELPFUL
```

System warning:

*Possible answer-format issue, not necessarily retrieval issue.*

Report me ye rules explicitly listed hain. `filecite\turn33file0\L13-L18`

---

## 46. Hamare example me mismatch case

Suppose:

```
Confidence = 0.92
```

Claims:

4/4 SUPPORTED

Retrieved context good.

But user says:

👉 NOT\_HELPFUL

Layer 8 blindly nahi kahega:

"All retrieval edges are wrong."

Instead:

⚠ Warning:  
High confidence + negative feedback  
Possible answer format / user expectation issue

Phir bhi report ke shown logic ke according feedback self-evolution path update trigger karta hai; small  $\eta$  is impact ko dampen karta hai. `filecite\turn32file0\L119-L130` `filecite\turn33file0\L13-L18`

47. Ab complete end-to-end example

Query

How does the AI Diagnostic Engine analyze patient symptoms and medical history to estimate emergency risk, and what role does the Machine Learning Model play?

Layer 6

Seed Nodes:  
NODE\_001  
NODE\_002  
  
Traversal:  
EDGE\_101  
EDGE\_102  
EDGE\_103  
EDGE\_104  
EDGE\_105  
  
Selected Nodes:  
6

Layer 7

```
Answer Generated
↓
Claims = 4
Supported = 4
Unsupported = 0
Confidence = 0.89
Latency = 1850 ms
```

## Layer 8 — Logging

```
EVT_000001 saved
```

Database:

```
QueryLog
├─ Query
├─ Intent
├─ Complexity
├─ Seed Nodes
├─ Selected Count
├─ Traversal Edges
├─ Claims
├─ Confidence
├─ Model
├─ Latency
└─ Feedback = None
```

## User Feedback

```
👍 HELPFUL
```

## Layer 8 — Feedback Attribution

```
EVT_000001
↓
Fetch original log
↓
Get traversal edges
↓
[EDGE_101...EDGE_105]
↓
Update only these edges
```

## Forest update

```
EDGE_101 weight ↑ slightly
EDGE_102 weight ↑ slightly
EDGE_103 weight ↑ slightly
EDGE_104 weight ↑ slightly
EDGE_105 weight ↑ slightly
```

## Future query

Similar query:

*"What role does the Machine Learning Model play in estimating emergency risk?"*

Layer 6 utility:

```
Node Confidence
x
Updated Edge Weight
x
Evidence Richness
x
Cost Efficiency
```

Relevant path ki utility increase ho sakti hai.

Therefore:

```
Higher chance
of selecting useful nodes
```

This closes the learning loop.

## 48. Layer 8 exception handling

### Exception 1 — Event ID not found

Feedback request:

```
event_id = EVT_999999
```

But database me event nahi.

Shown code:

```
if not log:
 raise ValueError(
 f"Event {event_id} not found"
)
```

filecite\turn32file0\L109-L113

### Issue

Exception raise ho raha hai.

But API layer is exception ko:

```
HTTP 404?
JSON error?
retry?
```

kaise convert karegi:

*Not specified in the shown Layer 8 code.*

## 49. Exception 2 — No logs available

Monitoring function:

```
logs = db.get_recent_logs(window)
```

Agar:

```
logs = []
```

Then:

```
{'status': 'NO_DATA'}
```

return hota hai. [filecite\turn33file0\L65-L69](#)

Ye proper handling hai.

---

## 50. Exception 3 — No hierarchical edges

Normalization me:

```
all_weights = [...]
```

Agar empty:

```
if not all_weights:
 return
```

So division by zero avoid hota hai. [filecite\turn33file0\L117-L130](#)

---

## 51. Exception 4 — Division by zero protections

Monitoring code multiple places par:

```
max(..., 1)
```

use karta hai.

Examples:

```
supported_claims / max(claim_count, 1)
```

and:

```
orphan_count / max(node_count, 1)
```

This avoids:

```
ZeroDivisionError
```

[filecite\turn33file0\L84-L99](#)

---

## 52. Layer 8 me major missing / weak parts

Ab report ko technically critical way me dekhte hain.

## Weakness 1 — Feedback is binary

Currently:

```
HELPFUL
NOT_HELPFUL
```

Only.

User ka feedback:

```
Answer correct but too long
```

ya:

```
Good retrieval but bad explanation
```

binary system capture nahi karta.

### Suggestion (Not from the report)

Feedback could be:

```
HELPFUL
PARTIALLY_HELPFUL
NOT_HELPFUL

+
optional reason:
RETRIEVAL
ACCURACY
COMPLETENESS
FORMAT
LATENCY
```

## 53. Weakness 2 — User feedback retrieval aur generation dono ka mixture hai

Report khud is issue ko recognize karta hai.

Protection:

```
small η
+
separate claim metrics
+
mismatch warnings
```

But fundamentally:

```
HELPFUL
```

Feedback still **pure retrieval label nahi hai**.

Therefore edge update signal noisy ho sakta hai.

## 54. Weakness 3 — Only exploitation risk

Agar repeatedly same successful paths reward hon:

```

Popular path
↓
Weight ↑
↓
More retrieval
↓
More feedback
↓
Weight ↑ again

```

Then system alternate paths explore kam kar sakta hai.

Report ke future work me explicitly:

```

ε-greedy
or
Thompson sampling

```

add karne ka suggestion hai, exploitation-only bias avoid karne ke liye. [filecite\turn31file3\L251-L259](#)

## 55. Weakness 4 — P95 latency mismatch

As discussed:

```

Config:
latency_p95_ms

```

but shown metrics:

```

avg_latency_ms

```

This is an implementation inconsistency. [filecite\turn33file0\L47-L57](#) [filecite\turn33file0\L73-L100](#)

## 56. Weakness 5 — Error rate definition too broad

```

confidence == 0

```

can mean:

```

No evidence

```

not necessarily:

```

Technical failure

```

So monitoring error rate may be misleading.

## 57. Weakness 6 — Feedback validation not explicitly shown

Function accepts:

```

feedback: str

```

But shown code me explicit:

```
if feedback not in ["HELPFUL", "NOT_HELPFUL"]:
```

validation visible nahi hai.

Therefore malformed feedback like:

```
"GOOD"
"YES"
"123"
```

ka exact handling:

*Not specified in the shown Layer 8 implementation.*

---

## 58. Weakness 7 — Duplicate feedback handling

Suppose same event par:

```
First:
HELPFUL

Then:
NOT_HELPFUL
```

What happens?

Shown code feedback field overwrite karta hua appear karta hai.

But:

```
multiple feedback submissions
idempotency
feedback history
```

ka explicit policy shown snippet me specified nahi hai.

---

## 59. Weakness 8 — Concurrent edge updates

Suppose 1,000 users simultaneously feedback bhej rahe hain.

Same edge:

```
EDGE_104
```

simultaneously update ho raha hai.

Potential issue:

```
Race Condition
```

Transaction locking / atomic update / concurrency strategy:

*Not specified in the shown Layer 8 code.*

---

## 60. Weakness 9 — Weight poisoning

Imagine malicious users repeatedly:



## HELPFUL

click karte hain ek poor retrieval path ke liye.

Weights artificially manipulate ho sakte hain.

User identity trust, rate limiting, anomaly detection:

*Not specified in the shown Layer 8 implementation.*

## 61. Layer 8 ka exact output

Layer 8 direct ek normal answer object nahi banata.

Its output:

1. Updated Forest Weights
2. Monitoring Metrics
3. Alerts

Example:

Updated Forest:

```
EDGE_101 = 1.020
EDGE_102 = 1.020
EDGE_103 = 1.020
EDGE_104 = 1.020
EDGE_105 = 1.020
```

Monitoring:

```
avg_confidence = 0.82
feedback_ratio = 0.74
feedback_coverage = 0.45
avg_claim_support_ratio = 0.91
orphan_rate = 0.03
error_rate = 0.02
```

Alerts:

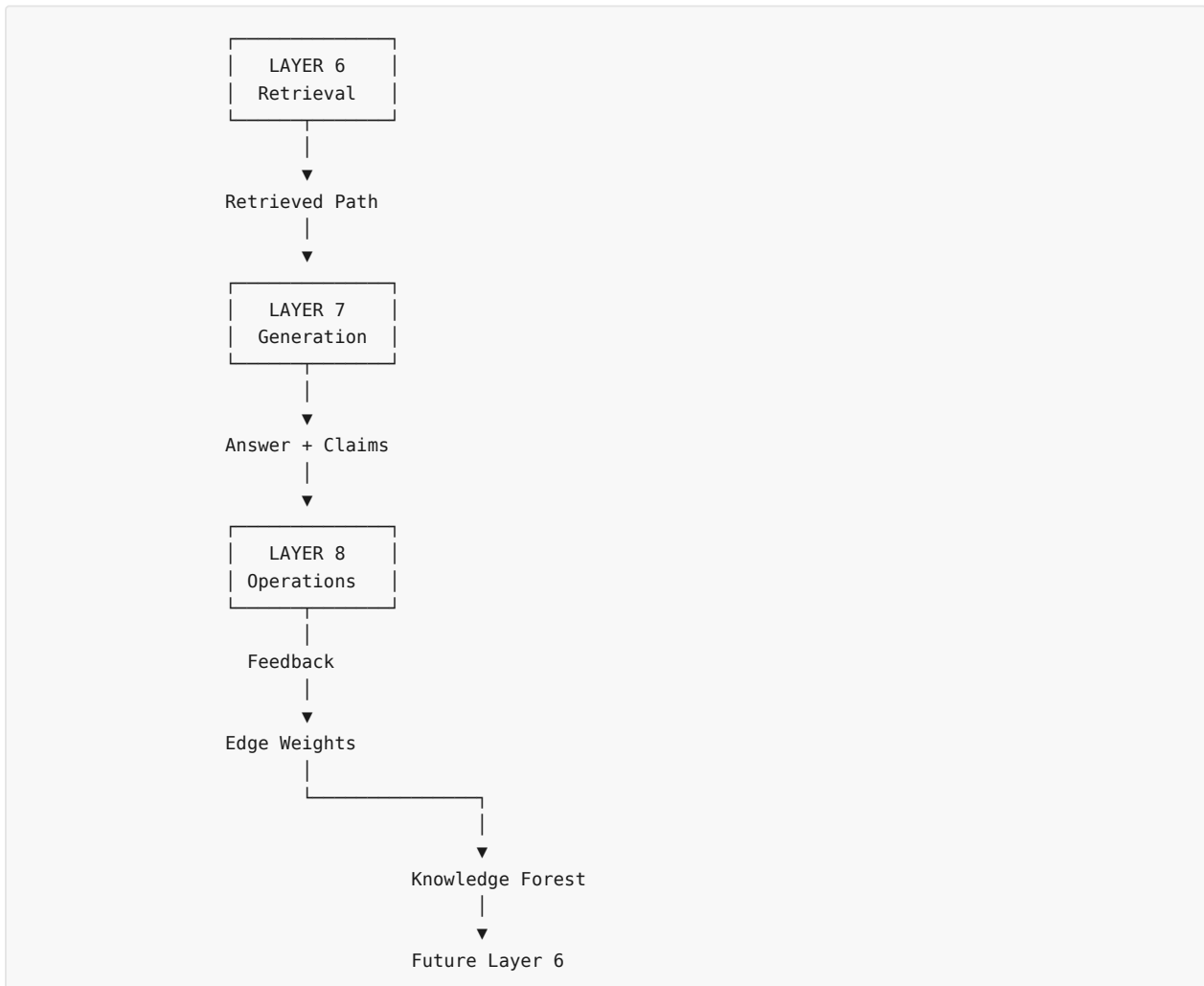
```
[]
```

## 62. Layer 8 ko ek table me summarize karo

| Step | Input           | Layer 8 kya karta hai              | Output          |
|------|-----------------|------------------------------------|-----------------|
| 1    | QueryEvent      | Complete event log karta hai       | QueryLog        |
| 2    | QueryLog        | User feedback ka wait              | Feedback        |
| 3    | Feedback + Path | Traversed edges identify karta hai | Attributed path |
| 4    | Attributed path | Edge weights update                | Updated Forest  |
| 5    | Recent logs     | Health metrics calculate           | Metrics         |
| 6    | Metrics         | Threshold check                    | Alerts          |
| 7    | Time/queries    | Normalize + decay                  | Stable weights  |

### 63. Layer 7 → Layer 8 → Layer 6 connection

Ye project ka **closed learning loop** hai:



Yahi Layer 8 ka main contribution hai.

### 64. Sabse important mental model

Layer 7 poochta hai:

*"Kya retrieved evidence se answer supported hai?"*

Layer 8 poochta hai:

*"Past queries me system ne kaisa perform kiya, user ko answer useful laga ya nahi, aur future retrieval ko kaise adapt karna chahiye?"*

### 65. Layer 8 ko ek sentence me yaad rakho

*Layer 8 SHIA-RAG ka continuous operations layer hai jo har query ka complete trace log karta hai, user feedback ko exact retrieval path se attribute karta hai, edge weights ko gradually update karta hai, future retrieval ko influence karta hai, aur system health ko continuously monitor karta hai. [filecite\turn32file0\L31-L36](#)*

*[filecite\turn33file0\L20-L42](#)*

Layer 0–8 complete flow

```
Layer 0 → Data Model
Layer 1 → Input Processing
Layer 2 → Document Parsing
Layer 3 → Knowledge Extraction
Layer 4 → Relation Management
Layer 5 → SHIA Core / Knowledge Forest
Layer 6 → Retrieval
Layer 7 → Generation + Verification
Layer 8 → Logging + Feedback + Self-Evolution + Monitoring
```

Iske saath SHIA-RAG ke saare 9 layers complete ho gaye. `filecite\turn31file0\L59-L71`

YOU 20/08/2026, 16:10:15

humne pura project dekha uske sbhi 9 layer ko smjha aur unme kya kamiya ,problem ,error , mistake hai wo bhi dekhi mujhe ab unsbhi mistakes , error problem chahiye jo sbhi layer me exist krti hai

CHATGPT 20/08/2026, 16:10:21

`{"paths":["files"],"query":"search"}`

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite\turn35file0`

- [L1] metadata:
- [L2] query\_result\_index: 0
- [L3] file\_id: file\_0000000012688211b4f57f5b8bcb8b42
- [L4] version\_id: 1
- [L5] name: SHIA-RAG\_Final\_Report(4).md
- [L6] mime\_type: text/markdown
- [L7] surface: conversation
- [L8] score: 0.032266458495966696
- [L9] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--6
- [L10] snippet:
- [L11]

```
[L12] graph TD
[L13] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L14] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Par
[L15] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L16] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L17] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L18] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L19] end
[L20]
[L21] subgraph "Phase 2: Online – Retrieval & Generation"
[L22] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L23] KF --> L6
[L24] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L25] L7 --> A["Answer + Citations"]
[L26] end
[L27]
[L28] subgraph "Phase 3: Continuous – Operations"
[L29] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L30] A --> L8
[L31] L8 -->|"Self-Evolution
Feedback"| KF
[L32] end
[L33]
```

[L34]

[L35] ### 3.2 Data Flow

[L36]

[L37]

```
[L38] Raw Document (PDF/DOCX/HTML)
[L39] ↓ Layer 1: Validate, detect format, extract text, OCR if needed
[L40] Normalized Text + Metadata
[L41] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order
[L42] [Page → Region → Block] hierarchy
[L43] ↓ Layer 3: Split sentences, extract concepts/definitions/relations
[L44] KnowledgeUnit objects (candidate concepts)
[L45] ↓ Layer 3: Alias resolution (merge synonyms via MinHash LSH)
[L46] KnowledgeNode objects (canonical, deduplicated)
[L47] ↓ Layer 4: Validate relations against ontology, score confidence
[L48] KnowledgeEdge objects (typed, scored)
[L49] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)
[L50] Knowledge Forest (rooted trees + cross-links)
[L51] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection
[L52] Selected context nodes (within token budget)
[L53] ↓ Layer 7: Build prompt, invoke LLM
[L54] ↓ Layer 7: Verify claims against evidence, bind citations
[L55] Answer with citations + confidence score
[L56] ↓ Layer 8: Log, collect feedback, update forest weights
[L57]
```

[L58]

[L59] ### 3.3 Layer Summary

[L60]

[L61] | Layer | Name | Input | Output | Key Algorithms |

[L62] |-----|-----|-----|-----|-----|

[L63] | 0 | Data Model | — | Schemas | KnowledgeNode, KnowledgeEdge |

[L64] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |

[L65] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |

[L66] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |  
[L67] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |  
[L68] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |  
[L69] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |  
[L70] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |  
[L71] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |  
[L72]  
[L73] ---  
[L74]  
[L75] ## 4. Data Model — Layer 0

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite[turn35file1]`

[L76] metadata:  
[L77] query\_result\_index: 1  
[L78] file\_id: file\_0000000012688211b4f57f5b8bcb8b42  
[L79] version\_id: 1  
[L80] name: SHIA\_RAG\_Final\_Report(4).md  
[L81] mime\_type: text/markdown  
[L82] surface: conversation  
[L83] score: 0.03149801587301587  
[L84] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--5  
[L85] snippet:  
[L86] **Gap:** No existing system combines (1) concept-level knowledge extraction, (2) automated hierarchy induction, (3) hierarchy-aware context selection, and (4) self-evolution via retrieval feedback.  
[L87]  
[L88] ### 2.3 Literature Search Methodology  
[L89]  
[L90] | Parameter | Details |  
[L91] |-----|-----|  
[L92] | **Databases Searched** | ACL Anthology, arXiv (cs.CL, cs.IR, cs.AI), Google Scholar, Semantic Scholar |  
[L93] | **Conferences/Venues** | ACL, EMNLP, NAACL, SIGIR, CIKM, WWW, NeurIPS, ICLR, KDD |  
[L94] | **Search Keywords** | "hierarchical RAG", "tree RAG", "graph RAG", "knowledge graph RAG", "structured retrieval augmented generation", "ontology-guided retrieval", "tree knapsack context selection", "adaptive depth retrieval" |  
[L95] | **Time Range** | 2020–2026 |  
[L96] | **Key Papers Reviewed** | RAPTOR (Sarathi et al., 2024), GraphRAG (Edge et al., 2024), TreeRAG (concept), HiChunk (LlamaIndex), REALM, RETRO, Atlas, Self-RAG |  
[L97]  
[L98] ---  
[L99]  
[L100] ## 3. System Architecture  
[L101]  
[L102] ### 3.1 Overview  
[L103]  
[L104] SHIA-RAG is organized as an **8-layer pipeline** divided into three phases:

[L105]

[L106] | Phase | Layers | Mode | Purpose |

[L107] |-----|-----|-----|-----|

[L108] | **Phase 1: Offline** | Layers 1-5 | Batch processing | Extract knowledge from documents and build the Knowledge Forest |

[L109] | **Phase 2: Online** | Layers 6-7 | Real-time | Retrieve context and generate answers for user queries |

[L110] | **Phase 3: Continuous** | Layer 8 | Background | Monitor, collect feedback, and evolve the Knowledge Forest |

[L111]

[L112] Additionally, **Layer 0** is the foundational data model (schemas, types, database contracts) shared by all layers.

[L113]

[L114]

```
[L115] graph TD
[L116] subgraph "Phase 1: Offline – Knowledge Extraction & Indexing"
[L117] L1["Layer 1: Input Processing
File upload, validation, OCR"] --> L2["Layer 2: Document Pa
[L118] L2 --> L3["Layer 3: Knowledge Extraction
Concept detection, alias resolution"]
[L119] L3 --> L4["Layer 4: Relation Management
Triple extraction, confidence scoring"]
[L120] L4 --> L5["Layer 5: SHIA Core
CPG++ → PRE → HV → FI → CLD"]
[L121] L5 --> KF["Knowledge Forest Store
(Graph DB + Vector DB)"]
[L122] end
[L123]
[L124] subgraph "Phase 2: Online – Retrieval & Generation"
[L125] Q["User Query"] --> L6["Layer 6: Retrieval Engine
Query analysis, forest traversal, RB0"]
[L126] KF --> L6
[L127] L6 --> L7["Layer 7: Generation Engine
Prompt assembly, LLM, verification"]
[L128] L7 --> A["Answer + Citations"]
[L129] end
[L130]
[L131] subgraph "Phase 3: Continuous – Operations"
[L132] L8["Layer 8: Operations
Logging, monitoring, feedback loop"]
[L133] A --> L8
[L134] L8 -->|"Self-Evolution
Feedback"| KF
[L135] end
[L136]
```

[L137]

[L138] ### 3.2 Data Flow

[L139]

[L140] ```

[L141] Raw Document (PDF/DOCX/HTML)

[L142] ↓ Layer 1: Validate, detect format, extract text, OCR if needed

[L143] Normalized Text + Metadata

[L144] ↓ Layer 2: Detect layout, segment into blocks, classify types, reading order

[L145] [Page → Region → Block] hierarchy

[L146] ↓ Layer 3: Split sentences,

[L147] metadata:

[L148] query\_result\_index: 2

[L149] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L150] version\_id: 1

[L151] name: SHIA\_RAG\_Final\_Report(4).md

[L152] mime\_type: text/markdown

[L153] surface: conversation

[L154] score: 0.03128054740957967

[L155] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--54

[L156] snippet:

[L157] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L158] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L159] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L160] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L161] 5. **SHEF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L162]

[L163] The complete system specification covers:

[L164] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L165] - **12 defined functions** with complete implementations

[L166] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L167] - **8 schemas** covering all data objects from document blocks to query logs

[L168] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L169] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L170]

[L171] ### 13.2 Key Novelties

[L172]

[L173] | # | Novelty | Level | Why It's New |

[L174] |---|-----|-----|-----|

[L175] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L176] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L177] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L178] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L179] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L180]

[L181] ### 13.3 Next Steps

[L182]

[L183] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L184] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L185] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

- [L186] 4. **Build Layer 3** (Knowledge extraction with LLM)
- [L187] 5. **Build Layer 4** (Relation management with ontology validation)
- [L188] 6. **Build Layer 5** (SHIA Core with algorithms)
- [L189] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)
- [L190] 8. **Build Layer 7** (Generation with verification)
- [L191] 9. **Build Layer 8** (Operations, monitoring, self-evolution)
- [L192] 10. **Run first evaluation** on OS textbook dataset using SHEF
- [L193] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)
- [L194] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027
- [L195]
- [L196] ### 13.4 Future Work

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite[turn35file3]`

- [L197] metadata:
- [L198] query\_result\_index: 3
- [L199] file\_id: file\_0000000012688211b4f57f5b8bcb8b42
- [L200] version\_id: 1
- [L201] name: SHIA\_RAG\_Final\_Report(4).md
- [L202] mime\_type: text/markdown
- [L203] surface: conversation
- [L204] score: 0.029827662395050816
- [L205] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--0
- [L206] snippet:
- [L207] # SHIA-RAG: Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation
- [L208]
- [L209] > **Project Type:** Minor Project (M.Tech)
- [L210] > **Date:** August 2026
- [L211] > **Author:** [Your Name]
- [L212]
- [L213] ---
- [L214]
- [L215] ## Abstract
- [L216]
- [L217] Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge. However, current RAG systems universally rely on flat text chunking — splitting documents into fixed-size segments and retrieving the top-k most similar chunks. This approach destroys semantic continuity, produces fragmented context, and fails at multi-hop reasoning.
- [L218]
- [L219] We present **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation), a novel 8-layer RAG system that replaces flat chunks with a dynamically constructed **Knowledge Forest** — a collection of rooted trees where each node represents a concept (with definitions, aliases, properties, and evidence) and edges encode hierarchical (IS\_A, PART\_OF) and semantic (USES, CAUSES) relationships.
- [L220]
- [L221] SHIA-RAG introduces five novel contributions: (1) **Precedence-Constrained Tree Knapsack** for context



selection under token budgets, ensuring hierarchical coherence; (2) **Self-Evolving Knowledge Forest** with a closed-loop feedback mechanism that modifies edge weights based on retrieval success/failure; (3) **Multi-Objective Energy Minimization** for automated hierarchy induction; (4) **Adaptive Depth Traversal** that adjusts retrieval depth based on query complexity; and (5) **SHEF** (SHIA-RAG Evaluation Framework), to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality.

[L222]

[L223] The system processes raw documents through an offline pipeline (input → parsing → extraction → relation management → hierarchy construction) and serves queries through an online pipeline (retrieval → generation with claim verification). A continuous operations layer closes the feedback loop by updating the Knowledge Forest based on user interactions.

[L224]

[L225] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L226]

[L227] ---

[L228]

[L229] ## Table of Contents

[L230]

[L231] 1. [Introduction](#)

[L232] 2. [Literature Review](#)

[L233] 3. [System Architecture](#)

[L234] 4. [Data Model — Layer 0](#)

[L235] 5. [Offline Pipeline — Layers 1-5](#)

[L236] 6. [Online Pipeline — Layers 6-7](#)

[L237] 7. [Operations Layer — Layer 8](#)

[L238] 8. [Novel Contributions](#)

[L239] 9. [Mathematical Framework](#)

[L240] 10. [Implementation](#)

[L241] 11. [Evaluation Framework](#)

[L242] 12. [Risk Analysis](#)

[L243] 13. [Conclusion & Future Work](#)

[L244]

[L245] ---

[L246]

[L247] ## 1. Introduction

[L248]

[L249] ### 1.1 Problem Statement

[L250]

[L251] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L252]

[L253] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L254] 2. Embed each chunk into a vector space

[L255] 3. At query time, retrieve the top-k most similar chunks

[L256] 4. Feed these chunks as context to an LLM for answer generation

[L257]

[L258] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `[]filecite[]turn35file4[]`

[L259] metadata:

[L260] query\_result\_index: 4

[L261] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L262] version\_id: 1

[L263] name: SHIA\_RAG\_Final\_Report(4).md

[L264] mime\_type: text/markdown

[L265] surface: conversation

[L266] score: 0.029551337359792925

[L267] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--35

[L268] snippet:

[L269] ##### Prompt Template

[L270]

[L271]

```
[L272] SYSTEM_PROMPT = """"You are a knowledge-grounded question answering system.
[L273] RULES:
[L274] 1. Answer ONLY from the provided evidence. Do NOT use prior knowledge.
[L275] 2. For each claim, cite the source using [NODE_ID] format.
[L276] 3. If evidence is insufficient, say "Insufficient evidence to answer."
[L277] 4. If evidence conflicts, present both views with citations.
[L278]
[L279] REQUIRED OUTPUT FORMAT:
[L280] {
[L281] "answer": "Your answer with [NODE_ID] citations inline.",
[L282] "claims": [{"text": "Factual claim", "evidence_ids": ["NODE_ID1"], "confidence": 0.0-1.0}],
[L283] "overall_confidence": 0.0-1.0,
[L284] "insufficient_evidence": false
[L285] }""""
[L286]
```

[L287]

[L288] ##### Claim Verification

[L289]

[L290] After the LLM generates an answer, each claim is verified against its cited evidence:

[L291]

[L292]

```

[L293] def verify_claim(claim_text, evidence_ids, node_map) -> tuple:
[L294] """Returns (status, confidence).
[L295] - SUPPORTED: evidence entails claim (confidence > 0.7)
[L296] - PARTIAL: evidence partially supports (0.4 < confidence ≤ 0.7)
[L297] - UNSUPPORTED: no evidence supports (confidence ≤ 0.4)
[L298] - CONFLICTING: evidence items contradict each other
[L299] """
[L300] scores = []
[L301] for eid in evidence_ids:
[L302] node = node_map.get(eid)
[L303] if node and node.evidence:
[L304] evidence_text = ' '.join(e.text for e in node.evidence)
[L305] scores.append(cosine_similarity(embed(claim_text), embed(evidence_text)))
[L306]
[L307] if not scores:
[L308] return 'UNSUPPORTED', 0.0
[L309]
[L310] avg_score = sum(scores) / len(scores)
[L311]
[L312] if len(scores) >= 2 and max(scores) - min(scores) > 0.4:
[L313] return 'CONFLICTING', avg_score
[L314] if avg_score > 0.7:
[L315] return 'SUPPORTED', avg_score
[L316] elif avg_score > 0.4:
[L317] return 'PARTIAL', avg_score
[L318] else:
[L319] return 'UNSUPPORTED', avg_score
[L320]

```

[L321]

[L322] ##### Unsupported Claim Policy

[L323]

[L324] | Status | Action |

[L325] |-----|-----|

[L326] | SUPPORTED | Keep as-is with citation |

[L327] | PARTIAL | Keep but mark as partially supported |

[L328] | UNSUPPORTED | Remove from answer, note at end |

[L329] | CONFLICTING | Present both views with note |

[L330]

[L331] ##### Answer Confidence Formula

[L332]

[L333] 
$$\text{confidence} = 0.4 \cdot \frac{\text{supported claims}}{\text{total claims}} + 0.3 \cdot \frac{\text{claim\_conf}}{\text{claim\_conf}} + 0.2 \cdot \frac{\text{node\_conf}}{\text{node\_conf}} + 0.1 \cdot \frac{\text{cited nodes}}{\text{selected nodes}}$$

[L334]

[L335] ##### Output Schema

[L336]

[L337]

```
[L338] @dataclass
[L339] class GenerationResult:
[L340] """Output of Layer 7 – ready for user and Layer 8."""
[L341] answer: str # Answer text with citations
[L342] claims: list # List of VerifiedClaim
[L343] citations: list # Formatted citation list
[L344] overall_confidence: float # 0.0-1.0
[L345] retrieval_result: object # Preserved for Layer 8
[L346] generation_metadata: dict # Model used, claim counts, etc.
[L347]
```

[L348]

[L349] ---

[L350]

[L351] ## 7. Operations Layer — Layer 8

[L352]

[L353] **Responsibility:** Log all query events, collect user feedback, update edge weights (self-evolution), monitor system health, and evaluate performance.

[L354]

[L355] ### 7.1 Logging

[L356]

[L357] Every query processed through the system generates a structured `QueryLog` entry:

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite`turn35file5

[L358] metadata:

[L359] query\_result\_index: 5

[L360] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L361] version\_id: 1

[L362] name: SHIA\_RAG\_Final\_Report(4).md

[L363] mime\_type: text/markdown

[L364] surface: conversation

[L365] score: 0.029273504273504274

[L366] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--53

[L367] snippet:

[L368] Real knowledge often has multiple valid parents (e.g., "Machine Learning" → both "AI" and "Statistics") | Deliberate design choice | Limits expressiveness | Future work: DAG support (Section 13.4) |

[L369] | L2 | **Greedy Tree Knapsack** — uses greedy approximation instead of exact DP. Exact DP is  $O(nB)$  where  $B$  = token budget, which is computationally prohibitive | Deliberate trade-off | Near-optimal, not optimal | Future work: Exact DP for small budgets |

[L370] | L3 | **No incremental updates** — adding new documents requires re-running the full offline pipeline (Layers 1-5) | Engineering gap | Limits production scalability | Future work: Incremental forest updates |

[L371] | L4 | **Abstraction-level heuristic** — the multi-factor heuristic for estimating abstraction level has not been empirically validated | Validation needed | May produce incorrect hierarchy | Planned: Empirical validation on domain datasets |

[L372] | L5 | **LLM extraction quality** — knowledge extraction relies on LLM accuracy. Extraction errors propagate through the entire pipeline | Inherent limitation | Garbage-in-garbage-out risk | Mitigated by confidence thresholds

and human review |

[L373] | L6 | **Self-evolution bias** — positive feedback on wrong paths can reinforce errors.  $\epsilon$ -greedy exploration mitigates but does not eliminate this risk | Research challenge | May degrade over time | Mitigated by  $\epsilon$ -greedy; future: Thompson sampling |

[L374] | L7 | **Novelty claims require verification** — "first" claims are based on systematic literature review but require exhaustive verification before publication | Research maturity | Affects publication readiness | All claims qualified with "to the best of our knowledge" |

[L375]

[L376] ---

[L377]

[L378] ## 13. Conclusion & Future Work

[L379]

[L380] ### 13.1 Summary

[L381]

[L382] This report presents **SHIA-RAG**, a novel 8-layer Retrieval-Augmented Generation system that replaces flat text chunks with a dynamically constructed Knowledge Forest. The system introduces five novel contributions:

[L383]

[L384] 1. **Precedence-Constrained Tree Knapsack** — to the best of our knowledge, the first formulation of RAG context selection as a precedence-constrained optimization problem

[L385] 2. **Self-Evolving Knowledge Forest** — to the best of our knowledge, the first closed-loop feedback mechanism that modifies RAG knowledge structures based on retrieval success/failure

[L386] 3. **Multi-Objective Energy Minimization** — a formal optimization framework for automated hierarchy induction combining semantic similarity, depth, confidence, and type coherence

[L387] 4. **Adaptive Depth Traversal** — query-complexity-aware retrieval depth adaptation for hierarchical RAG

[L388] 5. **SHF Evaluation Framework** — to the best of our knowledge, the first benchmark specifically designed for hierarchical RAG quality

[L389]

[L390] The complete system specification covers:

[L391] - **9 layers** (Layer 0-8) with schemas, algorithms, pseudocode, and inter-layer data contracts

[L392] - **12 defined functions** with complete implementations

[L393] - **12 relation types** across 4 hierarchical and 8 semantic categories

[L394] - **8 schemas** covering all data objects from document blocks to query logs

[L395] - A **6-month implementation roadmap** with technology stack, folder structure, and CI/CD pipeline

[L396] - A **comprehensive evaluation plan** with datasets, baselines, ablation studies, and statistical significance testing

[L397]

[L398] ### 13.2 Key Novelties

[L399]

[L400] | # | Novelty | Level | Why It's New |

[L401] |---|-----|-----|-----|

[L402] | 1 | **Tree Knapsack for Context Selection** | Incremental (Novel Application) | To our knowledge, first RAG system to formulate context selection as a precedence-constrained optimization |

[L403] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge,

[L404] metadata:

[L405] query\_result\_index: 6

[L406] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L407] version\_id: 1

[L408] name: SHIA\_RAG\_Final\_Report(4).md

[L409] mime\_type: text/markdown

[L410] surface: conversation

[L411] score: 0.029083245521601686

[L412] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--3

[L413] snippet:

[L414] > **Key Insight:** Humans organize knowledge by topics and subtopics (e.g., "Computer Science → Operating Systems → Scheduling → Round Robin"), not by arbitrary text windows. SHIA-RAG mirrors this natural organization for AI retrieval.

[L415]

[L416] ### 1.3 Objectives

[L417]

[L418] 1. Design a complete 8-layer architecture for hierarchical RAG

[L419] 2. Define algorithms for automated hierarchy induction (CPG++, PRE, HV, FI, CLD)

[L420] 3. Implement a self-evolving feedback loop that improves retrieval quality over time

[L421] 4. Create a context selection algorithm that respects hierarchical constraints (Tree Knapsack)

[L422] 5. Develop an evaluation framework (SHEF) specifically for hierarchical RAG quality

[L423] 6. Provide a fully implementable specification with schemas, pseudocode, and deployment plans

[L424]

[L425] ### 1.4 Contributions

[L426]

[L427] This work makes five novel contributions to the RAG literature:

[L428]

[L429] | # | Contribution | Novelty Level |

[L430] |---|-----|-----|

[L431] | 1 | Precedence-Constrained Tree Knapsack for context selection | Incremental (Novel Application) |

[L432] | 2 | Self-Evolving Knowledge Forest via retrieval feedback | Moderately Novel |

[L433] | 3 | Multi-Objective Energy Minimization for hierarchy induction | Moderately Novel |

[L434] | 4 | Adaptive Depth Traversal via query complexity classification | Incremental |

[L435] | 5 | SHEF — SHIA-RAG Evaluation Framework for hierarchical RAG | Incremental (Novel Combination) |

[L436]

[L437] ---

[L438]

[L439] ## 2. Literature Review

[L440]

[L441] ### 2.1 Related Work

[L442]

[L443] ##### 2.1.1 Flat-Chunk RAG Systems

[L444]

[L445] The foundational RAG approach was introduced by Lewis et al. (2020), combining dense passage retrieval with seq2seq generation. Subsequent systems — **REALM** (Guu et al., 2020), **RETRO** (Borgeaud et al., 2022), **Atlas** (Izacard et al., 2023), and **Self-RAG** (Asai et al., 2024) — improved retrieval quality, generation faithfulness, and self-

reflection, but all operate on flat text chunks with no structural awareness.

[L446]

[L447] ##### 2.1.2 Structured RAG Systems

[L448]

[L449] Recent work has begun exploring structured retrieval:

[L450]

[L451] | System | Approach | Key Limitation |

[L452] |-----|-----|-----|

[L453] | **RAPTOR** (Sarathi et al., 2024) | Recursive abstractive processing — clusters chunks and generates summaries at multiple levels | Top-down summaries only; no concept-level extraction or entity resolution |

[L454] | **GraphRAG** (Edge et al., 2024) | Builds a knowledge graph from text, retrieves subgraphs | No hierarchy; flat graph structure; no self-evolution |

[L455] | **TreeRAG** (Concept) | Organizes documents into topic trees for retrieval | No formal hierarchy induction algorithm; manual tree construction |

[L456] | **HiChunk** (LlamaIndex) | Hierarchical chunking with parent-child relationships between chunks | Chunks are still text fragments, not semantic concepts |

[L457]

[L458] ##### 2.1.3 Knowledge Graph Construction

[L459]

[L460] Ontology learning and taxonomy induction have a rich history in NLP:

[L461] - **Hearst patterns** (Hearst, 1992) extract hypernym relations from text

[L462] - **Taxonomy induction** (Velardi et al., 2013; Bansal et al., 2014) builds hierarchies from web corpora

[L463] - **Knowledge Graph Embedding** (TransE, RotatE) learns vector representations of entities and relations

[L464]

[L465] However, **none of these systems integrate hierarchy construction with RAG retrieval and self-evolution.**

[L466]

[L467] ### 2.2 Research Gap

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: filecite:turn35file7

[L468] metadata:

[L469] query\_result\_index: 7

[L470] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L471] version\_id: 1

[L472] name: SHIA\_RAG\_Final\_Report(4).md

[L473] mime\_type: text/markdown

[L474] surface: conversation

[L475] score: 0.028991596638655463

[L476] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--51

[L477] snippet:

[L478] ### 11.4 Statistical Significance Testing

[L479]

[L480] | Test | Application | Details |

[L481] |-----|-----|-----|

[L482] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |

[L483] | Significance level | All tests |  $\alpha = 0.05$  |

[L484] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |

[L485] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |

[L486] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |

[L487]

[L488] ### 11.5 Error Analysis

[L489]

[L490] | Analysis Type | Method | Details |

[L491] |-----|-----|-----|

[L492] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |

[L493] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |

[L494] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |

[L495] | Per-category breakdown | Stratified metrics | Report accuracy per query type |

[L496]

[L497] ### 11.6 Computational Cost Analysis

[L498]

[L499] | Metric | How Measured | Report Format |

[L500] |-----|-----|-----|

[L501] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L502] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L503] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L504] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L505] | Memory usage | Peak RAM + VRAM | GB |

[L506] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L507]

[L508] ---

[L509]

[L510] ## 12. Risk Analysis

[L511]

[L512] | Risk | Impact | Probability | Mitigation |

[L513] |-----|-----|-----|-----|

[L514] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L515] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L516] |  $O(N^2)$  operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L517] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning, conflict logging |

[L518] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L519] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L520] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L521] | Self-evolution reinforces wrong paths | Medium | Medium |  $\epsilon$ -greedy exploration ( $\epsilon=0.1$ ), periodic weight normalization, anomaly detection |

[L522]



[L523] ---  
[L524]  
[L525] ## 12.5 Limitations & Current Status  
[L526]  
[L527] This section explicitly documents known limitations and the current project status.  
[L528]  
[L529] ### Current Status  
[L530]  
[L531] SHIA-RAG is currently at the **specification stage**. This report provides a complete architectural specification with schemas, algorithms, pseudocode, and implementation contracts. However:  
[L532]  
[L533] - **No implementation exists yet** — the system has not been built or deployed  
[L534] - **No experimental results exist** — all evaluation metrics are planned, not measured  
[L535] - **Cost/latency numbers are projections** — actual measurements require a running system  
[L536]  
[L537] The implementation roadmap (Section 10.3) details the path from specification to working system.  
[L538]  
[L539] ### Known Limitations

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite[turn35file8]`

[L540] metadata:  
[L541] query\_result\_index: 8  
[L542] file\_id: file\_0000000012688211b4f57f5b8bcb8b42  
[L543] version\_id: 1  
[L544] name: SHIA\_RAG\_Final\_Report(4).md  
[L545] mime\_type: text/markdown  
[L546] surface: conversation  
[L547] score: 0.0288981288981289  
[L548] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--32  
[L549] snippet:  
[L550]

```

[L551] @dataclass
[L552] class RetrievalResult:
[L553] """Output of Layer 6 – input to Layer 7."""
[L554] selected_nodes: list # Nodes selected by Tree Knapsack
[L555] context_text: str # Assembled context string
[L556] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L557] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L558] query_intent: str # DEFINITION, COMPARISON, etc.
[L559] seed_node_ids: list # Initial seed nodes from vector search
[L560] token_count: int # Estimated tokens in context
[L561]
[L562] class RetrievalEngine:
[L563] def __init__(self, forest, vector_index, cache_manager, config):
[L564] self.forest = forest
[L565] self.vector_index = vector_index
[L566] self.cache = cache_manager
[L567] self.config = config
[L568]
[L569] def retrieve(self, query) -> RetrievalResult:
[L570] # Step 0: Cache check
[L571] cached = self.cache.get_query_result(hash_query(query))
[L572] if cached:
[L573] return cached
[L574]
[L575] # Step 1: Classify query intent and complexity
[L576] intent = detect_intent(query)
[L577] complexity = classify_complexity(intent)
[L578]
[L579] # Step 2: Embed query and find seed nodes
[L580] query_emb = embed_query(query) # Asymmetric embedding model
[L581] seed_nodes = self.vector_index.search(query_emb, k=5)
[L582]
[L583] # Step 3: Adaptive depth traversal
[L584] depth_config = {
[L585] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},
[L586] 'MODERATE': {'max_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},
[L587] 'COMPLEX': {'max_depth': 3, 'parents': True, 'children': True, 'crosslinks': True},
[L588] }
[L589] traversal = depth_config[complexity]
[L590]
[L591] # Step 4: BFS traversal from all seeds
[L592] candidate_nodes = set()
[L593] for seed in seed_nodes:
[L594] candidate_nodes |= self.traverse(seed, traversal)
[L595]
[L596] # Step 5: Tree Knapsack selection (per-tree, then merge)
[L597] selected = self.multi_tree_knapsack(candidate_nodes, budget=self.config.token_budget)
[L598]
[L599] # Step 6: Assemble context (definitions first, then core, sorted by confidence)
[L600] context_text = self.assemble_context(selected, intent)
[L601]
[L602] return RetrievalResult(
[L603] selected_nodes=selected, context_text=context_text,
[L604] traversal_path_edges=self._get_path_edges(selected),
[L605] query_complexity=complexity, query_intent=intent,
[L606] seed_node_ids=[s.id for s in seed_nodes],
[L607] token_count=estimate_tokens(context_text)
[L608])
[L609]

```

[L610]

[L611] ##### Multi-Tree Knapsack Selection

[L612]

[L613] For multi-seed queries, candidates may span multiple trees. The algorithm:

[L614]

[L615] 1. **Group** candidates by tree root

[L616] 2. **Allocate** token budget proportionally to each tree

[L617] 3. **Run** Tree Knapsack on each tree independently

[L618] 4. **Merge** results and trim to total budget

[L619]

[L620] ```python

[L621] def multi\_tree\_knapsack(self, candidate\_nodes, budget):

[L622] tree\_groups = {}

[L623] for node in candidate\_nodes:

[L624] root = self.forest.find\_root(node)

[L625] tree\_groups.setdefault(root.id, []).append(node)

[L626]

[L627] all\_selected = []

[L628] for root\_id, nodes in tree\_groups.items():

[L629] tree\_budget = int(budget \* len(nodes) / max(len(candidate\_nodes), 1))

[L630] \_, selected = tree\_knapsack(self.forest.get\_node(root\_id), tree\_budget, self.

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: □filecite□turn35file9□

[L631] metadata:

[L632] query\_result\_index: 9

[L633] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L634] version\_id: 1

[L635] name: SHIA\_RAG\_Final\_Report(4).md

[L636] mime\_type: text/markdown

[L637] surface: conversation

[L638] score: 0.028531243721117136

[L639] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--7

[L640] snippet:

[L641] deduplicated)

[L642] ↓ Layer 4: Validate relations against ontology, score confidence

[L643] KnowledgeEdge objects (typed, scored)

[L644] ↓ Layer 5: CPG++ → PRE → HV → FI → CLD (hierarchy construction)

[L645] Knowledge Forest (rooted trees + cross-links)

[L646] ↓ Layer 6: Query → embed → find seeds → traverse → Tree Knapsack selection

[L647] Selected context nodes (within token budget)

[L648] ↓ Layer 7: Build prompt, invoke LLM

[L649] ↓ Layer 7: Verify claims against evidence, bind citations

[L650] Answer with citations + confidence score

[L651] ↓ Layer 8: Log, collect feedback, update forest weights

[L652]

```
[L653]
[L654] ### 3.3 Layer Summary
[L655]
[L656] | Layer | Name | Input | Output | Key Algorithms |
[L657] |-----|-----|-----|-----|-----|
[L658] | 0 | Data Model | - | Schemas | KnowledgeNode, KnowledgeEdge |
[L659] | 1 | Input Processing | Raw files | NormalizedDocument | Format detection, OCR fallback |
[L660] | 2 | Document Parsing | NormalizedDocument | DocumentBlock[] | Block classification, reading order |
[L661] | 3 | Knowledge Extraction | DocumentBlock[] | KnowledgeNode[] | LLM extraction, alias resolution |
[L662] | 4 | Relation Management | KnowledgeNode[] | KnowledgeEdge[] | Ontology validation, confidence scoring |
[L663] | 5 | SHIA Core | Nodes + Edges | Knowledge Forest | CPG++, PRE, HV, FI, CLD |
[L664] | 6 | Retrieval Engine | User query | RetrievalResult | Adaptive depth, Tree Knapsack |
[L665] | 7 | Generation Engine | RetrievalResult | GenerationResult | Prompt assembly, claim verification |
[L666] | 8 | Operations | GenerationResult | Updated Forest | Feedback, self-evolution, monitoring |
[L667]
[L668] ---
[L669]
[L670] ## 4. Data Model – Layer 0
[L671]
[L672] Layer 0 defines the foundational schemas shared by all layers. The two core data structures are `KnowledgeNode` and `KnowledgeEdge`.
[L673]
[L674] ### 4.1 KnowledgeNode Schema
[L675]
[L676] Each concept in the Knowledge Forest is represented as a `KnowledgeNode`:
[L677]
[L678]
```

json

```
[L679] {
[L680] "node_id": "NODE_000789",
[L681] "uuid": "550e8400-e29b-41d4-a716-446655440000",
[L682] "canonical_name": "Round Robin",
[L683] "aliases": ["RR", "Round-Robin Scheduling"],
[L684] "definitions": ["A preemptive CPU scheduling algorithm that assigns a fixed time quantum to each process in circular order"],
[L685] "summaries": ["CPU scheduling method using time slices"],
[L686] "keywords": ["scheduling", "time quantum", "preemptive", "CPU"],
[L687] "domain": "Operating Systems",
[L688] "properties": {
[L689] "time_quantum": {"type": "integer", "value": 20, "unit": "ms"},
[L690] "preemptive": {"type": "boolean", "value": true}
[L691] },
[L692] "examples": ["Process P1 gets 20ms, then P2 gets 20ms, cycling back"],
[L693] "parent": "NODE_000456",
[L694] "children": ["NODE_001001", "NODE_001002"],
[L695] "related": ["NODE_000890"],
[L696] "incoming_edges": ["EDGE_001"],
[L697] "outgoing_edges": ["EDGE_002", "EDGE_003"],
[L698] "evidence": [
[L699] {
[L700] "evidence_type": "DEFINITION",
[L701] "text": "Round Robin is a preemptive scheduling algorithm...",
[L702] "source_doc": "DOC_000001",
[L703] "source_block": "BLK_000234",
```

[L704] "page": 42

[L705] }

[L706] ],

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite[turn35file10]`

[L707] metadata:

[L708] query\_result\_index: 10

[L709] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L710] version\_id: 1

[L711] name: SHIA\_RAG\_Final\_Report(4).md

[L712] mime\_type: text/markdown

[L713] surface: conversation

[L714] score: 0.028298204527712725

[L715] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--1

[L716] snippet:

[L717] **Keywords:** Retrieval-Augmented Generation, Knowledge Graph, Hierarchy Induction, Tree Knapsack, Self-Evolving Knowledge Systems, Natural Language Processing

[L718]

[L719] ---

[L720]

[L721] ## Table of Contents

[L722]

[L723] 1. Introduction

[L724] 2. Literature Review

[L725] 3. System Architecture

[L726] 4. Data Model — Layer 0

[L727] 5. Offline Pipeline — Layers 1-5

[L728] 6. Online Pipeline — Layers 6-7

[L729] 7. Operations Layer — Layer 8

[L730] 8. Novel Contributions

[L731] 9. Mathematical Framework

[L732] 10. Implementation

[L733] 11. Evaluation Framework

[L734] 12. Risk Analysis

[L735] 13. Conclusion & Future Work

[L736]

[L737] ---

[L738]

[L739] ## 1. Introduction

[L740]

[L741] ### 1.1 Problem Statement

[L742]

[L743] Retrieval-Augmented Generation (RAG) has become the standard approach for grounding LLM outputs in external knowledge. The typical RAG pipeline works as follows:

[L744]

[L745] 1. Split documents into fixed-size text chunks (e.g., 512 tokens)

[L746] 2. Embed each chunk into a vector space

[L747] 3. At query time, retrieve the top-k most similar chunks

[L748] 4. Feed these chunks as context to an LLM for answer generation

[L749]

[L750] While effective for simple factoid questions, this flat-chunk approach suffers from five fundamental limitations:

[L751]

[L752] | Problem | Description | Impact |

[L753] |-----|-----|-----|

[L754] | **Broken semantic continuity** | Fixed-size splits cut across paragraphs, definitions, and logical units | LLM receives incomplete or incoherent context |

[L755] | **Duplicate/fragmented information** | The same concept may appear in multiple chunks with slight variations | Token budget wasted on redundancy |

[L756] | **Fixed granularity** | All chunks are the same size regardless of content complexity | Cannot retrieve at different levels of abstraction |

[L757] | **Weak multi-hop reasoning** | No explicit connections between chunks | Cannot follow reasoning chains (e.g.,  $A \rightarrow B \rightarrow C$ ) |

[L758] | **No cross-document fusion** | Chunks from different documents are independent | Cannot merge knowledge from multiple sources |

[L759]

[L760] ### 1.2 Proposed Solution: SHIA-RAG

[L761]

[L762] **SHIA-RAG** (Semantic Hierarchy Induction Architecture for Retrieval-Augmented Generation) addresses all five limitations by replacing flat text chunks with a structured **Knowledge Forest**:

[L763]

[L764]

|                         |                                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| [L765] Traditional RAG: | Document $\rightarrow$ [Chunk <sub>1</sub> , Chunk <sub>2</sub> , ..., Chunk <sub>n</sub> ] $\rightarrow$ Top-k $\rightarrow$ LLM |
| [L766] SHIA-RAG:        | Document $\rightarrow$ Knowledge Forest (concepts + relations) $\rightarrow$ Adaptive Traversal $\rightarrow$ LLM                 |
| [L767]                  |                                                                                                                                   |

[L768]

[L769] The system:

[L770] 1. **Extracts knowledge** — concepts, definitions, relations, and evidence from raw documents

[L771] 2. **Builds a Knowledge Forest** — a hierarchy of semantically linked concept nodes organized as rooted trees

[L772] 3. **Retrieves knowledge nodes** — by traversing the hierarchy (parents, children, cross-links) with adaptive depth

[L773] 4. **Selects context optimally** — using a Tree Knapsack algorithm that respects hierarchical precedence constraints

[L774] 5. **Generates verified answers** — with claim-level evidence verification and citation binding

[L775] 6. **Learns from feedback** — a self-evolution loop that updates edge weights based on retrieval success/failure

[L776] metadata:  
 [L777] query\_result\_index: 11  
 [L778] file\_id: file\_0000000012688211b4f57f5b8bcb8b42  
 [L779] version\_id: 1  
 [L780] name: SHIA\_RAG\_Final\_Report(4).md  
 [L781] mime\_type: text/markdown  
 [L782] surface: conversation  
 [L783] score: 0.028169014084507043  
 [L784] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--31  
 [L785] snippet:  
 [L786] ##### 5.5.5 CLD — Cross-Link Discoverer  
 [L787]  
 [L788] After hierarchical placement, discover semantic (non-hierarchical) links:  
 [L789]  
 [L790]

```
[L791] def cld_discover(node, forest, config):
[L792] """Find semantic cross-links for a newly placed node."""
[L793] cross_links = []
[L794] neighbors = vector_index.search(node.embedding, k=config.cross_link_k)
[L795]
[L796] for neighbor in neighbors:
[L797] if neighbor.id == node.id or neighbor.id == node.parent:
[L798] continue
[L799]
[L800] sim = cosine_similarity(node.embedding, neighbor.embedding)
[L801] if sim > config.cross_link_threshold:
[L802] relation_type = classify_relation(node, neighbor) # USES, CAUSES, etc.
[L803] if relation_type:
[L804] forest.add_edge(node.id, neighbor.id,
[L805] edge_class='SEMANTIC', relation_type=relation_type,
[L806] confidence=sim, weight=1.0)
[L807] cross_links.append((neighbor.id, relation_type, sim))
[L808]
[L809] return cross_links
[L810]
```

[L811]  
 [L812] ---  
 [L813]  
 [L814] ## 6. Online Pipeline — Layers 6-7  
 [L815]  
 [L816] ### 6.1 Layer 6: Retrieval Engine  
 [L817]  
 [L818] **Responsibility:** Analyze user queries, traverse the Knowledge Forest, and select optimal context nodes within a token budget.  
 [L819]  
 [L820] **Input:** User query string → **Output:** RetrievalResult  
 [L821]  
 [L822] ##### Query Complexity Classification  
 [L823]  
 [L824] | Intent | Example | Complexity | Traversal Depth |

```

[L825] |-----|-----|-----|-----|
[L826] | DEFINITION | "What is OSPF?" | SIMPLE | 1 |
[L827] | COMPARISON | "Compare OSPF vs RIP" | MODERATE | 2 |
[L828] | EXPLANATION | "How does OSPF work?" | MODERATE | 2 |
[L829] | MULTI-HOP | "How does OSPF convergence affect reliability?" | COMPLEX | 3 |
[L830] | SYNTHESIS | "Summarize all routing protocols" | COMPLEX | 3 |
[L831]
[L832] ##### Retrieval Pipeline
[L833]
[L834] ```python
[L835] @dataclass
[L836] class RetrievalResult:
[L837] """Output of Layer 6 — input to Layer 7."""
[L838] selected_nodes: list # Nodes selected by Tree Knapsack
[L839] context_text: str # Assembled context string
[L840] traversal_path_edges: list # Edges traversed (for Layer 8 feedback)
[L841] query_complexity: str # SIMPLE, MODERATE, COMPLEX
[L842] query_intent: str # DEFINITION, COMPARISON, etc.
[L843] seed_node_ids: list # Initial seed nodes from vector search
[L844] token_count: int # Estimated tokens in context
[L845]
[L846] class RetrievalEngine:
[L847] def __init__(self, forest, vector_index, cache_manager, config):
[L848] self.forest = forest
[L849] self.vector_index = vector_index
[L850] self.cache = cache_manager
[L851] self.config = config
[L852]
[L853] def retrieve(self, query) -> RetrievalResult:
[L854] # Step 0: Cache check
[L855] cached = self.cache.get_query_result(hash_query(query))
[L856] if cached:
[L857] return cached
[L858]
[L859] # Step 1: Classify query intent and complexity
[L860] intent = detect_intent(query)
[L861] complexity = classify_complexity(intent)
[L862]
[L863] # Step 2: Embed query and find seed nodes
[L864] query_emb = embed_query(query) # Asymmetric embedding model
[L865] seed_nodes = self.vector_index.search(query_emb, k=5)
[L866]
[L867] # Step 3: Adaptive depth traversal
[L868] depth_config = {
[L869] 'SIMPLE': {'max_depth': 1, 'parents': True, 'children': False, 'crosslinks': False},

```



[L870] 'MODERATE': {'max\_depth': 2, 'parents': True, 'children': True, 'crosslinks': False},

[L871] 'COMPLEX': {'max\_depth': 3, 'parents': True,

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: □filecite□turn35file12□

[L872] metadata:

[L873] query\_result\_index: 12

[L874] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L875] version\_id: 1

[L876] name: SHIA\_RAG\_Final\_Report(4).md

[L877] mime\_type: text/markdown

[L878] surface: conversation

[L879] score: 0.028068137824235385

[L880] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--55

[L881] snippet:

[L882] first RAG system to formulate context selection as a precedence-constrained optimization |

[L883] | 2 | **Self-Evolving Knowledge Forest** | Moderately Novel | To our knowledge, first closed-loop feedback mechanism that modifies RAG knowledge structures |

[L884] | 3 | **Multi-Objective Energy Minimization** | Moderately Novel | Novel multi-objective optimization combining 4 objectives for hierarchy induction in RAG |

[L885] | 4 | **Adaptive Depth Traversal** | Incremental | Query-complexity-aware retrieval depth adaptation for hierarchical RAG |

[L886] | 5 | **SHEF Evaluation Framework** | Incremental (Novel Combination) | To our knowledge, first benchmark specifically designed for hierarchical RAG quality |

[L887]

[L888] ### 13.3 Next Steps

[L889]

[L890] 1. **Set up development environment** (Python, Neo4j, FAISS, FastAPI)

[L891] 2. **Implement Layer 0** (Pydantic schemas, database connections, migrations)

[L892] 3. **Build Layers 1-2** (Document ingestion and parsing pipeline)

[L893] 4. **Build Layer 3** (Knowledge extraction with LLM)

[L894] 5. **Build Layer 4** (Relation management with ontology validation)

[L895] 6. **Build Layer 5** (SHIA Core with algorithms)

[L896] 7. **Build Layer 6** (Retrieval with multi-tree knapsack)

[L897] 8. **Build Layer 7** (Generation with verification)

[L898] 9. **Build Layer 8** (Operations, monitoring, self-evolution)

[L899] 10. **Run first evaluation** on OS textbook dataset using SHEF

[L900] 11. **Conduct literature review** (systematic search across ACL Anthology, arXiv, Google Scholar)

[L901] 12. **Write paper** targeting ACL/EMNLP/NAACL 2027

[L902]

[L903] ### 13.4 Future Work

[L904]

[L905] - **DAG Support**: Allow multiple hierarchical parents per node (currently strict single-parent trees)

[L906] - **Multi-Tenancy**: Implement access control and document-level permissions

[L907] - **Exact Tree Knapsack DP**: Implement exact dynamic programming for small token budgets

[L908] - **Cross-Lingual Support:** Extend to multilingual documents and concept hierarchies

[L909] - **Incremental Updates:** Support adding new documents without full forest reconstruction

[L910] - **Advanced Verification:** Replace semantic similarity with NLI (Natural Language Inference) model for claim verification

[L911] - **Exploration in Self-Evolution:** Upgrade from  $\epsilon$ -greedy ( $\epsilon=0.1$ , currently implemented) to Thompson sampling for more principled exploration-exploitation trade-off

[L912]

[L913] ---

[L914]

[L915] > This report serves as the **single source of truth** for the SHIA-RAG project. All 9 layers (Layer 0-8) are fully specified with schemas, algorithms, pseudocode, and implementation contracts. Use this as the foundation for both implementation and thesis writing.

[L916]

[L917] ---

[L918]

[L919] ## Appendix: PDF to Markdown Conversion

[L920]

[L921] To convert this report to PDF, use Pandoc:

[L922]

[L923]

```
[L924] # Install pandoc (https://pandoc.org/installing.html)
[L925] # Basic conversion
[L926] pandoc SHIA_RAG_Final_Report.md -o SHIA_RAG_Final_Report.pdf --pdf-engine=xelatex
[L927]
[L928] # With better formatting (recommended)
[L929] pandoc SHIA_RAG_Final_Report.md \
[L930] -o SHIA_RAG_Final_Report.pdf \
[L931] --pdf-engine=xelatex \
[L932] --variable geometry:margin=1in \
[L933] --variable fontsize=11pt \
[L934] --variable mainfont="Times New Roman" \
[L935] --toc \
[L936] --toc-depth=3 \
[L937] --highlight-style=tango \
[L938] --number-sections
[L939]
[L940] # Alternative: use weasyprint for HTML-based PDF (better Mermaid support)
[L941] pandoc SHIA_RAG_Final_Report.md -o report.html --standalone --css=style.css
[L942] # Then use a browser to print to PDF
[L943]
```

---

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: `filecite[turn35file13]`

[L944] metadata:

[L945] query\_result\_index: 13

[L946] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L947] version\_id: 1

[L948] name: SHIA\_RAG\_Final\_Report(4).md

[L949] mime\_type: text/markdown

[L950] surface: conversation

[L951] score: 0.02749719416386083

[L952] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--50

[L953] snippet:

[L954] ## 11. Evaluation Framework

[L955]

[L956] ### 11.1 Datasets

[L957]

[L958] | Dataset | Domain | Why Use It |

[L959] |-----|-----|-----|

[L960] | Operating Systems textbook (Silberschatz) | CS Education | Deep hierarchical concepts (OS → Scheduling → RR) |

[L961] | Computer Networks textbook (Kurose) | CS Education | Cross-concept relations (TCP uses congestion control) |

[L962] | HotpotQA | Multi-hop QA | Standard benchmark for multi-hop reasoning |

[L963] | MuSiQue | Multi-hop QA | Harder multi-hop requiring 2-4 hops |

[L964] | Custom "OS Hierarchy" gold standard | CS Education | Hand-annotated concept taxonomy for Parent Assignment Accuracy |

[L965]

[L966] ### 11.2 Baselines

[L967]

[L968] | Baseline | What It Represents |

[L969] |-----|-----|

[L970] | Flat RAG (LangChain default) | Traditional chunk-based retrieval |

[L971] | RAPTOR | Recursive summarization tree |

[L972] | GraphRAG (Microsoft) | Knowledge graph + community summaries |

[L973] | TreeRAG | Document-structure tree with bidirectional traversal |

[L974] | HiChunk | Hierarchical chunking with auto-merge |

[L975]

[L976] ### 11.3 Experiments

[L977]

[L978] 1. **Retrieval Quality:** Compare Recall@k, NDCG@k, Hop Accuracy across all baselines

[L979] 2. **Answer Quality:** Compare F1, BLEU, BERTScore for generated answers

[L980] 3. **Hierarchy Quality:** Parent Assignment Accuracy, Tree Edit Distance vs gold taxonomy

[L981] 4. **Ablation Studies:**

[L982] - Remove CPG++ → use random parent selection

[L983] - Remove adaptive depth → use fixed depth=2

[L984] - Remove Tree Knapsack → use flat top-k

[L985] - Remove self-evolution → static forest

[L986] 5. **Scalability:** Measure indexing time, query latency, memory usage at 100/1K/10K/100K nodes

[L987]

[L988] ### 11.4 Statistical Significance Testing

[L989]

[L990] | Test | Application | Details |

[L991] |-----|-----|-----|

[L992] | Paired t-test | SHIA-RAG vs each baseline | Per-query metric comparison |

[L993] | Significance level | All tests |  $\alpha = 0.05$  |

[L994] | Effect size | All comparisons | Cohen's d (small: 0.2, medium: 0.5, large: 0.8) |

[L995] | Confidence intervals | All reported metrics | 95% CI via bootstrap (1000 resamples) |

[L996] | Multiple comparison correction | When comparing 5+ baselines | Bonferroni or Holm-Bonferroni |

[L997]

[L998] ### 11.5 Error Analysis

[L999]

[L1000] | Analysis Type | Method | Details |

[L1001] |-----|-----|-----|

[L1002] | Query categorization | Classify all test queries | DEFINITION, COMPARISON, MULTI-HOP, SYNTHESIS |

[L1003] | Failure pattern analysis | Cluster failed queries | Identify common failure modes |

[L1004] | Qualitative analysis | Manual inspection | Review 100 randomly sampled failed queries |

[L1005] | Per-category breakdown | Stratified metrics | Report accuracy per query type |

[L1006]

[L1007] ### 11.6 Computational Cost Analysis

[L1008]

[L1009] | Metric | How Measured | Report Format |

[L1010] |-----|-----|-----|

[L1011] | GPU hours | Total training + inference time | Hours on NVIDIA A100/V100 |

[L1012] | LLM API calls | Count per document, per query | Total calls + cost per 1K tokens |

[L1013] | Indexing time | Wall-clock time for Layers 1-5 | Seconds per document |

[L1014] | Query latency | End-to-end response time | P50, P95, P99 in milliseconds |

[L1015] | Memory usage | Peak RAM + VRAM | GB |

[L1016] | Cost (USD) | API costs + compute costs | Per 1K documents, per 1K queries |

[L1017]

[L1018] ---

[L1019]

[L1020] ## 12. Risk Analysis

DEEP RESEARCH 20/08/2026, 16:10:27

Citation Marker: fileciteturn35file14

[L1021] metadata:

[L1022] query\_result\_index: 14

[L1023] file\_id: file\_0000000012688211b4f57f5b8bcb8b42

[L1024] version\_id: 1

[L1025] name: SHIA\_RAG\_Final\_Report(4).md

[L1026] mime\_type: text/markdown

[L1027] surface: conversation

[L1028] score: 0.027479766610201392

[L1029] document\_chunk\_id: file\_0000000012688211b4f57f5b8bcb8b42--1--52

[L1030] snippet:

[L1031] ---

[L1032]

[L1033] ## 12. Risk Analysis

[L1034]

[L1035] | Risk | Impact | Probability | Mitigation |

[L1036] |-----|-----|-----|-----|

[L1037] | LLM extraction errors propagate to forest | High | High | Confidence thresholds, human-in-loop review for low-confidence nodes |

[L1038] | Forest grows too deep (>10 levels) | Medium | Medium | Hard depth limit (configurable, default=8), depth regularization in energy function |

[L1039] |  $O(N^2)$  operations at scale | High | Medium | LSH for canonicalization, ANN for CPG++, batch processing |

[L1040] | Cross-document entity conflicts | High | High | Evidence-weighted consensus voting, temporal versioning, conflict logging |

[L1041] | LLM hallucination in answers | High | Medium | Claim extraction + evidence verification, confidence scoring |

[L1042] | Cold start (new domain, no forest) | Medium | Low | Bootstrap from domain ontology (e.g., Wikipedia categories) |

[L1043] | Database sync issues (3 DBs) | Medium | Medium | Saga pattern for distributed transactions, idempotent operations |

[L1044] | Self-evolution reinforces wrong paths | Medium | Medium |  $\epsilon$ -greedy exploration ( $\epsilon=0.1$ ), periodic weight normalization, anomaly detection |

[L1045]

[L1046] ---

[L1047]

[L1048] ## 12.5 Limitations & Current Status

[L1049]

[L1050] This section explicitly documents known limitations and the current project status.

[L1051]

[L1052] ### Current Status

[L1053]

[L1054] SHIA-RAG is currently at the **specification stage**. This report provides a complete architectural specification with schemas, algorithms, pseudocode, and implementation contracts. However:

[L1055]

[L1056] - **No implementation exists yet** — the system has not been built or deployed

[L1057] - **\*\*No experimental results exist...** (truncated)

---

CHATGPT 20/08/2026, 16:10:27

Haan. Ab hume **poore SHIA-RAG project ko ek baar "problem audit" ki tarah dekhna chahiye.**

Ek important distinction:

- **A. Report me officially mentioned limitations/risks**
- **B. Hamne layer-by-layer analysis ke dauran identify kiye implementation/design issues**
- **C. Missing / unspecified parts**

Main inhe mix nahi karunga. Jahan report khud limitation bolti hai, main **Source-stated** bolunga; jahan hamari technical analysis thi, main **Critical analysis** bolunga.

Report khud bhi kehta hai ki project abhi **specification stage** me hai—implementation, experimental results, aur real cost/latency measurements abhi exist nahi karte. Isliye kuch issues **actual runtime bugs nahi**, balki **design gaps / unverified assumptions** hain. `filecite\turn35file7\525-L537`

## SHIA-RAG — Complete Mistakes, Errors, Problems & Gaps Audit

## Quick Summary

| Layer | Major issue category |

|---|---|

| **Layer 0** | Schema consistency, missing validation contracts, data duplication |

| **Layer 1** | File/OCR failure handling, unsupported inputs, extraction quality |

| **Layer 2** | Reading order and layout ambiguity |

| **Layer 3** | LLM extraction errors, alias merging mistakes, scalability |

| **Layer 4** | Wrong relation/confidence propagation, ontology limitations |

| **Layer 5** | Strict tree limitation, wrong hierarchy risk, no incremental updates |

| **Layer 6** | Query classification errors, greedy selection, retrieval-path ambiguity |

| **Layer 7** | Weak verification method, hallucination risk, confidence problems |

| **Layer 8** | Noisy feedback, wrong-path reinforcement, monitoring inconsistencies |

| **Project-wide** | No implementation/results, DB consistency, scalability, cold start |

---

## 1. Layer 0 — Data Model Problems

### 1.1 Schema duplication / consistency risk

`KnowledgeNode` me relationship information multiple jagah represent hoti hai:

```
parent
children
related
incoming_edges
outgoing_edges
```

Aur actual relationship `KnowledgeEdge` objects me bhi exist karti hai.

#### Problem

Agar:

```
Node.parent = NODE_A
```

lekin edge database me:

```
NODE_B → NODE_X
```

stored ho, to system inconsistent ho sakta hai.

#### Main question

*Single source of truth kaunsa hai—Node fields ya Edge records?*

Agar ye clearly enforce nahi hua, hierarchy corrupt ho sakti hai.

#### Type

Critical analysis — design consistency gap

---

### 1.2 Referential integrity explicitly enforced nahi dikh rahi

`parent` , `children` , `evidence` , edge IDs etc. reference karte hain doosre objects ko.

Lekin hume ensure karna hoga:

```
parent ID exists?
child ID exists?
edge source exists?
edge target exists?
evidence source document exists?
```

Agar nahi:

```
Dangling references
```

create ho sakte hain.

## Type

### Critical analysis — missing validation contract

---

## 1.3 Layer 0 sab layers ki dependency hai

Layer 0 me ek schema mistake:

```
↓
Layer 1 metadata wrong
↓
Layer 2 blocks wrong
↓
Layer 3 nodes wrong
↓
Layer 4 edges wrong
↓
Layer 5 forest wrong
↓
Layer 6 retrieval wrong
↓
Layer 7 answer wrong
```

### Biggest risk

Schema error propagates across the entire project.

---

## 2. Layer 1 — Input Processing Problems

### 2.1 OCR errors

Image/scanned PDF se OCR:

```
"Machine Learning"
```

ko:

```
"Machine Leaming"
```

read kar sakta hai.

Phir Layer 3 me:

Machine Learning

aur:

Machine Learning

do different concepts ban sakte hain.

## Result

```
OCR Error
 ↓
Wrong text
 ↓
Wrong concept extraction
 ↓
Wrong alias resolution
 ↓
Wrong hierarchy
```

## 2.2 OCR fallback itself enough nahi hai

Report OCR fallback ko input processing part maanta hai, lekin **OCR quality threshold**, re-OCR policy, ya human-review trigger ka complete contract hume layer analysis me clearly fixed nahi mila. Architecture OCR fallback mention karti hai. [filecite\turn35file0\L13-L18](#)

## Missing question

```
OCR confidence < X ?
→ retry?
→ another OCR engine?
→ reject?
→ human review?
```

## Type

Critical analysis — missing exception/recovery policy

## 2.3 Unsupported / corrupted files

Possible cases:

```
corrupted PDF
password-protected PDF
empty PDF
image-only PDF
malformed DOCX
very large file
unsupported encoding
```

Har case ke liye exact error contract clearly defined hona chahiye.

## Missing



Error code  
Retry policy  
Partial success policy  
Dead-letter / failed document storage

### 3. Layer 2 — Document Parsing Problems

#### 3.1 Reading order error

Complex PDF:

| Left Column | Right Column |
|-------------|--------------|
| A1          | B1           |
| A2          | B2           |
| A3          | B3           |

Parser galat order bana sakta hai:

A1 → B1 → A2 → B2

instead of:

A1 → A2 → A3 → B1 → B2 → B3

#### Impact

Layer 3 ko wrong semantic context milega.

#### 3.2 Table / figure / formula ambiguity

Document parser ko decide karna hota hai:

Ye paragraph hai?  
Table hai?  
Figure caption hai?  
Code hai?  
Formula hai?  
Header/footer hai?

Wrong classification:

Table row  
→ normal sentence

ban sakti hai.

#### Result

Knowledge extraction misleading ho sakta hai.

#### 3.3 Broken semantic blocks

Layer 2 block banata hai.

Agar ek definition:

Sentence 1 → Block A  
Sentence 2 → Block B

me toot gayi, Layer 3 incomplete definition extract kar sakta hai.

### Core problem

SHIA-RAG chunking ko replace kar raha hai, **but parsing layer me hi semantic segmentation galat hui to downstream system phir fragmented knowledge se start karega.**

## 4. Layer 3 — Knowledge Extraction Problems

Ye project ke **highest-risk layers** me se ek hai.

Report khud explicitly maanta hai:

*LLM extraction quality ek limitation hai; extraction errors poore pipeline me propagate kar sakte hain.*

`filecite[turn35file5L368-L374]`

### 4.1 LLM extraction hallucination

Document me agar likha hai:

ML Model estimates risk.

LLM extract kar sakta hai:

ML Model uses Deep Neural Network.

jabki source me DNN mention hi nahi tha.

### Result

False knowledge node create ho jayega.

### 4.2 Missing concept problem

LLM important concept miss kar sakta hai.

Document  
├ Symptoms  
├ Medical History  
├ Emergency Risk  
└ Model Calibration

Extraction output:

Symptoms  
Medical History  
Emergency Risk

Model Calibration missing.

Baad me retrieval us concept ko kabhi retrieve hi nahi kar payega.

### 4.3 Alias resolution false merge

Example:

```
"AI Model"
"Artificial Intelligence Model"
```

correct merge ho sakta hai.

Lekin similar names:

```
Risk Model
Risk Assessment Model
```

same nahi bhi ho sakte.

False merge:

```
Concept A + Concept B
 ↓
Wrong canonical node
```

### 4.4 Alias resolution false split

Opposite problem:

```
Machine Learning
ML
ML Model
Machine-Learning Model
```

same concept hain.

Agar merge nahi hua:

```
Duplicate Nodes
```

banenge.

### 4.5 Scalability issue

Report risk analysis explicitly:

```
O(N²) operations at scale
```

ko major risk maanta hai, with LSH/ANN/batching as mitigation. [filecite turn35file14 L1037-L1044](#)

#### Meaning

10K concepts manageable ho sakte hain.

But:

```
100K+
1M+
```

concepts par naive comparison impossible ya expensive ho sakta hai.

## 5. Layer 4 — Relation Management Problems

### 5.1 Wrong relation extraction

Sentence:

*"The ML model uses patient history to estimate risk."*

Correct:

```
ML Model --USES--> Patient History
ML Model --ESTIMATES--> Risk
```

Possible wrong extraction:

```
Patient History --CAUSES--> Risk
```

Ye serious semantic error hai.

### 5.2 Relation type ambiguity

Ek relation:

```
USES
DEPENDS_ON
RELATED_TO
PART_OF
IS_A
```

me ambiguous ho sakta hai.

Wrong relation:

```
↓
Wrong hierarchy candidate
↓
Wrong Layer 5 forest
```

### 5.3 Confidence propagation problem

Agar relation confidence:

```
0.95
```

hai but extraction itself galat hai, high confidence false security create karta hai.

**Important**

**Confidence ≠ correctness**

### 5.4 Ontology incomplete ho sakti hai

Report 12 relation types specify karta hai. `filecite` `turn35file2` `L163-L168`

But real-world knowledge me relation types infinite/complex ho sakte hain.

Agar correct relation ontology me exist nahi:

Forced classification

ho sakti hai.

## 6. Layer 5 — SHIA Core Problems

Ye sabse important algorithmic risk area hai.

### 6.1 Strict single-parent tree limitation

Report explicitly accept karta hai:

*Real knowledge ke multiple valid parents ho sakte hain, but current design strict single-parent trees use karta hai; future me DAG support planned hai.* [filecite\turn35file5\L368-L371](#)

Example:

```
Machine Learning
├─ Artificial Intelligence
└─ Statistics
```

Current strict tree me:

```
Machine Learning
 ↓
only one parent
```

choose karna padega.

#### Impact

Knowledge expressiveness lose hoti hai.

### 6.2 Wrong parent assignment

Layer 5 hierarchy induction automated hai.

Agar:

Emergency Risk

ko galat parent mil gaya:

Hospital Infrastructure

instead of:

Clinical Decision Support

to poori retrieval path affect hogi.

### 6.3 Abstraction-level heuristic unvalidated

Report explicitly kehta hai ki abstraction-level estimate karne wala multi-factor heuristic **empirically validated nahi hua**. `filecite\turn35file5\L370-L372`

### Problem

System assume kar raha hai:

Node A more abstract than Node B

without sufficient experimental proof.

---

## 6.4 Forest too deep

Report risk:

Forest grows too deep (>10 levels)

Mitigation:

Hard depth limit  
+  
depth regularization

`filecite\turn35file14\L1037-L1040`

### Problem

Too deep:

Retrieval expensive  
Parent context large  
Hierarchy difficult

Too shallow:

Loss of semantic detail

---

## 6.5 No incremental updates

Report explicitly:

*New documents add karne ke liye full offline pipeline Layers 1–5 rerun karni padti hai.* `filecite\turn35file5\L369-L371`

### Major production problem

New document:

Add 1 document

but system may need:

Reprocess entire corpus

This limits scalability.

---

## 7. Layer 6 — Retrieval Problems

### 7.1 Query intent misclassification

Current logic:

```
Intent
↓
Complexity
↓
Traversal Depth
```

Report retrieval code me intent se complexity derive hoti hai. `filecite\turn35file8\L575-L589`

But real query:

*"Compare the effect of X on Y and explain why."*

Ye:

```
COMPARISON?
EXPLANATION?
MULTI-HOP?
```

sab ho sakta hai.

Wrong classification:

```
Wrong complexity
↓
Wrong depth
↓
Wrong context
```

### 7.2 Fixed max depth problem

```
SIMPLE → 1
MODERATE → 2
COMPLEX → 3
```

Report current traversal mapping yahi define karti hai. `filecite\turn35file11\L822-L830`

Problem:

Kuch "simple-looking" query ko 4-hop evidence chahiye ho sakta hai.

Kuch complex query ko 1-hop.

### 7.3 Seed retrieval failure

Agar top-5 vector seeds wrong:

```
Wrong Seed
↓
Wrong Traversal
↓
Wrong Candidates
↓
Wrong Knapsack
```

Hierarchy traversal **bad initial seed ko automatically guarantee se fix nahi kar sakta.**

---

## 7.4 Multi-tree budget allocation heuristic

Current logic candidates ki count ke proportional budget allocate karta hai. `filecite\turn35file8\L611-L629`

### Problem

More candidate nodes:

More budget

but:

More candidates  $\neq$  More importance

Ek important tree ke 3 nodes ho sakte hain aur irrelevant tree ke 50 nodes.

Candidate-count proportional allocation important evidence ko under-budget kar sakta hai.

### Type

Critical analysis — heuristic weakness

---

## 7.5 Greedy Tree Knapsack

Report explicitly:

*Greedy approximation exact DP ke instead use hoti hai; therefore result near-optimal hai, guaranteed optimal nahi.*

`filecite\turn35file5\L368-L370`

### Meaning

System best possible context select karne ka claim mathematically guarantee nahi kar sakta.

---

## 7.6 Retrieval path ambiguity

Layer 8 feedback ke liye:

traversal\_path\_edges

store kiye ja rahe hain. `filecite\turn35file8\L552-L560`

But multi-seed, multi-tree traversal me:

Selected node tak multiple possible paths?

ho sakte hain.

Agar `_get_path_edges(selected)` exact causal path clearly reconstruct nahi karta, feedback wrong edges ko mil sakta hai.

### Type

Critical analysis — attribution ambiguity

---



## 8. Layer 7 — Generation & Verification Problems

### 8.1 LLM hallucination

Report khud:

LLM hallucination in answers

ko high-impact risk maanta hai. [filecite\turn35file14\L1040-L1044](#)

Prompt rule:

Answer ONLY from evidence

likhne se hallucination mathematically impossible nahi hoti. [filecite\turn35file4\L269-L285](#)

### 8.2 Claim verification semantic similarity par based hai

Current verification:

```
embed(claim)
embed(evidence)
↓
cosine similarity
```

[filecite\turn35file4\L288-L319](#)

#### Major problem

High semantic similarity  $\neq$  logical entailment.

Example:

Evidence:

*ML model estimates emergency risk.*

Claim:

*ML model accurately predicts emergency risk.*

Semantic similarity high ho sakti hai.

But:

accuracy

evidence me prove nahi hua.

### 8.3 Conflict detection heuristic weak hai

Current logic roughly score spread dekhta hai:

```
max(scores) - min(scores) > 0.4
→ CONFLICTING
```

[filecite\turn35file4\L310-L319](#)

## Problem

Score difference:

≠  
Logical contradiction

Do semantically different evidence pieces contradictory na hon.

Aur actual contradiction same similarity range me miss ho sakta hai.

## 8.4 Fixed thresholds unvalidated

> 0.7 → SUPPORTED  
> 0.4 → PARTIAL  
≤ 0.4 → UNSUPPORTED

filecite[turn35file4][L293-L319]

## Problem

Why exactly:

0.7?  
0.4?

Domain-specific calibration evidence report me shown implementation ke liye established nahi hai.

## 8.5 Confidence formula can be misleading

Report formula:

```
supported ratio
+
claim confidence
+
node confidence
+
citation coverage
```

combine karta hai. filecite[turn35file4][L331-L346]

Possible problem:

High node confidence  
+  
High citation coverage

answer ko high confidence de sakta hai even if verification itself weak similarity heuristic par based ho.

## 9. Layer 8 — Operations & Self-Evolution Problems

### 9.1 Feedback is noisy

User says:

NOT\_HELPFUL

Reason ho sakta hai:

Retrieval wrong  
OR  
Answer too long  
OR  
User expectation different  
OR  
LLM explanation poor

But edge update retrieval path ko affect karta hai.

### Core problem

User feedback is not a pure retrieval-quality signal.

## 9.2 Wrong path reinforcement

Report explicitly:

*Positive feedback on wrong paths errors reinforce kar sakta hai.* [filecite\[turn35file5\]\[L372-L374\]](#)

Aur risk analysis me:

Self-evolution reinforces wrong paths

listed hai. [filecite\[turn35file14\]\[L1042-L1044\]](#)

### Dangerous loop

```

Wrong Path
 ↓
Accidentally Helpful Answer
 ↓
Positive Feedback
 ↓
Weight Increase
 ↓
Wrong Path More Preferred
 ↓
More Wrong Retrieval

```

## 9.3 Exploration-exploitation bias

Current report future work me Thompson sampling propose karta hai aur current  $\epsilon$ -greedy approach ko more principled method se upgrade karne ki baat karta hai. [filecite\[turn35file12\]\[L903-L911\]](#)

Problem:

System successful paths ko repeatedly exploit kar sakta hai aur alternative paths sufficiently explore nahi karega.

## 9.4 Binary feedback limitation

HELPFUL  
NOT\_HELPFUL

rich feedback nahi deta.

User nahi bata sakta:

```
Retrieval wrong
Citation wrong
Answer incomplete
Answer too long
Latency high
```

## Type

Critical analysis — feedback granularity gap

---

## 9.5 Monitoring inconsistency

Hamare Layer 8 analysis me ek clear implementation mismatch mila tha:

```
Configuration:
latency_p95_ms
```

but shown monitoring calculation:

```
avg_latency_ms
```

track karti hai.

Isliye:

```
Configured threshold
```

aur:

```
Actually calculated metric
```

match nahi karte.

## Type

Critical analysis — implementation inconsistency

---

## 9.6 `confidence == 0` ko error count karna

Ye bhi critical issue tha.

```
confidence = 0
```

ka meaning:

```
Insufficient evidence
```

ho sakta hai—not necessarily:

```
Technical system error
```

To `error_rate` misleading ho sakta hai.

---

## 10. Project-Wide Problems

Ab sabse important part.

Ye problems **ek layer tak limited nahi hain**.

### 10.1 Error propagation — biggest system-wide problem

Report explicitly kehta hai:

*LLM extraction errors forest tak propagate ho sakte hain.* [filecite turn35file14 L 1035-L 1044](#)

Full chain:

```
Layer 1
Bad OCR
 ↓
Layer 2
Bad block
 ↓
Layer 3
Bad concept
 ↓
Layer 4
Bad relation
 ↓
Layer 5
Bad hierarchy
 ↓
Layer 6
Bad retrieval
 ↓
Layer 7
Bad answer
 ↓
Layer 8
Feedback may reinforce bad path
```

**This is the most dangerous end-to-end failure.**

### 10.2 No implementation exists yet

Report clearly says:

```
No implementation exists yet
No deployment exists
No experimental results exist
```

[filecite turn35file7 L529-L537](#)

So currently:

```
Algorithms = specified
Schemas = specified
Pseudocode = specified

BUT

Runtime correctness = unproven
```

### 10.3 No experimental validation

Novel algorithms ke actual results nahi hain.

Abhi hume nahi pata:

```
CPG++ actually better?
PRE useful?
HV correct?
FI improve karta hai?
Tree Knapsack better than top-k?
Self-evolution improves over time?
```

All require experiments.

---

### 10.4 Database synchronization risk

Project 3 databases ke sync issues ko risk maanta hai:

```
Database sync issues (3 DBs)
```

Mitigation:

```
Saga pattern
+
idempotent operations
```

filecite[turn35file14]L1040-L1044

#### Problem

Distributed transaction failure:

```
Graph DB updated ✓
Vector DB updated ✓
Metadata DB failed ✗
```

System inconsistent state me aa sakta hai.

---

### 10.5 Cross-document entity conflicts

Report explicitly:

```
Cross-document entity conflicts
```

ko high impact/high probability risk bolta hai. filecite[turn35file14]L1037-L1041

Example:

```
Document A:
Model version = 2.0

Document B:
Model version = 3.0
```

Dono same canonical node me merge hue to:

```
Conflicting evidence
```

handle karna padega.

---

## 10.6 Cold start problem

New domain:

```
No documents
No nodes
No hierarchy
No feedback
```

Then SHIA-RAG ka main advantage initially available nahi.

Report bootstrap ontology approach suggest karta hai. [filecite\turn35file14\L1041-L1044](#)

---

## 10.7 Scalability problem

Multiple expensive stages:

```
OCR
LLM extraction
Alias resolution
CPG++
Relation validation
Hierarchy construction
Vector search
Tree Knapsack
Claim verification
```

Report evaluation me specifically 100 → 1K → 10K → 100K nodes scalability measure karne ka plan rakhta hai.

[filecite\turn35file13\L976-L986](#)

Meaning:

Scalability abhi proven nahi hai.

---

## 11. Officially Documented Known Limitations

Report ki limitation table ke according:

### L1 — Strict tree structure

Real knowledge me multiple parents possible; current design single-parent tree hai. [filecite\turn35file5\L368-L371](#)

### L2 — Greedy Tree Knapsack

Near-optimal, but guaranteed optimal nahi. [filecite\turn35file5\L368-L370](#)

### L3 — No incremental updates

New documents ke liye full Layers 1–5 rerun. [filecite\turn35file5\L369-L371](#)

#### L4 — Abstraction heuristic unvalidated

Hierarchy wrong ho sakti hai. [\[filecite\]turn35file5\[L370-L372\]](#)

#### L5 — LLM extraction quality

Extraction errors propagate. [\[filecite\]turn35file5\[L371-L373\]](#)

#### L6 — Self-evolution bias

Wrong path reinforce ho sakta hai. [\[filecite\]turn35file5\[L372-L374\]](#)

#### L7 — Novelty claims require verification

Publication se pehle exhaustive verification chahiye. [\[filecite\]turn35file5\[L373-L374\]](#)

---

## 12. Mere according Top 10 Most Critical Problems

Agar mujhe **sirf sabse dangerous 10** choose karne ho, to:

### ● 1. Error Propagation

Early extraction error  
→ entire pipeline corrupted

### ● 2. LLM Extraction Reliability

Layer 3 wrong hua to baaki system wrong knowledge par chalega.

### ● 3. Strict Single-Parent Tree

Real-world knowledge ko forced tree me represent karna loss create karta hai.

### ● 4. Unvalidated Hierarchy Induction

System ki core novelty experimental proof ke bina hai.

### ● 5. Self-Evolution Wrong Feedback Reinforcement

System time ke saath improve bhi ho sakta hai, degrade bhi.

### ● 6. Claim Verification = Semantic Similarity

Similarity ko entailment samajhna logically weak hai.

### ● 7. No Incremental Updates

Production use ke liye major engineering limitation.

### ● 8. Scalability

$O(N^2)$ , LLM costs, multi-stage pipeline.



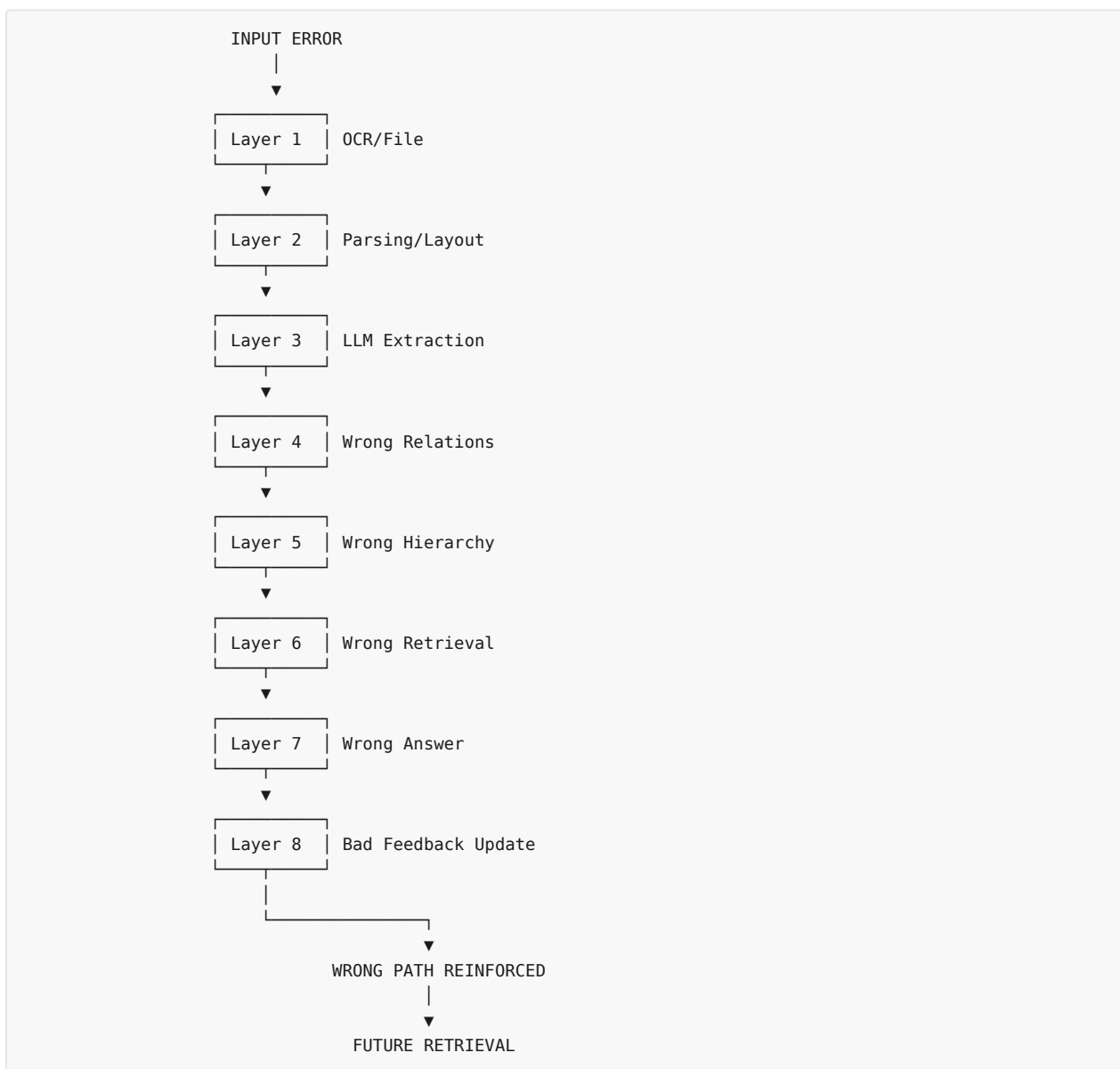
## 9. Cross-Database Consistency

Graph DB + Vector DB + Metadata DB sync failure.

## 10. No Real Implementation / Results

Abhi architecture theoretically detailed hai, but practically unproven.

### Final Master Error Flow



### Sabse important conclusion

SHIA-RAG ka sabse bada weakness ek single algorithm nahi hai.

Uska biggest problem hai:

*Ek early-stage error downstream har layer me propagate ho sakta hai, aur Layer 8 ka self-evolution mechanism kuch cases me us wrong structure ko future ke liye aur reinforce bhi kar sakta hai.*

Report me documented limitations strict-tree structure, greedy Tree Knapsack, no incremental updates, unvalidated abstraction heuristic, LLM extraction errors, self-evolution bias, aur novelty-verification concerns hain; iske alawa hamari layer-by-layer technical review ne schema consistency, validation, path attribution, monitoring, verification, aur feedback-design ke additional gaps identify kiye. [filecite\turn35file5\L368-L374](#) [filecite\turn35file7\L510-L537](#)

**Next logical step:** in sab problems ko main “**Problem** → **Kis layer me** → **Root cause** → **Impact** → **Severity** → **Exact solution** → **Implementation strategy**” wali ek complete master table me organize kar sakta hoon.